UIUC ECE 220 计算机系统与编程 · 学习笔记

> **ECE 220 — Computer Systems & Programming** University of Illinois Urbana-Champaign · The Grainger College of Engineering Department of Electrical and Computer Engineering
>
> 一门**自底向上**讲授 C 语言与系统编程的核心课程： 从 LC-3 汇编、内存映射 I/O、子程序与栈出发，一路走到 C 的指针、动态内存、 数据结构、面向对象设计与调试工具，建立"**从高级语言到机器代码的完整认知链条**"。
>
> 先修课：**ECE 120**（Introduction to Computing）· 4 学分 · 教材：Patt & Patel, *Introduction to Computing Systems: from bits and gates to C and beyond*

本笔记的结构

本笔记共 **21 讲 + 1 个速查表附录**，按"从汇编 → 到 C → 到数据结构 → 到工程实践"的顺序组织。 每一讲都严格采用六段式结构，其中**代码示例**部分包含五个子项 （代码 / 代码做什么 / 底层机制透视 / 内存布局图解 / 与汇编的对应）。

部分	讲次	主题
一 · 机器层	L1 — L4	课程概述与 LC-3 复习；内存映射 I/O 与 TRAP；子程序与调用约定；栈抽象与栈帧
二 · C 与机器码的映射	L5 — L8	类型/运算符/作用域/存储期；控制结构与 I/O；函数；运行时栈与栈帧
三 · 指针与数组	L9 — L12	指针；数组；字符串与多维数组；问题求解与函数指针
四 · 程序组织与内存	L13 — L16	递归；文件 I/O；结构体与信息隐藏；动态内存分配
五 · 数据结构与算法	L17 — L19	链表；树与 C-to-LC-3；排序算法
六 · 工程实践	L20 — L21	面向对象概念与 C/C++ 实现；调试工具与技术
附录	—	C 语言与系统编程核心概念速查表

资料来源

本笔记基于 ECE 220 的**公开可访问**资料编写：课程目录页的官方描述、主题列表、 Course Goals 与 9 条 Instructional Objectives；Fall 2025 课程网站的 syllabus、 逐周课程表与讲座索引；Resources 栏目下的**官方 GDB quick reference、 C coding conventions、C Programming Reference** 三个完整公开页面； 以及归档的 **Prof. Steven Lumetta ECE 220 公开站点**（58 份讲座幻灯片、15 份 Lab 讲义、 12 份 MP 讲义、历

年试卷与完整示例代码）。MP 与 Lab 的完整讲义位于当前网站的 `/secure/` 路径下、不对公众开放，仅有标题与截止日期公开。详细的资料来源与访问限制 记录见"课程概览"一节的最后一小节。

所有 C 代码示例均以 `gcc -g -std=c99 -Wall -Werror` 实际编译并运行验证过。

目录

- 课程概览 (Course Overview)
- ECE 220 是什么
- 为什么要"自底向上"（bottom-up）
- 官方主题列表（Topics Covered）
- 课程目标（Course Goals）
- 教学目标（Instructional Objectives）—— 本笔记的知识框架
- 与 ECE 120 的衔接
- 考核结构（Fall 2025）
- 本笔记如何使用
- 关于资料来源的说明（透明记录）
- 代码与汇编的验证方式
- 讲座序列与主文档结构
- Lecture 1: 课程概述与自底向上方法论；LC-3 机器模型复习 (Course Overview and the Bottom-Up Philosophy; LC-3 Review)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 2: 内存映射 I/O 与 TRAP (Memory-Mapped I/O: Input from the Keyboard, Output to the Monitor; TRAPs)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）

- Lecture 3: 子程序与调用约定 (Repeated Code: TRAPs, Subroutines and the Call Interface Specification)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 4: 栈抽象、栈帧与用栈做算术 (The Stack Abstraction, Stack Frames, and Arithmetic Using a Stack)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 5: C 语言入门：数据类型、运算符、作用域与存储期 (Introduction to C: Data Types, Operators, Scope and Storage)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 6: C 控制结构与基本 I/O (Introduction to C: Control Structures, Basic I/O)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 7: C 函数：定义、调用、参数传递与返回值 (Introduction to Functions in C)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析

- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 8: C 函数的实现：运行时栈与栈帧 (Implementing Functions in C, Run-Time Stack)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 9: 指针 (Pointers)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 10: 数组 (Arrays)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 11: 字符串与多维数组 (Strings; Multi-Dimensional Arrays)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 12: 用指针与数组解决问题；函数指针与回调 (Problem Solving with Pointers and Arrays; Function Pointers and Callbacks)
- 概述

- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 13: 递归 (Recursion)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 14: 文件 I/O (File I/O in C)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 15: 数据结构：结构体、typedef 与信息隐藏 (Data Structures: structs, typedef and Information Hiding)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 16: 动态内存分配 (Dynamic Memory Allocation)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）

- Lecture 17: 链表 (Linked Lists)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 18: 树、遍历与搜索；从 C 到 LC-3 汇编 (Trees, Traversal and Search; From C to LC-3 Assembly with Linked Data Structures)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 19: 基本排序算法 (Basic Sorting Algorithms)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 20: 面向对象编程概念及其在 C/C++ 中的实现 (Object-Oriented Concepts: Information Hiding and Encapsulation in C, and the Move to C++)
- 概述
- 核心概念与底层机制图解
- 代码示例与底层机制分析
- **mem220.c + demo.c # mem_system.c + handle_demo.c**
- **g++ -g -std=c++17 -Wall -Werror 真的把 new/delete 不匹配拦下来了！**
- **去掉 -Werror 让它跑起来：析构次数不对，然后崩溃**
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- Lecture 21: 调试工具与技术 (Testing, Debugging and Tooling: GDB, Valgrind and assert)
- 概述

- 核心概念与底层机制图解
- 代码示例与底层机制分析
- **程序本身的输出（它「正常」运行完了，退出码仍是 0）**
- **gcc -g -std=c99 -Wall -Werror -o assert_on assert_perror.c ; ./assert_on**
- **./assert_on crash-me ← 断言被违反**
- **gcc -g -std=c99 -Wall -Werror -DNDEBUG -o assert_off assert_perror.c**
- **./assert_off crash-me**
- gcc -g -std=c99 -Wall -o warn_demo warn_demo.c (退出码 0，仅警告)
- gcc -g -std=c99 -Wall -Werror -o warn_demo warn_demo.c (退出码 1，编译失败)
- gcc -g -std=c99 -Wall -Wextra -o warn_demo warn_demo.c (退出码 0)
- 常见错误与调试技巧
- 关键要点
- 思考题（带答案）
- **附录：C 语言与系统编程核心概念速查表**
- 目录

- 1. 编译与工具链
- 官方编译命令
- **ECE 220 标准编译命令（不要省略任何警告选项）**
- 调试与净化的编译变体
- **调试版：关闭优化，带符号**
- **净化版：AddressSanitizer + UndefinedBehaviorSanitizer**
- **泄漏检测版（Valgrind 不需要特殊编译，但需要 -g）**
- 多文件编译
- 2. 数据类型与内存表示
- 基本类型表（64 位机器，即 EWS 实验环境）
- 派生类型表
- 字面量与后缀
- 转义序列
- 精确宽度类型（推荐在 ECE 220 中使用）
- 3. 运算符与优先级
- 按优先级从高到低（常用部分）
- 三个最容易记错的点
- 位运算模板
- 4. 指针速查

- 核心规则
- 声明陷阱（课程重点强调）
- 指针的三条基本事实
- 指针常量与 NULL
- 用指针让函数修改调用者的变量（swap 模板）
- 指针的指针：让函数修改调用者的**指针**
- `realloc` 的正确写法（避免失败时泄漏）
- 5. 数组与指针算术速查
- 数组的本质
- 指针算术的步长
- 三种等价的遍历写法
- `sizeof` 陷阱
- 数组名不是指针变量
- 二维数组的行主序 (row-major) 布局
- 三种"多维"声明的区别（高频考点）
- 6. 字符串速查
- C 字符串 = NUL 结尾的 char 数组
- 标准字符串函数（`#include <string.h>`）
- 安全模板
- `strcmp` 的正确用法
- 7. 作用域与存储期
- 作用域（Scope）—— 名字在代码的哪里可见
- 存储期（Storage Duration）—— 变量的寿命
- 内存映射（Memory Map）
- 8. 函数与调用约定速查
- 声明的三种形态
- 参数传递：一切皆传值
- 数组参数
- 函数指针（高频考点）
- 回调与跳转表模板
- 9. 运行时栈与栈帧
- LC-3 调用约定寄存器
- LC-3 栈帧布局
- 调用序列（caller 侧）

- 被调用序列（callee 侧）
- 为什么参数从右往左压栈?
- 为什么同时需要 R5 和 R6?
- 编译器优化：栈帧可能不存在
- 返回局部变量指针 = 悬垂指针
- 10. 结构体、typedef 与信息隐藏
- 定义与访问
- 内存布局与填充（Padding）
- `typedef` 与 `enum`
- 信息隐藏：不透明类型（Opaque Type / Handle Idiom）
- 头文件模板（含 include guard）
- 11. 动态内存管理
- 四个函数的契约
- 标准使用模板
- `sizeof` 的正确写法
- 动态数组增长（倍增策略）
- 倍增策略的代价（课程幻灯片量化）
- 12. 动态内存分配器内部机制
- 堆与 break
- `sbrk` 系统调用
- 分配器的核心数据结构
- 分配策略
- 分裂与合并
- 两类碎片
- 13. 数据结构速查
- 动态数组 vs 链表
- 链表节点定义与核心操作
- 二叉树
- 14. 递归速查
- 递归的两个必要部分
- 标准模板
- 递归的代价：栈帧链
- 递归 vs 迭代
- 尾递归 (Tail Recursion)

- 回溯 (Backtracking) 模板
- 递归树与复杂度：朴素 Fibonacci
- Towers of Hanoi
- 15. 排序算法速查
- 总览对比表
- 插入排序模板
- 归并排序模板
- 快速排序模板（Lomuto 分区）
- `qsort` 与比较函数
- 16. 文件 I/O 速查
- 标准流
- `fopen` 模式
- 常用函数
- 读取整个文件的模板
- 逐行读取的模板
- `while (!feof(f))` 为什么是错的
- 命令行重定向与管道
- 解析文本的模板
- 17. 格式化输入输出速查
- `printf` / `scanf` 格式说明符（官方速查表）
- 关键差异：`printf` 传值，`scanf` 传地址
- 三个经典陷阱
- 检查返回值（必须做）
- 常用格式控制
- 18. LC-3 汇编速查
- 寄存器
- 内存映射 I/O 寄存器
- 轮询 I/O 模板
- 常用指令
- 压栈与弹栈
- 完整的子程序框架
- 19. C 到 LC-3 翻译模板
- 变量声明的翻译
- 赋值的翻译

- `if` 语句的翻译
- `while` 循环的翻译
- `for` 循环的翻译
- 数组访问的翻译
- 函数调用的翻译
- 结构体成员访问的翻译
- 20. 调试速查：GDB
- 启动
- 命令表（官方速查表命令）
- 读懂 `backtrace`
- 检查内存的 `x` 命令格式
- `.gdbinit` 配置（自动加载断点）
- 示例调试会话（真实的 off-by-one bug）
- 常用查看技巧
- **查看某个指针指向的内容和它的地址**
- **查看数组的原始字节**
- **注意：小端序！10 = 0x0a 存成 0a 00 00 00**
- **查看结构体**
- 21. 调试速查：Valgrind 与 Sanitizer
- Valgrind Memcheck
- 读懂泄漏分类
- Valgrind 能抓到的错误类型
- AddressSanitizer（更快，编译期插桩）
- 其他 Sanitizer
- 三种工具的选择
- 22. 错误分类与排查流程图
- 四类错误（课程的错误分类学）
- 排查流程
- 段错误的五个常见原因
- 23. 常见内存错误图鉴
- ① 内存泄漏 (Memory Leak)
- ② 悬垂指针 / 释放后使用 (Dangling Pointer / Use-After-Free)
- ③ 重复释放 (Double Free)
- ④ 越界访问 (Out-of-Bounds)

- ⑤ 返回局部变量的地址
- ⑥ 未初始化的内存
- ⑦ 结构体填充未初始化
- ⑧ `free` 非堆指针
- 内存错误速查表
- 24. C 编程准则清单
- 内存与指针
- 字符串
- 作用域与类型
- 函数与接口
- 流程与风格
- 测试与调试
- 附：LC-3 与 x86-64 概念对照表

课程概览 (Course Overview)

ECE 220 是什么

ECE 220 — Computer Systems & Programming（计算机系统与编程） 是伊利诺伊大学厄巴纳-香槟分校（UIUC）Grainger 工程学院电子与计算机工程系（ECE）的**大二核心课程**，4 学分，由计算机工程与 电气工程两个专业共同必修。它同时承担着一个更宏大的任务：**把 ECE 120 建立的"从比特到门电路" 的底层视角，一路缝合到现代软件工程实践之上。**

> **官方课程描述**
>
> Advanced use of LC-3 assembly language for I/O and function calling convention. C programming, covering basic programming concepts, functions, arrays, pointers, I/O, recursion, simple data structures, linked lists, dynamic memory management, and basic algorithms. Information hiding and object-oriented design as commonly implemented in modern software and systems programming.
>
> **先修要求**：ECE 120（Introduction to Computing）。不接受同期修读。 **限制**：仅限计算机工程 / 电气工程专业学生，或经 ECE 系批准的转学生。

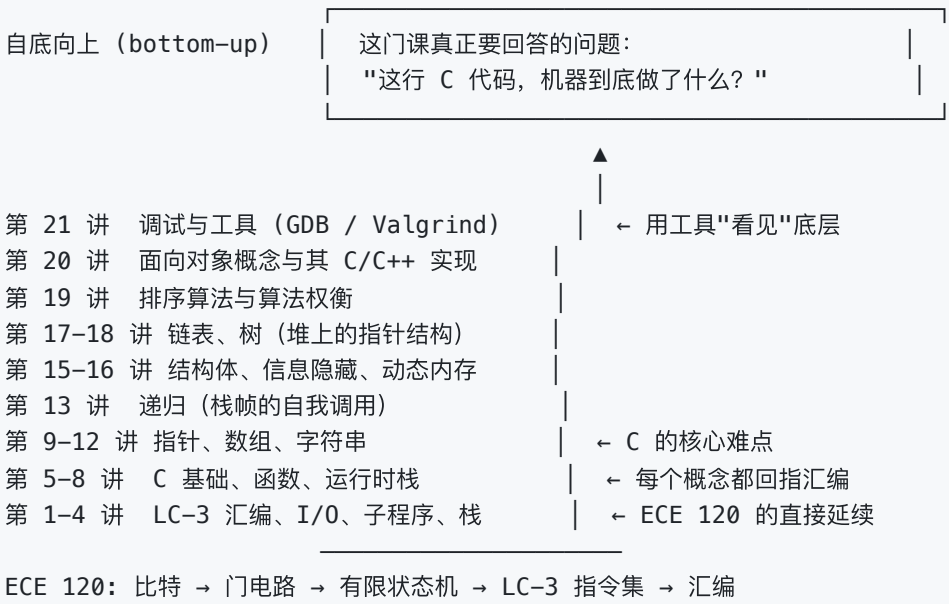
课程主任（Course Director）为 **Yih-Chun Hu** 教授。教材为 Patt & Patel 的 *Introduction to Computing Systems: from bits and gates to C and beyond*（第 2 版 2003 / 第 3 版 2019， McGraw-Hill）——这正是 ECE 120 使用的同一本教材，第 8 章之后的内容构成本课程的主体。

为什么要"自底向上"（bottom-up）

这是理解 ECE 220 全部教学设计的一把钥匙。课程官网的原文是：

> This course will focus on C programming, where **each new C concept will be related to the fundamental concepts described in ECE 120**. We will start by finishing our coverage of low-level concepts such as I/O, subroutines, and stacks in LC-3 assembly language, then move on to C. ... Such a **bottom-up understanding of computing systems** has proven more successful in helping students understand advanced computing concepts that follow in the ECE curriculum.

也就是说，ECE 220 **不是**一门普通的"C 语言入门课"。它的教学逻辑是反向的：



这条链条的每一环都不可跳过，因为**后面每一讲都在使用前面一讲建立的机器模型**：

C 概念	它底下的机器事实（ECE 120/220 前四讲建立）
变量与类型	一段内存 + 一段比特编码；类型是给编译器的"解读说明书"
函数调用	`JSR` 压栈返回地址 + 在栈上建立栈帧 + `RET`
局部变量	栈帧内 `R5` 相对偏移寻址的存储单元
数组	一段连续内存 + 基址加偏移寻址
指针	一个存着地址的字（64 位 = 8 字节）
结构体	一段连续内存 + 编译期确定的成员偏移
动态内存（`malloc`）	`sbrk` 系统调用移动 heap 的 break，再由分配器切块
递归	每次调用一份新的栈帧，栈就是递归的"记忆"
信息隐藏	头文件只暴露函数签名，结构体定义藏在 `.c` 里

学习建议：每学一个 C 概念，都问自己三个问题—— ① 它在内存里长什么样？② 它编译成什么指令？③ 它的生命周期由谁管理（作用域 / 存储期）？

官方主题列表（Topics Covered）

来自课程目录页，一字不改：

1. Assembly language programming with subroutines and stacks（汇编语言中的子程序与栈）
2. Basic programming concepts in C（C 基础编程概念）
3. Functions（函数）
4. Arrays（数组）
5. Pointers（指针）
6. I/O（输入输出）
7. Recursion（递归）
8. Simple data structures such as linked lists and trees（链表、树等简单数据结构）
9. Basic sorting algorithms（基本排序算法）
10. Concepts in object-oriented programming（面向对象编程概念）

课程目标（Course Goals）

> This course focuses on C programming, where each new C concept is introduced based on the fundamental concepts described in ECE 120. We cover basic programming concepts, functions, arrays, pointers, I/O, recursion, simple data structures, and concepts in object-oriented programming. A bottom-up understanding of computing systems has proven more successful in helping students to understand advanced concepts in computing that follow in the ECE curriculum.

教学目标（Instructional Objectives）—— 本笔记的知识框架

课程目录页列出 9 条教学目标，括号中的数字是 UIUC 工程学院的通识能力编号。 **这 9 条就是本笔记每一讲"学习目标"栏目的直接来源**：

#	教学目标（原文）	对应本笔记
1	Understand how statements written in high-level language such as C are transformed into machine code. **Be able to perform such a transformation manually.**	L1–L8, L18
2	Understand the idea of **scope and storage** for variables, and the role of **types** in high-level languages in providing information to the compiler.	L5

#	教学目标（原文）	对应本笔记
3	Understand the **stack abstraction** and the notion of a **calling convention** and its role in supporting the transfer of information between a caller and a subroutine.	L3, L4, L8
4	Understand the concepts of **arrays and pointers** and their representations in memory. Be able to use arrays and pointers for problem solving.	L9—L12
5	Be able to develop and use **data structures** for representing and aggregating information.	L11, L15, L17, L18
6	Be able to use **dynamic memory allocation** for storing values and objects in memory.	L16, L17
7	Understand the value of **recursion** as a problem-solving tool and be able to apply it for solving math and logical problems.	L13
8	Be able to **test and debug** programs written in C using standard debugging tools and techniques.	L21
9	Be familiar with the concepts of **object-oriented programming**.	L20

Fall 2025 版课程网站另外补充了 6 条更细的目标：

- be familiar with basic data organizations such as **arrays, structures, lists, trees, jump tables**, and **how they are laid out in memory**（熟悉基本数据组织及其内存布局）
- be able to write assembly language programs that make use of these data structures
- understand the transformation between programming constructs in languages such as C and their implementation on a **modern microprocessor**
- be able to write C programs to accomplish simple tasks, such as **functional simulation of a processor**
- understand the importance of structuring code in a way that it can be **tested**, be able to write effective tests, and be familiar with tools for aiding in this process
- be familiar with **implementations of basic data structures** and operations on them

以及四条工程素养目标：理解工程学科对**投入、质量、客观性**的要求；认识到**自我驱动与终身学习** 是工程成功的必要条件；能够阐述**权衡（tradeoff）**的重要性；能够识别基本的设计权衡。

> **注意第 1 条里的那句话："Be able to perform such a transformation manually."** 这不是修辞。ECE 220 的考试要求学生**手写 C 到 LC-3 的翻译**。这也是本笔记每一讲都保留 【与汇编的对应】栏目的原因——它直接对应考试能力。

与 ECE 120 的衔接

ECE 120 结束在 LC-3 指令集与基础汇编；ECE 220 从**同一本教材的下一章**接着讲：

	ECE 120	ECE 220
起点	比特、逻辑门、组合电路、有限状态机	LC-3 汇编的 I/O、子程序、栈
机器模型	LC-3 数据通路、指令集、汇编基础	调用约定、栈帧、内存布局、堆
语言	LC-3 汇编	LC-3 汇编 → **C** → C++ 概念
数据组织	单个变量、简单数组	数组、结构体、链表、树、跳转表
终点	"计算机如何执行一条指令"	"高级语言程序如何变成一组指令与内存布局"

教材进度对照（来自 Fall 2025 公开课表）：

- 第 8 章（2 版）/ 第 9 章（3 版）：Memory-mapped I/O、TRAP、子程序
- 第 10 章：栈数据结构与栈操作
- 第 11–13 章：C 的数据类型、变量、运算符、控制结构
- 第 14 章：C 函数、运行时栈
- 第 16 章：指针与数组
- 第 17 章：递归
- 第 18 章：C 文件 I/O
- 第 19 章：数据结构、动态内存、链表
- 第 20 章：数据结构续、树、C++ 与面向对象
- 第 5.4 / 9.4 节：中断与异常、中断驱动 I/O、TRAP

考核结构（Fall 2025）

```
MPs (Machine Problems)   ██████              15%
Midterms (2 次)          ████████████████    40%
Final Exam               ██████████          25%
Quizzes (6 次 CBTF)      ████████            20%
                         ────────────────────
                         100%      (Lab 仅提供加分，不计入 100%)
```

- **MPs**：约每周一个编程作业，每个 100 分。**前两个用 LC-3 汇编，其余用 C**（最后一个涉及 C++）。 通过 GitHub 提交，允许 3 人小组，但每人必须提交自己的副本。 迟交每小时扣 2 分，最多迟 48 小时；MP1–MP11 中最低分会被丢弃。
- **Quizzes**：6 次 CBTF（Computer-Based Testing Facility）机考，最低分丢弃。
- **Exams**：两次期中 + 一次期末，线下纸笔考试，**可以用手写方式考察 C 到 LC-3 的翻译**。
- **Labs**：每周五的编程练习课，完成 worksheet 可得 10 分加分，用于弥补 MP1–MP11 的失分。

工具链（Toolchain）——课程明确要求掌握三类工具：

1. **LC-3 工具**（`lc3tools` 汇编器 / 模拟器，含 `lc3sim-tk` 图形界面）

2. **Git**（作业分发与提交，SSH key 配置，远程工作）
3. **GCC / GDB 编译器–调试器组合**，以及 **Valgrind**

课程的官方编译命令（务必照抄，不要省略警告选项）：

```
gcc -g -std=c99 -Wall -Werror -o output_executable -l library1 source_file1.c [source_file2.
```

本笔记如何使用

每一讲严格按六段式组织，服务于上面 9 条教学目标：

段落	作用	对应教学目标
概述	本讲要解决什么问题	—
核心概念与底层机制图解	定义 / 直观解释 / **内存表示** / **作用域与存储期**	2, 5
代码示例与底层机制分析	可运行 C 代码 + 做什么 + **底层透视** + **内存布局** + **与汇编的对应**	1, 3, 4, 6
常见错误与调试技巧	真实 bug 与可执行的调试命令	8
关键要点	结论与编程准则	全部
思考题（带答案）	自测	全部

两条使用建议：

1. **不要跳过汇编部分。** 第 1–4 讲的 LC-3 内容不是"复习"，它是后面所有 C 概念的解释框架。 如果你不理解 `R5`（帧指针）和 `R6`（栈指针）的区别，第 8 讲"运行时栈"就只能是死记硬背。
2. **每个代码示例都亲手跑一遍。** 本笔记中所有 C 代码都已在 `gcc -g -std=c99 -Wall -Werror` 下实际编译运行验证过。请打开 GDB 单步执行它们， 用 `x/8xb` 看内存，用 `p &var` 看地址——**把笔记里的 ASCII 图变成你屏幕上真实的地址**， 这是把"知道"变成"理解"的唯一途径。

关于资料来源的说明（透明记录）

本笔记的资料获取过程如下，**已公开的内容与未公开的内容都如实记录**：

✅ **公开可访问并已获取：**

- 课程目录页的完整官方描述、主题列表、Course Goals、9 条 Instructional Objectives
- Fall 2025 课程网站：syllabus（含 6 条补充目标）、逐周课程表（含教材章节对照）、 27 讲的讲座索引页、MP 与 Lab 的**标题与截止日期**
- Resources 栏目下的三个完整公开页面：**GDB quick reference**（完整命令参考）、 **C coding conventions**（代码风格规范）、**C Programming Reference** （完整的基本类型/派生类型/字面量/运算符/printf–scanf 格式说明符表格）

- 归档的 **Prof. Steven Lumetta ECE 220 Fall 2020 (ZJUI) / Spring 2018 (Honors)** 公开站点： **58 份完整讲座幻灯片、15 份 Lab 讲义、12 份 MP 讲义、历年期中/期末试卷（含答案与勘误）**， 以及完整的 C 与 LC-3 示例代码（含一个真实可用的动态内存分配器 `mem220`）

⚠️ **未公开 / 需登录，仅记录名称：**

- `MP01` — `MP12` 的完整讲义：当前网站的路径为 `/ece220/fa2025/secure/mps/mpNN`， `/secure/` 路径**不对公众开放**。公开的只有标题（如 "MP 04 — Debugging with GDB"） 与截止日期。本笔记用其标题来佐证对应讲次的教学重点。
- `Lab01` — `Lab13` 的完整 worksheet：路径 `/ece220/fa2025/secure/labs/labNN`，同样不公开。 公开的只有标题（如 "Lab 10 — Linked List"、"Lab 12 — Lowest Common Ancestor"）。
- Spring 2026 讲座幻灯片 PDF：讲座索引页**公开列出了 27 讲的 PDF 链接**， 但截至获取时 **PDF 资产本身尚未上传**（访问返回 Grainger 模板占位页）。 本笔记因此改用上面归档的 Lumetta 幻灯片作为公开补充来源。
- 考试试卷（`/evaluation/exams`）、CBTF 测验内容、Piazza 论坛、Gradescope 提交： 需要登录，未访问。

Fall 2025 公开的 MP 标题（用于佐证教学重点）：

MP	标题	语言
MP 01	Printing histogram	LC-3 汇编
MP 02	Stack calculator	LC-3 汇编
MP 03	Pascal's triangle	C
MP 04	Debugging with GDB	C
MP 05	Codebreaker	C
MP 06	Game of Life	C
MP 07	Sudoku Solver	C
MP 08	2048	C
MP 09	Maze	C
MP 10	Sparse Matrix	C
MP 11	Introduction to C++	C++
MP 12	Anagrams	C

Fall 2025 公开的 Lab 标题：Printing hexadecimals；Problem solving with stack； Computing a math function；Printing Prime Numbers；Random Numbers；Matrix Multiplication in C； Mini Sudoku；2048 Preparation；Vectors；Linked List；C++ Classes；Lowest Common Ancestor；Recap。

> 这些标题本身就是极好的教学信号：**MP04 直接就是"用 GDB 调试"**， **MP11 是 C++ 入门**，**Lab 12 是"最低公共祖先"（树）**—— 说明调试、面向对象和树结构在本课程中都是一等公民，而不是附录。

代码与汇编的验证方式

本笔记不满足于"看起来对"。每一段代码都经过实际执行验证，方法如下：

① C 代码——全部用课程自己的命令编译：

```
gcc -g -O0 -std=c99 -Wall -Werror <file>.c -o <file>
```

并实际运行，逐项比对笔记中写出的输出。最终交付前，把文档里**每一个完整的 C 程序重新抽取出来再编译一遍** （共 30 个），确认 28 个可直接编译运行，另外 2 个是刻意设计的教学案例 （一个多文件示例的 `.h` 在独立代码块中，一个标注"仅供演示"的悬垂指针示例）。

② LC-3 汇编——为了不依赖"手算"，专门写了一个 LC-3 汇编器 + 模拟器 （`ece220_data/lc3sim.py`，实现完整 LC-3 指令集、`.ORIG` / `.FILL` / `.BLKW` / `.STRINGZ` / `.END` 伪指令，以及 `xFE00` — `xFE06` 的内存映射 I/O，因此轮询循环能真正终止）。它把本笔记的 **全部 90 段汇编**过了一遍：

类别	数量	处理
完整程序（含 `.ORIG`）	7	**逐条模拟执行**，并与笔记中手算的最终寄存器值/输出比对
指令片段	79	汇编通过（校验助记符、寄存器编号、字段宽度、立即数范围）
速查模板	4	明确标注为占位符示意，不参与汇编

模拟结果与笔记中所有**手算表逐项吻合**，例如：

讲次	笔记手算的结论	模拟器实测
L01 `SUM5`	`R0 = 150 = x0096`	30 条指令后 `R0 = x0096` ✓
L02 `printbinary`	输出 `1010101111001101`，结束 `R0=x0000, R1=FFFF`	159 条指令后完全一致 ✓
L03 `PRINT_BIN16`	两次调用输出 `xABCD`、`xBC3D`	370 条指令后输出两串首尾相接 ✓
L04 后缀求值器	`R0 = xFFFD = -3`	64 条指令后 `R0 = xFFFD` ✓
L04 `SUM2`	`R0 = 35 = 42 + (-7)`	24 条指令后 `R0 = x0023` ✓
L08 `add3`	`R0 = 15`，栈平衡	54 条指令后 `R0 = 15, R6 = xFE00` ✓

其中 L08 那份 C→LC-3 翻译被**两套互相独立实现的 LC-3 解释器**跑过，得到同一个指令计数（54）， 彼此印证。

③ **这套验证工具已随笔记保留**，你可以自己重跑：

```
cd ece220_data
python3 lc3sim.py myprog.asm          # 汇编并模拟一段 LC-3 程序
python3 verify_asm.py                 # 复查本笔记全部汇编片段
```

> ⚠️ **汇编器/模拟器是用于验证笔记的辅助工具，不是课程要求的工具链。** 课程使用的是 EWS 上的 `lc3tools`（含 `lc3sim-tk` 图形模拟器）； 如果你发现两者行为有差异，**以 `lc3tools` 为准**。

讲座序列与主文档结构

本笔记按"从汇编到 C 到数据结构"的逻辑顺序组织为 21 讲，覆盖官方全部主题与 9 条教学目标：

第一部分 · 机器层（承接 ECE 120） – Lecture 1：课程概述与自底向上方法论；LC-3 机器模型复习 – Lecture 2：内存映射 I/O 与 TRAP – Lecture 3：子程序与调用约定（Call Interface Specification） – Lecture 4：栈抽象、栈帧与用栈做算术

第二部分 · C 语言与机器码的映射 – Lecture 5：C 语言入门——数据类型、运算符、作用域与存储期 – Lecture 6：C 控制结构与基本 I/O – Lecture 7：C 函数——定义、调用、参数传递与返回值 – Lecture 8：C 函数的实现——运行时栈与栈帧

第三部分 · 指针与数组（课程核心难点） – Lecture 9：指针 – Lecture 10：数组 – Lecture 11：字符串与多维数组 – Lecture 12：用指针与数组解决问题；函数指针与回调

第四部分 · 程序组织与内存 – Lecture 13：递归 – Lecture 14：文件 I/O – Lecture 15：数据结构——结构体、typedef 与信息隐藏 – Lecture 16：动态内存分配

第五部分 · 数据结构与算法 – Lecture 17：链表 – Lecture 18：树、遍历与搜索；从 C 到 LC-3 汇编 – Lecture 19：基本排序算法

第六部分 · 工程实践 – Lecture 20：面向对象编程概念及其在 C/C++ 中的实现 – Lecture 21：调试工具与技术（GDB、Valgrind、assert）

附录：C 语言与系统编程核心概念速查表

Lecture 1: 课程概述与自底向上方法论；LC-3 机器模型复习 (Course Overview and the Bottom-Up Philosophy; LC-3 Review)

概述

ECE 220 要回答的核心问题是：从 ECE 120 中"用比特和门电路搭出来"的那台机器出发，怎样一步步走到能够写出 C 程序， 并且清楚地知道每一行 C 在机器里究竟发生了什么。本讲引入贯穿全课的两条主线：**自底向上 (bottom-up)** 的七层抽象链 （bits → gates → microarchitecture → ISA → assembly → C → algorithms），以及 **LC-3 机器模型**（内存、8 个通用寄存器、 指令格式、可寻址性、内存映射）。这套模型是

后面一切的地基：第 2 讲的内存映射 I/O 直接落在本讲画出的内存图最顶端， 第 3 讲的调用约定落在"寄存器约定"上，第 4 讲的栈帧落在"栈区"上，而 C 的变量作用域、数组、指针与函数调用， 也都会用同一套语言反复解释。

核心概念与底层机制图解

- **自底向上方法论 (Bottom-Up Philosophy)**：把"从问题到程序"的过程理解为**逐层构建**——每一层只使用下一层已经存在的机制。
  - *直观解释*：像盖楼而不是像堆沙。ECE 120 给了你砖（门电路）和一层楼（ISA）；ECE 220 要在这层楼上盖第二层（汇编）， 再盖第三层（C）。没有下面两层，上面写的一切都是悬空的。
  - *底层机制图解*：课程把数字系统分成七层，颜色标注每层通常用什么语言描述：

    ```
    问题 / 任务 Problems / Tasks ← 人类语言、理论 算法 Algorithms 机器 / 指令集架构 Machine / ISA ← ECE 220 从这一层开始 微架构 Microarchitecture 电路 Circuits ← ECE 120 在这里造出计算机 器件 Devices ------------------------------------------------ ECE 120: 从 bits 和 gates 造出机器；ECE 220: 站在 ISA 上走向 C；CS 374: 算法理论
    ```

    与机器码/汇编的对应关系在于：**向上走一步，代价是失去一层确定性**。写 C 时你不指定寄存器， 但编译器必须把它变成明确的 LC-3 指令；本课要求你能够**手工完成**这个翻译（教学目标 1）。 * *作用域与存储期*：这条方法论决定了"作用域"在本课中的含义——一个名字（变量名、标号、函数名）的可见范围， 总是由它所处的那一层决定：汇编里的标号受 `.ORIG` 与文件范围限制，C 里的标识符受块作用域与文件作用域限制， 而它们最终都映射为某个地址或某个寄存器偏移。
- **"计算机很笨" (Computers Are Dumb)**：处理器只会反复执行取指—译码—执行 (fetch-decode-execute) 循环，它对"这些比特是什么意思"完全没有概念；所有含义都是**人或编译器赋予的约定**。
  - *直观解释*：把 `"41,962"`、`"41321"`、`"9874"` 三个字符串按 ASCII 排序，计算机给出的大小顺序与人的直觉相反： 逗号 `x2C` 小于字符 `'3'`（`x33`），所以 `"41,962"` 排在 `"41321"` 前面。计算机没有错，它只是**严格按你写的规则做**。
  - *底层机制图解*：字符串在 LC-3 内存里就是"从某个地址开始、连续存放 ASCII 码、以 NUL（`x0000`）结束"的一段字， 而"字符串"这个值就是它的起始地址：

    ```
    地址 内容(bit) 含义 x4012 x0031 '1' ← 字符串 "19" 由地址 x4012 表示 x4013 x0039 '9' x4014 x0000 NUL ← 读到 0 就知道字符串结束 x7196 x0032 '2' ← 字符串 "23" 由地址 x7196 表示 x7197 x0033 '3' x7198 x0000 NUL 若 LC-3 执行: R1 ← x4012 R2 ← x7196 R3 ← R1 + R2 则 R3 = xB1A8 —— 这是一个"指向字符串 "23" 之后第四个字的地址"， 而不是 19 + 23 = 42。M[xB1A8] 里的内容与本题毫无关系。
    ```

    对应的机器码就是一条 `ADD R3,R1,R2`（`0001 011 001 000010`）：ALU 只做二进制加法，既不检查溢出，也不问这两个数是不是地址。 * *作用域与存储期*：这段内存里的字节是**数据段的存储期 = 整个程序运行期**（静态存储期）； 而"字符串"这个名字在 C 里可能只是一个指针变量（automatic storage duration）， 指针本身随栈帧消失，指向的内容却可能仍然存在——这是后面"悬垂指针"的根本原因。
- **系统化分解 (Systematic Decomposition)**：给定一个用人类语言描述的任务，**反复拆分**为更简单的子任务，直到每个子任务只需要几条指令（或几条 C 语句）就能表达。
  - *直观解释*：把"做一顿饭"拆成"买菜 / 洗菜 / 炒菜 / 盛盘"，每一项继续拆到"打开冰箱门"这个粒度。

- *底层机制图解*：拆分的结果最终只归结为三种构造 (construct)，它们各自映射到内存中的指令序列：

  `顺序 条件 迭代 子任务1 test condition ──FALSE──→ else ┌──→ 子任务 子任务2 ↓ TRUE ↓ │ ↓ 子任务3 then 子任务 ──→ 汇合 ←── └─ test condition（TRUE 则回去） 内存中的样子（条件构造）： x3000 …生成条件的指令（如 ADD R0,R0,#0 设置 N/Z/P）… x3001 BRn ELSE ; 0000 100 …（条件为假时跳走） x3002 …then 子任务的指令… x3003 BRnzp JOIN x3004 ELSE …else 子任务的指令… x3005 JOIN … BR` 的机器码形如 `0000 nzp PCoffset9`：只改 PC，不改任何寄存器，这正是"流程图能画进内存"的唯一手段。 * *作用域与存储期*：三种构造在 C 里对应顺序语句、`if/else`、循环语句；循环体是**块作用域**， 在块内声明的 automatic 变量每次进入块都会重新获得存储（生命周期只有一次迭代那么长）。

- **良好设计 (Good Design)**：软件的好坏没有单一指标——指令条数、内存用量、运行时间、能耗、正确性**全都算数**；课程给出两条可操作的指导原则：(1) 更简单（可行）的方案；(2) 易读、易测。
  - *直观解释*：先画流程图、先写注释，再写代码；能复用就复用，**每复制一次代码，就复制了一份 bug**。
  - *底层机制图解*：设计原则最终体现为可测量的机器代价。例如同一个"求数组和"的任务：

    `方案 A（简单直接） 方案 B（"聪明"但难读） LOOP LDR R3,R1,#0 ; 被完全展开的 5 条 LDR/ADD ADD R0,R0,R3 ; 无循环、无分支 ADD R1,R1,#1 ; 指令数 15，内存 15 字 ADD R2,R2,#-1 ; 跑得快，但数组长度一变就要重写 BRp LOOP ; 无法测试"长度 0"的情形 ; 指令数 5，内存 5 字 ; 长度改变只需改 R2 的初值`

    课程强烈建议先用**查表 (look-up table)** 与循环来表达重复，而不是把代码抄 N 遍。 * *作用域与存储期*：这条原则在 C 里表现为：函数的职责边界要清晰（接口 = 参数 + 返回值 + 副作用）， 这样每个函数才能**独立测试**；用 `static` 限制文件作用域，可以避免全局状态被意外修改。

- **LC-3 机器模型 (Machine Model)**：LC-3 是一台 16 位、**字可寻址 (word-addressable)** 的冯·诺依曼机。
  - *直观解释*：内存是一排编号的抽屉，每个抽屉里放一个 16 位的字；"地址"就是抽屉编号。
  - *底层机制图解*：核心参数必须背下来：

    `字长（word size）16 bit 地址宽度 16 bit → 2^16 = 65,536 个地址 可寻址空间 65,536 words = 128 KiB 可寻址单位（addressability）1 word（不是 1 byte！） 通用寄存器 8 个：R0..R7，每个 16 bit 条件码（condition codes）N / Z / P（每次写寄存器都会被更新） PC 16 bit，指向下一条要取的指令`

    与 C 的关键差异：C 的 `char *p; p + 1` 前进 **1 字节**，而 LC-3 的 `ADD R1,R1,#1` 前进 **1 个字（2 字节）**。 这就是同一个"指针加一"在两层的不同含义。 * *作用域与存储期*：8 个寄存器的"生命周期"是**整个程序运行期**，但它们的**内容归属**由第 3 讲的调用约定决定： R0–R3 是 caller-saved（子程序可以随便改），R4–R7 承担全局数据指针、帧指针、栈指针、返回地址。

- **指令格式 (Instruction Formats) 与可寻址性**：16 位指令中高 4 位是操作码 (opcode)，其余 12 位按格式划分为寄存器号和立即数/偏移。
  - *直观解释*：16 个比特要同时说明"做什么"和"对谁做"，所以必须精打细算；这就是为什么偏移量的位宽会限制跳转范围。
  - *底层机制图解*：三种基本格式（REGI 型、IMM 型、JMP 型）：

    `bits 15 14 13 12 | 11 10 9 | 8 7 6 | 5 4 3 | 2 1 0 ---------------------------------------------------------- ADD（reg）0001 | DR | SR1 | 0 0 0 | SR2 ; DR ← SR1 + SR2 ADD（imm5）0001 | DR | SR1 | 1 | imm5 ; DR ← SR1 + SEXT(imm5) LDR 0110 | DR | BaseR |`

`offset6 ; DR ← M[BaseR+SEXT(off6)] STR 0111 | SR | BaseR | offset6 ; M[BaseR+SEXT(off6)] ← SR BR 0000 | n z p | PCoffset9 ; if (cc 命中) PC ← PC + SEXT(off9) JSR 0100 | 1 | PCoffset11 ; R7 ← PC; PC ← PC + SEXT(off11) JSRR 0100 | 0 0 0 | BaseR | 0 0 0 0 0 0 ; R7 ← PC; PC ← BaseR LEA 1110 | DR | PCoffset9 ; DR ← PC + SEXT(off9)（取地址）`

立即数位宽直接决定能力边界：`imm5` 只能表示 −16..15，所以"把 15 放进 R1"必须用 `AND R1,R1,#0` + `ADD R1,R1,#15` 两条指令（立即数不够用，就先清零再加）；`offset6` 只能表示 −32..31， 所以访问远处的数据要用 `LEA` 先算出基址，再配合 `LDR/STR`。 * *作用域与存储期*：立即数偏移是**编译期常量**，它的"作用域"只在这一条指令内；而基址寄存器的内容是运行期值， 生命周期由程序员管理。

- **LC-3 内存映射 (Memory Map)**：整个 64K 字地址空间被**约定**划分为系统空间、代码、全局数据、堆、栈，以及最顶端的设备寄存器。
  - *直观解释*：像一座城市的规划图：市中心（低地址）是政府机关（陷阱向量表）与操作系统，中间是居民区（你的程序与数据）， 最北边（高地址）是海关与港口（I/O 设备寄存器）。
  - *底层机制图解*：



堆与栈**相向增长**：`malloc` 从下往上要空间，函数调用从上往下压栈。两边一旦相遇，就是"内存耗尽"。 这也是"栈溢出 (stack overflow)"在系统层面的真实含义。 * *作用域与存储期*：这张图就是 C 的存储期分类的物理来源—— 代码/全局数据 = static storage duration（整个程序期），栈 = automatic storage duration（进入块时创建、离开块时销毁）， 堆 = allocated storage duration（从 `malloc` 到 `free`，由程序员负责）。

- **ECE 120 如何喂养后面每一个 C 概念**：本课的每一个 C 主题都能追溯到 ECE 120 的一个底层机制。
  - *直观解释*：ECE 120 教会你"机器能做什么"，ECE 220 教你"用机器能懂的方式表达想法"。
  - *底层机制图解*：对应表如下——

`ECE 120 的底层机制 ECE 220 / C 中的概念 ---------- ---------- 二进制补码表示 int / unsigned / 溢出 / 类型转换 ASCII 与 NUL 结尾约定 字符串（char*）、strlen、字符串常量 8 个寄存器 + 条件码 表达式求值、控制流、局部变量的寄存器分配 内存映射 I/O printf / scanf 的底层对应物 JSR/RET + R7 函数调用、返回地址、递归 栈与 R6 / R4 全局数据指针 automatic 变量、栈帧、递归深度、static 变量 内存映射中的 heap malloc / free / 动态数据结构` 教学目标 2（作用域与存储）、3（调用约定）、4（数组与指针）、6（动态分配） 全部是这张

表的直接延伸。 * *作用域与存储期*：这张表本身就是"作用域与存储期"的分类标准：看到一个新变量， 先问"它在哪个区、活了多久"，答案决定它能不能被别的函数看到、能不能安全地被返回。

代码示例与底层机制分析

示例 1：一个完整的 LC-3 程序——用三种构造求数组元素之和

代码 (LC-3 assembly):

```
; SUM5 -- 把数组里 5 个字相加，结果放在 R0
; 寄存器用途表
;   R0 : 累加和（输出）
;   R1 : 指向当前数组元素的指针
;   R2 : 剩余元素计数
;   R3 : 当前元素（临时）

        .ORIG x3000
        LEA R1,ARRAY        ; x3000: R1 ← 数组首地址（顺序构造）
        AND R0,R0,#0        ; x3001: R0 ← 0
        AND R2,R2,#0        ; x3002: R2 ← 0
        ADD R2,R2,#5        ; x3003: R2 ← 5（立即数只有 5 位，所以先清零再加）
LOOP    LDR R3,R1,#0        ; x3004: R3 ← M[R1]       ← 迭代构造从这里开始
        ADD R0,R0,R3        ; x3005: sum ← sum + 元素
        ADD R1,R1,#1        ; x3006: 指针前进一个字
        ADD R2,R2,#-1       ; x3007: 计数减一
        BRp LOOP            ; x3008: 还有元素就回去（条件构造：只在 P 时跳）
        HALT                ; x3009: 停机
ARRAY   .FILL #10           ; x300A: 10
        .FILL #20           ; x300B: 20
        .FILL #30           ; x300C: 30
        .FILL #40           ; x300D: 40
        .FILL #50           ; x300E: 50
        .END
```

> ⚙️ **机器验证**：本清单已用 LC-3 汇编器 + 模拟器（`ece220_data/lc3sim.py`）实际汇编并运行： 执行 **30 条指令**后 `HALT`，最终 **`R0 = x0096 = 150`**，与下面手算表给出的结果完全一致。

【代码做什么？】

1. `LEA R1,ARRAY`：把标号 ARRAY 的**地址**（不是内容）装进 R1，此处 `R1 = x300A`。
2. 三条 `AND/ADD` 指令把 `R0`（和）清零、把 `R2`（计数）设置为 5；因为 `imm5` 只能到 15，这里虽然放得下 5，但同样两条指令的习惯写法在后面放 15 时是必需的。
3. 进入 `LOOP`：`LDR R3,R1,#0` 取出当前元素，`ADD R0,R0,R3` 累加。
4. `ADD R1,R1,#1` 让指针指向下一个**字**；`ADD R2,R2,#-1` 让计数减一。
5. `BRp LOOP`：若刚才的 `ADD` 使结果为正（P=1）就跳回；等于 0 时 P=0、Z=1，不跳，落到 `HALT`。
6. `HALT` 停机。

【底层机制透视】

- LC-3 没有"数组"这个类型，只有**连续的内存字**和"用指针加偏移去访问"的机制。数组名 `ARRAY` 在汇编里就是地址常量。
- 循环的循环变量、边界判断全部由程序员手工维护；`BRp` 只看条件码，而条件码是**上一条写寄存器的指令**留下的，所以 `ADD R2,R2,#-1` 与 `BRp LOOP` 必须紧挨着写——中间插入任何写寄存器的指令都会破坏判断。
- `printf` 这类"高级"操作在这里完全不存在：输出要靠第 2 讲的 `STI` / `TRAP`，所以本程序只能把结果留在 R0 里供调试器观察。

【内存布局图解】

```
地址      内容          含义
x3000     1110 001 000001001   LEA  R1,ARRAY   (PC+1+9 = x300A)
x3001     0101 000 000 1 00000 AND  R0,R0,#0
x3002     0101 010 010 1 00000 AND  R2,R2,#0
x3003     0001 010 010 1 00101 ADD  R2,R2,#5
x3004     0110 011 001 000000  LDR  R3,R1,#0    ← LOOP
x3008     0000 001 111111011   BRp  LOOP       (PC+1-5 = x3004)
x3009     1111 0000 0010 0101 HALT
x300A     0000 0000 0000 1010 10      ← ARRAY（连续 5 个字）
x300E     0000 0000 0011 0010 50      ← 最后一个元素
```

【与汇编的对应】（逐条手算执行结果）

时刻	R0	R1	R2	R3	说明
`LEA R1,ARRAY` 后	x0000	x300A	x0000	—	指针就位
`ADD R2,R2,#5` 后	x0000	x300A	#5	—	计数 = 5，P=1
第 1 次循环末	#10	x300B	#4	#10	P=1 → 跳回
第 2 次循环末	#30	x300C	#3	#20	P=1 → 跳回
第 3 次循环末	#60	x300D	#2	#30	P=1 → 跳回
第 4 次循环末	#100	x300E	#1	#40	P=1 → 跳回
第 5 次循环末	**#150**	x300F	**#0**	#50	Z=1, P=0 → 不跳，落到 HALT

即 `R0 = x0096 = 150`，正是 10+20+30+40+50。注意 `R1` 结束时指向 `x300F`（数组之后的第一个字），这已经"越界"了一个字——把指针留在越界位置是常见隐患，若紧接着去 `LDR` 就会读到垃圾。

示例 2：C 版本的"计算机很笨"——把地址当数字相加

代码 (C):

```
/* ECE 220 -- Lecture 1 example: "computers are dumb".
 *
 * The LC-3 executes R3 <- R1 + R2 with no idea what the bits mean.
 * Here we reproduce, in C, the slide example in which a student adds the
 * *addresses* of the strings "19" and "23" and gets a third address.
 */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void)
{
    int32_t addr1 = 0x4012;   /* address of the string "19" */
    int32_t addr2 = 0x7196;   /* address of the string "23" */
    int32_t sum   = addr1 + addr2;

    uint32_t bits = 0x3F800000u;   /* the 32 bits of the IEEE-754 float 1.0f */
    float    as_float;
    int32_t  as_int;

    /* copy the same 32 bits into a float and into an int32_t */
    memcpy(&as_float, &bits, sizeof(as_float));
    memcpy(&as_int,   &bits, sizeof(as_int));

    printf("R1 = x%04X   (address of \"19\")\n", (unsigned)(addr1 & 0xFFFF));
    printf("R2 = x%04X   (address of \"23\")\n", (unsigned)(addr2 & 0xFFFF));
    printf("R3 = x%04X   (R1 + R2: an address, NOT 19 + 23)\n",
           (unsigned)(sum & 0xFFFF));
    printf("M[x%04X] is one word past the NUL of \"23\" -- meaningless here\n",
           (unsigned)(sum & 0xFFFF));

    printf("\n");
    printf("the same 32 bits x%08X are:\n", (unsigned)bits);
    printf("  an IEEE-754 float -> %f\n", (double)as_float);
    printf("  a signed int32_t  -> %d\n", (int)as_int);
    printf("sizeof(int32_t) = %d bytes, sizeof(float) = %d bytes\n",
           (int)sizeof(int32_t), (int)sizeof(float));
    return 0;
}
```

编译与运行（本机 gcc 12.2.0）：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l01_dumb.c -o ece220_l01_dumb
$ ./ece220_l01_dumb
R1 = x4012   (address of "19")
R2 = x7196   (address of "23")
R3 = xB1A8   (R1 + R2: an address, NOT 19 + 23)
M[xB1A8] is one word past the NUL of "23" -- meaningless here

the same 32 bits x3F800000 are:
  an IEEE-754 float -> 1.000000
  a signed int32_t  -> 1065353216
sizeof(int32_t) = 4 bytes, sizeof(float) = 4 bytes
```

【代码做什么？】

1. 用两个 `int32_t` 保存两份"地址"，与幻灯片上 LC-3 的 R1、R2 完全对应。
2. `sum = addr1 + addr2` 就是那条 `ADD R3,R1,R2`；结果 `x4012 + x7196 = xB1A8`（16 位截断后），与 LC-3 的结果**逐位相同**。
3. 打印 `M[xB1A8]` 的说明：它落在 `"23"` 的 NUL 之后的第四个字节处，没有任何含义。
4. 后半段演示"同一串比特，不同类型解读不同"：`x3F800000` 当作浮点是 `1.000000`，当作有符号整数是 `1065353216`。
5. 打印 `sizeof` 说明"类型"的唯一实际作用之一就是**告诉编译器一次运算动多少字节**。

【底层机制透视】

- `memcpy(&as_float, &bits, 4)` 做的是**纯粹的比特复制**，不进行任何数值转换：这正是"机器只搬比特"的 C 级证据。 用 `as_float = (float)bits;` 则是数值转换，结果是 `1065353216.0f`，语义完全不同——这是 C 新手最常见的混淆点。
- `(unsigned)(addr1 & 0xFFFF)` 里的 `& 0xFFFF` 模拟了 LC-3 的 16 位寄存器宽度：LC-3 的 `ADD` 天然丢弃第 16 位以上的进位。 C 的 `int32_t` 不会自动截断，所以这里必须显式模仿。
- 在真正的机器上，`"19"` 和 `"23"` 是编译器放进只读数据段的两个字符数组，程序里出现的是它们的地址（指针）。 `"19" + "23"` 这种写法在 C 里甚至无法编译通过——两个指针不能相加——但 LC-3 没有类型，所以它能"顺利"执行并给你一个垃圾地址。

【内存布局图解】

```
        .rodata（只读数据，相当于 LC-3 的代码/数据区）
        +--------+--------+--------+
        |  '1'   |  '9'   | NUL    |   "19"   起始地址 0x4020（示例）
        +--------+--------+--------+
        |  '2'   |  '3'   | NUL    |   "23"   起始地址 0x4024
        +--------+--------+--------+

        栈帧（automatic storage duration）
        +------------------+  0x7ffd...
        |  addr1 = 0x4012  |
        +------------------+
        |  addr2 = 0x7196  |
        +------------------+
        |  sum   = 0xB1A8  |   ← 一个"指向垃圾"的地址
        +------------------+
        |  bits  = 0x3F800000
        +------------------+
        |  as_float        |   ← 与 bits 相同的 32 位，按 IEEE-754 解读
        +------------------+
        |  as_int          |   ← 与 bits 相同的 32 位，按补码解读
        +------------------+
```

【与汇编的对应】

```
; C: int32_t addr1 = 0x4012;  int32_t addr2 = 0x7196;  int32_t sum = addr1 + addr2;
; 汇编里变量存在内存（或寄存器）里，先搬到寄存器再运算
        LD   R1,ADDR1       ; R1 ← M[ADDR1]  = x4012
        LD   R2,ADDR2       ; R2 ← M[ADDR2]  = x7196
        ADD  R3,R1,R2       ; R3 ← xB1A8（第 16 位以上被丢弃）

; C: memcpy(&as_float, &bits, 4) —— 4 个字节的纯粹复制，不做任何数值转换
; LC-3 是字可寻址的，所以"4 字节"就是两个字的搬运（LDR/STR 各两次）

ADDR1   .FILL x4012
ADDR2   .FILL x7196
```

示例 3：内存映射在真实程序中的样子

代码 (C):

```c
/* ECE 220 -- Lecture 1 example: the memory map, seen from C. */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

static int32_t global_initialized = 0x220;   /* global data area  */
static int32_t global_zero;                  /* zero-filled (.bss) */

int main(void)
{
    int32_t  automatic = 1;          /* stack (automatic storage)  */
    static int32_t static_local = 2; /* still global data area     */
    int32_t *heap = malloc(4 * sizeof(int32_t));  /* heap          */
    uintptr_t a_heap;
    uintptr_t a_stack;
    uintptr_t a_global;

    if (heap == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    heap[0] = 0x220;

    a_global = (uintptr_t)&global_initialized;
    a_heap   = (uintptr_t)heap;
    a_stack  = (uintptr_t)&automatic;

    printf("&main              = %p   (code)\n", (void *)&main);
    printf("&global_initialized= %p   (global data)\n",
           (void *)&global_initialized);
    printf("&global_zero       = %p   (global data, zero-filled)\n",
           (void *)&global_zero);
    printf("&static_local      = %p   (global data, static duration)\n",
           (void *)&static_local);
    printf("heap (malloc)      = %p   (heap)\n", (void *)heap);
    printf("&automatic         = %p   (stack)\n", (void *)&automatic);

    printf("\n");
    printf("heap   is %lu bytes above global data\n",
           (unsigned long)(a_heap - a_global));
    printf("stack  is %lu bytes above heap\n",
           (unsigned long)(a_stack - a_heap));
    printf("global_zero-static_local = %ld (static locals sit with globals)\n",
           (long)((intptr_t)&static_local - (intptr_t)&global_zero));
    printf("on this machine the stack is at the %s addresses\n",
           (a_stack > a_heap) ? "HIGHER" : "lower");

    free(heap);
    return 0;
}
```

编译与运行：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l01_memmap.c -o ece220_l01_memmap
$ ./ece220_l01_memmap
&main              = 0x401166   (code)
&global_initialized= 0x404050   (global data)
&global_zero       = 0x40405c   (global data, zero-filled)
&static_local      = 0x404054   (global data, static duration)
heap (malloc)      = 0xcd42a0   (heap)
&automatic         = 0x7ffd355f679c   (stack)

heap   is 9241168 bytes above global data
stack  is 140725485446396 bytes above heap
global_zero-static_local = -8 (static locals sit with globals)
on this machine the stack is at the HIGHER addresses
```

（`heap` 与 `&automatic` 的数值以及两个"相距多少字节"每次运行都不同——本机开了 ASLR；上面是一次真实运行的输出， 但**相对次序**与 `global_zero - static_local = -8` 这条结论恒定不变。）

【代码做什么？】

1. 在五个不同的位置各声明一个对象：代码（函数）、全局数据、零初始化数据、静态局部变量、自动变量、堆。
2. 用 `%p` 打印各自的地址，注意**地址是运行期决定的**（本机开启了 ASLR，每次运行都会变），但**相对次序永远不变**。
3. 计算并打印相对距离，验证"代码 < 全局数据 < 堆 < 栈"这个次序与 LC-3 内存图一致。
4. `global_zero - static_local == -8` 说明：`static` 局部变量**并不在栈上**，它与全局变量住在一起。
5. `free(heap)` 归还堆空间，体现堆的存储期由程序员显式控制。

【底层机制透视】

- 编译产物分成若干**段 (section)**：`.text`（代码）、`.rodata`（字符串常量）、`.data`（有初值的全局/静态变量）、`.bss`（初值为 0 的全局/静态变量）。`global_initialized` 在 `.data`，`global_zero` 在 `.bss` —— 它们在内存图上相邻，但只有 `.data` 的内容需要存进可执行文件（`.bss` 只记录大小，运行时清零）。
- `&static_local` 落在全局数据区，正是"静态存储期 (static storage duration)"在物理上的表现： 它在程序启动前就存在、程序结束才消失，与函数调用无关。
- 栈地址比堆高得多、且两者相距极远，是因为**栈从高地址往下长、堆从低地址往上长**，在中间留着大块未映射的空隙。 这与 LC-3 的内存图完全同构；LC-3 只是把地址空间缩小到 64K 个字。
- 把函数指针转成 `void *` 打印属于实现定义行为（POSIX 保证可行），此程序在 x86-64 gcc 上稳定输出 `0x401166` 一类的低地址。

【内存布局图解】

```
低地址  0x401166  ┌──────────────┐  .text        &main（代码）
                  │     ...      │
        0x404050  ├──────────────┤  .data        global_initialized = 0x220
        0x404054  ├──────────────┤  .data        static_local = 2（静态存储期！）
        0x40405c  ├──────────────┤  .bss         global_zero（运行时清零）
                  │     ...      │
                  │   (空隙)     │  ← 堆向上长、栈向下长，在这中间相遇
                  │     ...      │
        0xcd42a0  ├──────────────┤  heap         malloc 返回的 16 字节
                  │     ...      │
高地址  0x7ffd... ├──────────────┤  stack       &automatic（automatic 存储期）
                  └──────────────┘
```

【与汇编的对应】

```
; LC-3 里同样的五个位置
;   代码          —— 从 x3000 开始，PC 在这里跑
;   全局数据      —— R4 指向这里（本课约定 x4000 起）
;   堆            —— 由 malloc 的实现（MP 中的 LC-3 分配器）用 R4 之上的空间管理
;   栈            —— R6 指向栈顶，向低地址增长；R5 是当前帧指针

        LEA  R4,GLOBAL_START   ; 初始化全局数据指针（程序启动代码负责）
        ; C 的 static int32_t static_local = 2; 编译成：
        LD   R0,STATIC_LOCAL   ; 直接从全局数据区取，不用 R5/R6
        ; C 的 int32_t automatic = 1; 编译成：
        ADD  R6,R6,#-1         ; 在栈帧里要一个槽
        AND  R0,R0,#0
        ADD  R0,R0,#1
        STR  R0,R5,#-1         ; 局部变量放在 R5-1

GLOBAL_START .FILL #0
STATIC_LOCAL .FILL #2
```

常见错误与调试技巧

- **把地址当数值参与运算**：像 `"19" + "23"` 那样把指针或地址直接相加，得到 `xB1A8` 这类垃圾地址。 现象是访问越界、结果莫名其妙。**调试**：在 C 中用 `printf("%p\n", (void *)p);` 确认拿到的确实是地址而不是数值； 在 LC-3 中用 `lc3sim` 的 `dump` 命令查看该地址处的内容，再用 `list` 对照它属于哪个数据块。
- **忘记 LC-3 是"字可寻址"**：以为 `ADD R1,R1,#1` 前进一个字节。现象是遍历 `char` 数组时每次跳过 2 字节而漏掉一半元素。 **调试**：在 C 中用 `printf("%td\n", (char *)p - (char *)q)` 观察真实字节差；记住 LC-3 上一律是"字"。
- **用错寄存器破坏了调用约定**：把 R4/R5/R6/R7 当临时寄存器乱用（例如用 R6 存循环计数）。 现象是返回后崩溃、`RET` 跳到错误位置。**调试**：在每条子程序开头写寄存器用途表注释； 在 `gdb` 中用 `info registers` 或 `layout regs` 观察 `rsp` / `rbp`（对应 R6/R5）是否被破坏。
- **`BR` 与条件码之间插入了别的指令**：`ADD R2,R2,#-1` 之后本应紧跟 `BRp`，中间插了 `LDR` 就按 `LDR` 的结果分支。 现象是循环次数差 1 或死循环。**调试**：在 `lc3sim` 里对循环回边设置断点（`break LOOP`），

每次停下用 `print R2` 与 `printpsr` 看 N/Z/P，逐次核对是否与手算一致。

- **偏移量超出位宽限制**：`LDR` 的 `offset6` 只有 −32..31，`BR` 的 `PCoffset9` 只有 −256..255，`JSR` 的 `PCoffset11` 只有 ±1024。 现象是汇编器报 "offset out of range"。**调试**：用 `LEA` 先算基址再用 `LDR/STR` 间接访问； 远距离跳转改用 `JSRR`（配合 `LEA R7,label` 一类技巧）或拆成两跳。
- **误以为内存有"类型"**：把 `x3F800000` 当成整数却期望它是 `1.0`。现象是打印出 `1065353216`。 **调试**：`gdb` 中 `x/4xb &bits` 看字节、`p *(float *)&bits` 与 `p *(int32_t *)&bits` 解读同一地址。
- **忘记 `HALT`**：程序"跑完"却没停机，PC 继续执行到 `.FILL` 的数据区，把数据当指令执行，仿真器行为完全不可解释。 **调试**：`lc3sim` 中单步 `step` 并开 `list` 观察 PC 是否越过最后一条指令；课程 MP 明确惩罚"执行数据"。

关键要点

- ECE 220 是**自底向上**的课程：每一个 C 概念都必须能追溯到 ECE 120 的底层机制（补码、ASCII、寄存器、内存映射、栈、调用约定）。 学习本课时，看到任何新概念都要问一句"它在机器里对应什么"。
- 计算机只做取指—译码—执行，**所有含义都来自约定**。"字符串是地址 + NUL"、"栈向低地址增长"、 "R6 是栈指针"都是约定；约定不遵守，机器照跑不误，但结果毫无意义。
- LC-3 的核心参数必须背熟：16 位字长、16 位地址、65536 个字、**字可寻址**、8 个通用寄存器、N/Z/P 条件码。 立即数位宽（imm5 = 5 位、offset6 = 6 位、PCoffset9 = 9 位）决定了哪些操作需要拆成多条指令。
- 内存映射的五段（系统空间 / 代码 / 全局数据 / 堆 / 栈）不是硬件规定，而是**软件约定 + 硬件配合**的产物； 堆向上、栈向下相向增长，这就是 C 中 static / automatic / allocated 三种存储期的物理来源。
- 良好设计的可操作版本：先画图、先写注释、避免复制代码、把功能切开以便单独测试——这些习惯在汇编阶段看起来"多余"， 到了 C、数据结构与递归阶段会决定你是"能debug"还是"被bug埋掉"。

思考题（带答案）

问题 1：LC-3 的地址是 16 位、可寻址单位是一个字（16 位）。请回答：(a) 总共有多少个可寻址的字？ (b) 如果换成字节可寻址（如真实机器），同一个 16 位地址最多能覆盖多少字节？ (c) 为什么 `ADD R1,R1,#1` 在遍历 `int32_t` 数组时在 C 里要写成 `p + 1` 而不是 `p + 2`？

答案：(a) `2^16 = 65,536` 个字，即 128 KiB。(b) 16 位地址最多寻址 `65,536` 个字节（64 KiB）—— 这就是为什么"字可寻址"的 LC-3 空间看起来比同地址宽度的字节寻址机器大：字长变大，地址仍只数"格子"。(c) 因为 C 的指针算术**按所指类型的大小缩放**：`int32_t *p; p + 1` 意味着"下一个 `int32_t`"， 编译器生成的是 `地址 + 4`（x86-64）或 `地址 + 2`（LC-3 上一个 `int32_t` 占两个字）。 LC-3 汇编里没有类型，`ADD R1,R1,#1` 就是加一个**字**，所以它等价于 C 中"下一个 16 位单元"。

问题 2：下面这段代码在 LC-3 上会打印出什么？为什么它与"人以为的答案"不同?

```
        .ORIG x3000
        LEA R0,A
        PUTS
        HALT
A       .STRINGZ "41,962"
B       .STRINGZ "41321"
        .END
```

> ⚙ **机器验证**：已用 `ece220_data/lc3sim.py` 汇编并运行，3 条指令后 `HALT`， 显示器上确实只出现 `41,962`（第二个 `.STRINGZ` 从未被访问）。

答案：只打印 `41,962`。若要比较两个字符串（幻灯片中的例子），LC-3 会逐字比较 ASCII 值： `,` = `x2C` 小于 `'3'` = `x33`，因此按 ASCII 顺序 `"41,962"` 排在 `"41321"` 之前，而人按数值认为 `41321 < 41962`。机器没有错——它执行的是"按字符编码逐位比较"这个约定； "字符串转成数值再比较"必须由程序显式完成（这也是 MP2/MP3 中要自己写转换代码的原因）。

问题 3：为什么"每复制一次代码就复制了一份 bug"？请用本讲的内存图解释代码复用（子程序）在机器层面的收益。

答案：复制出来的代码在内存里是**另一份独立的指令序列**，它与原版没有任何共享： 修 bug 时只改一处，另一处照旧出错；更糟的是两份副本会逐渐"漂移"，变成两套行为不同的语义。 子程序把指令序列放在内存中的一个位置，所有调用者用 `JSR` 跳过去、用 `RET` 跳回来，于是内存占用从 O(调用点数 × 代码长度) 降到 O(代码长度)，修改也只有一处。代价是必须约定好"怎么传参、怎么返回、 哪些寄存器会被改"——这就是第 3 讲的调用约定。

Lecture 2: 内存映射 I/O 与 TRAP (Memory-Mapped I/O: Input from the Keyboard, Output to the Monitor; TRAPs)

概述

本讲解决一个在 ECE 120 里被回避的问题：处理器如何与**外部世界**交换数据；LC-3 的答案是 **内存映射 I/O (memory-mapped I/O)**——把设备寄存器放在内存地址空间的最高端，用普通 `LDI`/`STI` 访问， 再用**状态位 (status bit)** 做同步（握手）。为了避免每个程序都手写轮询循环，LC-3 提供了 **TRAP 指令**与**陷阱向量表 (trap vector table)**，把 `GETC/OUT/PUTS/IN/HALT` 等系统服务做成"带特权切换的子程序调用"； 理解 TRAP 是理解 C 的 `printf`/`scanf`、系统调用 (system call) 与特权级 (privilege) 的第一步。 本讲的所有机制在第 3 讲会被重新解释为"子程序 + 调用约定"的一个特例。

核心概念与底层机制图解

- **内存映射 I/O (Memory-Mapped I/O)**：把 I/O 设备的寄存器**编入内存地址空间**，用装载/存储指令访问它们。

- *直观解释*：给设备也发一个"门牌号"，只不过这些门牌号集中在城市最北边；你不需要新的动词（新指令），只需要知道"这个门牌号后面不是仓库，而是海关窗口"。
- *底层机制图解*：访问 I/O 寄存器只有两种方案——

  `方案 1：新增指令与独立地址空间（端口 I/O） IN R0, KBDR / OUT DDR, R0 ；x86 的 IN/OUT 属于这一类，现代 ISA 中很少见 方案 2：内存映射 I/O（LC-3 采用，大多数现代 ISA 也采用） LDI R0, KBDR_ADDR / STI R0, DDR_ADDR ；用普通的存储访问指令即可 LC-3 的地址分配：xFE00 ~ xFFFF 属于 I/O（占全部 128K 字的 1/128） xFE00 KBSR xFE02 KBDR xFE04 DSR xFE06 DDR xFE08 ~ xFFFF 预留给未来的设备`

  为什么地址是隔一个用：LC-3 本身是字可寻址的，而 P&P 定义的 LC-3b 是**字节可寻址**的； 每对寄存器之间留一个地址，就能让同一份 I/O 地址映射在两种机器上通用。 硬件侧：地址控制逻辑 (address control logic) 根据 MAR 与 `MIO.EN`、`R.W` 决定是选中内存芯片， 还是把 KBSR/KBDR/DSR 的值送上 MDR；读 KBSR 时 `MAR=xFE00, R.W=read, MIO.EN=1`。 * *作用域与存储期*：这些"变量"**作用域全局**（任何有权限的代码都访问同一份设备状态）， **存储期等于机器通电期**——不随程序启动/结束创建或销毁，进程退出也不清零。这是"外设状态"与"程序变量"最根本的区别。

- **KBSR / KBDR：键盘寄存器对**：键盘这一侧用两个 16 位寄存器分别承载"状态"和"数据"。
  - *直观解释*：一个负责举手（状态位），一个负责说话（数据）；举手之后才有人听你说话。
  - *底层机制图解*：

    `KBDR (xFE02) — 键盘数据寄存器 15 ..... 8 | 7 ..... 0 未使用（读为 0） | 扩展 ASCII (8 bit) 字符，如 'A' = x41 KBSR (xFE00) — 键盘状态寄存器 15 | 14 ..... 0 状态位 | 未定义 (undefined)，软件必须忽略 1 = ready（有一个按键等待读取） 0 = not ready 关键：读入 KBSR 时 KBSR[15] 正好落在 N 条件码上， 所以检查"是否 ready"只需要一条 BRzp / BRn。` 协议违规的后果（幻灯片明确列出）：若在处理器读走上一个键之前又按了一个键，**只有一个键能被保存，另一个丢失**； 若在没有按键时读 KBDR，读到的比特**没有意义**。 * *作用域与存储期*：KBSR/KBDR 的值完全由硬件维护；软件唯一的责任是遵守协议——先看状态、再取数据。数据寄存器是"一到就取走"的单槽缓冲，不是队列。

- **DSR / DDR：显示器寄存器对**：输出侧的同一套结构，方向相反。
  - *直观解释*：DDR 是"投递口"，DSR 是"投递口是否空闲"的指示灯。
  - *底层机制图解*：

    `DDR (xFE06) — 显示器数据寄存器 bits 15 ..... 8 | 7 ..... 0 写入时填 0 | 扩展 ASCII (8 bit) 字符 DSR (xFE04) — 显示器状态寄存器 bits 15 | 14 ..... 0 状态位 | 未定义，软件必须忽略 1 = ready（可以接收下一个字符） 0 = not ready 时序（显示器初始为 ready)： 处理器：写字符进 DDR → （写动作本身把状态位翻转成 0） 显示器：读走并显示该字符 显示器：把自己的 SYNC 位翻转 → 状态位回到 1 处理器：观察到 ready 才能写下一个字符` * *作用域与存储期*：与键盘完全相同——全局可见、硬件维护、不随程序生命周期变化。 因为**别的代码可能刚用过显示器**，任何 LC-3 输出代码都必须**先等 ready** 再写，不能假设初始状态。

- **同步与握手 (Synchronization / Handshaking)**：设备与处理器没有公共时钟，因此必须用握手协议显式同步。

- *直观解释*：课堂上你举手（状态）老师才听你说话（数据）；没有举手就说话，问题就丢了。
- *底层机制图解*：

  `没有同步会怎样：你突然喊出问题 → 老师正在板书 → "嗯？什么？" → 问题已经丢了 硬件握手协议：生产者 1. 把数据放上线 2. 翻转自己的 SYNC（"新数据"）3. 等消费者翻转它的 SYNC 消费者 4. 读走数据 5. 翻转自己的 SYNC LC-3 把两个 SYNC 信号用 XOR 合成一个"状态位"（假设都从 0 开始）： SYNC_producer XOR SYNC_consumer = 1 → ready；= 0 → not ready 因此"翻转"在状态位上表现为 0 → 1 → 0 的交替。` 处理器比外设快得多（人按键间隔约 **100 毫秒**，对 LC-3 是**几千万个周期**），所以轮询几乎总在空转。 * *作用域与存储期*：握手状态跨越两个时钟域——**生产者与消费者各自保存一位**，组合后的状态位是唯一被软件看到的部分。

- **忙等 / 轮询 (Busy-Wait / Polling)**：在循环里反复读取状态寄存器，直到 ready 为止。
  - *直观解释*：站在门口反复问"好了吗？"——简单可靠，但把 CPU 时间全部浪费在等待上。
  - *底层机制图解*：LC-3 的典型轮询写法（一条 `LDI` + 一条 `BRzp`）：

    `POLL LDI R1,KBSR ; R1 ← M[M[KBSR]]：先取地址 xFE00，再间接读设备 BRzp POLL ; N=0（not ready）就继续等 LDI R0,KBDR ; N=1（ready），取走数据 KBSR .FILL xFE00 ; 注意：这个字的"内容"才是设备地址 xFE00` * *作用域与存储期*：轮询循环没有持久状态，其变量的生命周期就是循环本身；与之相对的**中断 (interrupt)** 方案 允许处理器去干别的事，代价是必须保存全部处理器状态（所有寄存器 + 条件码）并用 `RTI` 返回。

- **TRAP 指令与陷阱向量表 (TRAP Instruction and Trap Vector Table)**：用一个 8 位编号请求系统服务。
  - *直观解释*：TRAP 就像"呼叫总机报分机号"——不需要知道服务代码在哪个地址，只需要知道编号。
  - *底层机制图解*：

    `TRAP 的 RTL（与 JSR 的前半段完全相同）： R7 ← PC ; 保存返回地址 PC ← M[ZEXT16(vec8)] ; 用 8 位编号查表得到服务程序入口 Trap Vector Table 位于 x0000 ~ x00FF，每个单元存放对应服务的**起始地址**： x0020 → x0400 (GETC) x0021 → x0450 (OUT) x0022 → x0480 (PUTS) x0023 → x04A0 (IN) x0024 → x0520 (PUTSP) x0025 → x04E0 (HALT) (vector = 指针 = 地址；其余单元留给未来的服务) LC-3 的六个服务：GETC 读一个字符到 R0[7:0]（不回显）；OUT 输出 R0[7:0]； PUTS 输出 R0 指向的 NUL 结尾字符串；IN 提示并读入一行、回显，返回 R0[7:0]； PUTSP 按"两字符打包成一字"输出字符串；HALT 停机。` **TRAP 就是一个子程序调用**：返回地址放进 R7，服务程序以 `RET`（`JMP R7`）结束， 所以"TRAP 会覆盖 R7"与 `JSR` 完全一样——这就是课程 `readnumsub.asm` 里 `ST R7,SAVE_R7` 存在的原因。 系统调用还是一种特殊形式的**库**：通常以特权级执行、预先装进机器（有时在 ROM 里）、**按编号间接访问**。 这正是"任何问题都可以用另一层间接解决"的实例：编号 → 向量表 → 实现。 * *作用域与存储期*：陷阱向量表是**系统空间**的一部分，由操作系统在启动时填写，应用程序只读不写； 向量表的间接性带来可升级性：操作系统换了实现，应用程序的 `TRAP x21` 一行都不用改。

- **PSR 与特权级 (Processor Status Register and Privilege)**：硬件用一位标记"当前代码是否有权碰硬件"。
  - *直观解释*：设备寄存器就像配电箱，只有持证的电工（操作系统）能开箱；普通程序必须"请人代劳"（TRAP）。
  - *底层机制图解*：

`PSR (Processor Status Register) 的 bit 15 PSR[15] = 0 → privileged （特权态，可以做任何事） PSR[15] = 1 → unprivileged （用户态，必须依赖操作系统） TRAP x21 的执行过程（概念模型）： 用户代码 陷阱服务程序 返回 TRAP x21 → R7 ← PC, PC ← M[x0021] → 执行服务代码（ends with RET） → 回到用户态 （特权级提升，服务程序可以访问 DDR/DSR）` LC-3 的 TRAP **实际上并不切换特权级**（幻灯片明确指出："But not in the LC-3 ISA"），但它展示了真实机器上系统调用的结构：**编号 → 查表 → 特权代码 → 返回**。 * *作用域与存储期*：PSR 的生命周期是"当前正在执行的代码"；它决定一次 `LDI/STI` 能否作用于 I/O 地址，是"作用域"概念在**硬件权限**层面的对应物。库代码的存储期则是"链接进可执行文件后与程序同寿"（静态库）或"由加载器在运行时映射"（动态库），而 TRAP 服务由操作系统提供，跨越所有进程。

代码示例与底层机制分析

示例 1：把 R0 按 16 位二进制打印出来（课程原版 `printbinary.asm`）

代码 (LC-3 assembly):

```
        .ORIG   x3000

        LD      R0,NUMBER

        ; print the value in R0 as a 16-bit binary number

        ; R0 holds the number (shifted left to find bits)
        ; R1 holds the bit number being printed (15 down to 0)
        ; R2 holds the ASCII character for that bit
        ; R3 holds x30 (ASCII '0') for convenience

        AND     R1,R1,#0        ; start with bit 15
        ADD     R1,R1,#15
        LD      R3,ZERO         ; initialize R3
BITLOOP ADD     R2,R3,#0        ; copy R3 to R2 ('0')
        ADD     R0,R0,#0        ; check bit value
        BRzp    ZEROBIT         ; skip next instruction for 0 bit
        ADD     R2,R2,#1        ; print a 1 bit
ZEROBIT LDI     R4,DSR          ; wait for display to be ready
        BRzp    ZEROBIT
        STI     R2,DDR          ; write ASCII character to display
        ADD     R0,R0,R0        ; shift R0 left to find next bit
        ADD     R1,R1,#-1       ; count down in bit index
        BRzp    BITLOOP         ; print another until index < 0
        HALT

NUMBER  .FILL   xABCD           ; a number to print
ZERO    .FILL   x30             ; ASCII digit '0'
DSR     .FILL   xFE04           ; DSR address in LC-3
DDR     .FILL   xFE06           ; DDR address in LC-3

        .END
```

> ⚙️ **机器验证**：本清单已用 LC-3 汇编器 + 模拟器（`ece220_data/lc3sim.py`）实际汇编并运行： 执行 **159 条指令**后 `HALT`，显示器输出 **`1010101111001101`**（= `xABCD`）， 结束时 **`R0 = x0000`**、**`R1 = xFFFF`** —— 与下面 16 次迭代的手算表逐项吻合。 （模拟器把 `xFE04` DSR 的 bit 15 置 1 表示"显示器就绪"，轮询循环因此能正常退出。）

【代码做什么？】

1. `LD R0,NUMBER` 把待打印的数 `xABCD` 装入 R0； `AND`+`ADD` 把位计数器 R1 置为 15； `LD R3,ZERO` 把 ASCII `'0'`（`x30`）装入 R3。
2. `BITLOOP`：先把 R2 设为 `'0'`（`ADD R2,R3,#0` 就是复制 R3）； `ADD R0,R0,#0` 不改 R0 的值， 只为了**刷新条件码**：若 R0[15] = 1，则结果视为负数，N = 1。
3. `BRzp ZEROBIT`：N = 0（最高位是 0）就跳过下一条，于是 R2 保持 `'0'`；否则执行 `ADD R2,R2,#1` 得到 `'1'`。
4. `ZEROBIT` 处的轮询循环： `LDI R4,DSR` 反复读显示器状态， `BRzp ZEROBIT` 在"没 ready"时继续等；ready 后 `STI R2,DDR` 把字符写进数据寄存器（写动作本身让状态位变 0，显示器取走后变回 1）。
5. `ADD R0,R0,R0` 左移一位把下一位送到 bit 15， `ADD R1,R1,#-1` 计数减一， `BRzp BITLOOP` 在还有位时继续； 16 位打印完毕后 `HALT` 停机。

【底层机制透视】

- **为什么要用 `LDI` / `STI` 而不是 `LD` / `ST`**：`DSR` / `DDR` 这两个标号处存放的是**设备地址本身**（`xFE04` / `xFE06`）。 `LDI R4,DSR` 的语义是"先读 `M[DSR]` 得到 `xFE04`，再读 `M[xFE04]`"，正好命中设备寄存器。 如果误用 `LD R4,DSR`，只会把 `xFE04` 这个**地址常量**读进 R4，永远等不到 ready。 > 📌 **关于这里的 `R4`**：本清单是课程 `printbinary.asm` 的**原文**，它确实用 `R4` 暂存 DSR 的值。 > 这是**安全的**，因为该程序是独立程序——既不调用任何子程序、也不使用全局数据指针， > 所以 R4 空着。但**一旦你的程序里出现了 `JSR`**，就必须遵守第 3 讲的寄存器约定 > （`R4` 全局数据指针、`R5` 帧指针、`R6` 栈指针），临时值只能用 `R0`–`R3`。 > 课程 MP 的代码都属于后者。
- **轮询循环的方向很容易搞反**：状态位为 1 = ready 时 N = 1，所以"继续等"的条件是 N = 0， 应该写 `BRzp`（Z 或 P，即 N = 0）跳回；写成 `BRn` 就变成了"ready 时还在等、不 ready 时反而往下走"。
- **位选择用"左移"而不是"右移"**：R0 左移后最高位不断被替换成下一位，配合 `ADD R0,R0,#0` 就能用 N 条件码直接判断， 省掉了 `AND` 掩码。这是 LC-3 上最省指令的写法，因为 LC-3 没有"算术右移"也缺少位测试指令。
- `HALT` 本身也是一条 TRAP（`TRAP x25`）；本程序里写 `HALT` 就是它的伪指令形式。

【内存布局图解】

```
地址       指令/数据                          说明
x3000      LD   R0,NUMBER      (off=+14)   取要打印的数
x3003      LD   R3,ZERO        (off=+12)
x3004      ADD  R2,R3,#0       ← BITLOOP
x3005      ADD  R0,R0,#0                   刷新 N/Z/P
x3006      BRzp ZEROBIT        (off=+1)
x3007      ADD  R2,R2,#1
x3008      LDI  R4,DSR         (off=+8)    ← ZEROBIT: 轮询状态寄存器
x3009      BRzp ZEROBIT        (off=-2)    未 ready 就继续等
x300A      STI  R2,DDR         (off=+7)    写数据寄存器
x300D      BRzp BITLOOP        (off=-10)       x300E HALT
------- 以下为数据区 -------
x300F      xABCD  ← NUMBER     x3010 x0030 ← ZERO (ASCII '0')
x3011      xFE04  ← DSR        x3012 xFE06 ← DDR
设备侧（不在程序占用的地址范围里，而是硬件寄存器）:
xFE00 KBSR[15]=状态    xFE02 KBDR[7:0]=按键    xFE04 DSR[15]=状态    xFE06 DDR[7:0]=待显示字符
```

【与汇编的对应】（手算执行结果，R0 初值 = `xABCD` = `1010 1011 1100 1101`）

迭代	R1（计数）	移位前的 R0	R0[15]	打印	移位后的 R0
1	15	xABCD	1	`1`	x579A
2	14	x579A	0	`0`	xAF34
3	13	xAF34	1	`1`	x5E68
4	12	x5E68	0	`0`	xBCD0
5	11	xBCD0	1	`1`	x79A0
8	8	xE680	1	`1`	xCD00
12	4	x6800	0	`0`	xD000
16	0 → −1	x8000	1	`1`	x0000

（第 5–16 次迭代依次打印 `0 1 1 1 1 0 0 1 1 0 1`，中间行从略。） 显示屏上最终得到 `1010101111001101`，正是 `xABCD` 的 16 位二进制写法； 循环结束后：`R0 = x0000`、`R1 = xFFFF`(−1)、`R2 = x31`（最后一个字符 `'1'`）、`R3 = x0030`、`R4 = xFE04`。 注意 `ADD R0,R0,R0` 每次都会丢弃最高位（左移溢出），所以移位 16 次后 R0 必然归零。

示例 2：OUT 陷阱的真实实现（系统服务就是子程序）

代码 (LC-3 assembly，取自 lc3sim 的 OS 代码):

```
; OUT is TRAP x21.  M[x0021] contains x0450, and listing x0450 gives:

TRAP_OUT
        ST      R1,TOUT_R1      ; 保存 R1：子程序不允许偷偷改变调用者的寄存器
TRAP_OUT_WAIT
        LDI     R1,OS_DSR       ; 轮询显示器状态
        BRzp    TRAP_OUT_WAIT   ; 未 ready 就等待
        STI     R0,OS_DDR       ; 把 R0[7:0] 写进 DDR
        LD      R1,TOUT_R1      ; 恢复 R1
        RET                     ; JMP R7 —— 与普通子程序完全一样

TOUT_R1 .BLKW   1
OS_DSR  .FILL   xFE04
OS_DDR  .FILL   xFE06
```

【代码做什么？】

1. 进入服务程序时 R7 已由 `TRAP` 指令设为返回地址（`R7 ← PC`），R0 中是要输出的字符。
2. `ST R1,TOUT_R1` 保存 R1——服务程序要用 R1 做轮询，但**调用者并不期望 R1 被改**。
3. 轮询循环等到显示器 ready，然后 `STI R0,OS_DDR` 输出字符。
4. 恢复 R1，`RET` 返回。

【底层机制透视】

- 这段代码把第 3 讲的主题提前暴露出来：**调用约定 (calling convention)**。 它有输入（R0）、有副作用（写显示器）、对"其他寄存器"的所有权做了明确声明（R1 被保存/恢复）。 一份完整的调用接口说明必须写清这四件事。
- 服务程序**不保存 R0**：R0 是 caller-saved 的输入/输出寄存器，`OUT` 的语义就是"消费 R0 里的字符"。向量表的间接性还让"服务入口地址"成为可替换的数据项：操作系统升级只改 `M[x0021]`，应用程序无需重新汇编。

【内存布局图解】

```
x0000 ┌──────────────────────────────────────────────┐
      │ Trap Vector Table (256 words)                │
x0020 │  x0020: x0400 (GETC)     x0021: x0450 (OUT) ★ │
x0022 │  x0022: x0480 (PUTS)     x0023: x04A0 (IN)    │
x0024 │  x0024: x0520 (PUTSP)    x0025: x04E0 (HALT)  │
x00FF └──────────────────────────────────────────────┘
x0450 ┌──────────────────────────────────────────────┐
      │ TRAP_OUT: ST R1,TOUT_R1                      │
      │ WAIT: LDI R1,OS_DSR / BRzp WAIT              │
      │       STI R0,OS_DDR / LD R1,TOUT_R1 / RET    │
      └──────────────────────────────────────────────┘

执行 TRAP x21 的三个时刻：
  (1) 用户代码: PC = 调用点 → TRAP 令 R7 ← PC, PC ← M[x0021] = x0450
  (2) 服务代码: 轮询 DSR、写 DDR        (3) RET: PC ← R7, 回到用户代码的下一条指令
```

【与汇编的对应】：C 的 `putchar('A')` 在真实系统上会走到 `write(1, &c, 1)` 系统调用， 其结构与此处完全同构——用户态把参数放进约定好的位置（寄存器/栈），执行一条陷入指令， 内核用编号（x86-64 上是 `rax` 中的系统调用号）查表分派，服务完成后返回用户态； 区别只有两点：真实机器**真的切换特权级**，并且用专门的返回指令（LC-3 的 `RTI`）。

示例 3：C 中的轮询输入（`getchar` 循环）

代码 (C):

```c
/* ECE 220 -- Lecture 2 example: a polling (busy-wait) I/O loop in C.
 *
 * This is the C equivalent of the LC-3 idiom
 *     POLL LDI R1,KBSR / BRzp POLL      (wait for KBSR[15] = ready)
 *     LDI R0,KBDR                       (then take the data)
 * Replacing the status register with "did getchar() return EOF?" makes the
 * synchronisation idea visible without any hardware.
 */
#include <stdio.h>

int main(void)
{
    int ch;
    int count = 0;

    printf("echo> ");

    /* poll the keyboard once per iteration; stop at the end of the line */
    while ((ch = getchar()) != EOF) {
        if (ch == '\n') {
            break;
        }
        putchar(ch);            /* "store R2 to DDR" */
        count = count + 1;
    }
    putchar('\n');

    printf("read %d characters before the linefeed\n", count);
    return 0;
}
```

编译与运行：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l02_poll.c -o ece220_l02_poll
$ printf 'Hi 42\nignored line\n' | ./ece220_l02_poll
echo> Hi 42
read 5 characters before the linefeed
```

【代码做什么？】

1. 先打印提示 `echo>`（相当于 `PUTS`）。

2. 每次循环调用一次 `getchar()`：这就是"读一次 KBDR"，只不过函数内部替我们处理了 KBSR 的等待。
3. 读到换行符或 EOF（管道输入结束 / Ctrl-D）就结束循环。
4. 每读到一个普通字符就 `putchar(ch)` 回显（相当于 `STI R2,DDR`）并让计数加一，最后打印统计结果。

【底层机制透视】

- `getchar()` 与 LC-3 的 `GETC` (TRAP x20) 是同一层抽象：**阻塞式读一个字符**。 在终端上，标准库还会做行缓冲 (line buffering)——按键先攒在缓冲区里，直到按下回车才交给程序， 所以"轮询"发生在库/内核内部，而不是在每一行 C 代码里。
- `putchar` 与 TRAP x21 (`OUT`) 对应；二者都不返回"设备是否 ready"，因为同步已经被库/内核吸收了。
- 输入输出被抽象成**字节流 (stream)** 后，程序不再需要知道 DDR/DSR——这是"用一层间接换取可移植性"的又一次体现。 管道输入使 `EOF` 在读完所有字节后出现，程序能确定性地结束；交互式终端上则需要 Ctrl-D。

【内存布局图解】

```
栈帧（main 的帧，automatic storage duration）
+--------------------+  0x7ffd....
|  ch        (int)   |   ← 每次 getchar() 的返回值
|  count     (int)   |   ← 5（本例中）
|  调用现场（R7 等）   |   ← printf/putchar/getchar 都是函数调用
+--------------------+

库/内核侧（对应 LC-3 的设备寄存器）
  stdin 缓冲区:  'H' 'i' ' ' '4' '2' '\n' ...   ← 行缓冲（按下回车才交给程序）
  终端设备（概念上）: KBSR 状态位 / KBDR 数据      stdout 缓冲区: 'e' 'c' 'h' 'o' '>' ' '
```

【与汇编的对应】

```
; C: while ((ch = getchar()) != EOF) { ... }
POLL_IN
        GETC                ; TRAP x20：读一个字符到 R0[7:0]（内部已完成 KBSR 轮询）
        ADD  R1,R0,#-10     ; 与 ASCII 换行符 x0A 比较
        BRz  DONE           ; 读到换行 → 结束

; C: putchar(ch);   R0 此时就是刚才读到的字符
        OUT                 ; TRAP x21：输出 R0[7:0]

; C: count = count + 1;
        ADD  R2,R2,#1       ; R2 是计数器
        BRnzp POLL_IN
DONE
        ; C: printf("read %d characters ...\n", count);
        ; 需要把 R2 中的二进制数转成十进制字符串，再逐字符 OUT
```

示例 4：格式化 I/O 与返回值（以及一个被 `-Werror` 拦下的错误）

代码 (C):

```c
/* ECE 220 -- Lecture 2 example: formatted I/O and its return values. */
#include <stdio.h>

int main(void)
{
    int a = 0;
    int b = 0;
    int converted;
    int written;

    printf("Enter two integers: ");

    /* scanf returns the number of successful conversions */
    converted = scanf("%d%d", &a, &b);
    printf("scanf converted %d value(s)\n", converted);

    if (converted != 2) {
        printf("Bad input!\n");
        return 1;
    }

    /* every character of the format is printed literally except %... */
    written = printf("%d + %d = %d\n", a, b, a + b);
    printf("printf returned %d (the number of characters written)\n", written);

    return 0;
}
```

编译与运行：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l02_fmtio.c -o ece220_l02_fmtio
$ printf '6 7\n' | ./ece220_l02_fmtio
Enter two integers: scanf converted 2 value(s)
6 + 7 = 13
printf returned 11 (the number of characters written)
```

【代码做什么？】

1. `printf("Enter two integers: ")` 输出提示（`PUTS` 的角色）。
2. `scanf("%d%d", &a, &b)` 读两个十进制整数：`%d` 把 ASCII 数字序列**转换成补码**存进变量，`&` 传的是地址。
3. 检查返回值 `converted`：`scanf` 返回成功转换的个数（这里是 2），失败时返回 0 或 EOF（−1）。 幻灯片强调：**检查 `scanf` 的返回值是判断用户输入是否合法的标准做法**。
4. `printf("%d + %d = %d\n", ...)` 打印结果并把返回值存下来（`printf` 返回实际输出的字符数，这里是 11）， 最后打印它，说明"函数调用是表达式，有值"。

【底层机制透视】

- `scanf` 必须拿到**变量的地址**：它要往调用者的栈帧里写数据。这正是 LC-3 里"用寄存器传指针"的对应物——被调用者无法直接访问调用者的局部变量，只能通过地址间接写入。
- `printf("...%d...", a)` 的工作是"把二进制补码按十进制转成 ASCII 再输出"， 与示例 1 把二进制转成 `'0'` / `'1'` 是同一类算法，只是进制不同。这解释了为什么 C 里 "输出一个数"比 LC-3 里"输出一个数"省事：转换逻辑被搬进了库。
- 两个函数都遵循调用约定：参数按约定位置传递，返回值放在约定的位置（在 LC-3 上是 `R0`）。 本课 MP2 之后自己写的十进制打印子程序，就是在实现 `printf("%d")` 的一小部分。

【内存布局图解】

```
main 的栈帧
+---------------------+  0x7ffd...
|  a   = 6            |   ← scanf 通过 &a 直接写入调用者的帧
|  b   = 7            |
|  converted = 2      |   ← scanf 的返回值
|  written   = 11     |   ← printf 的返回值
+---------------------+
字符串常量（.rodata，静态存储期）："Enter two integers: "  "%d%d"
```

【与汇编的对应】

```
; C: scanf("%d%d", &a, &b)  —— 把地址压栈后调用（第 4 讲的帧布局）
        LEA  R0,FMT_DD          ; 参数 1：格式串地址
        ADD  R6,R6,#-1          ; 压参（右到左：先压最后一个参数）
        STR  R0,R6,#0
        LEA  R0,B_ADDR          ; &b 的地址（用 LEA 得到局部变量的地址）
        ADD  R6,R6,#-1
        STR  R0,R6,#0
        JSR  SCANF
        LDR  R0,R6,#0           ; 返回值 = 成功转换个数
        ADD  R6,R6,#4           ; 弹掉返回值与 3 个参数

; C: OUT (TRAP x21) 与 printf 的关系：printf 最终也只是一串字符输出
        ADD  R0,R2,#0           ; 把要打印的字符放进 R0
        OUT                     ; TRAP x21
```

仅供演示、请勿模仿——类型不匹配的 `printf`（编译期被 `-Werror` 拦下）：

```
#include <stdio.h>
int main(void)
{
    /* UB：%d 期待 int 却给了 double；%f 期待 double 却给了 int */
    printf("%d %f", 10.0, 17);
    return 0;
}
```

用课程命令 `gcc -g -std=c99 -Wall -Werror` 编译时它**根本不会通过**，本机 gcc 12.2.0 的真实报错是：

```
error: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-Werror
error: format '%f' expects argument of type 'double', but argument 3 has type 'int' [-Werror
cc1: all warnings being treated as errors
```

去掉 `-Werror` 后它能编译，本机运行输出 `17 10.000000`——但这是**未定义行为**，输出取决于 ABI 与寄存器分配， 换一台机器就会变（幻灯片给出的另一组结果 `0 0.000000` 同样"合法"）。 这正是课程要求 `-Wall -Werror` 的原因：**让编译器替你在运行前抓住这类错误**。

常见错误与调试技巧

- **不检查状态位就直接读数据**：在 KBSR[15] = 0 时读 KBDR，读到的是没有意义的比特。 现象是随机字符、程序行为不可复现。**调试**：在 `lc3sim` 里对轮询循环设置断点，用 `print R1` 查看状态位， 确认 `BRzp` 的方向（N = 0 = 未 ready 时才继续等）；在 C 里检查 `scanf` / `getchar` 的返回值。
- **用 `LD` / `ST` 而不是 `LDI` / `STI` 访问设备**：`DSR .FILL xFE04` 之后写 `LD R4,DSR` 只会拿到 `xFE04` 这个常量。 现象是轮询循环永远等不到 ready（或立刻通过）。**调试**：在 `lc3sim` 中用 `list` 查看该地址的**内容**， 再确认指令助记符是 `LDI` / `STI`； `print R4` 应显示 `xFE04` 而不是设备状态。
- **把 `xFE00` 当作普通内存写入**：例如用 `ST R0,ADDR`（而不是 `STI`）会覆盖"存放设备地址的那个字"，破坏程序数据。 现象是后续指令取到错误地址。**调试**：用 `dump xFE00 xFE10` 观察设备区，确认你把"地址常量"和"设备内容"分了清楚。
- **在子程序里用 TRAP 却忘了保存 R7**：`TRAP` 与 `JSR` 一样会覆盖 R7，导致 `RET` 跳回错误位置或陷入死循环。 现象是子程序返回后行为错乱（课程 `readnumsub.asm` 正是为此写 `ST R7,SAVE_R7`）。 **调试**：在每个用到 TRAP/JSR 的子程序出入口观察 R7；在 `gdb` 里用 `bt`（backtrace）看调用栈是否异常展开。
- **`printf` / `scanf` 的格式串与参数不匹配**：`%d` 配 `double`、`%f` 配 `int`、少写 `&`、 格式符个数与参数个数不符。现象是垃圾输出或段错误。**调试**：始终使用 `-Wall -Werror` 编译； 运行时用 `gdb` 看 `p a`、`x/2xb &a`； `scanf` 前打印 `&a` 确认传的是地址。
- **忘记 TRAP 会改变 R0**：`GETC`、`IN` 等以 R0 作为输出寄存器；若调用前把重要数据放在 R0，就会丢失。 现象是"某个值莫名其妙变成刚输入的字符"。**调试**：写寄存器用途表；用 `gdb` 的 `watch` 或 `lc3sim` 的断点逐条观察 R0 的变化。

关键要点

- LC-3 用**内存映射 I/O**：设备寄存器占据 `xFE00`–`xFFFF`，用 `LDI` / `STI` 访问；代价是这些地址不能再当普通内存用。
- 每个设备都是"**状态寄存器 + 数据寄存器**"的一对：`KBSR/KBDR`、`DSR/DDR`； 状态位固定在 **bit 15**，读入后**正好落在 N 条件码上**，所以判断"是否 ready"只需一条 `BRzp` / `BRn`。
- 同步只能靠**握手协议**：先查状态、再传数据；违反协议会丢字符或读到无意义比特。 处理器比人快几千万倍，因此轮询几乎总是在空转——这是后面引入中断的动机。
- `TRAP` 就是"**查表的子程序调用**"：`R7 ← PC; PC ← M[ZEXT16(vec8)]`， 服务程序以 `RET` 结束；六个常用服务是 `x20 GETC / x21 OUT / x22 PUTS / x23 IN / x24 PUTSP / x25 HALT`。 因为 R7 会被覆

盖，任何用到 TRAP 的子程序都必须保存 R7。

- TRAP 展示的是真实系统调用的结构（编号 → 向量表 → 特权代码 → 返回），并把"库"落到实处： 系统调用是一组**按编号访问的、预装的、带特权的子程序**。

思考题（带答案）

问题 1：为什么 LC-3 的键盘和显示器寄存器地址是 `xFE00, xFE02, xFE04, xFE06` 这样"隔一个"排列的？如果 LC-3 是字节可寻址的机器，同样的 I/O 映射会带来什么便利？

答案：因为 P&P 为高年级定义了一个字节可寻址的变体 **LC-3b**。在一个字占两个字节的字节寻址机器上，"每两个字地址之间空一格"正好给出字节粒度的相邻映射，使同一份 I/O 地址常量在两种机器上都可用（本课只关心字寻址）。 便利之处在于设备寄存器可以按字节访问，外设协议（如 8 位 ASCII 数据）与内存布局能一一对应，不需要"读一整字再屏蔽高位"。

问题 2：下面这段 LC-3 代码想等待键盘输入，但它有 bug。请指出问题并给出正确写法。

```
        LDI  R1,KBSR
        BRn  WAIT           ; "没准备好就继续等"
WAIT    LDI  R1,KBSR
        BRn  WAIT
        LDI  R0,KBDR
```

答案：`BRn` 的判断方向反了。KBSR[15] = 1 表示 **ready**，此时 N = 1； 所以"继续等待"的条件应该是 N = 0，即应写 `BRzp WAIT`（若还想排除 N=1 之外的情况可写 `BRz`）。 这段代码的效果恰好相反：**没按键时（N=0）它会直接往下走去读 KBDR**，读到无意义的比特； 而按键后（N=1）它反而卡在循环里不动。正确写法是 `LDI R1,KBSR` / `BRzp POLL` / `LDI R0,KBDR`， 其中 `POLL` 是 `LDI R1,KBSR` 那一行的标号。

问题 3：`TRAP x21`（OUT）与 `JSR` 调用一个打印字符的子程序，在机器层面有什么相同、什么不同？ 为什么课程说 TRAP 是"带特权切换的子程序调用"？

答案：**相同**：两者都是 `R7 ← PC` 然后改变 PC；服务程序/子程序都以 `RET`（`JMP R7`）结束； 都遵守"R7 是 caller-saved"这条约定，因此调用者若还要用 R7 必须自己保存。 **不同**：`JSR` 的目标地址由指令里的 `PCoffset11`（或 `JSRR` 的寄存器）直接给出，服务代码与调用者被链接进同一个程序； `TRAP` 的目标地址来自**陷阱向量表** `M[ZEXT16(vec8)]`，服务代码由操作系统预先装好，应用程序只提供 8 位编号。 在真实机器上，TRAP（系统调用）还会**提升特权级**——因为服务代码需要访问用户态代码无权触碰的硬件， 这正是 LC-3 中 PSR[15] 所代表的机制；LC-3 的 ISA 本身不实现这一切换，但结构已经完全具备， 这也是"用一层间接（向量表）换取可升级性"的经典例子。

Lecture 3: 子程序与调用约定 (Repeated Code: TRAPs, Subroutines and the Call Interface Specification)

概述

当同一段代码需要在程序里用很多次时，"复制粘贴"会把 bug 也一起复制，于是本讲引入**子程序 (subroutine)**： 把公共代码放在内存里的一个位置，用 `JSR` / `JSRR` 跳进去、用 `RET`（`JMP R7`）跳回来。 真正困难的部分不是跳转，而是**跳转前后的约定**——参数怎么传、结果怎么回、哪些寄存器会被改， 这就是**调用接口规范 (Call Interface Specification, CIS)**。本讲把 LC-3 的调用约定 （`R0–R3` caller-saved、`R4` 全局数据指针、`R5` 帧指针、`R6` 栈指针、`R7` 返回地址）讲清楚， 并说明为什么它必须**系统化**：编译器是程序，只能机械地生成代码，而且不同编译器生成的调用者与被调用者必须能互相配合； 下一讲把这个约定落实到栈帧上，C 的函数调用则是它的直接翻译。

核心概念与底层机制图解

- **代码复用 (Code Reuse)**：一段公共代码只写一次、被多处调用。
  - *直观解释*：不要每间教室都装发电机，而是建一个电厂拉线过去；代价是必须约定"电压、频率、接头"。
  - *底层机制图解*：LC-3 里"调用"的最原始形式是两步——**把返回地址放进寄存器**，然后跳过去：

    `顺序代码（复制粘贴） 子程序（复用） x3000 …公共代码…（第 1 份）x3000 JSR READNUM ; 调用点 1 x3010 …公共代码…（第 2 份）x3010 JSR READNUM ; 调用点 2 x3020 …公共代码…（第 3 份）x3050 READNUM …公共代码… ；改一次要改三处；占用 3×长度 RET ; JMP R7 ；改一次只改一处；占用 1×长度` LC-3 有两个"调用"指令：`JSR`（把指令里的 11 位偏移加到 PC 上）和 `JSRR`（跳到寄存器里的地址）；两者都做同一件事：**把返回地址保存到 R7**。 * *作用域与存储期*：子程序代码与全局数据一样具有 static storage duration；而"调用"产生的是**运行期的活动 (activation)**，其局部数据只活一次调用那么长——这就是 C 中 automatic 变量的来源。
- **JSR / JSRR / RET 的机器编码 (Encoding) 与可达范围**：决定一次调用能"跳多远"。
  - *直观解释*：`JSR` 像"往前走 N 步"（步数写死在指令里），`JSRR` 像"照着纸条上的地址走"（地址在寄存器里）。
  - *底层机制图解*：

    `bits 15 14 13 12 | 11 | 10 ..... 0 ----------------------------------------- JSR 0100 | 1 | PCoffset11 ; R7 ← PC; PC ← PC + SEXT(PCoffset11) JSRR 0100 | 0 | 0 0 BaseR 0 0 0 0 0 0 ; R7 ← PC; PC ← BaseR JMP(RET) 1100 | 0 | 0 0 BaseR 0 0 0 0 0 0 ; PC ← BaseR` `PCoffset11` 是 **11 位补码**（−1024 .. +1023），基准是"取指后已加一的 PC"。以 `.ORIG x3000` 调用入口在 `x4000` 的子程序为例：`JSR SUB` 从 `x3000` 出发时 `PC = x3001`，需要 `offset = x4000 − x3001 = 4095`，超过 `+1023` → **汇编器报错**；改用 JSRR：

    `LD R1,SUB_ADDR ; R1 ← M[SUB_ADDR] = x4000（地址从内存里取） JSRR R1 ; R7 ← x3001（返回地址！），PC ← x4000 SUB_ADDR .FILL SUB ; 一个字的数据，指向远处子程序的入口` **关键区别**：`JSRR` 里的寄存器放的是**被调用者的地址**，而 R7 收到的永远是**返回地址**——两者方向相反。 * *作用域与存储*

期：`PCoffset11` 是编译期常量（作用域仅这一条指令），`JSRR` 的 BaseR 是运行期值， 可以指向任何地址，灵活性换来了可读性的下降。

- **调用接口规范 (Call Interface Specification, CIS)**：一套说明"调用者与被调用者如何合作"的契约，包含四个部分。
  - *直观解释*：像两个陌生人合作搬家具——事先说好从哪个门进（输入）、放到哪里（输出）、 谁不碰对方的东西（寄存器所有权）、会不会顺手把墙刷了（副作用）。
  - *底层机制图解*：

    `CIS 的四个部分 1. 输入（inputs）：参数放在哪里？（LC-3 常用 R0..R3，或放在栈上） 2. 输出（outputs）：结果放在哪里？（返回寄存器/栈顶） 3. 其它寄存器的所有权 ：哪些寄存器可能被改（caller-saved），哪些保证不变（callee-saved） 4. 副作用（side effects）：改了哪些内存、做了哪些 I/O 以课程的 READNUM（readnumsub.asm）为例： 输入 : 键盘；输出 : R0 = 读到的二进制补码数 保存 : R1、R2、R3、R7（内部 ST/LD 保存并恢复） 副作用: 键盘输入、回显字符、可能打印错误信息` 课程 MP 要求**每个子程序都写下寄存器用途表**并把接口写成注释——"契约"必须可检查。 * *作用域与存储期*：CIS 决定了子程序作者的"作用域"边界：**只有写进接口的东西调用者才可能依赖**， 其它一切（内部临时变量、是否递归）都是可随时更换的实现细节。
- **为什么必须"系统化" (Why Systematic)**：编译器是程序，只能按规则生成代码；而且不同来源的代码必须能互操作。
  - *直观解释*：如果每个厂家都自定插头尺寸，电器就没法互通；调用约定就是编程世界的"插座标准"。
  - *底层机制图解*：

    `调用者编译器必须知道：参数放哪里 / 调用前保存哪些寄存器 / 返回值从哪里取 被调用者编译器必须知道：去哪里取参数 / 结果放哪里 / 哪些寄存器必须原样返回 └──────→ 同一份 CIS ←──────┘ 分开编译 (separate compilation)：caller.c --(gcc -c)--> caller.o ┐ callee.c --(gcc -c)--> callee.o ┴--(ld)--> 可执行文件 两个 .o 文件里只有符号名与约定，没有对方的源码` C 的**函数声明 (prototype)** 是 CIS 在语言层的表达（参数个数、类型、返回类型）， 但**没有**说明副作用与寄存器所有权——那是 ABI 规定的。 * *作用域与存储期*：分开编译意味着"作用域"跨文件——函数名与全局变量具有**外部链接 (external linkage)**， `static` 则限制在文件内，正对应子程序的"私有实现细节"。
- **LC-3 调用约定 (Calling Convention)**：五个位置各有专职，八个寄存器都被分配了角色。
  - *直观解释*：R0—R3 是"一次性便签"（用完就扔），R4—R7 是"常设机构"（职责固定，不能乱动）。
  - *底层机制图解*：

    `+------+--------------------+---+ 寄存器文件 | R0 | 参数 / 返回值 / I/O | caller-saved：被调用者可以随意改 | | R1 | 参数 / 返回值 / 临时 | caller-saved | | R2 | 参数 / 返回值 / 临时 | caller-saved | | R3 | 参数 / 返回值 / 临时 | caller-saved | | R4 | 全局数据指针 | 约定要求保持：指向全局数据区（本课约定 x4000 起） | | R5 | 帧指针（frame ptr） | 由帧机制恢复：指向当前栈帧（第 4 讲详解） | | R6 | 栈指针（stack ptr） | 由帧机制恢复：指向栈顶（第 4 讲详解） | | R7 | 返回地址 | **永远 caller-saved**：JSR/JSRR/TRAP 都会覆盖它 | +------+--------------------+---+` **R7 特殊在哪里**：它不是"可能被改"而是"一定会被改"（`JSR` 的语义就是 `R7 ← PC`）， 所以子程序**无法**为调用者保留 R7；调用者若在调用后还需要 R7，必须在调用前把它存到内

存或栈里。 **注意（课程 MT1 复习的原话）**：LC-3 在编译器层面**没有 callee-saved 寄存器**——"Callee-saved registers at the start and end of the function (none in LC-3)"； 上表"约定要求保持"只是说 R4–R6 的角色不随调用改变，具体保存/恢复仍必须由代码自己写出（第 4 讲的帧机制负责 R5/R6）。 * *作用域与存储期*：R4–R7 的"角色"在整个程序运行期不变，R0–R3 的值只保证到下一次调用为止——这就是"caller-saved"这个名字的含义。

- **寄存器保存纪律 (Register Saving Discipline)**：谁负责保存，取决于寄存器是 caller-saved 还是 callee-saved。
  - *直观解释*：借用规则只有两条——"易耗品你自己备份"（caller-saved），"借了家具要还回来"（callee-saved）。
  - *底层机制图解*：ECE 220 的两种保存方式（第 4 讲会升级为压栈）：

    `方式 A：调用者保存（把 R0..R3 存进自己的内存槽） ST R1,SAVE_R1 ; 调用前：子程序可能改 R1，先存起来 JSR SUB LD R1,SAVE_R1 ; 调用后恢复；子程序完全不用管 R1 方式 B：被调用者保存（子程序承诺不改某些寄存器） SUB ST R7,SUB_R7 ; JSR/TRAP 覆盖了 R7，先存起来 ST R2,SUB_R2 / LD R2,SUB_R2 ; 声明并兑现"R2 是 callee-saved" LD R7,SUB_R7 RET SAVE_R1 .BLKW 1 ; 注意：.BLKW 不初始化，内容是垃圾`

    选择原则：**用得多的寄存器让被调用者保存**（如 R4–R7 这类"常设"寄存器）， 只在少数调用点用到的让调用者保存，可以省掉大量无谓的存取。 * *作用域与存储期*：被保存的值具有"调用期间"的存储期；ECE 220 里它们存在固定的 `.BLKW` 槽，子程序一旦可重入或递归就会互相覆盖——所以第 4 讲必须把它们放进栈帧。
- **特权、陷阱与库 (Privilege, Traps and Libraries)**：系统服务也是子程序，只是多一层保护。
  - *直观解释*：普通程序像访客只能走前台（TRAP）；操作系统像内部员工，可以进机房（I/O 寄存器）。
  - *底层机制图解*：

    `保护的三层含义：1. 保护硬件（写错 BRz 可能让设备失效）2. 保护用户（进程不能互踩内存） 3. 保护系统（内核代码不能被随意改写） 实现手段：特权级（PSR[15]）；0 = privileged，1 = unprivileged。 TRAP = 子程序调用 + 特权切换（概念上）： 用户代码 --TRAP x21--> 陷阱向量表 --> 服务子程序（特权） --RET--> 用户代码 陷阱 vs 中断：陷阱**同步**（程序主动请求），中断**异步**（设备主动发起）；中断时被中断的代码毫无准备， 因此**所有寄存器（包括 R7）都必须是 callee-saved**，由处理程序保存全部状态并用 RTI 返回。`

    * *作用域与存储期*：特权级决定了同一条 `STI R0,DDR` 的**合法性**；"库"隐藏实现、只暴露 CIS。
- **与 C 的对应 (Connection to C)**：C 的函数 = 子程序 + 调用约定 + 类型信息。
  - *直观解释*：`int32_t f(int32_t x);` 是门面，门后面是 R0 放参数、JSR 进去、R0 取结果。
  - *底层机制图解*：

    `C 概念 对应的底层机制 函数声明（prototype） CIS：参数/返回值的类型与个数（ABI 补充寄存器所有权） 函数定义 一段以 RET 结束的指令序列 局部变量（automatic） 被调用者的栈帧（第 4 讲） 全局/static 变量 R4 指向的全局数据区 return 语句 把值放进约定位置（LC-3 上是 R0 / 栈顶），再 RET 递归 每次调用都要有**独立的**帧（第 4 讲）`

    教学目标 3 要求"理解栈抽象与调用约定在调用者与被调用者之间传递信息的作用"：本讲给出约定，下讲给出载体。 * *作用域与存储期*：C 的 scope 决定"名字能否被看到"、storage duration 决定"数据活多久"， 两者最终都由 CIS + 栈帧实现。

代码示例与底层机制分析

示例 1：写一个真正的子程序 PRINT_BIN16（含完整调用者与被调用者）

代码 (LC-3 assembly):

```
; ------------------------------------------------------------------
; PRINT_BIN16 -- 把 R0 的低 16 位按二进制打印到显示器
; 调用接口规范 (CIS)
;   输入: R0 = 要打印的值     输出: 无（写 16 个 ASCII '0'/'1' 到显示器）
;   改变: 无（R1/R2/R3 由子程序保存恢复；R0 返回时被破坏）
;   保存: R7（本子程序用 TRAP x21，TRAP 会覆盖 R7）
;   副作用: 向显示器输出 16 个字符
; 说明：本讲用固定 .BLKW 槽保存寄存器；第 4 讲改用栈帧。
; ------------------------------------------------------------------

        .ORIG x3000
        LD   R1,KEEP_ME      ; x3000: 调用者的活数据，必须活过两次调用
        LD   R0,VALUE_A      ; x3001: 参数 1
        JSR  PRINT_BIN16     ; x3002: 第一次调用
        LD   R0,VALUE_B      ; x3003: 参数 2
        JSR  PRINT_BIN16     ; x3004: 第二次调用
        HALT                 ; x3005

PRINT_BIN16                  ; x3006: 被调用者从这里开始
        ST   R7,PB_R7        ; x3006: TRAP 会覆盖 R7，先保存
        ST   R1,PB_R1        ; x3007: 保存 R1（本子程序承诺不改 R1）
        ST   R2,PB_R2        ; x3008: 保存 R2
        ST   R3,PB_R3        ; x3009: 保存 R3

        LD   R3,PB_ZERO      ; x300A: R3 ← x30 = ASCII '0'
        AND  R1,R1,#0        ; x300B: R1 ← 0
        ADD  R1,R1,#15       ; x300C: R1 ← 15（位下标，从最高位开始）

PB_LOOP ADD  R2,R3,#0        ; x300D: R2 ← '0'
        ADD  R0,R0,#0        ; x300E: 刷新 N/Z/P，用 N 检查 bit 15
        BRzp PB_SKIP         ; x300F: bit 15 是 0 就跳过
        ADD  R2,R2,#1        ; x3010: 否则 R2 ← '1'
PB_SKIP ADD  R0,R0,R0        ; x3011: 左移一位，把下一位送到 bit 15
        ST   R0,PB_R0        ; x3012: OUT 要用 R0，先把移位结果存起来
        ADD  R0,R2,#0        ; x3013: 把要打印的字符放进 R0
        OUT                  ; x3014: TRAP x21，输出 R0[7:0]
        LD   R0,PB_R0        ; x3015: 取回移位结果
        ADD  R1,R1,#-1       ; x3016: 位下标减一
        BRzp PB_LOOP         ; x3017: 还有位就继续

        LD   R3,PB_R3        ; x3018: 恢复调用者的寄存器
        LD   R2,PB_R2        ; x3019
        LD   R1,PB_R1        ; x301A
        LD   R7,PB_R7        ; x301B: 恢复返回地址
        RET                  ; x301C: JMP R7

; ---- 子程序的私有数据（.BLKW 不初始化，第一次调用前内容是垃圾）----
PB_R0   .BLKW 1              ; x301D
PB_R1   .BLKW 1              ; x301E
PB_R2   .BLKW 1              ; x301F
PB_R3   .BLKW 1              ; x3020
PB_R7   .BLKW 1              ; x3021
PB_ZERO .FILL x30            ; x3022: ASCII '0'
```

```
; ---- 调用者的数据 ----
KEEP_ME .FILL #7              ; x3023
VALUE_A .FILL xABCD           ; x3024
VALUE_B .FILL xBC3D           ; x3025
        .END
```

> ⚙ **机器验证**：已用 `ece220_data/lc3sim.py` 汇编并运行，**370 条指令**后 `HALT`， 显示器输出 `1010101111001101011110000111101` —— 正好是两次调用的结果首尾相接： `1010101111001101` (= `xABCD`) 与 `1011110000111101` (= `xBC3D`)。 这同时验证了"被调用者保存 R1–R3"这条承诺：调用者的值跨过两次调用后仍是原来的比特。

【代码做什么？】

1. 调用者先把"必须活过调用"的 `KEEP_ME` 装进 R1，然后装参数、`JSR`； `JSR` 做两件事：`R7 ← PC`（返回地址），`PC ← PRINT_BIN16`（第 1 次 `R7 = x3003`，第 2 次 `R7 = x3005`）。
2. 被调用者先保存 R7 与 R1–R3（它承诺不改 R1–R3），再进行计算。
3. 循环 16 次：用 `ADD R0,R0,#0` 把 bit 15 送进 N，选 `'0'` 或 `'1'`，左移，输出；每次输出前把移位结果 存进 `PB_R0`，因为 `OUT`（TRAP x21）会**消费 R0**；循环结束后恢复 R3、R2、R1、R7， `RET` 回到调用点。

【底层机制透视】

- **R7 的保存位置**：本讲把 R7 存进 `.BLKW` 槽，这在"不递归、不重入"时够用，但两次调用会互相覆盖同一槽； 递归时必须把保存区搬到栈帧里（第 4 讲）。同理 `.BLKW` **不初始化**，第一次调用前内容是垃圾——本程序总是"先 ST 再 LD"，所以安全；若直接 `LDR` 去读它就会拿到随机值。
- **caller-saved 的实际后果**：子程序结束时 R0 里是最后一次 `OUT` 的字符，调用者的原参数已经丢了； 这对本程序无害（每次都重新 `LD R0,VALUE_x`），但若调用者以为 `R0` 还是参数，就会写出难查的 bug。
- **为什么 `ST R0,PB_R0` 是必需的**：`OUT` 需要 R0 持有字符，而 R0 同时承担"待打印数值"的角色—— 这是 LC-3 只有 8 个寄存器时的典型权衡：**用一次内存访问换取一个寄存器**。

【内存布局图解】

```
地址       内容                          说明
x3000/x3001  LD R1,KEEP_ME / LD R0,VALUE_A  (off=+34)  调用者：装入活数据与参数
x3002      JSR PRINT_BIN16 (off=+3)     第一次调用的返回地址 = x3003
x3003/x3004  LD R0,VALUE_B / JSR (off=+1)  第二次调用的返回地址 = x3005
x3006      ST R7,PB_R7      (off=+26)   ← 被调用者从这里开始（x3006..x301C 为 16 条循环指令）
x301C      RET  (JMP R7)
x301D..x3021  PB_R0..PB_R7  .BLKW 1 ×5  子程序私有存储（第一次调用前是垃圾值）
x3022      PB_ZERO x0030
x3023..x3025  KEEP_ME #7 / VALUE_A xABCD / VALUE_B xBC3D   调用者的数据（code/global data 区）

两次调用期间 PB_R7 的变化：第 1 次 PB_R7 ← x3003（返回 x3003），第 2 次 PB_R7 ← x3005（返回 x3005）
```

【与汇编的对应】（逐条手算两次调用的关键状态）

时刻	R0	R1	R2	R3	R7	说明
执行第 1 次 `JSR` 前 / 进入子程序后	xABCD	#7	?	?	**x3003**	参数就位；JSR 写入返回地址
`ADD R1,R1,#15` 后	xABCD	#15	—	x30	x3003	计数 15，P=1
第 1 次循环迭代	x579A	#14	x31	x30	x3003	打印 `1`（xABCD 的 bit15）；16 次后 **x0000 / #−1**
恢复 + `RET` 之后	x0000	**#7**	原值	原值	x3003	R1 被恢复（承诺兑现），PC ← x3003
第 2 次调用返回后	x0000	#7	原值	原值	x3005	PC ← x3005（`HALT`）

两次调用在显示器上输出的字符串依次是 `1010101111001101`（= `xABCD`）与 `1011110000111101`（= `xBC3D`）； `R1` 中的 `KEEP_ME = 7` 在两次调用后**仍然等于 7**，这就是"callee-saved"承诺的实际含义。

示例 2：C 里"看得见"的调用约定（把寄存器模拟成变量）

代码 (C):

```
/* ECE 220 -- Lecture 3 example: the call interface specification, in C.
 *
 * The five file-scope variables stand in for LC-3 registers so that we can
 * literally watch a callee clobber a caller-saved register.
 */
#include <stdio.h>
#include <stdint.h>

static int32_t R0;   /* input / return value : caller-saved */
static int32_t R1;   /* input                : caller-saved */
static int32_t R2;   /* temporary            : caller-saved */
static int32_t R3;   /* temporary            : CALLEE-saved */
static int32_t R7;   /* return address       : always caller-saved */

/* PRINT_BIN16
 *   input : R0 (value to print, low 16 bits)
 *   output: none (writes 16 ASCII digits and a linefeed to the display)
 *   changes      : R2 (caller-saved)
 *   preserves    : R3 (callee-saved, saved and restored here)
 *   side effects : writes to the display
 */
static void PRINT_BIN16(void)
{
    int32_t saved_R3 = R3;      /* the callee keeps its promise here */
    int32_t i;

    R3 = R0;                    /* work on a copy inside the callee   */
    for (i = 15; i >= 0; i = i - 1) {
        R2 = '0';               /* R2 is caller-saved: free to clobber */
        if (((R3 >> i) & 1) != 0) {
            R2 = R2 + 1;        /* '0' + 1 == '1' */
        }
        putchar(R2);            /* "STI R2,DDR" */
    }
    putchar('\n');
    R3 = saved_R3;              /* restore the callee-saved register */
    return;                     /* "RET" (JMP R7) */
}

int main(void)
{
    R0 = 0xABCD;    /* same value as NUMBER in printbinary.asm */
    R1 = 0;         /* the caller's live values ... */
    R2 = 42;
    R3 = 7;
    R7 = 0;

    printf("before the call : R2 = %d, R3 = %d\n", (int)R2, (int)R3);

    /* the caller sets up the inputs, then JSRs */
    printf("PRINT_BIN16(x%04X) = ", (unsigned)(R0 & 0xFFFF));
    PRINT_BIN16();
```

```
    printf("after the call  : R2 = %d (clobbered), R3 = %d (preserved)\n",
           (int)R2, (int)R3);
    return 0;
}
```

编译与运行：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l03_printbin.c -o ece220_l03_printbin
$ ./ece220_l03_printbin
before the call : R2 = 42, R3 = 7
PRINT_BIN16(xABCD) = 1010101111001101
after the call  : R2 = 49 (clobbered), R3 = 7 (preserved)
```

【代码做什么？】

1. 用五个文件作用域变量模拟 LC-3 的 R0、R1、R2、R3、R7，使"寄存器纪律"变成可打印的量。
2. `main` 充当调用者：准备好 R0（参数）与 R2/R3（活数据），打印调用前状态，然后"JSR"。
3. `PRINT_BIN16` 用 R3 做工作副本、用 R2 承载 `'0'`/`'1'`，循环 16 次输出；返回前把 R3 恢复为入口值。 结果：R2 从 42 变成 49（`'1'` 的 ASCII 码），R3 仍是 7——**caller-saved 被改，callee-saved 保住**。

【底层机制透视】

- 打印的 `1010101111001101` 与示例 1 中 LC-3 版 PRINT_BIN16 的输出**逐位相同**（算法都是"从 bit 15 开始、每次左移一位"）； `R2 = R2 + 1` 把 `'0'`（`x30`）变成 `'1'`（`x31`），与 `ADD R2,R2,#1` 完全对应。
- 在这个模拟里，"保存"表现为 C 的局部变量 `saved_R3` ——它就是编译器在栈帧上开辟的槽，与 LC-3 的 `.BLKW` 槽同源。 真实的 C 编译器不会这样使用全局变量：它把参数放进寄存器（x86-64 上是 `rdi`、`rsi`...）， 把 callee-saved 寄存器（`rbx`、`rbp`、`r12`－`r15`）压栈保存——结构一模一样，只是名字不同。

【内存布局图解】

```
全局数据区（模拟"寄存器文件"）          PRINT_BIN16 的栈帧（automatic 存储期）
+----------------+  0x404040 附近      +--------------------+  0x7ffd....（高地址）
| R0 = 0         | ← 参数/返回值        | 保存的返回地址       |
| R1 = 0         |                     +--------------------+
| R2 = 49        | ← 被改成 '1'         | saved_R3 = 7       | ← callee-saved 存这里
| R3 = 7         | ← 仍是 7（被恢复）    | i                  |
| R7 = 0         |                     +--------------------+  0x7ffd....（低地址）
+----------------+
```

【与汇编的对应】

```
; C: int32_t saved_R3 = R3;       —— 被调用者的第一件事：保存 callee-saved 寄存器
        ST   R3,PB_SAVE_R3      ; 用内存槽（第 4 讲改为压栈）
; C: R3 = R0;   R2 = '0';  if (bit) R2 = R2 + 1;
        ADD  R3,R0,#0           ; 在工作副本上做位运算
        LD   R1,ZERO_CH         ; R1 ← x30（ASCII '0'），充当常量寄存器
        ADD  R2,R1,#0           ; R2 ← '0'
        ADD  R3,R3,#0           ; 刷新 N/Z/P：bit 15 决定 N
        BRzp SKIP
        ADD  R2,R2,#1           ; R2 ← '1'
SKIP    ; ... 输出 R2（OUT），然后 ADD R3,R3,R3 左移、计数减一，循环 16 次 ...
ZERO_CH .FILL x30               ; ASCII '0'
; C: R3 = saved_R3;  return;
        LD   R3,PB_SAVE_R3      ; 兑现"不改 R3"的承诺
        RET                     ; JMP R7
```

示例 3：分开编译——"约定"是唯一的纽带

代码 1 (C, 调用者 `ece220_l03_caller.c`):

```c
/* ECE 220 -- Lecture 3, part 1 of the separate-compilation demo: the CALLER.
 * It knows the callee only through the two declarations below. */
#include <stdio.h>
#include <stdint.h>

void    print_bin16(uint16_t value);   /* the call interface */
int32_t read_number(int32_t *out);

int main(void)
{
    int32_t a = 0;
    int32_t b = 0;
    int64_t product;
    int32_t conversions;

    printf("Enter two integers: ");
    conversions  = read_number(&a);
    conversions += read_number(&b);
    if (conversions != 2) {
        printf("Bad input!\n");
        return 1;
    }

    product = (int64_t)a * (int64_t)b;
    printf("a     = %6d = ", (int)a);
    print_bin16((uint16_t)a);
    printf("\nb     = %6d = ", (int)b);
    print_bin16((uint16_t)b);
    printf("\na * b = %6d = ", (int)(product & 0xFFFF));
    print_bin16((uint16_t)(product & 0xFFFF));
    printf("\n");
    return 0;
}
```

代码 2 (C, 被调用者 `ece220_l03_lib.c`):

```c
/* ECE 220 -- Lecture 3, part 2 of the separate-compilation demo: the CALLEE.
 * Everything the caller may rely on is in the two comments below. */
#include <stdio.h>
#include <stdint.h>

/* input: value; output: none; writes 16 ASCII '0'/'1' and a linefeed */
void print_bin16(uint16_t value)
{
    int32_t i;

    for (i = 15; i >= 0; i = i - 1) {
        if (((value >> i) & 1u) != 0u) {
            putchar('1');
        } else {
            putchar('0');
        }
    }
    return;
}

/* input: out; output: number of successful conversions (like scanf) */
int32_t read_number(int32_t *out)
{
    int n;

    if (out == NULL) {
        return -1;
    }
    n = scanf("%d", out);
    return (int32_t)n;
}
```

分开编译与链接（本机真实命令与输出）：

```
$ gcc -g -std=c99 -Wall -Werror -c ece220_l03_lib.c -o ece220_l03_lib.o
$ gcc -g -std=c99 -Wall -Werror -c ece220_l03_caller.c -o ece220_l03_caller.o
$ gcc -g -std=c99 -Wall -Werror ece220_l03_caller.o ece220_l03_lib.o -o ece220_l03_cis
$ printf '43981 5\n' | ./ece220_l03_cis
Enter two integers: a     =  43981 = 1010101111001101
b     =      5 = 0000000000000101
a * b =  23297 = 0101101100000001
```

（也可以一条命令完成：`gcc -g -std=c99 -Wall -Werror ece220_l03_caller.c ece220_l03_lib.c -o ece220_l03_cis`。）

【代码做什么？】

1. `caller.c` 与 `lib.c` 各自单独编译成 `.o`：编译 `caller.c` 时只看到两行声明，**完全不知道**实现； 编译 `lib.c` 时完全不知道谁会用这两个函数。
2. 链接器只做一件事：把 `caller.o` 里对 `print_bin16` 的"未定义引用"接到 `lib.o` 里的同名符号上。

3. 运行时 `main` 调用 `read_number` 两次读入 43981 与 5，再三次调用 `print_bin16`； 输出验证：`43981 = xABCD` → `1010101111001101`， `5` → `0000000000000101`， 乘积 `xABCD * 5 = x15B01` 截断到 16 位得 `x5B01` → `0101101100000001`。

【底层机制透视】

- 这就是"为什么 CIS 必须系统化"的物证：**两个翻译单元之间没有共享的源码或上下文**，唯一的沟通渠道是符号名 + 调用约定；任何一方偷偷改变约定（例如把参数从 R0 改到 R2），编译器都不会报错，运行时才崩溃。
- 链接器只检查**符号名与存储类别**，不检查参数个数/类型——写错声明时 `-Wall -Werror` 才是真正的守门人。
- 编译期看到的"声明"与运行期发生的"寄存器/栈操作"是 CIS 的两层：**文字形式**与**物理形式**； `(uint16_t)(product & 0xFFFF)` 则显式表达了 LC-3 中 16 位寄存器的天然截断行为。

【内存布局图解】

```
     caller.o                          lib.o
+-----------------------+          +-----------------------+
| .text: main           |          | .text: print_bin16    |
|  U print_bin16  ──────┼──┐       |  T print_bin16        |
|  U read_number  ──────┼──┼──┐    |  T read_number        |
+-----------------------+  │  │    +-----------------------+
                       └──┴──┘  ld（链接器）：把符号引用接到定义上
                                ↓
              ece220_l03_cis（两个 .text 段拼接，调用点填上真实地址）
```

【与汇编的对应】

```
; C: print_bin16((uint16_t)a);         —— 调用者的四个动作
        LDR  R0,R5,#0          ; ① 取出实参 a
        ADD  R6,R6,#-1         ; ② 压参数（第 4 讲的帧布局）
        STR  R0,R6,#0
        JSR  PRINT_BIN16       ; ③ 调用（R7 ← 返回地址）
        ADD  R6,R6,#2          ; ④ 弹掉返回值与参数
; 被调用者的入口：把自己承诺的寄存器存起来
PRINT_BIN16
        ST   R7,PB_R7          ; JSR/TRAP 会覆盖 R7，必须先保存
        ST   R3,PB_R3          ; R3 在 CIS 里声明为"不改"，所以必须保存
        ; ...（后续代码略：R2 是 caller-saved，可以随便用）
        RET
```

常见错误与调试技巧

- **忘记保存 R7**：子程序里用了 `JSR` / `TRAP`（包括 `OUT`、`PUTS`），返回地址被覆盖， `RET` 跳错地方。 现象是回到调用点附近的随机位置或死循环。**调试**：在子程序入口/出口打印或观察 R7； `gdb` 中用 `bt` 看调用栈是否合理展开； `lc3sim` 中对 `RET` 设断点，检查 R7 的值是否等于预期的返回地址。

- **破坏 callee-saved 寄存器**：CIS 里承诺"不改 R1"却用了 R1 当临时，调用者的数据被悄悄改掉，现象是"某个变量莫名其妙变了"。**调试**：`gdb` 中对可疑变量 `watch var`，断下后 `bt` 看是谁改的； LC-3 中给子程序出口设断点，逐一 `print R1`…`print R4` 与调用前对比。
- **用 `JSR` 调用太远的子程序**：目标超过 `PCoffset11` 的 ±1024 范围，汇编器报 offset 超范围。**调试**： 改成"先 `LD R1,SUB_ADDR` 再 `JSRR R1`"，确认 `SUB_ADDR .FILL SUB` 的地址正确；用 `lc3sim` 的 `list` 核对地址。
- **把 `JSRR` 的方向记反**：以为 `JSRR R1` 会把子程序地址写进 R7。实际上 R7 得到的是**返回地址**， R1 必须**事先**装有子程序入口地址。现象是 PC 跳到垃圾地址。**调试**：单步执行 `JSRR` 前后 `print R1` 与 `print R7`。
- **调用者假设 R0–R3 在调用后不变**：把循环计数放在 R2 里，调用一个"会改 R2"的子程序后计数被破坏，现象是循环次数错误或死循环。**调试**：`gcc -S` 看编译器如何在调用前后保存/恢复寄存器；`gdb` 里 `info registers` 对比。
- **接口文档缺失或不一致**：写了"输入 R0"却在代码里从 R1 取参数，返回值放在 R0 却让调用者去读栈； 现象是"单独测都对、连起来就错"。**调试**：给每个子程序写完整 CIS 注释（四个部分）； 用示例 3 的**分开编译**强制自己只依赖声明，用 `nm ece220_l03_caller.o` 检查未定义符号都能在 `lib.o` 里找到。

关键要点

- 子程序解决"写一次、用多次"的问题，代价是必须约定接口；LC-3 的调用机制是 `JSR`（11 位偏移）与 `JSRR`（寄存器地址）， 两者都执行 `R7 ← PC`。
- CIS 有四个部分：**输入、输出、其它寄存器的所有权、副作用**——缺任何一部分都无法安全协作。
- LC-3 调用约定：`R0–R3` caller-saved（参数/返回值），`R4` 全局数据指针，`R5` 帧指针，`R6` 栈指针，`R7` 返回地址； **R7 永远是 caller-saved**，因为 `JSR`/`JSRR`/`TRAP` 必然覆盖它。
- 约定必须系统化：编译器只能机械生成代码，分开编译的目标文件之间只有"符号 + 约定"这一条纽带。
- 系统调用（TRAP）是"带特权的子程序"；中断处理程序要求**所有寄存器都是 callee-saved**（被中断的代码毫无准备）。

思考题（带答案）

问题 1：从 `x3000` 开始的程序要用 `JSR` 调用位于 `x4000` 的子程序，为什么汇编器会报错？请给出两种替代方案与代价。

答案：`JSR` 用 11 位补码偏移，基准是"取指后已加一的 PC"：调用点 `x3000` 处 `PC = x3001`， 到 `x4000` 需要 `offset = 4095`，远超 `+1023`，所以汇编器拒绝。替代方案：(1) **`JSRR`**：`LD R1,SUB_ADDR` + `JSRR R1`（`SUB_ADDR .FILL SUB`）——代价是多一条指令与一个字的数据，R1 被占用；(2) **中继跳转 (trampoline)**：在 `JSR` 可达范围内放一个"入口转发"子程序，它再用 `JSRR` 跳到远处—— 代价是多一次调用（R7 要在转发处保存）。LC-3 的 11 位偏移是"指令字只有 16 位"的必然结果。

问题 2：下面这段被调用者的 CIS 注释有什么问题？会给调用者带来什么后果？

```
/*  输入 : R0
 *  输出 : R0
 *  说明 : 内部会使用 R1..R4 作为临时寄存器
 */
```

答案：它**只列出了输入/输出**，却把"R1..R4 会被改"写成实现说明而不是**所有权声明**：`R4` 在 LC-3 约定里是全局数据指针（callee-saved），一旦被改，调用者后续所有通过 R4 访问全局变量的代码都会用到错误地址；R1—R3 虽是 caller-saved，调用者也必须知道"它们会被改"才能决定是否提前保存。正确写法是 "输入 R0；输出 R0；改变 R1/R2/R3（caller-saved，调用者若需要请自行保存）；**R4 保持不变**（若确实要用必须保存并恢复）； 副作用：无"——把所有权写清楚，而不是藏在实现说明里。

问题 3：`JSR`、`TRAP`、`RTI` 三者都会改变 PC 与 R7（或更多状态），各对应什么场景？为什么中断处理必须保存全部状态?

答案：`JSR` 是**普通子程序调用**（程序主动发起，只覆盖 R7）；`TRAP` 是**系统调用**，同样是程序主动发起的 同步事件，语义与子程序调用一致（真实机器上还会提升特权级）；`RTI` 是**中断返回**，用于异步事件。 中断发生时被中断的代码完全不知道这一刻会发生什么，因此处理程序必须保存**所有寄存器（含 R7）与条件码** （在这个意义上"所有寄存器都是 callee-saved"），并用 `RTI` 一次性恢复状态；而陷阱是程序自己发出的请求，调用点清楚"接下来会发生什么"，只需遵守普通调用约定。这正是"同步 vs 异步"在机器层的差别。

Lecture 4: 栈抽象、栈帧与用栈做算术 (The Stack Abstraction, Stack Frames, and Arithmetic Using a Stack)

概述

第 3 讲给出了调用约定，但没有回答"参数、返回值、返回地址、局部变量究竟放在哪里"——本讲给出答案：放在**栈 (stack)** 上的一块连续区域里，这块区域叫**栈帧 (stack frame / activation record)**。 本讲引入 LIFO 语义、两条指令的 push/pop、`R6` 栈指针与 `R5` 帧指针的分工，以及完整的 LC-3 栈帧布局； 并用两个真实例子把它落地：用栈计算后缀表达式 `1 2 + 3 4 - *`（即 `(1+2)*(3-4)`），以及一个带完整帧的加法子程序。 这套机制正是 C 中 automatic 变量、递归与"栈溢出"的物理基础，也是后续数组、指针与动态数据结构的前提。

核心概念与底层机制图解

- **栈抽象 (Stack Abstraction)**：栈是一种**只能在顶端进出**的数据结构，提供 **LIFO（后进先出）** 语义——像一摞餐盘，只能把新盘子放在最上面、也只能从最上面取走。
  - *底层机制图解*:

    `操作 含义 与队列 (queue, FIFO) 的对比 (BFS 用队列，见第 1 讲) PUSH 把数据放到栈顶 队列：从尾部加入 POP 取走栈顶数据 队列：从头部取出 栈的生长方向 (关键约定): 高地址 +-----------+ ← 栈底 (base): 一开始 R6 指向这里，此时栈为空 低地址 +-----------+ ← 栈顶 (top): R6 指向最后一个被压入的字；压栈时地址减小`

内存里"栈顶以上（更高地址）"的内容是**旧数据**：还在内存里但**已不属于栈**——弹出并不擦除数据。 * *作用域与存储期*：栈上的数据是 **automatic storage duration**：进入函数/块时创建、离开时销毁，生命周期与"活跃的调用"严格对应；C 中函数内定义的普通变量就在这类存储里。

- **PUSH / POP 的两条指令实现**：LC-3 没有 `PUSH` / `POP` 指令，但每个操作恰好用两条指令完成——压栈 = 先把手指往下挪一格再放东西，弹栈 = 先拿起东西再把手指往上挪一格。
  - *底层机制图解*：

    `压栈（push R0） 弹栈（pop 到 R0） ADD R6,R6,#-1 ; 先腾出空间 LDR R0,R6,#0 ; 先取数据 STR R0,R6,#0 ; 再存入数据 ADD R6,R6,#1 ; 再收回空间`

    顺序不能颠倒：`ADD R6,R6,#-1` 放到后面就会写错格子、甚至覆盖别的帧；两条移动指令一条 `#-1`、一条 `#1`，对称且容易检查。 * *作用域与存储期*：被压入的值活到它被弹出为止；一旦弹出，它对应的存储期就结束了（比特还在内存里，但读取它是未定义行为——这正是"返回局部变量地址"错误的根源）。
- **R6 栈指针与 R5 帧指针 (Stack Pointer and Frame Pointer)**：`R6` 指向栈顶（一直在动，像"当前手的位置"），`R5` 指向当前帧的基准（本层内固定不动，像"本层书本的封面标签"）。
  - *底层机制图解*：本课与 MP 使用的完整帧布局（高地址在上、低地址在下）：

    `高地址 +--------------------------+ | caller 的栈帧 | 调用者的帧 +--------------------------+ | parameters | ← R5+4, R5+5, ...（第一个参数固定在 R5+4） +--------------------------+ | return value | ← R5+3 ┐ | return address (R7) | ← R5+2 ├ 这三格是 linkage（连接信息） | previous frame pointer | ← R5+1 ┘ +--------------------------+ | local variable 0 | ← R5+0 ← R5 指向局部变量底部 | local variable 1, ... | ← R5-1, R5-2, ... 低地址 +--------------------------+ ← R6 指向栈顶（最低的活动字）`

    关键顺序（课程真实约定，见 `translate.asm` 的 `FIND_ABS`）：`R5+0`（及 `R5-1`、`R5-2`、...）= 局部变量；`R5+1` = 旧帧指针；`R5+2` = 返回地址；`R5+3` = **返回值**；`R5+4`、`R5+5`、... = **参数**。返回值必须在 `R5+3`，因为它要紧贴在第一个参数 `R5+4` 之下，调用者才能用**一条** `ADD R6,R6,#(nparams+1)` 同时弹掉参数**和**返回值（`538-mt1-review`："Pop parameters and return value (destroy the params)"）。 * *作用域与存储期*：`R5` / `R6` 在子程序入口保存、出口恢复，因此每个活跃的调用都有自己的一对值；配合"每次调用都有自己的帧"，就得到 C 中**局部变量互不干扰**与**递归可行**的保证。
- **为什么两者缺一不可**：R6 在函数体内不断移动，R5 不动——R6 像电梯楼层号（一直在变），R5 像"我家在 5 楼"，用它来算相对位置。
  - *底层机制图解*：

    `时刻 R6 R5 R5-R6 刚建好帧 R5 R5 0 ← 两者相同 压 2 个参数后 R5-2 R5 2 ← R6 变了；参数仍在 R5+4、R5+5 调用返回后 R5 R5 0 又压 1 个临时值 R5-1 R5 1 用 R6 定位参数：LDR R0,R6,#2 的偏移每次都不同（无法使用） 用 R5 定位参数：LDR R0,R5,#4 永远成立`

    x86-64 的调用约定（`rbp` 帧指针 + `rsp` 栈指针）与 LC-3 完全同构；高优化下编译器会"省略帧指针"，代价是调试器无法可靠地还原调用栈。 * *作用域与存储期*：`R5` 让"帧内偏移"成为**编译期常量**（编译器符号表里 `find_abs/num → R5+4` 那一列），`R6` 只负责空间的分配与回收。

- **建立与拆除栈帧 (Prologue / Epilogue)**：调用者压参数，被调用者建帧，返回时按相反顺序拆帧——像进房间先挂外套摆好工具（prologue），离开前收好工具取回外套（epilogue）。
  - *底层机制图解*：

    `① 调用者压参数（右到左）：先压 p2（在更高地址），再压 p1（落在栈顶 = R6，它就是新帧的 R5+4） ② 调用者 JSR 子程序 ；R7 ← 返回地址 ③ 被调用者 prologue（FIND_ABS 的写法：k 个局部变量就压 3+k 格） ADD R6,R6,#-(3+k) ；3 格 linkage + k 个局部变量 STR R5,R6,#1 ；R5+1 ← 旧帧指针（此时 R6+1 正是将来的 R5+1） ADD R5,R6,#0 ；R5 = R6；R5+0 就是第一个局部变量 STR R7,R5,#2 ；R5+2 ← 返回地址 ④ 函数体：局部变量用 R5+0、R5-1、…，参数用 R5+4、R5+5、… ⑤ 被调用者 epilogue STR R0,R5,#3 ；返回值写进 R5+3（紧贴第一个参数之下） LDR R7,R5,#2 ；取回返回地址 LDR R5,R5,#1 ；恢复调用者的帧指针 ADD R6,R6,#(k+2) ；弹出局部变量与 linkage，只把 R5+3 的返回值留在栈顶 RET ；JMP R7 ⑥ 调用者：LDR R0,R6,#0 读返回值，再用一条 ADD R6,R6,#(nparams+1) 弹掉参数与返回值`（`FIND_ABS` 有 1 个局部变量：压 3+1=4 格、epilogue `ADD R6,R6,#3`；`FOO` 有 2 个局部变量：压 5 格、`ADD R6,R6,#4`。 没有局部变量时（`translate.asm` 的 `MAIN`）只压 3 格并令 `R5 = R6-1`，`R5+1`…`R5+3` 的语义完全不变。）
  - *作用域与存储期*：帧的存活期 = 一次调用；`R5` 的恢复使调用者的作用域不受影响，这就是"两个函数里的同名变量互不干扰"的机制来源。
- **用栈做算术 (Arithmetic Using a Stack)**：后缀（postfix）表达式天然对应"遇到数就压栈、遇到运算符就弹两次再压结果"；中缀 `1 + 2 × 3` 有歧义、需要优先级规则，而后缀 `1 2 3 × +` **无歧义**，连括号都不用。
  - *底层机制图解*：把 `(1+2)*(3-4)` 写成后缀 `1 2 + 3 4 - *` 并用 R6 执行：

    `输入记号 动作 栈内容（栈顶在左） R6 1 push 1 [1] base-1 2 push 2 [2, 1] base-2 + pop 2 和 1，push 1+2=3 [3] base-1 3 与 4 各压一次栈 [4, 3, 3] base-3 - pop 4 和 3，push 3-4=-1 [-1, 3] base-2 * pop -1 和 3，push 3×(-1) [-3] base-1 结束 结果在栈顶 结果 = -3`

    实现上只需要"弹两个、算、压一个"的子程序，每个都恰好是 `LDR R1,R6,#1`（更深的操作数）+ `LDR R0,R6,#0`（栈顶）+ `ADD R6,R6,#2` + 计算 + `ADD R6,R6,#-1` + `STR`。 * *作用域与存储期*：栈上每个操作数的存储期就是它参与运算的那一瞬间；这种**数据流驱动**的求值方式也是 JVM 字节码与许多解释器的模型。
- **栈与堆相向增长 (Stack and Heap Grow Toward Each Other)**：两个区域从内存两端出发，中间是自由空间——栈像从天花板往下挂的盘子，堆像从地面往上堆的箱子。
  - *底层机制图解*：

    `高地址 +---------------------+ | 系统空间 / I/O | xFE00 起；栈 stack 从上方往下增长 ↓ (R6/R5) | ↓ | | ↑ | | 堆 heap | 向上增长 ↑ malloc 从低处往高处要空间 | 全局数据 global data| R4 指向这里（本课约定 x4000 起） | 代码 code | 从 x3000 开始 低地址 +---------------------+`

    `malloc` 与函数调用在**同一个地址空间的两端**抢空间，这是"栈溢出"与"内存耗尽"在系统层面的真实图景。 * *作用域与存储期*：栈对应 automatic、堆对应 allocated（`malloc` → `free`）、全局数据对应 static；三种存储期在内存图上就是三段不同的区域。

- **栈溢出与下溢 (Overflow / Underflow)**：LC-3 的栈**没有任何检查**——一摞盘子堆到顶会塌，抽空了还继续抽也会塌，机器不会报警，只会悄悄写坏别人的数据。
  - *底层机制图解*：溢出（压栈过多，R6 撞上堆或全局数据）在 LC-3/嵌入式/内核里造成**静默的数据破坏 (silent data corruption)**，是最难查的一类 bug；在桌面/手机上则由硬件检测越界页、由操作系统让程序崩溃（`segmentation fault`）。下溢（弹出过多，R6 越过栈底往上跑）会读到别的帧的内容。递归深度过大是溢出的最常见原因。
  - *作用域与存储期*：栈的检查是**运行期**问题，与语言层无关——C 不检查，LC-3 不检查，只有操作系统 + 硬件通过"页保护"近似兜底。
- **与 C 的连接 (Connection to C)**：C 的每一次函数调用都是本讲这套动作的翻译结果。写 C 时你只管定义变量、调用函数，编译器负责建帧、算偏移、拆帧。
  - *底层机制图解*：

    `C 语言 LC-3 机制（本讲） 函数参数 调用者压栈；第一个参数固定在 R5+4 局部变量（automatic） R5+0, R5-1, ...（帧内偏移是编译期常量） return 值 写进 R5+3，调用者用 LDR R0,R6,#0 读回 递归 每层调用都有自己的帧（R5/R6 各不一样） 变量的作用域（scope）由符号表（R5+偏移）实现，与运行期无关`

    教学目标 2/3/4/7（作用域与存储、调用约定、数组与指针、递归）都直接建立在这张帧图上。 * *作用域与存储期*：C 的 storage duration 分类（automatic / static / allocated）与本讲的内存图一一对应；而 scope 是编译期概念，它决定编译器把名字翻译成哪个 `R5+偏移`。

代码示例与底层机制分析

示例 1：用栈执行后缀程序 `1 2 + 3 4 - *`

代码 (LC-3 assembly):

```
; 计算 (1+2)*(3-4)，即后缀式 1 2 + 3 4 - *
; 寄存器用途
;   R6 : 栈指针（指向栈顶，栈向低地址增长）
;   R0 : 操作数 / 返回值 / 算术临时
;   R1 : 第二个操作数（更深的那个）与乘法计数器
;   R2 : MULT 的累加器
        .ORIG x3000
        LEA  R6,STK_BASE     ; x3000: R6 = x4000，栈基址（空栈）

        ; ---- 执行 "1" 与 "2"：压栈 ----
        AND  R0,R0,#0        ; x3001
        ADD  R0,R0,#1        ; x3002
        ADD  R6,R6,#-1       ; x3003  压栈的固定两步
        STR  R0,R6,#0        ; x3004
        AND  R0,R0,#0        ; x3005
        ADD  R0,R0,#2        ; x3006
        ADD  R6,R6,#-1       ; x3007
        STR  R0,R6,#0        ; x3008

        ; ---- 执行 "+" ----
        JSR  STACKADD        ; x3009: 弹出 2 与 1，压回 3

        ; ---- 执行 "3" 与 "4" ----
        AND  R0,R0,#0        ; x300A
        ADD  R0,R0,#3        ; x300B
        ADD  R6,R6,#-1       ; x300C
        STR  R0,R6,#0        ; x300D
        AND  R0,R0,#0        ; x300E
        ADD  R0,R0,#4        ; x300F
        ADD  R6,R6,#-1       ; x3010
        STR  R0,R6,#0        ; x3011

        ; ---- 执行 "-" 与 "*" ----
        JSR  STACKSUB        ; x3012: 弹出 4 与 3，压回 -1
        JSR  STACKMUL        ; x3013: 弹出 -1 与 3，压回 -3

        LDR  R0,R6,#0        ; x3014: 结果出栈
        ADD  R6,R6,#1        ; x3015
        HALT                 ; x3016

; STACKADD -- 弹出两个操作数，压回它们的和
;   栈顶是 rhs，其下是 lhs；出口 R0 = lhs + rhs；改变 R0、R1
STACKADD
        LDR  R1,R6,#1        ; x3017: R1 ← lhs（更深的一个）
        LDR  R0,R6,#0        ; x3018: R0 ← rhs（栈顶）
        ADD  R6,R6,#2        ; x3019: 一次弹出两个
        ADD  R0,R0,R1        ; x301A: R0 ← lhs + rhs
        ADD  R6,R6,#-1       ; x301B: 压回结果
        STR  R0,R6,#0        ; x301C
        RET                  ; x301D

; STACKSUB -- 弹出两个操作数，压回 lhs - rhs
STACKSUB
```

```
        LDR  R1,R6,#1        ; x301E: R1 ← lhs
        LDR  R0,R6,#0        ; x301F: R0 ← rhs
        ADD  R6,R6,#2        ; x3020: 弹出两个
        NOT  R0,R0           ; x3021: 取 rhs 的补码
        ADD  R0,R0,#1        ; x3022: R0 ← -rhs
        ADD  R0,R1,R0        ; x3023: R0 ← lhs - rhs
        ADD  R6,R6,#-1       ; x3024: 压回结果
        STR  R0,R6,#0        ; x3025
        RET                  ; x3026

; STACKMUL -- 弹出两个操作数，压回 lhs * rhs
;   它要调用 MULT，所以必须先把 R7 保存到栈上（第 3 讲的 R7 纪律）
STACKMUL
        ADD  R6,R6,#-1       ; x3027: 在栈上腾一格保存 R7
        STR  R7,R6,#0        ; x3028: 现在栈布局是 [R7, rhs, lhs]
        LDR  R1,R6,#2        ; x3029: R1 ← lhs（跳过 R7 与 rhs）
        LDR  R0,R6,#1        ; x302A: R0 ← rhs
        JSR  MULT            ; x302B: R0 ← lhs * rhs
        LDR  R7,R6,#0        ; x302C: 恢复返回地址
        ADD  R6,R6,#1        ; x302D: 丢掉保存的 R7
        ADD  R6,R6,#2        ; x302E: 弹出两个操作数
        ADD  R6,R6,#-1       ; x302F: 压回乘积
        STR  R0,R6,#0        ; x3030
        RET                  ; x3031

; MULT -- 用重复加法计算 R0 * R1（假设 R1 >= 0），结果放 R0；改变 R1、R2
MULT    AND  R2,R2,#0        ; x3032: R2 ← 0
        ADD  R1,R1,#0        ; x3033: 测试乘数
        BRz  MULT_DONE       ; x3034
MULT_L  ADD  R2,R2,R0        ; x3035: 累加
        ADD  R1,R1,#-1       ; x3036
        BRnp MULT_L          ; x3037
MULT_DONE
        ADD  R0,R2,#0        ; x3038
        RET                  ; x3039

        .BLKW #64            ; x303A..x3079: 给栈留出空间
STK_BASE                     ; x307A: 空栈时 R6 指向这里
        .END
```

> ⚙️ **机器验证**：已用 `ece220_data/lc3sim.py` 汇编并运行，**64 条指令**后 `HALT`， 结束时 **`R0 = xFFFD`**（= −3），与下面手算的 `(1+2)*(3-4) = -3` 一致； `R6` 也回到了初始栈基址，说明每次压栈/弹栈严格配平。

【代码做什么？】

1. `LEA R6,STK_BASE` 把栈指针指到栈底；此时"栈是空的"（没有任何字低于 R6 属于栈）。
2. 每个数字记号（1、2、3、4）都用固定两条指令压栈：`ADD R6,R6,#-1` 然后 `STR R0,R6,#0`。
3. 每遇到运算符就 `JSR` 对应子程序：取两个操作数、一次弹出、算出结果、再压回；`STACKMUL` 在调用 `MULT` 前把 R7 压栈保存，返回前恢复。

4. 最后 `LDR R0,R6,#0` 取回栈顶结果并弹出：`R0 = −3`（= `xFFFD`），即 `(1+2)*(3-4)`。

【底层机制透视】

- **栈的"弹出"只是移动指针**：`ADD R6,R6,#2` 之后两个操作数仍在内存里，但已经**不属于栈**；再压一个新值就会覆盖它们。"一次弹两个"之所以安全，是因为栈是连续内存（等价于两次 `ADD R6,R6,#1`）；而任何"部分弹出"都会让栈失去平衡。
- **`MULT` 的循环次数取决于操作数的值**：`R1` 是乘数（此处 `lhs = 3`），每次减一，内层 `ADD R2,R2,R0` 把 `rhs` 累加进去。负的 `rhs` 能工作（累加负数），但负的 `R1` 会死循环——这就是"接口假设必须写进 CIS"的例子。
- **R7 保存在栈上**（比第 3 讲的固定 `.BLKW` 槽更安全），因此 `STACKMUL` 即使被再次进入也不会互相覆盖。

【内存布局图解】（STK_BASE = x4000，逐步执行）

```
初始:     R6 = x4000（空栈）
push 1/2: x3FFF = 1、x3FFE = 2      R6 = x3FFE    栈(顶在左): [2, 1]
STACKADD: 读 M[R6+1] = 1 (lhs)、M[R6+0] = 2 (rhs) → R6 = x4000 → 压回 3 于 x3FFF
push 3/4: x3FFE = 3、x3FFD = 4      R6 = x3FFD    栈: [4, 3, 3]
STACKSUB: 读 3 (lhs) 与 4 (rhs) → 3-4 = -1 → R6 = x3FFE, M[x3FFE] = -1    栈: [-1, 3]
STACKMUL: 压 R7 到 x3FFD → 读 3 (lhs) 与 -1 (rhs) → JSR MULT (累加 3 次 -1) → R0 = -3
          → 弹两格、压回 → R6 = x3FFF, M[x3FFF] = -3                        栈: [-3]
结束:     LDR R0,R6,#0 → R0 = xFFFD = -3; R6 = x4000（栈恢复为空）

内存快照: x3FFF = xFFFD(-3) 仍在栈上; x3FFE = xFFFD(-1)、x3FFD = R7、x3FFC = 3 都已弹出（属"旧数据"
```

【与汇编的对应】：这段程序与它的"高级语言版本"（示例 3）逐句对应——C 的 `push(x)` / `pop()` 就是那两条指令，`sp` 就是 R6，`stack[]` 就是 x4000 以下的 LC-3 内存；能手工完成的"栈机器求值"是理解所有表达式求值的通用语言。

示例 2：一个带完整栈帧的子程序 `SUM2`

代码 (LC-3 assembly):

```
; SUM2 -- 把栈上的两个参数相加，返回值留在栈顶
; 调用接口 (CIS)
;   输入  : 两个参数由调用者压栈（右到左：先压 p2，再压 p1，所以 p1 在栈顶 = R5+4）
;   输出  : 返回值在 M[R6]（即帧内的 R5+3）；调用者用一条 ADD 同时弹掉参数与返回值
;   改变  : R0、R1（caller-saved）
;   副作用: 无（只读写自己的帧）
; 帧布局（R5 相对，与 translate.asm 的 FIND_ABS 完全一致）：
;   R5+5 = p2   R5+4 = p1   R5+3 = 返回值   R5+2 = 返回地址
;   R5+1 = 旧帧指针        R5+0 = 局部变量
        .ORIG x3000
        LEA  R6,STK_BASE     ; x3000: R6 = x305A
        LD   R0,VAL_B        ; x3001: p2 先压（右到左 → p2 在更高地址）
        ADD  R6,R6,#-1       ; x3002
        STR  R0,R6,#0        ; x3003
        LD   R0,VAL_A        ; x3004: p1 后压 → 落在栈顶，成为新帧的 R5+4
        ADD  R6,R6,#-1       ; x3005
        STR  R0,R6,#0        ; x3006
        JSR  SUM2            ; x3007: R7 ← x3008
        LDR  R0,R6,#0        ; x3008: 读返回值（= 35）——R6 正指向它
        ADD  R6,R6,#3        ; x3009: 一条 ADD 弹掉 2 个参数 + 1 个返回值
        HALT                 ; x300A

SUM2                         ; x300B: ---- 被调用者 ----
        ADD  R6,R6,#-4       ; x300B: prologue：3 格 linkage + 1 个局部变量
        STR  R5,R6,#1        ; x300C: R5+1 ← 调用者的帧指针（必须在改 R5 之前）
        ADD  R5,R6,#0        ; x300D: R5 = R6，R5+0 就是第一个局部变量
        STR  R7,R5,#2        ; x300E: R5+2 ← 返回地址

        LDR  R1,R5,#4        ; x300F: R1 ← p1（第一个参数固定在 R5+4）
        LDR  R0,R5,#5        ; x3010: R0 ← p2
        ADD  R1,R1,R0        ; x3011: R1 ← p1 + p2
        STR  R1,R5,#0        ; x3012: 局部变量 ← 和

        STR  R1,R5,#3        ; x3013: epilogue：返回值写进 R5+3
        LDR  R7,R5,#2        ; x3014: 取回返回地址
        LDR  R5,R5,#1        ; x3015: 恢复调用者的帧指针
        ADD  R6,R6,#3        ; x3016: 弹出局部变量与 linkage（返回值除外）→ R6 = R5+3
        RET                  ; x3017

VAL_A   .FILL #42            ; x3018
VAL_B   .FILL #-7            ; x3019
        .BLKW #64            ; x301A..x3059: 栈空间
STK_BASE                     ; x305A: 空栈位置
        .END
```

> ⚙️ **机器验证**：已用 `ece220_data/lc3sim.py` 汇编并运行，**24 条指令**后 `HALT`， 结束时 **`R0 = x0023 = 35`**，与下面手算表给出的 `42 + (-7) = 35` 完全一致。

【代码做什么？】

1. 调用者把参数**从右向左**压栈：先 `p2 = -7`（落在更高地址），再 `p1 = 42`（落在栈顶 = 将来的 `R5+4`）；`JSR` 把返回地址放进 R7。
2. prologue 压 3+1=4 格：把旧帧指针存到 `R6+1`（`R5 = R6` 之后它就是 `R5+1`），令 `R5 = R6`，再把返回地址存到 `R5+2`。
3. 函数体用 `LDR R1,R5,#4`、`LDR R0,R5,#5` 从**固定偏移**取参数，相加后写进局部变量 `R5+0`。
4. epilogue 把结果写进 `R5+3`、恢复 R7 与 R5、用 `ADD R6,R6,#3` 让 R6 落在返回值上；调用者读结果（35）后用**一条** `ADD R6,R6,#3` 弹掉 2 个参数与返回值。

【底层机制透视】

- **参数在 R5+4 而不是 R5+0**：因为被调用者的 linkage 与局部变量都压在参数的**下面**（更低地址）；第一个参数是最后一个被压入的，所以位置固定。
- **`STR R5,R6,#1` 必须在 `ADD R5,R6,#0` 之前**：一旦 R5 被覆盖，调用者的帧指针就永远丢失。此时还没设 R5，所以用 `R6+1` 作基址——而 `R5 = R6` 之后 `R6+1` 正是 `R5+1`。
- **返回值的位置决定调用者如何弹栈**：`R5+3` 紧贴在第一个参数 `R5+4` 之下，所以 `RET` 之后 R6 指向返回值、参数就在它上面，调用者于是能用**一条** `ADD R6,R6,#(nparams+1)` 同时弹掉两者。若返回值放在别处（例如 `R5+1`），调用者就得先弹掉连接残留、再单独弹参数，一条指令完不成——这正是课程规定"返回值必须在 R5+3"的原因。
- **epilogue 的两条 `LDR` 不能交换**：`LDR R7,R5,#2` 必须用旧 R5 作基址；先恢复 R5 再去读 `R5+2` 只会读到调用者帧里的垃圾值。

【内存布局图解】（prologue 完成后，R5 = x3054）

```
        地址      内容                    R5 相对       说明
        x3059     xFFF9 (-7)              R5+5          参数 p2（先压 → 高地址）
        x3058     x002A (42)              R5+4          参数 p1（后压 → 栈顶方向）
        x3057     x0023 (35)              R5+3          返回值（紧贴第一个参数之下）
        x3056     x3008                   R5+2          返回地址（JSR 写入的 R7）
        x3055     x0000                   R5+1          旧帧指针（调用者的 R5）
        x3054     x0023 (35)              R5+0          局部变量（和）   ← R5 = R6 = x3054
        x305A                             STK_BASE      调用前的 R6（空栈位置；x3053 以下是 SUM2 的

函数体执行时 R5 = R6 = x3054；若又压了两个临时值，R6 = x3052（R5−R6 = 2），
但 LDR R1,R5,#4 仍正确取到 p1 —— 这就是"为什么要留着 R5"。
epilogue 之后 R6 = R5+3 = x3057（指向返回值）、R5 = x0000；
调用者 ADD R6,R6,#3 后 R6 = x305A，与调用前完全一致（栈平衡）。
```

【与汇编的对应】（逐条手算执行结果）

时刻	R0	R1	R5	R6	内存变化
`LD R0,VAL_B` 后	#−7	—	x0000	x305A	—
压 p2 后	#−7	—	x0000	x3059	M[x3059] = −7
`LD R0,VAL_A` 后 / 压 p1 后	#42	—	x0000	x3058	M[x3058] = 42（p1 成为 R5+4）

时刻	R0	R1	R5	R6	内存变化
`JSR` 后（prologue 前）	#42	—	x0000	x3058	R7 = x3008
`ADD R6,R6,#-4` 后	#42	—	x0000	x3054	—
`STR R5,R6,#1` 后（R5 还是旧值）	#42	—	x0000	x3054	M[x3055] = 0（旧帧指针 → R5+1）
`ADD R5,R6,#0` 后	#42	—	**x3054**	x3054	R5 = R6
`STR R7,R5,#2` 后	#42	—	x3054	x3054	M[x3056] = x3008（返回地址）
求和并存储后	#−7	#35	x3054	x3054	M[x3054]=35（局部变量）
epilogue 后	#−7	#35	**x0000**	**x3057**	M[x3057]=35（返回值 → R5+3）
`RET` 后 / `ADD R6,R6,#3` 后	**#35**	#35	x0000	**x305A**	R6 先指向返回值，读回后用一条 ADD 弹掉参数与返回值，栈完全恢复

即 `R0 = 35`，正是 `42 + (-7)`。

示例 3：C 版本的后缀求值器（同一算法的"高级语言形态"）

代码 (C):

```
/* ECE 220 -- Lecture 4 example: arithmetic on a stack.
 *
 * Running the postfix program "1 2 + 3 4 - *", which is (1 + 2) * (3 - 4).
 * The array below plays the role of LC-3 memory, sp plays the role of R6,
 * and push/pop are exactly ADD R6,R6,#-1 + STR and LDR + ADD R6,R6,#1.
 */
#include <stdio.h>
#include <stdint.h>

#define STK_SIZE 16

static int32_t stack[STK_SIZE];   /* LC-3 memory used as the stack */
static int32_t sp = STK_SIZE;     /* R6: index of the top element  */

static void push(int32_t value)
{
    sp = sp - 1;            /* ADD R6,R6,#-1 : make space first */
    stack[sp] = value;      /* STR R0,R6,#0  : then store       */
}

static int32_t pop(void)
{
    int32_t value = stack[sp];   /* LDR R0,R6,#0 */
    sp = sp + 1;                 /* ADD R6,R6,#1 : remove space */
    return value;
}

static void show(const char *label)
{
    int32_t i;

    printf("%-12s stack (top first):", label);
    if (sp == STK_SIZE) {
        printf(" <empty>");
    }
    for (i = sp; i < STK_SIZE; i = i + 1) {
        printf(" %d", (int)stack[i]);
    }
    printf("   [R6 offset = %d]\n", (int)(sp - STK_SIZE));
}

int main(void)
{
    const char *program[] = { "1", "2", "+", "3", "4", "-", "*" };
    int32_t i;

    show("start");
    for (i = 0; i < 7; i = i + 1) {
        const char *token = program[i];

        if (token[0] >= '0' && token[0] <= '9') {
            push((int32_t)(token[0] - '0'));
            printf("push %s\n", token);
```

```
        } else {
            int32_t rhs = pop();
            int32_t lhs = pop();
            int32_t result = 0;

            if (token[0] == '+') {
                result = lhs + rhs;
            } else if (token[0] == '-') {
                result = lhs - rhs;
            } else {
                result = lhs * rhs;
            }
            push(result);
            printf("%s: %d %c %d = %d\n", token, (int)lhs, token[0], (int)rhs,
                   (int)result);
        }
        show("  after");
    }

    printf("result = %d\n", (int)pop());
    return 0;
}
```

编译与运行：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l04_rpn.c -o ece220_l04_rpn
$ ./ece220_l04_rpn
start        stack (top first): <empty>   [R6 offset = 0]
push 1
  after      stack (top first): 1   [R6 offset = -1]
push 2
  after      stack (top first): 2 1   [R6 offset = -2]
+: 1 + 2 = 3
  after      stack (top first): 3   [R6 offset = -1]
push 3
  after      stack (top first): 3 3   [R6 offset = -2]
push 4
  after      stack (top first): 4 3 3   [R6 offset = -3]
-: 3 - 4 = -1
  after      stack (top first): -1 3   [R6 offset = -2]
*: 3 * -1 = -3
  after      stack (top first): -3   [R6 offset = -1]
result = -3
```

【代码做什么？】

1. `sp` 从 `STK_SIZE` 出发表示"空栈"；`push` 先减 `sp` 再写入，`pop` 先读出再增 `sp`——与 `ADD R6,R6,#-1` + `STR` / `LDR` + `ADD R6,R6,#1` 一一对应。
2. 主循环遍历记号数组：数字解析为整数后压栈；运算符先弹 `rhs`、再弹 `lhs`，按 `lhs op rhs` 计算后压回。

3. `show` 每步打印栈内容与"R6 偏移"，便于与手算的 LC-3 栈图对照；最后输出 `result = -3`，与示例 1 的 `R0 = xFFFD` 一致。

【底层机制透视】

- `sp` 用的是**数组下标**而非地址，`sp - STK_SIZE` 就是"R6 相对栈底移动了多少个字"。
- `stack[sp]` 的"先减指针、再写入"顺序保证了栈向**低地址**生长；反过来写就与 LC-3 语义相反。
- `pop()` 不会擦除 `stack[sp]`——与 LC-3 一样，"弹出"只是移动指针，旧值仍在内存里。
- 运算符分派用 `if/else` 链实现，对应 LC-3 的比较与分支；真实编译器会把它变成跳转表 (jump table)。

【内存布局图解】

```
全局数据区（static storage duration）
+------------------------------+  0x4040a0 附近
| stack[0] ... stack[15]       |  32 字节；sp 从 16 开始，向索引 0 方向"生长"
| sp = 0                       |  执行到最后：栈里只剩 1 个元素
+------------------------------+

栈内容（打印中"-"一步之后）：
        +---+---+---+---+---+
        | 4 | 3 | 3 | ? | ? |  ...      打印时从 sp 到 15 依次输出 → "4 3 3"
        +---+---+---+---+---+
          ↑
          sp（栈顶）；索引 13、14 的旧值已不在"栈上"
```

【与汇编的对应】

```
; C: sp = sp - 1;  stack[sp] = value;
        ADD  R6,R6,#-1          ; sp--
        STR  R0,R6,#0           ; stack[sp] = value
; C: value = stack[sp];  sp = sp + 1;
        LDR  R0,R6,#0           ; value = stack[sp]
        ADD  R6,R6,#1           ; sp++
; C: rhs = pop(); lhs = pop(); result = lhs - rhs; push(result);
        JSR  POP_R0             ; rhs 在 R0
        ADD  R3,R0,#0           ; 暂存 rhs（R3 是 caller-saved）
        JSR  POP_R0             ; lhs 在 R0
        NOT  R3,R3              ; -rhs
        ADD  R3,R3,#1
        ADD  R0,R0,R3           ; lhs - rhs
        JSR  PUSH_R0
```

一次调用一层帧：递归的前奏

把 `SUM2` 的 prologue/epilogue 放在一个**自我调用**的子程序里，就得到递归：每层调用都在更低地址压出**自己的一份完整帧**（自己的 `R5`、`R7`、局部变量），同一段代码于是可以有任意多份互不干扰的数据。在真实机器上可直接观察（本机实测每层相差 0x30 = 48 字节）：

```
$ gcc -g -std=c99 -Wall -Werror ece220_l04_frames.c -o ece220_l04_frames && ./ece220_l04_fra
main     : &automatic = 0x7ffe86cb5c68
frame 3: &local = 0x7ffe86cb5c4c   local = 30       (完整程序见 ece220_l04_frames.c)
frame 2: &local = 0x7ffe86cb5c1c   local = 20      地址随 ASLR 变化，但逐层下降
frame 1: &local = 0x7ffe86cb5bec   local = 10
frame 0: &local = 0x7ffe86cb5bbc   local = 0
sum of the four locals = 60
```

在 LC-3 上重复同样的实验，就是观察 `DEPTH_REPORT` 每次 prologue 之后的 `R5`：每递归一层就减小固定的字节数， 直到 `R6` 撞上代码或全局数据——那就是前面说的"栈溢出"。

常见错误与调试技巧

- **R6 失去平衡（弹栈数量算错）**：调用者忘记执行 `ADD R6,R6,#(nparams+1)`，或弹多了把调用者的数据当参数弹掉；现象是程序"跑一会儿就错"，多个函数间互相踩数据。**调试**：在每个调用点前后比较 R6；`gdb` 中用 `p $rsp`（对应 R6）核对；`lc3sim` 里 `print R6` 并用 `dump` 观察栈区。
- **prologue 顺序写错**：先 `ADD R5,R6,#0` 再 `STR R5,R6,#1`，把**新**帧指针当成旧帧指针存进 `R5+1`；或者把偏移写成 `#3`（那是返回值的位置）。现象是调用者返回后局部变量与 R5 全部错乱。**调试**：逐条单步 prologue，检查 `M[R5+1]` 是否等于调用前的 R5，`M[R5+2]` 是否等于 `JSR` 的下一条指令地址。
- **用 R6 而不是 R5 访问参数**：写成 `LDR R0,R6,#2`，一旦函数体内压过临时值，偏移就失效，现象是"参数偶尔读错"。**调试**：检查所有访问参数/局部变量的指令是否都以 R5 为基址；在 `gdb` 中打印 `$rbp` / `$rsp` 的差，确认帧内偏移恒定。
- **返回局部变量的地址**：`int32_t *f(void) { int32_t x = 1; return &x; }`——帧一拆，`x` 就没有存储期了，现象是调用后读到的值随机变化。**调试**：`-Wall -Werror` 会给出 `-Wreturn-local-addr`；`valgrind --tool=memcheck ./prog` 或 `gcc -fsanitize=address -g` 能直接指出"使用了已释放的栈内存"。
- **把"弹出"误解为"擦除"**：弹出后仍去读 `M[R6]` 之上的旧值，误以为那是"栈上的数据"，于是读到残留值。**调试**：用 `x/8xw $rsp`（gdb）或 `dump`（lc3sim）观察栈区，对照"R6 以上不属于栈"逐字检查。
- **栈溢出**：无限递归或每层帧过大（例如把大数组声明为局部变量）。LC-3 上是静默数据破坏，Linux 上是 `Segmentation fault`。**调试**：`gdb` 中 `bt 50` 看递归深度；`ulimit -s` 查栈上限；把递归改成用显式栈的迭代（如同示例 1）。

关键要点

- 栈提供 **LIFO** 语义；在 LC-3 上压栈是 `ADD R6,R6,#-1` + `STR R0,R6,#0`，弹栈是 `LDR R0,R6,#0` + `ADD R6,R6,#1`，**顺序不可颠倒**，而且"弹出"只移动指针、不擦除数据。
- `R6` 是栈指针（一直移动），`R5` 是帧指针（本帧内固定）；没有 R5，函数体一旦压入临时值，就无法再用固定偏移访问参数与局部变量。C 的自动变量、递归、作用域隔离全部依赖这一点。
- 栈帧布局（课程真实约定）：`R5+0` / `R5-1` /… 局部变量、`R5+1` 旧帧指针、`R5+2` 返回地址、`R5+3` **返回值**、`R5+4` / `R5+5` /… 参数；返回值必须在 `R5+3`，这样调用者才能用**一条** `ADD R6,R6,#(nparams+1)` 同时弹掉参数与返回值。

- 用栈求值是表达式求值的通用模型（遇到数压栈、遇到运算符就"弹两次、算、压一次"）：`1 2 + 3 4 - *` 最终得到 `-3`，与 C 版本一致；而栈与堆从地址空间两端**相向增长**，相遇即内存耗尽，栈的溢出/下溢在 LC-3 上没有任何检查，只会造成静默的数据破坏。

思考题（带答案）

问题 1：为什么 LC-3 的帧布局把**返回值**放在 `R5+3`（紧贴第一个参数 `R5+4` 之下），而不是像局部变量那样放在 R5 以下？如果 C 规定实参从左向右压栈，`R5+4` 还会是第一个参数吗？

答案：因为调用者返回后要用**一条** `ADD R6,R6,#(nparams+1)` 同时弹掉参数**和**返回值；由于参数在 `R5+4` 及以上连续排列，只有把返回值放在它们正下方的 `R5+3`，这一条指令才能一次清掉两者（`538-mt1-review` 的原话是 "Pop parameters and return value"）。那些"连接信息"（旧帧指针 `R5+1`、返回地址 `R5+2`）由被调用者的 epilogue 自己弹掉，调用者不必关心。如果改成**从左向右**压栈，最后一个参数会落在 `R5+4`、第一个参数跑到更远处；对固定参数个数的函数两者都可行，但**变参函数**（如 `printf`）就无法知道第一个参数在哪里，所以 C 选择右到左压栈。

问题 2：某个 LC-3 子程序的 epilogue 写成下面这样，哪里有问题？会造成什么后果？

```
        STR  R0,R5,#3        ; 返回值 → R5+3
        LDR  R5,R5,#1        ; 先恢复调用者的帧指针
        LDR  R7,R5,#2        ; 再想用 R5+2 取回返回地址
        ADD  R6,R6,#3
        RET
```

答案：**`LDR R5,R5,#1` 与 `LDR R7,R5,#2` 的顺序反了**：`R5+1` 是旧帧指针、`R5+2` 才是返回地址，一旦先用 `LDR R5,R5,#1` 恢复 R5，`R5+2` 就变成**调用者帧**里的某个字，`LDR R7,R5,#2` 取到垃圾值，`RET` 跳向随机位置（`ADD R6,R6,#3` 也会因为基准不清而算错栈顶）。正确顺序是先取回返回地址、再恢复帧指针、最后调 R6——即 `STR R0,R5,#3` / `LDR R7,R5,#2` / `LDR R5,R5,#1` / `ADD R6,R6,#3` / `RET`。

问题 3：下面的 C 函数返回了局部变量的地址。请解释为什么它危险，并给出两种修法。

```
int32_t *make_value(int32_t v)
{
    int32_t local = v * 2;
    return &local;          /* 危险 */
}
```

答案：`local` 具有 automatic storage duration，家在 `make_value` 的栈帧里；函数一返回帧就被拆除（LC-3 上就是 `R5` / `R6` 被恢复），那块空间随时会被下一次调用覆盖，因此返回的指针指向**已经结束存储期的对象**，读取它是未定义行为。两种修法：(1) 让调用者提供存储 `void make_value(int32_t v, int32_t *out) { *out = v * 2; }`，或直接返回值 `int32_t make_value(int32_t v) { return v * 2; }`；(2) 改用 allocated storage：`int32_t *p = malloc(sizeof(int32_t)); *p = v * 2; return p;`（调用者负责 `free`）。编译器在 `-Wall` 下会给出 `-Wreturn-local-addr` 警告——让工具替你发现存储期错误。

Lecture 5: C 语言入门：数据类型、运算符、作用域与存储期 (Introduction to C: Data Types, Operators, Scope and Storage)

概述

本讲回答一个自底向上 (bottom-up) 的核心问题：**为什么高级语言需要"类型"，以及类型信息如何决定机器码的生成**。 我们引入 C 的数据类型表（基本类型与派生类型）、字面量 (literal) 与后缀 (suffix)、六大类运算符及其优先级与短路求值 (short-circuit evaluation)、 隐式转换 (implicit conversion) 与整型提升 (integer promotion)，以及作用域 (scope) 与存储期 (storage duration) 这两把"变量住在哪里、活多久"的尺子。 这些概念直接承接 ECE 120 的 LC-3 汇编与内存映射 (memory map)：类型是编译器决定该生成 `ADD`、`MUL` 还是浮点库调用 (library call) 的唯一依据， 作用域与存储期则决定一个变量落在全局数据区 (global data)、栈 (stack) 还是堆 (heap)——这正是后续函数、指针、动态内存分配三讲的共同地基。

核心概念与底层机制图解

- **C 是"高级汇编" (C as a high-level assembly language)**：C 与典型 ISA 之间有一层**透明的映射 (transparent mapping)**，人容易理解、编译器也容易生成。
  - *直观解释*：写汇编时你自己决定"哪个数据放哪个寄存器、哪块内存"；写 C 时你只给变量起个符号名 (symbolic name) 并声明类型，**由编译器决定存放位置**——像把"手工点名分座位"换成"报人数让系统自动排座"。
  - *底层机制图解*：编译器分前端 (front end，语言相关) 与后端 (back end，ISA 相关) 两半，中间的接口叫中间表示 (intermediate representation, IR)。 这样 10 种语言 × 10 种 ISA 只需写 10 + 10 = 20 个模块，而不是 100 个编译器。 `C 源文件 + 头文件 │ C 预处理器 (preprocessor)：展开 #include / #define ▼ 预处理后的源码 ──前端──▶ IR ──后端──▶ LC-3 / x86-64 汇编 │ 汇编器 (assembler) ▼ 目标文件 (.o) │ 链接器 (linker) ▼ 可执行文件 + 动态链接库` `gcc` 默认把预处理器、汇编器、链接器一路跑完；只想看中间产物时用 `-E`（预处理）、`-S`（汇编）、`-c`（目标文件）。
  - *作用域与存储期*：这一层映射是"编译期"概念——变量的位置在编译时就被**静态决定**（写成 `R5+0`、`R4+2` 这样的偏移）， 运行期改变的只有这些位置里的**位 (bits)**，而不是位置本身。例外是后面要学的动态分配 (dynamic allocation)，它的位置在运行期才确定。
- **数据类型 (Data Type)**：类型告诉编译器三件事——需要**多少位 (size)**、这些位用哪种**编码 (encoding)** 解释、以及该生成哪条指令去运算。
  - *直观解释*：同一串 32 位 `0xFFFFFF88`，当 2 的补码 (2's complement) 是 −120，当无符号数 (unsigned) 是 4 294 967 176；类型就是贴在内存上的标签，决定这串位被当成什么。
  - *底层机制图解*：ECE 220 官方 C 编程参考给出的基本类型表（与 `res_c_prog_ref` 一致）：

type name	size	encoding	precision	min value	max value
`bool`	1 bit **	0:false 1:true	exact integer	0	1
`char`	8 bits	signed integer ***	exact integer	−128	127

type name	size	encoding	precision	min value	max value
`unsigned char`	8 bits	unsigned integer	exact integer	0	255
`short`	16 bits	signed integer	exact integer	−32768	32767
`unsigned short`	16 bits	unsigned integer	exact integer	0	65535
`int`	32 bits *	signed integer	exact integer	−2.1 billion *	2.1 billion *
`unsigned int`	32 bits *	unsigned integer	exact integer	0	4.3 billion *
`long`	64 bits *	signed integer	exact integer	-9.2×10^{18} *	9.2×10^{18} *
`unsigned long`	64 bits *	unsigned integer	exact integer	0	1.8×10^{19} *
`float`	32 bits	floating point	7 decimal digits	-3.4×10^{38}	3.4×10^{38}
`double`	64 bits	floating point	16 decimal digits	-1.8×10^{308}	1.8×10^{308}

* `int` 与 `long` 的尺寸/范围假定运行在 EWS 实验机的 64 位处理器上；在 32 位、16 位平台（例如 LC-3）上可能更小。 ** `bool` 只需 1 位，但编译器会把它补齐 (pad) 到 1 字节（或机器的可寻址单位 addressability）。 *** `char` 更常见的用途是保存 ASCII 字符，此时可以把整数值看成它对应的 ASCII 字符。

LC-3 是 16 位可寻址机器，一个字 (word) = 16 位，所以课程里的 C 代码常用 `<stdint.h>` 提供的**与 ISA 无关的类型 (ISA-independent integer types)**：`int8_t / uint8_t`（8 位）、`int16_t / uint16_t`（16 位）、`int32_t / uint32_t`（32 位）、`int64_t / uint64_t`（64 位）。除了 `main` 与部分库函数调用，ECE 220 一律建议用这些名字，因为它们的位数在任何平台上都一样。 * *作用域与存储期*：类型本身没有作用域与存储期，但**类型决定了变量名在机器码里如何被翻译成偏移**：`int32_t` 在 LC-3 上占 2 个内存字，`int16_t` 占 1 个字，于是 `symbol_table` 中相邻变量的偏移相差 1 或 2。 * **派生类型 (Derived Type)**：由基本类型构造出来的类型——指针 (pointer)、数组 (array)、结构体 (struct)。 * *直观解释*：指针像门牌号（内容是一个地址），数组像一排等大紧挨的储物柜（从 0 号开始），结构体像把几件行李捆成一箱（成员依次摆放，可能要垫泡沫）。 * *底层机制图解*：官方参考给出的派生类型表：

type	operator	example	size	description
pointer to Z	`*`	`int * x;`	64 bits (8 bytes) §	存放某个 Z 类型对象的地址
array of n Zs	`[]`	`int x[4];`	n × sizeof(Z) §§§	连续分配的 n 个 Z 类型对象
structure	`struct`	`struct { int x, y; } vector;`	各成员大小之和 §§	依次分配的成员对象集合

§ 假定 64 位处理器；§§ 编译器可能插入填充字节 (padding)，使结构体比预期更大；§§§ **例外**：数组作为函数参数时会被立刻转换成指向首元素的指针，见下。

`int32_t arr[4]; /* 4 个 32 位元素，共 16 字节 */ 地址(假设) 0x1000 0x1004 0x1008 0x100C +--------+--------+--------+--------+ arr: | 10 | 20 | 30 | 40 | +--------+--------+--------+--------+ ^ arr 在表达式里退化为 &arr[0]；int32_t *p = arr; 后 p 里就是 0x1000`

数组参数退化是官方参考中明确写出的例外：`void foo(int x[4]);` 会被编译器立刻转换成 `void foo(int * x);`。 所以在被调用者里 `sizeof(x)` 得到的是指针大小（ECE 220 实验机上是 8 字节），而**不是**数组大小——数组长度必须另开一个参数传进去。 * *作用域与存储期*：派生类型变量的存储期由声明位置决定。函数内 `int32_t arr[4];` 是 automatic，随栈帧建立与销毁； 文件外层的 `static int32_t arr[4];` 是 static，落在全局数据区，随程序存在。 * **字面量 (Literal) 与后缀 (Suffix)**：字面量是直接写在代码里的常量；后缀用来**强制指定**这个常量的类型。 * *直观解释*：后缀像单位标签：写 `42` 是"整数 42"，写 `42.0f` 是"单精度浮点 42"，写 `42ul` 是"无符号长整数 42"。这在你想让表达式走浮点运算时非常关键。 * *底层机制图解*：官方参考的后缀表：`ul` → `unsigned long`，`u` → `unsigned int`，`l` → `long`，无后缀 → `int`，`.f` → `float`，`.` → `double`。字符字面量 `'c'` 的类型是 `int`（值是 ASCII 码）；字符串字面量 `"hello"` 的类型是 `char *`，并且**隐式地在静态内存区分配并初始化一个以 NUL 结尾的字符数组**。 允许的转义序列包括 `\0`(NUL)、`\n`(换行)、以及 `\\`（一个反斜杠）。字面量本身分配在临时空间（例如寄存器），但它赋值给的变量不一定。 **注意**：后缀不会改变被赋值变量的类型——变量类型永远由声明决定。 * *作用域与存储期*：字面量没有名字也没有作用域；字符串字面量所在的字符数组是 static storage duration，程序运行期间一直存在。 * **运算符 (Operator)**：C 提供算术、关系、逻辑、位运算、赋值、自增自减、条件、逗号等运算符，用来把变量与字面量组成表达式 (expression)。 * *直观解释*：运算符就是"计算的动作"，麻烦之处是**同一符号在不同上下文含义不同**——`&` 在位运算里是"按位与"，在单目位置上是"取地址"。 * *底层机制图解*：ECE 220 关注的六类运算符及其机器码对应：

类别	运算符	结果类型	与 LC-3 的对应
算术	`+ - * / %`	操作数"较大"的那个类型	`ADD`；`*` 需要循环加或库调用；`/`、`%` 需要除法子程序
位运算	`& | ~ ^ << >>`	整数类型	`AND`、`NOT`+`ADD #1`、`XOR`、移位循环
关系	`< <= == != >= >`	恒为 `int` 的 0 或 1	条件码 (condition code) `N/Z/P` + `BRnzp`
逻辑	`&& || !`	恒为 `int` 的 0 或 1	短路求值 → 多段 `BR` 跳转
赋值	`= += -= *= /= %= &= |= ^= <<= >>=`	右值 (right-hand side) 的类型	`STR`：把结果写回左值的地址
其它	`++ -- ?: ,`	见下	自增=读+改+写回；`?:` = `BR` + 两个赋值块

必须记住的语义细节： 1. **整数除法向 0 取整**：`11 / 3` 得 3，`-11 / 3` 得 −3；`%` 定义为满足 `(A / B) * B + (A % B) == A`，所以 `-11 % 3` 得 **−2**（`%` 的结果不一定非负）； 又因为整数除法丢精度，`(100 / 8) * 8` 得 **96**，不是 100。 2. **移位**：`<<` 相当于乘 2^N，左端溢出的位直接丢失；`>>` 的行为**取决于类型**——有符号数做算术右移（复制符号位），无符号数做逻辑右移（左边补 0）。 3. **关**

系运算符恒产生 0 或 1；逻辑运算符只看"真/假"，0 是假、非 0 是真，结果也恒为 0 或 1。 4. **短路求值**：`&&` 在左操作数为假时不再算右边，`||` 在左操作数为真时不再算右边。这是"保护危险操作"的常用手法，例如 `if (0 <= dist_sq && walk_p (me, sqrt (dist_sq)))`。 5. **`=` 是表达式**，其值是右值，所以 `A = B = 0;` 合法，等价于 `A = (B = 0);`；左值 (l-value) 必须有地址，`A + B = 42;` 一定编译失败。 6. **`++` / `--`** 的前后缀差别只在"取用表达式的值"时才出现：`i++` 先读值再自增，`++i` 先自增再读值；`?:` 是三目运算符，`A = (B > 0 ? C : D);` 等价于一段 `if/else` 赋值；逗号运算符依次求值并取最后一个的值（常用于 `for (i = 0, j = 5; i < 3; i++, j--)`）。 7. **优先级**：不要背表。乘法类 > 加减 > 移位 > 关系 > 相等 > 位与 > 异或 > 位或 > 逻辑与 > 逻辑或 > 条件 > 赋值 > 逗号。**只要一眼看不出顺序，就加括号。** * *作用域与存储期*：运算符没有作用域；但"求值顺序"在 C 中**大部分未指定 (unspecified)**——`f(g(), h())` 里哪个先算由编译器决定， 所以不要在同一个表达式里对同一变量既读又写（例如 `j = (++i) + (j++)`），那属于未定义行为 (undefined behavior, UB)。 * **隐式转换 (Implicit Conversion)、整型提升 (Integer Promotion) 与截断 (Truncation)**：不同类型混在一个表达式里时，编译器会按规则"往较大的类型"转。 * *直观解释*：像把两种货币放进同一个收银机：编译器先都换成"大面额"再算，算完塞回你声明的那个小盒子里——塞不进去的部分就被截掉了。 * *底层机制图解*：`int x; x = 3 + 4.6;` 的四步：①把 `3` 从 `int` 转成 `double`；②两个 `double` 相加得 `7.6`；③把和转回 `int`，**截断**成 `7`；④把 `7` 的位写进 `x`。更隐蔽的是**有符号与无符号混用**：`c unsigned a = 10; int b = -20; if (a + b < 0) { /* 不会进来！ */ printf ("ok"); }` `a + b` 先把 `b` 转成 `unsigned`，结果按无符号解释是 4 294 967 286，不可能小于 0。加上显式强制转换 `((int)a) + b < 0` 才会得到 −10。 **整型提升**：比 `int` 窄的类型（`char`、`short`、`bool`）在参与算术前会先提升为 `int`，所以 `int16_t big = 32767; big + 1` 得到的是 `int` 的 32768，而把它存回 `int16_t` 又会绕回 −32768。 * *作用域与存储期*：转换只发生在表达式求值期间，产生临时值 (temporary)；临时值通常放在寄存器或编译器安排的溢出槽 (spill slot) 里，不改变原变量的类型与存储。 * **作用域 (Scope)**：一个名字在程序的哪一段里"看得见"。 * *直观解释*：作用域像办公室的权限范围：全局变量是"全公司都能改的公告板"，文件作用域是"本部门内部备忘录"，块作用域是"会议室白板，散会就擦掉"。 * *底层机制图解*：C 允许三种作用域： * **文件作用域 (file scope)**：写在所有函数外面并用 `static` 修饰，如 `static int my_var;`，只在本文件可见； * **函数 / 块作用域 (function / block scope)**：写在花括号里面，即局部变量 (local variable)，只在那个块里能用； * **全局作用域 (global scope)**：写在所有函数外面且**不加** `static`，全程序可见（本讲建议避免）。 编译器内部维护一张**符号表 (symbol table)** 决定每个名字怎么寻址：

scope	identifier	type	from	offset
translate.c	`the_number`	`int32_t`	`R4`	0
find_abs	`abs_value`	`int32_t`	`R5`	0
find_abs	`num`	`int32_t`	`R5`	4

表里 "from/offset" 就是机器码的寻址方式：`LDR R0,R4,#0` 读全局变量，`LDR R0,R5,#4` 读参数，`STR R0,R5,#0` 写局部变量。 **`static` 在函数外与函数内含义不同**：函数外改变的是**作用域**（把全局名字收窄到本文件），函数内改变的是**存储期**（automatic → static）。 * *作用域与存储期*：作用域

是"能否用名字访问"，存储期是"这块内存何时存在"，两者**互相独立**：`static` 局部变量的名字只在块内可见，存储却从程序开始活到结束。 * **存储期 (Storage Duration) 与内存映射 (Memory Map)**：变量在内存里"活多久、住在哪一段"。 * *直观解释*：把内存想成一栋楼：低层是物业（操作系统），中层是办公楼（代码与全局数据），高层是员工储物柜（栈，从顶往下用），中层与高层之间还有一片"临时租用的仓库"（堆）。 * *底层机制图解*：C 有三种存储类别 (storage class)：

存储类别	何时创建/销毁	存放位置	LC-3 中的指针
static	程序开始到程序结束	全局数据区 (global data)	`R4` 指向该区域顶部
automatic	进入块时创建，离开块时销毁	栈 (stack)	`R6` 指向栈顶，`R5` 是帧指针
dynamic	按需创建与销毁（没有名字，必须由程序自己跟踪地址）	堆 (heap)	由程序自己保存地址

LC-3 内存映射（地址用十六进制，LC-3 共 2^{16} 个 16 位字）：

```
高地址 +--------------------------+ xFFFF | 内存映射 I/O (MMIO) | xFE00 – xFFFF ← 系统空间 +--------------------------+ | 栈 stack（向低地址增长） | R6 → 栈顶，R5 → 当前帧 | main / foo / ... 的数据 | 每调用一个函数就长出一帧 +--------------------------+ | 堆 heap（动态分配） | malloc 一类调用管理，向下生长 +--------------------------+ | 全局数据 global data | R4 → 区域顶部 | 静态变量、字符串字面量 | +--------------------------+ | 代码 code | x3000 起（惯例） +--------------------------+ | 中断/异常向量表 | x0100 – x01FF | 陷阱向量表（trap vector） | x0000 – x00FF 低地址 +--------------------------+
```

栈与堆从两端向中间生长；**一旦相撞**：在 LC-3、嵌入式 ISA 或操作系统内部是静默的数据损坏 (silent data corruption)，多数 ISA 的用户程序则由硬件检测并让程序崩溃。 **全局数据里的变量是程序镜像 (program image) 的一部分**，默认初值为 0；自动变量的初值是"位 (bits)"——可能有 0，也可能没有。 * *作用域与存储期*：三者的组合由"在哪声明 + 是否 `static`"决定：

```
声明位置 + 是否 static → （作用域, 存储期） 函数/块之外 + 不加 static → (全局作用域, static) 函数/块之外 + 加 static → (文件作用域, static) 函数/块之内 + 不加 static → (块作用域, automatic) 函数/块之内 + 加 static → (块作用域, static) dynamic 存储期必须靠显式分配（malloc）获得，稍后一讲
```

* **为什么要避免全局变量 (Why Avoid Globals)**：全局变量让"名字管理"变成噩梦，收益极小、代价极大。 * *直观解释*：想象 1 000 000 行程序、20 个程序员共用一个命名空间，还要和库代码的名字共存——你不可能记得住哪些名字已被用过。 * *底层机制图解*：全局名字不能被"局部化"，链接器 (linker) 必须把它们放进同一个符号表，一个名字冲突就是一次链接错误，而**意外同名**（你的 `count` 与库里的 `count`）可能导致静默的行为改变。 相比之下，文件作用域的 `static` 名字不进全局符号表，不会与其他文件冲突。 * *作用域与存储期*：全局变量是 static 存储期 + 全局作用域，因此它的**每一次读写都跨越了模块边界**，把"局部推理"变成"全局推理"。

代码示例与底层机制分析

代码 (C) — 类型、运算符、转换、作用域与存储期总览：

```c
/*
 * ECE220 Lecture 5 demo -- types, sizes, operators, conversion, scope.
 * Build: gcc -g -std=c99 -Wall -Werror l05_types.c -o l05_types
 */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

/* file scope + static storage duration: exists for the whole program */
static int32_t file_wide_counter = 0;

/* a function whose local has static storage duration */
static int32_t how_many_calls(void)
{
    static int32_t call_count = 0;   /* block scope, static duration */
    call_count = call_count + 1;
    file_wide_counter = file_wide_counter + 1;
    return call_count;
}

/* proof that short-circuit evaluation really skips work */
static int32_t side_effect(const char *name)
{
    printf("    [side_effect(%s) ran]\n", name);
    return 1;
}

struct point {
    int32_t x;
    int32_t y;
};

int main(void)
{
    /* ---------- 1. sizes and encodings ---------- */
    printf("sizeof(bool)=%zu sizeof(char)=%zu sizeof(short)=%zu\n",
           sizeof(bool), sizeof(char), sizeof(short));
    printf("sizeof(int)=%zu sizeof(long)=%zu sizeof(int32_t)=%zu\n",
           sizeof(int), sizeof(long), sizeof(int32_t));
    printf("sizeof(float)=%zu sizeof(double)=%zu sizeof(void *)=%zu\n",
           sizeof(float), sizeof(double), sizeof(void *));
    printf("sizeof(struct point)=%zu\n", sizeof(struct point));

    /* ---------- 2. integer vs floating point arithmetic ---------- */
    printf("11/3=%d  -11/3=%d  -11%%3=%d  (100/8)*8=%d\n",
           11 / 3, -11 / 3, -11 % 3, (100 / 8) * 8);
    printf("11.0/3=%f  3+4.6=%f  (int)(3+4.6)=%d\n",
           11.0 / 3, 3 + 4.6, (int)(3 + 4.6));

    /* ---------- 3. wrapping of narrow and unsigned values ---------- */
    {
        uint8_t byte = 255;
        uint32_t word = 0;
```

```
    int16_t big = 32767;         /* largest int16_t */
    int promoted = big + 1;      /* computed as int -> 32768 */
    int16_t back = (int16_t)promoted;   /* conversion back: wraps */

    byte = byte + 1;             /* wraps around */
    word = word - 1;             /* wraps around */
    printf("uint8 255+1=%u  uint32 0-1=%u\n", (unsigned)byte, word);
    printf("int16 32767+1 as int=%d, stored back as int16=%d\n",
           promoted, (int)back);
}

/* ---------- 4. bitwise operators ---------- */
{
    int a = 120;                 /* 0x00000078 */
    int b = 42;                  /* 0x0000002A */
    unsigned int u = 0xFFFFFF00u;
    int negative = -120;         /* 0xFFFFFF88 */
    printf("a&b=%d a|b=%d a^b=%d ~a=%d a<<2=%d\n",
           a & b, a | b, a ^ b, ~a, a << 2);
    printf("negative>>2=%d  u>>4=0x%08X\n", negative >> 2, u >> 4);
}

/* ---------- 5. signed / unsigned comparison trap ---------- */
{
    unsigned int a = 10;
    int b = -20;
    if (a + b < 0) {
        printf("a+b < 0 is true\n");
    } else {
        printf("a+b < 0 is false (a+b printed with %%u is %u)\n", a + b);
    }
    if ((int)a + b < 0) {
        printf("with an explicit (int) cast the test is true\n");
    }
}

/* ---------- 6. precedence, ternary, comma, assignment ---------- */
{
    int x = 2 + 3 * 4;           /* * binds tighter than + */
    int y = (2 + 3) * 4;         /* parentheses win */
    int z = 10 - 4 - 3;          /* left associative -> 3 */
    int big = 42;
    int small = 0;
    int t = (big > 8 ? 8 : big);
    int i;
    int j;
    printf("2+3*4=%d  (2+3)*4=%d  10-4-3=%d  ternary=%d\n", x, y, z, t);
    for (i = 0, j = 5; i < 3; i++, j--) {
        printf("comma-update: i=%d j=%d\n", i, j);
    }
    small = big = 0;             /* assignment is an expression */
    printf("after small = big = 0: small=%d big=%d\n", small, big);
}
```

```
    /* ---------- 7. short-circuit evaluation ---------- */
    printf("left side of || is true:\n");
    if (1 || side_effect("first ||")) {
        printf("    result of || is true\n");
    }
    printf("left side of && is false:\n");
    if (0 && side_effect("first &&")) {
        printf("    never printed\n");
    } else {
        printf("    result of && is false\n");
    }

    /* ---------- 8. scope and storage duration ---------- */
    {
        int32_t inner = 7;           /* block scope, automatic duration */
        int32_t first;
        int32_t second;

        /* one statement per call: argument evaluation order is unspecified */
        first = how_many_calls();
        second = how_many_calls();
        printf("call 1 -> %d, call 2 -> %d\n", (int)first, (int)second);
        printf("file-wide counter=%d, inner=%d\n",
               (int)file_wide_counter, (int)inner);
    }
    printf("the static local survived its block: next call -> %d\n",
           how_many_calls());

    return 0;
}
```

实际编译运行结果（`gcc -g -std=c99 -Wall -Werror l05_types.c -o l05_types && ./l05_types`，本机 gcc 12.2.0 / x86-64 Linux）：

```
sizeof(bool)=1 sizeof(char)=1 sizeof(short)=2
sizeof(int)=4 sizeof(long)=8 sizeof(int32_t)=4
sizeof(float)=4 sizeof(double)=8 sizeof(void *)=8
sizeof(struct point)=8
11/3=3  -11/3=-3  -11%3=-2  (100/8)*8=96
11.0/3=3.666667  3+4.6=7.600000  (int)(3+4.6)=7
uint8 255+1=0  uint32 0-1=4294967295
int16 32767+1 as int=32768, stored back as int16=-32768
a&b=40 a|b=122 a^b=82 ~a=-121 a<<2=480
negative>>2=-30  u>>4=0x0FFFFFF0
a+b < 0 is false (a+b printed with %u is 4294967286)
with an explicit (int) cast the test is true
2+3*4=14  (2+3)*4=20  10-4-3=3  ternary=8
comma-update: i=0 j=5
comma-update: i=1 j=4
comma-update: i=2 j=3
after small = big = 0: small=0 big=0
left side of || is true:
    result of || is true
left side of && is false:
    result of && is false
call 1 -> 1, call 2 -> 2
file-wide counter=2, inner=7
the static local survived its block: next call -> 3
```

【代码做什么？】 1. 第 1 段用 `sizeof` 打印每种基本类型与一个结构体的字节数，验证基本类型表（`bool` 被补齐成 1 字节，`struct point` 两个 `int32_t` 共 8 字节）。 2. 第 2 段对照整数与浮点运算（整数除法向 0 取整、`3 + 4.6` 被截断）；第 3 段让 `uint8_t` / `uint32_t` 溢出回绕，并展示整型提升：`int16_t` 的 32767 加 1 在 `int` 里得 32768，存回 `int16_t` 又绕回 −32768。 3. 第 4、5 段对 `a = 120 (0x78)`、`b = 42 (0x2A)` 做全部位运算，对比有符号/无符号右移，并复现"有符号无符号混用"陷阱。 4. 第 6、7 段验证优先级、`?:`、逗号运算符、"赋值是表达式"，并用带打印的 `side_effect` 证明短路求值真的跳过了右操作数。 5. 第 8 段证明 `static` 局部变量的存储期跨越了它所在的块，而 `inner` 出了块就不可见。

【底层机制透视】 * `3 + 4.6` 的过程完全是编译期决定的：编译器看到 `int` 与 `double`，生成"把 `3` 转成 `double`（`cvtsi2sd`）→ 浮点加 → 转回 `int`（`cvttsd2si`，向 0 截断）"三条指令。它不会报错，也**不会警告**——自动转换是静默的。 * `uint8_t byte = 255; byte = byte + 1;` 中，`byte` 先被**整型提升**成 `int` 参与加法得 256，再截断回 8 位得 0——所以窄类型的加法"不会提前溢出"，赋值时才回绕。 * `negative >> 2` 得到 −30（`0xFFFFFFE2`）而不是正数，是因为 `int` 的右移是**算术右移**，左侧补的是符号位； 同一份位模式用 `unsigned int` 解释时右移是**逻辑右移**，所以 `0xFFFFFF00u >> 4` 得 `0x0FFFFFF0`。**同一个 `>>` 运算符，编译器根据类型生成两条不同的指令序列**——这正是"类型为编译器提供信息"的最直接证据。 * `a + b < 0` 中，`b` 被转换成 `unsigned int`（转换规则：有符号与同宽无符号相遇时，转成无符号）， 于是 −20 变成 4294967276，加 10 得 4294967286（打印出来正是 `4294967286`），自然不小于 0。 * `static int32_t call_count;` 放在函数里：它**不占栈帧**，而是落在全局数据区并默认为 0，名字只在该函数体内可见——"作用域 ≠ 存储期"的活教材。 * `for (i = 0, j = 5; i < 3; i++, j--)` 的初始化与更新部分各是一个**逗号表达式**：先算左边、再算右边，整个表达式的值是右边那个。

【内存布局图解】

编译器为这个程序安排的内存（示意；LC-3 中以 R4/R5/R6 + 偏移表示）：

```
高地址  +-----------------------------------------------+
        | 栈 main 的栈帧：inner / first / second / i / |
        |   j / x / y / z / big / small / t / a / b     |  ← automatic，离开块即销毁
        +-----------------------------------------------+
        | 堆 heap（本讲未使用 malloc）                   |
        +-----------------------------------------------+
        | 全局数据区：file_wide_counter = 2 (static)    |
        |   how_many_calls.call_count = 3               |  ← 块作用域，但 static 存储期
        |   字符串字面量 "    [side_effect(%s) ran]\n"  |
        +-----------------------------------------------+
        | 代码 code（main、how_many_calls、...）        |
低地址  +-----------------------------------------------+
```

【手工位级推演 (hand trace)：这些位到底落在哪里】

语句	表达式的类型	位模式（按该类型编码）	存进变量后的解释
`int a = 120;`	`int`	`0x00000078`	120
`int b = 42;`	`int`	`0x0000002A`	42
`a & b`	`int`	`0x00000028`	40
`a | b`	`int`	`0x0000007A`	122
`a ^ b`	`int`	`0x00000052`	82
`~a`	`int`	`0xFFFFFF87`	−121（2 的补码）
`a << 2`	`int`	`0x000001E0`	480
`-120 >> 2`	`int`	`0xFFFFFFE2`	−30（算术右移）
`0xFFFFFF00u >> 4`	`unsigned int`	`0x0FFFFFF0`	4 026 531 824（逻辑右移）
`(uint8_t)255 + 1`	先提升为 `int` = 256，存回 8 位	`0x00`	0（回绕）；`int16_t 32767 + 1` 则得 `int` 的 `0x00008000` = 32768，存回 `int16_t` 是 −32768

【与汇编的对应】（LC-3：把上表中的关键表达式翻译出来）

```
; 假设 a 在 R5+0，b 在 R5-1，结果写到 R5-2（都是 int16_t 以适配 LC-3）

; ---- a & b ----   类型是"有符号整数"，用 AND
        LDR     R0,R5,#0        ; R0 <- a
        LDR     R1,R5,#-1       ; R1 <- b
        AND     R0,R0,R1        ; 按位与，与类型的有符号/无符号无关
        STR     R0,R5,#-2

; ---- ~a（按位取反）与 -a（取负）是两回事：~a 是 NOT，-a 是 NOT 后 ADD #1 ----
        LDR     R0,R5,#0
        NOT     R0,R0           ; ~a：每位取反
        STR     R0,R5,#-2

; ---- a << 2 ----   左移 2 位 = 自己加自己两次
        LDR     R0,R5,#0
        ADD     R0,R0,R0        ; << 1
        ADD     R0,R0,R0        ; << 2
        STR     R0,R5,#-2

; ---- 短路求值 if (0 && side_effect(...)) ----
        ; 编译器只生成"求值左操作数 -> 若为假则跳到 else"的代码，
        ; 右操作数的代码块根本不会被跳进去：
        AND     R0,R0,#0        ; 左操作数 0
        BRz     ELSE_BLOCK      ; 结果为假，直接跳过右操作数的求值代码
        JSR     SIDE_EFFECT     ; 这行永远不会被执行
ELSE_BLOCK
        ; ...

; ---- 右移：int 补符号位（算术右移），unsigned 补 0（逻辑右移）----
```

代码 (C) — 求值顺序未指定（第二例，短）：

```c
/*
 * ECE220 Lecture 5 demo -- argument evaluation order is unspecified.
 * Build: gcc -g -std=c99 -Wall -Werror l05_eval_order.c -o l05_eval_order
 */
#include <stdint.h>
#include <stdio.h>

static int32_t trace[4];
static int32_t produced = 0;

static int32_t label(int32_t tag)
{
    trace[produced] = tag;
    produced = produced + 1;
    return tag;
}

static int32_t combine(int32_t first, int32_t second, int32_t third)
{
    return first * 100 + second * 10 + third;
}

int main(void)
{
    int32_t i;
    int32_t packed;

    /* The C standard fixes no order for the arguments of a call; gcc on
       x86-64 evaluates them right to left.  The value of the expression is
       the same either way, but the side effects are observable. */
    packed = combine(label(1), label(2), label(3));
    printf("combine=%d, argument evaluation order:", (int)packed);
    for (i = 0; i < produced; i++) {
        printf(" %d", (int)trace[i]);
    }
    printf("\n");

    return 0;
}
```

实际编译运行结果：

```
combine=123, argument evaluation order: 3 2 1
```

【代码做什么？】 `label()` 每次被调用就记录自己的编号并返回该编号。`printf` 打印出的顺序是 `3 2 1`，说明**实参**被从右往左求值； 而 `combine` 的结果仍然是 123——"哪个先算"不影响表达式的值，只影响副作用 (side effect) 的发生顺序。

【底层机制透视】 C 标准只规定函数调用**实参的值**都被算出来（且不能互相干扰），却没有规定先后；x86-64 的 System V 调用约定把前几个实参放进寄存器， gcc 为图方便从最右边的实参开始填，于是出现 3 2 1。

LC-3 的调用约定同样"从右往左压栈"——这不是巧合：它让第一个参数总是落在固定偏移上（下一讲详解），也正是 `printf` 这类**可变参数函数 (variable-argument function)** 能工作的前提。

【内存布局图解】

```
栈上（gcc 12.2.0 -O0，x86-64）：combine 的三个实参各占调用者栈帧里的一个槽

        高地址  +-----------------+
                | return address |
                +-----------------+
                | third  = 3     |  ← 先求值，先占位（最右边的实参）
                +-----------------+
                | second = 2     |
                +-----------------+
                | first  = 1     |  ← 最后求值（最左边的实参）
        低地址  +-----------------+
        trace[] 在全局数据区，按求值顺序记录：3, 2, 1
```

【与汇编的对应】

```
; 从右往左求值 = 先算第三个实参、最后算第一个实参：
        LEA     R1,THIRD_VAL        ; 最右边的实参
        JSR     LABEL
        STR     R0,R6,#-1           ; 压入第三个实参的副本
        LEA     R1,SECOND_VAL
        JSR     LABEL
        STR     R0,R6,#-2           ; 压入第二个实参的副本
        LEA     R1,FIRST_VAL
        JSR     LABEL
        STR     R0,R6,#-3           ; 第一个实参的副本（地址最低）
        JSR     COMBINE
; 第一个实参永远在最"新"的位置（地址最低）——这就是"右到左压栈"的机器码形态。
```

常见错误与调试技巧

- **有符号与无符号混用 (signed/unsigned mixing)**：`if (a + b < 0)` 永不成立、`0 <= i` 对无符号 `i` 恒真，原因是混合表达式整体转成无符号。**调试**：`gcc -g -std=c99 -Wall -Werror -Wextra -Wsign-compare file.c -o file`；在 `gdb` 里用 `p (unsigned int)b`、`p a + b` 看提升后的值；把操作数统一成 `int32_t` 或 `uint32_t`。
- **整数除法与截断 (integer division / truncation)**：期望 `1.67` 却得到 `1`，期望 `100` 却得到 `96`，因为两个 `int` 相除仍是 `int`。**调试**：`gcc -Wall` 不报警，只能自查；用 `gdb` 的 `p 100/8` 与 `p 100.0/8` 对比；代码里让**一个**操作数带小数点，或写 `(double)sum / count`。
- **优先级猜错 (operator precedence)**：例如 `int x = 1 << 2 + 3;` 想得到 40，实际 `+` 先算。**调试**：`$ gcc -g -std=c99 -Wall -Werror prec_err.c -o prec_err prec_err.c: In function ‘main’: prec_err.c:2:26: error: suggest parentheses around ‘+’ inside ‘<<’ [-Werror=parentheses] 2 | int main(void){int x = 1 << 2 + 3; printf("%d\n", x); return 0;} | ^~ cc1: all warnings`

`being treated as errors` 这种代码在 ECE 220 的 `-Werror` 策略下**根本编译不过**——这是特性，不是障碍。养成"看不清就加括号"的习惯。

- **忘记初始化局部变量 (uninitialized automatic variable)**：局部变量初值是"位"，可能为 0 也可能不是，打印出天文数字常常就是这个原因。**调试**： `valgrind --track-origins=yes ./prog` 报 `Conditional jump or move depends on uninitialised value(s)`；gcc 的 `-Wmaybe-uninitialized -O2` 也能抓；习惯上**声明时就初始化**。
- **有符号溢出与未定义行为 (signed overflow / UB)**：`int32_t` 最大值加 1 是 UB，编译器可以任意处理（例如把整个循环优化掉）。 **调试**：`gcc -g -std=c99 -O1 -fsanitize=undefined -fno-sanitize-recover=all file.c -o file`，运行时会精确报出溢出所在的行； `-ftrapv` 让溢出直接触发陷阱 (trap)。修法：改用 `uint32_t` （回绕是良好定义的）或在做加法前先检查边界。
- **同一表达式里既读又写同一变量，或忘记包含头文件**：`j = (++i) + (j++) + (++j) - (i--);` 是 UB（不同编译器结果不同）； 调用未声明的函数会得到 `implicit declaration of function`，返回值被默认当成 `int`、参数被默认自动转换，在 64 位机器上很容易崩。 **调试**：前者靠代码审查 + `gdb` 单步 `next` 观察子表达式顺序；后者用 `gcc -std=c99 -Wall -Werror` 直接拦住， `gcc -E file.c | less` 确认 `#include` 真的被展开， `nm -u file.o` 查看还有哪些未解析符号。

关键要点

- **类型是给编译器的说明书**：它决定变量的**宽度、编码与解释方式**，从而决定生成 `ADD`、算术右移还是浮点库调用；同一串位在不同类型下可以是完全不同的值。
- **整数与浮点是两个世界**：整数运算会截断、会回绕， `/` 与 `%` 向 0 取整， `(100/8)*8 == 96`；想让表达式走浮点，至少让一个操作数或常量是浮点类型。
- **隐式转换静默发生**： `int` 与 `unsigned` 混合、窄类型提升为 `int`、`double` 转 `int` 截断，全都不报错——**不确定就写显式强制转换**。
- **`static` 是双面词**：在函数外它收窄**作用域**到本文件（代替全局变量），在函数内它把**存储期**变成 static（变量活到程序结束，但仍只在块内可见）。
- **作用域决定"能否用名字访问"，存储期决定"内存何时存在"**：三张表——全局数据（static）、栈（automatic）、堆（dynamic）——就是后面函数、指针、动态分配三讲的舞台。

思考题（带答案）

1. 下面的程序会打印什么？为什么？ （`int16_t a = 300; int b = a * a; printf("%d\n", b);`） **答案**：打印 `90000`。`a` 在乘法前被**整型提升**为 `int`， `300 * 300` 在 `int` 里算，90000 放得下 32 位。若把 `b` 声明成 `int16_t`，90000 会被截断（`90000 & 0xFFFF = 24464`，按 2 的补码是 −41072），打印出负数——类型决定结果。
2. 为什么下面这段代码"看起来没问题"却永远不会打印 `ok`？ `c unsigned int a = 10; int b = -20; if (a + b < 0) { printf("ok\n"); }` **答案**：`a + b` 的公共类型是 `unsigned int`， `b` 被转换成 4294967276，加上 10 得 4294967286，永远不小于 0。 改为 `if ((int)a + b < 0)` 或把 `a` 声明成 `int` 即可。用 `gcc -Wextra -Wsign-compare` 可以让编译器提前提示这类混合。

3. 一个函数里的 `static int32_t counter = 0;` 与写在所有函数外面的 `static int32_t counter = 0;` 有什么本质区别？ **答案**：**存储期相同**（都是 static，位于全局数据区，默认初值 0）；区别在**作用域**：前者块作用域（只有该函数体能用这个名字）， 后者文件作用域（本文件内所有函数都能用，别的源文件看不见）。若函数内那个**去掉** `static`，它变成 automatic，每次进函数重新创建、初值不确定、出函数即销毁，`counter` 就永远数不到 2。

Lecture 6: C 控制结构与基本 I/O (Introduction to C: Control Structures, Basic I/O)

概述

本讲解决两个问题：**如何让程序按条件选择、按次数重复**，以及**如何与人和机器交换数据**。为此引入四类控制语句（`if` / `switch`、`for` / `while` / `do-while`、`break` / `continue`、`return`）与 `printf` / `scanf` 的格式说明符 (format specifier)、返回值语义和缓冲 (buffering) 行为。 这些机制的机器码形态都只是 `LDR` / `BR` / `JSR` / `RET` 的不同编排；而"用库函数做 I/O"这件事本身，又正好引出下一讲的主题——把任务分解成函数。

核心概念与底层机制图解

- **真值性 (Truthiness)**：C 的条件判断只看"是不是 0"——**0 是假 (false)，任何非 0 都是真 (true)**。
  - *直观解释*：像问"箱子里还有货吗"：0 件表示没有，其余都算有。于是 `if (x)` 就是 `if (x != 0)`，`if (!x)` 就是 `if (x == 0)`。
  - *底层机制图解*：关系运算符 `< <= == != >= >` 与逻辑运算符 `&& || !` **恒产生 `int` 的 0 或 1**； 机器码就是"把值装进寄存器 → 条件码 (N/Z/P) 随之更新 → `LDR R0,R5,#0` 之后的 `BRz ELSE_BLOCK` 决定走 then 还是 else"。
  - *作用域与存储期*：条件表达式的值只是"当下"的临时值，通常留在寄存器里，不跨越语句边界。
- **`if` / `else if` / `else`**：二选一或链式多选一的条件执行。
  - *直观解释*：像分诊台：先问"有没有生命危险"，否则问"是不是骨折"，再否则按普通号处理——**顺序本身就是逻辑**。
  - *底层机制图解*：`else if` 只是"`else` 里再嵌一个 `if`"的简写，编译器把它变成一串"测试失败就跳到下一个测试"的 `BR`。两条书写纪律： ① **始终使用花括号 `{ }`**（ECE 220 编码规范）；② **比较时把常量写在左边**（`42 == A`）——手误成 `42 = A` 时编译器一定报错，写成 `A = 42` 则只得到警告、逻辑已变成"恒为真"。 `c if (input < 0) { printf ("Negative\n"); } else if (0 == input) { printf ("Zero\n"); } else { printf ("Positive\n"); }`
  - *作用域与存储期*：每个分支都是一个复合语句 (compound statement)，即一个块；块内声明的变量是 automatic，离开块即销毁。
- **`while` 循环**：先测后做的迭代，可能一次也不执行。
  - *直观解释*：像"只要还有邮件就继续处理"——一开始就没有邮件，就一封也不用处理。

    - *底层机制图解*：`while (<test>) { <body> }` 完全等价于 `<init>`、`<update>` 都空着的 `for`；测试在循环**顶部**，所以 update 必须由循环体自己完成。 机器码形态是 `WHILE_TEST: LDR R0,R5,#-1 / BRzp WHILE_DONE / …循环体与 update… / BRnzp WHILE_TEST`，即"每次回到顶部都要重新读一遍控制变量"。
    - *作用域与存储期*：循环体是块，块内 automatic 变量**每次迭代都重新创建**（初值仍是"位"）；要跨迭代保存状态就得在循环外声明。
- **`do` / `while` 循环**：先做后测，**至少执行一次**。
    - *直观解释*：像"先上菜，再问客人还要不要"——第一轮一定发生，适合"必须先读一个值才能判断"的场景（如 `do { scanf (...) } while (bad);`）。
    - *底层机制图解*：与 `while` 的唯一差别是测试被搬到循环体**后面**：循环体先执行，`BRnzp DO_BODY` 在条件为真时回到顶部。
    - *作用域与存储期*：与 `while` 相同；控制变量一般声明在循环外，因此具有函数级的 automatic 生存期。
- **`for` 循环**：把初始化、测试、更新写在一行的迭代。
    - *直观解释*：像点名册的三栏——"从第 1 号开始、念到第 40 号、每念一个加 1"。
    - *底层机制图解*：执行顺序严格是 init → (test → body → update) → (test → body → update) → …，init 只做一次。课程例子（打印 1–1000 中 42 的倍数）：`c int N; for (N = 1; 1000 >= N; N = N + 1) { if (0 == (N % 42)) { printf ("%d\n", N); } }` 斐波那契例子（`C = A + B; A = B; B = C;`）则展示多个变量在循环里"滚动"。
    - *作用域与存储期*：C99 允许 `for (int32_t i = 0; ...)`，此时 `i` 只活在循环内；ECE 220 代码习惯把循环变量声明在函数开头，以便循环结束后还能检查它的值。
- **`switch`：贯穿 (fall-through)、`break` 与 `default`**：对**一个表达式的多个常量取值**做多路分支。
    - *直观解释*：像电梯面板——按 3 楼停 3 楼，按 5 楼停 5 楼；`default` 是"按了无效楼层时停哪儿"。**没有 `break` 就会一路坐到底。**
    - *底层机制图解*：表达式可以是任意整型表达式，但 `case` 后**必须是编译期常量**；取值少时编译器生成"比较链"，取值密集时生成**跳转表 (jump table)**——把各 `case` 的地址放进全局数据区，用 `LDR` 取出地址再 `JSRR` 跳过去。 `c switch (operator) { case '+': /* add */ result = a + b; break; /* 漏掉它就会继续执行 '-' 的代码 */ case 1: case 2: /* 故意贯穿：1 和 2 共用同一段代码 */ printf ("one or two\n"); break; default: /* 兜住其它取值，应放在最后 */ printf ("unknown\n"); break; }`
    - *作用域与存储期*：整个 `switch` 体**只有一个块作用域**，所有 `case` 共享；在 `case` 里声明变量必须加花括号，否则会与其它 `case` 冲突，还可能"跳过初始化"（未定义行为）。
- **`break` 与 `continue`**：迭代控制语句，只作用于**最内层**的循环或 `switch`。
    - *直观解释*：`continue` 是"这道题跳过，回到答题流程"；`break` 是"这场考试不考了，交卷走人"。
    - *底层机制图解*：`continue` 对 `for` 会**跳到 update**，对 `while` / `do-while` 会**跳到 test**（它们没有 update）；`break` 跳到当前结构之后的第一条语句。 `for (...) { while (...) { if (skip) continue; ⟶ update if (skip) continue; ⟶ test if (done) break; ⟶ 循环之后 if`

`(done) break;` ⟶ `循环之后 } }` 嵌套循环里的 `break` **不会**跳出外层循环；要跳两层就用标志变量配合外层判断，或把内层封装成函数用 `return`。
    - *作用域与存储期*：两者不创建作用域，只是把控制流（LC-3 里就是把 `PC`）搬到另一处。
- `return` **语句**：立即结束当前函数并把一个值交回调用者。
    - *直观解释*：像"交卷"——交完就不再答题，哪怕后面还有代码没执行。
    - *底层机制图解*：LC-3 上 `return <expr>;` 分四步，与调用约定严格对应：`assembly LDR R0,R5,#0 ; ① 求表达式的值（先放 R0） STR R0,R5,#3 ; ② 写进栈帧的"返回值"槽 LDR R7,R5,#2 ; ③ 拆栈帧：恢复 R7 与 R5 LDR R5,R5,#1 ADD R6,R6,#3 ; 弹出局部变量与 linkage（保留返回值） RET ; ④ PC ← R7，回到调用者` 正因如此，`return` 可以出现在任何位置（包括循环体内），它一次完成"带返回值 + 跳回"两件事。
    - *作用域与存储期*：`return` 触发**整个栈帧的销毁**，该函数所有 automatic 变量在 `RET` 之后失效，内存会被下一个被调用者复用——**绝不能返回指向局部变量的地址**。
- **循环设计五步法 (designing loops)**：课程给出的系统化流程，用来"证明"循环的正确性。
    - *直观解释*：动笔前先回答五个问题，像施工前的五张图纸，避免边写边补。
    - *底层机制图解*：`0. 重复的任务是什么？（循环体） 1. 每次进入"测试"时什么一定为真？（不变式 invariant） 2. 迭代何时停止？（可能有多个条件） 3. 迭代结束后做什么？（不同停止条件可能不同） 4. 第一次迭代前做什么？（init 让不变式首次成立） 5. 迭代之间如何更新？（update 维持不变式）` 课程的内存转储例子：任务="每行打印 12 个内存单元"；不变式="`start` 是行首地址且为 12 的倍数"；测试 = `start >= addr_e`；init= `start = (addr_s / 12) * 12`；update= `start = start + 12`。技巧：给 `addr_e` 加 `0x10000` 造"虚拟副本"，就能用**一个**循环处理地址回绕 (wrap-around)。
    - *作用域与存储期*：不变式涉及的变量必须在循环外声明（要跨迭代保存）；只在单次迭代内用到的临时量可以在循环体内声明。
- `printf` **与格式说明符**：把值按指定格式写到标准输出 (stdout)。
    - *直观解释*：格式串是一张"填空表"：普通字符原样输出，`%` 开头的是空位，后面的表达式按顺序填进去。
    - *底层机制图解*：课程 C 编程参考给出的完整表格（`printf` 类型 / `scanf` 类型）：| specifier | format description | printf type | scanf type | |---|---|---|---| | `%c` | ASCII character | `char` | `char *` | | `%d` | signed decimal integer | `int` | `int *` | | `%f` | decimal with fractional part | `float` | `float *` | | `%hd` | signed decimal integer | `short` | `short *` | | `%hhd` | signed decimal integer | `char` | `char *` | | `%hhu` | unsigned decimal integer | `unsigned char` | `unsigned char *` | | `%hhx` | hexadecimal | `unsigned char` | `unsigned char *` | | `%hu` | unsigned decimal integer | `unsigned short` | `unsigned short *` | | `%hx` | hexadecimal | `unsigned short` | `unsigned short *` | | `%ld` | signed decimal integer | `long` | `long *` | | `%lf` | decimal with fractional part | `double` | `double *` | | `%lu` | unsigned decimal integer | `unsigned long` | `unsigned long *` | | `%lx` | hexadecimal | `unsigned long` | `unsigned long *` | | `%p` | hexadecimal address | `T *` | `T **` | | `%s` | ASCII string | `char *` | `char *` | | `%u` | unsigned decimal integer | `unsigned int` | `unsigned int *` | | `%x` | hexadecimal | `unsigned int` | `unsigned int *` |

要点：① 除说明符外字符**原样输出**，想要空格就自己写（`printf("%d%d%d", 12, -34, 56)` 打印 `12-3456`）；② 转义序列 `\n`、`\\`、`%%`； ③ 宽度与填充：`%04X` = 至少 4 字符宽、右对齐、前导 0 填充（课程打印地址时使用）；④ 类型必须严格匹配，`printf("%d %f", 10.0, 17)` 会按错误的类型解释位模式（系统相关，可能是 `0 0.000000`），说明符多于实参时会去读"位"（行为未指定）；⑤ **返回值是写出的字符个数**（出错时 < 0），例如 `printf("hello\n")` 返回 6（含换行符）。 * *作用域与存储期*：格式串是字符串字面量，位于静态内存区，具有 static 存储期，程序运行期间一直存在。 * **`scanf` 与"为什么要传地址"**：从标准输入 (stdin) 按格式读取并转换数值。 * *直观解释*：`printf` 是"把值拿出来给人看"，只要**值**；`scanf` 是"把值塞进你的盒子"，必须知道**盒子在哪**，所以传地址（`&`）。 * *底层机制图解*：`scanf ("%d", &A);` 的机器码形态是"把 `A` 在栈帧里的偏移算出来当参数压栈"，`scanf` 通过这个地址把结果 `STR` 回去。**忘记写 `&` 会把结果写到地址 0 或垃圾地址上（行为未指定）。**其它关键语义： 1. `%d` / `%f` / `%s` 会**跳过前导空白**，所以 `5 42` 与 `5\n42` 都能被 `scanf("%d%d", &A, &B)` 正确读入；而 `%c` **不跳过任何字符**，读到的可能是上一行遗留的换行，要读"下一个非空白字符"必须写 `" %c"`。 2. 格式串里的其它字符必须原样输入：`scanf("%d<>%d", &A, &B)` 要求输入 `5<>42`；`%s` 则扫描下一个"单词"并连同结尾 NUL 写入数组，**数组必须足够大**。 3. **返回值是成功转换的个数**，没有成功转换时返回 −1（EOF）。**必须检查**：`c if (2 != scanf ("%d%d", &A, &B)) { printf ("Bad input!\n"); A = 42; B = 10; /* 使用默认值 */ }` 实测（输入 `12 34\nAB\n`）：读两个整数后 `scanf` 返回 2；紧接着 `scanf("%c", &c1)` 返回 1 且 `c1 = 0x0A`（遗留的换行）；改成 `scanf(" %c", &c2)` 后返回 1 且 `c2 = 'A'`。 * *作用域与存储期*：`scanf` 写入的是调用者栈帧里的自动变量（或静态数据、堆对象），所以必须拿到地址；这也要求那些变量在调用期间一直存活。 * **缓冲 (Buffering)**：标准 I/O 不保证每次调用都立刻访问设备，而是先攒在内存缓冲区里。 * *直观解释*：像超市收银台——每位顾客单独结账太慢，先把手推车装满再一起去结，或等收银员喊"下一位"（遇到换行）。 * *底层机制图解*：三种模式——**行缓冲**（stdout 连终端，遇换行就冲刷）、**全缓冲**（stdout 被重定向到文件/管道，缓冲区满才写出，通常 4096 字节）、**无缓冲**（stderr 总是立刻写出）。实测程序与结果：`c printed = printf("1: to stdout\n"); /* 先进缓冲区 */ fprintf(stderr, "2: to stderr\n"); /* 立刻写出 */ printf("3: printf returned %d characters for line 1\n", (int)printed); fflush(stdout); /* 强制冲刷 */` `2: to stderr 1: to stdout 3: printf returned 13 characters for line 1` **先打印的那一行反而后出现**——因为 stdout 全缓冲、stderr 无缓冲。调试时 `printf` 不带 `\n` 又想知道"执行到哪了"，要么 `fflush(stdout);`，要么临时改用 `fprintf(stderr, ...)`。 * *作用域与存储期*：stdout/stdin/stderr 及其缓冲区是库在静态存储区维护的对象，从程序启动活到程序退出；正常结束会自动冲刷，被信号杀死或调用 `_exit` 则可能丢掉缓冲区内容。

代码示例与底层机制分析

代码 (C) — 一元二次方程求解器（课程示例 `04-quadratic.c`）：

```
/* solution of the quadratic equation ax^2+bx+c=0
   Adapted from V. Kindratenko's notes on 30 August 2016.
   Build: gcc -g -std=c99 -Wall -Werror l06_quadratic.c -o l06_quadratic -lm
*/

#include <stdio.h>   /* needed for printf and scanf */
#include <math.h>    /* needed for sqrtf */

int
main()
{
    float a, b, c;   /* quadratic equation coefficients */
    float D;         /* discriminant */
    float x1, x2;    /* solution(s) */

    /* Get equation coefficients. */
    printf ("Enter a, b, and c: ");
    if (3 != scanf ("%f %f %f", &a, &b, &c)) {
        printf ("Three real coefficients are required.\n");
        return 3; /* Program failed. */
    }

    printf ("Solving equation %fx^2+%fx+%f=0.\n", a, b, c);

    /* Compute discriminant. */
    D = b * b - 4 * a * c;

    /* Compute solution. */
    if (0 < D) {           /* Two real roots exist. */
        x1 = (-b + sqrtf (D)) / (2 * a);
        x2 = (-b - sqrtf (D)) / (2 * a);
        printf ("x1=%f, x2=%f\n", x1, x2);
    } else if (0 == D) {   /* Only one root exists. */
        x1 = -b / (2 * a);
        printf ("x=%f\n", x1);
    } else {
        printf ("No real roots exist\n");
    }

    /* End program successfully. */
    return 0;
}
```

实际编译运行结果（四种输入，均用管道送入）：

```
$ echo "1 -3 2" | ./l06_quadratic
Enter a, b, and c: Solving equation 1.000000x^2+-3.000000x+2.000000=0.
x1=2.000000, x2=1.000000
$ echo "1 2 1" | ./l06_quadratic
Enter a, b, and c: Solving equation 1.000000x^2+2.000000x+1.000000=0.
x=-1.000000
$ echo "1 0 1" | ./l06_quadratic
Enter a, b, and c: Solving equation 1.000000x^2+0.000000x+1.000000=0.
No real roots exist
$ echo "1 2" | ./l06_quadratic ; echo "exit status = $?"
Enter a, b, and c: Three real coefficients are required.
exit status = 3
```

【代码做什么？】 声明系数 `a, b, c`、判别式 `D` 与两根；打印提示后用 `scanf("%f %f %f", ...)` 读三个实数，**返回值不等于 3** 就打印错误并 `return 3`； 计算 `D = b*b - 4*a*c`；按 `D > 0` / `D == 0` / `D < 0` 三分支求两根、求唯一根、报"无实根"；成功返回 0，失败返回 3（`echo $?` 可见）。

【底层机制透视】 * `scanf` 必须传地址（`&a`），因为它的任务是**写入**；`a` 是 `float` 所以必须用 `%f`，写成 `%lf`（`double`）会按 8 字节写入并覆盖相邻变量。`3 != scanf(...)` 一次覆盖三种情形：正常返回 3、非法输入时返回已成功的个数、在第一个字符前遇到文件结束返回 −1。 * `float` 作为可变参数传给 `printf` 时会**自动提升为 `double`**，这就是表中 `%f` 在 `printf` 侧写 `float`、实际却接受 `double` 的原因；`scanf` 传地址无法提升，故 `%f` 对应 `float *`、`%lf` 对应 `double *`。 * `(2 * a)` 是 `float`，不会踩整数除法陷阱（若系数是 `int`，`-b / (2 * a)` 会被截断）。LC-3 没有浮点指令，`sqrtf` 与浮点算术全由库子程序模拟，`-lm` 就是去数学库找它们。

【内存布局图解】

```
main 栈帧（x86-64, -O0）：高地址到低地址依次是 c、b、a、D、x1、x2，各占 4 字节（float）
        &a = 0x7ffd....f44   &b = 0x7ffd....f48   &c = 0x7ffd....f4c   ← scanf 收到的是地址
        D = 9 - 8 = 1.0f；输入 "1 -3 2" 时 x1 = 2.0f, x2 = 1.0f
```

【与汇编的对应】（LC-3：三分支结构 + 库调用 + 返回）

```
        LDR     R0,R5,#-3       ; R0 <- D
        BRp     TWO_ROOTS       ; D > 0 → 两个实根
        BRn     NO_ROOTS        ; D < 0 → 无实根
ONE_ROOT
        LDR     R0,R5,#-2       ; 计算 -b / (2a)：float 运算由库子程序完成
        LDR     R1,R5,#-1
        ADD     R1,R1,R1        ; 2*a
        JSR     FLOAT_DIV       ; R0 / R1 → R0
        STR     R0,R5,#-4       ; x1
        BRnzp   QUAD_DONE
TWO_ROOTS
        LDR     R0,R5,#-3       ; 压入唯一实参 D → JSR → 读 R6+0 → 弹栈
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     SQRTF           ; R0 <- sqrtf (D)
        ADD     R6,R6,#2
NO_ROOTS
        LEA     R0,MSG_NO_ROOTS ; 对应 printf ("No real roots exist\n")
        JSR     PRINT_STRING
QUAD_DONE
        AND     R0,R0,#0        ; return 0：写返回值槽，再拆栈帧
        STR     R0,R5,#3
        LDR     R7,R5,#2
        LDR     R5,R5,#1
        ADD     R6,R6,#6
        RET
```

代码 (C) — 素数打印器（课程示例 `primes.c` 的整理版）：

```c
/*
 * ECE220 Lecture 6 demo -- a prime number printer.
 * Adapted from the course example primes.c (ECE220 Spring 2018).
 * Build: gcc -g -std=c99 -Wall -Werror l06_primes.c -o l06_primes
 */

#include <stdint.h>
#include <stdio.h>

/* A "static" function is usable only inside this file. */
static int32_t is_prime (int32_t num);
static int32_t divides_evenly (int32_t divisor, int32_t value);

int
main ()
{
    int32_t check;
    int32_t count = 0;

    /* 2 is the smallest prime number. */
    for (check = 2; 1000 > check; check++) {
        if (is_prime (check)) {
            printf ("%d is prime.\n", check);
            count = count + 1;
        }
    }
    printf ("%d primes below 1000.\n", count);

    return 0;
}

static int32_t
is_prime (int32_t num)
{
    int32_t divisor;

    /* Check every possible divisor from 2 to num - 1. */
    for (divisor = 2; num > divisor; divisor++) {
        if (divides_evenly (divisor, num)) {
            return 0;
        }
    }

    return 1;
}

static int32_t
divides_evenly (int32_t divisor, int32_t value)
{
    int32_t multiple;

    /* Integer arithmetic: (value / divisor) * divisor rounds down, so it
       equals value exactly when divisor divides value evenly. */
```

```
    multiple = (value / divisor) * divisor;

    return (multiple == value);
}
```

实际编译运行结果（`./l06_primes` 共 169 行，首尾如下）：

```
2 is prime.
3 is prime.
5 is prime.
7 is prime.
11 is prime.
13 is prime.
...
983 is prime.
991 is prime.
997 is prime.
168 primes below 1000.
```

【代码做什么？】 `main` 用 `for` 从 2 遍历到 999，对每个数调用 `is_prime`，为真就打印并累加 `count`；`is_prime` 用 `for` 从 2 试到 `num - 1`，发现能整除就 `return 0`（一次结束循环与函数），循环自然结束说明没有因子，`return 1`；`divides_evenly` 用 `(value / divisor) * divisor == value` 判断整除；最后打印总数 168。

【底层机制透视】 * `return` 是 `is_prime` 的"双出口"：只写 `break` 的话，函数会继续执行到 `return 1`，把合数判成素数——初学者最常见的逻辑错误。`for` 的测试在每次迭代**之前**求值，所以 `num = 2` 时第一次测试就为假并直接 `return 1`；写成 `num >= divisor` 就会把 2 判成合数（边界必须逐个手推）。 * 用整数除法表达整除（而非 `%`）是课程的有意选择：`(value / divisor) * divisor` 直接对应 LC-3 上"调用除法库子程序、再乘回去比较"的机器码序列。 * `static` 函数只在本文件可见、不进全局符号表，避免与库或其它文件重名——与上一讲"避免全局变量"是同一思路。

【内存布局图解】

```
调用链上的三个栈帧（正在检查 num = 5 时），每个函数各持一份副本：
高地址  +---------------------------------+
        | main 的栈帧：check = 5, count = 3 |
        +---------------------------------+
        | is_prime 的栈帧：num 副本 = 5     |   ← 参数是值的拷贝
        |                  divisor = 2     |   ← automatic，每次调用重新创建
        +---------------------------------+
        | divides_evenly 的栈帧：           |
        |   divisor 副本 = 2, value = 5    |
        |   multiple = (5 / 2) * 2 = 4     |
低地址  +---------------------------------+
三份 num/divisor 互不影响——这就是"按值传递"。
```

【与汇编的对应】（LC-3：`for` 循环骨架 + "调用函数并取回返回值"的标准五步）

```
MAIN
        ADD     R6,R6,#-5       ; 2 个局部变量（check, count）+ 3 个 linkage 字
        STR     R5,R6,#2
        ADD     R5,R6,#1        ; R5 -> 局部变量底部（check）
        STR     R7,R5,#2
        AND     R0,R0,#0        ; check = 2
        ADD     R0,R0,#2
        STR     R0,R5,#0
        AND     R0,R0,#0        ; count = 0
        STR     R0,R5,#-1
FOR_TEST
        LDR     R0,R5,#0        ; R0 <- check
        ADD     R1,R0,#-9       ; 与 1000 比较（示意）
        BRzp    FOR_DONE        ; 测试失败 → 跳出循环
        LDR     R0,R5,#0        ; ① 求值并压入实参 check
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     IS_PRIME        ; ② 调用
        LDR     R0,R6,#0        ; ③ 从栈顶读返回值
        ADD     R6,R6,#2        ; ④ 弹出返回值与实参
        BRz     FOR_UPDATE      ; 返回 0（非素数）→ 跳过打印
        ; ...打印 "%d is prime.\n" 并把 count 加 1
FOR_UPDATE
        LDR     R0,R5,#0        ; check++（for 的 update 部分）
        ADD     R0,R0,#1
        STR     R0,R5,#0
        BRnzp   FOR_TEST        ; 回到测试
FOR_DONE
        AND     R0,R0,#0        ; return 0：写返回值槽、拆栈帧、RET
        STR     R0,R5,#3
        LDR     R7,R5,#2
        LDR     R5,R5,#1
        ADD     R6,R6,#4
        RET
```

代码 (C) — 循环设计实例：打印空心正方形（课程的 think-pair-share 任务）：

```
/*
 * ECE220 Lecture 6 demo -- loop design: print a hollow square.
 * Adapted from the think-pair-share task in the course lecture
 * "Designing Loops" (531).
 * Build: gcc -g -std=c99 -Wall -Werror l06_square.c -o l06_square
 */
#include <stdint.h>
#include <stdio.h>

/*
 * Function: print_square
 * Description: prints a hollow square of asterisks of the given size
 * Parameters: size -- the side length, in characters, of the square
 * Return Value: 0 on success, -1 if size is not positive
 * Side effects: writes to the display
 */
int32_t
print_square (int32_t size)
{
    int32_t row;
    int32_t col;

    /* Argument checking: reject anything that has no meaning here. */
    if (1 > size) {
        return -1;
    }

    for (row = 0; size > row; row++) {
        for (col = 0; size > col; col++) {
            /* Only the four edges of the square are filled. */
            if (0 == row || 0 == col || row == size - 1
                || col == size - 1) {
                printf ("*");
            } else {
                printf (" ");
            }
        }
        printf ("\n");
    }

    return 0;
}

int
main ()
{
    int32_t size;

    for (size = 1; 6 > size; size++) {
        if (0 != print_square (size)) {
            printf ("size %d rejected\n", (int)size);
        }
    }
```

```
    /* The error case: a size that cannot be drawn. */
    printf ("print_square(0) returned %d\n", (int)print_square (0));
    printf ("print_square(-3) returned %d\n", (int)print_square (-3));

    return 0;
}
```

实际编译运行结果（`./l06_square`，此处只列出 size = 1、2、5 与两个错误返回值）：

```
*
**
**
 (size = 3、4 的输出按同样规则展开)
*****
*   *
*   *
*   *
*****
print_square(0) returned -1
print_square(-3) returned -1
```

【代码做什么？】 `print_square` 先做参数检查（`size < 1` 直接 `return -1`），再用外层 `for` 走行、内层 `for` 走列； 只有处于四条边之一时才打印 `*`，否则打印空格，每行末尾打印换行；`main` 打印 1–5 的图形并演示两个非法尺寸的返回值。

【底层机制透视】 * 五步法的落地：① 任务是"打印一整行"；② 内层测试时的不变式是"该行已开头、已打印 `col` 个字符"；③ 停止条件是 `col >= size`；④ 每开新行必须重新 `col = 0`（由 `for` 的 init 保证）；⑤ 更新是 `col++`，行末再 `row++`。 * 边界值手推：`size == 1` 时 `row`、`col` 只取 0，条件 `0 == row` 成立，输出恰好一行一个星号；`size == 2` 时四条边条件全成立，输出 2×2 的实心块。若把 `col = 0` 写到循环外，第二行会从 `col = size` 开始、什么都不打印。逐字符 `printf("*")` 每次都要进入库函数，这也是"逐字符打印很慢"的原因。

【内存布局图解】

```
print_square 的栈帧 (size = 5):            输出 (每行 5 个字符):
+--------------------------+                row=0: * * * * *   ← row == 0 恒真
| 参数 size       |   5    |               row=1: *       *   ← col == 0 或 col == size-1
+--------------------------+                row=2: *       *
| 局部 row        |  0..4  |               row=3: *       *
+--------------------------+                row=4: * * * * *   ← row == size-1 恒真
| 局部 col        |  0..4  |  ← 每行重新从 0 开始
+--------------------------+
| linkage (R7 / R5 / 返回值) |
+--------------------------+
row、col 是 automatic，函数返回即销毁；size 是参数副本，改它不影响调用者。
```

【与汇编的对应】（LC-3：双重循环，重点是"每次进内层前重新初始化 col"）

```
        AND     R1,R1,#0        ; row = 0
ROW_TEST
        LDR     R0,R5,#4        ; size
        NOT     R2,R1
        ADD     R2,R2,#1
        ADD     R2,R2,R0        ; size - row
        BRzp    ROW_DONE        ; <= 0 → 外层结束
        AND     R2,R2,#0        ; col = 0   ← 每开一行都要重新执行!
COL_TEST
        LDR     R0,R5,#4        ; size
        NOT     R3,R2
        ADD     R3,R3,#1
        ADD     R3,R3,R0        ; size - col
        BRzp    COL_DONE
        ADD     R3,R1,#0
        BRz     PRINT_STAR      ; row == 0（其余三条边的判断同理）
        LEA     R0,SPACE_CHAR
        JSR     PRINT_CHAR
        BRnzp   COL_NEXT
PRINT_STAR
        LEA     R0,STAR_CHAR
        JSR     PRINT_CHAR
COL_NEXT
        ADD     R2,R2,#1        ; col++
        BRnzp   COL_TEST
COL_DONE
        LEA     R0,NEWLINE
        JSR     PRINT_CHAR      ; 行末换行
        ADD     R1,R1,#1        ; row++
        BRnzp   ROW_TEST
ROW_DONE
```

常见错误与调试技巧

- **`=` 与 `==` 混淆**：`if (A = 42)` 恒为真且改写了 `A`，因为赋值是表达式、其值是右值。**调试**：`gcc -std=c99 -Wall -Werror` 给出 `suggest parentheses around assignment used as truth value`；把常量写左边（`42 == A`）让手误变成编译错误；`gdb` 里断点后 `p A` 看它是否被意外改写。
- **缺少花括号**：`if (x > 0) sum += x; count++;` 中 `count++` 不在条件内。**调试**：`gcc -Wall -Wmisleading-indentation`（gcc 6+ 默认开启）会直接报警；`clang-format -i file.c` 让"缩进骗人"暴露；`gdb` 单步 `next` 观察是否真的跳过了那行。
- **`switch` 漏写 `break`（贯穿）**：某个 `case` 执行完继续执行下一个 `case`。**调试**：`gcc -Wall -Wimplicit-fallthrough -Werror`；`gdb` 在各 `case` 首行设断点并用 `bt` 看从哪儿跳进来；确实要贯穿时补 `/* fall through */` 注释。
- **`for` 的 init/update 放错位置**：把 `col = 0` 写到外层循环之外，内层只跑一轮。**调试**：循环体开头打印 `row`、`col`；`gdb` 里 `watch col` 在值变化时自动停下，`break l06_square.c:41` 精确停在某一行。
- **循环边界差一 (off-by-one)**：`num >= divisor` 把 2 判成合数；`0 <= i` 对无符号 `i` 是死循环。**调试**：先手推最小/最大/边界三个输入；`gcc -fsanitize=undefined` 抓可疑比较；`gdb` 的 `until` 跑完

已知正确的一轮再 `p` 变量。

- **`scanf` 忘写 `&` 或说明符与类型不匹配**：忘 `&` 会写到垃圾地址；`%f` 配 `double`、`%d` 配 `long` 会读写错误宽度。**调试**：`gcc -std=c99 -Wall -Werror` 的 `-Wformat` 会检查这对函数；运行时加 `-fsanitize=address -g` 或 `valgrind --track-origins=yes ./prog` 定位非法写入。
- **不检查 `scanf` 返回值 / 缓冲让提示"迟到"**：前者使变量保持"位"、后续结果不可预测；后者让不带换行的提示在被重定向时最后才出现。**调试**：用 `printf 'abc\n' | ./prog` 故意喂坏输入，`gdb` 里在 `scanf` 之后 `p conversions`（x86-64 也可 `p $eax`）；给提示加 `\n` 或 `fflush(stdout);`，用 `strace -e write ./prog` 看真正的写顺序，诊断信息临时改用无缓冲的 `fprintf(stderr, ...)`。

关键要点

- **0 是假、非 0 是真**，关系与逻辑运算符恒产生 0/1；因此 `if (x = 42)` 不会报语法错误，必须靠"常量写左边 + 始终写花括号"这两条书写纪律来防。
- **控制结构的角色分工**：`if` / `switch` 做条件分解，`for` / `while` / `do-while` 做迭代分解，`break` / `continue` 决定跳到 test、update 还是循环之外，`return` 一次完成"返回值 + 拆栈帧 + RET"。
- **写循环前先回答五个问题**（任务、不变式、停止条件、结束后的处理、init 与 update）并**手推三个边界输入**；循环的 bug 几乎都藏在初始化位置与边界比较上。
- **`printf` / `scanf` 的说明符必须与类型严格匹配、`scanf` 必须传地址**：`" %c"` 才是"读下一个非空白字符"，`"%c"` 会读到上一次输入遗留的换行；而返回值是判断 I/O 成功的唯一手段（`printf` 返回字符数，`scanf` 返回成功转换个数，EOF 为 −1）——stdout 的缓冲更意味着"你以为已经打印了"不等于"真的写出去了"。

思考题（带答案）

1. 下面两段循环各执行多少次循环体？（`int32_t i = 0; while (i < 3) { i++; }` 与"从 `i == 3` 开始的 `do { i--; } while (i > 0);`"） **答案**：都是 3 次（前者 `i` 从 0 到 3，后者从 3 减到 0）。若把前者的初值改成 5，`while` 执行 0 次而 `do-while` 仍执行 1 次——这就是两者唯一的区别。
2. 用户依次输入 `42`、回车、`A`、回车。下面代码中 `c` 得到什么？如何修好？ `c int32_t n; char c; scanf ("%d", &n); scanf ("%c", &c);` **答案**：`c` 得到 `'\n'`（0x0A），因为 `%d` 把数字后的换行留在输入缓冲区里。修法：改成 `scanf (" %c", &c);`（`%` 前加空格跳过所有空白），或先清空缓冲区（`while ('\n' != getchar ()) { }`）。
3. 为什么课程要求把常量写在比较式的左边（`42 == size`）？举一个能被编译器抓住、而写反后抓不住的例子。 **答案**：`42 == size` 手误成 `42 = size` 时编译器立刻报 `lvalue required as left operand of assignment`（常量不是左值）；而 `size == 42` 手误成 `size = 42` 完全合法（值是 42、恒为真），通常只得到一条容易忽略的警告。更隐蔽的 `if (x = y)`（本意是比较）既让条件恒等于 `y` 的值，又把 `x` 写坏。

Lecture 7: C 函数：定义、调用、参数传递与返回值 (Introduction to Functions in C)

概述

本讲把"一个 `main` 打天下"的程序拆成多个函数，回答**函数如何成为 C 里最基本的分解工具**这一问题。 我们引入函数签名 (signature) 与原型 (prototype)、按值传递 (call by value)、返回值、多文件编译与头文件、数组参数退化为指针、`void`， 并预览递归 (recursion)。函数是"系统分解"落地的单元：它把接口（签名）与实现（函数体）分开，让不同的人、不同的源文件可以独立开发与测试， 而"参数是值的拷贝"这条规则又直接决定了下一讲的栈帧布局——调用者把自己的值压到栈上，被调用者拿到的永远是一份副本。

核心概念与底层机制图解

- **函数签名 (Function Signature)**：函数的名字、参数的个数与类型、返回值的类型。
  - *直观解释*：签名就是"插座规格"——几孔、什么形状、能出多少电；插头（调用）不合规格，编译器直接拒绝。
  - *底层机制图解*：签名让编译器① 核对实参个数、② 做必要的类型转换（或拒绝）、③ 决定返回值放哪里。LC-3 上调用函数固定四步： `assembly LDR R0,R5,#0 ; ① 求值实参到 R0 → 压栈（先 ADD R6,R6,#-1 再 STR） ADD R6,R6,#-1 STR R0,R6,#0 JSR MY_FUNC ; ② 调用（R7 ← 返回地址） LDR R0,R6,#0 ; ③ 从栈顶读返回值 ADD R6,R6,#2 ; ④ 弹出返回值与实参`
  - *作用域与存储期*：函数名具有 file scope 或 global scope（加 `static` 则只在本文件可见）；函数**代码**的存储期是整个程序，与调用次数无关。
- **声明 (Declaration) 与定义 (Definition)、原型 (Prototype)**：声明只给签名（以分号结束），定义还给函数体。*直观解释*：声明是"菜单上写着有这道菜"，定义是"厨房里真的会做"。
  - *底层机制图解*： `int32_t f (int32_t h, int32_t w);` 是声明，去掉分号并跟上 `{ ... }` 就是定义。 **声明里的参数名一定要写**： `int32_t f (int32_t, int32_t);` 语法合法，但两个类型相同，谁也看不出哪个是 height。 **早期 C 允许不声明就调用**，编译器只能假设"整型实参转 `int`、浮点转 `double`、返回值是 `int`"——这些假设在 64 位机器上经常是错的，而编译器没有签名就**无法提醒你**。
  - *作用域与存储期*：声明不分配存储，只往符号表里放一条"名字 → 签名"的记录。
- **按值传递 (Call by Value)**：C 把**实参的值拷贝**给被调用者，被调用者拿到的是自己的副本。*直观解释*：像把文件复印一份交给同事——他可以在复印件上随意写字，你的原件不会有任何变化。
  - *底层机制图解*：① 调用者求值实参；② 把**值**压栈；③ 副本构成被调用者栈帧的参数区；④ 被调用者随便改自己那份；⑤ 返回时调用者把副本弹掉（ `ADD R6,R6,#3` = 2 个实参 + 1 个返回值）。 **关键推论**：若参数是"指向某物"的指针，被调用者改不了指针本身，却能**顺着指针改它所指向的对象**——这正是数组能被函数修改的原因。
  - *作用域与存储期*：形参的作用域是整个函数体，存储期是 automatic，函数一返回副本即消失。**绝不要返回指向形参或局部变量的地址。**

- **返回值 (Return Value)**：函数用 `return <表达式>;` 交回一个值，类型由签名规定。*直观解释*：像外卖员把餐送到门口——送到那一刻（`return`）服务就结束，之后的代码都不会执行。
  - *底层机制图解*：LC-3 上 `return` 与调用约定严格对应——求值 → 写返回值槽（`R5+3`）→ 恢复 `R7` / `R5` → 弹出局部与 linkage → `RET`。 函数可以有多个 `return`（"发现就提前返回"），但**每条控制路径都必须返回值**，否则返回的是寄存器里剩下的位；返回类型是 `void` 时不能返回值（可以 `return;`）。
  - *作用域与存储期*：返回值是**临时值**，由调用者立刻使用或存进自己的变量，不占用被调用者的栈帧。
- **多文件、头文件与分别编译 (Separate Compilation)**：接口放 `.h`，实现放 `.c`，各自独立编译再由链接器拼起来。
  - *直观解释*：头文件是"对外公布的产品说明书"，`.c` 是"车间图纸"；客户只看说明书，车间改工艺不影响客户。
  - *底层机制图解*：`#include "x.h"` 是**预处理器的文本替换**，把头文件原样插进当前文件，于是每个 `.c` 都是自包含的编译单元； 头文件必须加**包含卫士 (include guard)**（`#ifndef / #define / #endif`）防止重复插入。链接器看到的是符号而不是源码： `assembly ; main.o : U array_sum （未定义，需要在别处找） ; stats.o : T array_sum （已定义，地址确定） ; 链接器把 U 填成 T 的地址` —这就是多文件能"拼"起来的原因。
  - *作用域与存储期*：`.h` 里只放声明（原型、类型、宏）；放定义会让每个包含它的 `.c` 各生成一份实体，链接时报"重复定义"。`static` 函数只在本编译单元可见，不会与其它文件冲突。
- **数组作为参数：退化为指针 (decay to pointer)**：形参写 `int32_t values[]` 时会被编译器**立刻转换成 `int32_t *values`**。
  - *直观解释*：把一整排储物柜搬进函数太贵，所以只把"第一排柜子的门牌号"抄一份递进去。
  - *底层机制图解*：课程参考明确写出这条例外：`void foo(int x[4]);` 会被立刻转换成 `void foo(int * x);`。两个直接后果： ① 函数里 `sizeof (values)` 得到**指针大小**（8 字节）而不是数组大小，`gcc -Wall` 会用 `-Wsizeof-array-argument` 直接报错； ② 数组长度无法从地址推出，**必须另传一个长度参数**。访问 `values[i]` 的机器码就是"指针 + i × 元素大小"： `assembly LDR R1,R5,#4 ; R1 <- values（只是个地址，占 1 个字） ADD R1,R1,R2 ; 加上 i × 元素大小（LC-3 上 int32_t 是 2 个字） ADD R1,R1,R2 LDR R3,R1,#0 ; R3 <- values[i]`
  - *作用域与存储期*：数组元素本身属于**调用者**的栈帧；被调用者只拿到首元素地址，因此可以读写调用者的数据，但调用者一返回这些元素就失效。
- **`void`**：C 里"什么都不是"的类型：`void f (void)` 表示不收参数，返回类型写 `void` 表示不返回值。*直观解释*：像一台只打印、不找零的售货机——用了它就知道"没有回执可查"。
  - *底层机制图解*：课程建议**尽量少用 `void` 返回类型**。理由很实际：函数现在总能成功，100 个调用点就都不检查失败；将来它需要处理失败，你得改 100 处。 让函数返回 `int32_t`（0 成功、非 0 是错误码）或 `bool`，调用点从第一天起就有地方检查： `c void print_slot (int32_t slot); /* 只输出，永远"成功"，可以用 void */ int32_t print_square (int32_t size); /* 参数非法时返回 -1，调用者必须检查 */`
  - *作用域与存储期*：`void` 只是类型信息，不涉及存储；返回 `void` 的函数在 LC-3 上不写返回值槽，栈顶剩下的就是实参。

- **参数与局部变量的作用域与存储期**：形参与局部变量都属函数/块作用域 + automatic 存储期。
  - *直观解释*：它们住在"临时工位"上：上班（进入函数）时分配，下班（返回）时收回，第二天来的是另一个人。
  - *底层机制图解*：被调用者执行期间的栈帧（自高地址向低地址）：`高地址 +------------------------ | 调用者的栈帧 | +-------------------------+ | 参数副本（实参的拷贝） | ← R5+4, R5+5, ... +-------------------------+ | 返回值槽 | ← R5+3（返回后位于栈顶） +------------------------ | 返回地址（R7） | ← R5+2 +-------------------------+ | 上一个帧指针（R5） | ← R5+1 +-------------------------+ | 局部变量 | ← R5+0, R5-1, ... 低地址 +------------------------` **同名不冲突**：`f` 里的 `arg` 与 `main` 里的 `arg` 是两个完全不同的内存位置，一个在 `f` 的帧里、一个在 `main` 的帧里。
  - *作用域与存储期*：作用域决定"名字能在哪段代码里使用"，存储期决定"这块内存何时存在"；局部变量的名字只在函数体内可见，内存只在函数执行期间有效。
- **递归 (Recursion) 预览**：函数直接或间接调用自己。
  - *直观解释*：像俄罗斯套娃：打开一个里面还有一个同样结构的小娃娃，直到最小的那个（**基准情形 base case**）为止。
  - *底层机制图解*：每次递归都**再压一个新栈帧**（新的参数副本、新的局部变量）。课程的递归策略与循环五步法同构：`c ______ recursive ( ______ ) { // 1. 检查停止条件（base case） // 2. 处理当前这一个节点 // 3. 处理"孩子"（递归调用） }` 递归、数学归纳法与硬件位切片 (bit-slicing) 是同一思想的三种形式：先解决一小块，再与"剩余同类问题"的解组合。 **忘记 base case 会无限递归**，每次调用消耗一个栈帧，最终栈溢出 (stack overflow)。
  - *作用域与存储期*：每层递归都有独立的 automatic 存储；递归深度就是同时存活的栈帧数，深递归会消耗大量栈空间（LC-3 的栈从 `xFE00` 向下生长，空间有限）。
- **函数设计准则 (Function Design Guidelines)**：把"能跑的代码"变成"可维护的代码"。
  - *直观解释*：一个函数应该像一件称手的工具——只干一件事、拿起来就知道怎么用、坏了能单独送去修。
  - *底层机制图解*：① **单一职责**：`read_values` 只负责读、`array_sum` 只负责求和； ② **小而可测**：短到能在脑子里跑完，每个函数都能用几组输入单独验证； ③ **在边界处检查参数**：`print_square` 先判 `size < 1`、`guessing_game` 先判取值区间，非法输入立即返回错误码，**不要"过度解释"含义**（`print_square(-10)` 不该被理解成"画三角形"）； ④ **文档化**：按 ECE 220 约定在定义上方写清 INPUTS / OUTPUTS / RETURN VALUE / SIDE EFFECTS；⑤ **优先返回值而不是 `void`**，给调用者留出检查失败的位置。
  - *作用域与存储期*：良好的函数边界让每个函数的 automatic 变量都"短命"——状态不会悄悄地跨越很远的地方存活，这正是减少 bug 的来源。

代码示例与底层机制分析

代码 (C) — 按值传递：被调用者改不动调用者的变量：

```
/* * ECE220 Lecture 7 demo -- C passes arguments by value.
   * Build: gcc -g -std=c99 -Wall -Werror l07_byvalue.c -o l07_byvalue
*/
#include <stdint.h>
#include <stdio.h>

/* The address of the callee's first parameter, kept so that main can
   compare it with the address of its own variable. */
static intptr_t callee_first_address;

/* This function tries to swap its two parameters.  It cannot succeed:
   first and second are copies that live in this function's stack frame. */
static void
try_to_swap (int32_t first, int32_t second)
{
    int32_t temp;

    callee_first_address = (intptr_t)&first;

    printf ("  inside try_to_swap: &first=%p &second=%p\n",
            (void *)&first, (void *)&second);

    temp = first;
    first = second;
    second = temp;

    printf ("  inside try_to_swap: first=%d second=%d\n",
            (int)first, (int)second);
}

/* The parameter n is also a copy; the caller's variable is untouched. */
static int32_t
add_one_by_value (int32_t n)
{
    n = n + 1;
    return n;
}

int
main ()
{
    int32_t x = 7;
    int32_t y = 42;
    int32_t result;

    printf ("in main:            &x=%p &y=%p\n", (void *)&x, (void *)&y);
    printf ("in main:            x=%d y=%d\n", (int)x, (int)y);
    printf ("x and y are %ld bytes apart inside main\n",
            (long)((intptr_t)&x - (intptr_t)&y));

    try_to_swap (x, y);

    printf ("x is %ld bytes above the callee's first parameter\n",
```

```
            (long)((intptr_t)&x - callee_first_address));
    printf ("in main after call: x=%d y=%d\n", (int)x, (int)y);

    result = add_one_by_value (x);
    printf ("add_one_by_value(x) returned %d, x is still %d\n",
            (int)result, (int)x);

    return 0;
}
```

实际编译运行结果（`./l07_byvalue`；栈地址每次运行都会变，这里给出一次真实运行的输出）：

```
in main:            &x=0x7ffe8f859438 &y=0x7ffe8f859434
in main:            x=7 y=42
x and y are 4 bytes apart inside main
  inside try_to_swap: &first=0x7ffe8f85940c &second=0x7ffe8f859408
  inside try_to_swap: first=42 second=7
x is 44 bytes above the callee's first parameter
in main after call: x=7 y=42
add_one_by_value(x) returned 8, x is still 7
```

【代码做什么？】 `main` 打印 `x`、`y` 的地址与值；`try_to_swap (x, y)` 在函数内部确实完成了交换，但回到 `main` 后 `x`、`y` **完全没有变化**；`add_one_by_value (x)` 返回 8，而 `x` 仍是 7。

【底层机制透视】 * **地址就是证据**：`main` 里 `&x = 0x7ffe8f859438`，`try_to_swap` 里 `&first = 0x7ffe8f85940c`——两者相差 44 字节，属于**不同的栈帧**。`first` 是 `x` 的副本，交换副本当然不会影响原件。`y` 与 `second` 之间同理。 * **被调用者的参数区由调用者准备**：LC-3 上就是"求值 → 压栈 → `JSR`"；x86-64 上前几个实参走寄存器、再由被调用者存进自己的帧，所以 `&first` 落在被调用者的帧里。 * **想让函数改变调用者的数据只能传地址**：把"门牌号"（指针）按值传进去，函数改不了门牌号本身，却能按门牌号改房间里的东西；返回值同样是临时值。

【内存布局图解】

```
高地址  +--------------------------------+  ← main 的栈帧
        |  int32_t x = 7      (0x...438) |     try_to_swap 的帧在更低 44 字节处:
        |  int32_t y = 42     (0x...434) |       参数副本 first  (0x...40c)
        +--------------------------------+       参数副本 second (0x...408)
        |  ... try_to_swap 的栈帧 ...    |       局部变量 temp
低地址  +--------------------------------+      x、y 与 first、second 是完全不同的内存
交换只发生在 first / second 上，x / y 一动不动。
```

【与汇编的对应】（LC-3：按值传递的完整调用序列与"改不到原件"的事实）

```
; main 里: try_to_swap (x, y) —— 压入的是 x、y 的"值"
        LDR     R0,R5,#0        ; R0 <- x 的值
        ADD     R6,R6,#-1
        STR     R0,R6,#0        ; 压入 x 的副本（第一个实参最后压，地址最低）
        LDR     R0,R5,#-1       ; 同理压入 y 的副本
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     TRY_TO_SWAP
        ADD     R6,R6,#2        ; 返回类型是 void，只弹掉两个实参
; TRY_TO_SWAP 内部只读写自己帧里的 R5+4、R5+5（副本），
; 交换完成后 RET；main 帧里的 x、y 从未被写过，所以值不变。
```

代码 (C) — 三文件程序：头文件 + 两个源文件：

`l07_stats.h`（只有声明，带包含卫士）：

```
/* ECE220 Lecture 7 -- header for the array-statistics module (declarations only). */
#ifndef L07_STATS_H
#define L07_STATS_H

#include <stdint.h>

/* read_values: reads integers into values[] (room for capacity elements);
   returns how many were read, or -1 if none could be read. */
int32_t read_values (int32_t values[], int32_t capacity);

/* print_array: prints the first count elements of values. */
void print_array (const int32_t values[], int32_t count);

/* array_sum: returns the sum of the first count elements (0 if count <= 0). */
int32_t array_sum (const int32_t values[], int32_t count);

/* array_mean: returns the mean of the first count elements (0.0 if count <= 0). */
double array_mean (const int32_t values[], int32_t count);

/* pointer_size_in_callee: reports sizeof() of an array parameter, to show that
   an array parameter is really a pointer. */
int32_t pointer_size_in_callee (const int32_t values[]);

#endif /* L07_STATS_H */
```

`l07_stats.c`（实现）：

```c
/* ECE220 Lecture 7 -- implementation of the array-statistics module. */
#include <stdint.h>
#include <stdio.h>

#include "l07_stats.h"

int32_t
read_values (int32_t values[], int32_t capacity)
{
    int32_t count = 0;

    if (1 > capacity) {                 /* check arguments first */
        return -1;
    }
    while (capacity > count && 1 == scanf ("%d", &values[count])) {
        count = count + 1;              /* keep reading until input ends */
    }
    if (0 == count) {
        return -1;
    }
    return count;
}

void
print_array (const int32_t values[], int32_t count)
{
    int32_t i;

    printf ("values:");
    for (i = 0; count > i; i++) {
        printf (" %d", (int)values[i]);
    }
    printf ("\n");
}

int32_t
array_sum (const int32_t values[], int32_t count)
{
    int32_t i;
    int32_t total = 0;

    for (i = 0; count > i; i++) {
        total = total + values[i];
    }
    return total;
}

double
array_mean (const int32_t values[], int32_t count)
{
    if (1 > count) {
        return 0.0;
    }
```

```
    /* The (double) cast forces the division to be done in floating point. */
    return (double)array_sum (values, count) / (double)count;
}

int32_t
pointer_size_in_callee (const int32_t values[])
{
    /* Writing sizeof (values) here is a bug that gcc catches with
       -Werror=sizeof-array-argument; an array parameter IS a pointer. */
    const int32_t *as_pointer = values;

    return (int32_t)sizeof (as_pointer);
}
```

`l07_main.c`（调用者）：

```
/* ECE220 Lecture 7 -- the main file of a three-file program. */
#include <stdint.h>
#include <stdio.h>

#include "l07_stats.h"

#define MAX_VALUES 6

int
main ()
{
    int32_t numbers[MAX_VALUES];
    int32_t count;

    printf ("enter up to %d integers: ", MAX_VALUES);
    count = read_values (numbers, MAX_VALUES);
    if (0 > count) {
        printf ("no numbers were read\n");
        return 1;
    }

    print_array (numbers, count);
    printf ("sum  = %d\n", (int)array_sum (numbers, count));
    printf ("mean = %f\n", array_mean (numbers, count));

    /* sizeof() is answered by the compiler, and the answer depends on
       whether the name is still an array or has decayed to a pointer. */
    printf ("sizeof(numbers) in main = %d bytes\n", (int)sizeof (numbers));
    printf ("sizeof(values) in the callee = %d bytes\n",
            (int)pointer_size_in_callee (numbers));

    return 0;
}
```

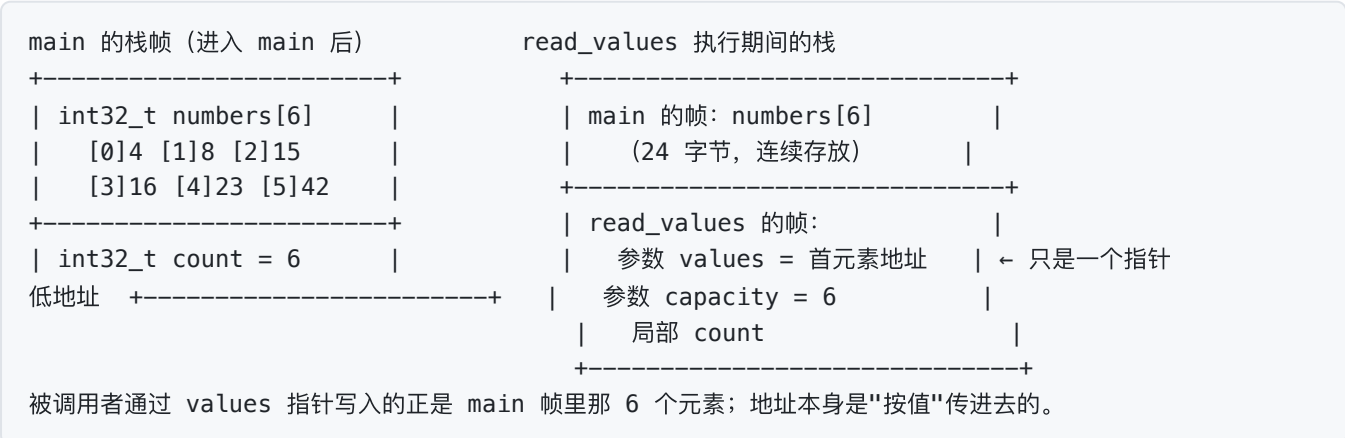
精确的编译命令与实际的链接结果（三个文件一起编译，链接成功）：

```
$ gcc -g -std=c99 -Wall -Werror -o l07_stats_demo l07_main.c l07_stats.c
$ printf '4 8 15 16 23 42' | ./l07_stats_demo
enter up to 6 integers: values: 4 8 15 16 23 42
sum  = 108
mean = 18.000000
sizeof(numbers) in main = 24 bytes
sizeof(values) in the callee = 8 bytes
$ printf '5 5 5' | ./l07_stats_demo
enter up to 6 integers: values: 5 5 5
sum  = 15
mean = 5.000000       (两次运行的 sizeof 两行完全相同：24 与 8)
```

【代码做什么？】 `l07_main.c` 声明数组并调用 `read_values`、`print_array`、`array_sum`、`array_mean`；`l07_stats.c` 提供全部实现，两个文件都靠 `#include "l07_stats.h"` 拿到签名；`read_values` 用 `scanf` 反复读整数直到输入结束或数组满。打印出的 24 与 8 分别证明"数组名在 `main` 里是数组（6 × 4 字节）"与"在函数里已退化成指针（8 字节）"。

【底层机制透视】 * **`#include` 是文本替换**：预处理器把 `l07_stats.h` 原样插进两个 `.c`，让两个编译单元都看到完整原型；包含卫士保证它被多次包含时声明只出现一次。 * **分别编译 + 链接**：`gcc -c l07_stats.c` 会产出 `l07_stats.o`，其中 `array_sum` 是**已定义符号 (T)**；`l07_main.c` 编译出的 `l07_main.o` 里 `array_sum` 是**未定义符号 (U)**。链接器的工作就是把所有 `U` 接到对应的 `T` 上；缺一个就是 `undefined reference`，多一个（同名定义两次）就是 `multiple definition`。 * **数组退化与长度**：`read_values (numbers, MAX_VALUES)` 传入的是"首元素地址 + 长度"两个值。`read_values` 里的 `values[count]` 被编译成"从 `values` 出发、偏移 `count × sizeof(int32_t)`"，所以它写进去的正是 `main` 的数组元素—— 这就是"数组可以被函数修改"的机制。 * **`const` 的用处在签名里**：`const int32_t values[]` 承诺"我不修改你的数组"，于是 `print_array`、`array_sum` 这些只读函数不会意外写入调用者的数据； 同时 `const` 也让"传字符串字面量"这样的调用成为合法。 * **两处 `printf` 的证据**：`sizeof (numbers)` 在 `main` 里是 24（6 个 `int32_t`），在函数里对数组参数得 8（`int32_t *`）。

【内存布局图解】



【与汇编的对应】（LC-3：传"数组指针 + 长度"并按下标访问）

```
; main 里调用 array_sum (numbers, count)：传"首元素地址 + 长度"
        LEA     R0,numbers      ; 第一个实参：数组首元素地址（只占 1 个字）
        ADD     R6,R6,#-1
        STR     R0,R6,#0        ; 注意：压的是地址，不是 6 个元素
        LDR     R0,R5,#3        ; 第二个实参：count
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     ARRAY_SUM
        LDR     R0,R6,#0        ; 返回值（和）
        ADD     R6,R6,#3        ; 弹出返回值 + 2 个实参
; array_sum 内部：values 在 R5+4、count 在 R5+5；
;   LDR R1,R5,#4 / ADD R1,R1,R2 / ADD R1,R1,R2 / LDR R3,R1,#0
; 就是 values[i]（指针 + i × 2 个字），所以它写进的正是 main 的数组。
```

代码 (C) — 递归预览：

```c
/* ECE220 Lecture 7 demo -- a preview of recursion.
   Build: gcc -g -std=c99 -Wall -Werror l07_recursion.c -o l07_recursion */
#include <stdint.h>
#include <stdio.h>

static int32_t fib_calls = 0;   /* static storage: survives between calls */

/* factorial prints its own call chain: each level adds two spaces. */
static int32_t
factorial (int32_t n, int32_t depth)
{
    int32_t result;

    printf ("%*sfactorial(%d)\n", 2 * (int)depth, "", (int)n);
    if (1 >= n) {
        return 1;               /* base case: stop recursing */
    }
    result = n * factorial (n - 1, depth + 1);
    printf ("%*sfactorial(%d) = %d\n", 2 * (int)depth, "", (int)n,
            (int)result);
    return result;
}

/* naive Fibonacci: the number of calls grows exponentially */
static int32_t
fib (int32_t n)
{
    fib_calls = fib_calls + 1;
    if (2 > n) {
        return n;               /* base cases: fib(0) = 0, fib(1) = 1 */
    }
    return fib (n - 1) + fib (n - 2);
}

int
main ()
{
    int32_t i;
    int32_t value;

    printf ("factorial(5) = %d\n", (int)factorial (5, 0));
    for (i = 0; 11 > i; i++) {
        fib_calls = 0;
        value = fib (i);        /* one statement per call: argument
                                   evaluation order is unspecified */
        printf ("fib(%2d) = %4d, using %d calls\n",
                (int)i, (int)value, (int)fib_calls);
    }
    return 0;
}
```

实际编译运行结果（`./l07_recursion`）：

```
factorial(5)
  factorial(4)
    factorial(3)
      factorial(2)
        factorial(1)
      factorial(2) = 2
    factorial(3) = 6
  factorial(4) = 24
factorial(5) = 120
factorial(5) = 120
fib( 0) =    0, using 1 calls
fib( 5) =    5, using 15 calls
fib(10) =   55, using 177 calls
```

【代码做什么？】 `factorial` 先打印当前层（缩进表示深度），到基准情形 `n <= 1` 就返回 1，否则求出 `n-1` 的阶乘再乘 `n`； `fib` 用最朴素的 `fib(n-1) + fib(n-2)` 递归，并用静态存储期的 `fib_calls` 统计调用次数（每次调用前用单独的语句清零，避免求值顺序问题）。

【底层机制透视】 * **基准情形是刹车**：两个 base case 都必须在递归调用**之前**检查，否则永远到不了出口；打印出的缩进层级就是同时存活的栈帧数（`factorial(5)` 最深时 5 个帧）。 * **每层都有自己的副本**：每层递归的 `n`、`result` 都在新的栈帧里，这正是"按值传递 + automatic 存储"的直接结论。 * **调用次数指数增长**：1, 1, 3, 5, 9, 15, 25, 41, 67, 109, 177——同一子问题被反复求解，这就是"记忆化"与动态规划的动机。 > **注意 Fibonacci 有两种起点约定**：本讲用的是常见的 `fib(0)=0, fib(1)=1`，所以 `fib(5)=5`、`fib(10)=55`。 > **第 13 讲**用的是 Patt & Patel 教材的 `F(0)=1, F(1)=1`，同一函数会得到 `fib(5)=8`、`fib(25)=121393`。 > 两种约定下**调用次数完全相同**（上面这串数字两种约定都成立），因为它们满足同一个递推 > `T(n) = 1 + T(n-1) + T(n-2)`；只有返回值差一位下标。看清基准情形，就不会被数字差异迷惑。 * **`fib_calls` 必须是 static 存储期**：它要在两次调用之间保留值；若声明成循环体内的 automatic 变量，计数永远是 1。 * **递归与迭代等价**：任何递归都能改写成"显式栈 + 循环"；但树、图这类结构本身递归的问题，递归写法几乎总是更短更清晰。

【内存布局图解】

```
factorial(5, 0) 执行到最深处的栈（自高地址向低地址）：
高地址   +-----------------------------+    返回顺序与调用顺序相反（后进先出）：
         | main 的栈帧                 |    1 → 2 → 6 → 24 → 120，与打印结果一致
         +-----------------------------+
         | factorial(5,0): n=5         |
         +-----------------------------+
         | factorial(4,1): n=4         |
         +-----------------------------+
         | factorial(3,2): n=3         |
         +-----------------------------+
         | factorial(2,3): n=2   … 直到 factorial(1,4): n=1（基准情形，最先销毁）
低地址   +-----------------------------+
fib_calls 是 static 存储期，放在全局数据区，不属于任何一层栈帧。
```

【与汇编的对应】（LC-3：递归调用的骨架——与普通调用完全相同，被调用者就是自己）

```
FACTORIAL                       ; 参数 n 在 R5+4，depth 在 R5+5
        ADD     R6,R6,#-4       ; 1 个局部变量（result）+ 3 个 linkage 字
        STR     R5,R6,#1
        ADD     R5,R6,#0
        STR     R7,R5,#2        ; 必须保存 R7：递归调用会覆盖它！
        LDR     R0,R5,#4
        ADD     R0,R0,#-1       ; if (1 >= n) → base case
        BRnz    BASE_CASE
        LDR     R0,R5,#4        ; 压入实参 n-1（depth+1 同理），然后：
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     FACTORIAL       ; 递归调用自己，被调用者就是本函数
        LDR     R1,R6,#0        ; R1 <- factorial(n-1)
        ADD     R6,R6,#3        ; 弹出返回值 + 2 个实参
BASE_CASE
        LDR     R7,R5,#2        ; 每层恢复的是"本层保存的"返回地址
        LDR     R5,R5,#1
        ADD     R6,R6,#3
        RET
```

常见错误与调试技巧

- **以为按值传递能改变调用者的变量**：写了 `void swap (int32_t a, int32_t b)` 却发现没换成功。**调试**：打印 `&a` 与调用者变量的地址就能看出是两个不同的地址； `gdb` 里 `break swap` 后 `p &a` 与 `p &x` 对比，或 `up` 到调用者帧看 `x` 的值。
- **忘记声明（原型）**：C99 会给 `implicit declaration of function` 警告，并假设返回值是 `int`、实参按默认规则转换，在 64 位机器上常导致崩溃。**调试**： `gcc -std=c99 -Wall -Werror` 直接拦住； `gcc -E l07_main.c | grep array_sum` 确认头文件真的被包含； `nm -u l07_main.o` 查还有哪些未解析符号。
- **`sizeof` 用在数组参数上**：在函数里写 `sizeof (values)` 得到 8 而不是 24。**调试**： `$ gcc -g -std=c99 -Wall -Werror -c l07_stats.c l07_stats.c:60:28: error: 'sizeof' on array function parameter 'values' will return size of 'const int32_t *' {aka 'const int *'} [-Werror=sizeof-array-argument]` 修法：在数组仍然"是数组"的地方（如 `main`）算好长度，再作为参数传进去。
- **头文件缺少包含卫士 / 在头文件里放定义**：前者导致 `redefinition` 或重复声明，后者导致链接时报 `multiple definition of 'array_sum'` 。**调试**： `gcc -E -H l07_main.c 2>&1 | head -20` 打印实际的头文件包含树； `nm l07_main.o | grep array_sum` 看符号是 `T`（定义）还是 `U`（引用）—— 头文件里只留声明，定义放回 `.c` 。
- **递归缺少基准情形或基准情形太晚**：程序跑一会儿以 `Segmentation fault` 结束（栈溢出）。**调试**： `gdb ./prog` 后 `run`，崩溃时 `bt 20` 会打印几十层重复的递归帧； 也可以在函数开头打印参数（像本例那样），观察它是否朝基准情形前进。
- **数组越界写坏调用者的栈帧**： `capacity` 判错或忘了判断时，多读进来的元素会覆盖相邻变量甚至返回地址。**调试**： `gcc -g -fsanitize=address -std=c99 -Wall file.c -o file` 会在越界那一刻精确报

错；`valgrind --leak-check=full --track-origins=yes ./prog` 也能定位；`gdb` 里 `p count`、`p capacity` 检查边界条件。

关键要点

- **函数是 C 的基本分解单元**：签名定义接口（名字、参数、返回类型），函数体定义实现；只要签名不变，实现的改动不会影响任何调用点。
- **C 只有按值传递**：被调用者拿到参数的**拷贝**，改它不会改调用者；想改调用者的数据必须传"指向它的地址"（指针），这也是下一讲栈帧机制的出发点。
- **数组作为参数会退化为指针**：函数里 `sizeof` 得到的是指针大小，因此**长度必须单独传**；这也意味着函数拥有读写调用者数组元素的能力。
- **多文件程序 = 声明与实现分离**：头文件放原型（带包含卫士），`.c` 放定义，`gcc` 一次列出所有 `.c` 交给链接器把未定义符号接上。
- **递归 = 每层一个新栈帧 + 一个可靠的基准情形**：先写停止条件并确认每次调用都在向它靠近；深度过大时改用循环以免栈溢出。

思考题（带答案）

1. 下面两次调用之后 `x` 分别是多少？为什么？（`static void bump (int32_t n) { n = n + 1; }` 与 `static void real_bump (int32_t *n) { *n = *n + 1; }`；调用序列：`int32_t x = 5; bump (x); printf ("%d\n", x); real_bump (&x); printf ("%d\n", x);`） **答案**：先打印 `5`，再打印 `6`。`bump` 改的是副本 `n`；`real_bump` 收到的虽然是"地址的副本"，但顺着这个地址改的是 `x` 本身。这说明"按值传递"限制的是**参数本身**，而不是参数所指向的对象。
2. 同一个数组在 `main` 里 `sizeof` 得 24，在被调函数的参数上 `sizeof` 得 8，解释这两个数字。 **答案**：`main` 里它是真正的数组（6 个 `int32_t`，6 × 4 = 24 字节）；作为实参传给函数后它退化为指向首元素的**指针**，所以 `sizeof` 得到指针大小（64 位机器 8 字节）。这也说明函数无法从参数得知数组长度，必须另传长度参数；在 ECE 220 的编译选项下，gcc 会用 `-Werror=sizeof-array-argument` 把这类写法直接判为错误。
3. 为什么课程建议"函数尽量少返回 `void`"？不返回 `void` 的函数该怎么设计返回值？ **答案**：`void` 意味着"调用者没有地方检查失败"；一旦这个函数将来需要处理错误，你就得回头修改所有调用点。更好的做法是返回 `int32_t`（0 成功、非 0 是错误码）或 `bool`，并在函数开头检查参数、在文档注释里写清每个返回值的含义——这样调用点从第一天起就能写 `if (0 != print_square (size)) { ... }`。

Lecture 8: C 函数的实现：运行时栈与栈帧 (Implementing Functions in C, Run-Time Stack)

概述

本讲回答"编译器究竟怎样把函数调用变成机器码"这一问题，核心是**运行时栈 (run-time stack) 与栈帧 (stack frame)** 这两个抽象；我们引入活动记录 (activation record)、LC-3 调用约定（`R0–R3` caller-saved、`R4` 全局数据指针、`R5` 帧指针、`R6` 栈指针、`R7` 返回地址）、 实参**自右向左压栈**的理由、完整的调用与返回序列，并手工把一段 C 代码翻译成 LC-3 汇编。 这一讲是 ECE 220 的枢纽：上一讲的"按值传递"在这里得到机器级解释（副本就在栈上）， 而后面要学的数组、指针、递归、动态内存分配，全部建立在"谁能看见谁的栈帧、谁能改谁的副本"这一套约定之上。

核心概念与底层机制图解

- **活动记录 / 栈帧 (Activation Record / Stack Frame)**：一次函数调用在栈上占用的那一整块内存，装着这次调用所需的全部"私人数据"。
  - *直观解释*：像去餐厅吃饭时服务员给你划出的那块桌面：你的餐盘、账单、别人给你的便条都放这里；吃完走人，桌面立刻被下一位客人使用。
  - *底层机制图解*：栈在 LC-3 中**向低地址增长**；压栈 (push) 是先把 `R6` 减 1，再 `STR`；弹栈 (pop) 是 `ADD R6,R6,#1`。 每个函数调用都会在栈顶**长出一个新帧**，返回时整个帧被"弹掉"——所谓弹掉，只是把 `R6` 抬回去，内存里的旧位还在，但**逻辑上已经无效**。 `栈（自高地址向低地址生长，R6 指向栈顶）`

    ```
    +--------------------------------+ | 调用者的栈帧 | 高地址 +-------------------------------
    --+ | 被调用者的栈帧（本次调用） | ← R6 与 R5 都在这一块里 +--------------------------------+
    | ...（更深的调用继续向下） | 低地址
    ```
  - *作用域与存储期*：帧内所有 automatic 变量的生命周期 = 这次调用的生命周期；`return`（或 `RET`）之后它们**立即失效**，内存会被下一次调用复用。
- **调用约定 (Calling Convention)**：寄存器分工与栈帧格式的固定约定，让不同的人（甚至不同的编译器）写出的代码能互相调用。
  - *直观解释*：像机场的行李转运规则：哪件行李放哪个传送带、标签怎么写、谁负责搬——所有航站楼都照同一份规则办，行李才不会丢。
  - *底层机制图解*：ECE 220 使用的 LC-3 约定：

寄存器	角色	谁负责保存
`R0–R3`	传递参数与返回值用的通用寄存器	**caller-saved**（调用者保存）
`R4`	全局数据指针 (global data pointer)，指向全局数据区	全程序公用
`R5`	帧指针 (frame pointer)，指向当前帧的局部变量区	被调用者保存进自己的帧
`R6`	栈指针 (stack pointer)，指向栈顶	由调用/返回序列维护

寄存器	角色	谁负责保存
R7	返回地址 (return address)，JSR 写入、RET 使用	被调用者保存进自己的帧

为什么要"约定"而不是"随便"：① 编译器本身是个程序，只能按固定规则生成代码；② 不同编译器（或手写汇编）产生的子程序必须能互相调用，所以对调用接口的选择必须一致。编译器可以在这个约定**之内**自由优化（见后文），因为帧的内部布局不是接口的一部分。 * *作用域与存储期*：约定决定了"跨函数可见"的东西——只有栈上的实参、返回值槽与 R0 的返回值能跨越函数边界；帧内部的局部变量对外完全不可见。 * **栈帧布局 (Stack Frame Layout)**：帧被分成"linkage（链接信息）+ 局部变量 + 参数"三部分。 * *直观解释*：像一份三明治：最下面（地址最低）是别人递进来的原料（参数），中间是保存回去的路（linkage），最上面是你自己加工出来的东西（局部变量）。 * *底层机制图解*：R5 指向局部变量区的**底**，也就是该帧的第一个局部变量（R5+0）；其余局部变量依次落在 R5-1、R5-2...，R6 是栈顶： 高地址 +----------+ ← 调用者的栈帧 | caller's stack frame | +----------+ | 参数（实参的副本） | R5+4, R5+5, R6+6, ... ← 第一个实参在 R5+4 +----------+ | 返回值槽 | R5+3 +----------+ | 返回地址（R7） | R5+2 +----------+ | 上一个帧指针（调用者的 R5）| R5+1 +----------+ | 局部变量 | R5+0, R5-1, R5-2, ... ← R5 指向第一个局部变量（R5+0） 低地址 +----------+ ← R6（执行中会因压栈而暂时更低） R5+1/R5+2/R5+3 这三格就是 linkage：把它们连起来，就能从任意深度的帧一路"走"回 `main`。 **偏移是编译器算好的常量**：num 在 R5+4、abs_value 在 R5+0，于是 C 语句被翻译成 LDR R0,R5,#4、STR R0,R5,#0 这样的定长指令。 **这个顺序就是课程的真实约定**（已对照 lc3code/translate.asm 的 FIND_ABS 与 MAIN、以及 538-mt1-review 讲义核实）：R5+0（及 R5-1、R5-2...）= 局部变量，R5+1 = 旧帧指针，R5+2 = 返回地址，R5+3 = **返回值**，R5+4 起 = **参数**。 **为什么返回值必须在 R5+3（紧贴参数区之下）**：被调用者返回后，调用者要用**一条** ADD R6,R6,#(nparams+1) 同时弹出参数**和**返回值（538-mt1-review 的原话是 "Pop parameters and return value (destroy the params)"）；只有返回值紧邻第一个参数之下，这一条指令才成立。 没有局部变量的 MAIN 也满足同一组偏移：ADD R6,R6,#-3 / STR R5,R6,#0（即 R5+1）/ ADD R5,R6,#-1（使 R5+1 = R6+0），于是 R5+1 旧帧指针、R5+2 返回地址、R5+3 返回值、R5+4 第一个参数，与 FIND_ABS 完全一致。 * *作用域与存储期*：参数与局部变量都是 automatic；linkage 的作用域是"这次调用的整个生命周期"，它在 RET 之前被用掉，随后随帧一起失效。 * **为什么实参自右向左压栈 (right-to-left)**：调用 f (A, B, C) 时，编译器先压 C、再压 B、最后压 A，于是 **A 落在地址最低处，也就是 R5+4**。 * *直观解释*：想象给收银员一叠订单：最上面那张必须永远是"总单"（第一个参数），后面的明细按顺序往下排。这样收银员不必知道一共几张，就能先看总单再往下翻。 * *底层机制图解*：决定性理由是 **C 允许可变参数函数 (variable-argument function)**，printf 就是典型： c printf ("%d and %f\n", i, x); /* 参数个数由格式串决定，编译器不告诉 printf */ printf 必须能"先拿到第一个参数（格式串），再按格式串里的说明符个数去取后面的参数"。若第一个参数总在固定的 R5+4，它就能顺序读到 R5+5、R5+6.....；若顺序反过来（第一个参数在最高的地址），printf 连"一共几个参数"都不知道，就无法定位起点： 压栈顺序：C（最高地址）→ B → A（最低地址 = R5+4） R5+6: C R5+5: B R5+4: A ← 第一个实参永远是 R5+4，与参数个数无关 * *作用域与存储期*：参数副本属于**被调用者**的帧；被调用者可以随意修改它们，返回时调用者把这些字全部弹掉（对可变参数函数也一

样，只是弹出长度由调用者决定）。 * **调用序列 (Call Sequence)**：调用者做的四件事 + 被调用者做的三件事。 * *直观解释*：像寄快递：先装箱（压参数）→ 下单（`JSR`）→ 取回执（读返回值）→ 清理包装（弹栈）；收件方则负责开箱、干活、贴回执。 * *底层机制图解*：`调用者： 被调用者： ① 求值实参，自右向左压栈 ① 为 linkage + 局部变量腾出空间（ADD R6,R6,#-N） ② JSR（R7 ← 下一条指令地址，PC ← 函数入口） ② 保存调用者的 R5：STR R5,R6,#k ③ 从栈顶读返回值：LDR R0,R6,#0 ③ R5 ← 局部变量底部：ADD R5,R6,#(n-1) ④ 弹出返回值与所有实参：ADD R6,R6,#(n+1) ④ 保存返回地址：STR R7,R5,#2 ⑤ 执行语句；返回值写入 R5+3 ⑥ 拆帧：LDR R7,R5,#2 / LDR R5,R5,#1 / ADD R6,R6,#(n+2) ⑦ RET（PC ← R7）` 具体指令：`LDR R0,R5,#0`（求值）→ `ADD R6,R6,#-1` + `STR R0,R6,#0`（压栈：必须先移动 `R6` 再写）→ `JSR MY_FUNC` → `LDR R0,R6,#0`（读栈顶返回值）→ `ADD R6,R6,#2`（弹掉返回值与实参）。 * *作用域与存储期*：调用序列决定了一次调用的"内存波纹"：先长高（压参数）、再长出一个完整的帧、最后全部收回，栈指针必须回到平衡位置——不平衡就会慢慢耗尽栈空间。 * **返回值与返回值槽 (Return Value Slot)**：返回值放在帧里的固定位置 `R5+3`，返回后它就是栈顶。 * *直观解释*：像交作业时把作业本放在桌子最上面——老师（调用者）一伸手就能拿到，不需要问"你放哪了"。 * *底层机制图解*：`return <expr>;` 的四步：`assembly LDR R0,R5,#0 ; ① 求值 ② STR R0,R5,#3：写进返回值槽 LDR R7,R5,#2 ; ③ 恢复 R7 与 R5（此时 R6 仍指向帧内） LDR R5,R5,#1 ADD R6,R6,#3 ; ④ 弹出局部变量与 linkage，只留下返回值 RET` 被调用者弹出 `n+2` 个字（n 个局部变量 + linkage 中除返回值外的两格）之后，`R6` 正好指向返回值槽，调用者一条 `LDR R0,R6,#0` 就读到返回值；紧接着调用者再用**一条** `ADD R6,R6,#(nparams+1)` 把返回值与所有参数一起弹掉。**返回值必须落在参数区正下方（`R5+3`），这两条指令才成立**——这正是"linkage 三格顺序不能随便换"的根本原因。 * *作用域与存储期*：返回值是**调用者帧里的一段临时空间**，由调用者负责弹出；被调用者的局部变量在 `RET` 之后已全部失效。 * **谁保存什么：caller-saved 与 callee-saved**：`R0–R3` 是 caller-saved，`R5` / `R6` / `R7` 的状态由栈帧机制维护。 * *直观解释*：会议室里的白板（`R0–R3`）谁用谁擦；会议室本身的结构（`R5` / `R6` / `R7`）由每次开会的人负责恢复原样。 * *底层机制图解*：LC-3 上的实际做法是： * `R0–R3`：**调用者保存**——调用者若在调用之后还需要这些寄存器里的值，必须自己在调用前压栈保存（编译器在寄存器不够用时才这么做）； * `R5`、`R7`：**被调用者保存**——但保存位置不是别的寄存器，而是**自己栈帧里的两个槽**（`R5+1` 与 `R5+2`），这样天然支持嵌套调用； * `R6`：由"压栈/弹栈"的算术共同维护，返回时必须回到调用者期望的位置； * `R4`：全局数据指针，通常整个程序共用，不随调用改变。 `assembly ; 如果调用者需要保留 R1，就自己压栈保护（caller-saved 的含义） ADD R6,R6,#-1 STR R1,R6,#0 JSR SOME_FUNC LDR R1,R6,#0 ; 读完返回值后恢复 R1 ADD R6,R6,#1` * *作用域与存储期*：`R0–R3` 的内容只保证"到下一次调用之前"有效；跨调用存活的数据必须放进栈帧（automatic）或全局数据区（static）。 * **编译器可以优化（所以编译出的帧与你的心理模型可能不同）**：栈帧的内部结构不是接口，编译器有充分的自由。 * *直观解释*：约定只规定了"行李怎么交接"，没规定"你在自己房间里怎么摆"；有人把衣服挂起来（寄存器），有人干脆不用箱子（省掉帧）。 * *底层机制图解*：常见的优化包括： * **把变量放进寄存器**：局部变量只在函数内使用，不必真的分配栈槽；**不保存 `R7`**：函数体内不再调用别的子程序时可以省掉 `STR R7,R5,#2`； * **完全不建立栈帧**：叶子函数 (leaf function) 可能直接算完就返回；**省略帧指针**（x86-64 在 `-O2` 下把 `RBP` 当普通寄存器，导致调试器难以还原调用栈）。 实测证据：同一份 `l08_add3.c`，`-O0` 下被调用者的参数副本位于调用者局部变量的**更低地址**，`-O2` 下 `&a < &b < &c` 与 `&total` 的相对位置整体反转—— **同一份 C 代码，帧内布局可以完全不同**；不变的是**接口层面**（实参顺序、返回值位置、寄存器角色），否则不同模块无法链接在一起。 * *作用域与存储期*：优化改变的是"变量住在哪"（寄存

器、栈槽、甚至被消除），但 C 语言层面**automatic 变量的可见性与生命周期不变**——语言语义不因优化而改变。 * **返回局部变量地址的危险 (Dangling Pointer)**：函数返回后，它的栈帧已经失效，任何指向帧内的指针都变成悬空指针 (dangling pointer)。 * *直观解释*：像把"我家冰箱第二层"这个地址告诉别人，然后你就搬走了；下一位住户往那一层放什么，你完全无法预料。 * *底层机制图解*：`c int32_t *bad (void) { int32_t local = 1234; return &local; /* 帧一拆，local 的槽就"归下一位调用者所有" */ }` 返回时 `ADD R6,R6,#n` 只是抬高了栈指针，**内存里的 1234 还在**——所以紧接着读可能"看起来正确"，但任何一次新的调用（甚至 `printf` 内部的调用）都会复用这块内存，把 1234 覆盖成别的位。 这正是"未定义行为 (undefined behavior, UB)"的典型：**结果依编译器与调用时序而定**。 * *作用域与存储期*：这就是"作用域 ≠ 存储期"的报应：变量的名字只在该函数里可见，而它的存储也在函数返回时结束了；指针却把两个边界都带了出去。

代码示例与底层机制分析

代码 (C) — 小函数与其调用者，用地址揭示帧的边界：

```c
/*
 * ECE220 Lecture 8 demo -- a small function and its caller, with the
 * addresses of every variable printed so that the frame layout is visible.
 * Build: gcc -g -std=c99 -Wall -Werror l08_add3.c -o l08_add3
 */
#include <stdint.h>
#include <stdio.h>

/*
 * Function: add3
 * Description: adds three integers
 * Parameters: a, b, c -- the values to add
 * Return Value: a + b + c
 */
static int32_t
add3 (int32_t a, int32_t b, int32_t c)
{
    int32_t total;      /* the only local variable of add3 */

    printf ("  add3 frame: &a=%p &b=%p &c=%p\n",
            (void *)&a, (void *)&b, (void *)&c);
    printf ("  add3 frame: &total=%p\n", (void *)&total);
    printf ("  add3 frame: (intptr_t)&b - (intptr_t)&c = %ld\n",
            (long)((intptr_t)&b - (intptr_t)&c));
    printf ("  add3 frame: (intptr_t)&a - (intptr_t)&b = %ld\n",
            (long)((intptr_t)&a - (intptr_t)&b));

    total = a + b + c;

    return total;
}

int
main ()
{
    int32_t x = 7;
    int32_t y = -2;
    int32_t z = 10;
    int32_t answer;

    printf ("main frame: &x=%p &y=%p &z=%p &answer=%p\n",
            (void *)&x, (void *)&y, (void *)&z, (void *)&answer);

    answer = add3 (x, y, z);

    printf ("add3(7, -2, 10) = %d\n", (int)answer);

    /* Call by value: the caller's variables are untouched. */
    printf ("main still holds x=%d y=%d z=%d\n", (int)x, (int)y, (int)z);

    return 0;
}
```

实际编译运行结果（`./l08_add3`，gcc 12.2.0 `-O0`；地址每次运行都变）：

```
main frame: &x=0x7ffc6edef64c &y=0x7ffc6edef648 &z=0x7ffc6edef644 &answer=0x7ffc6edef640
  add3 frame: &a=0x7ffc6edef61c &b=0x7ffc6edef618 &c=0x7ffc6edef614
  add3 frame: &total=0x7ffc6edef62c
  add3 frame: (intptr_t)&b - (intptr_t)&c = 4
  add3 frame: (intptr_t)&a - (intptr_t)&b = 4
add3(7, -2, 10) = 15
main still holds x=7 y=-2 z=10
```

【代码做什么？】 `main` 声明四个 `int32_t` 并打印它们的地址；调用 `add3 (x, y, z)`；`add3` 打印自己三个参数与局部变量 `total` 的地址， 并打印参数之间的地址差；计算 `a + b + c` 返回后， `main` 打印结果，并确认 `x`、`y`、`z` 没有被改动。

【底层机制透视】 * **地址差 4 说明参数是各自独立的槽**：`&b - &c = 4`、`&a - &b = 4`，正好是一个 `int32_t` 的宽度，说明 `a`、`b`、`c` 按 `a`（最低）、`b`、`c`（最高）的顺序连续摆放——**与 LC-3 上"自右向左压栈、第一个实参地址最低"完全一致**。 * **参数在被调用者的帧里**：`&a = 0x...61c` 比 `main` 的 `&x = 0x...64c` 低 48 字节，属于新长出来的那一段栈；这就是"按值传递"的物理形态——**副本在栈上**。 * **`main` 的值不变**：`add3` 改不改自己的 `a` 都无所谓， `x` 所在的槽根本没被写过。所以函数若想改变调用者的数据，只能接收一个指向它的地址。 * **优化会改变布局**：同一份代码用 `gcc -O2` 编译后， `&a < &b < &c` 与 `&total` 的相对位置整体反转（实测），提醒我们**帧的内部布局不是接口**。

【内存布局图解】

```
x86-64, -O0 实测（地址为一次真实运行的输出）：
高地址  +-----------------------------+ 0x7ffc6edef64c   ← main 的栈帧（地址更高）
        |  int32_t x = 7              |
        +-----------------------------+ 0x7ffc6edef648
        |  int32_t y = -2             |
        +-----------------------------+ 0x7ffc6edef644
        |  int32_t z = 10             |
        +-----------------------------+ 0x7ffc6edef640
        |  int32_t answer             |
        +-----------------------------+ 0x7ffc6edef62c
        |  add3 的局部变量 total      |   ← add3 的栈帧：整体在 main 之下
        +-----------------------------+ 0x7ffc6edef61c
        |  参数副本 a = 7             |   ← 最低地址 = 第一个实参
        +-----------------------------+ 0x7ffc6edef618
        |  参数副本 b = -2            |
        +-----------------------------+ 0x7ffc6edef614
        |  参数副本 c = 10            |   ← 最高地址 = 最后一个实参
低地址  +-----------------------------+
main 的 x 在 0x...64c，add3 的 a 在 0x...61c，相差 48 字节：两个完全不同的世界。
```

【与汇编的对应】（LC-3：调用 `add3 (x, y, z)` 的压栈顺序）

```
; 自右向左压栈：先 z (c)，再 y (b)，最后 x (a)
        LDR     R0,R5,#-2       ; 第三个实参 z
        ADD     R6,R6,#-1
        STR     R0,R6,#0        ; z 落在最高地址
        LDR     R0,R5,#-1       ; 第二个实参 y
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        LDR     R0,R5,#0        ; 第一个实参 x（最后压 → 地址最低）
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     ADD3
        LDR     R0,R6,#0        ; 返回值位于栈顶
        ADD     R6,R6,#4        ; 弹掉返回值 + 3 个实参
; 进入 ADD3 后：R6 指向 a；被调用者建好帧之后
;   a = R5+4、b = R5+5、c = R5+6（第一个参数永远在 R5+4）
```

代码 (C) 与手工翻译的 LC-3 汇编 — 完整的 `main` + `add3`：

待翻译的 C 代码：

```
int32_t add3 (int32_t a, int32_t b, int32_t c)
{
    int32_t total;              /* one local variable */
    total = a + b + c;
    return total;
}

int main (void)
{
    int32_t x = 7, y = -2, z = 10;
    int32_t answer;
    answer = add3 (x, y, z);
    return answer;
}
```

手工翻译的 LC-3 汇编（与课程示例 `translate.asm` 风格一致；用一个小型 LC-3 解释器实际执行过）：

```
        .ORIG   x3000
        LEA     R4,GLOBAL_DATA
        LD      R6,STACK_TOP    ; R6 <- xFE00, 栈基址
        JSR     MAIN            ; 调用 main (R7 <- 下一条指令)
        LDR     R0,R6,#0        ; 读 main 的返回值
        ADD     R6,R6,#1        ; 弹掉返回值
        HALT

;---------------------------------------------------------------
; int main (void)
;   locals: x at R5+0, y at R5-1, z at R5-2, answer at R5-3
;---------------------------------------------------------------
MAIN
        ADD     R6,R6,#-7       ; 4 个局部变量 + 3 个 linkage 字
        STR     R5,R6,#4        ; 保存调用者的帧指针 (R6+4 = R5+1)
        ADD     R5,R6,#3        ; R5 -> 局部变量底部 (x)
        STR     R7,R5,#2        ; 保存返回地址
        AND     R0,R0,#0        ; x = 7
        ADD     R0,R0,#7
        STR     R0,R5,#0
        AND     R0,R0,#0        ; y = -2
        ADD     R0,R0,#-2
        STR     R0,R5,#-1
        AND     R0,R0,#0        ; z = 10
        ADD     R0,R0,#10
        STR     R0,R5,#-2
        LDR     R0,R5,#-2       ; 自右向左压入实参: 先 z
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        LDR     R0,R5,#-1       ; 再 y
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        LDR     R0,R5,#0        ; 最后 x (第一个实参, 地址最低)
        ADD     R6,R6,#-1
        STR     R0,R6,#0
        JSR     ADD3            ; 调用
        LDR     R0,R6,#0        ; 读返回值
        ADD     R6,R6,#4        ; 弹出返回值与 3 个实参
        STR     R0,R5,#-3       ; answer = add3 (x, y, z)
        LDR     R0,R5,#-3       ; return answer
        STR     R0,R5,#3        ; 写返回值槽
        LDR     R7,R5,#2        ; 拆栈帧
        LDR     R5,R5,#1
        ADD     R6,R6,#6        ; 弹出局部变量与 linkage (留下返回值)
        RET

;---------------------------------------------------------------
; int32_t add3 (int32_t a, int32_t b, int32_t c)
;   a at R5+4, b at R5+5, c at R5+6;  local total at R5+0
;---------------------------------------------------------------
ADD3
        ADD     R6,R6,#-4       ; 1 个局部变量 + 3 个 linkage 字
        STR     R5,R6,#1        ; 保存调用者的帧指针
```

```
        ADD     R5,R6,#0        ; R5 -> 局部变量（total）
        STR     R7,R5,#2        ; 保存返回地址
        LDR     R0,R5,#4        ; R0 <- a
        LDR     R1,R5,#5        ; R1 <- b
        ADD     R0,R0,R1        ; a + b
        LDR     R1,R5,#6        ; R1 <- c
        ADD     R0,R0,R1        ; a + b + c
        STR     R0,R5,#0        ; total = a + b + c
        LDR     R0,R5,#0        ; return total
        STR     R0,R5,#3        ; 写返回值槽
        LDR     R7,R5,#2        ; 拆栈帧：恢复返回地址
        LDR     R5,R5,#1        ; 恢复调用者的帧指针
        ADD     R6,R6,#3        ; 弹出局部变量与 linkage（留下返回值）
        RET

STACK_TOP       .FILL   xFE00
GLOBAL_DATA
        .END
```

实际执行结果（用一个小型 LC-3 解释器运行上面的汇编；栈基址 `xFE00`）：

```
instructions executed: 54
R0 (return value of main) = 15
R6 after returning to the caller = xFE00      ← 栈指针回到初始位置：完全平衡
```

> ⚙️ **独立复核**：这份清单后来又被**另一套独立实现**的 LC-3 汇编器 + 模拟器（`ece220_data/lc3sim.py`，与本讲使用的解释器不是同一份代码）跑了一遍，结果逐项吻合： **54 条指令**、**`R0 = x000F` (= 15)**、**`R6 = xFE00`**（栈完全平衡）、 参数按约定落在 `R5+4` / `R5+5` / `R5+6`。两套独立实现给出同一个指令计数， 说明这份手写 C→LC-3 翻译在寄存器与栈的使用上是自洽的。

执行结束时留在栈上的字（地址: 十进制值）：

```
xFDF2:   15        ← add3 的局部变量 total（R5_add3+0）
xFDF3:  65020      ← add3 保存的调用者帧指针 = xFDFC = main 的 R5（R5+1）
xFDF4:  12317      ← add3 的返回地址 = x301D（JSR ADD3 的下一条指令）（R5+2）
xFDF5:   15        ← add3 的返回值槽（R5+3）——RET 后它就是栈顶
xFDF6:    7        ← 实参 a = x（第一个实参，地址最低）（R5+4）
xFDF7:  65534      ← 实参 b = y = -2（用 16 位 2 的补码表示）
xFDF8:   10        ← 实参 c = z（最后一个实参，地址最高）（R5+6）
xFDF9:   15        ← main 的局部变量 answer（R5_main-3）
xFDFA:   10        ← main 的局部变量 z（R5_main-2）
xFDFB:  65534      ← main 的局部变量 y = -2（R5_main-1）
xFDFC:    7        ← main 的局部变量 x（R5_main+0）= xFDFC 正是 main 的 R5
xFDFD:    0        ← main 保存的"上一个帧指针"（最外层没有调用者，置 0）（R5+1）
xFDFE:  12291      ← main 的返回地址 = x3003（JSR MAIN 的下一条指令）（R5+2）
xFDFF:   15        ← main 的返回值槽（R5+3）；最外层用 LDR R0,R6,#0 读它
```

【代码做什么？】 最外层先设置 `R4`（全局数据指针）与 `R6 = xFE00`（栈基址），`JSR MAIN` 进入 C 的 `main`； `MAIN` 建立 7 个字的帧（4 个局部变量 + 3 个 linkage），给 `x`、`y`、`z` 赋值，**自右向左**压入三个

实参，`JSR ADD3`； `ADD3` 建立 4 个字的帧（1 个局部变量 + 3 个 linkage），从 `R5+4`、`R5+5`、`R5+6` 取三个参数相加，写入局部变量 `total`， 把结果写进返回值槽（`R5+3`），拆帧后 `RET`； `MAIN` 从栈顶取回 15 存进 `answer`，最后按同样的方式返回给自己的调用者。

【底层机制透视】 * **参数偏移固定**：`ADD3` 里 `a` 在 `R5+4`、`b` 在 `R5+5`、`c` 在 `R5+6`，始终不变——因为 `ADD3` 只有 1 个局部变量，`R5 = R6 + 0`，被调用时的 `R6`（指向 `a`）正好是 `R5+4`。 n 个局部变量时建帧要写 `ADD R6,R6,#-(n+3)`、`STR R5,R6,#n`、`ADD R5,R6,#(n-1)`：**增量随局部变量个数变化，但 linkage 相对 `R5` 的偏移（+1/+2/+3）永远不变**，参数一律从 `R5+4` 起。 * **linkage 是"回家的路"**：`R5+1` 存调用者的 `R5`、`R5+2` 存返回地址 `R7`；递归或深层调用时每个帧都存着自己那一层的这两个值，于是可以逐层 `LDR R5,R5,#1` 走回去（调试器打印调用栈 backtrace 的原理）。 * **栈指针必须平衡**：`main` 的帧 7 个字 + `add3` 的帧 4 个字 + 3 个实参，最终 `R6` 回到 `xFE00`，说明"压多少、弹多少"完全对上；**不平衡是隐蔽 bug 的常见来源**。 * **实测的地址关系**：`xFDF6 < xFDF7 < xFDF8` 对应 `a < b < c`，证明实参自右向左压栈；`xFDF3 = xFDFC` 说明 `add3` 保存的正是 `main` 的帧指针。 * **与 C 语言级的对应**：`total = a + b + c;` 被翻译成 6 条指令（2 条 `LDR` + 2 条 `ADD` + 1 条 `LDR` + 1 条 `STR`）， 完全没有"优化"；真实编译器会做出与这里不同的选择（见"编译器可以优化"一节）。

【内存布局图解】

```
执行到 ADD3 内部时（R5_main = xFDFC，R5_add3 = xFDF2），栈的实际内容：
高地址  +-------------------------------+ xFE00  ← 栈基址（初始 R6）
        +-------------------------------+ xFDFF    （以下每格 = 1 个 LC-3 字）
        |  main 的返回值槽          = 15 | R5_main+3
        +-------------------------------+ xFDFE
        |  main 的返回地址      = x3003  | R5_main+2   ┐
        +-------------------------------+ xFDFD         │ main 的 linkage
        |  main 保存的上个帧指针    = 0  | R5_main+1   ┘  ← xFDFC = main 的 R5
        |  main 的局部变量 x         = 7 | R5_main+0  ← R5_main
        +-------------------------------+ xFDFB
        |  main 的局部变量 y       = -2  | R5_main-1
        |  main 的局部变量 z        = 10 | R5_main-2
        |  main 的局部变量 answer   = 15 | R5_main-3
        +-------------------------------+ xFDF8
        |  实参 c = z                = 10 | R5_add3+6   ┐
        +-------------------------------+ xFDF7          │ 参数区：第一个实参
        |  实参 b = y               = -2  | R5_add3+5    │ 地址最低（R5+4）
        +-------------------------------+ xFDF6          ┘
        |  实参 a = x                = 7  | R5_add3+4
        +-------------------------------+ xFDF5
        |  add3 的返回值槽          = 15 | R5_add3+3
        +-------------------------------+ xFDF4
        |  add3 的返回地址      = x301D  | R5_add3+2   ┐
        +-------------------------------+ xFDF3         │ add3 的 linkage
        |  add3 保存的帧指针 = xFDFC     | R5_add3+1   ┘
        +-------------------------------+ xFDF2
        |  add3 的局部变量 total    = 15 | R5_add3+0  ← R5_add3（也是 R6）
低地址  +-------------------------------+
```

【与汇编的对应】 上面整段汇编就是本例的"C → 机器码"翻译，三条关键规律可以单独记住：

```
; ① 访问局部变量：R5 + 编译期算好的偏移
        LDR     R0,R5,#0        ; total（局部变量在 R5+0 及以下）
        STR     R0,R5,#0
; ② 访问参数：R5 + (3 + 参数序号)
        LDR     R0,R5,#4        ; 第一个参数 a
        LDR     R1,R5,#6        ; 第三个参数 c
; ③ 返回值：永远写 R5+3，返回前把 R6 抬到 R5+3
        STR     R0,R5,#3        ; 返回值槽
        LDR     R7,R5,#2        ; 拆 linkage：R7 ← 返回地址
        LDR     R5,R5,#1        ;              R5 ← 调用者的帧指针
        ADD     R6,R6,#3        ;              R6 指向返回值槽
        RET
```

代码 (C) — 危险示例：返回/保存指向局部变量的地址（仅供演示，请勿模仿）：

```
/*
 * DANGER: demonstration only, do not imitate.
 *
 * gcc 12 在 ECE 220 的警告级别下就能抓住这两种写法：
 *   -Werror=return-local-addr        （直接返回局部变量地址）
 *   -Werror=dangling-pointer=        （把局部变量地址存进全局指针）
 * 所以下面这两份程序必须去掉 -Werror 才能编译、运行：
 *   gcc -g -std=c99 -Wall l08_dangling2.c -o l08_dangling2
 */
#include <stdint.h>
#include <stdio.h>

static int32_t *dangling;          /* points into a frame that no longer exists */
static volatile int32_t sink;      /* keeps the "reuser" from being optimized away */

static void
keep_a_local (void)
{
    int32_t local = 1234;

    dangling = &local;             /* the slot dies when keep_a_local returns */
}

static void
reuse_the_stack (void)
{
    volatile int32_t junk[8];
    int32_t i;

    for (i = 0; 8 > i; i++) {
        junk[i] = 0x5A5A;
        sink = junk[i];
    }
}

int
main ()
{
    keep_a_local ();
    printf ("right after the call:  *dangling = %d\n", (int)*dangling);

    reuse_the_stack ();
    printf ("after the second call: *dangling = %d\n", (int)*dangling);

    return 0;
}
```

实际编译与运行结果：

```
$ gcc -g -std=c99 -Wall l08_dangling2.c -o l08_dangling2
l08_dangling2.c: In function ‘keep_a_local’:
l08_dangling2.c:21:14: warning: storing the address of local variable ‘local’ in ‘dangling’
$ ./l08_dangling2
right after the call:  *dangling = 1234
after the second call: *dangling = 8         (未定义行为：结果依编译器/时序而定)

$ gcc -g -std=c99 -Wall -Werror l08_dangling2.c -o l08_dangling2
cc1: all warnings being treated as errors        ← ECE 220 的编译选项会直接拦下它

$ gcc -g -std=c99 -Wall -Werror dangling_min.c -o dangling_min     # 直接 return &local
dangling_min.c:3:56: error: function returns address of local variable [-Werror=return-local
```

【代码做什么？】 `keep_a_local` 把一个**局部变量**的地址存进全局指针 `dangling` 后返回；`main` 立刻读该地址，得到 1234； 接着调用 `reuse_the_stack`（它在自己的帧里写 8 个 `0x5A5A`），再读同一个地址，值已经被覆盖成 8。整个过程没有任何编译错误（只去掉 `-Werror` 时）。

【底层机制透视】 * **"帧被弹掉"不等于"内存被清空"**：`RET` 只把 `R6` 抬高，1234 仍然躺在原来的地址上，所以第一次读"看起来是对的"——这是陷阱中最危险的部分：**程序可能在测试时一直正常，直到某次调用把它踩坏**。 * **下一位调用者会复用这段内存**：`reuse_the_stack` 的 `junk[8]` 正好覆盖了 `local` 的槽，于是读出来变成 8（`0x5A5A` 循环计数器的残留）。缓存、寄存器分配、优化等级都会改变实际结果。 * **编译器能抓住它**：`-Wreturn-local-addr` 与 `-Wdangling-pointer=` 都在 `-Wall` 里，配合 ECE 220 的 `-Werror` 会**直接变成编译错误**——这是把 UB 挡在门外的第一道防线；用 `-fsanitize=address` 还能在运行时精确报出"访问已释放栈内存"。 * **正确的替代做法**：要么让调用者**传入**存放结果的地址（`void f (int32_t *out)`），要么把结果作为**返回值**交回（`return local;` 返回的是值的拷贝），要么在需要长期存活时用动态分配（后面的讲次）——三者都不会把"已失效的帧"带出去。

【内存布局图解】

```
keep_a_local 返回前后，同一段栈内存的命运：
高地址  +---------------------------+
        |  main / 其它帧            |
        +---------------------------+ ← keep_a_local 调用期间的帧
        |  int32_t local = 1234     | ← dangling 指向这里（0x...6a0 之类）
        +---------------------------+ ← keep_a_local 返回后 R6 抬高，这段内存"逻辑上无效"
        |                           |   但 1234 的位还在
        +---------------------------+
        |                           | ← reuse_the_stack 的 junk[8] 恰好覆盖同一段
        |  0x5A5A 0x5A5A ...        |    （写 8 个 int32_t，其中最后一个写进原 local 的槽）
低地址  +---------------------------+
读 *dangling：第一次 1234（残留），第二次 8（被复用后的位）——两次都能"读到数"，却都不是被承诺的语义。
```

【与汇编的对应】（LC-3：`keep_a_local` 返回时做了什么，以及为什么地址会失效）

```
KEEP_A_LOCAL
        ADD     R6,R6,#-4       ; 1 个局部变量（local）+ 3 个 linkage 字
        STR     R5,R6,#1
        ADD     R5,R6,#0        ; R5 -> local
        STR     R7,R5,#2
        AND     R0,R0,#0        ; local = 1234
        LD      R1,C1234        ;  （1234 超过 ADD 的 imm5 范围，从字面量池取）
        ADD     R0,R0,R1
        STR     R0,R5,#0
        STR     R5,R4,#2        ; dangling = &local（全局数据区 R4+2）
        LDR     R7,R5,#2        ; 拆帧：R7、R5 恢复，R6 抬高
        LDR     R5,R5,#1
        ADD     R6,R6,#3
        RET
        ; 注意：R6 抬高之后，[旧 R5+0] 这个地址仍然写着 1234，
        ; 但它已经不在"活的栈"里了；下一个被调用者会用 ADD R6,R6,#-n 把同样的地址划进自己的帧。
C1234   .FILL   #1234
```

常见错误与调试技巧

- **忘记保存/恢复 `R7`（手写汇编时）**：函数里再调用别的子程序后，`RET` 就跳不回来了。**调试**：`gdb` 里 `info registers r7`（LC-3 用 `lc3sim` 的 `print R7`）看返回值是否被覆盖；在函数入口/出口分别打印 `R7`；规则是"只要函数体内有 `JSR` / `JSRR` /陷阱指令，建帧时必须 `STR R7,R5,#2`"。
- **栈指针不平衡**：压了 3 个实参却只 `ADD R6,R6,#2`，栈会逐次下沉，最终撞上堆或越界。**调试**：在函数入口与 `RET` 前各打印一次 `R6`（或用 `gdb` 的 `p $rsp`），两者必须相等；x86-64 上还可以用 `-fsanitize=address` 直接报出栈越界。 与之相关，以为"编译出的帧长什么样"是固定的**
- **返回或保存局部变量地址（悬空指针）**：函数返回后指针仍被使用，读到的值随调用时序变化。**调试**：`gcc -g -std=c99 -Wall -Werror` 会在编译期报 `-Wreturn-local-addr` / `-Wdangling-pointer=`；运行时用 `gcc -fsanitize=address -g` 或 `valgrind --track-origins=yes ./prog` 精确指出"使用了已失效的栈内存"；`gdb` 里 `bt` 看当前帧是否还有效。
- **局部变量未初始化，误以为"新帧里是 0"**：帧只是把 `R6` 挪了位置，里面的位是上一次调用留下的垃圾。**调试**：`gcc -O2 -Wmaybe-uninitialized`、`valgrind` 的 `Conditional jump or move depends on uninitialised value(s)`；养成声明即初始化的习惯。
- **传结构与数组时想当然"传的是本体"**：数组会退化为指针（传地址），结构体按值传递则**整个拷进参数区**，帧会突然变大。**调试**：打印 `sizeof` 与 `&param`；用 `gcc -S` 观察建帧语句（`ADD R6,R6,#-N` 里的 N 会明显增大）。
- **在 `printf` 里混用错误的类型/个数导致读到"栈上的位"**：可变参数函数靠格式串决定读几个参数，写错就会读到别的槽。**调试**：`gcc -std=c99 -Wall -Werror` 的 `-Wformat` 会检查参数与说明符；`gdb` 在该 `printf` 前中断后用 `x/8xw $rsp`（LC-3 用 `x/8xw R6`）查看即将被读走的那些字。

关键要点

- **一次调用 = 一个栈帧**：帧里依次是参数副本、返回值槽、返回地址、上一个帧指针、局部变量；`R6` 指向栈顶，`R5` 指向局部变量底部，`R5+4` 起是第一个参数。

- **实参自右向左压栈**，因此**第一个实参永远位于 `R5+4`**；这是 `printf` 这类可变参数函数能"先拿到格式串、再依次取参"的前提，也是所有调用者/被调用者能对上的唯一方式。
- **调用与返回是两个对称的序列**：压参 → `JSR` → 读返回值 → 一条 `ADD R6,R6,#(nparams+1)` 弹栈；建帧（保存 `R5` / `R7`、分配局部变量）→ 执行 → 写返回值槽（`R5+3`）→ 拆帧 → `RET`。**栈指针必须平衡。**
- **接口固定、内部自由**：寄存器分工与参数顺序是跨模块的契约，而"变量放寄存器还是栈槽、要不要帧指针"完全由编译器决定——编译出来的帧可能与你心里的模型很不一样。 同样地，帧一失效，指向它的指针就是悬空指针**：`return` 只是抬高栈指针，旧内存会被下一位调用者复用；`-Werror` 与 `-fsanitize=address` 是发现这类未定义行为最有效的工具。

思考题（带答案）

1. 一个函数有 **3 个参数、2 个局部变量**。它建立栈帧时 `R6` 要减多少？参数分别对应哪个偏移？返回到调用者之前 `R6` 又要加多少？ **答案**：帧 = 2 个局部变量 + 3 个 linkage 字 = 5 个字，所以 `ADD R6,R6,#-5`（`538-mt1-review` 的 `FOO` 正是这个例子，注释写着 "three for linkage, two for local vars"）。 接着 `ADD R5,R6,#1` 让 `R5` 指向两个局部变量中地址较高的那个，于是 `STR R5,R6,#2` 恰好把调用者的帧指针存到 `R5+1`，再 `STR R7,R5,#2` 存返回地址；参数依次在 `R5+4`、`R5+5`、`R5+6`，返回值槽是 `R5+3`。返回前 `ADD R6,R6,#4`（弹出 2 个局部变量与 linkage 中的两格）使 `R6` 正好指向返回值槽；调用者读完返回值后再用**一条** `ADD R6,R6,#(3+1)` 同时弹出返回值与三个参数。 三个参数依次在 `R5+4`、`R5+5`、`R5+6`。返回前 `ADD R6,R6,#4`（弹出 2 个局部变量 + linkage 中的两格，留下返回值槽），此时 `R6` 正好指向返回值。
2. 为什么 `printf` 要求实参自右向左压栈？如果改成从左往右，`printf` 会遇到什么问题？ **答案**：`printf` 的参数个数由**第一个参数（格式串）**决定，编译器不会告诉它一共传了几个。自右向左压栈让第一个实参固定落在 `R5+4`，于是 `printf` 可以先取格式串、再按其中的说明符个数依次读 `R5+5`、`R5+6`……。若从左往右压栈，第一个实参会在**最高的地址**上，而 `printf` 既不知道总共有几个参数，也就无法算出"第一个参数在哪里"——它连起点都找不到。
3. 下面的函数为什么危险？在实际系统上它可能"看起来正常"，为什么？给出两种正确的替代写法。 `c int32_t *make (void) { int32_t local = 100; return &local; }` **答案**：`local` 是 automatic 变量，函数返回时它的栈帧被拆掉，返回的指针成为悬空指针。之所以"看起来正常"，是因为 `RET`/拆帧只是把栈指针抬高，**并没有清空内存**，紧接着读往往还能读到 100；但任何一次新的调用都会复用这段内存并把它覆盖，于是程序会在毫无规律的时刻给出错误结果（还可能被编译器直接优化掉，见 `-Wreturn-local-addr`）。 正确写法：① 让调用者传入存放结果的地址——`void make (int32_t *out) { *out = 100; }`；② 直接返回值本身——`int32_t make (void) { return 100; }`（返回的是值的拷贝，与帧无关）；若结果必须长期存活，则使用动态分配。

Lecture 9: 指针 (Pointers)

概述

本讲解决的问题是：C 程序如何通过"值的存放位置"而不是"值本身"来操作数据——这是"修改调用者的变量""表示字符串" "表示结构化数据"三类需求的共同基础。为此 C 引入了指针类型 `X*`、解引用运算符 (dereference operator) `*` 与取地址运算符 (address operator) `&`；在机器层面它们只是"把一个地址装进寄存器或内存，再用它做一次内存访问"。 指针是 ECE 220 从 LC-3 汇编走向 C 的枢纽：上一讲的栈帧与调用约定解释了参数为何是**值传递 (call by value)**， 本讲说明要改调用者的变量就必须传地址，而下一讲会看到数组名本身就是地址，于是"数组"与"指针"被缝成同一件事。

核心概念与底层机制图解

- **指针 (Pointer) 就是内存地址**：指针是一个变量，它的值是某个内存地址，程序用这个值去"指名"另一块存储。
  - *直观解释*：指针像**门牌号**，不是房子本身。把门牌号抄给朋友（值传递），朋友去的是同一栋房子， 但他手里那张纸并不是房子。
  - *底层机制图解*：`*p` 在机器上就是"把 `p` 的值装入地址寄存器，再发起一次访存"。LC-3 里只有两条指令： `LDR R2,R1,#0` 读出 `*p` （R1 装 `p`）， `STR R2,R1,#0` 写入 `*p`。 64 位机器（EWS 实验室机器）上一个指针占 8 字节，LC-3 上占 1 个字（16 位地址）。 `C: int32_t value = 42; int32_t *iptr = &value; 机器: M[&value] ← 42 M[&iptr] ← &value LDR R1,R5,#-1 ; R1 ← iptr（一个地址） LDR R2,R1,#0 ; R2 ← M[R1] = 42 ← 这次间接访问就是"指针"`
  - *作用域与存储期*：指针变量本身与普通变量一样（函数内 automatic、加 `static` 则 static)， 但**被指向对象有独立存储期**：字符串常量活到程序结束，局部数组随栈帧消失（于是产生悬垂指针）。
- **指针类型 `X*` 从右往左读 (read pointer types right to left)**： `int*` 是 "pointer to int"， `char**` 是 "pointer to pointer to char"。
  - *直观解释*：英文里重心在最后被修饰的名词： `a pointer to a pointer to char`，真正存放的是 `char`。
  - *底层机制图解*：类型决定"解引用取几个字节"。同一个地址 `p`， `*(char*)p` 取 1 字节， `*(int32_t*)p` 取 4 字节； 编译器为 `int32_t*` 生成的 `p + 1` 是"地址 + 4"，为 `char*` 生成的是"地址 + 1"。 类型信息**只存在于编译期**，运行时的内存里没有类型，只有位。
  - *作用域与存储期*：类型属于声明，不占运行时空间； `sizeof (int32_t*)` 与 `sizeof (char*)` 都是 8。
- **声明指针只为指针分配空间 (declaring a pointer only makes space for the pointer)**： `int32_t* iptr;` 只创建了一个"能装地址"的变量，**不创建**被指向的对象。
  - *直观解释*：买了信封不等于买了房子；信封上没写门牌号（未初始化）时按它去找房子必然出乱子。
  - *底层机制图解*： `iptr（8 字节，内容未定义）` 有存储，而 `??? 被指向的对象` **不存在**，必须另行声明或 `malloc`。

- *作用域与存储期*：`iptr` 是 automatic；它指向的对象可能是 static（字符串常量）、automatic（别的局部变量） 或 allocated（`malloc`），三者的生命周期互不相干。

- **解引用 `*` 与取地址 `&`**：二者都是**一元**运算符，对可取地址的对象互为逆运算。
  - *直观解释*：`&` 是"问门牌号"，`*` 是"按门牌号上门取东西"。
  - *底层机制图解*：`&x` **不产生访存**，编译期就算出地址（栈帧偏移、全局符号，或 `LEA`）；`*p` 才产生真实访存。这解释了 `scanf ("%d", &value)` 为什么必须写 `&`：被调用者需要地址才能写回来。
  - *作用域与存储期*：`&` 只作用于有存储位置的对象 (lvalue)；函数返回后，其局部变量的地址即失效。

- **陷阱：`*` 绑定到变量而不是类型（`int *A, B;`）**：声明符里的 `*` 属于**被声明的变量**，不属于类型关键字。
  - *直观解释*：`int *A, B;` 读作"`*A` 是 int，`B` 是 int"，所以 `A` 是 `int*`，`B` 只是 `int`。
  - *底层机制图解*：`int *A, B;` 中 `sizeof A` 为 8、`sizeof B` 为 4；写成 `int *A, *B;` 则两者都是 8。 编译器不会为此报错，只会静默地少一层间接——这是最快的自检手段。
  - *作用域与存储期*：两个变量的作用域与存储期完全相同，差异只在类型。

- **`char*` 与字符串常量 (string constants)**：`char* cptr = "My favorite string";` 中字符串是**常量**，由编译器放在全局数据区 (global data area)；`cptr` 只指向它的第一个字符。
  - *直观解释*：`cptr` 是写着地址的便条，字符串是印刷好的标语牌；便条可以换，标语牌不能涂改。
  - *底层机制图解*：两处存储、两种存储期：`全局数据区（static，只读）0x402008 处： 'M' 'y' ' ' 'f' ... '\0' ← cptr 的值指向这里 栈（automatic） ： cptr（8 字节）= 0x402008 ← &cptr 就是这 8 字节的地址`
  - *作用域与存储期*：字符串常量 static，`cptr` automatic；因此**返回指向局部字符数组的指针一定是 bug**， 而返回指向字符串常量的指针是安全的。

- **指向指针的指针（`char**`）与 LDI/STI 类比**：把"指针的地址"也存起来，就是两级间接。
  - *直观解释*：门牌号本身被写在另一张纸条上。幻灯片的玩笑很准确：指针的指针到处有用； 指针的指针的指针是考查学生懂不懂指针的好工具，此外没用。
  - *底层机制图解*：LC-3 的间接寻址 `LDI` / `STI` 正是硬件版的 `**`（两次访存）：`LDR R1,R5,#-2 ; R1 ← cptr_ptr LDR R1,R1,#0 ; R1 ← *cptr_ptr = cptr LDR R2,R1,#0 ; R2 ← **cptr_ptr = 'M' LDI R2,CPTR ; 若 cptr 在全局数据区，硬件一次完成两次访存`
  - *作用域与存储期*：`cptr_ptr`（automatic）、`cptr`（automatic）、被指向的字符数组（static） 是**三种不同的存储期**，这是理解双指针的关键。

- **NULL 与空指针 (null pointer)**：NULL 是"不指向任何对象"的特殊指针值，位模式全 0。
  - *直观解释*：门牌号那一栏写着"无"——不是随便一个号，而是一个**可检测**的"无"。
  - *底层机制图解*：没有 NULL，函数就无法用返回值表示"没找到"，因为**几乎任何位模式都可能是合法地址**； 全 0 的好处是可以直接参与判断：`if (NULL != p)` 编译成 `LDR` + `BRz`。 别混淆四个都"像 0"的东西：`NUL` 是 ASCII 字符 `'\0'`，`NULL` 是**指针**值，`null` 只是英文单词， `0` 是**数值**。幻灯片还提醒：在很多微控制器上解引用 NULL **不会**崩溃。

- *作用域与存储期*：NULL 来自 `<stdio.h>` / `<stdlib.h>` 的宏；把 `free` 后的指针赋为 NULL， 能让后续误用立刻暴露，而不是静默破坏堆。

- **指针是让函数修改调用者变量的手段**：C 用**值传递**，形参是实参的副本。
  - *直观解释*：把门牌号抄一份给被调用者：他换不掉你的纸条（`w = ...` 无效）， 却能改房子里面的东西（`*w = ...` 有效）。
  - *底层机制图解*：幻灯片 `string_equal` 里 `s1++`、`s2++` 只改副本，调用者的 `w`、`x` 不变； 而 `*s1 = ...` 会真的改写调用者能看到的内存。要改**调用者的指针变量本身**，必须传 `&pointer`（形参 `T**`）： `调用者栈帧 被调用者栈帧 +--------------+ +--------------------+ | x = 3 |◄─&x─┐| a = &x （8 字节）|──► 指向 x | y = 8 |◄─&y─┼| b = &y （8 字节）|──► 指向 y +--------------+ └+-----------------+ *a = *b 改的是调用者的变量，不是形参 a、b 自身。`
  - *作用域与存储期*：形参 `a`、`b` 随被调用者栈帧销毁，被指向的 `x`、`y` 属于调用者且活得更久， 所以"写回"合法；反之返回指向自身局部变量的指针就是错误。

- **`&` 不能作用于临时值**：`&(value + 1)` 必然是编译错误。
  - *直观解释*："值 43"这种**中间结果**没有被存放在任何地方，自然没有门牌号。
  - *底层机制图解*：`value + 1` 的结果可能只存在于寄存器里，甚至在编译期被折叠成常量； `&` 要求操作数是 lvalue。同理 `&&cptr`（对 `&cptr` 再取地址）也是错误，但 `*(*(&cptr))` 合法且等于 `*cptr`。
  - *作用域与存储期*：这是 C 存储模型的一部分——**只有具有存储期的对象才有地址**。

内存布局总图：

```
高地址  +------------------------------+  栈 (automatic)
        | cptr_ptr  (8 字节) → &cptr    |  ← &cptr 合法
        | cptr (8) → 0x402008 / value (4) = 42 / iptr (8) → &value
        +------------------------------+
        |        ... 空闲 ...           |  堆 (allocated), malloc 从这里向上要空间
低地址  +------------------------------+  全局数据区 (static): "My favorite string\0" (只读, cpt
                                             代码 (text) 在更低地址
```

代码示例与底层机制分析

示例 1：指针的读写、类型大小与"重新指向"

代码 (C)（`/tmp/ece220_ptr/01_pointer_basics.c`，用 `gcc -g -std=c99 -Wall -Werror 01_pointer_basics.c -o 01_pointer_basics` 实测）：

```
#include <stdint.h>
#include <stdio.h>

int
main (void)
{
    int32_t  value = 42;
    int32_t  other = 7;
    int32_t* iptr = &value;
    int32_t* jptr = &other;

    printf ("value = %d\n", value);
    printf ("*iptr = %d\n", *iptr);
    printf ("iptr == &value -> %d\n", iptr == &value);
    printf ("&iptr = %p (address of the POINTER variable)\n", (void*) &iptr);

    printf ("sizeof (int32_t) = %d, sizeof (int32_t*) = %d\n",
            (int) sizeof (int32_t), (int) sizeof (int32_t*));
    printf ("sizeof (value) = %d, sizeof (iptr) = %d\n",
            (int) sizeof value, (int) sizeof iptr);

    *iptr = 100;
    printf ("after *iptr = 100: value = %d, *iptr = %d\n", value, *iptr);

    iptr = jptr;
    printf ("after iptr = jptr: *iptr = %d, value is still %d\n",
            *iptr, value);

    return 0;
}
```

实际输出：

```
value = 42
*iptr = 42
iptr == &value -> 1
&iptr = 0x7ffd7cfb5238 (address of the POINTER variable)
sizeof (int32_t) = 4, sizeof (int32_t*) = 8
sizeof (value) = 4, sizeof (iptr) = 8
after *iptr = 100: value = 100, *iptr = 100
after iptr = jptr: *iptr = 7, value is still 100
```

【代码做什么？】 1. 栈上分配 4 字节放 `value = 42`，另 4 字节放 `other = 7`。 2. `iptr`、`jptr` 各占 8 字节，分别写入 `&value`、`&other`。 3. `*iptr` 打印 42（解引用产生一次访存）；`&iptr` 打印**指针变量自己**在栈上的地址，与 `iptr` 的内容不同。 4. `*iptr = 100` 通过指针写入，`value` 变成 100——"修改外层变量"的最小形态。 5. `iptr = jptr` 只改指针自己的 8 字节；`value` 仍为 100，说明**重新指向不搬动任何数据**。

【底层机制透视】 `sizeof (int32_t) = 4` 而 `sizeof (int32_t*) = 8`，说明"指针的存储"与"被指向对象的存储"是两件独立的事。 `iptr == &value` 为 1，因为 `&value` 在编译期就是"R5（帧指针）+ 固定偏移"，

运行期与 `iptr` 中的位模式逐位相同。 `iptr = jptr` 后 `*iptr` 为 7，正是幻灯片 `string_equal` 中 `s1++`、`s2++` 不影响调用者的同一机制：被复制的只是地址这个值。

【内存布局图解】（地址为示意值）

```
栈
0x7ffd..e4  +----------------------+  value = 100（被 *iptr 改写）
            |  42 → 100            |
0x7ffd..e8  +----------------------+
            |  iptr = 0x7ffd..e4   |──────┐  解引用走这条箭头，读/写 4 字节
0x7ffd..f0  +----------------------+      ↓
            |  jptr = 0x7ffd..e0   |──────→ other = 7
0x7ffd..f8  +----------------------+
            iptr = jptr 之后 iptr 的内容变成 0x7ffd..e0，*iptr 读出 7。
            注意 &iptr = 0x7ffd..e8（指针变量住哪）≠ iptr = 0x7ffd..e4（它指向哪）。
```

【与汇编的对应】（LC-3；局部变量在 `R5` 帧指针下方，`R6` 为栈指针）

```
; ---- int32_t value = 42;  int32_t other = 7; ----
        AND  R0,R0,#0
        ADD  R0,R0,#15
        ADD  R0,R0,#15
        ADD  R0,R0,#12          ; R0 = 42
        STR  R0,R5,#0           ; value  (R5+0)
        AND  R0,R0,#0
        ADD  R0,R0,#7
        STR  R0,R5,#-1          ; other  (R5-1)

; ---- int32_t *iptr = &value;  int32_t *jptr = &other; ----
        ADD  R1,R5,#0           ; R1 = &value（栈上取地址用 R5+offset）
        STR  R1,R5,#-2          ; iptr
        ADD  R2,R5,#-1          ; R2 = &other
        STR  R2,R5,#-3          ; jptr

; ---- *iptr 读 / 写 ----
        LDR  R1,R5,#-2          ; R1 ← iptr（一个地址）
        LDR  R2,R1,#0           ; R2 ← M[R1] = value = 42  ← 解引用 = 一次 LDR
        ; R3 ← 100（由若干 ADD 构造）
        STR  R3,R1,#0           ; M[iptr] ← 100，改的是 value

; ---- iptr = jptr; ----（只动 8 字节的指针副本）
        LDR  R2,R5,#-3
        STR  R2,R5,#-2

; 全局/静态对象用 LEA Rd,LABEL 取地址；栈上局部变量没有汇编期标号，只能 ADD Rd,R5,#offset 后再 LDR/STR
```

示例 2：`char*`、字符串常量的位置与 `char**` 的两级间接

代码 (C)（`/tmp/ece220_ptr/02_string_and_pointer_to_pointer.c`）：

```
#include <stdio.h>

int
main (void)
{
    char*  cptr = "My favorite string";
    char** cptr_ptr = &cptr;

    printf ("*cptr = %c\n", *cptr);
    printf ("cptr = %p -> \"%s\"\n", (void*) cptr, cptr);
    printf ("&cptr = %p (where the pointer variable lives)\n", (void*) &cptr);
    printf ("cptr + 3 = \"%s\"\n", cptr + 3);
    printf ("*(cptr + 3) = %c\n", *(cptr + 3));

    printf ("**cptr_ptr = %c\n", **cptr_ptr);
    printf ("*cptr_ptr == cptr -> %d\n", *cptr_ptr == cptr);
    printf ("*(*(&cptr)) = %c\n", *(*(&cptr)));

    printf ("sizeof (cptr) = %d, sizeof (cptr_ptr) = %d\n",
            (int) sizeof cptr, (int) sizeof cptr_ptr);

    return 0;
}
```

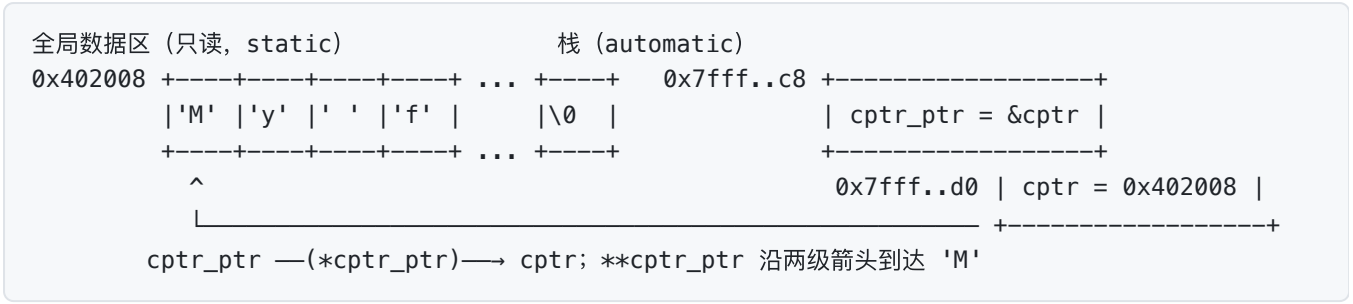
实际输出：

```
*cptr = M
cptr = 0x402008 -> "My favorite string"
&cptr = 0x7fffdc4790d0 (where the pointer variable lives)
cptr + 3 = "favorite string"
*(cptr + 3) = f
**cptr_ptr = M
*cptr_ptr == cptr -> 1
*(*(&cptr)) = M
sizeof (cptr) = 8, sizeof (cptr_ptr) = 8
```

【代码做什么？】 1. 编译器把 `"My favorite string"` 放进全局数据区并取得地址（本次运行是 `0x402008`），写进局部变量 `cptr`。 2. `*cptr` 读出 `'M'`；`cptr + 3` 前进 3 个**字符**，得到 `"favorite string"`。 3. `cptr_ptr` 存放 `&cptr`；`**cptr_ptr` 做两次解引用得到 `'M'`，而 `*cptr_ptr` 恰好等于 `cptr`。

【底层机制透视】 `&cptr`（栈地址）与 `cptr`（静态数据地址）处于完全不同的地址区域，这就是"两种存储期"的直接证据： 函数返回后 `cptr` 消失而字符串仍在。`%p` 要求实参为 `void*`，必须显式转换（`-Wall -Werror` 的硬性要求）。 指针算术按元素大小缩放：`char*` 加 3 是加 3 字节，若换成 `int32_t*` 加 3 就是加 12 字节。

【内存布局图解】



【与汇编的对应】（`LEA` 与 `LDI` 的用法）

```
; ---- char* cptr = "My favorite string"; ----
        LEA  R0,STR_FAV        ; R0 = 字符串常量地址（标号汇编期已知 → LEA）
        STR  R0,R5,#-1         ; cptr
; ---- *cptr ----（一次间接：LDR）
        LDR  R1,R5,#-1         ; R1 ← cptr
        LDR  R2,R1,#0          ; R2 ← 'M'
; ---- char** cptr_ptr = &cptr; 然后 **cptr_ptr ----
        ADD  R1,R5,#-1         ; R1 = &cptr（栈上取地址用 R5+offset）
        STR  R1,R5,#-2         ; cptr_ptr
        LDR  R1,R5,#-2         ; R1 ← cptr_ptr
        LDR  R1,R1,#0          ; R1 ← *cptr_ptr = cptr
        LDR  R2,R1,#0          ; R2 ← **cptr_ptr = 'M'（两次间接 = LDI）
; 若 cptr 位于全局数据区，硬件一步完成两次访存：LDI  R2,CPTR_SLOT   ; R2 ← M[M[CPTR_SLOT]]
STR_FAV  .STRINGZ "My favorite string"
```

示例 3：用指针交换两个整数，并用指针"返回"第二个值

代码 (C)（`/tmp/ece220_ptr/03_swap_and_second_return.c`）：

```c
#include <stdint.h>
#include <stdio.h>

static void
swap (int32_t* a, int32_t* b)
{
    int32_t temp = *a;
    *a = *b;
    *b = temp;
}

static int32_t
divmod (int32_t num, int32_t den, int32_t* remainder)
{
    *remainder = num % den;     /* side effect on the caller's variable */
    return num / den;           /* the one real return value            */
}

int
main (void)
{
    int32_t x = 3;
    int32_t y = 8;
    int32_t q;
    int32_t r;

    printf ("before swap: x = %d, y = %d\n", x, y);
    swap (&x, &y);
    printf ("after  swap: x = %d, y = %d\n", x, y);

    q = divmod (47, 5, &r);
    printf ("47 / 5 = %d remainder %d\n", q, r);

    /* The addresses the callee received are the addresses of x, y, r. */
    printf ("&x = %p, &y = %p, &r = %p\n",
            (void*) &x, (void*) &y, (void*) &r);

    return 0;
}
```

实际输出：

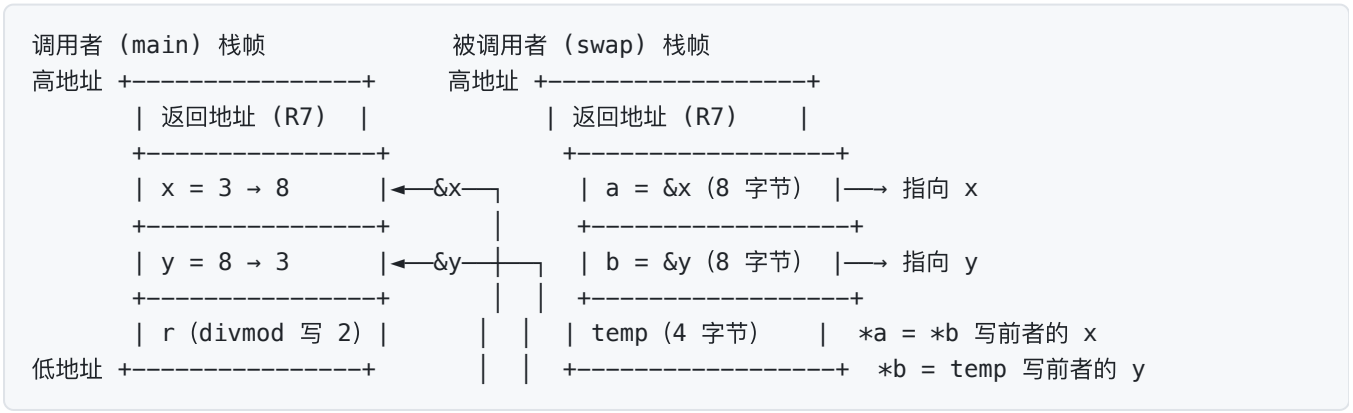
```
before swap: x = 3, y = 8
after  swap: x = 8, y = 3
47 / 5 = 9 remainder 2
&x = 0x7ffd06ed2468, &y = 0x7ffd06ed2464, &r = 0x7ffd06ed2460
```

【代码做什么？】 1. `swap (&x, &y)` 把两个**地址**按值传入形参 `a`、`b`。 2. 函数体先用 `temp = *a` 保存 `x` 的值，否则第一条写入就把它覆盖了。 3. `*a = *b`、`*b = temp` 通过指针写回，调用者的 `x`、`y` 完成交换。 4. `divmod (47, 5, &r)` 把商作为返回值，余数经 `*remainder` 写回调用者的 `r`——C 里"返回多

个值"的标准做法； 最后一行打印三个变量的地址，可见 `|&x - &y| = 4`、`|&y - &r| = 4`（三个 `int32_t` 相邻）。

【底层机制透视】 `swap` 改动的是**调用者栈帧里的 4 字节**，而形参 `a`、`b` 自身是**被调用者栈帧里的 8 字节**， 两者通过"写入到 `a` 所指地址"联系起来。这也解释了 `swap (x, y)`（漏写 `&`）为什么不行： 若把 `int` 值当地址用，函数会去写地址 3 和地址 8。地址相差 4 说明同一函数内的 automatic 变量被紧凑排布，但**相对顺序与是否相邻由编译器决定**，程序不应依赖。

【内存布局图解】



【与汇编的对应】（幻灯片"函数可以修改按值传入的地址上的比特"的机器版本）

```
; ---- 调用者：swap (&x, &y) ----
        ADD  R0,R5,#0          ; R0 = &x   (参数经 R0–R3 传递)
        ADD  R1,R5,#-1         ; R1 = &y
        JSR  SWAP              ; R7 ← 返回地址；返回后 x、y 已被改写

; ---- 被调用者 SWAP ----
SWAP    ; 进入时 R7 = 返回地址；若本子程序还要调用别人，必须先把 R7 压栈保存
        LDR  R2,R0,#0          ; R2 = *a  (= x)
        LDR  R3,R1,#0          ; R3 = *b  (= y)
        STR  R3,R0,#0          ; *a = R3 → 调用者的 x = 旧 y
        STR  R2,R1,#0          ; *b = R2 → 调用者的 y = 旧 x
        RET                    ; JMP R7

; ---- divmod：进入时 R0 = num, R1 = den, R2 = remainder 的地址 ----
        ; ... LC-3 无除法指令，商/余数由库子程序算好，设在 R3/R1（R1 的除数已用掉）...
        STR  R1,R2,#0          ; *remainder = 余数  ← "第二个返回值"
        ADD  R0,R3,#0          ; R0 = 商            ← 真正的返回值
        RET
```

示例 4：NULL 的用法与 `int *A, B;` 陷阱

代码 (C)（`/tmp/ece220_ptr/04_null_and_declaration_pitfall.c`）：

```
#include <stdint.h>
#include <stdio.h>

static int32_t*
find (int32_t* data, int32_t n, int32_t value)
{
    int32_t i;

    for (i = 0; i < n; i++) {
        if (data[i] == value) {
            return &data[i];
        }
    }
    return NULL;
}

int
main (void)
{
    int32_t  data[5] = {10, 20, 30, 40, 50};
    int32_t* hit;
    int32_t* miss;

    hit = find (data, 5, 30);
    miss = find (data, 5, 31);

    if (NULL != hit) {                      /* always test before use */
        printf ("found %d at index %lu\n", *hit,
                (unsigned long) (hit - data));
    }
    if (NULL == miss) {
        printf ("31 is not in the array (find returned NULL)\n");
    }

    printf ("hit is %s, miss is %s\n",
            (NULL != hit ? "valid" : "NULL"),
            (NULL != miss ? "valid" : "NULL"));

    /* ---- the declaration pitfall ---- */
    {
        int  *A, B;     /* A is int*, but B is a plain int!  */

        A = &B;
        *A = 5;
        printf ("sizeof (A) = %d, sizeof (B) = %d\n",
                (int) sizeof A, (int) sizeof B);
        printf ("B was set through A: B = %d\n", B);
    }

    return 0;
}
```

实际输出：

```
found 30 at index 2
31 is not in the array (find returned NULL)
hit is valid, miss is NULL
sizeof (A) = 8, sizeof (B) = 4
B was set through A: B = 5
```

【代码做什么？】 1. `find` 遍历数组，找到就返回该元素的**地址**（`&data[i]`），否则返回 NULL。 2. 调用者先判断 `NULL != hit` 再解引用——这是所有返回指针的函数的调用契约。 3. `hit - data` 是同类型指针相减，得到"相隔几个**元素**"（2），不是字节数。 4. 第二个代码块演示声明陷阱：`int *A, B;` 中 `A` 占 8 字节，`B` 只占 4 字节；`A = &B; *A = 5;` 合法且真的改了 `B`。

【底层机制透视】 `find` 的返回类型是 `int32_t*`，所以"没找到"必须借助一个**带外 (out-of-band)** 的地址值，标准选定全 0 的 NULL。 这也说明指针的本质：**任何非零位模式都可能是合法地址**，不能靠"看起来奇怪"判断有效性，只能靠约定。 指针相减得到 2，是因为编译器生成"字节差 ÷ `sizeof (int32_t)`"；且两指针必须指向同一数组（或其末尾下一位）才有定义。

【内存布局图解】

```
data 数组（main 的栈帧）                指针变量
0x7ffd..e0 +------+                    +--------------------+
           |  10  |  ← data[0]          | hit  = 0x7ffd..e8  |──┐
           +------+                     +--------------------+  |
0x7ffd..e4 |  20  |  ← data[1]          | miss = NULL（全 0）|  |
           +------+                     +--------------------+  |
0x7ffd..e8 |  30  |  ← data[2] ◄──────────────────────────────┘
           +------+   hit - data = (0x7ffd..e8 - 0x7ffd..e0) / 4 = 2
           |  50  |   ← 一维数组必须连续，否则 &data[i] 之后的指针算术没有意义
           +------+
```

【与汇编的对应】（NULL 判断就是条件码 `Z` 的判断）

```
; ---- hit = find (data, 5, 30); ----
        LEA  R0,DATA            ; R0 = 数组首地址（数组是全局对象 → LEA）
        AND  R1,R1,#0
        ADD  R1,R1,#5           ; R1 = 长度 5
        AND  R2,R2,#0
        ADD  R2,R2,#15
        ADD  R2,R2,#15          ; R2 = 30
        JSR  FIND
        STR  R0,R5,#-1          ; hit ← 返回的地址
; ---- if (NULL != hit) { *hit ... } ----
        LDR  R1,R5,#-1          ; R1 ← hit
        BRz  SKIP               ; 全 0 位模式 → Z=1 → 跳过；NULL 判断就是 BRz
        LDR  R2,R1,#0           ; R2 ← *hit（只有非 NULL 才敢解引用）
SKIP    ; FIND 内部循环 LDR 比较，失败时用 AND R0,R0,#0 造出 NULL 再 RET
```

> **演示（仅供演示、请勿模仿）：两种典型的未定义行为** 幻灯片里的经典 bug 是 `int* ptr; scanf ("%d", ptr);`：`ptr` 是 automatic 变量且从未赋值， 里面是栈上的**旧比特**，`scanf` 会往那个随机地址写数据。 `c > int32_t* ptr; /* 未初始化 */ > scanf ("%d", ptr); /* UB：写入随机地址 */ > int32_t value = 42; > int32_t* bad = &(value + 1); /* 编译错误：临时值没有地址 */ >` 实测（gcc 12.2.0，x86-64 Linux）：`-Wall -Werror` 下第一种**编译失败**，报 `error: 'ptr' is used uninitialized [-Werror=uninitialized]`；去掉 `-Werror` 后可编译， 运行时以**段错误（退出码 139）**结束。第二种报 `error: lvalue required as unary '&' operand`。 **这属于未定义行为，结果随编译器、优化级别与平台而异**，不能推理成"一定会崩溃"。

常见错误与调试技巧

- **用未初始化的指针**：`int32_t* p; *p = 1;`。现象是随机段错误，或悄悄破坏别的变量后在别处爆炸。**调试**：`-Wall -Werror` 以 `-Werror=uninitialized` 直接拒绝编译；`valgrind --track-origins=yes ./prog` 定位来源； `gdb` 中 `p p`、`bt`、`watch *p`。
- **`int *A, B;` 声明陷阱**：以为 `B` 也是指针， `B = &value;` 报类型错误（32 位平台上更隐蔽）。 **调试**：`gdb -tui --args ./prog` 后 `ptype A`、`ptype B`；或打印 `(int) sizeof A` 与 `(int) sizeof B`（8 与 4 立刻暴露）。
- **`scanf` 忘记取地址**：`scanf ("%d", value);` 会把变量的值当地址写进去。**调试**：`-Wall` 报 `format '%d' expects argument of type 'int *'`；已崩溃时 `gdb` 的 `bt` 看是否停在 `scanf` 内， `p &value` 与 `p value` 对比。
- **修改字符串常量或返回局部变量地址**：`char* s = "hi"; s[0] = 'H';` 段错误（只读段）；`return &local;` 是"有时能跑"的悬垂指针。 **调试**：`gdb` 中 `x/s s` 与 `info proc mappings` 确认只读映射；`gcc -Wall` 报 `function returns address of local variable`；再用 `-fsanitize=address -g` 复核。
- **混淆 `NUL`、`NULL`、`0`，或 `%p` 实参不是 `void*`**：字符串循环不结束，或地址打印错乱。 **调试**：`gdb` 中 `x/16xb str` 确认末尾是否真有 0 字节；统一写 `printf ("%p", (void*) p);` 并开启 `-Wall -Werror`。

关键要点

- **指针就是一个带类型的地址**：`X*` 从右往左读；类型只影响编译期两件事——解引用取几个字节、指针算术按几字节缩放，运行时的内存里没有类型，只有位。
- **声明指针不等于创建对象**：`int32_t* p;` 只提供装地址的空间；被指向的对象必须另行声明、来自字符串常量、或由 `malloc` 分配。
- **C 是值传递**：函数改不了调用者的变量本身，但可以改"调用者变量地址上的内容"； 要改调用者的指针变量就传 `&pointer`（形参 `T**`）——这正是 LC-3 `LDI` / `STI` 在 C 里的形态。
- **`&` 要有存储，`*` 要有有效地址**：这两条规则覆盖本讲绝大多数 bug。NULL 是让"无对象"可检测的约定（一条 `BRz` 即可判断），但要记住 `NUL`（字符）、`NULL`（指针）、`0`（数值）不是同一个东西。

思考题（带答案）

问题 1：下面两段代码，哪一段能把调用者的 `p` 改成指向新分配的内存？为什么？

```
static void alloc_a (int32_t* p)  { p = malloc (10 * sizeof (int32_t)); }
static void alloc_b (int32_t** p) { *p = malloc (10 * sizeof (int32_t)); }
```

答案：只有 `alloc_b`。`alloc_a` 的形参 `p` 是调用者指针值的**副本**：函数内让 `p` 指向新块， 调用者的指针毫无变化，而且那块内存立刻泄漏。`alloc_b` 收到的是"指针变量的地址"， `*p = ...` 写入的是**调用者的指针变量本身**。对应到 LC-3，就是被调用者并没有换掉调用者栈帧里那个指针槽， 而是**写入到它所指向的槽**（`STR Rd,Rbase,#0`）。

问题 2：`char* cptr = "My favorite string";` 之后，`*cptr`、`&cptr`、`**&cptr`、`&*cptr` 各是什么？哪个是编译错误?

答案：`*cptr` 是 `'M'`；`&cptr` 是 `cptr` 这个**指针变量**的地址（类型 `char**`）；`**&cptr` 等价于 `*cptr`，即 `'M'`； `&*cptr` 等价于 `cptr` 本身，类型 `char*`。真正报错的是 `&&cptr`： `&cptr` 的结果是临时值，没有存储位置 （gcc：`lvalue required as unary '&' operand`）。

问题 3：幻灯片里 `string_equal` 把 `if (*s1 != *s2) { return 0; }` 改成 `if (*s1 != *s2) { *s1 = *s2 = '\0'; return 0; }` 后，为什么 `printf ("%s %s\n", w, x)` 打印的**仍然**是 `word1 word2`？

答案：其一，`w`、`x` 是 main 的局部变量，`s1`、`s2` 是被调用者的形参副本， **函数内对指针本身的任何修改都不影响** `w`、`x`（只有 `*s1` 才可能影响调用者看到的内容）； 其二，它们指向的是**字符串常量**（static、只读），`*s1 = ...` 是未定义行为，在把字面量放入只读段的平台上会被硬件拒绝。 另外，想知道两个指针是否指向同一段字符，不能写 `s1 == s2`，必须逐字符比较（`strcmp` 做的事）。

Lecture 10: 数组 (Arrays)

概述

本讲讨论 C 中最基础的聚合机制——数组 (array)：**连续存储**的同类型元素，用从 0 开始的下标 (index) 访问。 "连续布局"这一个事实直接决定了访问方式（地址 = 基址 + 下标 × 元素大小），而"数组名在表达式中退化为指向首元素的指针" 这条规则把数组与上一讲的指针缝成同一件事。 它同时是字符串（下一讲）、二维数组、动态数组与 MP4/MP6 的基础，也是本课"理解数组与指针在内存中的表示"这条教学目标的核心。

核心概念与底层机制图解

- **数组 (Array)：连续存储的同类型元素，下标从 0 开始。**
  - *直观解释*：一排紧挨着的储物柜，每个柜子一样大，编号从 `region[0]` 开始； 第 N 个柜子的位置 = 第 0 个柜子的位置 + N × 柜子大小。
  - *底层机制图解*：`int32_t region[5]` 占 `5 * 4 = 20` 字节，元素紧挨着排： `int32_t region[5] = {10, 20, 30, 40, 50};` 地址（示意）：`base+0 base+4 base+8 base+12 base+16 +--------+--------+--------+--------+--------+ region: | 10 | 20 | 30 | 40 | 50 | +--------+--------+--------+--------+--------+`

`------+--------+--------+ region[0] region[1] region[2] region[3] region[4] ^ └─ 数组名 region 的值就是 base（元素 0 的地址） 实测：&region[1] − &region[0] = 4 字节，&region[4] − &region[0] = 16 字节` "连续"不是语法规定而是**内存布局规定**：正因为连续，才能用"加一个偏移量"算出任意元素的位置—— 这就是 `LDR Rd, Rbase, #offset` 这种"基址 + 偏移"寻址在硬件层面的对应。

- *作用域与存储期*：函数内声明的数组是 automatic，整块 20 字节随栈帧生灭； 加 `static` 则成为 static 存储期； `malloc` 得到的则是 allocated 存储期（本讲不涉及）。

- **方括号是"加法 + 解引用"的语法糖**：`region[N]` 与 `*(region + N)` 是**同一个表达式**。
  - *直观解释*：下标不是"第几个柜子"这种高级概念，只是"从首地址往前走 N 步，然后开门"的简写。
  - *底层机制图解*： `region[N] ≡ *(region + N) 编译器的步骤：1) 把 region 当成 int32_t*；2) 加上 N * sizeof (int32_t) = 4N 字节； 3) 对结果解引用（一次 LDR/STR） 推论一：既然加法可交换，2[region] 也合法且等于 region[2]（实测输出 30）。 推论二：region[N] 与 N[region] 都只是 *(region + N)，写成前者才是人读得懂的代码。 推论三：*(region + 2) = 99 与 region[2] = 99 改的是同样的 4 字节（实测两者一致）。`
  - *作用域与存储期*： `region[N]` 本身只是一个表达式，不引入存储； 它读写的是数组对象（automatic/static/allocated 由数组自身的声明决定）。
- **指针算术按元素大小缩放 (stride = `sizeof (element)`)**：`p + 1` 是"下一个**元素**"，不是"下一个字节"。
  - *直观解释*：步长由"柜子有多大"决定： `int32_t` 柜子 4 字节， `char` 柜子 1 字节， `double` 柜子 8 字节。
  - *底层机制图解*：幻灯片的问题——若 `region` 是 `0x12345000`， `region + 5` 是不是 `0x12345005`？ **不是**：答案是"5 个 int 所需要的地址数"，即 `0x12345000 + 20`： `实测：region + 5 is 20 bytes above region ← 5 * sizeof (int32_t) = 20 对照：char* c; c + 5 会前进 5 字节；double* d; d + 5 会前进 40 字节 两种寻址方式的换算（LC-3 按"字"寻址，C 按"字节"寻址）： C 里的 1 个 int32_t = 4 字节 = LC-3 的 2 个字` 这就是为什么参数必须带上类型：编译器要靠它把"第 N 个元素"翻译成"地址 + N × 4"。
  - *作用域与存储期*：指针算术不改变任何对象的存储期，只是产生一个新的地址值； 但**这个新地址必须落在同一个数组对象内**（或末尾的下一个位置），否则行为未定义。
- **数组名是元素 0 的地址，在表达式中"退化"为指针**。
  - *直观解释*： `region` 这个"名字"在大多数场合等价于"首柜子的地址"，而不再是"一整排柜子"。
  - *底层机制图解*：C Programming Reference 给出的派生类型规则：数组参数会被编译器**立即**转换成指针： `void foo (int x[4]); 立刻被编译器改写成: void foo (int* x); 实测: region == &region[0] -> 1` 但有两个例外： `sizeof (region)` 得到 20（整块数组）， `&region` 得到"指向整个数组的指针"。
  - *作用域与存储期*：数组名不是变量，没有自己的存储；它是编译器为那块存储起的名字。
- **`sizeof (array)` 与 `sizeof (pointer)` 是两件完全不同的事**。
  - *直观解释*： `sizeof` 问的是"这个名字代表的对象有多大"；数组是 20 字节的柜子排，指针只是一张纸条（8 字节）。

- *底层机制图解*：`实测：sizeof (region) = 20 sizeof (region[0]) = 4 sizeof (p) = 8 (p 是 int32_t*) number of elements = sizeof (region) / sizeof (region[0]) = 5` **这条规则可以反过来当工具用**：`sizeof (arr) / sizeof (arr[0])` 是"在数组自己的作用域内"求元素个数的标准写法； 而 `sizeof` 是编译期运算（对定长数组），不产生任何运行时代码。

- **数组作为参数传递：传的是地址，不是副本；函数无法从参数恢复长度。**
  - *直观解释*：把整排柜子搬到另一个房间（复制）太贵；C 只把"首柜子的地址"告诉被调用者—— 代价是**对方不知道这排柜子有几个**。
  - *底层机制图解*：幻灯片的示范函数签名 `int32_t min_value (int32_t const values[]);` 与 `int32_t const* values` 完全等价，它只有一个"地址"，因此必须**再传一个长度参数**： `调用者栈帧 被调用者（min_value）栈帧 +------------------+ +--------------------------+ | my_nums[0..3] |◄———| values （1 个地址，8 字节）| | 93 100 79 42 | +------------------ --------+ +------------------+ | n_values = 4 | 整块 16 字节留在调用者栈帧；被调用者只拿到一个地址 → 数组不会被复制，因此被调用者通过 a[i] 的写入会直接改变调用者的数据。` 幻灯片特别指出：`values`（类型 `int32_t const*`）在栈帧里**只占一个内存位置**。
  - *作用域与存储期*：形参是 automatic 的**指针**，被指向的数组属于调用者； `const` 只是编译期承诺（"我只读"），不会改变存储期。

- **C 不检查数组边界 (no bounds checking)**；同时**允许"末尾的下一个位置"这个指针存在。**
  - *直观解释*：机器只做"地址 + 偏移"，它不知道柜子排到哪里结束；越界读到的就是邻居的东西。 而"末尾下一个位置"像"停车场最后一个车位之后的那个点"——可以用来比较，但不能停车（不能解引用）。
  - *底层机制图解*：两种规则的边界： `合法： int32_t* end = data + 6; /* 一过末尾 (one past the end)：只比较，不解引用 */ 非法： *end = 1; data[6] = 1; data[-1] = 1; /* 全部是 UB */` 幻灯片关于"边界不检查"的经典例子是 `char name[20]; scanf ("%s", name);`：输入超过 19 个字符会顺着栈帧 覆盖返回地址（缓冲区溢出攻击）； `scanf ("%19s", name)` 能限制长度，但幻灯片提醒"依赖人来维护的防护措施本身容易出错"。
  - *作用域与存储期*：越界访问不会创建新对象，它只是访问了**别的对象**的内存； 那些对象有自己的存储期，被破坏后症状往往在别处才出现。

- **用指针遍历数组 (walking with a pointer)**：`for (p = a; p != a + n; p++)` 是最地道的写法。
  - *直观解释*：与其每次用"下标 × 元素大小"重新算地址，不如让游标自己往前走。
  - *底层机制图解*： `下标写法：for (i = 0; i < n; i++) { total += a[i]; } 每次迭代都要算 base + i*4（编译器通常会优化成自增） 指针写法：for (p = a; p != a + n; p++) { total += *p; } p++ 直接加 4，比较用 != 而不是 <，语义是"走到末尾就停" 实测（12_array_parameters.c）：sum_by_pointer (data, data + 6) = 21` 两种写法在汇编层面几乎一样：取数、累加、指针加、比较、分支。
  - *作用域与存储期*：游标 `p` 是 automatic；遍历不改变元素的存储期，但可以对它们赋值。

- **LC-3 视角：数组访问就是"基址 + 偏移"寻址。**
  - *直观解释*：`LEA` 算出首地址， `LDR/STR` 用一个小偏移量读写——这就是"数组"在机器层的全部内容。

- *底层机制图解*：指令编码限制了"一步走多远"：`LEA Rd, LABEL ; PCoffset9：±256 个字，只能指向汇编期已知的标号` `LDR Rd, Rbase, #o ; offset6：−32..31，超范围就必须先 ADD 改基址 运行期下标 i：先 ADD R3,R1,R2 算出地址，再 LDR R0,R3,#0 编译期常量下标 i ≤ 31：可以直接 LDR R0,R1,#i`
- *作用域与存储期*：数组通常放在全局数据区（`LEA` 取址）或栈帧里（`R5 + 偏移` 取址）； 局部数组随栈帧回收，全局数组从程序开始存在到结束。

代码示例与底层机制分析

示例 1：连续性、方括号语法糖与 `sizeof` 的区别

代码 (C)（`/tmp/ece220_arr/10_array_basics.c`，用 `gcc -g -std=c99 -Wall -Werror 10_array_basics.c -o 10_array_basics` 实测）：

```
#include <stdint.h>
#include <stdio.h>

int
main (void)
{
    int32_t  region[5] = {10, 20, 30, 40, 50};
    int32_t* p;

    /* 1. The elements are contiguous: each one is 4 bytes above the last. */
    printf ("&region[0] = %p, &region[1] = %p\n",
            (void*) &region[0], (void*) &region[1]);
    printf ("&region[1] - &region[0] = %lu bytes\n",
            (unsigned long) ((unsigned long) (void*) &region[1] -
                             (unsigned long) (void*) &region[0]));
    printf ("&region[4] - &region[0] = %lu bytes\n",
            (unsigned long) ((unsigned long) (void*) &region[4] -
                             (unsigned long) (void*) &region[0]));

    /* 2. Pointer arithmetic multiplies by sizeof (int32_t). */
    printf ("region + 5 is %lu bytes above region\n",
            (unsigned long) ((unsigned long) (void*) (region + 5) -
                             (unsigned long) (void*) region));

    /* 3. region[N] and *(region + N) are the same expression. */
    printf ("region[2] = %d, *(region + 2) = %d, 2[region] = %d\n",
            region[2], *(region + 2), 2[region]);
    printf ("region[2] == *(region + 2) -> %d\n",
            region[2] == *(region + 2));

    /* 4. Writing through either form changes the same 4 bytes. */
    *(region + 2) = 99;
    printf ("after *(region + 2) = 99: region[2] = %d\n", region[2]);
    region[3] = 77;
    printf ("after region[3] = 77: *(region + 3) = %d\n", *(region + 3));

    /* 5. An array name decays to a pointer to element zero... */
    printf ("region == &region[0] -> %d\n", region == &region[0]);
    p = region;                       /* same as p = &region[0];        */
    printf ("*p = %d, p[4] = %d\n", *p, p[4]);

    /* 6. ...but sizeof still sees the ARRAY, not a pointer. */
    printf ("sizeof (region) = %d, sizeof (region[0]) = %d\n",
            (int) sizeof region, (int) sizeof region[0]);
    printf ("sizeof (p) = %d (a pointer is one address, not 5 elements)\n",
            (int) sizeof p);
    printf ("number of elements = %d\n",
            (int) (sizeof region / sizeof region[0]));

    return 0;
}
```

实际输出：

```
&region[0] = 0x7fff60c21670, &region[1] = 0x7fff60c21674
&region[1] - &region[0] = 4 bytes
&region[4] - &region[0] = 16 bytes
region + 5 is 20 bytes above region
region[2] = 30, *(region + 2) = 30, 2[region] = 30
region[2] == *(region + 2) -> 1
after *(region + 2) = 99: region[2] = 99
after region[3] = 77: *(region + 3) = 77
region == &region[0] -> 1
*p = 10, p[4] = 50
sizeof (region) = 20, sizeof (region[0]) = 4
sizeof (p) = 8 (a pointer is one address, not 5 elements)
number of elements = 5
```

【代码做什么？】 1. `&region[1] - &region[0] = 4`、`&region[4] - &region[0] = 16`：元素连续，每个占 4 字节；`region + 5` 则高 **20 字节**（不是 5）。 2. `region[2]`、`*(region + 2)`、`2[region]` 都得 30；`*(region + 2) = 99` 与 `region[3] = 77` 的写入彼此等价。 3. `region == &region[0]` 为 1（数组名退化为首元素地址），而 `sizeof (region)` 仍是 20、`sizeof (p)` 是 8。

【底层机制透视】 第 1 步里 `region + 5` 是全讲最容易被直觉误导的一行：它不是"地址 + 5"，而是"地址 + 5 × `sizeof (int32_t)`"（幻灯片里 `0x12345000 + 5` 的错误直觉正是指这个）。第 2、3 步展示了"退化"与"不退化"的分界： 数组名在绝大多数表达式里是 `int32_t*`，但 `sizeof` 与 `&` 看到的是数组类型本身。 `sizeof region / sizeof region[0]` 是本课推荐的长度计算法，但它**只在数组自己的作用域内有效**（见示例 3）。

【内存布局图解】

```
region（main 的栈帧，20 字节连续）
base+0      base+4      base+8      base+12     base+16     base+20
+-----------+-----------+-----------+-----------+-----------+
|    10     |    20     |    99     |    77     |    50     |  ← 被步骤 4/5 改写
+-----------+-----------+-----------+-----------+-----------+
  region[0]   region[1]   region[2]   region[3]   region[4]   ← region+5 指向这里（末尾之后）
  ^ region = &region[0] = base（实测 0x7fff60c21670）；region + 2 = base + 8
p（另 8 字节）→ base；p[4] 读的就是 region[4] = 50；region + 5 指向末尾之后（20 字节处）
```

【与汇编的对应】（LC-3：`LEA` 取基址、`LDR/STR` 带偏移访问——"基址 + 偏移"就是数组）

```
; int32_t region[5] 放在全局数据区（用 LEA 取址），元素按字排列
        LEA  R1,REGION          ; R1 = region 的首地址（标号汇编期已知 → LEA）
        LDR  R2,R1,#0           ; R2 = region[0] = 10   ← 常量下标直接用 offset6
        LDR  R3,R1,#2           ; R3 = region[2] = 30
        ; R0 ← 99（由若干 ADD 构造）
        STR  R0,R1,#2           ; region[2] = 99        ← 与 *(region + 2) = 99 等价
        LDR  R0,R1,#3           ; R0 = region[3] = 77

; 运行期下标 i（放在 R2）：必须"先算地址、再访问"
        ADD  R3,R1,R2           ; R3 = region + i（LC-3 上每个 int 1 个字）
        LDR  R0,R3,#0           ; R0 = region[i]
; 注意 offset6 只有 -32..31：i 可能更大，所以不能写 LDR R0,R1,#i

REGION  .FILL #10
        .FILL #20
        .FILL #30
        .FILL #40
        .FILL #50
```

示例 2：课堂手算示例——求数组最小值

代码 (C)（`/tmp/ece220_arr/11_min_value.c`；这就是幻灯片上的 `min_value`，手算数据也用幻灯片里的 `{93, 100, 79, 42}`）：

```
#include <stdint.h>
#include <stdio.h>

/*
 * min_value -- return the smallest element of an array of int32_t.
 * INPUTS: values -- pointer to the first element (an address alone does
 *                   NOT define a length, so the caller must pass one)
 *         n_values -- number of elements in the array
 * OUTPUTS: none
 * RETURN VALUE: the smallest element (undefined if n_values < 1)
 * SIDE EFFECTS: none
 */
static int32_t
min_value (int32_t const values[], int32_t n_values)
{
    int32_t min = values[0];        /* assume the first value is smallest */
    int32_t check;

    for (check = 1; n_values > check; check++) {
        if (min > values[check]) {  /* found something smaller           */
            min = values[check];
        }
    }
    return min;
}

int
main (void)
{
    int32_t my_nums[4] = {93, 100, 79, 42};
    int32_t least;

    least = min_value (my_nums, 4);

    printf ("my_nums = {%d, %d, %d, %d}\n",
            my_nums[0], my_nums[1], my_nums[2], my_nums[3]);
    printf ("min_value (my_nums, 4) = %d\n", least);

    /* The parameter was an address, not a copy of the data. */
    printf ("inside main, sizeof (my_nums) = %d\n", (int) sizeof my_nums);
    printf ("my_nums == &my_nums[0] -> %d\n", my_nums == &my_nums[0]);

    return 0;
}
```

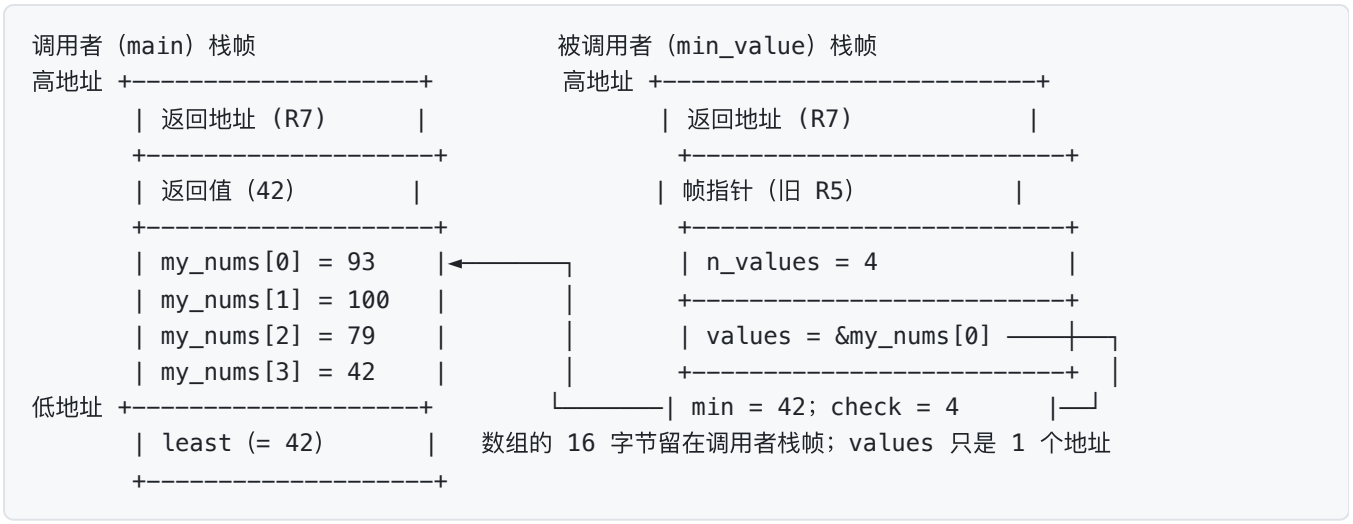
实际输出：

```
my_nums = {93, 100, 79, 42}
min_value (my_nums, 4) = 42
inside main, sizeof (my_nums) = 16
my_nums == &my_nums[0] -> 1
```

【代码做什么？】 1. 调用 `min_value (my_nums, 4)`：传的是**首地址**与**长度**两个值（数组本身不复制）。 2. 被调用者先假设 `values[0] = 93` 最小，然后让 `check` 从 1 走到 3。 3. `check=1`：`93 > 100`？否，`min` 仍为 93；`check=2`：`93 > 79`？是，`min` 变成 79；`check=3`：`79 > 42`？是，`min` 变成 42；循环结束后返回 **42**。 4. 调用者打印 `sizeof (my_nums) = 16`（4 个 `int32_t`），并验证 `my_nums == &my_nums[0]`。

【底层机制透视】 形参写成 `int32_t const values[]` 与写成 `int32_t const* values` **完全等价**： C 的派生类型规则规定，"数组类型"作为函数参数时会被编译器立即改写为"指向元素的指针"。 所以 `sizeof (values)` 在函数内部只能得到 8（指针大小），**函数无法从参数恢复数组长度**—— 幻灯片因此把"再加一个 `n_values` 参数"作为唯一可行的解法。 `const` 是对调用者的承诺："这个函数只读数组"，它也让编译器在函数体里拒绝 `values[i] = ...`。 注意 `min = values[0]` 在 `n_values < 1` 时是越界读，所以文档必须写明这一前提——C 的接口约定要写全。

【内存布局图解】



【与汇编的对应】（LC-3：`min_value` 的循环）

```
; 约定：R0 = 数组地址（values），R1 = 元素个数（n_values），返回值放在 R0
MINVAL  ; 入口先建立自己的栈帧（保存旧 R5、R7 并取局部空间）
        ADD  R6,R6,#-1
        STR  R5,R6,#0           ; 保存调用者的帧指针
        ADD  R5,R6,#0           ; R5 = 本帧基址（全程只读：R5 是帧指针，不能当临时寄存器）
        ADD  R6,R6,#-3
        STR  R0,R5,#-2          ; values（只占 1 个地址！）
        STR  R1,R5,#-3          ; n_values
        LDR  R2,R0,#0           ; R2 = min = values[0]
        AND  R3,R3,#0
        ADD  R3,R3,#1           ; R3 = check = 1
        ; R0–R3 已被 values/n_values/min/check 占满：临时值只借 R0/R1，用完再从帧里取回
MLOOP   LDR  R0,R5,#-3          ; R0 ← n_values（从帧里取回，R0 让出来做临时）
        NOT  R0,R0
        ADD  R0,R0,#1
        ADD  R0,R0,R3           ; R0 = check − n_values：check >= n_values 就结束
        BRzp MDONE
        LDR  R1,R5,#-2          ; R1 ← values（从帧里取回，R1 让出来做临时）
        ADD  R1,R1,R3           ; R1 = &values[check]  ← 基址 + 下标
        LDR  R1,R1,#0           ; R1 = values[check]
        NOT  R0,R1
        ADD  R0,R0,#1
        ADD  R0,R0,R2           ; R0 = min − values[check]
        BRnz MNEXT              ; min <= values[check] → 不更新
        ADD  R2,R1,#0           ; min = values[check]
MNEXT   ADD  R3,R3,#1           ; check++
        BRnzp MLOOP
MDONE   ADD  R0,R2,#0           ; 返回值 = min；R5 未被改写，恢复 R5/R7 并退回 R6 即可
        RET
```

示例 3：把数组传给函数、以及"一过末尾"的遍历

代码 (C)（`/tmp/ece220_arr/12_array_parameters.c`）：

```
#include <stdint.h>
#include <stdio.h>

/* Declaring the parameter as "int32_t a[]" would be EXACTLY equivalent to
 * "int32_t* a": the compiler rewrites an array parameter into a pointer to
 * its first element.  We write the pointer form here because gcc's -Wall
 * refuses to let us sizeof an array parameter (it would return the size of
 * a pointer, which is a bug in almost every case).                        */
static void
show_parameter_size (int32_t* a)
{
    printf ("inside callee: sizeof (a) = %d (a pointer, not an array)\n",
            (int) sizeof a);
}

/* One-past-the-end walking: we never form a pointer past a + n, and we
 * never dereference a + n.  This is the standard C loop idiom.          */
static int32_t
sum_by_pointer (int32_t const* begin, int32_t const* end)
{
    int32_t const* cursor;
    int32_t        total = 0;

    for (cursor = begin; cursor != end; cursor++) {
        total += *cursor;
    }
    return total;
}

/* Doubling every element: the callee writes THROUGH the pointer, so the
 * caller sees the change.  No copying of the array takes place.         */
static void
double_all (int32_t* a, int32_t n)
{
    int32_t i;

    for (i = 0; i < n; i++) {
        a[i] = 2 * a[i];
    }
}

int
main (void)
{
    int32_t data[6] = {1, 2, 3, 4, 5, 6};
    int32_t i;

    printf ("sizeof (data) in main = %d, elements = %d\n",
            (int) sizeof data, (int) (sizeof data / sizeof data[0]));
    show_parameter_size (data);

    printf ("sum = %d\n", sum_by_pointer (data, data + 6));
```

```
    double_all (data, 6);
    printf ("after double_all:");
    for (i = 0; i < 6; i++) {
        printf (" %d", data[i]);
    }
    printf ("\n");

    /* Passing data + 2 gives the callee a "shorter array". */
    double_all (data + 2, 4);
    printf ("after double_all (data + 2, 4):");
    for (i = 0; i < 6; i++) {
        printf (" %d", data[i]);
    }
    printf ("\n");

    return 0;
}
```

实际输出：

```
sizeof (data) in main = 24, elements = 6
inside callee: sizeof (a) = 8 (a pointer, not an array)
sum = 21
after double_all: 2 4 6 8 10 12
after double_all (data + 2, 4): 2 4 12 16 20 24
```

【代码做什么？】 1. `main` 里 `sizeof (data) = 24`；同一数组传进 `show_parameter_size` 后 `sizeof (a) = 8`——**长度信息在函数边界上丢失了**。 2. `sum_by_pointer (data, data + 6)` 用"末尾的下一个位置"作终止条件，累加得到 21。 3. `double_all (data, 6)` 得 `2 4 6 8 10 12`（改的是**调用者的数组**）；`double_all (data + 2, 4)` 只看后 4 个元素，得 `2 4 12 16 20 24`。

【底层机制透视】 第 1 步是**本讲最重要的教训**：数组参数是指针，所以 `sizeof` 在函数内部只能看到 8 字节； "函数无法恢复数组长度"不是编译器的缺陷，而是"只传地址"这一设计的必然结果——必须由调用者显式传长度 （或用一个哨兵值，如字符串的 `'\0'`）。 第 3 步的 `cursor != end` 使用 `!=` 而不是 `<`，是因为 C 标准只保证"同一数组内的指针可以比较"， 用 `!=` 表达的语义最清楚：走到末尾就停。第 5 步说明"数组的起点"只是一种约定： `data + 2` 让被调用者看到的是长度 4 的另一段内存，这既是 C 的灵活性，也是它容易出错的地方。

【内存布局图解】

```
main 的栈帧（data 24 字节）
base+0   base+4   base+8   base+12  base+16  base+20  base+24
+--------+--------+--------+--------+--------+--------+
|   1    |   2    |   3    |   4    |   5    |   6    |   ← 初始值
+--------+--------+--------+--------+--------+--------+
  ^ begin = data                                ^ end = data + 6（一过末尾，不可解引用）
  |  第一次 double_all(data, 6) 之后：2 4 6 8 10 12
  └─ 第二次 double_all(data + 2, 4)：只碰 base+8 起的 4 个元素 → 2 4 12 16 20 24
     begin/end 只是两个地址，被调用者不知道这块内存总共有多大
```

【与汇编的对应】（LC-3：传入的是地址与长度，循环用"指针 != 末尾"）

```
; ---- 调用者：sum_by_pointer (data, data + 6) ----
        LEA  R0,DATA            ; R0 = begin = data（数组在全局数据区）
        ADD  R1,R0,#6           ; R1 = end   = data + 6（每个 int 占 1 个字）
        JSR  SUMBYPTR           ; 返回值（21）回到 R0

; ---- 被调用者：按指针遍历，直到游标等于 end ----
SUMBYPTR AND R2,R2,#0           ; total = 0
        ADD  R3,R0,#0           ; cursor = begin
SUMLOOP NOT  R0,R1              ; begin 已复制进 R3，R0 空出来做临时
        ADD  R0,R0,#1
        ADD  R0,R3,R0           ; R0 = cursor - end
        BRz  SUMDONE            ; cursor == end → 结束（只比较，不解引用 end）
        LDR  R0,R3,#0           ; R0 = *cursor
        ADD  R2,R2,R0           ; total += *cursor
        ADD  R3,R3,#1           ; cursor++
        BRnzp SUMLOOP
SUMDONE ADD R0,R2,#0            ; 返回 total
        RET
DATA    .FILL #1
        .FILL #2
        .FILL #3
        .FILL #4
        .FILL #5
        .FILL #6
; begin/end 是两个地址（经 R0–R3 传参）；"指针 != 末尾"就是一次减法看 Z 条件码。
```

> **演示（仅供演示、请勿模仿）：数组越界** C 不检查下标，越界读写会碰到相邻对象：`c > int32_t region[5] = {10, 20, 30, 40, 50}; > int32_t neighbour = 0x5A5A; > printf ("%d\n", region[5]); /* UB：读一过末尾 */ > printf ("%d\n", region[-1]); /* UB：读下界之外 */ > region[5] = 0; /* UB：写一过末尾，可能覆盖 neighbour，也可能覆盖别的 */ >` 实测（gcc 12.2.0，x86-64 Linux，未加 `-Werror`）：本次运行 `region[5]` 与 `region[-1]` 都打印 `0`，`neighbour` 仍为 `0x5A5A` ——**这只是本次编译的巧合**：那些位置是填充字节，换个编译选项就会不同；把越界写成循环时， 被写坏的甚至可能是循环计数器本身（本笔记的早期验证版本就因此陷入死循环）。**UB 的结果随编译器与平台而异。** 作为对照，课程的编译命令能拦住"用 `sizeof` 求数组参数长度"：实测 gcc 报 `error: 'sizeof' on array function parameter 'a' will return size of 'int32_t *' [-Werror=sizeof-array-argument]`。

常见错误与调试技巧

- **把 `sizeof` 用在数组参数上求长度**：函数内 `sizeof (a) / sizeof (a[0])` 得到 2（8/4）而不是真实元素个数。 **调试**：`gcc -g -std=c99 -Wall -Werror` 会以 `-Werror=sizeof-array-argument` 直接拒绝编译； 已在运行的代码用 `gdb` 的 `ptype a` 确认参数类型是 `int32_t *`，并把长度作为额外参数传入。

- **误以为 `p + 1` 前进 1 字节**：对 `int32_t*` 而言是 4 字节；现象是索引算错 4 倍。**调试**：`gdb` 中 `p p`、`p p+1`、`p (char*)(p+1) - (char*)p` 三步核对，或临时改成 `(char*)` 做字节运算验收。
- **越界读写或解引用"一过末尾"的指针**：`for (i = 0; i <= n; i++) a[i] = ...;` 多写一个元素；`*end = 1;` 更直接。 **调试**：`gcc -fsanitize=address -g` 立即报 `stack-buffer-overflow`；`valgrind --track-origins=yes ./prog` 报 invalid write；`gdb` 中 `x/8dw a` 看相邻变量、`watch *end` 看是否被写。
- **数组名赋值**：`a = b;`（a 是数组）报 `assignment to expression with array type`。**调试**：复制内容用 `memcpy (a, b, sizeof a)` 或逐元素赋值；要"改指向"就改用指针变量。
- **把 `int*` 与 `int (*)[N]` 混用**：`int (*p)[4] = m;` 与 `int* q = m;` 步长不同（下一讲详述）。 **调试**：`gdb` 的 `ptype p`、`ptype q`，或打印 `(int) sizeof *p`（16 与 4 的差别立刻暴露）。

关键要点

- **数组 = 连续存储 + 从 0 开始的下标**：正因为连续，"地址 = 基址 + 下标 × `sizeof (元素)`"才成立，这也正是 LC-3 里 `LDR/STR` 的"基址 + 偏移"寻址形式。
- **方括号只是加法与解引用的语法糖**：`region[N] ≡ *(region + N)`（所以 `2[region]` 也合法）；指针算术的步长是 `sizeof (元素)`，`int32_t* p; p + 5` 前进 **20 字节**。
- **数组名退化为指针，但 `sizeof` 与 `&` 例外**：`sizeof (arr)` 给出整块大小，是在数组作用域内求元素个数的唯一可靠写法。
- **数组参数就是指针**：`void f (int a[])` 被编译器改写为 `void f (int* a)`，函数**无法从参数恢复长度**，必须另传长度参数（或用哨兵值）。
- **C 不检查边界**，只允许"末尾的下一个位置"作为比较用的指针；越界读写是未定义行为，攻击者可借此覆盖返回地址， 防御手段是限制长度的输入函数与显式的范围检查。

思考题（带答案）

问题 1：`int32_t region[20];`，已知 `region` 的地址是 `0x12345000`。`region + 5` 的值是多少？ 为什么幻灯片说"不一定是 `0x12345005`"？ 如果把数组类型换成 `char region[20]` 呢？

答案：`region + 5 = 0x12345000 + 5 × sizeof (int32_t) = 0x12345000 + 20 = 0x12345014`。 幻灯片强调"不一定是 `0x12345005`"，是因为**指针算术按所指类型的大小缩放**：步长取决于"一个元素占多少地址"。换成 `char region[20]` 时 `sizeof (char) = 1`，`region + 5` 就真的是 `0x12345005`。 本笔记的实测输出直接验证了这一点：`region + 5 is 20 bytes above region`。

问题 2：下面的函数想返回数组长度，但它错了。错在哪里？给出两种修法。

```
static int32_t count (int32_t a[])
{
    return (int32_t) (sizeof a / sizeof a[0]);
}
```

答案：`a` 虽然写成 `int32_t a[]`，但作为函数参数它被编译器**立即改写为 `int32_t* a`**， 所以 `sizeof a` 是 8（指针）而不是整块数组的大小，`8 / 4 = 2` 是垃圾结果；加 `-Wall -Werror` 时 gcc 直接以 `-`

`Werror=sizeof-array-argument` 拒绝编译。修法一：把长度作为参数显式传入（幻灯片的 `min_value` 就是这样）。 修法二：传"首尾指针"（`end - begin` 得到元素个数，但两个指针必须指向同一个数组）。 **在数组自己的作用域内**（例如 `main` 里），`sizeof (a) / sizeof (a[0])` 仍然是正确且推荐的写法。

问题 3：为什么允许 `int32_t* end = data + n;`（一过末尾）存在，却不允许 `*end = 0;`？

答案：C 标准允许指针指向"数组最后一个元素之后的那一个位置"，因为这是**循环终止条件**的自然写法：`for (p = data; p != end; p++)` 需要一个"刚好在末尾"的地址来比较；若连这个地址都不能形成， 遍历就只能改用计数变量。但"允许形成这个地址"不等于"允许访问那里的对象"——那里**没有对象**：`*end = 0` 会写到相邻对象（或填充字节）上，是未定义行为，可能悄无声息地破坏别的变量，也可能立刻崩溃。 一句话：**一过末尾的指针可以比较、不可以解引用**；而 `data[-1]` 连形成都已经是未定义行为。

Lecture 11: 字符串与多维数组 (Strings; Multi-Dimensional Arrays)

概述

本讲解决两组彼此关联的问题：C 如何表示文本（以 NUL 结尾的 `char` 数组），以及 C 如何表示表格（"数组的数组"）。 字符串部分要讲清 `char s[]` 与 `char *s` 这个**一字之差、语义完全不同**的区分，以及不检查边界的字符串函数 如何导致缓冲区溢出 (buffer overflow)；多维数组部分要讲清行主序 (row-major) 布局与地址公式， 以及 `int m[3][4]`、`int *m[3]`、`int (*m)[4]` 三种写法的内存含义。 二者都建立在上一讲的"数组名即地址、指针算术按元素大小缩放"之上，又是课程 MP2（课表）、MP6（方块棋盘）的直接工具。

核心概念与底层机制图解

- **C 字符串 (C string) 是以 NUL 结尾的 `char` 数组**：字符串的"值"就是起始地址，长度靠扫描到 `'\0'` 才知道。
  - *直观解释*：像一列没有编号的储物柜，末尾放一面红旗；要数有多少格，只能一直走到红旗。
  - *底层机制图解*：`地址 0x402008 0x402009 0x40200A 0x40200B 0x40200C 内容 'H' 'i' '!' '\0' ?? (无关字节)  strlen = 3 (不含 NUL), sizeof ("Hi!") = 4 (含 NUL)` `strlen` 的机器实现就是"从首地址开始 `LDR`，为 0 就停，否则地址加一"。
  - *作用域与存储期*：字符串本体没有类型，存储期取决于来源：字面量 static、`char buf[32]` automatic、`malloc` 来的 allocated。"字符串"这个概念 = 一个 `char*` + "以 0 结尾"这个契约。
- **字符串字面量 (string literal) 在静态存储区，且不可修改**：编译期放入全局数据区（通常只读），相同字面量可能被合并。
  - *直观解释*：标语牌是印刷好的；`char* p = "hi";` 只是拿了一张写着地址的便条，便条可换，标语牌不能涂。
  - *底层机制图解*：`sizeof ("hello")` 为 6，说明"字面量本身是一个 6 字节的数组"；出现在表达式里时退化为 `char*`。 写入字面量是未定义行为，在把字面量放进只读段的平台上直接段错误。
  - *作用域与存储期*：static storage duration，程序整个运行期都在；不要释放也不能释放。
- **`char s[]` 与 `char *s`：一个分配数组，一个只分配指针**：

- *直观解释*：`char s[] = "hi";` 是"照着标语牌抄一份自己的副本"；`char *s = "hi";` 是"只记下标语牌的位置"。
- *底层机制图解*：`char a[] = "hello"; char* p = "hello";` 栈（automatic，可写，6 字节） 栈（automatic，8 字节） +---+---+---+---+---+---+ +------------------+ |'h'|'e'|'l'|'l'|'o'|\0 | | p = 0x402008 | +---+---+---+---+---+---+ +------------------+ sizeof (a) = 6 | sizeof (p) = 8 v 全局数据区（只读） +---+---+---+---+---+----+ |'h'|'e'|'l'|'l'|'o'|\0 | +---+---+---+---+---+----+ a[0] = 'H'; 合法（改自己的副本） p[0] = 'H'; 未定义行为（改只读数据） a = "bye"; 非法（数组名不可赋值） p = "bye"; 合法（只改这 8 字节）
- *作用域与存储期*：`a` 与 `p` 都是 automatic，但 `p` **指向的数组是 static**；因此 `char* f (void) { char buf[8] = "hi"; return buf; }` 是悬垂指针，而 `return "hi";` 完全正确。实测（`/tmp/ece220_str/21t_char_array_vs_pointer.c`，课程编译命令）：`sizeof (array_form) = 6`、`sizeof (pointer_form) = 8`、`sizeof ("hello") = 6`；用双指针原地反转 `array_form` 后得到 `"olleh"`，而 `pointer_form` 指向的字面量仍然是 `"hello"`（一个字面量、一份数组，两处互不影响）。

- **转义序列 (escape sequences)**：字符串与字符字面量里，反斜杠引导"无法直接键入"的字符。

转义	含义	转义	含义
`\0`	NUL（字符串结束符）	`\\`	反斜杠本身
`\n`	换行 newline	`\'`	单引号
`\t`	水平制表符 tab	`\"`	双引号
`\r`	回车 carriage return	`\?`	问号
`\a`	响铃 alert	`\ooo`	1–3 位八进制（`\101` = `'A'`）
`\f`	换页 form feed	`\xhh`	1–2 位十六进制（`\x41` = `'A'`）

  - *直观解释*：反斜杠是"下一个字符按编码解释、不要按字面理解"的开关。`"a\0b"` 在内存里是 4 个字节 `'a' 0 'b' 0`——**中间的 NUL 让字符串提前结束**，`strlen` 只看到 1。
  - *作用域与存储期*：转义只是书写形式，不改变存储期；`'\0'` 是值为 0 的 `int` 字符常量。
- **字符串库 (string library)**：`<string.h>` 的函数都以 NUL 为界，**都不检查目标缓冲区大小**。`strlen (s)` 返回字符数（不含 NUL）；`strcpy (dst, src)` 连 NUL 一起复制；`strcat (dst, src)` 从 `dst` 的 NUL 处追加；`strcmp (s1, s2)` 逐字符比较，返回负/0/正（比的是**字符编码**，不是字典序）；`strncpy (dst, src, n)` 最多复制 n 字节，**源太长时不补 NUL**；`strchr (s, c)` 返回第一次出现的指针或 NULL。
  - *直观解释*：这些函数像"搬箱子工人"：你说搬多少就搬多少，它不知道你的仓库有多大。
  - *底层机制图解*：`strcpy` 的机器模型就是"读一字节、写一字节、见 0 才停"：`LOOP LDR R2,R1,#0 ; R2 ← *src STR R2,R3,#0 ; *dst ← R2 BRz DONE ; 连 NUL 一起复制完，循环结束 ADD R1,R1,#1 ADD R3,R3,#1 BRnzp LOOP` 幻灯片的缓冲区溢出攻击正是利用这一点：输入超出数组长度时，多余字节会覆盖保存的返回地址。

- *作用域与存储期*：这些函数不分配内存；目标缓冲区由调用者提供，其大小是调用者的责任 （`scanf("%19s", name)` 用字段宽度把这个责任"部分"交给格式串）。

- **二维数组是"数组的数组"，按行主序 (row-major) 存放**：`int32_t m[3][4]` 是"3 个元素、每个元素是 4 个 `int32_t`"。
  - *直观解释*：像 3 排 4 列的储物柜，物理上是一长排；先排完第 0 行再排第 1 行。
  - *底层机制图解*：地址公式 `&m[i][j] = base + (i * numCols + j) * sizeof(int32_t)`：`int32_t m[3][4]; 行主序展开（一行接一行） m[0][0] m[0][1] m[0][2] m[0][3] base+0 base+4 base+8 base+12 m[1][0] m[1][1] m[1][2] m[1][3] base+16 base+20 base+24 base+28 m[2][0] m[2][1] m[2][2] m[2][3] base+32 base+36 base+40 base+44 行跨距 (row stride) = numCols * 4 = 16 字节；总大小 = 3*4*4 = 48 字节` **`m[i]` 本身是一个数组**（类型 `int32_t[4]`），在表达式里退化为 `int32_t*`； 而 **`m` 退化为"指向一整行的指针"**，类型 `int32_t (*)[4]`——所以 `m + 1` 前进 16 字节，`m[0] + 1` 前进 4 字节。
  - *作用域与存储期*：48 字节是一个 automatic 对象，随栈帧生灭；行与行之间没有独立的生命周期。

- **三种"表格"写法**：`int32_t m[3][4]`、`int32_t* ap[3]`、`int32_t (*pa)[4]`。
  - *直观解释*：第一种是"一整排连着的仓库"；第二种是"3 张写着仓库地址的便条"； 第三种是"一张便条，并写明它指向一整排 4 格"。
  - *底层机制图解*：`int32_t m[3][4] int32_t* ap[3] int32_t (*pa)[4] （48 字节，连续） （24 字节的指针数组） （8 字节的一个指针） +----+----+----+----+ +----------+ +----------+ |行0 |行1 |行2 | | | ap[0] ---┼--→ r0[4] | pa ---------┼--→ m[0][0] +----+----+----+----+ | ap[1] ---┼--→ r1[4] +----------+ sizeof(m) = 48 | ap[2] ---┼--→ r2[4] sizeof(pa) = 8 +----------+ pa + 1 前进 16 字节 sizeof(ap) = 24 ap + 1 前进 8 字节（一个指针） 各行可以不相邻 (jagged / ragged)` 三者都能写 `x[i][j]`，但机器代价不同：`m[i][j]` 是"乘加 + 一次访存"（列数是编译期常量），`ap[i][j]` 是"先取指针再取元素"，**多一次指针追逐 (pointer chase)**。
  - *作用域与存储期*：`m` 的各行是同一对象的组成部分；`ap` 指向的各行可以是独立对象（各有各的存储期）； `pa` 只是一个指针，指向谁就属于谁的存储期。

- **字符串数组 `char* names[]`**：这是"指针数组"，每个元素指向一个字符串，字符本体在静态存储区。
  - *直观解释*：像通讯录，每页只写"名字在第几号柜子"。
  - *底层机制图解*：`char* names[4] = {"Ada", "Grace", "Linus", "Ken"}; names（栈上 32 字节） 静态数据区（只读，长度各不相同） +----------+ "Ada\0" "Grace\0" "Linus\0" "Ken\0" | names[0] -┼----------→ | names[1] -┼----------→ | names[2] -┼----------→ | names[3] -┼----------→ +----------+ sizeof(names) = 32, sizeof(names[0]) = 8 交换两个名字只需交换两个指针（16 字节），一个字符都不用搬`
  - *作用域与存储期*：指针数组常是 automatic，被指向的字符串是 static；若写成 `char names[4][16]`， 则 4×16 字节全在栈上（固定宽度、浪费空间，但内容可修改）。

- **参差不齐的表 (jagged / ragged table)**：长度不一致的"二维"数据要用"指针数组 + 逐行分配"。

- *直观解释*：课程 MP2 的课表是"15 个时段 × 5 天"的**指针数组**：每个格子存一个指向事件标签的指针（或 NULL 表示空闲）；课程 MP6 的棋盘 `space_type_t b[BOARD_HEIGHT][BOARD_WIDTH]` 则是**真正的二维数组**， 因为每行长度相同。
- *底层机制图解*： `MP2 课表（LC-3 视角）：数组的 15 个数组，每个数组 5 个指针 x3800 +----+----+----+----+----+ ← 第 0 个时段（周一..周五） | p | p |NULL| p | p | +----+----+----+----+----+ x3805 | p |NULL| p | p |NULL| ← 第 1 个时段 +----+----+----+----+----+ 行 r、列 c 的地址 = x3800 + r*5 + c（每格一个"字"，内容是指向事件标签的地址或 NULL）` 在 C 里同样可以写 `char* schedule[15][5];`（固定宽度），或逐行 `malloc` 得到可变的参差行。
- *作用域与存储期*：固定宽度表的存储期由数组本身决定；逐行 `malloc` 的表是 allocated，必须逐行 `free`（否则泄漏），通常要先把每行的指针保存好再释放外层数组。

代码示例与底层机制分析

示例 1：字符串库函数与 NUL 终止

代码 (C)（`/tmp/ece220_str/20t_string_library.c`，用 `gcc -g -std=c99 -Wall -Werror 20t_string_library.c -o 20t_string_library` 实测）：

```
#include <stdio.h>
#include <string.h>

int
main (void)
{
    char  buf[32];
    char  truncated[8];
    char const* found;

    /* strcpy copies the NUL terminator; strlen does not count it. */
    strcpy (buf, "Hello");
    strcat (buf, ", ");
    strcat (buf, "world");
    printf ("buf = \"%s\"\n", buf);
    printf ("strlen (buf) = %lu, bytes used including NUL = %lu\n",
            (unsigned long) strlen (buf),
            (unsigned long) strlen (buf) + 1);
    printf ("sizeof (buf) = %d (the array is 32 bytes, the string is not)\n",
            (int) sizeof buf);

    /* strcmp returns a negative, zero, or positive value. */
    printf ("strcmp (\"abc\", \"abc\") -> %d\n", strcmp ("abc", "abc"));
    printf ("strcmp (\"abc\", \"abd\") is %s\n",
            (0 > strcmp ("abc", "abd") ? "negative" : "not negative"));
    /* 'B' (0x42) < 'a' (0x61): ASCII order is not dictionary order. */
    printf ("strcmp (\"Zebra\", \"apple\") is %s (ASCII order!)\n",
            (0 > strcmp ("Zebra", "apple") ? "negative" : "positive"));

    /* strchr returns a pointer into the string, or NULL. */
    found = strchr (buf, 'w');
    if (NULL != found) {
        printf ("'w' is at index %lu; the rest is \"%s\"\n",
                (unsigned long) (found - buf), found);
    }
    printf ("strchr (buf, 'z') == NULL -> %d\n", NULL == strchr (buf, 'z'));

    /* strncpy copies AT MOST n bytes and does NOT terminate the result
     * when the source is too long -- you must terminate it yourself.    */
    strncpy (truncated, "truncated", sizeof truncated - 1);
    truncated[sizeof truncated - 1] = '\0';
    printf ("strncpy result = \"%s\" (length %lu)\n",
            truncated, (unsigned long) strlen (truncated));

    return 0;
}
```

实际输出:

```
buf = "Hello, world"
strlen (buf) = 12, bytes used including NUL = 13
sizeof (buf) = 32 (the array is 32 bytes, the string is not)
strcmp ("abc", "abc") -> 0
strcmp ("abc", "abd") is negative
strcmp ("Zebra", "apple") is negative (ASCII order!)
'w' is at index 7; the rest is "world"
strchr (buf, 'z') == NULL -> 1
strncpy result = "truncat" (length 7)
```

【代码做什么？】 1. `strcpy` 把 `"Hello"` 复制进 `buf`（含 NUL），`strcat` 两次追加得到 `"Hello, world"`。 2. `strlen` 返回 12：最后一个字符下标是 11，NUL 不计入；`sizeof buf` 是 32——**数组大小 ≠ 字符串长度**。 3. `"Zebra" < "apple"` 是因为 `'Z'`(0x5A) < `'a'`(0x61)，`strcmp` 比的是编码。 4. `strchr(buf, 'w')` 返回 `buf + 7`，指针相减得到下标；没找到时返回 NULL。 5. `strncpy` 只复制 7 字节且**不写 NUL**，所以代码手工补 `truncated[7] = '\0'`。

【底层机制透视】 `strlen` / `strcpy` / `strcmp` / `strchr` 全都依赖"某个字节为 0"这一约定——它们没有长度参数，必须靠扫描。 这意味着：若字符数组里**没有** NUL（例如用 `memcpy` 复制了整块数据），这些函数会一直读下去， 直到偶然碰到 0，这就是最常见的"字符串越界读"。`strncpy` 是少数带长度上限的函数，但语义别扭：源比 n 长时不补 NUL，源比 n 短时**用 NUL 填满**。

【内存布局图解】

```
buf（栈上 32 字节）
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+ ... +----+
|'H' |'e' |'l' |'l' |'o' |',' |' ' |'w' |'o' |'r' |'l' |'d' |'\0'| ?? |     | ?? |
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+ ... +----+
  0    1    2    3    4    5    6    7    8    9   10   11   12   ...      31
                                 ^
                                 └─ found = buf + 7（strchr 的结果）
strlen = 12（下标 0..11），sizeof = 32；buf[13..31] 从未被写，内容无意义。
```

【与汇编的对应】（LC-3：`strlen` 的循环就是"见 0 即止"）

```
; ---- strlen: R0 = 字符串首地址，返回长度放在 R1 ----
LEN     AND  R1,R1,#0          ; 计数清零
LENLP   LDR  R2,R0,#0          ; R2 ← *s
        BRz  LENDONE           ; 遇到 NUL 字节（0）就结束——这就是"字符串"的全部机制
        ADD  R1,R1,#1
        ADD  R0,R0,#1          ; 前进一个**字符**（1 字节 = 1 个 LC-3 字）
        BRnzp LENLP
LENDONE RET

; ---- strchr: R0 = 字符串，R2 = 目标字符，地址返回在 R3（找不到置 0）----
CHR     ADD  R3,R0,#0
CHRLP   LDR  R1,R3,#0
        BRz  CHRFAIL           ; 先到 NUL：没找到
        NOT  R1,R1
        ADD  R1,R1,#1          ; R1 = -(*s)
        ADD  R1,R1,R2          ; R1 = c - *s
        BRz  CHRDONE           ; 差为 0 → 找到
        ADD  R3,R3,#1
        BRnzp CHRLP
CHRFAIL AND  R3,R3,#0          ; 返回 NULL（全 0 位模式）
CHRDONE RET
```

> **演示（仅供演示、请勿模仿）：缓冲区溢出 (buffer overflow)** 幻灯片用 `char name[20]; scanf ("%s", name);` 说明"用户输入超过 19 个字符会怎样"： 多余字节顺着栈帧向上覆盖保存的返回地址，攻击者可以让 `RET` 跳到任意位置（乃至跳到刚写进栈的代码里）。 `c > char small[8]; > char big[64] = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdef"; > strcpy (small, big); /* UB: 向 8 字节的数组写入 53 字节 */ >` 实测（gcc 12.2.0，x86-64 Linux，未开 `-Werror`）：程序以**段错误（退出码 139）**结束。 更隐蔽的一例是 `strncpy (dst, "abcdefghijk", 8)`：实测 gcc 报 `warning: 'strncpy' output truncated copying 8 bytes from a string of length 11 [-Wstringop-truncation]` （加 `-Werror` 即编译失败），因为结果**没有 NUL 终止**。防御手段：`fgets`、带字段宽度的 `scanf ("%19s", ...)`、`strncpy` 后手工补 NUL——并记住幻灯片的话："依赖人来维护的防护措施本身就容易出错"。 **这些都是未定义行为，实际结果随编译器、优化级别与平台而异。**

示例 2：`int32_t m[3][4]` 的行主序布局与地址公式

代码 (C)（`/tmp/ece220_str/22t_two_d_arrays.c`）：

```
#include <stdint.h>
#include <stdio.h>

int
main (void)
{
    int32_t  m[3][4] = {{1, 2, 3, 4},
                        {5, 6, 7, 8},
                        {9, 10, 11, 12}};
    int32_t  i;
    int32_t  j;
    int32_t (*row_ptr)[4] = m;      /* pointer to an array of 4 ints */
    int32_t*  elem_ptr    = m[0];   /* pointer to a single int       */

    printf ("sizeof (m) = %d, sizeof (m[0]) = %d, sizeof (m[0][0]) = %d\n",
            (int) sizeof m, (int) sizeof m[0], (int) sizeof m[0][0]);
    printf ("rows = %d, columns = %d\n",
            (int) (sizeof m / sizeof m[0]),
            (int) (sizeof m[0] / sizeof m[0][0]));

    printf ("(char*) &m[1][0] - (char*) &m[0][0] = %ld bytes\n",
            (long) ((char*) &m[1][0] - (char*) &m[0][0]));
    printf ("m[1] - m[0] = %ld (in units of int, not bytes)\n",
            (long) (m[1] - m[0]));

    for (i = 0; i < 2; i++) {
        for (j = 0; j < 4; j++) {
            long offset = (long) ((char*) &m[i][j] - (char*) m);
            printf ("m[%d][%d]: offset %2ld = (i*4 + j)*4 = %2ld, value %d\n",
                    i, j, offset, (long) ((i * 4 + j) * 4), m[i][j]);
        }
    }

    printf ("(char*) (row_ptr + 1) - (char*) row_ptr = %ld bytes\n",
            (long) ((char*) (row_ptr + 1) - (char*) row_ptr));
    printf ("(char*) (elem_ptr + 1) - (char*) elem_ptr = %ld bytes\n",
            (long) ((char*) (elem_ptr + 1) - (char*) elem_ptr));
    printf ("(*(row_ptr + 1))[2] = m[1][2] = %d\n",
            (*(row_ptr + 1))[2]);

    printf ("m[2][3] = %d, *(*(m + 2) + 3) = %d, *(m[2] + 3) = %d\n",
            m[2][3], *(*(m + 2) + 3), *(m[2] + 3));

    return 0;
}
```

实际输出：

```
sizeof (m) = 48, sizeof (m[0]) = 16, sizeof (m[0][0]) = 4
rows = 3, columns = 4
(char*) &m[1][0] - (char*) &m[0][0] = 16 bytes
m[1] - m[0] = 4 (in units of int, not bytes)
m[0][0]: offset  0 = (i*4 + j)*4 =  0, value 1
m[0][1]: offset  4 = (i*4 + j)*4 =  4, value 2
m[0][2]: offset  8 = (i*4 + j)*4 =  8, value 3
m[0][3]: offset 12 = (i*4 + j)*4 = 12, value 4
m[1][0]: offset 16 = (i*4 + j)*4 = 16, value 5
m[1][1]: offset 20 = (i*4 + j)*4 = 20, value 6
m[1][2]: offset 24 = (i*4 + j)*4 = 24, value 7
m[1][3]: offset 28 = (i*4 + j)*4 = 28, value 8
(char*) (row_ptr + 1) - (char*) row_ptr = 16 bytes
(char*) (elem_ptr + 1) - (char*) elem_ptr = 4 bytes
(*(row_ptr + 1))[2] = m[1][2] = 7
m[2][3] = 12, *(*(m + 2) + 3) = 12, *(m[2] + 3) = 12
```

【代码做什么？】 1. 在三种粒度上取 `sizeof`：整体 48、一行 16、一个元素 4；用比值算出"3 行 4 列"。 2. 打印每行的字节间距（16）与"以 int 为单位"的间距（4）——同一个差值，**单位不同**。 3. 逐元素比较"实测字节偏移"与公式 `(i*4 + j)*4`，两列完全一致，验证行主序。 4. `row_ptr + 1` 前进 16 字节（跨一行），`elem_ptr + 1` 前进 4 字节（跨一个元素）——**类型决定步长**。 5. 最后一行用三种等价写法读出同一个元素：`m[i][j] ≡ *(*(m+i)+j) ≡ *(m[i]+j)`。

【底层机制透视】 `m[i][j]` 被编译成 `base + (i * numCols + j) * sizeof (element)`：一次乘法、一次加法、一次访存。 **列数 `numCols` 是编译期常量**，编译器可把 `i * 16` 优化为移位加法；这也解释了为什么 `m` 的类型必须是"指向 4 个 int 的数组的指针"——只有知道一行多长，才能算出第 i 行从哪里开始。 若换成"指针的指针"（`int32_t**`），编译器必须多做一次访存去读第 i 行的地址。

【内存布局图解】

```
m（栈上 48 字节，行主序，一行紧接一行）
偏移:  0    4    8   12   16   20   24   28   32   36   40   44
      +----+----+----+----+----+----+----+----+----+----+----+----+
      | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 |
      +----+----+----+----+----+----+----+----+----+----+----+----+
        └──── 第 0 行 ────┘└──── 第 1 行 ────┘└──── 第 2 行 ────┘
        m[0] = base+0      m[1] = base+16     m[2] = base+32
        m + 1    = base+16（m 的类型是 int32_t (*)[4]）
        m[0] + 1 = base+4  （m[0] 的类型是 int32_t*）
```

【与汇编的对应】（LC-3：base+offset 寻址正是二维数组的天然实现）

```
; 计算 &m[i][j]：m 在全局数据区，i 在 R1，j 在 R2
; int32_t 在 LC-3 上占 2 个字（4 字节），故偏移（以字计） = (i*4 + j)*2
        AND  R3,R3,#0
        ADD  R3,R3,#4          ; R3 = 4（列数）
        JSR  MULT              ; R0 = i * 4（LC-3 无乘法指令，用库子程序）
        ADD  R0,R0,R2          ; R0 = i*4 + j
        ADD  R0,R0,R0          ; R0 = 2 * (i*4 + j)  ← 每个 int32_t 占 2 个字
        LEA  R3,M_BASE         ; R3 = 数组首地址（全局对象 → LEA）
        ADD  R3,R3,R0          ; R3 = &m[i][j]        ← base + offset
        LDR  R2,R3,#0          ; R2 = m[i][j]（j 已并入 R0，R2 可以覆盖）
M_BASE  .BLKW 24               ; 3*4 个 int32_t = 24 个字
; 课程为简化常把 int 当作 16 位（1 个字）：此时偏移就是 (i*4 + j) 个字；
; 但 int32_t 在 C 里确实是 4 字节，所以真实代码要乘 2——这个"2 倍"正是
; "LC-3 按字寻址、C 按字节寻址"两个世界的换算。
```

示例 3：三种表格形式与字符串数组（用 `sizeof` 分辨）

代码 (C)（`/tmp/ece220_str/23s_row_forms.c`）：

```
#include <stdint.h>
#include <stdio.h>
#include <string.h>

int
main (void)
{
    int32_t  m[3][4]  = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int32_t  r0[4] = {1, 2, 3, 4};
    int32_t  r1[4] = {5, 6, 7, 8};
    int32_t  r2[4] = {9, 10, 11, 12};
    int32_t* ap[3] = {r0, r1, r2};      /* array of POINTERS */
    int32_t (*pa)[4] = m;               /* pointer to an array of 4 ints */
    char*    names[4] = {"Ada", "Grace", "Linus", "Ken"};
    int32_t  i;

    printf ("sizeof (m) = %d, sizeof (ap) = %d, sizeof (pa) = %d\n",
            (int) sizeof m, (int) sizeof ap, (int) sizeof pa);
    printf ("sizeof (m[0]) = %d, sizeof (ap[0]) = %d, sizeof (names) = %d\n",
            (int) sizeof m[0], (int) sizeof ap[0], (int) sizeof names);
    printf ("m[1] - m[0] = %ld (row stride in int units)\n",
            (long) (m[1] - m[0]));
    printf ("gap between ap[0] and ap[1] targets = %ld bytes\n",
            (long) ((char*) ap[1] - (char*) ap[0]));
    printf ("(*(pa + 2))[1] = %d, m[2][1] = %d\n",
            (*(pa + 2))[1], m[2][1]);

    /* Both forms index the same way at the source level. */
    printf ("m[1][2] = %d, ap[1][2] = %d, (*pa)[6] = %d\n",
            m[1][2], ap[1][2], (*pa)[6]);

    /* Modifying through ap changes the ROW ARRAY, not ap itself. */
    ap[1][0] = 50;
    printf ("after ap[1][0] = 50: r1[0] = %d, ap[1][0] = %d\n",
            r1[0], ap[1][0]);

    printf ("names:");
    for (i = 0; i < 4; i++) {
        printf (" %s(%lu)", names[i], (unsigned long) strlen (names[i]));
    }
    printf ("\n");
    {
        char* temp = names[0];
        names[0] = names[2];
        names[2] = temp;
    }
    printf ("after swapping names[0] and names[2]:");
    for (i = 0; i < 4; i++) {
        printf (" %s", names[i]);
    }
    printf ("\n(no characters were copied; only two pointers moved)\n");
```

```
    return 0;
}
```

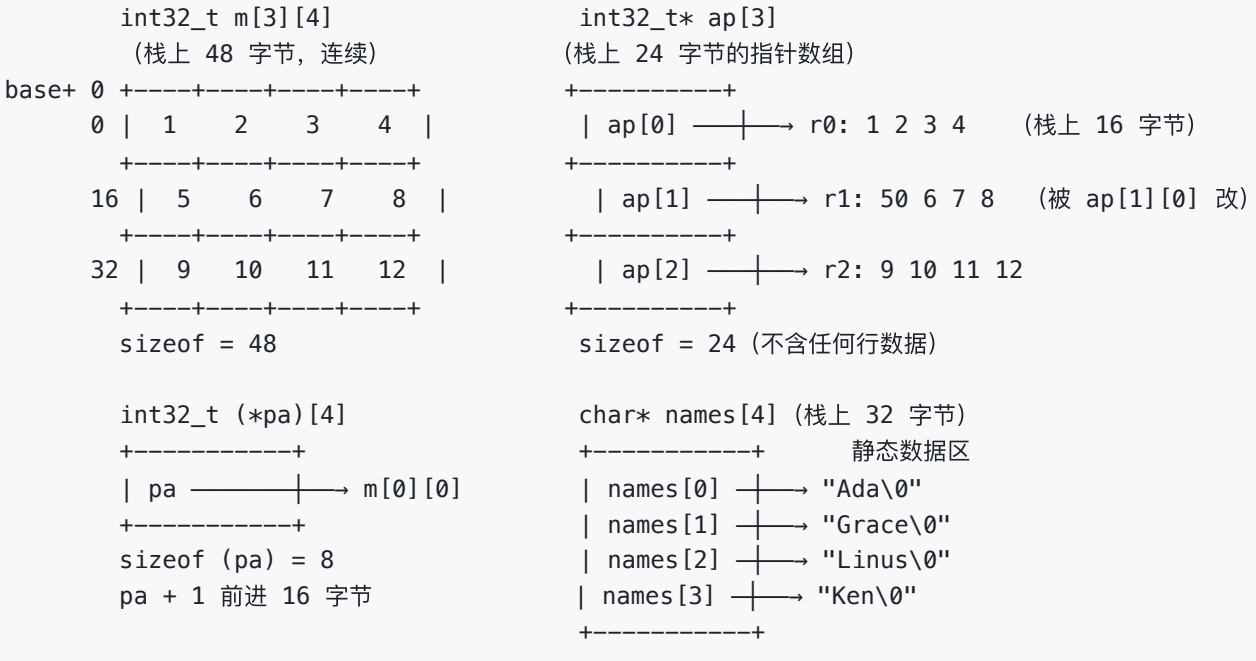
实际输出：

```
sizeof (m) = 48, sizeof (ap) = 24, sizeof (pa) = 8
sizeof (m[0]) = 16, sizeof (ap[0]) = 8, sizeof (names) = 32
m[1] - m[0] = 4 (row stride in int units)
gap between ap[0] and ap[1] targets = -16 bytes
(*(pa + 2))[1] = 10, m[2][1] = 10
m[1][2] = 7, ap[1][2] = 7, (*pa)[6] = 7
after ap[1][0] = 50: r1[0] = 50, ap[1][0] = 50
names: Ada(3) Grace(5) Linus(5) Ken(3)
after swapping names[0] and names[2]: Linus Grace Ada Ken
(no characters were copied; only two pointers moved)
```

【代码做什么？】 1. `sizeof m = 48`（二维数组）、`sizeof ap = 24`（指针数组：3×8）、`sizeof pa = 8`（一个指针）。 2. `m[1] - m[0] = 4`（元素单位），而 `ap` 指向的 `r0/r1/r2` 相距 16 字节且**地址递减**，所以打印 `-16`： 独立对象的相对位置由编译器决定，不能依赖。 3. `m[1][2] = ap[1][2] = (*pa)[6] = 7`：三种语法访问同一数据，`x[i][j]` 只是"加法 + 解引用"的语法糖。 4. `ap[1][0] = 50` 改的是 `r1[0]`（被指向的行），`ap` 本身（24 字节）没变。 5. 名字数组的"交换"只搬动两个指针，字符一个都没动——这正是"索引排序"的原理。

【底层机制透视】 `sizeof (m[0]) = 16` 是因为 `m[0]` 的类型是"4 个 int 的数组"，而 `sizeof (ap[0]) = 8` 是因为 `ap[0]` 是**指针**。 访问 `ap[i][j]` 需要两次访存（先读行地址再读元素），`m[i][j]` 只需一次——代价换来灵活性： `ap` 的各行可以是长度不同的独立数组（参差表），`m` 做不到。`char* names[4]` 同理：32 字节的指针表 指向静态区 4 个长度各不相同的字符串。

【内存布局图解】



【与汇编的对应】（LC-3：一次访存 vs 两次访存）

```
; ---- int32_t m[3][4]：算出地址后一次 LDR ----
        LEA  R3,M_BASE
        ADD  R3,R3,R0          ; R0 = (i*4 + j)*2 个字
        LDR  R2,R3,#0          ; R2 = m[i][j]        ← 1 次数据访存

; ---- int32_t* ap[3]：先取行指针，再取元素（多一次 pointer chase）----
        LEA  R0,AP_BASE
        ADD  R0,R0,R3          ; R3 = i（每个指针占 1 个字）
        LDR  R0,R0,#0          ; R0 = ap[i]    ← 第 1 次访存，得到某一行地址
        ADD  R2,R2,R2          ; R2 = 2*j（ap[i] 是 int32_t*，每个元素占 2 个字）
        ADD  R0,R0,R2          ; R0 = &ap[i][j]
        LDR  R1,R0,#0          ; R1 = ap[i][j] ← 第 2 次访存

; ---- char* names[4]：取出字符串地址后逐字符输出到 NUL ----
        LEA  R3,NAMES
        ADD  R3,R3,R1          ; R1 = i
        LDR  R0,R3,#0          ; R0 = names[i]（一个字符串地址）
        JSR  PRINT_STRING      ; 内部用 LDR + BRz 扫到 NUL
```

常见错误与调试技巧

- **忘记 NUL 终止**：用 `strncpy` 或手写循环复制后没补 `'\0'`，之后 `printf ("%s")` 一直越界读（乱码或崩溃）。 **调试**：`gcc -Wall` 报 `-Wstringop-truncation`；`gdb` 中 `x/16xb dst` 看有无 0 字节；`valgrind` 抓越界读。
- **混淆 `char s[]` 与 `char *s`**：对 `char* s = "...";` 执行 `s[0] = 'H'`。现象是段错误（只读段）。**调试**：打印 `(int) sizeof s`（6 还是 8）瞬间判定；`gdb` 中 `info proc mappings` 确认只读映射。
- **`scanf ("%s", ...)` 不带字段宽度**：超长输入冲垮相邻变量与返回地址。**调试**：改成 `scanf ("%19s", name)`（宽度 = 数组长度 − 1）；用 `gcc -fsanitize=address -g` 复现；`gdb` 的 `bt` 看崩溃是否发生在函数返回处。
- **字符串比较用 `==`，或二维数组下标写反**：前者比的是**地址**（改用 `strcmp`，"内容相同却判不等"即此）；后者越界却不报错。**调试**：`gdb` 中 `p s1` / `p s2` 看地址、`x/s s1` 看内容； 打印 `(char*)&m[i][j] - (char*)m` 与手算的 `(i*numCols+j)*4` 对照，或 `x/12dw m` 按内存顺序列出全部元素。
- **`int *m[3]` 与 `int (*m)[4]` 写错，或把二维数组传给 `int**` 参数**：前者一个是"指针数组"（24 字节）、一个是"指向数组的指针"（8 字节）；后者报 `incompatible pointer type` 或运行时崩溃。 **调试**：`gdb` 的 `ptype m` 直接读出真实类型，`(int) sizeof m` 对照预期；正确写法是 `int32_t (*p)[4] = m;`。

关键要点

- **C 字符串不是一种类型，而是一种约定**："一串连续 `char`，最后有一个 0"；所有字符串函数都靠扫描到 0 定界，忘记终止符就等于无限越界读。
- **`char s[]` 分配数组（可写、`sizeof` 是长度），`char *s` 只分配指针**（指向只读静态数据、`sizeof` 为 8）；要修改内容用数组，只引用常量用指针。

- **没有任何字符串函数会检查目标缓冲区大小**：`strcpy` / `strcat` / `scanf("%s")` 是缓冲区溢出的经典来源；要用带长度上限的手段，并记住 `strncpy` 不会自动补 NUL。
- **多维数组是"数组的数组"，按行主序连续存放**：`&m[i][j] = base + (i*numCols + j)*sizeof(element)`；`m` 退化为指向一整行的指针，所以 `m + 1` 跨一行，`m[0] + 1` 只跨一个元素。
- **参差数据要用"指针的数组"或"指针的指针"**：`T m[R][C]` 只能表示等长行；MP2 的课表（15×5 个字符串指针）与 MP6 的棋盘（固定大小的二维数组）分别是这两种需求的代表。

思考题（带答案）

问题 1：下面两段代码都得到"内容是 hello 的字符串"，它们在内存占用、可修改性、生命周期上有何不同?

```
char  a[] = "hello";        /* (1) */
char* p   = "hello";        /* (2) */
```

答案：(1) 在栈上分配 6 字节的**数组**，内容可写，`sizeof a = 6`，存储期到函数返回为止；(2) 只在栈上分配 8 字节的**指针**，字符串本体在静态数据区（通常只读），`sizeof p = 8`，存储期是整个程序。所以 `a[0] = 'H'` 合法，`p[0] = 'H'` 是未定义行为（常见结果是段错误）；反过来 `a = "bye"` 非法（数组名不可赋值），`p = "bye"` 合法。

问题 2：`int32_t m[3][4];` 中 `sizeof (m)`、`sizeof (m[0])`、`sizeof (m[0][0])` 各是多少？`m`、`m[0]`、`&m[0][0]` 的**值**与**类型**分别是什么?

答案：`48`、`16`、`4`。三者的值相同（都是数组首字节的地址），但类型不同：`m` 退化后是 `int32_t (*)[4]`，`m[0]` 退化为 `int32_t*`，`&m[0][0]` 也是 `int32_t*`。类型差异直接体现在步长上：`m + 1` 前进 16 字节，`m[0] + 1` 前进 4 字节。这也解释了为什么"用 `int32_t**` 接收二维数组"是错的。

问题 3：课程 MP2 的课表在 LC-3 上是"15 个数组，每个数组 5 个指针"，格子的地址是 `x3800 + r*5 + c`。 为什么这里每个格子只占**一个**内存字，而 C 里 `int32_t schedule[15][5]` 每个格子占 4 字节?

答案：LC-3 的格子里存放的是"指向事件标签的**指针**（地址）"，而 LC-3 的地址就是一个 16 位字，所以每格 1 个字； C 里 `int32_t` 是 4 字节，所以每格 4 字节——**格子的大小由格子里放的东西决定**，而不是由"数组"这个词决定。 遍历时用两层循环：外层 `r` 从 0 到 14、内层 `c` 从 0 到 4，每次 `LEA R2,SCHEDULE` 取基址、`ADD R2,R2,offset`（`offset = r*5 + c`）算出格子地址、`LDR R0,R2,#0` 取出内容，再用 `BRz` 判断是否为 NULL： 是 NULL 就打印空白，否则把 `R0` 当作字符串地址调用打印子程序。 （这里用 `R2` 而不是 `R4` 暂存格子地址：`R4` 是全局数据指针，按约定不能当临时寄存器用—— 见第 1 讲与第 3 讲的寄存器约定表。）

Lecture 12: 用指针与数组解决问题；函数指针与回调 (Problem Solving with Pointers and Arrays; Function Pointers and Callbacks)

概述

本讲把指针与数组从"机制"变成"工具"：就地 (in-place) 算法如何在不额外分配内存的前提下完成反转、分区、去重、旋转与查找， 以及数据长度事先未知时如何用"指针的指针"与 `realloc` 让被调用者改掉调用者的指针。 在此之上引入**函数指针 (function pointer)**：函数的入口地址也是一个值，可以赋给变量、放进数组当跳转表 (jump table)、 作为参数传给别的函数（回调，callback），于是同一个排序框架能对"任意类型"工作——这正是标准库 `qsort` 的设计。

核心概念与底层机制图解

- **就地算法 (In-place Algorithm)**：所有工作都在调用者提供的存储里完成，额外空间 O(1)。
  - *直观解释*：在原书架上重排书，而不是先搬到另一间屋子再搬回来。
  - *底层机制图解*：额外空间只体现为寄存器与一两个栈上的临时变量： `reverse (int32_t* d, int32_t n) 的机器模型 R1 = d（左指针） R2 = d + (n-1)*4（右指针） 循环：交换 M[R1] 与 M[R2], R1 += 4，R2 -= 4，直到 R1 >= R2` 对比"另分配一个数组再拷回"：额外空间 O(n)，还要 `malloc` / `free` 与失败处理。
  - *作用域与存储期*：只读写调用者的数组，**不引入新的存储期**；幻灯片强调 `malloc` 只有"没有内存"一个失败原因，能省就省。
- **双指针技术 (Two-Pointer Technique)**：两个指针从两端（或同向）扫描，把 $O(n^2)$ 的朴素解法降到 O(n)。
  - *直观解释*：两个人从书架两头向中间整理，比一个人反复从头找到尾快得多。
  - *底层机制图解*：三种典型形态——
    - **分区 (partition)**：`left` 停在第一个 ⩾ pivot 的元素、`right` 停在第一个 < pivot 的元素，交换后各进一步； 循环条件必须是 `left <= right`，否则落在同一元素上的那个值从未被归类。
    - **有序配对 (pair sum)**：`sum < target` 就 `left++`（左边需要更大），`sum > target` 就 `right--`；**依赖数组有序**。
    - **去重 (remove duplicates)**：读指针扫全数组，写指针仅在"值变化"时前进；写指针永不越过读指针： `已排序 {1,1,1,2,3,3,5,5,5,8}，w（写）与 r（读）同起点 r: 1 1 1 2 3 3 5 5 5 8 值与前一个不同 → *w = *r; w++ w: 1 2 3 5 8 w 前进 5 次 → 逻辑长度 5，数组本身没有变小`
  - *作用域与存储期*：两个指针只是 automatic 变量；数据仍是调用者的数组，结果留在原地，所以函数必须用返回值报告"新长度/边界"。
- **旋转与三反转技巧 (rotation; the three-reversal trick)**：设 `a = [A|B]`，三步得到 `[B|A]`。
  - *直观解释*：一摞牌分上下两半互换位置，做法是"各自翻面、再整体翻面"。
  - *底层机制图解*： `原始: A = 1 2 3 B = 4 5 6 7 8 反转 A: 3 2 1 | 4 5 6 7 8 反转 B: 3 2 1 | 8 7 6 5 4 全体反转: 4 5 6 7 8 1 2 3 ← 示例 2 的实测输出（每个元素恰好移动一次）` 先做 `k = k % n`

（`k=11`、`n=8` 等价于 `k=3`），`k <= 0` 直接返回。 实测片段（`/tmp/ece220_algo/41b_rotate_bsearch.c`，已编译运行，输出 `rotate_left (a, 8, 3): 4 5 6 7 8 1 2 3`）：

```
c static void rotate_left (int32_t* d, int32_t n, int32_t k) { k = (n > 0) ? k % n : 0; if (k <= 0) { return; } reverse (d, k); /* reverse the first block */ reverse (d + k, n - k); /* reverse the second block */ reverse (d, n); /* reverse everything */ }
```

  - *作用域与存储期*：只有整数 `k` 是 automatic；数组没有新增存储，也没有新的生命期。

- **查找：线性 vs 二分 (linear vs binary search)**。
  - *直观解释*：查纸质电话簿不会从第一页翻起（线性），而是翻开中间判断目标在前还是在后（二分）。
  - *底层机制图解*：二分每轮把区间**至少减半**，比较次数 O(log n)；但小数组上它未必赢，因为它每轮做两次比较 （判等 + 定方向）。实测（`/tmp/ece220_algo/31c_rotate_search.c`，数组 `{1,3,5,7,9,11,13,15}`）：`target linear(idx/cmp) binary(idx/cmp) 1 0 / 1 0 / 5 15 7 / 8 7 / 7 8 -1 / 8 -1 / 6` **正确性论证**：(1) *不变量*——若 `v` 存在，其下标始终在 `[low, high]` 内；`v < d[mid]` 时由有序性排除 `mid` 及其右侧，`[low, mid-1]` 仍满足不变量，反向同理。(2) *终止*——区间每轮至少减半，`low > high` 时为空， 由不变量知 `v` 不存在。(3) *溢出陷阱*——`mid = (low + high) / 2` 会溢出成负数，必须写 `low + (high - low) / 2`； 幻灯片指出标准库曾用错这个表达式二十多年。 实测的二分核心（同一文件；对 8 个元素的目标 1、15、8 分别返回下标 0、7、−1， 比较次数 5、7、6）：

```
c static int32_t binary_search (int32_t const* d, int32_t n, int32_t v, long* cmps) { int32_t low = 0; int32_t high = n - 1; int32_t mid; *cmps = 0; while (high >= low) { mid = low + (high - low) / 2; /* NOT (low + high) / 2 */ (*cmps)++; if (v == d[mid]) { return mid; } (*cmps)++; if (v < d[mid]) { high = mid - 1; } else { low = mid + 1; } } return -1; }
```

  - *作用域与存储期*：`low` / `high` / `mid` 都是 automatic；数组只读，参数用 `int32_t const*`。

- **指针的指针：`alloc (int32_t**)` 与 `realloc` 惯用法**。
  - *直观解释*：要让别人替你换掉写着门牌号的纸条，你得先把"放纸条的抽屉"告诉他。
  - *底层机制图解*：`T*` 的副本改不了调用者的指针，`T**` 才能；实测片段（`/tmp/ece220_algo/32c_alloc.c` 已编译运行：`a = 0x14fd2a0` 是堆地址，`&a = 0x7ffcd2c88500` 是栈地址）：

```
c static int32_t alloc_ints (int32_t** out, int32_t n) { int32_t* fresh = malloc ((size_t) n * sizeof (int32_t)); if (NULL == fresh) { *out = NULL; /* never leave a stale value */ return 0; } *out = fresh; /* writes the CALLER's pointer */ return 1; }
```

`realloc` 同理，但**必须用临时指针接住返回值**（失败时它返回 NULL 且不释放旧块）：`p = realloc (p, n);` 会丢失旧地址造成泄漏，正确写法是 `temp = realloc (p, n); if (NULL != temp) { p = temp; }`。
  - *作用域与存储期*：`*out` 是 **allocated** 存储期，由 `free` 结束；`free` 后把指针置 NULL，可让悬垂误用立刻暴露。

- **函数指针 (Function Pointer)**：函数入口地址是一个值，类型 = 返回值类型 + 参数列表。
  - *直观解释*：函数名像门牌号，函数指针是"把门牌号抄进变量"；通讯录里存号码，不存谈话内容。

- *底层机制图解*：幻灯片最关键的对比——`int (*f)(int)` 与 `int *f(int)` **完全不是一回事**：`c int32_t (*f) (int32_t); /* f 是指针：指向"int32_t → int32_t"的函数 */ int32_t* g (int32_t); /* g 是函数：接收 int32_t，返回 int32_t* */` 括号 `(*f)` 是分水岭：没有它，`*` 会与返回类型结合，变成"返回指针的函数"。 两条等价关系（示例 2 实测）：`fp = □` 与 `fp = square;` 同值；`(*fp) (7)` 与 `fp (7)` 都是 49。
- *作用域与存储期*：函数具有 **static 存储期**（代码段，程序全程存在），函数指针永不悬垂；指针变量自身是 automatic，占 8 字节。

- **函数指针数组 = 跳转表 (Jump Table)**：一组同签名函数地址放进数组，用下标选择行为。
  - *直观解释*：电视遥控器：按不同的键（下标）触发不同功能（函数）。
  - *底层机制图解*：这就是 LC-3"`JSRR` + 跳转表"的 C 版本：`static int32_t (*table[2]) (int32_t) = {&square, &negate}; +----------------------+ 调用 table[opcode] (x): | &square （代码地址） | LDR 取出地址 → 间接跳转 = 一次访存 + 一次间接调用 +----------------------+ | &negate （代码地址） | C 不检查下标：table[2] 会读到别的数据再跳过去（UB） +----------------------+`
  - *作用域与存储期*：带 `static` 的表是 static 存储期，程序启动即存在；函数内不加 `static` 则表在栈上，每次调用都要重新初始化。
- **回调 (Callback) 与泛型排序 (Generic Sort)**：把函数指针作为参数交给"框架函数"，框架在合适时机回头调用它。
  - *直观解释*：算法是"流程"，回调是"规则"；框架只管搬元素，谁大谁小由回调决定。
  - *底层机制图解*：`qsort` 的签名是标准形态：`c void qsort (void* base, size_t nmemb, size_t size, int (*compar) (const void*, const void*));` 框架要能移动任意大小的元素，所以 `base` 是 `void*` 而内部转成 `char*` 做字节算术：`a + j * size /* 第 j 个元素的地址：只有 char* 能按字节算 */ (*compar) (a + j * size, a + best * size) /* 问回调：谁在前？ */ swap_bytes (..., size) /* 逐字节交换，不知道类型也能搬 */` 回调的语义**必须写进文档**（幻灯片特别强调）：返回负数表示第一个参数"更小/在前"。
  - *作用域与存储期*：框架不分配内存（字节交换就地完成），没有失败路径；回调位于 static 代码段，调用它不影响框架的栈帧布局。

代码示例与底层机制分析

示例 1：双指针就地算法（反转、去重、有序配对）

代码 (C)（`/tmp/ece220_algo/40_two_pointer.c`，用 `gcc -g -std=c99 -Wall -Werror 40_two_pointer.c -o 40_two_pointer` 实测）：

```
#include <stdint.h>
#include <stdio.h>

/* Reverse n elements; return the number of swaps (n / 2). */
static int32_t
reverse (int32_t* d, int32_t n)
{
    int32_t* left  = d;
    int32_t* right = d + n - 1;
    int32_t  swaps = 0;

    while (left < right) {
        int32_t t = *left;
        *left++ = *right;       /* read right, write left, then advance */
        *right-- = t;
        swaps++;
    }
    return swaps;
}

/* Keep one copy of each value of a SORTED array; return the new length. */
static int32_t
dedupe (int32_t* d, int32_t n)
{
    int32_t* write = d;
    int32_t* read  = d;
    int32_t* end   = d + n;

    while (read != end) {
        if ((write == d) || (*read != *(write - 1))) {
            *write++ = *read;
        }
        read++;
    }
    return (int32_t) (write - d);   /* the write pointer IS the length */
}

/* In a SORTED array, find two elements summing to target. */
static int32_t
find_pair (int32_t const* d, int32_t n, int32_t target, int32_t* out)
{
    int32_t const* left  = d;
    int32_t const* right = d + n - 1;

    while (left < right) {
        int32_t sum = *left + *right;

        if (sum == target) {
            out[0] = *left;
            out[1] = *right;
            return 1;
        }
        if (sum < target) {
```

```
            left++;                 /* the left value must grow   */
        } else {
            right--;                /* the right value must shrink */
        }
    }
    return 0;
}

int
main (void)
{
    int32_t a[5] = {1, 2, 3, 4, 5};
    int32_t dup[10] = {1, 1, 1, 2, 3, 3, 5, 5, 5, 8};
    int32_t sorted[8] = {1, 3, 4, 7, 9, 11, 15, 20};
    int32_t pair[2];
    int32_t i;
    int32_t n;

    printf ("swaps performed = %d\n", reverse (a, 5));
    printf ("after reverse:");
    for (i = 0; i < 5; i++) {
        printf (" %d", a[i]);
    }
    n = dedupe (dup, 10);
    printf ("\nafter dedupe (length %d):", n);
    for (i = 0; i < n; i++) {
        printf (" %d", dup[i]);
    }
    if (find_pair (sorted, 8, 16, pair)) {
        printf ("\npair summing to 16: %d + %d\n", pair[0], pair[1]);
    }
    if (0 == find_pair (sorted, 8, 5, pair)) {
        printf ("no pair sums to 5\n");
    }

    return 0;
}
```

实际输出：

```
swaps performed = 2
after reverse: 5 4 3 2 1
after dedupe (length 5): 1 2 3 5 8
pair summing to 16: 1 + 15
```

【代码做什么？】 1. `reverse (a, 5)` 交换 2 次（`5/2`），数组变成 `5 4 3 2 1`；中间那个元素不用动。 2. `dedupe (dup, 10)` 在已排序的 10 个元素里留下 5 个不同值并返回 5：**数组没有变小**，只是逻辑长度变成 5。 3. `find_pair (sorted, 8, 16, pair)` 从两端出发扫描一次，找到 `1 + 15` 并写入调用者的 `pair[0..1]`。 4. `find_pair (sorted, 8, 5, pair)` 返回 0（不存在这样的两项），于是打印 "no pair sums to 5"。 5. 全程没有一次 `malloc`：函数只写调用者的数组，用返回值报告"发生了什么"。

【底层机制透视】 `*left++ = *right;` 的语义是"取 `*right` 作为右值、写入 `*left`、然后左指针自增"，一条 C 语句编译成"取数—存数—指针加 4"。`dedupe` 里 `write == d` 的特判避免读 `*(write-1)` 越界——用不变量守住边界比事后检查更可靠。`find_pair` 传入 `int32_t* out` 而不是返回结构体， 是为了让函数"返回两个值"（第二讲/第九讲讲过的指针参数的典型用途）。 `write - d` 得到的是**元素个数**（指针相减按元素大小缩放），它直接就是新的长度。

【内存布局图解】

```
dup 数组（栈上 10 个 int32_t）与两个指针（去重过程中）
下标:  0    1    2    3    4    5    6    7    8    9
     +----+----+----+----+----+----+----+----+----+----+
     | 1  | 1  | 1  | 2  | 3  | 3  | 5  | 5  | 5  | 8  |
     +----+----+----+----+----+----+----+----+----+----+
       ^w  ^r
       └──┘  r 前进；当 *r 与前一个写入值不同时 *w = *r 且 w++
       结束时：w 停在 d+5（元素单位），于是 return 5
     +----+----+----+----+----+----+----+----+----+----+
     | 1  | 2  | 3  | 5  | 8  | 3  | 5  | 5  | 5  | 8  |  ← 下标 5..9 是"垃圾但仍在数组内"
     +----+----+----+----+----+----+----+----+----+----+
       └─── 逻辑上有效的 5 个元素 ──┘
```

【与汇编的对应】（LC-3：双指针就是两个寄存器 + 一次比较）

```
; reverse (d, n)：R0 = d，R1 = n。约定 R2 = 左指针，R3 = 右指针，
; 临时量取自 R0/R1（d 与 n 用完就空闲了），R4~R7 一律不能当临时寄存器
        ADD  R2,R0,#0          ; R2 = d         （左指针）
        ADD  R3,R1,#-1
        ADD  R3,R0,R3          ; R3 = d + n - 1（右指针）
REVLOOP
        NOT  R0,R3             ; 比较 R2 与 R3；R2 >= R3 就结束
        ADD  R0,R0,#1
        ADD  R0,R2,R0          ; R0 = R2 - R3
        BRzp REVDONE
        LDR  R0,R2,#0          ; R0 = *left
        LDR  R1,R3,#0          ; R1 = *right
        STR  R1,R2,#0          ; *left  = 旧 *right
        STR  R0,R3,#0          ; *right = 旧 *left
        ADD  R2,R2,#1          ; left++（LC-3 上一个 int 槽 = 一个字）
        ADD  R3,R3,#-1         ; right--
        BRnzp REVLOOP
REVDONE RET
; 提示：R4 = 全局数据指针、R5 = 帧指针、R6 = 栈指针、R7 = 返回地址，它们各有
; 固定职责，绝不能像上面 R0/R1 那样拿来当临时量。另外 R0~R3 是调用者保存的：
; 若 reverse 还要调用别的子程序，返回后 R0~R3 可能已被改写，
; 必须先按调用约定把它们压栈保存。
```

示例 2：函数指针、跳转表、回调与泛型排序

代码 (C)（`/tmp/ece220_algo/42b_fptrs.c`）：

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char     name[16];
    int32_t  score;
} player_t;

static int32_t square (int32_t x) { return x * x; }
static int32_t negate (int32_t x) { return -x; }

/* A function that RETURNS a pointer: note where the '*' sits. */
static int32_t*
identity (int32_t* p)
{
    return p;
}

/* An operation chosen at run time by an opcode: a jump table. */
static int32_t
apply_op (int32_t opcode, int32_t x)
{
    static int32_t (*table[2]) (int32_t) = {&square, &negate};

    if (0 > opcode || 1 < opcode) {
        return 0;                       /* the bound check is OURS to do */
    }
    return (*table[opcode]) (x);        /* like LC-3 JSRR through a table */
}

/* Exchange two elements of size bytes without knowing their type. */
static void
swap_bytes (char* x, char* y, size_t size)
{
    size_t k;

    for (k = 0; k < size; k++) {
        char t = x[k];
        x[k] = y[k];
        y[k] = t;
    }
}

/* mysort -- a generic sort with the same parameters and meaning as qsort:
 * base is the array address, nmemb the element count, size the bytes per
 * element, compar a callback returning <0, 0, or >0 for (first, second). */
static void
mysort (void* base, size_t nmemb, size_t size,
        int (*compar) (const void*, const void*))
{
    char*  a = base;            /* byte arithmetic needs char* */
```

```
    size_t i;
    size_t j;

    for (i = 0; i + 1 < nmemb; i++) {
        size_t best = i;        /* index of the smallest remaining */
        for (j = i + 1; j < nmemb; j++) {
            if (0 > (*compar) (a + j * size, a + best * size)) {
                best = j;
            }
        }
        if (best != i) {
            swap_bytes (a + i * size, a + best * size, size);
        }
    }
}

static int
cmp_score_desc (const void* a, const void* b)
{
    player_t const* p1 = a;         /* restore the real type */
    player_t const* p2 = b;

    if (p1->score != p2->score) {
        return (p1->score > p2->score) ? -1 : 1;
    }
    return strcmp (p1->name, p2->name);
}

static void
show (char const* label, player_t const* p, size_t n)
{
    size_t i;

    printf ("%s:", label);
    for (i = 0; i < n; i++) {
        printf (" %s/%d", p[i].name, p[i].score);
    }
    printf ("\n");
}

int
main (void)
{
    int32_t  value = 21;
    int32_t  (*fp) (int32_t) = &square;
    int32_t  (*named) (int32_t) = square;   /* the same address */
    int32_t* ip = identity (&value);
    player_t players[3] = {{"Ada", 91}, {"Grace", 88}, {"Linus", 91}};
    player_t copy[3];

    printf ("(&square == square) -> %d, (*fp) (7) = %d, fp (7) = %d\n",
            fp == named, (*fp) (7), fp (7));
    printf ("sizeof (fp) = %d, sizeof (ip) = %d, sizeof (identity (&value))"
            " = %d (return type is int32_t*)\n",
```

```
            (int) sizeof fp, (int) sizeof ip, (int) sizeof identity (&value));
    fp = &negate;                   /* re-point it at run time */
    printf ("after fp = &negate: fp (7) = %d, *ip = %d\n", fp (7), *ip);
    printf ("apply_op (0, 6) = %d, apply_op (1, 6) = %d, apply_op (2, 6) = %d\n",
            apply_op (0, 6), apply_op (1, 6), apply_op (2, 6));

    show ("original", players, 3);
    mysort (players, 3, sizeof (player_t), &cmp_score_desc);
    show ("mysort by score", players, 3);

    memcpy (copy, players, sizeof players);
    qsort (copy, 3, sizeof (player_t), &cmp_score_desc);
    printf ("library qsort agrees with mysort -> %d\n",
            0 == memcmp (players, copy, sizeof players));

    return 0;
}
```

实际输出:

```
(&square == square) -> 1, (*fp) (7) = 49, fp (7) = 49
sizeof (fp) = 8, sizeof (ip) = 8, sizeof (identity (&value)) = 8 (return type is int32_t*)
after fp = &negate: fp (7) = -7, *ip = 21
apply_op (0, 6) = 36, apply_op (1, 6) = -6, apply_op (2, 6) = 0
original: Ada/91 Grace/88 Linus/91
mysort by score: Ada/91 Linus/91 Grace/88
library qsort agrees with mysort -> 1
```

【代码做什么？】 1. `fp == named` 为 1：`&square` 与裸函数名 `square` 得到同一地址；`(*fp) (7)` 与 `fp (7)` 都是 49。 2. `sizeof (fp) = 8`（函数指针也是 8 字节地址），`sizeof (identity (&value)) = 8` 是**调用结果**（`int32_t*`）的大小。 3. `fp = &negate` 后 `fp (7) = -7`：同一个指针变量在运行期换成另一个函数。 4. `apply_op (0, 6) = 36`、`apply_op (1, 6) = -6`（跳转表命中），`apply_op (2, 6) = 0` ——**越界时返回 0 是我们自己写的边界检查救的场**。 5. `mysort` 按分数降序排好 3 个结构体；`qsort` 得到逐字节相同的结果（`memcmp` 为 0），证明"用回调实现的泛型接口"与标准库语义一致。

【底层机制透视】 `mysort` 把 `void* base` 转成 `char* a`：**只有 `char*` 能做字节算术**，`a + j * size` 就是第 `j` 个元素的地址，`memcpy` / `swap_bytes` 按字节搬移，框架完全不知道类型。比较交给回调 `compar`，它接收**元素的地址**而不是元素本身（`const void*`），所以 `cmp_score_desc` 必须做类型还原 `player_t const* p1 = a;`。`sizeof (player_t) = 20`（16 字节名字 + 4 字节分数）——每次交换搬 20 字节，这就是"泛型"的代价：框架不知道类型，只能逐字节处理。`qsort` 的 `size` 参数正是为此存在。

【内存布局图解】

```
跳转表（static）
table （2 × 8 = 16 字节）
+----------------------+
| &square  （代码地址） |  ← table[0] (6) = 36
+----------------------+
| &negate  （代码地址） |  ← table[1] (6) = -6
+----------------------+
  table[2] 越界 → 读到别的数据再跳过去（UB）
```

```
mysort 中的字节地址算术
 a ──→ +----+----+----+ ... +----+
       |   第 0 个元素（20 字节） |
       +----+----+----+ ... +----+
       |   第 1 个元素           |
       +----+----+----+ ... +----+
       |   第 2 个元素           |
     +----+----+----+ ... +----+
      a + j * 20 = 第 j 个元素的地址
```

【与汇编的对应】（LC-3：跳转表 + `JSRR`）

```
; apply_op (opcode, x)：R0 = opcode，R1 = x
        LEA  R2,TABLE          ; R2 = 跳转表首地址（标号汇编期已知 → LEA）
        ADD  R2,R2,R0          ; R2 = &TABLE[opcode]（每项 1 个字）
        LDR  R3,R2,#0          ; R3 = 函数入口地址 ← 从表里取出"函数指针"
        ADD  R0,R1,#0          ; R0 = x，作为被调用函数的参数（R0–R3 传参）
        JSRR R3                ; 间接跳转：跳到 R3 所指的代码  ← C 的 (*table[opcode]) (x)
        RET                    ; 返回值在 R0 中，直接交还调用者

TABLE   .FILL SQUARE           ; 表项就是函数的入口地址
        .FILL NEGATE
SQUARE  ; ... 计算 R0 * R0 ...
        RET
NEGATE  NOT  R0,R0
        ADD  R0,R0,#1
        RET
; 通用排序（回调）在 LC-3 上的做法完全相同：调用者把"比较子程序的地址"
; 作为参数压栈传进去，排序框架用 LDR 取出该地址再用 JSRR 调用 —— 这就是汇编里的回调。
```

> **演示（仅供演示、请勿模仿）：悬垂指针与同一块内存被释放两次** `free` 之后的指针仍然保存着旧地址，它指向的存储已经不属于本程序：`c > int32_t* p = malloc (4 * sizeof (int32_t)); > for (i = 0; i < 4; i++) { p[i] = i + 1; } > free (p); > printf ("%d\n", p[0]); /* UB：读已释放的存储 */ > free (p); /* UB：二次释放，可能摧毁分配器的簿记结构 */ >` 实测（gcc 12.2.0，x86-64 Linux，加 `-Wall` 但不加 `-Werror`）：编译器报 `warning: pointer 'p' used after 'free' [-Wuse-after-free]`；加上 `-Werror` 后**根本编译不过**。 若去掉警告直接运行，读出的往往是分配器留下的簿记数据而不是原来的 `1`； 用 `valgrind --leak-check=full ./prog` 会明确报出 invalid read 与 double free。 **这是未定义行为，实际结果随编译器、优化级别与平台而异**，不要把它当成"总能读到旧值"来推理。

常见错误与调试技巧

- **`realloc` 的返回值直接写回原指针**：`p = realloc (p, n)`，失败时旧块地址丢失造成泄漏。 **调试**：`valgrind --leak-check=full --track-origins=yes ./prog` 会报 definitely lost；改用临时指针接住返回值并检查 `NULL`。

- **`free` 之后继续使用指针或重复 `free`**：读/写已释放的存储、二次释放。 **调试**：`gcc -Wall` 报 `-Wuse-after-free`；`gcc -fsanitize=address -g` 或 `valgrind` 直接报 invalid read/write 与 double free。
- **函数指针声明写错**：把 `int (*f)(int)` 写成 `int* f(int)`，或在数组声明里漏括号写成 `int* table[2](int)`（非法）。 **调试**：`gdb` 的 `ptype f` 读出真实类型；打印 `(int) sizeof f`（8 说明是指针）；逐字读编译器的错误信息。
- **跳转表越界**：`table[opcode]` 没检查 `opcode` 范围，会跳到随机地址或调用错误函数。 **调试**：调用前加范围检查（示例 2 就是这么做并实测到 `apply_op (2, 6) = 0`）； `gdb` 中 `x/2gx &table` 看表内容、`p opcode` 看下标；用 `-fsanitize=address` 捕获越界。
- **二分中点溢出**：`mid = (low + high) / 2` 在大数组上溢出成负下标。 **调试**：改用 `low + (high - low) / 2`；`gdb` 里 `watch mid` 观察是否出现负值；`-fsanitize=undefined` 捕获有符号溢出。
- **把两个"有副作用"的调用塞进同一个 `printf` 参数表**：参数求值顺序未定义， 计数器可能在被写入前被读出（本讲早期版本实测打印出 `0 / 524288`）。**调试**：拆成独立语句分别赋值再打印。

关键要点

- **就地算法用 O(1) 额外空间完成变换**：交换是基本功，双指针是主力（分区、有序配对、去重）， 三反转法用 n 次移动完成旋转；共同点是结果留在调用者的数组里，函数用返回值报告新长度或边界。
- **二分的正确性来自不变量**（目标若存在则下标在 `[low, high]`），每次比较丢弃一半区间；中点必须写 `low + (high - low) / 2`，否则加法会溢出成负下标。
- **要改调用者的指针就传"指针的地址"**（`void alloc (int32_t** p)`）； `realloc` 必须用临时指针接返回值， 失败时它返回 NULL 且不释放旧块， `ptr = realloc (ptr, n)` 是经典的泄漏写法。
- **函数指针让函数成为数据**：`int (*f)(int)` 与 `int *f(int)` 的区别全在括号； `&func` 与 `func` 等价、 `(*f)(x)` 与 `f(x)` 等价；函数指针数组就是跳转表（LC-3 里对应 `LEA` + `LDR` + `JSRR`），下标边界要自己检查。
- **回调是"把规则交给框架"**：`qsort` 式签名让同一个排序框架服务任意类型，框架按字节搬移元素（`size` 参数因此必需）， 回调负责把 `void*` 还原成真实类型；其返回值的语义必须写进文档。

思考题（带答案）

问题 1：`partition` 的循环条件为什么是 `left <= right` 而不是 `left < right`？请举例说明差别。

答案：因为当两个指针落在**同一个元素**上时，这个元素还没被归类。以 `{9, 1, 3}`、`pivot = 5` 为例：`left` 指向 9（≥ pivot），`right` 指向 3（< pivot），若用 `<`，当 `left == right`（都指向 3）时循环提前退出， 中间的 `3` 既没被移走也没被计数，返回的 `cut` 就夸大了"小于 pivot 的元素个数"。 改用 `<=` 时，两指针在同一元素上仍会执行一次判断与归类，返回的 `cut` 恰好是"真正小于 pivot 的元素个数"。 一般规则：**当循环体可能改变当前元素所属的一侧时，必须让指针在同一元素上走完一次逻辑**。

问题 2：下面两个声明分别是什么？ `sizeof` 各是多少？怎样一句话区分？

```
int32_t (*f) (int32_t);
int32_t*  g (int32_t);
```

答案：`f` 是**函数指针变量**（指向"接收 `int32_t`、返回 `int32_t`"的函数），`sizeof (f) = 8`； `g` 是**函数**（接收 `int32_t`、返回 `int32_t*`），函数名没有"大小"，`sizeof (g (3)) = 8` 只是调用结果（指针）的大小。 一句话：**看括号**——`(*f)` 表明被声明的标识符 `f` 是指针；`g (int32_t)` 中标识符后面直接跟参数表，表明 `g` 是函数。 所以 `fp = □` 合法（给指针赋值），而 `g = □` 非法（给函数名赋值）。

问题 3：为什么 `mysort`（以及 `qsort`）必须接收 `size` 参数？去掉它会怎样？

答案：泛型排序**不知道元素类型**，只能用字节地址与字节偏移定位元素： `a + j * size` 才是第 `j` 个元素的地址，`swap_bytes (..., size)` 才知道要搬多少字节。 若没有 `size`，框架只能假设每个元素 1 字节，那么对 `int32_t` 数组第二个元素会算成"首地址 + 1"而不是 "+4"， 结果完全错乱；对 20 字节的 `player_t` 更是灾难。 反过来，`size` 也解释了回调为什么接收**元素的地址**而不是元素本身：框架只能说"两个元素在哪里"， 由回调按自己的类型去解释那 20 个字节——这正是 `const void*` 参数与 `player_t const* p1 = a;` 这种类型还原存在的原因。

Lecture 13: 递归 (Recursion)

概述

本讲要解决的问题是：**如何用"函数调用自己"来描述那些天然自相似的问题**——斐波那契、迷宫、汉诺塔、 二分查找、树的遍历。引入的机制是**递归函数 (recursive function)**：由**基本情况 (base case)** 与 **递归情况 (recursive case)** 构成，每一次调用在运行时都对应一次真实的 `JSR` 与一个**新栈帧 (stack frame)**。 递归把 ECE 120 的栈抽象与调用约定同"分而治之"思维缝合在一起：向上承接栈帧与调用约定， 向下开启基于指针的树、链表以及归并/快速排序。

核心概念与底层机制图解

- **递归函数 (Recursive Function)**：在自身定义中调用自己，把大问题化成同形的更小问题。
  - *直观解释*：俄罗斯套娃——打开一层，里面还是同样形状、只是更小；最小的那层打不开，就是基本情况。
  - *底层机制图解*：递归在机器层面**没有任何新指令**，它就是 `JSR`（返回地址存入 `R7`）加栈操作； 每次调用压入一个新帧，放参数、局部变量、返回地址与返回值槽。于是 "递归深度 = 同时存活的帧数"，"递归成本 = 帧大小 × 深度"。
  - *作用域与存储期*：帧内局部变量是 automatic storage duration，每次调用重新创建、地址各异、随帧销毁； 这正是递归成立的关键——第 4 层的 `n` 与第 1 层的 `n` 是两个不同单元。若声明为 `static` （static storage duration），所有层共享同一单元，递归立刻出错。
- **基本情况与递归情况 (Base Case / Recursive Case)**：前者给出最小问题的答案并停止递归，后者把问题缩小后交给自己。
  - *直观解释*：上楼梯——"再上一级"是递归情况，"到一楼就不再往上问"是基本情况。

    - *底层机制图解*：二者的分界就是"不再执行 `JSR` 的那条分支"。课程模板是 **①检查停止条件 ②处理当前结点 ③处理子结点**；②与③可互换，互换后得到先序/后序两种顺序 （`print_reverse` 正是靠它把字符串反着打印）。
    - *作用域与存储期*：停止条件必须保证参数向基本情况**收敛**，否则栈持续向低地址增长， 越过栈区边界后进程被操作系统以段错误终止。
- **递归树 (Recursion Tree)**：每次调用画成一个结点，其子结点是它发起的调用。
    - *直观解释*：家族树——每个"人"下面挂着它生出的所有"孩子"。
    - *底层机制图解*：树的形状直接决定代价：链状树是线性时间；带大量重复子树的二叉树（朴素 Fibonacci） 是指数时间；完全二叉树（汉诺塔）有 2^n 个结点。必须区分两个量： **树高 = 栈帧峰值 = 内存占用，结点总数 = 调用次数 = 时间开销**。
    - *作用域与存储期*：同层不同分支的结点互相不可见（局部变量各在帧里）， 这正是回溯算法退出分支后能"自动恢复现场"的原因。
- **返回阶段 / 回退 (Unwinding)**：基本情况返回后控制权逐层回到调用者，每层继续执行调用点之后的语句。
    - *直观解释*：往里走时把待办事项写在便签上贴墙，走到尽头后往回走，一张张撕下来照做。
    - *底层机制图解*：`RET` 把 `R7` 装回 `PC`，同时复位 `R6` / `R5`，该帧内存随即失效（不是清零，只是不再属于你）。 `print_reverse` 靠回退阶段完成工作：**打印发生在递归调用返回之后**，所以字符被反序输出。
    - *作用域与存储期*：回退后被调用帧的局部变量存储期结束，因此返回指向本帧局部变量的指针（`char* f(void){ char buf[10]; return buf; }`）是经典的悬空指针错误。
- **递归 vs 迭代 (Recursion vs Iteration)**：
    - *直观解释*：递归像贴一叠便签再逐张撕下，迭代像用一张便签反复改写。
    - *底层机制图解*：递归把进度信息交给**栈帧**（编译器管理），迭代把它放进**循环变量**（程序员管理）。 递归代码更短、更贴近数学定义，但有帧建立/拆除开销且深度受栈限制；迭代没有帧开销， 但表达树形结构与回溯时往往要手写显式栈。
    - *作用域与存储期*：递归的中间状态随帧自动生灭，迭代的中间状态由你负责初始化—— 忘记重置循环变量正是"脏状态"错误的来源。
- **尾递归 (Tail Recursion)**：递归调用是整个函数的最后一个动作（`return f(...);`，返回后不再计算）。
    - *直观解释*：接力赛——棒子交出去后自己就可以离场，不必站在原地等结果。
    - *底层机制图解*：调用者的帧此后不再被使用，编译器可**复用当前帧**，把 `JSR 自己` 改成 `BRnzp 函数入口` （尾调用优化, tail-call optimization），栈深度由 O(n) 降到 O(1)。课程的二分查找就是尾递归。
    - *作用域与存储期*：是否优化取决于编译器与优化级别，`-O0` 下不优化，深度仍受栈限制。 随附的 `r8_tail_opt.c` 让一个尾递归函数递归**一千万层**，三种编译方式的实测结果：

编译方式	实测结果
`gcc -g -std=c99` （即 `-O0`）	段错误，退出码 139

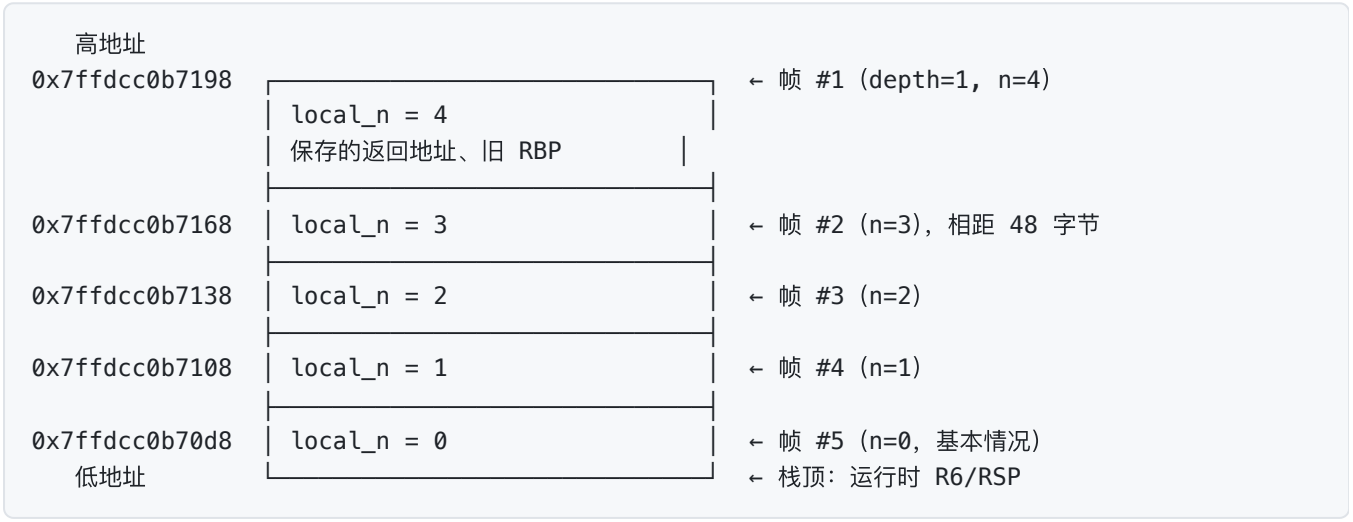
编译方式	实测结果
`gcc -O1`	正常输出 `count_down (10000000) = 10000000`，退出码 0
`gcc -O2 -fno-optimize-sibling-calls`	段错误，退出码 139

`objdump -d` 显示 `-O0` 版本里是一条**自己调用自己的 `callq`**（真的压帧），而 `-O1` 版本已经没有自己的帧，改用 `jne` **跳回函数开头**：`text -O0: 401159: callq 401126 <count_down> ← 一千万层把 8 MiB 栈撑爆 -O1: 40112c: jne 40112f <count_down+0x9> ← 尾调用优化：回跳，不压帧` **结论：尾递归是否省栈取决于编译器是否做尾调用优化，`-O0` 下不省。** 不要指望"我写的是尾递归所以不会爆栈"。

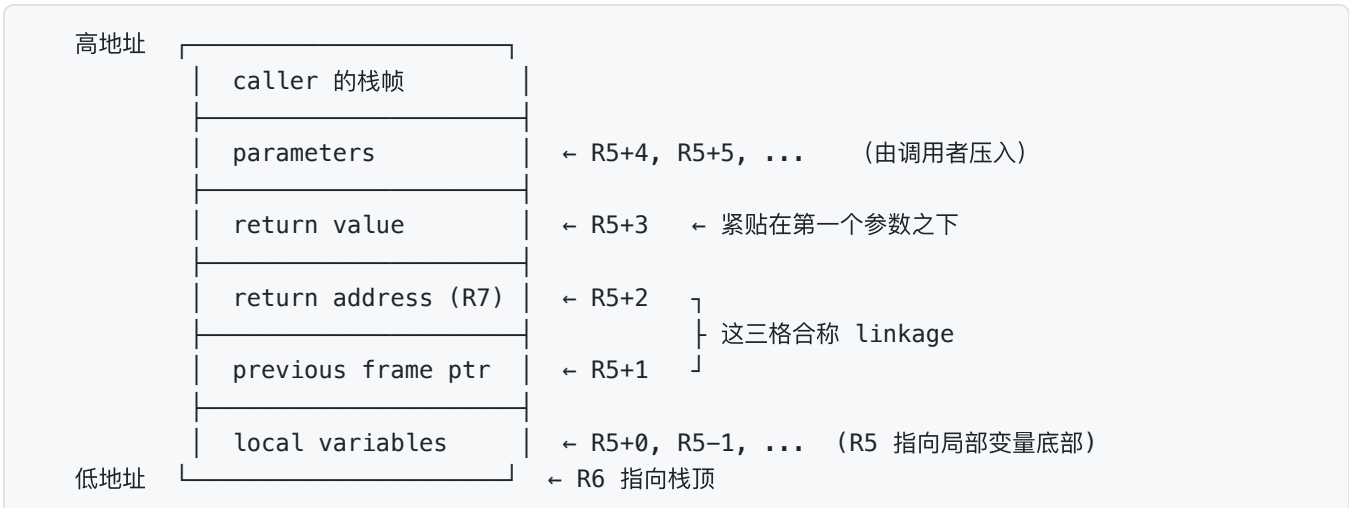
- **数组与字符串上的递归 (Recursion over Arrays and Strings)**：把"数组的其余部分"当作子问题。
  - *直观解释*：数一串珠子——"这一颗 + 剩下那一串的数目"，剩下那一串空了就返回 0。
  - *底层机制图解*：数组名传参时**退化为指向首元素的指针**，所以递归靠 `a + 1` 前进一格，用 `n - 1` 表示"还剩几个"。课程原版 `print_reverse` 是"**先递归、回来时打印**"，因此字符从最后一个开始输出；二分查找每次把区间折半（9 个元素最多 4 层），两次递归调用都直接 `return`，属于尾递归。随附的 `r2c_array_recursion.c` 实测输出 `woN` / `length = 3` / `sum_tail = 360` / `find 31 -> index 4` / `find 60 -> index 7` / `find 7 -> index -1`。
  - *作用域与存储期*：数组元素本身不在帧里（它们在调用者的帧或堆上），递归传递的只是**指针**；累加器版本（`sum_tail (a+1, n-1, acc + a[0])`）把"进度"放进参数，使帧可以被复用，是尾递归的典型写法。
- **互递归 (Mutual Recursion)**：A 调用 B，B 又调用 A。
  - *直观解释*：两人轮流接话，一句问一句答，直到某句"答完了"为止。
  - *底层机制图解*：机器层面与普通递归相同，只是调用图成环；必须至少有一处基本情况，并在使用前**前向声明 (forward declaration)** 另一个函数，否则编译器解析函数体时不知道它的原型。例如 `static int32_t is_odd (int32_t n); static int32_t is_even (int32_t n) { if (0 == n) { return 1; } return is_odd (n - 1); }`，`is_odd` 对称地返回 `is_even (n - 1)`；实测 `is_even(4) = 1, is_odd(4) = 0`，`is_even(10)` 的递归深度为 **11 帧**。
  - *作用域与存储期*：两个函数都用文件作用域 `static`（否则要在头文件里声明）；深度仍由栈决定。
- **回溯 (Backtracking)**：尝试一个候选 → 递归求解剩余问题 → 失败就**撤销这次尝试**换下一个。
  - *直观解释*：走迷宫时在岔路口画粉笔记号；走进死胡同就退回最近一个还有未试方向的岔路口。
  - *底层机制图解*：核心是"**修改 → 递归 → 撤销**"三步。撤销之所以必要，是因为解的状态通常放在 文件作用域数组（所有层共享）里；标记数组 `found` 同时充当①已访问集合 ②停止条件 ③输出结果。
  - *作用域与存储期*：共享状态（`found`、`col[]`）必须是 static storage duration 或显式传指针；只属于单次调用的状态才放帧里。哪些要撤销、哪些要累计，是回溯最容易搞错的地方。
- **分治 (Divide and Conquer)**：把问题切成若干同形子问题，分别求解再合并。
  - *直观解释*：整理扑克牌——分成两摞各自理好，再并成一摞（归并排序）。

- *底层机制图解*：汉诺塔的 `T(n) = 2T(n-1) + 1` 解出 `T(n) = 2^n - 1`； 归并排序的 `T(n) = 2T(n/2) + O(n)` 解出 `O(n log n)`。前者递归树"窄而深"，后者"宽而浅"，代价天差地别。 汉诺塔的函数体只有三行（`hanoi(n-1,from,via,to)` → 搬第 n 个盘子 → `hanoi(n-1,via,to,from)`），递推式直接从"2 次递归 + 1 次搬动"读出。`r3_hanoi.c` 的实测结果为： `n = 3` 时打印出 7 步且 `moves counted = 7` （与 `2^3 - 1` 吻合）， `n = 1, 4, 10, 20` 分别对应 `1, 15, 1023, 1048575` 步。 递归树是完全二叉树：结点总数 `2^(n+1) - 1 = 15` （n=3），而同时存活的栈帧峰值只有 `n + 1 = 4`。 注意 `n = 64` 需 `2^64 - 1 ≈ 1.8 × 10^19` 步，每秒一亿步也要五千年以上—— 分治能把问题描述得很优雅，却不改变问题本身的难度。
- *作用域与存储期*：合并阶段通常需要额外空间（归并的临时数组），多来自堆，须自行管理生命周期。

栈帧链图解（`factorial(4)` 在 x86-64 上的实测地址）：栈**向低地址增长**，相邻两帧相距 `0x30 = 48` 字节。



LC-3 视角下同一件事（`R5` 帧指针、`R6` 栈指针、`R7` 返回地址；`R0–R3` 为 caller-saved 的参数/返回值寄存器， `R4` 是全局数据指针，**不要拿 R4–R7 当临时寄存器**）：



务必记准这个顺序：`R5+0` （及 `R5-1`, `R5-2`, ...） = **局部变量**； `R5+1` = **旧帧指针**； `R5+2` = **返回地址**； `R5+3` = **返回值**； `R5+4`, `R5+5`, ... = **参数**。压栈 = `ADD R6, R6, #-1` 后 `STR`。

为什么返回值必须在 `R5+3`（紧邻参数之下）？ 因为调用者取回返回值后要用**一条** `ADD R6, R6, #(nparams + 1)` 同时弹出**参数和返回值**（538 讲义的原文是 "Read return value / Pop parameters and return value (destroy the params)"）。 课程真实汇编 `translate.asm` 的 `FIND_ABS` 正是这样写的：

```
FIND_ABS
        ADD  R6,R6,#-4    ; 4 个位置：3 个 linkage + 1 个局部变量
        STR  R5,R6,#1     ; 保存旧帧指针       -> R5+1
        ADD  R5,R6,#0     ; 设置帧指针
        STR  R7,R5,#2     ; 保存返回地址       -> R5+2
        LDR  R0,R5,#4     ; 第一个参数 num     -> R5+4
        STR  R0,R5,#0     ; 局部变量 abs_value -> R5+0
        STR  R0,R5,#3     ; 返回值             -> R5+3
        LDR  R7,R5,#2     ; 恢复返回地址
        LDR  R5,R5,#1     ; 恢复旧帧指针
        ADD  R6,R6,#3     ; 弹出局部变量与 linkage（**返回值槽除外**）
        RET
```

代码示例与底层机制分析

示例 1：`factorial(4)` 的逐帧轨迹

代码 (C):

```
/* r1_factorial_frames.c
 * 编译:  gcc -g -std=c99 -Wall -Werror r1_factorial_frames.c -o r1_factorial_frames */
#include <stdio.h>
#include <stdint.h>

static int32_t depth = 0;               /* 只为打印轨迹，不属于算法本身 */

static int32_t
factorial (int32_t n)
{
    int32_t local_n = n;                /* 每一帧都有自己的这一份 */
    int32_t result;

    depth++;
    printf ("push  depth=%d  &local_n=%p  n=%d\n", depth, (void*)&local_n, (int)n);
    if (0 == n) {
        result = 1;                     /* 基本情况 */
    } else {
        result = n * factorial (n - 1);  /* 递归情况 */
    }
    printf ("pop   depth=%d  &local_n=%p  returns %d\n",
            depth, (void*)&local_n, (int)result);
    depth--;
    return result;
}

int
main (void)
{
    printf ("factorial(4) = %d\n", (int)factorial (4));
    return 0;
}
```

真实运行输出:

```
push  depth=1  &local_n=0x7ffdcc0b7198  n=4
push  depth=2  &local_n=0x7ffdcc0b7168  n=3
push  depth=3  &local_n=0x7ffdcc0b7138  n=2
push  depth=4  &local_n=0x7ffdcc0b7108  n=1
push  depth=5  &local_n=0x7ffdcc0b70d8  n=0
pop   depth=5  &local_n=0x7ffdcc0b70d8  returns 1
pop   depth=4  &local_n=0x7ffdcc0b7108  returns 1
pop   depth=3  &local_n=0x7ffdcc0b7138  returns 2
pop   depth=2  &local_n=0x7ffdcc0b7168  returns 6
pop   depth=1  &local_n=0x7ffdcc0b7198  returns 24
factorial(4) = 24
```

【代码做什么？】 1. `main` 调用 `factorial(4)`，第 1 帧把 `local_n = 4` 放在 `0x7ffdcc0b7198`。 2. `n != 0` 走递归情况求 `factorial(3)`；**当前帧保持存活挂在那里等待**，于是压出第 2 帧。 3. 重复到 `n = 0`，

共 5 帧，每帧相距 48 字节，地址依次递减。 4. `n = 0` 走基本情况返回 1——唯一"不再调用自己"的点。 5. 回退：第 5 帧交出 1，第 4 帧算 `1*1=1`，第 3 帧算 `2*1=2`，第 2 帧算 `3*2=6`，第 1 帧算 `4*6=24`。

【底层机制透视】 4 次乘法**不是往下走时做的，而是回来时做的**：`n * factorial(n-1)` 必须先拿到子调用的返回值， 所以乘法指令位于 `CALL` 之后。这就是"递归成本 = 帧数"的直接后果：第 1 帧在整个递归期间都不能回收。 48 字节/帧是本机实测值，换编译器或 `-O2` 都会变。栈默认上限 8 MiB（`ulimit -s` 可查），本机大约能撑 `8 MiB / 48 B ≈ 17 万` 层——这解释了课程那句"宽搜索适合递归，深搜索容易把栈压垮"。

【内存布局图解】 见前面的帧链图（`R6` / `RSP` 始终指向最深的帧）。

【与汇编的对应】

```
; ---- 调用者：压参数后 JSR —— 这一条指令就是"C 里的递归调用" ----
        LDR R0, R5, #0          ; R0 = n（本帧的局部变量在 R5+0）
        ADD R1, R0, #-1
        ADD R6, R6, #-1
        STR R1, R6, #0          ; 压参数 -> 被调用者的 R5+4
        JSR FACT                ; R7 <- 返回地址；PC <- FACT，压出新的一帧
        LDR R2, R6, #0          ; R2 = 返回值（被调用者的 R5+3，紧邻参数之下）
        ADD R6, R6, #2          ; 一条指令弹出参数与返回值槽

; ---- FACT 的进入/退出序列（与 translate.asm 的 FIND_ABS 完全同构）----
FACT    ADD R6, R6, #-4         ; 3 个 linkage 槽 + 1 个局部变量
        STR R5, R6, #1          ; 保存旧帧指针       -> R5+1
        ADD R5, R6, #0          ; R5 = 新帧基址（局部变量底部）
        STR R7, R5, #2          ; 保存返回地址       -> R5+2
        LDR R1, R5, #4          ; R1 = 参数 n        -> R5+4
        ; ... n 为 0 走基本情况，否则再次 JSR FACT ...
        STR R1, R5, #0          ; local_n（局部变量）-> R5+0
        STR R0, R5, #3          ; 返回值             -> R5+3
FACTD   LDR R7, R5, #2          ; 恢复返回地址
        LDR R5, R5, #1          ; 恢复旧帧指针
        ADD R6, R6, #3          ; 弹出局部变量与 linkage（返回值槽除外）
        RET                     ; PC <- R7
```

示例 2：朴素 Fibonacci 的指数爆炸

代码 (C):

```c
/* r4_fib_calls.c -- 课程约定：F(0)=1, F(1)=1, F(N)=F(N-1)+F(N-2)
 * 编译：  gcc -g -std=c99 -Wall -Werror r4_fib_calls.c -o r4_fib_calls */
#include <stdio.h>
#include <stdint.h>

static int64_t calls = 0;                       /* fib 被进入的次数 */

static int32_t
fib_naive (int32_t n)
{
    calls++;
    if (0 == n || 1 == n) { return 1; }
    return fib_naive (n - 1) + fib_naive (n - 2);
}

static int32_t
fib_iter (int32_t n)                            /* 迭代版本 */
{
    int32_t i = 1, j = 1, k = 1, t;

    while (n > k) { t = j; j = j + i; i = t; k++; }
    return j;
}

int
main (void)
{
    int32_t n;

    printf (" n |      fib(n) | calls for naive fib\n");
    for (n = 1; 25 >= n; n++) {
        int32_t f;

        calls = 0;
        f = fib_naive (n);              /* 先调用，把计数定下来 */
        printf ("%2d | %11d | %ld\n", (int)n, (int)f, (long)calls);
    }
    calls = 0;
    printf ("\nfib_iter(25) = %d (recursive calls made: %ld)\n",
            (int)fib_iter (25), (long)calls);
    return 0;
}
```

> ⚠️ **为什么必须分成两句写？** 如果写成 `c > printf ("%2d | %11d | %ld\n", (int)n, (int)fib_naive (n), (long)calls); /* ❌ */ >` 那么这一行里"调用 `fib_naive`"与"读取 `calls`"这两个实参的**求值顺序是未指定的 (unspecified order of evaluation)**。GCC 12 会**先读 `calls`**（此时刚被置 0）再调用 `fib_naive`，于是整张表的 `calls` 列全部打印 **0**——程序照样编译、照样运行、毫无警告， 只是结果是错的。这是一个"逻辑错误"的活标本：**编译器抓不到它，只有核对输出才能发现**。 凡是"某个函数的副作用会影响同一表达式里另一个实参的值"，都必须拆成独立的语句。

真实运行输出（节选）：

```
 n |      fib(n) | calls for naive fib
 1 |           1 | 1
 2 |           2 | 3
 3 |           3 | 5
 4 |           5 | 9
 5 |           8 | 15
10 |          89 | 177
15 |         987 | 1973
20 |       10946 | 21891
25 |      121393 | 242785

fib_iter(25) = 121393 (recursive calls made: 0)
```

【代码做什么？】 1. `fib_naive` 每次进入都把 `calls` 加一，把"调用次数"变成可测量的量。 2. `n = 0` 或 `n = 1` 返回 1（两个基本情况，也是递归树的叶子）；否则返回 `fib(n-1) + fib(n-2)`， 一次调用分裂成两个孩子。 3. 主程序对 `n = 1..25` 打印 `fib(n)` 与调用次数，最后用迭代版本算 `fib(25)`（调用次数 0）。

【底层机制透视】 课程强调的数字在这里得到实测印证：**`fib(5)` 一共调用 15 次**，因为同一个 `n` 被反复计算：

```
                    fib(5)                    ← 1 个结点
                   /      \
              fib(4)         fib(3)            ← 2 个
             /     \        /     \
        fib(3)   fib(2)  fib(2)  fib(1)        ← 4 个
        /   \     /  \    /  \
    fib(2) f(1) f(1) f(0) f(1) f(0)            ← 7 个（只展开左半）
    /   \
 fib(1) fib(0)                                 ← 叶子层
```

数一数：`1 + 2 + 4 + 7 + 1 = 15`，其中 `fib(3)` 被算了 2 次、`fib(2)` 被算了 3 次。 调用次数满足 `C(n) = C(n-1) + C(n-2) + 1`，与 Fibonacci 同阶，即 `C(n) = θ(φ^n)`， `φ = (1+√5)/2 ≈ 1.618`。 实测比例印证：`C(25)/C(24) = 242785/150049 ≈ 1.618`； `C(20)/C(15) = 21891/1973 ≈ 11.1 ≈ φ^5 = 11.09`。 **指数爆炸的根因不是"递归慢"，而是"重复子问题"**：迭代版本自底向上算，每个 `n` 只算一次，于是降到 O(n)； 而栈深度始终只有 O(n)。内存与时间必须分开看。

【内存布局图解】

```
内存（栈）: O(n)                    时间（调用次数）: O(φ^n)

┌──────────────────────────┐
│ 帧1 fib(5) → … → 帧5 fib(1) │   只有一条根到叶的路径同时存活；
└──────────────────────────┘   兄弟分支的帧"用完即弹、需要再压"
```

【与汇编的对应】

```
; R0 = n；两条 JSR FIB 就是"一次调用分裂成两个孩子"（示意：真实编译器把 n 与中间
; 结果放在 R5+0、R5-1 局部变量里；这里直接借用栈顶，注意每次调用后弹出"参数+返回值"两格）
        ADD R1, R0, #0
        BRz FIB1                ; n == 0 -> 基本情况
        ADD R2, R1, #-1
        BRz FIB1                ; n == 1 -> 基本情况
        ADD R6, R6, #-1
        STR R1, R6, #0          ; n 暂存栈上（两次调用之间还要用）
        ADD R6, R6, #-1
        ADD R0, R1, #-1
        STR R0, R6, #0          ; 压参数 n-1
        JSR FIB                 ; 递归调用 #1
        LDR R2, R6, #0          ; R2 = fib(n-1)（返回值在参数正下方）
        ADD R6, R6, #2          ; 弹出参数与返回值槽
        ADD R6, R6, #-1
        STR R2, R6, #0          ; fib(n-1) 也要存起来（R0-R3 是 caller-saved）
        LDR R1, R6, #1          ; R1 = n（从栈上取回）
        ADD R6, R6, #-1
        ADD R0, R1, #-2
        STR R0, R6, #0          ; 压参数 n-2
        JSR FIB                 ; 递归调用 #2
        LDR R3, R6, #0          ; R3 = fib(n-2)
        ADD R6, R6, #2          ; 弹出参数与返回值槽
        LDR R2, R6, #0          ; R2 = fib(n-1)
        ADD R6, R6, #1          ; 弹掉暂存
        ADD R0, R2, R3
FIB1    RET
```

示例 3：回溯与洪水填充 (flood fill)

代码 (C):

```c
/* r5d_maze_compact.c -- 迷宫用位向量表示：L=1, R=2, U=4, D=8, 出口=16
 * 编译：  gcc -g -std=c99 -Wall -Werror r5d_maze_compact.c -o r5d_maze_compact */
#include <stdio.h>
#include <stdint.h>

enum { LEFT_WALL = 1, RIGHT_WALL = 2, UPPER_WALL = 4, LOWER_WALL = 8, HAS_EXIT = 16 };
#define W 3
#define H 3

/* maze[x][y]：(2,2) 是出口，但被四面墙围死 */
static uint8_t maze[W][H]  = {{5, 9, 9}, {4, 2, 10}, {6, 10, 31}};
static uint8_t found[W][H];         /* 0 = 未到过，1 = 到过（文件作用域，初值全 0）*/
static int32_t saw_exit = 0, calls = 0, depth = 0, max_depth = 0;

static void
can_reach (int32_t x, int32_t y)
{
    calls++;
    if (++depth > max_depth) { max_depth = depth; }
    if (found[x][y]) {                      /* 停止条件：这一格已经到过 */
        depth--;
        return;
    }
    found[x][y] = 1;                        /* 处理当前结点 */

    if (0 == (LEFT_WALL & maze[x][y]))  { can_reach (x - 1, y); }   /* 子结点 */
    if (0 == (RIGHT_WALL & maze[x][y])) { can_reach (x + 1, y); }
    if (0 == (UPPER_WALL & maze[x][y])) { can_reach (x, y - 1); }
    if (0 == (LOWER_WALL & maze[x][y])) { can_reach (x, y + 1); }

    if (0 != (HAS_EXIT & maze[x][y])) { saw_exit = 1; }
    depth--;
}

int
main (void)
{
    int32_t x, y;

    can_reach (0, 0);
    printf ("spaces reachable from (0,0)  ('#' = reached, '.' = not reached)\n");
    for (y = 0; H > y; y++) {
        for (x = 0; W > x; x++) { printf ("%c", found[x][y] ? '#' : '.'); }
        printf ("\n");
    }
    printf ("saw_exit = %d\n", (int)saw_exit);
    printf ("calls = %d, maximum stack depth = %d frames\n",
            (int)calls, (int)max_depth);
    return 0;
}
```

真实运行输出:

```
spaces reachable from (0,0)  ('#' = reached, '.' = not reached)
###
###
##.
saw_exit = 0
calls = 19, maximum stack depth = 8 frames
```

【代码做什么？】 1. `maze[x][y]` 的每个字节是位向量：第 0 位左墙、第 1 位右墙、第 2 位上墙、第 3 位下墙、第 4 位出口。 `maze[0][0] = 5 = 1|4` 表示左上角同时有左墙与上墙； `maze[2][2] = 31` 表示出口被四面墙围死。 2. `can_reach(x,y)`：若 `found[x][y]` 非 0 就立刻返回（**停止条件**），否则标记该格， 再对四个方向中"没有墙"的邻居分别递归。 3. `main` 从 (0,0) 做洪水填充，然后把 `found` 打印成 `#` / `.` 图。 4. 输出显示 8 格可达、右下角 (2,2) 不可达，于是 `saw_exit = 0`：**出口存在，但从起点走不到**。

【底层机制透视】 `found` 数组同时承担三个角色：①已访问集合 ②停止条件 ③输出结果。 课程演示的"没有停止条件会怎样"值得亲自体验：把 `if (found[x][y]) return;` 注释掉， `A → B → A → B …` 无限互相调用，栈持续向下生长，最终**段错误**。随附的 `r5b_no_stop_demo.c` 就是这种情况，实测退出码 **139 (SIGSEGV)**，而且**第一行 `printf` 的输出也一并丢失**—— 它还在 stdout 缓冲区里，进程没来得及 flush（第 14 讲会再遇到这个现象）。 再注意实测的 `calls = 19` 与 `max_depth = 8`：**调用次数（时间）与栈深度（空间）是两个不同的量**。

【内存布局图解】

```
static storage duration（进程整个生命周期）
┌──────────────────────────────────────────┐
│ maze[3][3]   9 字节   ← 墙的位向量       │
│ found[3][3]  9 字节   ← 访问标记（初值 0）│
│ saw_exit / calls / depth / max_depth     │
└──────────────────────────────────────────┘
栈（向低地址增长，最深 8 层）：帧#1 can_reach(0,0) → 帧#8 栈顶（R6/RSP）
```

【与汇编的对应】

```
; CANREACH(x, y): R0 = x, R1 = y。帧布局与 538 讲义的 FOO 同构 (R5 = R6+1):
; R5+0 = x、R5-1 = y、R5+1 = 旧帧指针、R5+2 = 返回地址、R5+3 = 返回值、R5+4/+5 = 参数
CANREACH
        ADD  R6, R6, #-5        ; 3 个 linkage 槽 + 2 个局部变量
        STR  R5, R6, #2         ; 保存旧帧指针
        ADD  R5, R6, #1         ; 设置本帧指针 (局部变量底部)
        STR  R7, R5, #2         ; 保存返回地址       -> R5+2
        STR  R0, R5, #0         ; 局部 x             -> R5+0
        STR  R1, R5, #-1        ; 局部 y             -> R5-1
        LDR  R3, R5, #-1        ; R3 = y
        ADD  R2, R3, R3
        ADD  R2, R2, R3         ; R2 = 3y
        LDR  R3, R5, #0         ; R3 = x
        ADD  R2, R2, R3         ; R2 = x + y * W (课程强调的展平公式)
        LEA  R3, FOUND
        ADD  R3, R3, R2         ; R3 = &found[x][y]
        LDR  R1, R3, #0         ; R1 = found[x][y]
        BRnp CRTEARDOWN         ; 非 0 -> 已到过, 停止条件成立, 直接返回
        ADD  R1, R1, #1
        STR  R1, R3, #0         ; found[x][y] = 1 (处理当前结点)
        LEA  R3, MAZE
        ADD  R3, R3, R2         ; R3 = &maze[x][y]
        LDR  R1, R3, #0         ; R1 = 位向量
        AND  R1, R1, #1         ; 取"左墙"位
        BRnp SKIP_LEFT          ; 有墙 -> 不递归
        LDR  R0, R5, #-1        ; R0 = y
        LDR  R1, R5, #0
        ADD  R1, R1, #-1        ; R1 = x - 1
        ADD  R6, R6, #-1
        STR  R0, R6, #0         ; 压 y (先压的在上面 -> 被调用者的 R5+5)
        ADD  R6, R6, #-1
        STR  R1, R6, #0         ; 压 x-1 (-> 被调用者的 R5+4)
        JSR  CANREACH           ; ← 新帧, 递归
        ADD  R6, R6, #3         ; 一条指令弹出 2 个参数与返回值槽
SKIP_LEFT
        ; 右、上、下三个方向同理
CRTEARDOWN
        LDR  R7, R5, #2         ; 恢复返回地址
        LDR  R5, R5, #1         ; 恢复旧帧指针
        ADD  R6, R6, #4         ; 弹出 2 个局部变量与 linkage (返回值槽除外)
        RET
```

二维数组 `maze[x][y]` 在 C 中等价于 `*(*(maze + x) + y)`，展平偏移为 `x + y * W`； MP8 的洪水填充文档要求写成 `red[x + y * width]`，正是这个公式。

常见错误与调试技巧

- **忘记基本情况或基本情况不收敛**：现象是卡死或段错误。`can_reach` 少了 `if (found[x][y]) return;` 就会 `A→B→A→B…` 无限递归。**调试**：`gdb -tui --args ./prog` 运行后 Ctrl-C，用 `bt` 看栈； 同一函数重复出现几百次就是缺停止条件， `bt 20` 只看前 20 层。

- **参数没有向基本情况靠近**：例如把 `can_reach(x - 1, y)` 写成 `can_reach(x + 1, y)`， 或数组递归传 `a` 而不是 `a + 1`。**调试**：在 `gdb` 中反复 `p x` / `p y` 观察参数是否单调靠近基本情况； 也可临时加 `if (depth > 100) { printf("%d %d\n", x, y); abort(); }`。
- **返回指向本帧局部变量的指针**：`char* f(void){ char buf[10]; return buf; }`， 现象是返回的字符串内容随机（帧内存已被后续调用覆盖）。**调试**：`gcc -fsanitize=address -g` （本机受 `ulimit -v` 限制无法启动 ASan 时改用 `valgrind --track-origins=yes ./prog`， 它会报 "Address ... on thread 1's stack"）。
- **以为写了尾递归就不会爆栈**：`a[0] + rec(...)` 之后还有加法，编译器无法复用帧。 **调试**：`objdump -d ./prog | awk '/<func>:/,/ret/'` 看是否存在指向自己的 `callq`； 再用 `gcc -O2` 与 `gcc -O2 -fno-optimize-sibling-calls` 各编译一次对比运行结果。
- **回溯时忘记撤销状态**：`col[row]` 没还原、`found` 没清理，后续分支看到脏数据。 **调试**：`gdb` 里 `watch col[3]`，每次变化都会停住，配合 `bt` 可看出是哪一层改的、有没有改回来。
- **低估递归深度**：深度 10 万 × 48 字节 ≈ 4.8 MB，已吃掉默认 8 MB 栈的一半。 **调试**：`ulimit -s` 查看栈上限；`gcc -fstack-usage` 生成 `.su` 文件给出静态帧大小； 也可在 `gdb` 中打印相邻两层局部变量的地址差，算出真实帧大小。

关键要点

- 递归函数由**基本情况**与**递归情况**构成；停止条件不仅要存在，还要保证参数向基本情况收敛。
- 每次递归调用都是一次 `JSR` 加一个新栈帧：**递归深度直接等于内存占用， 递归树的结点数等于时间开销**，两者必须分开评估。
- 回退 (unwinding) 阶段能做真正的功：`print_reverse` 靠它反序输出， `factorial` 靠它完成乘法； 语句放在递归调用之前还是之后，决定了算法是自顶向下还是自底向上。
- 尾递归只在编译器做尾调用优化时才省栈（`-O0` 下与普通递归相同），不能用它来规避栈溢出风险。
- 递归的价值在**表达力**（回溯、分治、树形结构），不在速度：朴素 Fibonacci 说明 "能递归地写"不等于"应该递归地写"，遇到重叠子问题应改用迭代或记忆化。

思考题（带答案）

问题 1：下面的函数在 `n = 5` 时输出什么？如果把两个 `printf` 互换位置，输出如何变化？

```
static void f (int32_t n)
{
    if (0 == n) { printf ("B"); return; }
    printf ("a");
    f (n - 1);
    printf ("b");
}
```

答案：输出 `aaaaaBbbbbb`：下行阶段每层打印 `a` （5 个），基本情况打印 `B`，回退阶段每层打印 `b` （5 个）。 这说明**递归调用之前的语句属于"下行段"，之后的语句属于"回退段"**；互换后 `a`、`b` 角色对调，变成 `bbbbbBaaaaa`。 用 `gdb` 的 `break f` 配合 `continue` 观察调用序列即可验证。

问题 2：为什么 `fib(25)` 只要 242785 次调用，而 `fib(40)` 就要上亿次？给出把它降到 O(n) 的最小改动。

答案：调用次数满足 `C(n) = C(n-1) + C(n-2) + 1`，与 Fibonacci 同阶，故 `C(n) = Θ(φ^n)`，`φ ≈ 1.618`。从 25 到 40 差 15 层，放大倍数约 `φ^15 ≈ 1364`，于是 `242785 × 1364 ≈ 3.3 × 10^8`，确实是上亿。最小改动有两条路：① 改成迭代（保留两个前驱值，O(n) 时间、O(1) 空间）；② 保留递归但加一张表做记忆化 (memoization)。两者都只消除了"重复子问题"。

问题 3：走迷宫的 `can_reach` 中，如果把 `found` 改成局部变量（每帧一份），程序还能正确工作吗？

答案：不能。`found` 是**跨越所有分支共享的已访问集合**，必须对所有递归层可见；放进帧里就变成每层一份副本，"已经到过 (x,y)"这一信息无法传给兄弟分支，不同分支会反复访问同一格，最终退化为无限递归。这说明 storage duration 的选择直接决定算法是否成立：需要**跨调用共享**的状态要用文件作用域 `static` 或显式传指针，只属于**单次调用**的状态才放帧里。顺带一提，MP8 的洪水填充文档特意提醒"不要使用 static 存储"—— 那里的标记数组由包装函数在堆上分配后传入，这样函数才能被反复调用而不留残留状态。两种做法都对，关键是清楚状态的生命周期边界。

Lecture 14: 文件 I/O (File I/O in C)

概述

本讲要解决的问题是：**程序如何把数据从文件、键盘、管道读进来，再写出去**，并且做到"同一份代码既能对文件工作，也能对键盘和管道工作"。引入的机制是 C 的**流 (stream)** 抽象—— `FILE*` 加上 `fopen` / `fclose` / `fgetc` / `fgets` / `fscanf` / `fprintf` / `fread` / `fwrite` 这一整套库函数，它们在 Unix **文件描述符 (file descriptor)** 之上再加一层**缓冲 (buffering)**。这一讲把第 1–4 讲的 LC-3 内存映射 I/O（`KBSR` / `KBDR` / `DSR` / `DDR` 那几个寄存器）提升到操作系统级抽象，也是第 15–16 讲"把数据结构存进文件、再读回来"的前提。

核心概念与底层机制图解

- **流 (Stream) 与 `FILE` 抽象**：C 程序把每个 I/O 通道看成一串连续的字节，用一个 `FILE*` 句柄操作它。
  - *直观解释*：`FILE*` 像水管的接头：你只关心倒水与接水，不关心水来自井里、河里还是水厂—— **文件、键盘、管道、网络在程序眼里是同一种东西**。
  - *底层机制图解*：Unix 里所有 I/O 都归结为**文件描述符 (file descriptor)**，即内核中"每进程打开文件表"的**小整数下标**。C 的 `FILE` 结构包住一个描述符，并自带一块缓冲区：`FILE* ──►` `┌──────────────────────────┐ 内核侧（打开文件表）: │ 文件描述符 fd = 3 │──┐` `┌──────────────────────────┐ │ 缓冲区指针 / 读写位置 │ └─► │ [0]=键盘 [1]=屏幕 │ │ 缓冲区数组 / EOF / 错误标志 │ │ [2]=屏幕(错误) [3]=文件 │ └──────────────────────────┘` `└──────────────────────────┘` 这也是著名的"一切皆文件"：`read` / `write` 系统调用对普通文件、终端、管道、套接字都适用，所以早期 Internet 服务大多先在键盘/屏幕上调试，然后由 `inetd` 把网络连接替换成标准输入输出就上线了。

- *作用域与存储期*：三个标准流 `stdin`（描述符 0）、`stdout`（1）、`stderr`（2） 具有**静态存储期 (static storage duration)**，由 C 运行时在 `main` 之前建立、在退出时自动关闭。 `fopen` 返回的 `FILE*` 指向**堆上**由库分配的对象，必须靠 `fclose` 释放。

- **打开与关闭文件 (`fopen` / `fclose`)**：`FILE* fopen (const char* path, const char* mode);` 失败返回 `NULL`。
  - *直观解释*：`fopen` 像去图书馆借书——书可能不在（路径错）、可能没权限，所以**每次都要查返回值**。
  - *底层机制图解*：`mode` 是字符串，决定打开方式与是否截断：

mode	含义	mode	含义
`"r"` / `"rb"`	只读，文件必须存在	`"r+"` 系列	读写，文件必须存在
`"w"` / `"wb"`	只写，**先清空**（不存在则创建）	`"w+"` 系列	先清空，再读写
`"a"` / `"ab"`	只写，追加到末尾	`"a+"` 系列	追加并允许读

  `"b"` 是历史遗留的二进制标记；在 MS-DOS 等系统上它会阻止 CR/LF 被悄悄改写，在 Unix 上无实际差别。 `int fclose (FILE* stream)` 返回 0 成功、`EOF` 失败——**失败常常发生在关闭时**， 因为此时才把缓冲区真正写回磁盘，所以 `fclose` 的返回值必须检查。 * *作用域与存储期*：每个进程能同时打开的描述符数量有上限（`ulimit -n`，本机 16384）。 忘记 `fclose` 就是**泄漏文件描述符 (file descriptor leak)**：循环里反复 `fopen` 却不关闭， 程序会先耗尽描述符，随后所有 `fopen` 都返回 `NULL`——这类 bug 在长跑服务里很致命。

- **三种粒度的读写：字符、行、格式化**：
  - *直观解释*：字符 I/O 像用吸管一滴一滴喝，行 I/O 像按杯喝，格式化 I/O 像按菜谱配好料再端上来。
  - *底层机制图解*：
    - `int fgetc (FILE*)` / `int getc (FILE*)`：读一个字节；**返回 `int` 而不是 `char`**， 因为必须有一个"不属于任何字节"的值表示失败，那就是 `EOF`（-1），而 `0xFF` 是合法字节。 `getc` 是宏（内联进你的函数，代码更大但更快），`fgetc` 是库函数（代码更小）。
    - `int fputc (int c, FILE*)` / `putc`：写一个字节；`getchar` / `putchar` 是 stdin/stdout 的快捷方式。
    - `char* fgets (char* s, int size, FILE*)`：读**最多 `size-1` 个字节**，遇**行尾/文件尾/缓冲区满**即停， 把换行符**留在数组里**并补 `'\0'`；它是处理行式输入的最佳工具。
    - `int fputs (const char* s, FILE*)`：写字符串，**不加换行**；`puts` 会自动补一个 `'\n'`。
    - `int fprintf`：格式化写，返回写入字符数或负值；`int fscanf`：格式化读，**返回成功转换的字段个数**。
    - `gets` **已被 C11 删除，永远不要使用**：它没有长度参数，任何长度的输入都会溢出缓冲区。
  - *作用域与存储期*：`fgets` 写入的是**调用者提供的数组**，其存储期由调用者决定； 函数本身不分配任何内存，所以绝不要把栈上的缓冲区地址返回出去。

- **解析文本：`strtok`、`strtol`、`sscanf`、`snprintf`**：

- *直观解释*：`fgets` 负责"把整行端上桌"，解析函数负责"把这一行切成小块并看懂它们"。
- *底层机制图解*：
  - `char* strtok (char* s, const char* delim)`：按分隔符切分，**会就地修改原字符串**（把分隔符改成 `'\0'`）； 第一次传字符串，之后传 `NULL` 表示"继续切同一个串"。它不是线程安全的，也不能处理嵌套。
  - `long strtol (const char* s, char** endptr, int base)`：把字符串转成长整型； `endptr` 指回**第一个未能转换的字符**，配合 `errno == ERANGE` 可以区分"转换成功""后面有垃圾""溢出"三种情况。
  - `int sscanf (const char* s, const char* fmt, ...)`：从**字符串**里做格式化读取（与 `fscanf` 同族）， 适合"先用 `fgets` 取一行、再反复尝试不同格式"的健壮解析策略：解析失败的行可以原样回显给用户。
  - `int snprintf (char* s, size_t size, const char* fmt, ...)`：把格式化结果写进字符串，**带长度限制**。
- *作用域与存储期*：`strtok` 内部用一个**静态变量**记录位置（static storage duration）， 所以它跨调用保存状态——这正是它不可重入的原因。解析出来的指针都指向**原字符串**内部， 原字符串一旦被覆盖或释放，这些指针立刻失效。

- **EOF 与错误是两回事（`feof` / `ferror`）**：
  - *直观解释*：`feof` 是"上一次读为什么停下来"的**事后报告**，不是"接下来还有没有数据"的**事前预测**。
  - *底层机制图解*：`feof(f)` 只有在**一次读取已经因为到达文件尾而失败之后**才变为真。 因此 `while (!feof (f)) { fscanf (f, ...); ... }` 必然多执行一轮：最后一轮里 `fscanf` 失败， 但变量没有被写入，于是把那**上一次的旧值又处理了一遍**。正确写法是**测试输入函数的返回值**： `while (1 == fscanf (f, "%d", &v))` 或 `while (NULL != fgets (buf, size, f))`。 同理， `fgetc` 返回 `EOF` 后要用 `ferror(f)` 区分"正常结束"与"真的出错了"。
  - *作用域与存储期*：EOF 与错误标志都存在 `FILE` 对象里，存储期与流相同； `clearerr(f)` 可以清除它们， `rewind(f)` 会同时清标志并把位置移回开头。
- **文本格式 vs 二进制格式（`fread` / `fwrite`）**：
  - *直观解释*：文本像"用文字写下来"，二进制像"把内存原样复印一份"。
  - *底层机制图解*：`size_t fwrite (const void* p, size_t size, size_t n, FILE*)` 把 `n` 个 `size` 字节的"东西"原样写出； `size_t fread (...)` 原样读回，二者返回**成功处理的个数**（不是字节数）。 二进制更省空间也更快（不需要十进制转换），但有三个代价：**不可读**、**不可移植**（大小端、浮点表示、结构体填充都可能不同）、**难以升级格式**。 另外要"把数据结构存进文件"就必须先**扁平化（flatten）**：把指针换成下标或偏移， 否则下次运行进程地址变了，指针就毫无意义。
  - *作用域与存储期*：`fread` / `fwrite` 不分配内存，读写的是调用者给的缓冲区； 若要把整个数组一次读写，缓冲区通常来自 `malloc`（第 16 讲）。
- **重定向、管道与缓冲 (redirection, pipes, buffering)**：
  - *直观解释*：重定向是"把水管接到别的水龙头上"，缓冲是"在水管中间装一个水箱"。
  - *底层机制图解*：shell 在启动程序前就把描述符 0/1/2 换成别的文件：

`./prog > out.txt` 把 stdout 重定向到文件（先清空），`>>` 改为追加，`< in.txt` 重定向 stdin，`2> err.txt` 只重定向 stderr，`cat in.txt | ./prog` 用管道把前一个进程的 stdout 接到后一个进程的 stdin。 同时写 `stdout` 与 `stderr` 的程序，因此可以做到"正常结果进文件、错误信息留在屏幕"。 缓冲方面的关键事实：**缓冲区在 `exit` / `return` 时自动刷新，但 `abort` /段错误不会**。 写到终端时 stdout 通常是**行缓冲**（每遇换行就刷），重定向到文件或管道时变成**全缓冲**（默认 4 KB 左右），于是"程序崩了但输出不见了"变成一个与平台有关的坑。`fflush(stdout)` 能立刻把缓冲区推出去；`setvbuf(stdout, NULL, _IOLBF, 0)` 可以强制行缓冲，让程序在重定向后行为一致。 * *作用域与存储期*：缓冲区属于 `FILE` 对象；进程死亡时它随进程消失，未刷出的数据就永远丢了。

代码示例与底层机制分析

示例 1：读记录文件 → 排序 → 写出汇总

代码 (C):

```c
/* n1_record_sort.c   用法：  ./n1_record_sort <input> <output>
 * 编译：  gcc -g -std=c99 -Wall -Werror n1_record_sort.c -o n1_record_sort */
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define MAX_RECORDS 50
#define MAX_LINE    64

typedef struct {
    char    name[16];
    int32_t score;
} record_t;

/* Return the number of records read, or -1 on a malformed line. */
static int32_t
read_records (FILE* in, record_t* recs, int32_t max_records)
{
    char    line[MAX_LINE];
    int32_t n = 0;
    int32_t value;

    while (NULL != fgets (line, MAX_LINE, in)) {
        if (2 != sscanf (line, "%15s %d", recs[n].name, &value)) {
            return -1;                    /* not "name score" */
        }
        recs[n].score = value;
        n++;
        if (max_records == n) {
            break;
        }
    }
    if (0 != ferror (in)) {               /* EOF and an error differ! */
        perror ("read");
        return -1;
    }
    return n;
}

static void
sort_records (record_t* recs, int32_t n)   /* insertion sort, descending */
{
    int32_t  i, j;
    record_t current;

    for (i = 1; n > i; i++) {
        current = recs[i];
        for (j = i - 1; 0 <= j; j--) {
            if (current.score <= recs[j].score) {
                break;
            }
            recs[j + 1] = recs[j];
        }
```

```
        recs[j + 1] = current;
    }
}

int
main (int argc, char* argv[])
{
    FILE*    in;
    FILE*    out;
    record_t recs[MAX_RECORDS];
    int32_t  n, i, total = 0;

    if (3 != argc) {
        fprintf (stderr, "syntax: %s <input> <output>\n", argv[0]);
        return 2;
    }
    if (NULL == (in = fopen (argv[1], "r"))) {      /* always check! */
        perror (argv[1]);
        return 1;
    }
    n = read_records (in, recs, MAX_RECORDS);
    if (0 != fclose (in)) {                          /* fclose can fail   */
        perror ("close input");
        return 1;
    }
    if (0 > n) {
        return 1;
    }

    sort_records (recs, n);

    if (NULL == (out = fopen (argv[2], "w"))) {
        perror (argv[2]);
        return 1;
    }
    fprintf (out, "records: %d\n", (int)n);
    for (i = 0; n > i; i++) {
        fprintf (out, "%-3d %-8s %4d\n", i + 1, recs[i].name, (int)recs[i].score);
        total += recs[i].score;
    }
    fprintf (out, "average: %.2f, best: %s\n",
             (double)total / (double)n, recs[0].name);
    if (0 != fclose (out)) {                         /* flushes the buffer */
        perror ("close output");
        return 1;
    }

    printf ("wrote %d records to %s\n", (int)n, argv[2]);
    return 0;
}
```

输入文件 `n1_input.txt` 的内容是 `alice 91`、`bob 72`、`carol 85`、`dave 64`、`erin 91`、`frank 100`，每行一条。

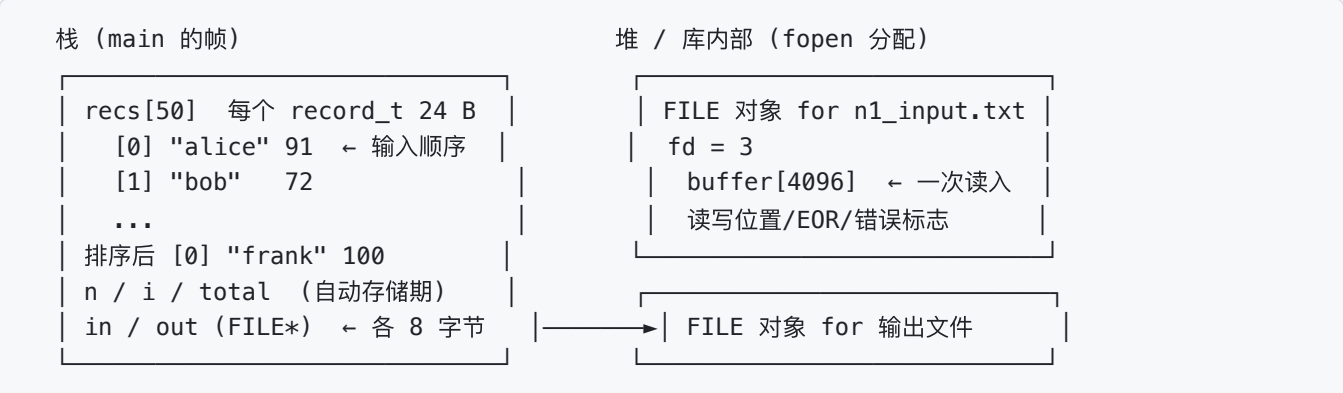
真实运行输出:

```
$ ./n1_record_sort n1_input.txt n1_summary.txt
wrote 6 records to n1_summary.txt
$ cat n1_summary.txt
records: 6
1   frank     100
2   alice      91
3   erin       91
4   carol      85
5   bob        72
6   dave       64
average: 83.83, best: frank
```

【代码做什么？】 1. `main` 检查参数个数，然后 `fopen(argv[1], "r")` 打开输入文件，**立刻判断是否为 `NULL`**。 2. `read_records` 用 `fgets` 一行一行读（因此天然处理行式文本），再用 `sscanf(line, "%15s %d", ...)` 解析出姓名与分数；返回值不等于 2 就判定该行格式错误并返回 –1。`%15s` 防止姓名写爆 `name[16]`。 3. 读完输入后用 `ferror` 区分"正常读完"与"读出错"，随后 `fclose` 并**检查返回值**。 4. `sort_records` 做插入排序（降序）。实测 alice 与 erin 同为 91 分，因为插入排序在相等时停止（稳定排序）， 输出里 alice 排在 erin 前面。 5. 以 `"w"` 打开输出文件，逐行 `fprintf` 写出排名、姓名、分数，最后写平均分与最高分； `fclose` 把缓冲区写回磁盘（失败会返回 `EOF`，所以这里也要检查）。 6. 最后向 `stdout` 打印一行简短的状态信息。

【底层机制透视】 这段代码几乎把整讲的 I/O 规则用了一遍：`fgets` 提供**行边界**、`sscanf` 提供**字段解析**、`ferror` 提供**错误与 EOF 的区分**、`fclose` 提供**缓冲区的最终落盘**。 注意 `"w"` 模式的语义是**先截断**，与 `"a"` 的追加语义完全不同； `read_records` 里"读到上限就 `break`"的检查若省略，超长输入会持续写 `recs[MAX_RECORDS]` 之后的栈内存—— 这是第 10 讲强调过的经典越界错误。最后， `fprintf` 的返回值（写入字符数）在本例中没有检查； 磁盘写满、管道破裂这类错误只有靠检查它（以及 `fclose` 的返回值）才能发现。

【内存布局图解】



【与汇编的对应】 LC–3 只有内存映射 I/O（`KBSR` / `KBDR`、`DSR` / `DDR` 各占一个地址），没有操作系统、没有文件系统； C 里的一次 `fgets` 在 LC–3 上会展开成一串 `LDI` / `STI`（轮询就绪位、搬运数据）， 而在 Unix 上它变成"从 `FILE` 缓冲区拷一行，必要时再发一次 `read` 系统调用"。 两者共享的机器事实是：**数据必须经过寄存器逐字搬运，缓冲只是减少搬运次数**。 可以用 TRAP 类比标准流的入口：

```
; LC-3 没有文件概念：读一个字符要轮询键盘状态寄存器
POLL    LDI R0, KBSR        ; R0 = 键盘状态寄存器内容
        BRzp POLL           ; 就绪位为 0 就继续等
        LDI R0, KBDR        ; R0 = 键入的字符
; C 里对应的概念是 fgetc(stdin)：一次调用返回一个字符或 EOF
        JSR FGETC_CALL      ; 进入 C 库函数（内部可能触发 read 系统调用）
        ADD R1, R0, #1      ; 比较是否为 EOF(-1)
        BRz GOT_EOF
```

用第 12 讲的字符处理循环（`getc` 逐字符 + `putc` 逐字符）就能写出与 `cat` 等价的程序， 只不过 C 版本读写的是 `FILE*`，而 LC-3 版本读写的是设备寄存器。

示例 2：读 `stdin` 的过滤器——与管道和重定向协作

代码 (C):

```c
/* n2_wc_stdin.c  用法:  cat file | ./n2_wc_stdin    或   ./n2_wc_stdin < file
 * 编译:  gcc -g -std=c99 -Wall -Werror n2_wc_stdin.c -o n2_wc_stdin */
#include <stdio.h>
#include <stdint.h>

int
main (int argc, char* argv[])
{
    FILE*   in = stdin;                 /* default: standard input */
    int64_t lines = 0, words = 0, chars = 0;
    int32_t in_word = 0;
    int32_t c;

    if (2 == argc) {                    /* optional file argument */
        if (NULL == (in = fopen (argv[1], "r"))) {
            perror (argv[1]);
            return 1;
        }
    } else if (1 != argc) {
        fprintf (stderr, "syntax: %s [file]\n", argv[0]);
        return 2;
    }

    while (EOF != (c = fgetc (in))) {   /* fgetc returns int, not char */
        chars++;
        if ('\n' == c) {
            lines++;
        }
        if (' ' == c || '\t' == c || '\n' == c || '\r' == c) {
            in_word = 0;
        } else if (0 == in_word) {
            words++;
            in_word = 1;
        }
    }

    if (0 != ferror (in)) {             /* a real error, not end of file */
        perror ("read");
        if (stdin != in) {
            (void)fclose (in);
        }
        return 1;
    }
    if (stdin != in) {
        (void)fclose (in);
    }

    printf ("lines %ld, words %ld, chars %ld\n", (long)lines, (long)words, (long)chars);
    return 0;
}
```

真实运行输出（三种调用方式结果一致）：

```
$ cat n1_input.txt | ./n2_wc_stdin
lines 6, words 12, chars 51
$ ./n2_wc_stdin < n1_input.txt
lines 6, words 12, chars 51
$ ./n2_wc_stdin n1_input.txt
lines 6, words 12, chars 51
```

【代码做什么？】 1. `in` 默认指向 `stdin`；只有在命令行给了文件名时才 `fopen`。于是**同一个程序既当过滤器又当普通工具**。 2. 用 `fgetc` 逐字符读取，直到返回 `EOF`。用 `int32_t c` 接住返回值——**不能声明成 `char`**。 3. 每读一个字符 `chars++`；遇到换行 `lines++`；用 `in_word` 状态机统计单词边界（连续的非空白字符算一个单词）。 4. 循环结束后用 `ferror(in)` 判断是"正常到达文件尾"还是"读取出错"，并在出错时报 `perror` 并返回 1。 5. 若 `in` 不是标准输入则 `fclose`；随后把三个计数打到 `stdout`。

【底层机制透视】 关键在于"**同一份代码对文件、管道、键盘都工作**"，靠的是 shell 在启动程序之前就把描述符 0 换掉了： `< file` 让 `stdin` 指向一个普通文件，`cat x | prog` 让它指向管道读端，程序无需知道差异。 另一个细节是 `c` 的类型：`fgetc` 要返回 257 种可能的值（256 个字节 + 一个 EOF），`char` 装不下； 本机 `char` 有符号，字节 `0xFF` 会被当成 −1，于是"遇到 0xFF 就以为文件结束"，文件被提前截断。 `in_word` 这类**跨迭代保存的状态**必须声明在循环之外。

【内存布局图解】

```
  进程启动时（由 shell/内核建立）：
  ┌──────────────────────────────────────────────────────────┐
  │ 描述符 0  ──►  管道读端  │  文件 n1_input.txt  │  键盘      │
  │ 描述符 1  ──►  终端      │  文件 out.txt（若 > out.txt）    │
  │ 描述符 2  ──►  终端（错误专用，永远不被 > 影响）            │
  └──────────────────────────────────────────────────────────┘
  FILE* in = stdin 只是给"描述符 0"套了一层缓冲壳：
  ┌──────────────────┐    read(fd=0, buf, 4096)
  │ FILE{fd=0,buf}   │ ──────────────────────────► 内核/管道/磁盘
  └──────────────────┘    fgetc 先吃 buf 里的字节，吃空了才再发一次系统调用
```

【与汇编的对应】

```
; LC-3 里"读一个字符"是轮询设备寄存器；C 里是 fgetc(stdin)。
; 真正把两者连起来的是同一件事：数据必须经寄存器搬进内存。
POLL_IN LDI R0, KBSR            ; 轮询状态寄存器（GETC 是 TRAP x20 别名，不能作标签）
        BRzp POLL_IN            ; 设备就绪（bit 15 = 1）才往下走
        LDI R0, KBDR            ; R0 = 一个字符
        ; 判断是否为 EOF（-1）：
        ADD R1, R0, #1
        BRz  DONE               ; R0 == -1 -> 结束
        ADD R1, R1, #0          ; 否则统计逻辑（chars++ / 判断空白）
        BRnzp POLL_IN
DONE    ; 退出循环，写结果
```

LC-3 版本的"stdin"就是键盘设备寄存器；C 版本把同样的轮询循环藏在 `fgetc` 与缓冲层之下。

示例 3：解析一行文本——`strtok`、`strtol`、`sscanf`、`snprintf`

代码 (C):

```c
/* n4_parse.c
 * 编译： gcc -g -std=c99 -Wall -Werror n4_parse.c -o n4_parse */
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>

int
main (void)
{
    char    line[] = "42, alice , 91.5";
    char*   field;
    char*   end;
    char    out[32];
    int32_t n = 0;
    int32_t id = 0;
    char    name[16] = "";
    double  score = 0.0;
    long    value;

    printf ("--- strtok: split on commas and blanks ---\n");
    field = strtok (line, ", \t");         /* modifies line in place */
    while (NULL != field) {
        printf ("field %d: [%s]\n", (int)n, field);
        n++;
        field = strtok (NULL, ", \t");
    }

    printf ("\n--- strtol with full error checking ---\n");
    errno = 0;
    value = strtol ("-1234xyz", &end, 10);
    printf ("\"-1234xyz\" -> %ld, errno=%d, stopped at '%s'\n", value, errno, end);
    errno = 0;
    value = strtol ("99999999999999999999", &end, 10);
    printf ("overflow case -> %ld, errno=%d (ERANGE=%d)\n", value, errno, ERANGE);

    printf ("\n--- sscanf: check the return value! ---\n");
    n = sscanf ("id=7 name=bob score=88.5", "id=%d name=%15s score=%lf",
                &id, name, &score);
    printf ("converted %d fields: id=%d name=%s score=%.1f\n",
            (int)n, (int)id, name, score);
    n = sscanf ("garbage here", "id=%d name=%15s score=%lf", &id, name, &score);
    printf ("garbage input converted %d fields (values left unchanged)\n", (int)n);

    printf ("\n--- snprintf builds a line safely ---\n");
    n = snprintf (out, sizeof (out), "%d:%s:%.1f", 7, "bob", 88.5);
    printf ("snprintf wrote %d characters: [%s]\n", (int)n, out);

    return 0;
}
```

真实运行输出:

```
--- strtok: split on commas and blanks ---
field 0: [42]
field 1: [alice]
field 2: [91.5]

--- strtol with full error checking ---
"-1234xyz" -> -1234, errno=0, stopped at 'xyz'
overflow case -> 9223372036854775807, errno=34 (ERANGE=34)

--- sscanf: check the return value! ---
converted 3 fields: id=7 name=bob score=88.5
garbage input converted 0 fields (values left unchanged)

--- snprintf builds a line safely ---
snprintf wrote 10 characters: [7:bob:88.5]
```

【代码做什么？】 1. `strtok(line, ", \t")` 把 `"42, alice , 91.5"` 就地切成三段：`42`、`alice`、`91.5`， 连空格都被跳过了（因为空格也在分隔符集合里）。 2. `strtol("-1234xyz", &end, 10)` 得到 -1234， `end` 指向 `'x'`，于是"数字后面有垃圾"被精确识别。 3. 第二个 `strtol` 演示**溢出**：字符串超出 `long` 范围，函数返回 `LONG_MAX`（本机 `9223372036854775807`） 并把 `errno` 设为 `ERANGE`（34）。**只看返回值会把溢出误当成一次成功的巨大数值**。 4. `sscanf` 在格式匹配时返回转换成功的字段数（3），遇到垃圾输入返回 0， 且**不会修改**输出变量——所以调用前应把它们初始化，否则读到的是不确定的旧值或垃圾值。 5. `snprintf` 有长度上限，返回"假如空间足够本应写入的字符数"（10），并把 `out` 变成 `7:bob:88.5`。

【底层机制透视】 这四个函数代表两种互补的解析策略：**先切分再转换**（`fgets` + `strtok` + `strtol`）与**直接格式匹配**（`sscanf`）。 课程推荐前者处理"人写的文件"：人可能少写字段、多打空格，而 `sscanf` 一次匹配失败后不知道失败在哪， `strtok` 分出的每一段却都能单独校验并**回声给用户**。实测 `errno=34` 提醒我们： **返回值 + `errno` + `endptr` 三者要一起看**，才能把"成功""部分成功""溢出""完全不是数字"分开。 最后注意 `strtok` **就地改写了 `line`**（把分隔符换成 `'\0'`），所以必须传入可写字符串 （`char line[] = "..."`，而不是 `char* line = "..."` 这种指向只读字面量的写法）。

【内存布局图解】

```
  char line[] = "42, alice , 91.5";     ← 可写的栈数组
  0    1  2  3 4 5 6 7  8  9 ...
  '4' '2' '\0' 'a' 'l' 'i' 'c' 'e' '\0' ...
        ▲      ▲                   ▲
        |      |                   └── 第 3 段 "91.5"（先跳过空格）
        |      └── 第 2 段 "alice"
        └── 第 1 段 "42"：strtok 把 ',' 与空格写成 '\0'
  每个 field 指针都指向 line 内部 —— line 一旦失效，所有 field 都变成悬空指针
```

【与汇编的对应】

```
; 字符分类在汇编里就是若干次比较；下面判断 R0 是否为分隔符 ','
; 以及把 ',' 就地改成 '\0'（这就是 strtok 的核心动作）
        LDI R1, COMMA           ; R1 = ',' 的 ASCII 码 44
        NOT R1, R1
        ADD R1, R1, #1          ; R1 = -44
        ADD R2, R0, R1          ; R2 = c - ','
        BRnp NEXT_FIELD         ; 不是分隔符 -> 继续扫描
        AND R2, R2, #0
        STR R2, R3, #0          ; 把分隔符位置写为 '\0'：字段到此结束
        ; 记录下一个字段的起始地址（相当于 strtok 内部的静态变量）
NEXT_FIELD
        ; 数字转换则是循环：digit = c - '0';  acc = acc * 10 + digit
```

`strtok` 的"就地写 `'\0'`"与 `strtol` 的"逐位乘十累加"在汇编里都是极短的循环—— 它们的困难不在指令，而在**边界条件的判断**（溢出、无数字、只有分隔符）。

常见错误与调试技巧

- **不检查 `fopen` 的返回值**：文件不存在时 `fopen` 返回 `NULL`，随后 `fgets(NULL)` 直接崩溃。 **调试**：`perror(argv[1])` 打印失败原因； `strace -e trace=openat ./prog` 看内核拒绝了哪次打开； `gdb` 下 `p in` 确认是否为 `(FILE *) 0x0`。
- **经典的 `while (!feof(f))` 多处理一条（不存在的）记录**：`feof` 只在一次读取失败**之后**才为真。 实测对比（`n6_eof.c`）：正确写法输出 `read 10 / read 20 / read 30` 后循环结束； 错误写法输出 `read 10 / read 20 / read 30 / read 30` ——**最后一行是被重复处理的旧值**。 > 下面是故意写错的演示，请勿模仿： `c while (0 == feof (in)) { /* 错误写法：多跑一轮 */ (void)fscanf (in, "%d", &value); /* 返回值被忽略 */ printf ("%d\n", value); /* EOF 时打印的是上一次的旧值 */ }` **调试**：把循环条件换成对 `fscanf` / `fgets` 返回值的判断； `gdb` 里 `break n6_eof.c:<行号>` 后反复 `p value`、`next`，可以看见最后一轮 `fscanf` 返回 -1 而变量未变。
- **用 `char` 接 `fgetc` / `getc` 的返回值**：`char c = fgetc(f);` 在 `char` 有符号的平台上会把 `0xFF` 变成 -1， 与 `EOF` 混淆，导致文件被提前当成"读完"。**调试**：把类型改成 `int`； `gdb` 下用 `x/16xb buf` 检查输出是否在中途被截断。
- **用 `gets` 读字符串**：它没有长度限制，长输入直接冲垮栈。 **调试**：`gcc -Wall` 会报 `implicit declaration` / `dangerous`；改用 `fgets(buf, sizeof (buf), stdin)`； 用 `gcc -fsanitize=address`（或 `valgrind ./prog`）可以看到 `Invalid write ... on thread 1's stack`。
- **忘记 `fclose` 造成文件描述符泄漏**：循环里反复 `fopen` 不关闭，最后所有 `fopen` 都失败。 **调试**：`ulimit -n` 查看上限； `ls /proc/<pid>/fd | wc -l` 观察描述符数量是否持续增长； `valgrind --track-fds=yes ./prog` 会在退出时列出未关闭的描述符。
- **崩溃前输出"消失"（缓冲区没刷）**：重定向到文件后程序中途 `abort()`/段错误，缓冲区内容全部丢失。 实测（`n7_buffering.c`）：不加 `fflush` 时输出文件**大小为 0**，加了 `fflush(stdout)` 后同样崩溃却写出 **36 字节**。 > 下面是故意让程序崩溃的演示，请勿模仿（结果依平台与缓冲模式而异）： `c for (i = 1; 3 >= i; i++) { printf ("step %d of 3\n", (int)i); } fflush (stdout); /* ← 去掉这一行，重定向到文件时输出会全部丢失 */ abort (); /* abort 不刷新 stdio 缓冲区 */` **调试**： `./prog > out.txt;`

`wc -c out.txt` 检查文件大小是否符合预期； 需要"边跑边看"时用 `stdbuf -oL ./prog` 或调用 `setvbuf(stdout, NULL, _IOLBF, 0)`。

关键要点

- C 的 I/O 是**两层抽象**：内核的文件描述符（0/1/2 与其它小整数）+ 库的 `FILE*` 缓冲流。 程序因此可以完全不知道"对面"是文件、管道还是设备。
- **每一次 I/O 调用都必须检查返回值**： `fopen` 要查 `NULL`， `fscanf` / `fgets` / `fgetc` 要查转换结果， `fclose` 要查是否写出成功； `feof` 只能用来事后判断原因，不能用来当循环条件。
- 行式文本用 `fgets` + `sscanf` / `strtok` / `strtol` 解析（因为能对失败的行做回声与重试）， 二进制数据用 `fread` / `fwrite`（更省空间更快，但不可移植、需要扁平化指针）。
- 重定向（ `>`、`<`、`2>` ）与管道（ `|` ）只是**在启动前替换描述符**， 把输出写到 `stdout`、错误写到 `stderr` 的程序天然就能与它们协作。
- 缓冲区在 `return` / `exit` 时自动刷新，在 `abort` /崩溃时**不会**； 在乎输出的顺序与完整性时，要显式 `fflush` 或设置缓冲模式。

思考题（带答案）

问题 1：下面的代码想读出一个文件里所有的数字之和，它错在哪里？给出两种正确写法。

```
int sum = 0, v;
while (!feof (f)) {
    fscanf (f, "%d", &v);
    sum += v;
}
```

答案：错在两处。① `feof(f)` 在**一次读取失败之后**才为真，因此循环体会多执行一轮， 此时 `fscanf` 返回 0/EOF 且**不修改** `v`，于是把上一次的数字**重复累加**了一次； ② `fscanf` 的返回值被完全忽略，无法区分"读到一个 0""格式不匹配""到达文件尾"。 正确写法一： `while (1 == fscanf (f, "%d", &v)) { sum += v; }`； 正确写法二：先用 `fgets` 读行，再在行内用 `sscanf` / `strtol` 解析并统计成功次数。 两种写法都把**输入函数的返回值**放在循环条件里，这是处理 EOF 的唯一可靠方式。

问题 2： `./prog < in.txt > out.txt` 中，如果程序用 `printf` 打印进度、用 `fprintf(stderr, ...)` 打印错误， 用户会看到什么？为什么这样设计有用？

答案：正常输出（进度信息）进入 `out.txt`，错误信息仍显示在终端上。 因为重定向只替换**被指定的描述符**： `>` 默认只改描述符 1（stdout），描述符 2（stderr）仍指向终端。 这样程序可以把"结果数据"与"给人看的诊断信息"分流：结果便于交给下一个程序（ `|` ）， 错误也不会被淹没在数据里。若希望错误也进文件，需显式写 `2> err.txt` 或 `> out.txt 2>&1`； 反过来，把错误打进 stdout 的程序在 `prog > out.txt` 时会让用户"看不到任何错误"。

问题 3：为什么 `fread` / `fwrite` 存下来的结构体文件通常不能跨机器读取？举出三个具体原因。

答案：① **字节序 (endianness)**：x86-64 是小端， `int32_t 1` 写成 `01 00 00 00`，大端机器读出来是 `0x01000000`； ② **结构体填充 (padding)**：本机实测 `struct {int32_t id; char tag[8]; double`

`value;}` 的 `sizeof` 是 24 （`4 + 8 + 8 = 20`，另有 4 字节填充），换编译器或换 ABI 后字段偏移就可能全部错位； ③ **表示差异**：`double` 的 IEEE 754 编码、`long` 的宽度、甚至 `char` 是否有符号都可能不同。再加上指针本身是地址（必须扁平化成下标或偏移）。所以可移植的持久化格式要么用**文本**， 要么显式规定"大端、固定宽度、无填充"的二进制协议。

Lecture 15: 数据结构：结构体、typedef 与信息隐藏 (Data Structures: structs, typedef and Information Hiding)

概述

本讲要解决的问题是：**当程序需要同时记录一件事物的多个属性（书的作者、标题、ISBN、页数、价格）时，如何把它们作为一个整体来组织、传递与保护**。引入的机制是 C 的**结构体 (struct)**、 **类型别名** `typedef`、**枚举** `enum`，以及把"接口"与"实现"分开的**头文件 + 不透明类型 (opaque type)** 惯用法。 这一讲是第 16 讲动态内存分配的直接前提——`malloc` 返回的每个对象本质上都是一块"由头文件声明的类型、 由 `.c` 文件私有定义其字段"的数据，而课程真实的 `mem220` 分配器正是这种设计的完整范例。

核心概念与底层机制图解

- **结构体：定义 (definition) 与声明 (declaration)**：`struct book_t {...};` 只是**定义类型**，不分配内存；`struct book_t book;` 才是**声明一个变量**（对象）。
  - *直观解释*：定义像"表格模板"（有哪些栏），声明像"填好的一张表"；模板本身不占抽屉。
  - *底层机制图解*：字段按定义顺序**连续排列**，编译器为每个字段记住固定的**偏移 (offset)**； `book.pages` 因此编译成"取 `book` 的地址 + 偏移 160，再按 `int32_t` 访问"，对齐由编译器负责。
  - *作用域与存储期*：类型名是文件作用域；变量的存储期取决于声明位置——函数内 automatic（随帧生灭）、 函数外 static、`malloc` 出来的 dynamic（见第 16 讲）。
- **`sizeof` 与"成员之和"不是一回事**：实测 `pad_t` 的成员之和是 6 字节， `sizeof` 却是 12。
  - *直观解释*：`sizeof` 是"抽屉实际占多大"，包含为对齐留的空隙。
  - *底层机制图解*：编译器要保证每次字段访问都对齐，因此在字段之间与结构体末尾插入**填充 (padding)**。 实测 `struct {char author[50]; char title[100]; uint64_t isbn; int32_t pages; double price;}` 的 `sizeof` 是 **176**、成员之和是 **170**：`title` 结束于 150， `isbn` 需 8 字节对齐，于是插了 2 字节空隙。 **永远不要手算结构体大小**，用 `sizeof (变量名)`，因为它随 ISA、OS、编译器与编译选项变化。
  - *作用域与存储期*：`sizeof` 是编译期运算符（变长数组除外），不产生运行时代码。
- **数组的结构体 vs 结构体的数组 (AoS vs SoA)**：
  - *直观解释*：AoS 是"每本书一张卡片，卡片排成一摞"；SoA 是"把所有书的作者抄在一张纸上、 所有书的价格抄在另一张纸上"。

- *底层机制图解*：`book_t shelf[10]` 中每个元素占 `sizeof(book_t)` 字节， 所以 `shelf[i]` 的地址是 `base + i * 176`——**跨步 (stride) 就是结构体大小**， 循环访问同一字段时每次要跳过一大段内存，对缓存不友好；SoA （`char author[10][50]; double price[10];`）让每个字段各自连续，适合批量处理单个字段。 两者存储期一致，差别在"复制一本书"的粒度：176 字节还是 50 字节。

- **按值传递/返回结构体 vs 传指针**：
  - *直观解释*：按值传递像"把整本书复印一份交给对方"，传指针像"告诉他书在第几排第几格"。
  - *底层机制图解*：C 的参数传递是**按值 (call-by-value)**，所以把结构体作为参数会**复制整个结构体**到栈上； 课程示例里的 `stack_t` 含 `char data[500][200]`，约 **100,000 字节**， 每次调用复制它是不可接受的。因此约定是：**传结构体的地址**（`const stack_t* s`）， 并用 `->` 访问字段；只在结构体很小（如两个 `double` 构成的复数）时才考虑按值。 返回结构体同理：`return s;` 会把整个结构体复制回调用者。
  - *作用域与存储期*：被调用者拿到的是**调用者对象的别名**（通过指针）， 所以它能修改调用者的数据；这正是 `stack_init (&s)` 能改变 `s.top` 的原因。

- **`typedef` 与 `enum`：给类型取名字**：
  - *直观解释*：`typedef` 像给类型起绰号；`enum` 像给一组整数贴标签（`SPACE_FULL` 比 `1` 好读）。
  - *底层机制图解*：`typedef struct player_t player_t;` 让类型名与结构体标签同名，从此不必写 `struct`； 课程代码约定用 `typedef struct {...} name_t;`。`enum` 成员就是**整型常量**，默认从 0 递增、可显式赋值； 两个常用惯用法：① 末尾放"计数"名（`NUM_SPACE_TYPES`），数组大小随枚举自动调整； ② 作**位向量**给 bit 命名（`LEFT_WALL=1, RIGHT_WALL=2, UPPER_WALL=4, LOWER_WALL=8, HAS_EXIT=16`）。 实测 `sizeof (space_type_t) = 4`， `SPACE_EMPTY/FULL/BLOCK = 0/1/2`，与 `switch` 天然搭配。
  - *作用域与存储期*：两者都只影响"名字"，不产生运行时对象；`enum` 常量是编译期常量，可当数组长度。

- **结构体里的函数指针：穷人的虚函数 (poor man's virtual method)**：
  - *直观解释*：给每张表格附上一栏"该找谁办这件事"，不同的表格填不同的办事员。
  - *底层机制图解*：`struct shape_t { double a, b; double (*area)(const shape_t*); }` 把"数据"和"操作"放进同一个对象， `s->area (s)` 先取函数指针再间接调用—— 在 x86-64 上是一条 `call *%rax`，与 C++ 虚函数表 (vtable) 的机制同源。 实测 `sizeof (shape_t) = 40`，其中两个函数指针各占 8 字节（`area` 在偏移 24、`name` 在偏移 32）。 调用点 `describe (&table[0])` 完全不需要知道对象究竟是矩形还是圆，这就是**多态 (polymorphism)** 的雏形。
  - *作用域与存储期*：函数名在表达式里**退化为函数指针**；结构体赋值会连同函数指针一起复制 （实测 `table[0] = table[1]` 之后，两次 `describe` 都打印 circle）。

- **信息隐藏 (Information Hiding)：头文件是接口， `.c` 文件是私有实现**：
  - *直观解释*：`stdio.h` 也是这个套路——你天天用 `FILE*`，但从不（也不应该）去碰它的字段。
  - *底层机制图解*：把**类型的前向声明 (forward declaration)** 放进头文件、 把**字段定义**放进 `.c` 文件，使用者的代码就只能操控指针，无法依赖内部布局： `c /* memory_system.h — 只有接口，没有字段 */ #if !defined(_MEMORY_SYSTEM_H) #define _MEMORY_SYSTEM_H typedef struct memory_system`

`memory_system_t; /* 不完整类型 */ memory_system_t* memory_system_create (size_t capacity); void memory_system_destroy (memory_system_t* ms); #endif /* !defined(_MEMORY_SYSTEM_H) */` `memory_system_t` 在这里是**不完整类型 (incomplete type)**：编译器知道这个名字， 但不知道它有多大，所以 `memory_system_t ms;` 或 `ms->capacity` 都会**编译失败**—— 这正是我们想要的保护。课程的真实案例 `mem220.h` 更彻底：它**只声明四个函数**， 连 `mem_block_t` 这个名字都不暴露，把块头结构、空闲链表数组、`log2_ceil` 全部关在 `mem220.c` 里。 头文件还必须写**包含守卫 (include guard)**，否则同一个头被两条 `#include` 路径各包含一次时， 类型会被重复定义而编译报错。

- *作用域与存储期*：不透明类型对象的**存储期由创建/销毁函数管理**（`_create` 里 `malloc`，`_destroy` 里 `free`），使用者只持有"**句柄 (handle)**"——一个不透明指针。 典型签名会带上 `const`（如 `memory_system_fetch (const memory_system_t*, ...)`）来表达"只读"。

结构体布局图解（`pad_t` 的实测字节）：

```
struct pad_t { char a; int32_t b; char c; };      sizeof = 12（成员之和只有 6）

偏移:   0     1     2     3     4     5     6     7     8     9    10    11
      +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
      |  a  | ### | ### | ### |        b (4 字节)      |  c  | ### | ### | ### |
      +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
        'A'=41  填充    填充    填充   44 33 22 11    'C'=43  填充（尾部填充）
      实测字节: 41 00 00 00 44 33 22 11 43 00 00 00
                           ↑ b 必须在 4 的倍数地址上，所以先塞 3 个洞
                                                  ↑ 数组里下一个元素也要 4 字节对齐，故尾部再补 3

换成 struct tight_t { char a; char c; int32_t b; };  sizeof = 8（没有内部填充）
      实测字节: 41 43 00 00 44 33 22 11        ← 字段顺序影响大小!
```

AoS / SoA 图解：

```
数组的结构体 (Array of Structs):  book_t shelf[3]
base+0                    base+176                  base+352
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ book0 的 176 字节 │      │ book1 的 176 字节 │      │ book2 的 176 字节 │
└──────────────────┘      └──────────────────┘      └──────────────────┘
取 shelf[i].price 的地址 = base + i*176 + 168     ← 跨步很大

结构体的数组 (Struct of Arrays):
char   author[3][50]  ← 连续      double price[3]  ← 连续
批量取价格时内存访问是顺序的，缓存友好
```

代码示例与底层机制分析

示例 1：填充、对齐与 `sizeof`

代码 (C):

```c
/* l1_layout.c
 * 编译： gcc -g -std=c99 -Wall -Werror l1_layout.c -o l1_layout */
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>

/* The layout from lecture: one byte, four bytes, one byte. */
typedef struct {
    char    a;
    int32_t b;
    char    c;
} pad_t;

/* The same fields reordered so that no interior padding is needed. */
typedef struct {
    char    a;
    char    c;
    int32_t b;
} tight_t;

/* A book, from the structured-data lecture. */
typedef struct {
    char     author[50];
    char     title[100];
    uint64_t isbn;
    int32_t  pages;
    double   price;
} book_t;

static void
dump (const char* label, const void* p, size_t n)
{
    const unsigned char* b = p;
    size_t i;

    printf ("%-11s:", label);
    for (i = 0; n > i; i++) {
        printf (" %02X", b[i]);
    }
    printf ("\n");
}

int
main (void)
{
    pad_t  p;
    tight_t t;

    memset (&p, 0, sizeof (p));
    memset (&t, 0, sizeof (t));
    p.a = 'A'; p.b = 0x11223344; p.c = 'C';
    t.a = 'A'; t.c = 'C';        t.b = 0x11223344;
```

```
    printf ("--- pad_t { char a; int32_t b; char c; } ---\n");
    printf ("sizeof = %ld, offsets: a=%ld b=%ld c=%ld\n", (long)sizeof (pad_t),
            (long)offsetof (pad_t, a), (long)offsetof (pad_t, b),
            (long)offsetof (pad_t, c));
    dump ("bytes", &p, sizeof (p));

    printf ("\n--- tight_t { char a; char c; int32_t b; } ---\n");
    printf ("sizeof = %ld, offsets: a=%ld c=%ld b=%ld\n", (long)sizeof (tight_t),
            (long)offsetof (tight_t, a), (long)offsetof (tight_t, c),
            (long)offsetof (tight_t, b));
    dump ("bytes", &t, sizeof (t));

    printf ("\n--- book_t ---\n");
    printf ("sizeof = %ld bytes, sum of members = %ld bytes\n", (long)sizeof (book_t),
            (long)(50 + 100 + 8 + 4 + 8));
    printf ("offsets: author=%ld title=%ld isbn=%ld pages=%ld price=%ld\n",
            (long)offsetof (book_t, author), (long)offsetof (book_t, title),
            (long)offsetof (book_t, isbn), (long)offsetof (book_t, pages),
            (long)offsetof (book_t, price));
    printf ("title ends at %ld, isbn starts at %ld -> %ld bytes of padding\n",
            (long)(offsetof (book_t, title) + 100), (long)offsetof (book_t, isbn),
            (long)(offsetof (book_t, isbn) - (offsetof (book_t, title) + 100)));
    return 0;
}
```

真实运行输出:

```
--- pad_t { char a; int32_t b; char c; } ---
sizeof = 12, offsets: a=0 b=4 c=8
bytes      : 41 00 00 00 44 33 22 11 43 00 00 00

--- tight_t { char a; char c; int32_t b; } ---
sizeof = 8, offsets: a=0 c=1 b=4
bytes      : 41 43 00 00 44 33 22 11

--- book_t ---
sizeof = 176 bytes, sum of members = 170 bytes
offsets: author=0 title=50 isbn=152 pages=160 price=168
title ends at 150, isbn starts at 152 -> 2 bytes of padding
```

【代码做什么？】 1. `pad_t` 把 `a='A'` 放在偏移 0，`b=0x11223344` 放在偏移 4，`c='C'` 放在偏移 8。 2. 用 `dump` 把结构体的**原始字节**打出来：`41 00 00 00 44 33 22 11 43 00 00 00` ——`a` 后面 3 个 `00` 是**内部填充**，`c` 后面 3 个 `00` 是**尾部填充**，所以 `sizeof = 12`。 3. `tight_t` 只是把 `b` 挪到最后：`sizeof` 变成 **8**，字节为 `41 43 00 00 44 33 22 11`。 4. `book_t` 打印每个字段的 `offsetof`，并算出 `title`（结束于 150）与 `isbn`（起始于 152）之间的 2 字节空隙。 5. 三次输出共同说明一件事：**结构体的大小与布局由编译器按对齐规则决定，不能手算**。

【底层机制透视】 对齐规则的表述是：**每个字段必须放在它自身大小的整数倍偏移上**（`int32_t` → 4 的倍数，`double`/`uint64_t` → 8 的倍数），而结构体整体的对齐是所有字段对齐要求的最大值，因此结构体总

大小会被补齐到该对齐的倍数（这解释了 `pad_t` 尾部的 3 个字节： `a` 占 1 字节、`b` 占 4 字节、`c` 占 1 字节共 6 字节，但数组 `pad_t v[2]` 里第二个元素的 `b` 也必须 4 字节对齐，所以整体补到 12）。 多数支持字节寻址的 ISA 要求"N 字节的载入/存储必须落在 N 的倍数地址上" （从 `0x20000001` 载入 32 位值会崩溃；即使允许未对齐访问，速度也可能慢两个数量级）， 所以编译器**必须插入填充**。实用技巧：**大字段排在前面、小字段排在后面**通常能减少填充 （`tight_t` 比 `pad_t` 小 4 字节）；确实需要紧凑布局时，应该按字节手工序列化，而不是依赖 `#pragma pack`。

【内存布局图解】（`pad_t` vs `tight_t`，见前面的"结构体布局图解"）

【与汇编的对应】 LC-3 没有"结构体"：字段访问完全是**基址 + 偏移**。设 `R2` 存放 `book_t* book`， 那么 `book->pages` （偏移 160）与 `book->price` （偏移 168）就是：

```
        ; 假设 R2 = &book；LC-3 的 LDR 偏移只有 6 位（0..63），大偏移要先算地址
        ; 注意：R4 是全局数据指针、R5 是帧指针、R6 是栈指针，临时值只用 R0-R3
        LEA R3, BOOK_T_PAGES_OFF   ; R3 = 160
        ADD R3, R2, R3             ; R3 = &book.pages
        LDR R0, R3, #0             ; R0 = book.pages
        LEA R3, BOOK_T_PRICE_OFF   ; R3 = 168
        ADD R3, R2, R3             ; R3 = &book.price
        LDR R0, R3, #0             ; R0 = book.price 的低 16 位
        ADD R3, R3, #1
        LDR R1, R3, #0             ; R1 = 高 16 位（两半拼成一个 32 位值）
; x86-64 上同一段代码会编译成带立即数偏移的取数指令：
;     mov 0xa0(%rdi), %eax        ; book->pages，偏移 0xa0 = 160
;     movsd 0xa8(%rdi), %xmm0     ; book->price，偏移 0xa8 = 168
```

`->` 在机器层面**就是一条带立即数偏移的取数指令**，没有别的玄机；填充的存在只是让这些偏移"对齐"。

示例 2：结构体 + 操作函数（把接口写成函数，把实现留在帧外）

代码 (C):

```
/*
 * l2b_stack_t.c -- a struct plus the operations that go with it.
 * Compile: gcc -g -std=c99 -Wall -Werror l2b_stack_t.c -o l2b_stack_t
 */

#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define MAX_LINES 4
#define MAX_LEN   24

typedef struct {
    char    data[MAX_LINES][MAX_LEN];   /* the lines, stored newest-first */
    int32_t top;                        /* index of the top; MAX_LINES = empty */
} stack_t;

static int32_t stack_empty (const stack_t* s) { return (MAX_LINES == s->top); }
static int32_t stack_full  (const stack_t* s) { return (0 == s->top); }

/* Returns 1 on success, 0 if the stack is full or the string does not fit. */
static int32_t
stack_push (stack_t* s, const char* str)
{
    char*   write;
    int32_t i;

    if (stack_full (s)) {
        return 0;
    }
    write = s->data[--s->top];              /* decrement, then use the index */
    for (i = 0; '\0' != *str; i++) {
        if (MAX_LEN - 1 == i) {
            s->top++;                       /* undo the decrement, then fail */
            return 0;
        }
        *write++ = *str++;
    }
    *write = '\0';
    return 1;
}

/* Returns 1 on success, 0 on failure.  Fills buf, truncating if needed. */
static int32_t
stack_pop (stack_t* s, char* buf, int32_t len)
{
    const char* read = s->data[s->top];
    int32_t     i;

    if (stack_empty (s)) {
        return 0;
    }
    for (i = 1; len > i && '\0' != *read; i++) {
```

```
        *buf++ = *read++;
    }
    *buf = '\0';
    s->top++;
    return 1;
}

int
main (void)
{
    stack_t s;
    char    buf[MAX_LEN];

    /* Passing a stack_t by value would copy all 100 bytes; we pass a pointer. */
    printf ("sizeof (stack_t) = %ld bytes\n", (long)sizeof (stack_t));
    printf ("&s = %p, &s.data[0][0] = %p (same object, offset 0)\n",
            (void*)&s, (void*)&s.data[0][0]);
    printf ("offset of top = %ld (measured with pointer subtraction)\n",
            (long)((char*)&s.top - (char*)&s));

    s.top = MAX_LINES;                      /* stack_init (&s) */
    printf ("push \"first\"  -> %d\n", (int)stack_push (&s, "first"));
    printf ("push \"second\" -> %d\n", (int)stack_push (&s, "second"));
    printf ("push \"third\"  -> %d\n", (int)stack_push (&s, "third"));
    printf ("push a 30-char string -> %d (too long for %d bytes)\n",
            (int)stack_push (&s, "012345678901234567890123456789"), MAX_LEN);
    printf ("top is still %d after the failed push\n", (int)s.top);

    while (!stack_empty (&s)) {
        (void)stack_pop (&s, buf, MAX_LEN);
        printf ("pop -> %s\n", buf);
    }

    return 0;
}
```

真实运行输出:

```
sizeof (stack_t) = 100 bytes
&s = 0x7ffd095fb560, &s.data[0][0] = 0x7ffd095fb560 (same object, offset 0)
offset of top = 96 (measured with pointer subtraction)
push "first"  -> 1
push "second" -> 1
push "third"  -> 1
push a 30-char string -> 0 (too long for 24 bytes)
top is still 1 after the failed push
pop -> third
pop -> second
pop -> first
```

> 上面的 `&s` 地址是**示意值**（每次运行因 ASLR 而不同）；关键是它与 `&s.data[0][0]` **完全相同**， 而 `top` 的偏移恒为 96 = `4 × 24`，即结构体前 96 字节全是 `data`， `top` 紧接其后。

【代码做什么？】 1. `stack_t` 把"400 字节的二维字符数组 + 一个栈顶下标"聚合在一起， `sizeof` 实测 100 字节 （4×24 = 96 字节数据 + 4 字节 `top`，对齐到 4 的倍数）。 2. `&s` 与 `&s.data[0][0]` 打印出**同一个地址**，证明 `data` 就在偏移 0 处、没有额外开销； `top` 的偏移实测是 96。 3. `main` 把 `top` 设为 4（空栈）； `stack_push` 先 `--s->top` 再写字符串，所以栈是**向下增长**的。 4. 前三次 push 都成功；第四次传入 30 字符的串超过了 23 字符的上限， `stack_push` **撤销了已经做的** `--s->top` 并返回 0，因此 `top` 仍然是 1。 5. 最后循环 `stack_pop` 直到空栈，输出顺序是 `third / second / first` （后进先出）。

【底层机制透视】 `stack_empty` 的参数是 `const stack_t*`： `const` 承诺"我不会修改你指向的对象"， 指针则避免复制 100 字节。 `->` 完全等价于 `(*s).top` ——**因为** `.` **的优先级高于** `*`， **所以必须写括号**，C 才提供 `->` 作为简写。实测 `offset of top = 96` 也说明了 "字段访问 = 基址 + 常量偏移"这一本质： `s->top` 编译成从 `s` 的第 96 字节处取 4 字节。 `stack_push` 里"失败时把 `top` 加回来"体现了**要么完整成功、要么完全不动**的接口原则； 忘了这一步，失败的 push 会永久吃掉一个栈槽，这类"状态泄漏"是最难查的错误之一。 另外， `stack_push` **复制**了字符串， `stack_pop` 也必须复制到调用者提供的数组里—— 因为栈内那份随时会被下一次 push 覆盖。课程对此的总结是 "**信息隐藏与性能有时互相冲突**"：复制两次是浪费，但换来调用者不必关心栈内部的生命周期。

【内存布局图解】

```
stack_t s（在 main 的栈帧里，起始地址实测 0x7fffa87c6980）
┌──────────────────────────────────────────────────┐
│ s.data[0][0..23] ← 偏移 0    │ s.data[1] ← 偏移 24│
│ s.data[2] ← 偏移 48          │ s.data[3] ← 偏移 72│
├──────────────────────────────────────────────────┤
│ s.top ← 偏移 96（实测，地址 0x7fffa87c69e0）      │   共 100 字节
└──────────────────────────────────────────────────┘

push: top=4（空）→ "first" 存 data[3]，top=3 → "second" 存 data[2]，top=2
      → "third" 存 data[1]，top=1；pop 读 data[top] 后 top++，顺序反过来。
```

【与汇编的对应】

```
; stack_init: s->top = MAX_LINES    (R0 = 指向 stack_t 的指针)
STKINIT LEA R1, MAX_LINES_VAL
        ADD R2, R0, #96         ; top 的偏移是 96 (实测)
        STR R1, R2, #0          ; (*s).top = 4
        RET
; stack_push: 先 --s->top, 再把字符逐个写进 s->data[top]
STKPUSH ADD R2, R0, #96
        LDR R1, R2, #0          ; R1 = s->top
        ADD R1, R1, #-1         ; --s->top
        STR R1, R2, #0          ; 写回
        ; data[top] 的地址 = s + top*24 (每行 24 字节): 用循环连加 24 次求偏移
        ; 注意: R5 是帧指针、R4 是全局数据指针, 临时值只用 R0-R3
COPYLP  LDR R2, R1, #0          ; R2 = *str (R1 = 读指针, R3 = 写指针)
        STR R2, R3, #0          ; *write = *str
        ADD R3, R3, #1
        ADD R1, R1, #1
        ; ... 判断是否到达 '\0' 或长度上限 ... ; BRnzp COPYLP
```

`->` 与 `[]` 在汇编里都是"**基址 + 偏移**"的算术：`s->top` 是固定偏移 96， 而 `s.data[top]` 的偏移要先算 `top * 24`。这正是讲指针与数组时反复出现的同一条规则。

案例研究：不透明类型与真实的 `mem220` 分配器

接口（`memory_system.h`，节选，完整文件见随附源码）：

```
/* 只有接口, 没有任何字段 */
#if !defined(_MEMORY_SYSTEM_H)
#define _MEMORY_SYSTEM_H

typedef struct memory_system memory_system_t;      /* 不完整类型: 只知道名字 */

memory_system_t* memory_system_create (size_t capacity);
void             memory_system_destroy (memory_system_t* ms);
int32_t          memory_system_store (memory_system_t* ms, int32_t value);
int32_t          memory_system_fetch (const memory_system_t* ms, size_t index, int32_t* out)
size_t           memory_system_count (const memory_system_t* ms);

#endif /* !defined(_MEMORY_SYSTEM_H) */
```

实现（`memory_system.c`，节选）——字段定义只出现在这个文件里：

```
#include "memory_system.h"
struct memory_system {                  /* 使用者永远看不到这个定义 */
    int32_t* data;                      /* 动态分配的数组 */
    size_t   capacity;
    size_t   count;
};
memory_system_t*
memory_system_create (size_t capacity)
{
    memory_system_t* ms;

    if (0 == capacity || NULL == (ms = malloc (sizeof (*ms)))) { return NULL; }
    ms->data = malloc (capacity * sizeof (*ms->data));   /* 大小只有本文件知道 */
    if (NULL == ms->data) { free (ms); return NULL; }    /* 不要泄漏外层对象   */
    ms->capacity = capacity;  ms->count = 0;
    return ms;
}
```

使用者视角（`s5_main.c`，节选）与实测输出:

```
    memory_system_t* ms = memory_system_create (4);
    /* 下面这行无法编译——这正是信息隐藏的效果:
       printf ("%ld\n", ms->capacity);   // error: incomplete type */
```

```
store   0 : ok
store  10 : ok
store  20 : ok
store  30 : ok
store  40 : rejected (full)
store  50 : rejected (full)
count = 4
fetch [0] : ok, value = 0
fetch [4] : out of range
destroyed
```

真实课程代码：`mem220.h` / `mem220.c`。这一对文件把不透明做到极致—— 头文件**连一个结构体名都不暴露**，只声明四个函数（与 C 库的 `malloc` / `calloc` / `realloc` / `free` 一一对应）:

```
/* mem220.h（节选） */
#if !defined(_MEM220_H)
#define _MEM220_H                       /* ← 包含守卫: 防止重复定义 */
#include <stdint.h>
#define MEM220_MAX_ALLOC_LOG  20
#define MEM220_MAX_ALLOC      (1UL << MEM220_MAX_ALLOC_LOG)   /* 1 MiB */
void*    mem220_allocate (size_t n_bytes);            /* 对应 malloc  */
void*    mem220_allocate_and_zero (size_t n_bytes);   /* 对应 calloc  */
int32_t  mem220_reallocate (void** ptr_to_ptr, size_t n_bytes);  /* realloc */
void     mem220_free (void* ptr);                     /* 对应 free    */
#endif /* !defined(_MEM220_H) */
```

实现在 `mem220.c` 里，**全部私有**：块头类型、空闲链表数组、`log2_ceil` 都是 `static`（文件作用域），外部代码既看不到也改不了：

```
/* mem220.c（节选）—— 这些名字在 mem220.h 里完全不出现 */
typedef struct mem_block_t mem_block_t;
struct mem_block_t {
    size_t       size;        /* 块的字节数（2 的幂） */
    mem_block_t* next;        /* 空闲链表指针 */
};

static uint8_t*     free_bytes;                      /* 未分配的内存起点   */
static size_t       n_free_bytes;                    /* 未分配的字节数     */
static mem_block_t* mem_bin[MEM220_MAX_ALLOC_LOG+1]; /* 按 2^k 分箱的表头  */
static int32_t      init_done = 0;

void*
mem220_allocate (size_t n_bytes)
{
    size_t       block_size;
    int32_t      bin;
    mem_block_t* new_block;

    if (!init_done) { mem220_init (); }              /* 首次调用时初始化   */
    block_size = n_bytes + sizeof (*new_block);      /* 加上块头的开销     */
    if (0 == n_bytes || MEM220_MAX_ALLOC < block_size) {
        return NULL;                                 /* 0 字节或超大请求   */
    }
    bin = log2_ceil (block_size);                    /* 找到 2^k 的箱子    */

    if (NULL != mem_bin[bin]) {                      /* 箱子里有旧块?      */
        new_block = mem_bin[bin];                    /* 从链表头摘下来     */
        mem_bin[bin] = new_block->next;
    } else {
        n_bytes = (1UL << bin);                      /* 真正分配 2^k 字节  */
        if (n_bytes > n_free_bytes) { return NULL; }
        new_block = (mem_block_t*)free_bytes;        /* 从堆前面切一块     */
        free_bytes += n_bytes;
        n_free_bytes -= n_bytes;
        new_block->size = n_bytes;                   /* 把大小写进块头     */
    }
    return (new_block + 1);                          /* 返回块头之后的数据 */
}
```

`mem220_free` 只有四行有效代码：用 `mem_block[-1].size` **从 `ptr` 前面读回块头**，算出 bin 号，再把块头插进 `mem_bin[bin]` 链表的表头——这就是"自由链表头插、LIFO 复用"的全部秘密。

真实运行输出（用课程自带的测试 + 一个自写驱动）：

```
$ gcc -g -std=c99 -Wall -Werror mem220.c mem220_test.c -o mem220_test && ./mem220_test
（无输出，退出码 0：1000 字节数据经两次 realloc 后逐字节校验全部通过）

$ ./mem220_demo
MEM220_MAX_ALLOC = 1048576 bytes
five requests of 100 bytes : small[1] - small[0] = 128 bytes    （块大小 2^7）
five requests of 1000 bytes: big[1]   - big[0]   = 1024 bytes   （块大小 2^10）
free big[1], big[3], then allocate two more:
  again[0] = 0x7f4b493ffea0  ← 最后释放的 big[3] 先被复用（链表头插，LIFO）
  again[1] = 0x7f4b493ff6a0
reallocate to 3000 bytes: returned 0, bytes that changed during the move: 0
interface limits: allocate(0) = (nil); allocate(1048577) = (nil);
                  allocate(1000000) = 0x7f4b49401aa0
```

这个案例把本讲的几条原则全部体现出来：① **接口与实现分离**——使用者只需要四个原型， 看不到 `mem_block_t` 与分箱表；② **`static` 是头文件之外的第二道墙**—— `mem220_init` 与 `log2_ceil` 在链接器层面就不可见；③ **头文件必须能被安全地包含多次**—— `#if !defined(_MEM220_H)` 守卫保证了这一点； ④ **值-结果参数 (value-result argument)**—— `mem220_reallocate (void** ptr_to_ptr, size_t)` 接收"指针的地址"且只在成功时改写 `*ptr_to_ptr`，从签名上避免了 `ptr = realloc (ptr, n)` 的泄漏陷阱。

常见错误与调试技巧

- **手工计算结构体大小或字段偏移**： `malloc (4 + 50)` 之类的写法在换平台后立刻出错。 **调试**：用 `sizeof (变量)` 与 `offsetof (类型, 字段)`； `gcc -Wpadded` 会提示每个填充位置。
- **用 `==` 比较结构体**：C 不支持整体比较（ `a == b` 编译报错），手写比较又容易漏字段。 **调试**：写显式的比较函数；按字节比较要用 `memcmp (&a, &b, sizeof (a))`， 但必须先用 `memset` 清零，因为填充字节不会被赋值、内容是随机的。
- **按值传递大结构体**： `int32_t f (stack_t s)` 会复制 100 KB，栈可能直接爆掉。 **调试**： `gcc -fstack-usage`； `gdb` 里 `bt` 后 `info frame`。
- **混淆 `.` 与 `->`**： `s->top` 与 `(*s).top` 等价，而 `*s.top` 是错的（ `.` 优先级更高）。 **调试**：编译器会报 `request for member 'top' in something not a structure or union`； 用 `gdb` 的 `p *s` 与 `p s->top` 交叉验证。
- **忘记写包含守卫**：同一个头被间接包含两次 → `redefinition of 'struct ...'`。 **调试**： `gcc -E file.c | grep -n "struct book_t"` 看预处理结果里出现了几次。
- **在不透明类型上写 `p->field`**：报 `dereferencing pointer to incomplete type`。 这不是 bug 而是**设计生效了**。**调试**：确认自己用的是接口函数； 若确实需要该字段，说明接口缺少一个访问函数（例如 `memory_system_count()`），应该加函数而不是暴露字段。

关键要点

- 结构体定义只是**定义类型**，不分配内存；字段按定义顺序连续存放，访问即"基址 + 编译期常量偏移"。
- 编译器为对齐插入**填充**： `sizeof` 通常大于成员之和，字段顺序会影响大小；永远用 `sizeof` 而不要手算。

- C 的参数传递是按值：**结构体参数会被整体复制**，所以约定是传 `const T*` 并用 `->`（`->` 与 `(*p).m` 等价）。
- `typedef` 让 `struct` 关键字消失，`enum` 把整数命名化（可作 `switch` 标签、位向量与数组长度）。
- **信息隐藏 = 头文件放接口 + `.c` 文件放表示**：不透明类型让使用者只能拿句柄、调函数；`mem220.h` / `mem220.c` 是这一原则在课程里的完整范例。

思考题（带答案）

问题 1：下面的结构体在 64 位 Linux/x86-64 上 `sizeof` 是多少？怎样改写能变小？为什么小了的版本更快也更容易缓存？

```
struct s_t { char a; double b; char c; int32_t d; };
```

答案：`a` 在偏移 0；`double b` 需 8 字节对齐，故填 7 字节，`b` 在 8..15；`c` 在 16；`int32_t d` 需 4 字节对齐，填 3 字节后落在 20..23 → 总大小 **24**。 按"大字段优先"重排为 `{ double b; int32_t d; char a; char c; }`：`b` 在 0..7、`d` 在 8..11、`a` 在 12、`c` 在 13，尾部补 2 字节 → `sizeof = 16`，省下 33%。元素更小意味着同样大小的 cache line 能装下更多元素、遍历时 miss 更少，所以更快。

问题 2：为什么使用者拿到 `memory_system_t*` 之后写 `ms->count` 会编译失败？如果确实需要知道元素个数，应该怎么办？

答案：因为 `memory_system.h` 里只有 `typedef struct memory_system memory_system_t;` 这条**前向声明**， `struct memory_system` 是**不完整类型**，编译器不知道它有哪些字段、也不知道它多大， 因此无法计算 `->count` 的偏移，也无法为 `memory_system_t ms;` 分配空间。 这正是**信息隐藏的目的**：使用者只能依赖接口，不能依赖内部布局， 于是实现可以在不改动使用者代码的前提下更换数据结构（例如把数组换成链表）。需要元素个数时应当**在接口里提供访问函数**，本讲示例中的 `memory_system_count()` 就是这样一个函数；`mem220` 里也有类似的思路——使用者想知道块大小，只能通过接口语义（自己记住请求的字节数）， 而不该去读 `ptr[-1]` 那个私有块头（虽然 `mem220.c` 内部确实这么做了）。

问题 3：`stack_push` 为什么要“复制”字符串？如果改成只保存调用者传入的指针，会出现什么问题？

答案：只保存指针时，栈里存的是**调用者缓冲区的地址**。调用者一旦改写该缓冲区 （例如循环里反复用同一个 `char buf[200]` 读入新行），栈中所有元素会**同时变成新内容**， 栈就失去意义；若缓冲区来自 `malloc`，还可能产生悬空指针（第 16 讲）。 所以课程选择"push 时复制一份进栈内部"，代价是复制开销与容量上限。 这背后的取舍是：**谁负责数据的生命周期**——容器复制（安全但费时间/内存），还是调用者保证（快但易错）。`stack_pop` 同样必须复制到调用者提供的数组里，因为栈内那份随时会被下一次 push 覆盖； 这正是"信息隐藏与性能有时互相冲突"的具体体现。

Lecture 16: 动态内存分配 (Dynamic Memory Allocation)

概述

本讲要解决的问题是：**当程序在编译时无法知道需要多少内存（用户会输入多少行、图里有多少个结点）时，如何向操作系统申请、使用并归还内存**。引入的机制是**堆 (heap)** 与 `malloc` / `calloc` / `realloc` / `free` 这一组函数，以及它们背后操作系统提供的 `sbrk` / `brk` 接口和**分配器 (allocator)** 的内部数据结构。 这一讲把第 15 讲的结构体与指针提升为"可变大小的对象"，是链表、树、动态数组、字符串处理 以及所有真实程序（包括课程 `mem220` 分配器）的基础，也解释了为什么 `free` 必须依赖**元数据 (metadata)**。

核心概念与底层机制图解

- **静态分配与栈分配为什么不够**：`static` /全局数组的大小编译期固定；栈上局部数组随帧生灭， 大小也必须编译期已知（C99 变长数组也不能超过帧的预算）。
  - *直观解释*：静态数组像"买一套固定大小的房子"，栈数组像"借会议室开会"—— 会开完就收回；真实需求却常是"人来了再定房间"。
  - *底层机制图解*：程序的内存分为代码段、全局数据段、**堆**与**栈**； 堆的末端地址叫 **break**， `sbrk` 可以移动它： `高地址 ┌──────────────────────┐ │ 系统空间（内核） │ ├──────────────────────┤ │ 栈 (stack) │ ← 向低地址增长（局部变量、返回地址、帧） │ ↓ │ ├──────────────────────┤ │ ↑ │ │ 堆 (heap) │ ← 向高地址增长（malloc / mem220） ├──────────────────────┤ ← break（sbrk 改变这里） │ 全局数据段 (.bss/.data)│ ├──────────────────────┤ │ 代码段 (text) │ 低地址 └──────────────────────┘` 两段增长方向相反，中间的空隙就是它们共同的可用空间；堆撞上栈时进程就会崩溃。
  - *作用域与存储期*：堆对象的存储期由**程序显式控制**——从 `malloc` 返回开始， 到 `free` 调用为止，**与任何函数的作用域无关**。这是它与栈对象最根本的区别， 也是"函数返回后指针仍然有效"（以及忘记 `free` 就永久泄漏）的原因。
- **`sbrk` 与 `intptr_t`**：Linux 用 `void *sbrk (intptr_t increment);` 调整 break。
  - *直观解释*：`sbrk` 像问物业"再给我扩 N 平方米"，物业把旧边界的位置告诉你，你就能算出新场地在哪。
  - *底层机制图解*：调用 `sbrk(n)` 请求把 break 移动 `n` 字节，**返回旧的 break** （失败时返回 `(void*)-1`）；参数为负则缩小堆。参数类型是 `intptr_t` —— "足够装下一个指针的**有符号**整数"。为什么不允许用 `int`？ 因为 64 位地址空间下 `int` 只有 32 位，装不下指针，也无法表示"负数增量"与地址的差值。 本讲示例里 `sbrk ((intptr_t)HEAP_BYTES)` 一次拿到 `0x831000..0x835000` 共 16384 字节。
  - *作用域与存储期*：`sbrk` 改变的是**进程**的内存映射，与函数作用域无关； 它申请的内存需要在程序结束前自己管理（我们的分配器从不把它还给 OS）。
- **四个函数的契约**：
  - *直观解释*：`malloc` 是"给我一块地"， `calloc` 是"给我一块**除过草**的地"， `realloc` 是"我想把地扩大/缩小"， `free` 是"这块地我不要了"。

- *底层机制图解*：

调用	契约（必须逐条记住）
`void* malloc (size_t size)`	返回至少 `size` 字节的**未初始化**内存块，失败返回 `NULL`
`void* calloc (size_t n, size_t size)`	申请 `n * size` 字节并**全部置 0**，失败返回 `NULL`
`void* realloc (void* p, size_t size)`	改变已分配块的大小；**可能搬到新地址**（此时旧内容被复制、旧块被释放）；失败返回 `NULL` 且**旧块保持不变**
`void free (void* p)`	归还 `p` 指向的块；`p` 必须是 `malloc` / `calloc` / `realloc` 返回的**块首地址**；`free(NULL)` 是合法的空操作

  `size_t` 是无符号整数类型（本机 64 位），"字节数"永远用它而不用 `int`； 注意无符号的后果：`malloc (n * sizeof (int))` 中若 `n` 很大，乘法会**回绕**成一个小数， 于是申请到一块太小的内存，随后越界写入。 `malloc` 返回 `void*`：它可以被自动转换成任何指针类型（C 的隐式转换）， 但**既不能解引用也不能做指针算术**——必须先赋给具体类型的指针。 * *作用域与存储期*：这些对象都是 dynamic storage duration； `free` 之后指针值仍然存在，但它指向的存储期已经结束，**继续使用是未定义行为 (UB)**。

- **经典错误家族**：全部与存储期有关。
  - *直观解释*：把内存想成停车位——忘了开走叫"泄漏"，车位被收回后还继续用叫"悬空指针"， 再退一次叫"重复释放"，停到线外叫"越界"，拿别人的车牌退租叫"释放非堆指针"， 把唯一的租约弄丢叫"丢失指针"。
  - *底层机制图解*：每种错误的机器层面后果如下：

错误	机理	典型后果
内存泄漏 (leak)	忘记 `free`，块永远占用堆	内存持续增长，最终 `malloc` 返回 `NULL`
悬空指针 (dangling pointer)	`free` 后指针值未变但存储期已结束	读到别人的数据或已回收的元数据
释放后使用 (use-after-free)	同上，且**写**会破坏分配器元数据	程序"看起来正常"却悄悄损坏堆
重复释放 (double free)	同一块被插进自由链表两次	两个申请者拿到同一块内存
越界写 (buffer overflow)	写入 `p[n]` 之外	覆盖相邻块的头部（大小与链表指针）
释放非堆指针	`free` 栈地址/全局地址/块中间地址	分配器 abort 或堆结构被破坏
丢失唯一指针	指针被覆盖或离开作用域	等价于泄漏，且**再也无法回收**

- *作用域与存储期*：判断一段代码是否安全只需两个问题： ① 该指针指向对象的**存储期**何时结束？② 结束时还有没有别的指针引用它？

- **动态数组的增长策略与摊销分析 (amortized analysis)**：
  - *直观解释*：容量翻倍像"每次搬家都换一间大一倍的房子"——搬得次数少，虽然每次搬的东西多。
  - *底层机制图解*：容量**翻倍**时，第 k 次扩容复制 `2^(k-1)` 个元素， 总复制量 `1 + 2 + … + n/2 < n`，再加最后一次至多 `n`，合计 `< 2n`， 于是"每次追加的平均复制成本"是常数，即**摊销 O(1)**；若每次只加固定量（如 +4）， 总复制量 `4 + 8 + … + n ≈ n²/8`，即**总时间 O(n²)**——实测（示例 1）加倍策略在 `n = 1000000` 时共复制 1048572 个元素（`copies/n = 1.05`），而固定 +4 策略 在 `n = 4000` 时就已复制 1998000 个（`copies/n = 499.5`）。代价是空间： 平均浪费约 38%（课程推导 `2(ln 2 − 1/2)`），且 `realloc` 可能整块搬家导致指针失效 （**扩容后必须使用 `realloc` 的返回值**）。
  - *作用域与存储期*：数组在堆上（dynamic），描述它的 `count` / `capacity` 通常在栈上， 两者必须一起更新，否则出现"容量说 100、实际只有 10"的不一致状态。

- **内部碎片与外部碎片 (internal / external fragmentation)**：
  - *直观解释*：内部碎片是"租了一间 100 平米的房子只用了 60 平米"； 外部碎片是"空房间加起来有 200 平米，但没有一间能装下你要的 150 平米"。
  - *底层机制图解*：**内部碎片**来自分配粒度： `mem220` 按 2 的幂分配， 请求 1000 字节（加 16 字节头 = 1016）会拿到 1024 字节的块，浪费 8 字节； 实测反复申请 1000 字节时相邻块地址相差**恰好 1024 字节**。 **外部碎片**来自空洞：反复分配/释放不同大小的块会留下许多小空洞， 即使总空闲量足够，也可能没有一块连续区域满足大请求。 分配器的对策是**分裂 (splitting)** 与**合并 (coalescing)**（见示例 3）， 以及把块大小**分箱 (binning)** 成 2 的幂——这正是 `mem220` 的"最佳适配对数分配器"。
  - *作用域与存储期*：碎片是**堆的全局性质**，与任何单个变量无关； 它说明"程序的内存行为"不能只看单个对象，还要看整个生命周期的分配模式。

- **分配器如何工作：自由链表、首次适配、分裂与合并**：
  - *直观解释*：自由链表是"空房间登记表"；首次适配是"从表头开始找第一间够大的房间"； 最佳适配是"找最接近需求的那一间"；分裂与合并是"把大房间隔成两间"与"把相邻空房间打通"。
  - *底层机制图解*：每个块前面有**头部 (header)** 记录大小与空闲标志，空闲块用头部的 `next` 串成链表。分配时找到合适的块，必要时**分裂**成"用掉的 + 剩下的"；释放时插回链表并与 **物理相邻**的空闲块**合并**，否则堆会碎成一地小块。两种经典策略： **首次适配 (first fit)** 取第一块够大的（快，但前部易留小碎片）； **最佳适配 (best fit)** 取最接近需求的（省空间，但通常要遍历整个链表）。 `mem220` 用第三种：**对数最佳适配 (best-fit logarithmic)**——把大小量化成 2 的幂、 每种大小一条链表，"最佳适配"就变成查 `ceil(log2(size))` 那张表，O(1)。
  - *作用域与存储期*：链表里的块都来自 `sbrk` 拿到的整片堆区，存储期贯穿整个进程； 分配器自己不把它们还给 OS（`mem220` 只在初始化时 `malloc` 一次大块）。

- **`malloc(0)`、`free` 为什么需要元数据**：
  - *直观解释*：`free(p)` 只收到一个地址，却必须知道这块有多大、属于哪张表—— 就像退房时只看房卡就知道房号、面积与租约，靠的是"卡片旁边的登记信息"。

- *底层机制图解*：大小必须**存在块旁边**：`mem220` 把大小放在**返回指针之前**的 `mem_block_t.size` 里，`free` 用 `mem_block[-1].size` 读回 （`ptr[-1]` 等价于 `*(ptr - 1)`，指针算术按 `sizeof (mem_block_t)` 缩放）。 示例分配器把头部放在 `(uint8_t*)ptr - sizeof (block_t)` 处，并用 `free` 标志 + 地址范围做检查。 `malloc(0)` 的返回值是实现定义的：可能返回 `NULL`，也可能返回一个"不能解引用但可以 `free`"的唯一指针； `mem220` 明确选择返回 `NULL` （其注释写明 0 字节请求返回 `NULL`），实测 `mem220_allocate (0) = (nil)`。 所以**不要把 `malloc(0) == NULL` 当成"内存不足"的信号**。
- *作用域与存储期*：元数据与块同生共死； `free` 之后头部属于分配器， 此时读写 `p[-1]` 就是对分配器内部结构的破坏。

代码示例与底层机制分析

示例 1：动态数组的倍增长与摊销代价

代码 (C):

```c
/* l4_grow.c
 * 编译:  gcc -g -std=c99 -Wall -Werror l4_grow.c -o l4_grow */
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

static int64_t copied = 0;      /* elements moved by realloc */
static int64_t grows = 0;

static int32_t*
append (int32_t* arr, int32_t* n, int32_t* cap, int32_t value, int32_t doubling)
{
    if (*n == *cap) {
        int32_t  new_cap = doubling ? (2 * *cap) : (*cap + 4);
        int32_t* bigger = realloc (arr, (size_t)new_cap * sizeof (*arr));

        if (NULL == bigger) {
            exit (1);                   /* arr is still valid here */
        }
        copied += *n;                   /* realloc copies the live elements */
        grows++;
        arr = bigger;
        *cap = new_cap;
    }
    arr[*n] = value;
    (*n)++;
    return arr;
}

static void
run (int32_t n, int32_t doubling, const char* label)
{
    int32_t* arr = malloc (4 * sizeof (*arr));
    int32_t  count = 0, capacity = 4, i;

    copied = 0;
    grows = 0;
    for (i = 0; n > i; i++) {
        arr = append (arr, &count, &capacity, i, doubling);
    }
    printf ("%-10s n=%7d: grows=%5ld copies=%9ld copies/n=%7.2f\n",
            label, (int)n, (long)grows, (long)copied,
            (double)copied / (double)n);
    free (arr);
}

int
main (void)
{
    printf ("--- doubling the capacity (amortized O(1) per append) ---\n");
    run (1000, 1, "double");
    run (10000, 1, "double");
    run (1000000, 1, "double");
```

```
    printf ("\n--- adding a fixed increment of 4 (quadratic total work) ---\n");
    run (1000, 0, "fixed +4");
    run (2000, 0, "fixed +4");
    run (4000, 0, "fixed +4");
    return 0;
}
```

真实运行输出:

```
--- doubling the capacity (amortized O(1) per append) ---
double     n=   1000: grows=    8 copies=     1020 copies/n=   1.02
double     n=  10000: grows=   12 copies=    16380 copies/n=   1.64
double     n=1000000: grows=   18 copies=  1048572 copies/n=   1.05

--- adding a fixed increment of 4 (quadratic total work) ---
fixed +4   n=   1000: grows=  249 copies=   124500 copies/n= 124.50
fixed +4   n=   2000: grows=  499 copies=   499000 copies/n= 249.50
fixed +4   n=   4000: grows=  999 copies=  1998000 copies/n= 499.50
```

【代码做什么？】 1. `run` 先 `malloc` 一个容量为 4 的数组，然后反复调用 `append` 追加元素。 2. `append` 在 `count == capacity` 时扩容：加倍策略新容量为 `2 * cap`，固定策略为 `cap + 4`。 3. 每次扩容用 `realloc`，把被搬动的元素数累计进 `copied`；失败则退出（此时旧块仍有效）。 4. 三组加倍实验的 `copies/n` 稳定在 1.02—1.64（**与 n 无关**，这正是摊销 O(1) 的含义）。 5. 三组固定增量实验的 `copies/n` 随 n **线性增长**（124 → 249 → 499），总复制量 $O(n^2)$。

【底层机制透视】 `realloc` **可能把整块内存搬到新地址**（本机实验中确实发生了），所以必须用返回值更新 `arr`， 并把新地址传回调用者（`append` 返回 `arr`）。写成 `realloc (arr, new_cap)` 而忽略返回值， 就会同时造成"指针可能失效"与"失败时丢地址（泄漏）"两个问题。 另外 `copies/n` 在 `n = 10000` 时是 1.64 而不是 1.0x：容量从 4 开始翻倍， 总复制量是前面所有容量之和（`4+8+…+8192 = 16380`），相对当前 n 会有波动， 但永远小于 2n——**摊销上界与某一时刻的具体值不是一回事**。 加倍策略时间上最优，空间上却平均浪费约 38%（课程推导）， 且一次大搬家会造成短暂的"两份内存同时存在"的峰值需求。

【内存布局图解】

```
堆上的数组随扩容在地址上"跳":
容量 4:    [0][1][2][3]                          ← malloc(16)
append 第 5 个元素时 → realloc 到容量 8:
           旧块被释放（内容复制）                  新块可能是完全不同的地址
容量 8:    [0][1][2][3][4][5][6][7]              ← 32 字节
...
容量 2^k: 复制了 2^(k-1) 个已有元素
总复制量 = 4 + 8 + ... + 2^(k-1) < 2^k ≤ 2n    →  摊销 O(1)

栈:  arr(指针) / count / capacity / i / n      ← 每次调用 append 都会复制这 4 个整数
```

【与汇编的对应】

```
; append 的核心：比较 count 与 capacity，不等就直接写入，相等就扩容
; 注意：R5 是帧指针、R6 是栈指针、R4 是全局数据指针，临时值只用 R0–R3
APPEND  LDR R0, R1, #0          ; R0 = count (R1 = &count, R3 = value)
        LDR R2, R2, #0          ; R2 = capacity (R2 原本指向 capacity)
        NOT R2, R2
        ADD R2, R2, #1
        ADD R2, R0, R2          ; R2 = count − capacity
        BRnp STORE              ; 不相等 -> 直接存
        LDR R2, R0, #0          ; （示意）重新取 capacity
        ADD R2, R2, R2          ; 2 * capacity (乘 2 就是左移一位)
        ADD R6, R6, #-1
        STR R2, R6, #0          ; 压入新容量作为参数 (-> 被调用者的 R5+4)
        JSR REALLOC             ; 返回值放在参数正下方 (被调用者的 R5+3)
        LDR R1, R6, #0          ; R1 = 新块地址 (失败时为 0)
        ADD R6, R6, #2          ; 一条指令弹出参数与返回值槽
        ; ... R1 == 0 时保留旧指针并报错，否则 arr = R1 ...
STORE   ADD R0, R0, #1          ; (*n)++
        STR R0, R1, #0
        RET
```

`2 * capacity` 在汇编里就是一条 `ADD R2, R2, R2`（左移一位等于乘 2，用到的是临时寄存器 `R2`， 不是帧指针 `R5`）；`realloc` 是一次子程序调用—— "搬家"发生在库函数内部（可能 `malloc` 新块 + `memcpy` + `free` 旧块）。

示例 2：一个真正的分配器——自由链表 + 首次适配 + 分裂 + 合并

代码 (C)（完整的可编译程序）：

```
/*
 * l6c_alloc.c -- a complete, compact first-fit free-list allocator on sbrk().
 * Compile: gcc -g -std=c99 -Wall -Werror l6c_alloc.c -o l6c_alloc
 */

#define _DEFAULT_SOURCE 1                    /* sbrk() under -std=c99 */
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>
#include <unistd.h>

#define HEAP_BYTES  (16 * 1024)
#define ALIGN       16

/* The header lives immediately BEFORE the bytes handed to the caller.
   Its size is deliberately 32 bytes (a multiple of ALIGN) so that the
   payload always starts on a 16-byte boundary.                          */
typedef struct block {
    size_t        size;          /* usable payload bytes              */
    struct block* next;          /* link of the free list             */
    size_t        free;          /* 1 = free, 0 = in use              */
    size_t        padding;       /* keeps sizeof (block_t) == 32      */
} block_t;

static uint8_t* heap;
static size_t   heap_bytes;
static block_t* free_list;       /* kept sorted by address */

static block_t*
following (block_t* b)
{
    return (block_t*)((uint8_t*)b + sizeof (block_t) + b->size);
}

static void
list_remove (block_t* b)
{
    block_t** link = &free_list;

    while (NULL != *link) {
        if (*link == b) {
            *link = b->next;
            b->next = NULL;
            return;
        }
        link = &(*link)->next;
    }
}

static void
list_insert (block_t* b)         /* insert, keeping the list sorted by address */
{
```

```
    block_t** link = &free_list;

    while (NULL != *link && *link < b) {
        link = &(*link)->next;
    }
    b->next = *link;
    *link = b;
}

static void*
my_malloc (size_t n_bytes)
{
    block_t* b;
    size_t   need = (n_bytes + (ALIGN - 1)) & ~((size_t)ALIGN - 1);   /* align up */

    if (0 == n_bytes) {
        return NULL;
    }
    for (b = free_list; NULL != b; b = b->next) {          /* first fit */
        if (need > b->size) {
            continue;
        }
        if (b->size >= need + sizeof (block_t) + ALIGN) {  /* split */
            block_t* rest = (block_t*)((uint8_t*)b + sizeof (block_t) + need);

            rest->size = b->size - need - sizeof (block_t);
            rest->free = 1;
            b->size = need;
            list_remove (b);
            list_insert (rest);
        } else {
            list_remove (b);
        }
        b->free = 0;
        return (uint8_t*)b + sizeof (block_t);
    }
    return NULL;
}

static void
my_free (void* ptr)
{
    block_t* b;
    block_t* next;

    if (NULL == ptr) {
        return;
    }
    b = (block_t*)((uint8_t*)ptr - sizeof (block_t));
    if (b < (block_t*)heap || b >= (block_t*)(heap + heap_bytes) || 0 != b->free) {
        fprintf (stderr, "my_free: %p is not a live block (ignored)\n", ptr);
        return;
    }
    b->free = 1;
```

```
    list_insert (b);

    next = following (b);                    /* coalesce with every following
                                                free block, not just one     */
    while ((uint8_t*)next < heap + heap_bytes && 0 != next->free) {
        list_remove (next);
        b->size += sizeof (block_t) + next->size;
        next = following (b);
    }
}

static void
dump (void)
{
    uint8_t* p = heap;

    while (p < heap + heap_bytes) {
        const block_t* b = (const block_t*)p;

        printf ("  [offset %6ld] size %6ld %s\n", (long)(p - heap),
                (long)b->size, b->free ? "FREE" : "in use");
        p += sizeof (block_t) + b->size;
    }
}

int
main (void)
{
    void*    base = sbrk ((intptr_t)HEAP_BYTES);
    uint8_t *a, *b, *c, *d;
    int32_t  local = 7;

    if ((void*)-1 == base) {
        perror ("sbrk");
        return 1;
    }
    heap = base;
    heap_bytes = (size_t)HEAP_BYTES;

    free_list = (block_t*)heap;                /* one big free block */
    free_list->size = heap_bytes - sizeof (block_t);
    free_list->next = NULL;
    free_list->free = 1;
    printf ("heap %p..%p (%ld bytes); first usable byte %p\n", (void*)heap,
            (void*)(heap + heap_bytes), (long)heap_bytes,
            (void*)(heap + sizeof (block_t)));

    a = my_malloc (100); b = my_malloc (100); c = my_malloc (100);
    printf ("a=%p b=%p c=%p ; b-a = %ld bytes = 112 payload + 32 header\n",
            (void*)a, (void*)b, (void*)c, (long)(b - a));

    my_free (b);
    d = my_malloc (64);
    printf ("a 64-byte request returned %p (%s b)\n", (void*)d,
```

```
            (d == b) ? "the same address as" : "a different address from");
    printf ("after splitting the hole:\n");
    dump ();

    my_free (c);  my_free (d);  my_free (a);   /* descending address order */
    printf ("after freeing c, d, a:\n");
    dump ();

    printf ("my_malloc (0) = %p\n", my_malloc (0));
    printf ("freeing the stack address %p:\n", (void*)&local);
    my_free (&local);
    dump ();
    return 0;
}
```

真实运行输出:

```
heap 0x831000..0x835000 (16384 bytes); first usable byte 0x831020
a=0x831020 b=0x8310b0 c=0x831140 ; b-a = 144 bytes = 112 payload + 32 header
a 64-byte request returned 0x8310b0 (the same address as b)
after splitting the hole:
  [offset      0] size    112 in use
  [offset    144] size     64 in use
  [offset    240] size     16 FREE
  [offset    288] size    112 in use
  [offset    432] size  15920 FREE
after freeing c, d, a:
  [offset      0] size  16352 FREE
my_malloc (0) = (nil)
freeing the stack address 0x7fff0a13cad4:
  [offset      0] size  16352 FREE
 (stderr: my_free: 0x7fff0a13cad4 is not a live block (ignored))
```

【代码做什么？】 1. `main` 用 `sbrk((intptr_t)16384)` 向操作系统要一整片堆，并把它做成**一个巨大的空闲块**（16352 字节有效载荷）。 2. `my_malloc(100)` 把请求向上取整到 16 的倍数（112），在自由链表里做**首次适配**： 大块够大，于是**分裂**成"用掉的 112 字节 + 剩下的空闲块"。 3. 三次 100 字节请求后，块以 **144** 字节（112 数据 + 32 头部）为步长依次排列。 4. `my_free(b)` 把 b 标记为空闲并插回链表（按地址有序）；随后 `my_malloc(64)` **复用了 b 的地址** （`0x8310b0`），并再次分裂出 16 字节的空闲尾巴（64 + 32 头部 = 96，112 − 96 = 16）。 5. 按地址从高到低依次 `my_free(c); my_free(d); my_free(a);` 时，**合并循环**把相邻空闲块逐个吞并， 最终重新拼成单个 16352 字节的空闲块——堆完全回到初始状态。 6. `my_malloc(0)` 返回 `NULL`；对栈地址调用 `my_free` 被"范围 + 已释放标志"检查拦下，堆保持完好。

【底层机制透视】 `b - a = 144` 把三件事一起说清了：**头部开销**（32 字节）、**对齐粒度**（16 字节）、 以及**请求会被向上取整**（100 → 112）。这 12 字节的差额就是**内部碎片**。 头部之所以刻意做成 32 字节（`size` / `next` / `free` 之后还留一个 `padding`）， 就是为了让"头部 + 16 的倍数"仍然落在 16 字节边界上——**对齐要求会反过来影响分配器的设计**。 **分裂**让大块满足小请求而不浪费太多；**合并**让相邻空闲块重新变大，否则堆会永久碎裂。 本例只做**向前合并**（`following (b)` 的地址 = `b + 32 + size`，看它是否也空闲，并循环直到遇到使用中的块）， 因此 `my_free` 需要按**从高到低**的地址顺序释放才能一次合并干净； 如果

按从低到高释放，会留下多个相邻但未合并的空闲块——这正是"分裂容易、合并难"的根源。 真实分配器（如glibc）在块尾也放一个**边界标记 (boundary tag)**，这样无需遍历链表就能找到前一个块并做双向合并。 最后，`my_free` 的"范围 + 已释放标志"检查说明：**分配器必须对错误输入有防御**， 否则一次 `free` 栈地址就会破坏整条链表；真实 `glibc` 会打印 `free(): invalid pointer` 并 `abort`。

【内存布局图解】

```
sbrk 拿到的 16 KiB 堆，每个块 = 32 字节头部 + 对齐到 16 的数据

偏移 0      32       144      176      288      320      432
┌──────────┬─────────┬─────────┬─────────┬─────────┬─────────┬──────────────────┐
│ header   │ a: 112  │ header  │ b: 112  │ header  │ c: 112  │ 空闲 15920       │
│          │ in use  │         │ in use  │         │ in use  │ （整片剩余）     │
└──────────┴─────────┴─────────┴─────────┴─────────┴─────────┴──────────────────┘
 释放 b 并再申请 64 字节后（复用 + 分裂出 16 字节尾巴）：
┌──────────┬─────────┬─────────┬─────┬─────────┬──────────────────────────────┐
│ a: 112   │ d: 64   │ 空洞 16 │ c:112 │ 空闲 15920                       │
└──────────┴─────────┴─────────┴─────┴─────────┴──────────────────────────────┘
 依次 free(c)、free(d)、free(a) → 合并成单个 16352 字节的空闲块（偏移 0 起）

free(ptr) 如何找到头部？  ptr − 32 字节 ──► header.size（这就是"元数据"）
返回给调用者的是 (uint8_t*)b + sizeof (block_t)，即"跳过一个头部"之后的地址
```

【与汇编的对应】

```
; my_malloc 的骨架：遍历自由链表 + 首次适配
MYMALLOC
        LD   R1, FREE_LIST      ; R1 = 链表头（文件作用域变量，用 PC 相对的 LD 取）
FITLOOP ADD  R1, R1, #0
        BRz  NOMEM              ; 链表空了 -> 返回 NULL
        LDR  R2, R1, #0         ; R2 = b->size
        NOT  R3, R0
        ADD  R3, R3, #1
        ADD  R3, R2, R3         ; R3 = b->size - need
        BRn  NEXTBLK            ; 不够大 -> 看下一个
        LDR  R3, R1, #1         ; R3 = b->next（R3 比较结果已用完，可复用）
        ST   R3, FREE_LIST      ; 从链表摘下来（文件作用域变量用 ST）
        ; ... 够大时分裂成两块，并把后半块插回自由链表 ...
        ADD  R0, R1, #2         ; R0 = 返回给调用者的地址（跳过头部）
        RET
NEXTBLK LDR  R1, R1, #1
        BRnzp FITLOOP
NOMEM   AND  R0, R0, #0
        RET
```

`free` 侧的关键只有两步：**从 `ptr` 往下取头部读出大小**（负偏移访问）， 以及**把块插回链表并检查地址是否相邻**——相邻就是 `prev + sizeof (header) + prev->size == b`。

常见错误与调试技巧

- **内存泄漏**：忘记 `free`，或中途把唯一的指针覆盖。实测 `valgrind --leak-check=full ./d4r_plain leak` 给出 `64 bytes in 1 blocks are definitely lost ... by 0x4011CB: get_block`。 **调试**：`valgrind --leak-check=full --show-leak-kinds=all ./prog`； ASan（`gcc -fsanitize=address -g`）在本机因 `ulimit -v = 32 GB` 无法预留影子内存 （启动即报 `ReserveShadowMemoryRange failed`），此时改用 valgrind。
- **释放后使用 / 悬空指针**：`free(p)` 之后 `printf("%d", p[0])`。**普通运行可能"看起来正常"**： 实测该程序输出 `uaf: after free, [0] reads as 1711` 并继续运行（值是垃圾，每次可能不同）， 而 valgrind 立刻报 `Invalid read of size 4 ... Address 0x4a6f040 is 0 bytes inside a block of size 16 free'd`。 **调试**：`valgrind --track-origins=yes ./prog`；把指针 `free` 后置为 `NULL` 是好习惯。
- **重复释放**：实测普通运行时 `glibc` 报 `free(): double free detected in tcache 2` 并 `abort`（退出码 134）， 而 valgrind 报 `Invalid free()` 后继续给出报告——**同一份代码在两种环境下表现不同**，这正是 UB 的特征。 **调试**：`valgrind ./prog`； `MALLOC_CHECK_=3 ./prog` 让 glibc 做额外校验。
- **越界写**：`p[4] = 1234`（只分配了 4 个 `int32_t`）。实测普通运行时**毫无提示**、输出仍是 `0 1 2 3`；valgrind 报 `Invalid write of size 4 ... 0 bytes after a block of size 16 alloc'd`。 **调试**：`valgrind ./prog`； `gdb` 里 `x/8xw p` 观察块外内容是否被改写。
- **`realloc` 直接赋回原指针**：`p = realloc (p, n);` 一旦失败，旧块地址丢失、永远泄漏。 正确写法是先用临时变量接住返回值（`l5_realloc.c` 实测：请求 64 TiB 时返回 `NULL` 且 `errno = 12 (ENOMEM)`，旧块内容 `p[0]=100 p[9]=109` 完好；随后一次真正的扩容成功并搬家）。 **调试**：`valgrind --leak-check=full` 会指出失败路径上泄漏的块对应的源码行。
- **释放非堆指针**：对栈地址、全局地址或块中间地址调用 `free`。 实测 glibc 报 `free(): invalid pointer` 并 `abort`；valgrind 报 `Address 0x1ffeffd0dc is on thread 1's stack`。**调试**：`valgrind ./prog`； `gdb` 下 `p ptr` 与 `bt` 对照，确认它是否来自 `malloc` 的返回值。

关键要点

- 堆对象具有 dynamic storage duration：生命周期由 `malloc` / `free` 决定，与函数作用域无关—— 这既让函数可以返回新对象，也让忘记 `free` 变成永久泄漏。
- 每个 `malloc` 都必须检查 `NULL`，每个 `realloc` 都必须用**临时变量**接住返回值， 每个 `free` 都必须恰好一次、且只能传回 `malloc` 家族返回的**块首地址**。
- 动态数组用**容量翻倍**获得摊销 O(1) 的追加成本（总复制量 < 2n），固定增量则是 $O(n^2)$； 代价是约 38% 的平均空间浪费与 `realloc` 搬家的可能。
- 分配器的核心是**元数据 + 自由链表**：头部记录大小，分配时首次/最佳适配并**分裂**， 释放时插回链表并**合并**； `free` 无法知道大小，所以大小必须存在块旁边。
- 内存错误（泄漏、悬空、重复释放、越界）在普通运行中**往往不可见**， 必须用 `valgrind` /ASan 这类工具才能可靠发现——"跑起来没崩"绝不等于"内存管理正确"。

思考题（带答案）

问题 1：为什么 `free (ptr)` 只需要一个指针，就能把块归还给正确的链表？ 如果调用者传入的是 `ptr + 1`（块中间的地址），会发生什么？

答案：因为分配器把**元数据放在返回指针之前**。`mem220` 的 `mem_block_t` 存了块的字节数， `free` 用 `mem_block[-1].size` 读回它，再用 `log2_ceil(size)` 算出 bin 号并插进对应链表； 示例分配器同理：`b = (block_t*)((uint8_t*)ptr - sizeof (block_t))`。 传入 `ptr + 1` 时 `b` 落在块**内部**而非头部，读出的"大小"是用户数据的前几个字节， 算出的 bin 号毫无意义，插回链表时还会把用户数据当成链表指针——轻则下次 `malloc` 返回重叠内存，重则立即崩溃。示例分配器靠"地址范围 + `free` 标志"拦下这种错误 （实测对栈地址调用 `my_free` 被拒绝并打印警告，堆保持完好），真实分配器则直接 `abort`。

问题 2：`mem220` 为什么按 2 的幂分配块？这样做的收益与代价各是什么？

答案：收益有两个。① **最佳适配变成 O(1)**：块大小只有 2^k 种，于是可以用数组 `mem_bin[k]` 存"大小为 2^k 的空闲块链表"，请求 `n` 字节时直接算 `bin = ceil(log2(n + sizeof(header)))` 查表，无需遍历所有空闲块。 ② **对齐自动满足**：块都是 2 的幂且最小 32 字节，起始地址天然满足 `malloc` 的对齐要求。 代价是**内部碎片**：请求 1000 字节拿到 1024 字节的块（实测相邻地址相差 1024）， 请求 100 字节拿到 128 字节的块，请求 513 字节也要 1024 字节（浪费近一半），最坏浪费接近 50%； 而且它**不分裂也不合并**，被释放的 1024 字节块无法满足 2048 字节的请求—— "总空闲够"却可能分配失败。

问题 3：下面两段代码都想把数组增长到 `2n`，第二段为什么是危险的？请给出正确写法。

```
/* A */  arr = realloc (arr, 2 * n * sizeof (int32_t));

/* B */  int32_t* bigger = realloc (arr, 2 * n * sizeof (int32_t));
         if (NULL == bigger) { /* 处理失败 */ } else { arr = bigger; }
```

答案：代码 A 把 `realloc` 的返回值**直接写回唯一的指针**：一旦失败返回 `NULL`， `arr` 立刻变成 `NULL`，原来那块内存的地址**永久丢失**——既不能使用也不能 `free`， 这就是经典的"失败路径泄漏"。代码 B 用临时变量接住返回值，失败时 `arr` 仍指向旧块 （内容完好，可继续使用或正常 `free`），成功时才更新。实测证据：`l5_realloc.c` 请求 64 TiB 时 `realloc` 返回 `NULL`、`errno = 12 (ENOMEM)`，而旧块的 `p[0] = 100`、`p[9] = 109` 仍可读， 随后一次正常扩容成功并把数据复制到新地址。另外 `2 * n * sizeof (int32_t)` 本身也可能溢出 `size_t`， 生产代码通常要检查 `n > SIZE_MAX / (2 * sizeof (int32_t))`。

Lecture 17: 链表 (Linked Lists)

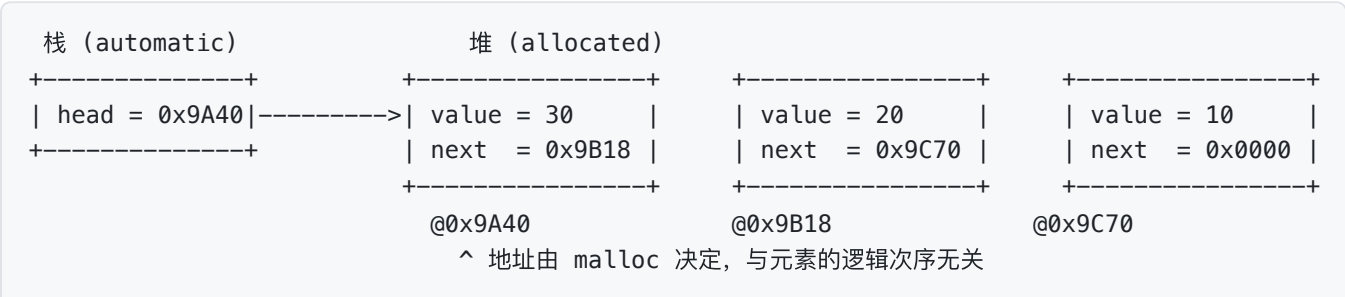
概述

本讲解决的核心问题是：当元素个数在编译期未知、且需要在中间频繁插入与删除时，数组不再是合适的容器。 我们引入**自引用结构 (self-referential structure)**、**头指针 (head pointer)** 与指针链，把散布在堆 (heap) 上的结点串成**单链表 (singly-linked list)**，并给出建表、遍历、查找、有序插入、删除、销毁的标准惯用法。 链表是内核就绪队列、编译器符号表与抽象语法树 (AST) 子结点链的共同基础，也是下一讲树结构（每个结点带两个链接指针）的直接前驱。

核心概念与底层机制图解

- **数组作为容器的三个结构性限制 (limits of arrays)**：数组只能表达"定长、连续、按位序访问"的数据，像一排焊死的储物柜：数量固定，想在中间塞进一个，得把后面所有柜子整体搬开。
  - *底层机制图解*：`int32_t a[1000]` 在编译期决定 `4000` 字节连续空间；中间插入/删除都要整体搬移元素（`O(n)` 次内存写）；动态数组扩容 (dynamic resizing) 用 `realloc` 申请更大块并**整体拷贝**，按 2 倍增长时累计拷贝 ≤ `2N`，平均浪费约 38% 空间。
  - *作用域与存储期*：`int32_t a[1000];` 是 automatic storage duration，随栈帧销毁；`malloc` 出来的动态数组是 allocated storage duration，必须显式 `free`。
- **自引用结构 (self-referential structure)**：结构体内含一个指向**同类型**的指针成员，从而表达"任意多个自己"。
  - *直观解释*：每个结点像一张卡片，卡片上写着"下一张卡片放在哪个房间"——房间不挨着也没关系，顺着地址能走完全部卡片。
  - *底层机制图解*：`struct node_t { int32_t value; struct node_t* next; };` 中 `value` 占 4 字节，`next` 是指针（64 位平台上 8 字节），含填充共 16 字节。**必须用指针而不是内嵌 `struct node_t next;`**：内嵌会让类型大小无限递归，编译器直接报 `field 'next' has incomplete type`。
  - *作用域与存储期*：类型名 `struct node_t` 在文件作用域可见；每个 `malloc` 出来的结点是 allocated storage duration，生命周期从 `malloc` 到 `free`，与任何栈帧无关——这正是链表能跨函数存活的原因。
- **头指针与 NULL 终止符 (head pointer and NULL terminator)**：整个链表由一个指针变量代表。
  - *直观解释*：头指针是"第一张卡片所在房间的号码"；最后一个结点的 `next` 写 `NULL`，等于卡片上写"到此为止"。
  - *底层机制图解*：LC-3 中 `NULL` 就是 `x0000`，而代码从 `x3000` 装载、全局数据区在 `x4000` 以上，`0` 永远不是合法结点地址——这就是 `NULL` 能当哨兵的原因。
  - *作用域与存储期*：`node_t* head;` 定义在函数内时头指针本身是 automatic（栈上 8 字节），但它指向的结点在堆上；函数返回后头指针消失、结点仍在堆上，这就是**内存泄漏 (memory leak)** 的物理来源。

链表的内存布局图解（结点在堆上散布、由指针串联）：



- **头插法与尾插法 (head insertion / tail insertion)**：决定新结点接到链表的哪一端。

- *直观解释*：头插像"往牌堆顶放牌"，永远 `O(1)`；尾插像"把牌放到牌堆底"，没有尾指针就得从顶翻到底。
- *底层机制图解*：头插两步且**顺序不可颠倒**：`n->next = head; head = n;`。若颠倒则旧链表地址永久丢失（见本讲最后的演示）。尾插的 `O(n)` 版本用**指向指针的指针 (pointer to pointer)** 一次遍历定位到那个值为 `NULL` 的链接字段：`c node_t** find; for (find = &head; NULL != *find; find = &(*find)->next) { } *find = make_node(value); /* 直接改写"最后一个 next 字段" */` 若维护 `node_t* tail`，尾插降为 `O(1)`：`tail->next = n; tail = n;`。
- *作用域与存储期*：`find` 是 automatic 变量（栈上 8 字节），保存的是**堆中某个 `next` 字段的地址**，解引用它就是对堆中链接字段的读或写。

- **遍历惯用法 (traversal idiom)**：`for (p = head; NULL != p; p = p->next)`。
  - *直观解释*：像一只手沿铁链逐节往前摸，摸到"没有下一节"就停。
  - *底层机制图解*：每次迭代两次内存访问——读 `p->value`（偏移 0）与读 `p->next`（偏移 1），对应 LC-3 的 `LDR R2,R0,#0` 与 `LDR R0,R0,#1`。**绝不能写 `p++`**：结点地址不连续，`p++` 只会走到相邻的无意义内存。
  - *作用域与存储期*：迭代变量 `p` 是 automatic；循环体若 `free(p)`，`p->next` 立刻失效（见"删除"与"销毁"）。
- **有序插入、查找与删除 (sorted insertion, search and deletion)**：三者共用"维护待改写的那个链接字段"这一手法。
  - *直观解释*：有序插入像排队时走到第一个比自己高的人前面站定；删除像从链条上摘下一节，必须同时捏住它前面那一节。
  - *底层机制图解*：用 `find = &head` 起步、`find = &(*find)->next` 前进，`find` 始终指向"待改写的链接字段"。于是有序插入是 `n->next = *find; *find = n;`，删除是 `*find = dead->next; free(dead);`，**删头、删中间、删尾三种情况合并为同一句**。这正是课程 `player_delete` 中 `player_t** find` 的写法（`for (find = &player_list; p != *find; find = &(*find)->next)`）。查找则是 `O(n)` 的线性扫描，命中即返回结点地址。
  - *作用域与存储期*：`free(dead)` 后该结点进入"已释放"状态，内容随时可能被下一次 `malloc` 覆盖；任何仍指向它的指针都是**悬垂指针 (dangling pointer)**。
- **销毁整个链表 (destroying a list)**：必须**先保存 `p->next`，再 `free(p)`**。
  - *直观解释*：拆链条时先把"下一节在哪"记在手心，再把当前节扔掉。
  - *底层机制图解*：`free` 会把内存交还分配器，分配器可能立刻写入自己的元数据，因此 `free(p)` 之后读 `p->next` 是**未定义行为 (undefined behaviour, UB)**。正确写法是 `next = p->next; free(p);`，循环结束后务必 `*head = NULL`。
  - *作用域与存储期*：销毁后头指针归零是接口契约的一部分——调用者不能再使用旧头指针。
- **双向链表与哨兵 (doubly-linked list and sentinel)**：每个结点多一个 `prev` 指针，换取 `O(1)` 删除与双向遍历。
  - *直观解释*：每张卡片同时写"上一张"和"下一张"的房间号，于是从任意一张都能前后走；代价是每结点多 8 字节（开销增加 50%），且插入/删除要改写 4 个或 2 个指针。

    - *底层机制图解*：课程推荐的简化写法是**带哨兵的循环双向链表 (cyclic doubly-linked list with a sentinel)**：`c static double_list_t my_list = {&my_list, &my_list}; /* 空表：指向自己 */ void dl_insert (double_list_t* head, double_list_t* elt) { elt->next = head->next; /* 1 */ elt->prev = head; /* 2 */ head->next->prev = elt; /* 3 */ head->next = elt; /* 4 */ } void dl_remove (double_list_t* elt) { /* 已知结点，O(1)，无需前驱 */ elt->prev->next = elt->next; elt->next->prev = elt->prev; }` 没有 `NULL` 检查、没有头/尾特例：空表时哨兵的前后指针都指向自己。
    - *作用域与存储期*：哨兵可以是 `static`（静态存储期，程序全程存在），也可以是 `malloc` 出来的；把 `double_list_t` 放在 "thing" 结构的**首字段**，就能在同一地址上自由转换"结点"与"数据"两种身份（`&my_thing.dl == &my_thing`）。
- **链式栈与链式队列 (stack and queue on a list)**：同一个结点结构，改变增删端就得到两种容器。
    - *直观解释*：栈像弹夹（后进先出，只在顶端操作）；队列像排队买饭（队首出、队尾进）。
    - *底层机制图解*：栈只需一个 `top` 指针，`push` 是头插、`pop` 是删头，均 `O(1)`；队列需要 `head` 与 `tail`，`enqueue` 改 `tail->next` 与 `tail`，`dequeue` 改 `head`，均 `O(1)`。**删掉最后一个元素时必须把 `tail` 也置 `NULL`**，否则它成为悬垂指针。
    - *作用域与存储期*：`stack` / `q_head` / `q_tail` 定义为 `static` 时具有 static storage duration，生命周期等于整个程序，但只在定义它的文件内可见。
- **链表与动态数组的权衡 (linked list vs dynamic array)**：

维度	单链表	动态数组
随机访问第 k 个元素	`O(k)`，必须顺序走	`O(1)`：基址 + `k * sizeof(T)`
头部插入/删除	`O(1)`（改两个指针）	`O(n)`（整体搬移）
中间插入/删除（已知前驱）	`O(1)`	`O(n)`
按值查找	`O(n)`	`O(n)`；有序时可二分 `O(log n)`
每元素内存开销	1 个指针 + 填充（16 字节/结点）	0（容量 > 长度时浪费）
缓存局部性 (cache locality)	差：每结点一次潜在 cache miss	好：预取器能识别连续访问
增长代价	单个 `malloc`，无拷贝	倍增扩容，累计拷贝 ≤ `2N`
适用场景	频繁中间增删、长度剧烈变化、地址需稳定	频繁按下标访问、遍历密集、内存敏感

代码示例与底层机制分析

示例 1：单链表的全套基本操作

代码 (C) — `/tmp/ece220_l17/ll_core.c`（完整文件已编译运行）：

```c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct node_t node_t;

struct node_t {
    int32_t value;
    node_t* next;
};

static node_t* make_node(int32_t value)
{
    node_t* n = malloc(sizeof(*n));

    if (NULL == n) {
        fprintf(stderr, "out of memory\n");
        exit(2);
    }
    n->value = value;
    n->next = NULL;
    return n;
}

static node_t* push_front(node_t* head, int32_t value)      /* O(1) 头插 */
{
    node_t* n = make_node(value);

    n->next = head;         /* 先接旧链表 */
    return n;               /* 再成为新头 */
}

static node_t* append_slow(node_t* head, int32_t value)     /* O(n) 尾插 */
{
    node_t** find;

    for (find = &head; NULL != *find; find = &(*find)->next) {
    }
    *find = make_node(value);
    return head;
}

static node_t* find_value(node_t* head, int32_t value)      /* 查找：O(n) */
{
    node_t* p;

    for (p = head; NULL != p; p = p->next) {
        if (value == p->value) {
            return p;
        }
    }
    return NULL;
}
```

```
static node_t* insert_sorted(node_t* head, int32_t value)   /* 有序插入 */
{
    node_t** find = &head;
    node_t* n;

    while (NULL != *find && value > (*find)->value) {
        find = &(*find)->next;
    }
    n = make_node(value);
    n->next = *find;
    *find = n;
    return head;
}

static node_t* delete_classic(node_t* head, int32_t value)  /* prev 指针：三种情况 */
{
    node_t* p = head;
    node_t* prev = NULL;

    while (NULL != p && value != p->value) {
        prev = p;
        p = p->next;
    }
    if (NULL == p) {
        return head;                    /* 没找到 */
    }
    if (NULL == prev) {
        head = p->next;                 /* 情况 1：删头结点 */
    } else {
        prev->next = p->next;           /* 情况 2/3：删中间或尾结点 */
    }
    free(p);
    return head;
}

static node_t* delete_pp(node_t* head, int32_t value)       /* 指针的指针 */
{
    node_t** find = &head;
    node_t* dead;

    while (NULL != *find && value != (*find)->value) {
        find = &(*find)->next;
    }
    if (NULL == *find) {
        return head;
    }
    dead = *find;
    *find = dead->next;                 /* 三种情况合并为一句 */
    free(dead);
    return head;
}

static void destroy(node_t** head)
```

```
{
    node_t* p;
    node_t* next;

    for (p = *head; NULL != p; p = next) {
        next = p->next;                 /* 必须在 free 之前保存 */
        free(p);
    }
    *head = NULL;
}
```

验证到的真实输出（`gcc -g -std=c99 -Wall -Werror ll_core.c -o ll_core && ./ll_core`）：

```
push_front 10..50 head -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL
append_slow      head -> 10 -> 20 -> 30 -> 40 -> 50 -> NULL
append_fast      head -> 10 -> 20 -> 30 -> 40 -> 50 -> NULL
find 30         -> found
find 35         -> not found
insert_sorted    head -> 10 -> 20 -> 30 -> 40 -> 50 -> NULL
delete 30 (mid)  head -> 10 -> 20 -> 40 -> 50 -> NULL
delete 10 (head) head -> 20 -> 40 -> 50 -> NULL
delete 50 (tail) head -> 20 -> 40 -> NULL
delete 99 (none) head -> 20 -> 40 -> NULL
rebuilt          head -> 15 -> 20 -> 25 -> 35 -> 40 -> NULL
pp delete 15     head -> 20 -> 25 -> 35 -> 40 -> NULL
pp delete 35     head -> 20 -> 25 -> 40 -> NULL
pp delete 25     head -> 20 -> 40 -> NULL
pp delete 25 x2  head -> 20 -> 40 -> NULL
after destroy    head -> NULL
allocated = 23, freed = 23
```

【代码做什么？】 1. `push_front` 依次插入 10、20、30、40、50，每次都放到最前面，因此打印为 50→10，即**逆序**。 2. `append_slow` 用 `find` 走到末尾的 `NULL` 链接后挂上新结点，得到正序 10→50；带尾指针的 `append_fast`（写法与示例 2 的 `enqueue` 相同）借助 `tail` 得到同样结果，但不再每次遍历。 3. `find_value(head, 30)` 命中后返回结点地址，打印 `found`；`find_value(head, 35)` 走完全表返回 `NULL`。 4. `insert_sorted` 以 50、30、20、40、10 的顺序插入，最终仍为升序，说明有序不变量被维持。 5. `delete_classic` 依次删除 30（中间）、10（头）、50（尾），各走一个分支；删除不存在的 99 时链表不变；`delete_pp` 用同一段代码完成同样的三种情况。 6. `destroy` 逐结点释放并把 `head` 置 `NULL`；计数器 23 = 23，说明无泄漏。

【底层机制透视】 `malloc(sizeof(*n))` 在堆上分配 16 字节（`int32_t` 4 字节 + 4 字节填充 + 指针 8 字节），返回地址与上次分配**无任何可预测关系**——这正是必须显式保存 `next` 的物理原因。用 `sizeof(*n)` 而非 `sizeof(node_t)` 是 ECE 220 约定：它随指针类型自动变化。 `(*find)->next` 是一个 `node_t*` 左值，取地址得到"链接字段本身的地址"（类型 `node_t**`）；下一次 `*find` 读取该字段。把链接字段当变量来改写，正是删除三种情况能坍缩成一句的原因。 计数器 `n_alloc` / `n_free` 是 `static int32_t`，位于数据段而非栈上，因此跨调用保留数值。

【内存布局图解】 以 `insert_sorted` 建好的 10→20→30→40→50 为例（地址为示意）：

```
 栈 (automatic)                          堆 (allocated)
+-----------------+       +----------------+     +----------------+     +----------------+
| head = 0x9A40   |-----> | value=10       |     | value=20       |     | value=30       |
| tail = 0x9C70   |--+    | next  = 0x9B18 |---> | next  = 0x9C70 |---> | next  = 0x9D88 |--
+-----------------+  |    +----------------+     +----------------+     +----------------+
                     +--------> tail 指向尾结点，尾插 O(1)
 find = &head -> 不匹配;  find = &head->next -> 命中 20;
 dead = *find;  *find = dead->next;  free(dead)      /* head/中间/尾部同一套代码 */
```

【与汇编的对应】 设 `head` 是文件作用域变量（符号表：`from = R4, offset = 0`），`value` 在偏移 0、`next` 在偏移 1（LC-3 按 16 位字编址）：

```
; ---- 遍历：for (p = head; p != NULL; p = p->next) ----
        LDR   R0,R4,#0        ; R0 = head
LOOP    BRz   DONE            ; p == NULL ? 退出
        LDR   R2,R0,#0        ; R2 = p->value    (偏移 0)
        ; ... 使用 R2 ...
        LDR   R0,R0,#1        ; p = p->next      (偏移 1)
        BRnzp LOOP
DONE

; ---- 头插：n->next = head; head = n;（顺序不可颠倒）----
        LDR   R0,R4,#0        ; R0 = head (旧链表)
        ADD   R6,R6,#-1
        STR   R0,R6,#0        ; 把 head 作为参数传给 MAKE_NODE
        JSR   MAKE_NODE       ; 返回新结点地址在栈顶
        LDR   R1,R6,#0        ; R1 = n
        ADD   R6,R6,#2        ; 弹出返回值与参数
        LDR   R0,R4,#0        ; R0 = head (旧链表地址)
        STR   R0,R1,#1        ; n->next = head   <- 先接旧链表
        STR   R1,R4,#0        ; head = n         <- 再改头指针
```

示例 2：用链表实现栈与队列

代码 (C) — `/tmp/ece220_l17/ll_stack_queue.c`：

```
static node_t* stack = NULL;

static void push(int32_t value)          /* LIFO：头插 */
{
    node_t* n = malloc(sizeof(*n));
    n->value = value;
    n->next = stack;
    stack = n;
}

static int32_t pop(int32_t* value)       /* LIFO：删头 */
{
    node_t* old = stack;
    if (NULL == stack) {
        return 0;
    }
    *value = stack->value;
    stack = stack->next;
    free(old);
    return 1;
}

static node_t* q_head = NULL;
static node_t* q_tail = NULL;

static void enqueue(int32_t value)       /* FIFO：尾插，O(1) */
{
    node_t* n = malloc(sizeof(*n));
    n->value = value;
    n->next = NULL;
    if (NULL == q_head) {
        q_head = n;
    } else {
        q_tail->next = n;
    }
    q_tail = n;
}

static int32_t dequeue(int32_t* value)   /* FIFO：删头，O(1) */
{
    node_t* old = q_head;
    if (NULL == q_head) {
        return 0;
    }
    *value = q_head->value;
    q_head = q_head->next;
    if (NULL == q_head) {
        q_tail = NULL;                   /* 队列变空，tail 必须同步归零 */
    }
    free(old);
    return 1;
}
```

验证到的真实输出：

```
stack pops : 4 3 2 1
pop empty  : empty
queue pops : 100 200 300 400
deq empty  : empty
```

【代码做什么？】 1. `push(1..4)` 把 1、2、3、4 压栈，栈顶为 4。 2. `while (pop(&v))` 连续弹出得到 4、3、2、1（后进先出）；栈空时 `pop` 返回 0，循环结束。 3. 再次 `pop` 打印 `empty`，验证空栈判断；`enqueue` / `dequeue` 对 100..400 得到先进先出的 100、200、300、400。

【底层机制透视】 栈与队列共用同一结点结构，区别只在**在哪一端增删**：栈只改一个指针；队列必须同时维护 `q_head` 与 `q_tail`。 `dequeue` 里 `if (NULL == q_head) { q_tail = NULL; }` 是关键：若只把 `q_head` 置空而不管 `q_tail`，`q_tail` 就指向已释放结点，下一次 `enqueue` 会写 `q_tail->next`，即**写已释放内存**（堆破坏）。 `pop` / `dequeue` 用 `int32_t*` 输出参数返回数据、用返回值表达成败，这与 `mem220_reallocate` 的 `-1` / `0` 约定一致：调用者必须先检查返回值再使用 `*value`。

【内存布局图解】

```
栈（stack 容器，只动 stack 指针）          队列（queue 容器，两端各一个指针）
stack -> 4 -> 3 -> 2 -> NULL              q_head -> 100 -> 200 -> 300 -> NULL
                                          q_tail --------------------^
          push/pop 只动 stack；enqueue 动 q_tail，dequeue 动 q_head
```

【与汇编的对应】（`value` 在偏移 0、`next` 在偏移 1）

```
; ---- push(value)：value 已在 R0 ----
        ADD   R6,R6,#-1
        STR   R0,R6,#0        ; 参数入栈
        JSR   MALLOC_NODE     ; 返回新结点地址在栈顶
        LDR   R1,R6,#0        ; R1 = n
        ADD   R6,R6,#2        ; 弹出返回值与参数
        LDR   R2,R4,#0        ; R2 = stack
        STR   R2,R1,#1        ; n->next = stack
        STR   R1,R4,#0        ; stack = n

; ---- pop：删头 ----
        LDR   R0,R4,#0        ; R0 = stack
        BRz   POP_EMPTY       ; 空栈
        LDR   R1,R0,#0        ; R1 = stack->value
        LDR   R2,R0,#1        ; R2 = stack->next
        STR   R2,R4,#0        ; stack = stack->next
        ; ... 调用 FREE(R0) ...
```

示例 3：带哨兵的循环双向链表

代码 (C) — `/tmp/ece220_l17/dll.c`（核心部分）：

```
typedef struct dnode_t dnode_t;

struct dnode_t {
    int32_t value;
    dnode_t* prev;
    dnode_t* next;
};
static dnode_t sentinel;            /* static：静态存储期，程序全程存在 */

static void dl_init(void)
{
    sentinel.prev = &sentinel;      /* 空表：哨兵指向自己 */
    sentinel.next = &sentinel;
}

static void dl_insert_after(dnode_t* head, dnode_t* elt)   /* 4 次指针改写，无循环 */
{
    elt->next = head->next;
    elt->prev = head;
    head->next->prev = elt;
    head->next = elt;
}

static void dl_remove(dnode_t* elt)                        /* 2 次指针改写，无搜索 */
{
    elt->prev->next = elt->next;
    elt->next->prev = elt->prev;
    elt->prev = NULL;
    elt->next = NULL;
}
```

验证到的真实输出：

```
forward    head <-> 10 <-> 20 <-> 30 <-> head (cyclic)
backward   head <-> 30 <-> 20 <-> 10 <-> head (cyclic)
after removing 30:
forward    head <-> 10 <-> 20 <-> head (cyclic)
```

【代码做什么？】 1. `dl_init` 让哨兵的前后指针都指向自己，表示空表。 2. 依次 `dl_insert_after(&sentinel, a=20)`、`dl_insert_after(a, b=30)`、`dl_insert_after(&sentinel, c=10)`，每次都插在头部之后，最终顺序为 10、20、30。 3. `print_forward` 从 `sentinel.next` 出发、以 `&sentinel != p` 为终止条件，绕回哨兵即停；`print_backward` 把 `next` 换成 `prev` 得到逆序。 4. `dl_remove(b)` 摘除 30，只需两次指针改写，**不需要从头搜索前驱**。

【底层机制透视】 单链表删除要"找前驱"（`O(n)`），因为只有 `next` 一条线索；双向链表结点自带 `prev`，已知结点指针即可 `O(1)` 删除。 哨兵把"表头"从可为 `NULL` 的指针变成**永远存在的结点**，于是插入/删除代码不再需要任何边界判断——这是系统编程用哨兵消除特例的经典手法（Linux 内核的 `list_head` 即此结构）。 代价是每结点多 8 字节，且指针改写必须成对：`dl_insert_after` 四步少做一步，前驱的 `prev` 就永久失去同步。

【内存布局图解】

```
 数据段 (static sentinel)              堆
+----------------------+          +-----------------+     +-----------------+
| sentinel.prev = ----|--+       | value = 10      |     | value = 20      |
|   &sentinel (0x4100)|  |       | prev = --------|--+  | prev = --------|--+
| sentinel.next = ----|--+       | next = --------|--+->| next = --------|--+-> 回哨兵
|   &sentinel (0x4100)|  |       +-----------------+  |  +-----------------+  |
+----------------------+  |         @0x9A40           |    @0x9B18            |
      @0x4100             |                            |                       |
                          +----------------------------+-----------------------+
   两个方向都能回到哨兵，所以循环终止条件是 &sentinel != p，而不是 p != NULL
```

【与汇编的对应】（`prev` 在偏移 1、`next` 在偏移 2）

```
; ---- dl_insert_after(head, elt)：head 在 R5+4，elt 在 R5+5 ----
        LDR   R0,R5,#4        ; R0 = head
        LDR   R1,R5,#5        ; R1 = elt
        LDR   R2,R0,#2        ; R2 = head->next
        STR   R2,R1,#2        ; elt->next = head->next
        STR   R0,R1,#1        ; elt->prev = head
        LDR   R2,R1,#2        ; R2 = elt->next
        STR   R1,R2,#1        ; head->next->prev = elt
        STR   R1,R0,#2        ; head->next = elt
        RET
```

演示（仅供演示，请勿模仿）：丢失整条链表

演示 A：头插顺序颠倒（`/tmp/ece220_l17/ll_bug_lost.c`）

```
/* DEMONSTRATION ONLY -- DO NOT IMITATE */
static node_t* broken_push_front(node_t* head, int32_t value)
{
    node_t* n = make_node(value);

    head = n;          /* BUG：n->next 仍是 NULL，旧链表被彻底孤立 */
    return head;
}
```

真实输出（程序本身是良定义的，但**泄漏**了 3 个结点）：

```
before broken insert   head -> 10 -> 20 -> 30 -> NULL
after broken insert    head -> 40 -> NULL
```

【为什么丢失】 `head = n` 只把栈上的头指针改指 40；10、20、30 仍在堆上，但**再没有任何指针指向它们**，程序既无法访问也无法释放。

```
错误写法（head = n）：head -> 40 -> NULL，而 10 -> 20 -> 30 成为孤立结点（不可达 = 泄漏）
正确写法（先 n->next = head 再 head = n）：head -> 40 -> 10 -> 20 -> 30 -> NULL
```

演示 B：释放后继续使用（悬垂指针）（`/tmp/ece220_l17/ll_bug_uaf2.c`）

```
/* DEMONSTRATION ONLY -- DO NOT IMITATE.  UB. */
static int32_t peek(node_t* p) { return p->value; }
static void release(node_t* p) { free(p); }

int main(void)
{
    node_t* head = malloc(sizeof(*head));

    head->value = 7;
    head->next = NULL;
    release(head);
    printf("value after free = %d\n", peek(head));  /* UB：读已释放内存 */
    return 0;
}
```

若把 `head->value` 直接写在 `free` 之后，`-Werror` 会在编译期拦下：`error: pointer 'head' used after 'free' [-Werror=use-after-free]`（GCC 12+ 的静态分析）。把读写拆进两个函数后编译器无法静态证明，程序得以编译，此时输出**完全不确定**（一次运行得到 `value after free = 3491`，另一次得到 `7`）。Valgrind 能稳定抓住它：

```
$ valgrind --error-exitcode=9 -q ./ll_bug_uaf2
==1212390== Invalid read of size 4
==1212390==    at 0x401152: peek (ll_bug_uaf2.c:19)
==1212390==  Address 0x4a6f040 is 0 bytes inside a block of size 16 free'd
==1212390==    by 0x40116D: release (ll_bug_uaf2.c:24)
```

UB：此处的输出完全依赖分配器实现与运行环境，绝不可依赖。

常见错误与调试技巧

- **头插顺序颠倒**：`head = n; n->next = head;` 会让 `n->next` 指向自己（遍历死循环），或让旧链表丢失。 **调试**：`valgrind --leak-check=full --show-leak-kinds=all ./prog` 报告 "definitely lost" 块及其分配栈；在 GDB 中 `print head`、`print n->next` 逐步核对。
- **`free` 后访问 `p->next`**：销毁链表时写 `free(p); p = p->next;` 是 UB，可能读到分配器元数据而崩溃。 **调试**：`valgrind -q ./prog` 报 "Invalid read of size 8"；或 `gcc -fsanitize=address -g`（该选项需要足够虚拟地址空间，受限环境下改用 Valgrind）。
- **销毁后忘记置 `head = NULL`**：调用者继续使用旧头指针即悬垂访问。 **调试**：销毁接口用 `node_t** head`，从类型上强制"我能改写你的头指针"；销毁后用 `print head` 确认其为 `0x0`。
- **在遍历中删除当前结点**：`for (p = head; p; p = p->next) { if (...) free(p); }` 会在 `free` 后读 `p->next`。 **调试**：改用 `node_t** find` 循环（先取 `next` 再改链接）；`gdb` 中 `watch -l p->next`

观察字段何时被改写。

- **结构体自引用写成内嵌**：`struct node_t { int32_t value; struct node_t next; };` 报 `field 'next' has incomplete type`。**调试**：`gcc -std=c99 -Wall -Werror -c file.c` 直接定位错误行；规则是"自引用必须是指针"。
- **删空队列后忘记 `q_tail = NULL`**：`q_tail` 成为悬垂指针，下一次 `enqueue` 写坏堆。**调试**：`valgrind -q ./prog` 报 "Invalid write of size 8"；每次 `enqueue` 前用 `p q_head` / `p q_tail` 确认二者同为 `0x0` 或都非空。
- **`sizeof` 用错**：`malloc(sizeof(node_t*))` 只分配指针大小的 8 字节，写 `n->next` 就越界。 **调试**：`valgrind -q ./prog` 报 "Invalid write of size 8"；养成 `malloc(sizeof(*n))` 的习惯。

关键要点

- 数组与链表的差别源于**内存布局**：连续布局换来 `O(1)` 随机访问，链式布局换来 `O(1)` 插入删除；选择容器本质是在"访问模式"与"修改模式"之间取舍。
- 自引用结构的成员**必须是指针**；`NULL` 是链表唯一的终止符，一切循环都写成 `NULL != p`。
- **指针的指针** `T**` 是消除链表边界情况的统一工具：`find = &head` 起步、`find = &(*find)->next` 前进，最后 `*find = ...` 一句话完成头/中/尾的插入与删除。
- 多指针改写**顺序即正确性**：头插先接后换、销毁先存 `next` 再 `free`、双向链表四个指针成对更新。
- 越"强大"的结构越要维护不变量：带哨兵的循环双向链表用"空表时哨兵指向自己"换掉了所有 `NULL` 判断，代价是每结点 8 字节与更严格的改写顺序。

思考题（带答案）

问题 1：下面这个 `append` 为什么在第二次调用时破坏链表？

```
static node_t* append(node_t* head, node_t* tail, int32_t value)
{
    node_t* n = make_node(value);

    if (NULL == head) { head = n; } else { tail->next = n; }
    tail = n;
    return head;
}
```

答案：`tail` 是按值传递的指针，函数内 `tail = n` 只改到副本，调用者的尾指针永远停在第一个结点。第二次调用时 `tail` 仍指向旧结点，`tail->next = n` 把新结点插在旧结点之后而不是队尾，链表结构被破坏；若调用者传入的 `tail` 已是已释放结点则直接 UB。修正是传 `node_t** tail`（`*tail = n;`）或让函数返回新的尾指针。

问题 2：给定长度 `n = 100000` 的链表，需要"按值查找"与"在第 50000 个元素前插入"各 1000 次。你会坚持用链表吗？

答案：不会无条件坚持。单链表按值查找是 `O(n)`，1000 次即 `10^8` 次结点访问；"已知位置的插入"虽只需 `O(1)` 次指针改写，但**定位**该位置同样要 `O(n)`。若改用动态数组：先排序后二分查找为 `O(log n)`，中间插

入虽要 `O(n)` 次搬移，但搬移的是连续内存（`memcpy`、缓存友好、常数极小）。因此对"查找/插入混合"的场景动态数组通常更快；链表只在元素本身巨大（搬移代价高）或需要结点地址长期稳定（外部已握有结点指针）时才占优。

Lecture 18: 树、遍历与搜索；从 C 到 LC-3 汇编 (Trees, Traversal and Search; From C to LC-3 Assembly with Linked Data Structures)

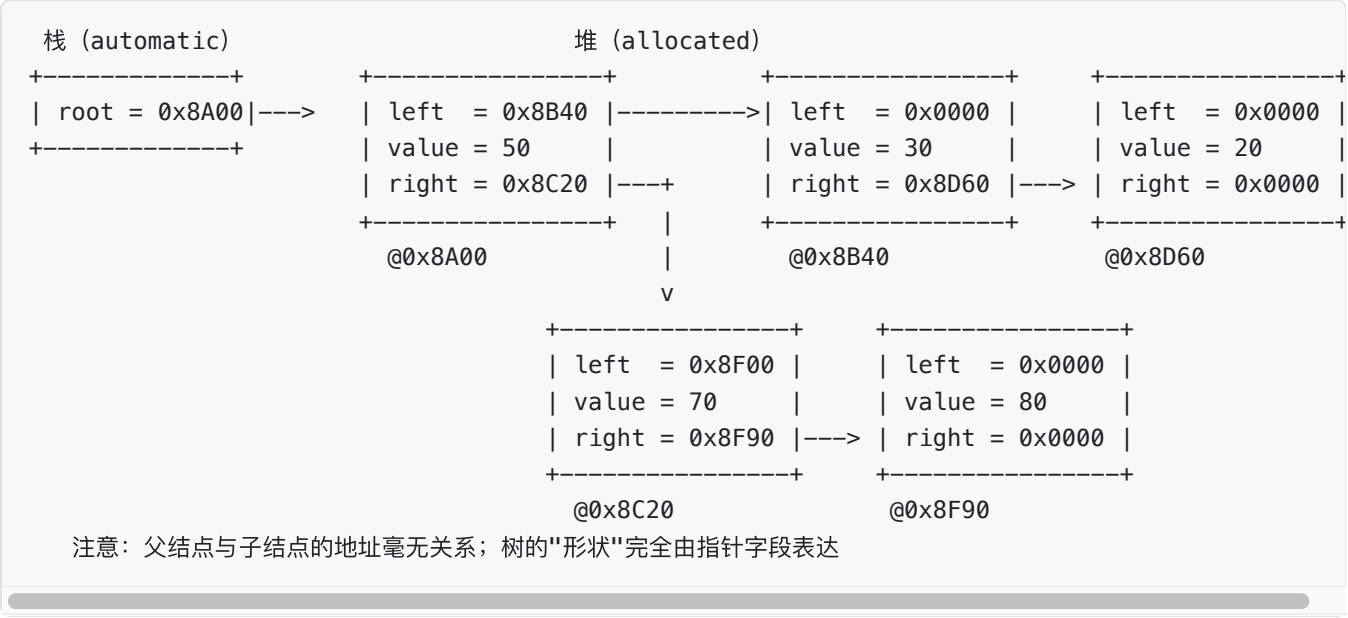
概述

本讲把上一讲的"一个结点带一条链"推广为"一个结点带两条链"，得到**二叉树 (binary tree)**，并在此基础上实现**二叉搜索树 (binary search tree, BST)** 的插入、查找与三种深度优先遍历。 关键的新机制是**递归**与**自引用结构的组合**：遍历、求高度、统计结点数、整树释放全部写成递归函数，其中整树释放**必须使用后序遍历 (postorder)**。 最后我们把一个递归树函数手工翻译成 LC-3 汇编，看清 `p->value`、`p = p->left` 如何变成带常量偏移的 `LDR`，以及为什么**任何含子程序调用的函数都必须在栈帧中保存 R7**——这正是 ECE 220 "C 语句如何变成机器码"这一教学目标的核心演练。

核心概念与底层机制图解

- **树 (tree) 作为层级式指针结构**：每个结点可以有多个后继，且无环；从根到任一结点恰有一条路径。
  - *直观解释*：像家族谱系或文件系统的目录树——每个"父亲"可以有若干个"儿子"，但每个结点只有一个"父亲"。
  - *底层机制图解*：树的实现方式有两类。**指针式 (pointer-based)**：结点用 `malloc` 分配在堆上，用指针成员连接（本讲 `bst.c`）；**数组式 (array-based)**：所有结点放在一个数组里，用下标算式表达父子关系——课程的**金字塔树 (pyramid tree)** 就是后者，`mp9.c` 中下标 `N` 的结点其子结点下标为 `4N+1` … `4N+4`，叶子结点直接对应图的顶点，从而整棵树只需一次 `malloc`、遍历时缓存友好。
  - *作用域与存储期*：指针式树的结点是 allocated storage duration，必须逐结点 `free`；数组式树的结点随那一个数组一起生死，释放只需一次 `free`。
- **二叉树结点 (binary tree node) 与内存布局**：`struct node_t { node_t* left; int32_t value; node_t* right; };`
  - *直观解释*：每个结点像一张卡片，卡片上写着"左孩子在哪个房间、右孩子在哪个房间、我自己是多少"。
  - *底层机制图解*：`left` 在偏移 0、`value` 在偏移 1、`right` 在偏移 2（LC-3 按 16 位字编址）；64 位平台上含填充共 24 字节，其中 16 字节是"结构开销"。`p->value` 编译成 `LDR R1,R0,#1`，`p = p->left` 编译成 `LDR R0,R0,#0`，都是一条指令——**结构体成员访问就是"基址 + 编译期常量偏移"**。

二叉树的内存布局图解（结点散布在堆上，用两个指针连接）：



注意：父结点与子结点的地址毫无关系；树的"形状"完全由指针字段表达

- **二叉搜索树的有序不变量 (BST ordering invariant)**：对任意结点 `n`，其**左子树所有值 < `n->value` < 右子树所有值**。
  - *直观解释*：像一本按字母排好的电话簿：左边全是"更小的"，右边全是"更大的"。
  - *底层机制图解*：不变量约束的是**整棵子树**而不是直接孩子：`left->value < n->value` 只是它的必要条件，不是充分条件。它带来的直接好处是"每次比较都能丢弃一棵子树"，于是查找路径长度等于树高 `h`，平均 `O(log n)`。
  - *作用域与存储期*：不变量是**所有修改函数共同维护的契约**（`insert`/`delete`/旋转都要保证），一旦某个函数破坏它，`search` 会静默地找不到存在的元素——这类 bug 最难调试。
- **插入 (insertion)**：沿查找路径下降，走到 `NULL` 处挂上新结点。
  - *直观解释*：像按字母顺序往书架上插书，一路比较，找到空位就放进去。
  - *底层机制图解*：递归写法直接使用返回值回填父子链接：`root->left = insert(root->left, value);`——这是"用返回值修改指针字段"的技巧；迭代写法则需要 `node_t** find`（与上一讲链表的指针的指针完全同源）。时间复杂度 `O(h)`，插入顺序决定树的形状。
  - *作用域与存储期*：新结点由 `malloc` 分配，挂到树上后其生命周期由"谁负责 `free` 整棵树"决定；函数返回时只有局部变量消失，结点留在堆上。
- **查找与其 `O(height)` 代价 (search and its cost)**：从根出发，比较后只走一边。
  - *直观解释*：像查字典——比目标小就翻左边，比目标大就翻右边。
  - *底层机制图解*：迭代版本只用一个指针变量，无需递归，空间 `O(1)`：`while (NULL != root && value != root->value) { root = (value < root->value) ? root->left : root->right; }`。代价取决于**树高 `h`**：平衡时 `h ≈ log2(n)`，退化时 `h = n - 1`（下一节）。
  - *作用域与存储期*：`search` 返回指向堆中结点的指针；只要不 `free`，该指针一直有效；但一旦整树被释放，所有旧指针立即变成悬垂指针。
- **三种深度优先遍历 (depth-first traversals)**：区别只在于"什么时候处理根结点"。
  - *直观解释*：把每个结点的工作分成"打印自己、走左子树、走右子树"三件事，三种顺序对应三种遍历。

- *底层机制图解*：以本讲 `bst.c` 实际插入顺序 `50, 30, 70, 20, 40, 60, 80` 建成的树为例：

```
              50
            /    \
          30      70
         /  \    /  \
       20   40  60   80

前序 preorder  (根, 左, 右): 50 30 20 40 70 60 80
中序 inorder   (左, 根, 右): 20 30 40 50 60 70 80   <- 升序!
后序 postorder (左, 右, 根): 20 40 30 60 80 70 50
层序 breadth-first (用队列): 50 30 70 20 40 60 80
```

```
    **为什么 BST 的中序是升序**：对任意结点，中序先完整输出它的左子树（全比它小），再输出它自己，最后输出右子
*   *作用域与存储期*：递归遍历的每个活动调用在自己的栈帧里保存一份局部变量与返回地址，递归深度等于树高 h，因此
```

- **递归统计与整树释放 (recursive metrics and teardown)**：高度、结点数、求和都可以用同一种"左右递归 + 合并"模板；释放则**必须后序**。
  - *直观解释*：数一棵树有多少片叶子，先数左半边再数右半边；拆房子则必须先拆完上面两层，才能拆地基。
  - *底层机制图解*：`count(n) = 1 + count(left) + count(right)`，`sum(n) = n->value + sum(left) + sum(right)`，`height(n) = 1 + max(height(left), height(right))`（空树高度定义为 `-1`，使单结点树高度为 0）。释放必须是 `free_tree(left); free_tree(right); free(n);`——若先 `free(n)` 就再也拿不到两个子结点地址了。课程的 `trees.c` 里 `free_tree` 正是三叉版本：先递归释放 `right`、`mid`、`left`，最后 `free (n)`。
  - *作用域与存储期*：递归函数每层的局部变量都是 automatic，随该层栈帧销毁；堆上的结点只有在对应的 `free` 之后才结束生命周期。
- **退化与平衡 (degeneration and balance)**：按升序插入 `1, 2, …, n` 得到的是一棵"只有右孩子的链"，高度 `n - 1`。
  - *直观解释*：每次新元素都最大，于是永远往右走——树长成了一根竹竿。
  - *底层机制图解*：本讲 `bst.c` 实测：插入 `1..8` 得到的树 `height = 7, nodes = 8`，`search` 退化为 `O(n)`，递归的栈深度也变成 `n`。平衡带来的收益是 `h = O(log n)`：`n = 10^6` 时 `h ≈ 20` 而不是 `10^6`。保持平衡需要**旋转 (rotation)**（AVL 树、红黑树）或随机化插入顺序；平衡的代价是每次修改多几次指针改写。
- **广度优先遍历 (breadth-first traversal)**：用队列逐层访问，不需要递归。
  - *直观解释*：像水波纹一圈圈扩散：先看根，再看根的两个孩子，再看四个孙子……
  - *底层机制图解*：把根入队，然后循环"出队一个、访问它、把它的非空孩子依次入队"。队列可以用上一讲的链式队列，也可以用环形数组（`bst.c` 用的是简单数组 + `head`/`tail` 下标）。BFS 与 DFS 的差别只在容器：**栈 → DFS，队列 → BFS**。

    - *作用域与存储期*：BFS 的队列所需空间正比于**树的最大宽度**（最坏 `n/2`），DFS 的栈空间正比于**树高**——这是选择遍历策略时的重要工程权衡。
- **结构层级与"首字段包含" (hierarchies of structures)**：把共同字段放进父类型，并让父类型作为子类型的**第一个字段**，就能安全地把子类型指针向上转型。
    - *直观解释*：所有证件的第一页都是"姓名 + 照片"，于是不管后面的内容是驾驶证还是护照，只看第一页都能当"身份证"来处理。
    - *底层机制图解*：课程例子里 `book_t` 的第一个字段是 `reference_t`，`reference_t` 的第一个字段是 `double_list_t`；因为偏移为 0，`book_t*`、`reference_t*`、`double_list_t*` 三者的地址**完全相同**，向上转型（`(reference_t*)elt`）是安全的。反过来从父类型指针转到子类型**不安全**：仅凭 `reference_t*` 无法知道后面跟着的是什么，必须先有一个 `type` 字段做动态类型标记，再用 `switch` 分派。这正是 C 语言里"虚函数"的手工实现方式（对比 Lecture 12 的函数指针回调）。

代码示例与底层机制分析

示例 1：二叉搜索树的插入、查找、四种遍历与统计

代码 (C) — `/tmp/ece220_l18/bst.c`（完整文件已编译运行）：

```c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct node_t node_t;

struct node_t {
    node_t* left;
    int32_t value;
    node_t* right;
};

static node_t* make_node(int32_t value)
{
    node_t* n = malloc(sizeof(*n));

    if (NULL == n) {
        fprintf(stderr, "out of memory\n");
        exit(2);
    }
    n->left = NULL;
    n->right = NULL;
    n->value = value;
    return n;
}

/* 返回新的子树根；维持 left < node < right 的不变量 */
static node_t* insert(node_t* root, int32_t value)
{
    if (NULL == root) {
        return make_node(value);
    }
    if (value < root->value) {
        root->left = insert(root->left, value);
    } else if (value > root->value) {
        root->right = insert(root->right, value);
    }
    return root;
}

static node_t* search(node_t* root, int32_t value)      /* 迭代版：O(height)，空间 O(1) */
{
    while (NULL != root && value != root->value) {
        root = (value < root->value) ? root->left : root->right;
    }
    return root;
}

static void preorder(node_t* root)                       /* 根，左，右 */
{
    if (NULL == root) {
        return;
    }
```

```
    printf(" %d", root->value);
    preorder(root->left);
    preorder(root->right);
}

static void inorder(node_t* root)                          /* 左, 根, 右 */
{
    if (NULL == root) {
        return;
    }
    inorder(root->left);
    printf(" %d", root->value);
    inorder(root->right);
}

static void postorder(node_t* root)                        /* 左, 右, 根 */
{
    if (NULL == root) {
        return;
    }
    postorder(root->left);
    postorder(root->right);
    printf(" %d", root->value);
}

static int32_t height(node_t* root)
{
    int32_t lh;
    int32_t rh;

    if (NULL == root) {
        return -1;                        /* 空树高度 -1, 单结点树高度 0 */
    }
    lh = height(root->left);
    rh = height(root->right);
    return 1 + ((lh > rh) ? lh : rh);
}

static int32_t count(node_t* root)
{
    if (NULL == root) {
        return 0;
    }
    return 1 + count(root->left) + count(root->right);
}

static int32_t sum(node_t* root)
{
    if (NULL == root) {
        return 0;
    }
    return root->value + sum(root->left) + sum(root->right);
}
```

```
static void free_tree(node_t* root)                    /* 必须后序：先孩子后自己 */
{
    if (NULL == root) {
        return;
    }
    free_tree(root->left);
    free_tree(root->right);
    free(root);
}
```

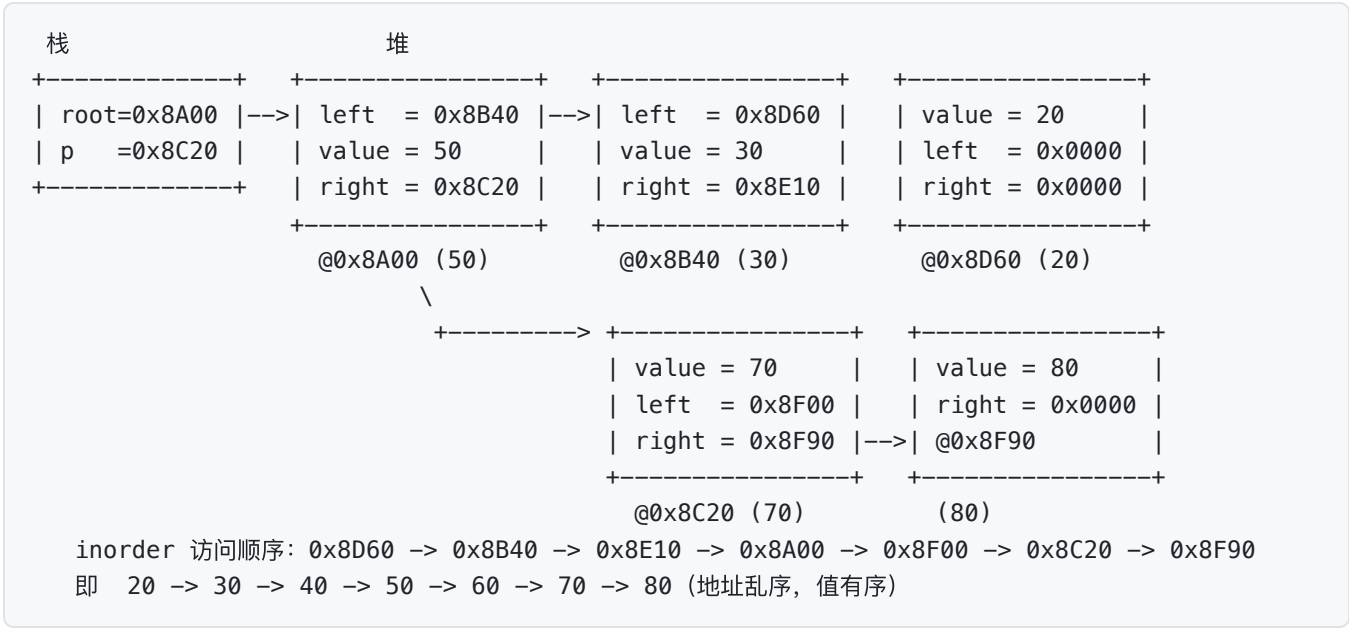
验证到的真实输出（`gcc -g -std=c99 -Wall -Werror bst.c -o bst && ./bst`）：

```
preorder  : 50 30 20 40 70 60 80
inorder   : 20 30 40 50 60 70 80
postorder : 20 40 30 60 80 70 50
breadth   : 50 30 70 20 40 60 80
height = 2, nodes = 7, sum = 350
search 60 -> found
search 65 -> no
balanced tree: allocated = 7, freed = 7
degenerate tree 1..8: height = 7, nodes = 8
inorder of degenerate tree is still sorted: 1 2 3 4 5 6 7 8
total allocated = 15, freed = 15
```

【代码做什么？】 1. 以 `50, 30, 70, 20, 40, 60, 80` 的顺序调用 `insert`，每次下降比较后挂到 `NULL` 位置，得到根为 50、高度为 2 的完全二叉树。 2. `preorder` / `inorder` / `postorder` 分别按"根左右 / 左根右 / 左右根"输出，得到 `50 30 20 40 70 60 80` / `20 30 40 50 60 70 80` / `20 40 30 60 80 70 50`。 3. `breadth_first` 用数组队列逐层输出 `50 30 70 20 40 60 80`。 4. `height` 返回 2（以边数计），`count` 返回 7，`sum` 返回 350 = 50+30+70+20+40+60+80，三者都由递归合并子树结果得到。 5. `search(root, 60)` 命中返回结点地址，`search(root, 65)` 走到 `NULL` 返回空。 6. `free_tree` 后分配/释放计数 7 = 7；再插入 `1..8` 得到退化树，实测 `height = 7, nodes = 8`，但其中序仍然是升序，说明不变量未被破坏、退化的只是形状。

【底层机制透视】 `insert` 用 `root->left = insert(root->left, value);` 这种"把递归结果写回指针字段"的写法，好处是**插入新结点时不需要区分特例**：当 `root->left == NULL` 时递归返回新结点地址，赋值语句恰好把它挂上。 递归函数的每一层都在栈上占一个栈帧，因此遍历的空间开销是 `O(h)`；`height` 的递归分叉是"两次调用 + 取较大值"，属于**树形递归**，总调用次数为 `2n + 1`。 `search` 写成迭代版是为了强调：**递归不是必需的**，只要每步只走一条分支就可以用循环。反之，需要访问两条分支（遍历、统计、释放）时，用递归保存"另一半还没做"的状态最自然。 `free_tree` 若改成前序（先 `free(root)` 再递归孩子），会在 `free` 之后读 `root->left`，属于 UB；这也是本讲最常见的错误。

【内存布局图解】 以插入顺序 `50, 30, 70, 20, 40, 60, 80` 建成的树为例（地址为示意）：



inorder 访问顺序：0x8D60 -> 0x8B40 -> 0x8E10 -> 0x8A00 -> 0x8F00 -> 0x8C20 -> 0x8F90
即 20 -> 30 -> 40 -> 50 -> 60 -> 70 -> 80（地址乱序，值有序）

【与汇编的对应】 `p->value`、`p->left`、`p->right` 都是一条 `LDR`；比较后用条件分支选择走哪边：

```
; ---- search: while (root != NULL && value != root->value) ----
; 参数: root 在 R5+4, value 在 R5+5
SEARCH  LDR   R0,R5,#4        ; R0 = root
        LDR   R1,R5,#5        ; R1 = value
S_LOOP  BRz   S_NULL          ; root == NULL ? 没找到
        LDR   R2,R0,#1        ; R2 = root->value    (value 在偏移 1)
        NOT   R3,R2
        ADD   R3,R3,#1        ; R3 = -root->value
        ADD   R3,R3,R1        ; R3 = value - root->value
        BRz   S_FOUND         ; 相等, 命中
        BRn   S_GO_LEFT       ; value < root->value
        LDR   R0,R0,#2        ; root = root->right   (right 在偏移 2)
        BRnzp S_LOOP
S_GO_LEFT
        LDR   R0,R0,#0        ; root = root->left    (left 在偏移 0)
        BRnzp S_LOOP
S_NULL  AND   R0,R0,#0        ; 返回 NULL
        BRnzp S_DONE
S_FOUND LDR   R0,R5,#4        ; 返回命中的结点地址
S_DONE  STR   R0,R5,#3        ; 写入返回值槽
        RET
```

示例 2：把树"压平"成数组再复原（课程的 flattening 例子）

代码 (C) — `/tmp/ece220_l18/flatten.c`（核心部分；课程原版为 `Ccode/flattening/trees.c`，三叉树）：

```
#define ABSENT 0x80000000

/* 孩子先写、结点最后写 —— 这正是后序（左，中，右，根）*/
static int32_t pack(node_t* root, int32_t ar[], int32_t pos)
{
    if (NULL == root) {
        ar[pos] = ABSENT;
        return pos + 1;
    }
    pos = pack(root->left, ar, pos);
    pos = pack(root->mid, ar, pos);
    pos = pack(root->right, ar, pos);
    ar[pos] = root->value;
    return pos + 1;
}

/* 从数组末尾倒着读：先读根，再读右、中、左 */
static node_t* build(const int32_t ar[], int32_t* pos)
{
    int32_t v = ar[--(*pos)];
    node_t* n;

    if (ABSENT == v) {
        return NULL;
    }
    n = make_node(v);
    n->right = build(ar, pos);
    n->mid = build(ar, pos);
    n->left = build(ar, pos);
    return n;
}
```

验证到的真实输出：

```
packed      : ABSENT ABSENT ABSENT 5 ABSENT ABSENT ABSENT ABSENT 6 2 ABSENT ABSENT ABSENT 3
build consumed 22 of 22 words
repacked    : ABSENT ABSENT ABSENT 5 ABSENT ABSENT ABSENT ABSENT 6 2 ABSENT ABSENT ABSENT 3
repacked identical: yes
```

课程原版 `trees.c` 用真实输入文件做**往返 (round trip)** 测试，我也实际编译运行过：

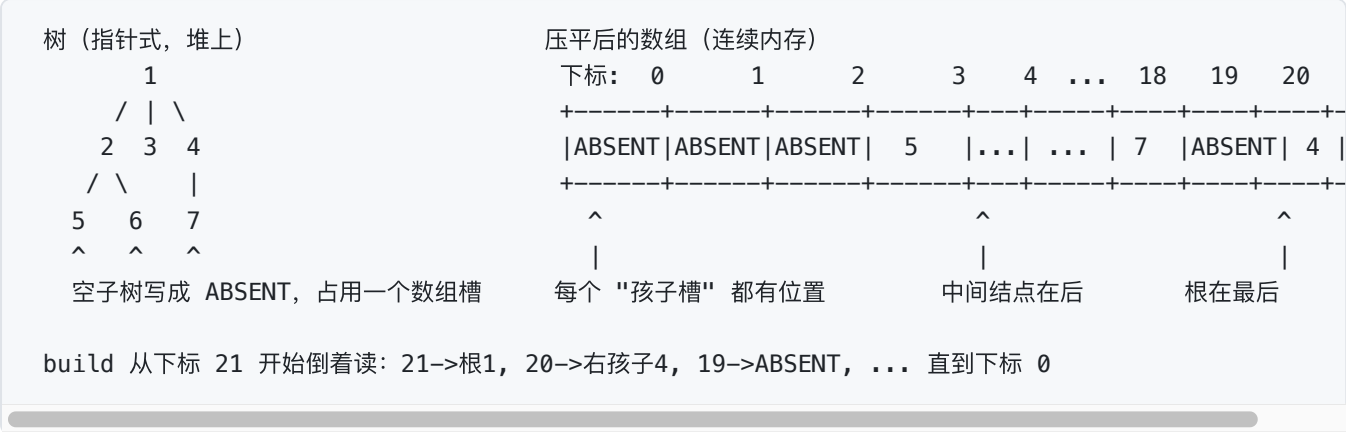
```
$ gcc -g -std=c99 -Wall -Werror trees.c -o trees
$ ./trees sample out          # 读入 sample，反压平成树，再压平写回 out
$ diff sample out && echo IDENTICAL
IDENTICAL
```

（`sample` 的首行是数组长度 `25`，随后 25 个 `int32_t` 数据，其中 `-2147483648` 即 `ABSENT` 标记，代表"空子树"。）

【代码做什么？】 1. `pack` 递归压平三叉树：遇到 `NULL` 子树就写一个 `ABSENT` 标记，否则先写左、中、右三棵子树，最后写自己的值。 2. 输出显示 22 个数据字，末尾的 `4 1` 说明根结点 `1` 的值写在**数组最后**——因为它是最后被"完成"的。 3. `build` 从数组末尾倒着读：第一个读出的是根 `1`，然后依次递归还原 `right`、`mid`、`left`，与写入顺序严格互逆。 4. `build consumed 22 of 22 words` 说明数组被完整消费；重新 `pack` 得到的数组与原始数组逐字相同（`repacked identical: yes`），证明压平/复原是**无损可逆**的。 5. 课程的 `trees.c` 更进一步：它故意构造三种错误输入（数组过长、数组有剩余、结构非法），并检查程序是否正确地报错退出；对合法输入则 `diff sample out` 完全一致。

【底层机制透视】 压平的本质是**遍历顺序即存储顺序**：`pack` 写出的是"左–中–右–根"的后序序列，而 `build` 从尾部倒着读，恰好按"根–右–中–左"消费。之所以要倒着读，是因为**根必须最先被创建**，而根在数组末尾。 `pack` 用返回值把"下一个待写位置"传回上层（`return pos + 1;`），这与 `insert` 用返回值回填指针是同一种函数式风格：**用返回值串起递归的状态**。 这种"把指针结构序列化成整数数组"的技术在系统编程中极常见：网络协议要把对象树打包成字节流、编译器要把 AST 序列化到文件、`pyr_tree.c` 干脆就把整棵金字塔树存成一个 `pyr_node_t` 数组（`struct pyr_tree_t { int32_t n_nodes; pyr_node_t* node; }`），因为数组版本可以一次性 `malloc`、一次性 `free`、并且对缓存友好。

【内存布局图解】



【与汇编的对应】 `pack` 的"孩子先、自己后"决定了两条递归 `JSR` 必须写在 `STR` 之前：

```
; ---- pack(root, ar, pos)：root 在 R5+4，ar 在 R5+5，pos 在 R5+6 ----
; 局部：R5+0 = 当前 pos（因为递归调用会破坏 R0–R3，必须存回栈帧）
PACK    LDR   R0,R5,#4        ; R0 = root
        LDR   R1,R5,#5        ; R1 = ar
        LDR   R2,R5,#6        ; R2 = pos
        STR   R2,R5,#0        ; 把 pos 存进局部变量
        BRnp  P_NOT_NULL
        ; root == NULL：写 ABSENT，返回 pos + 1
        LEA   R3,ABSENT_VAL   ; 以 LEA + LDR 取出 0x80000000
        LDR   R3,R3,#0
        ADD   R2,R2,R1        ; 若 ar 已换算为绝对地址，这里直接算出目标地址
        STR   R3,R2,#0
        LDR   R2,R5,#0
        ADD   R2,R2,#1
        STR   R2,R5,#3        ; 返回值槽 = pos + 1
        BRnzp P_DONE
P_NOT_NULL
        ; 依次递归 left / mid / right，每次都用上一次的返回值更新 pos
        LDR   R0,R5,#4
        LDR   R0,R0,#0        ; left 在偏移 0（若结构为 value,left,mid,right 需相应调整）
        ; ... 压栈调用 PACK，读回返回值写回局部 pos ...
        ; ... 对 mid、right 重复 ...
        LDR   R2,R5,#0        ; 取出最终的 pos
        LDR   R3,R5,#4
        LDR   R3,R3,#3        ; 取 root->value
        ; ar[pos] = value; 返回值 = pos + 1
P_DONE  LDR   R7,R5,#2
        LDR   R5,R5,#1
        ADD   R6,R6,#3
        RET
```

示例 3：把递归树函数翻译成 LC-3（本讲核心）

代码 (C) — 取自 `/tmp/ece220_l18/bst.c` 的 `sum`，为便于逐句翻译写成显式局部变量的形式：

```
int32_t tree_sum (node_t* root)
{
    int32_t total;

    if (NULL == root) {
        return 0;
    }
    total = root->value;
    total += tree_sum (root->left);
    total += tree_sum (root->right);
    return total;
}
```

验证到的真实输出：`bst.c` 中同一逻辑的 `sum(root)` 打印 `sum = 350`（对 7 个结点 `20+30+40+50+60+70+80`），与手算一致。

【代码做什么？】 1. 若 `root` 为空，直接返回 0（递归的**基例 (base case)**）。 2. 否则把 `root->value` 存入局部变量 `total`。 3. 递归求左子树之和，把返回值累加到 `total`。 4. 递归求右子树之和，再次累加。 5. 返回 `total`。

【底层机制透视】 这个函数是**非叶函数 (non-leaf function)**：它自己会调用子程序。这带来两个硬约束：① `JSR` 会把返回地址写进 `R7`，覆盖本函数自己的返回地址，所以**必须在序言里把 `R7` 存进栈帧**（`STR R7,R5,#2`），否则第一次递归调用后本函数就无法 `RET`； ② `R0–R3` 是 caller-saved，两次递归调用之间的 `total` 必须放在**栈帧里的局部变量**（内存）而不是寄存器里。 第二个递归调用之前需要重新读取 `root`：它保存在本帧的参数槽 `R5+4`，而参数槽属于**本帧**，在整个函数执行期间保持有效——这正是"参数按值传递并存放在被调用者栈帧里"的工程价值。

栈帧布局（与课程讲义及 `translate.asm` 一致）：

```
高地址 ┌──────────────────────────────┐
       │  caller's stack frame        │
       ├──────────────────────────────┤
       │  parameters: root            │  <- R5+4    (由调用者压栈)
       ├──────────────────────────────┤
       │  previous frame pointer      │  <- R5+3
       ├──────────────────────────────┤
       │  return address (R7)         │  <- R5+2    (STR R7,R5,#2)
       ├──────────────────────────────┤
       │  return value                │  <- R5+1    (STR R0,R5,#1)
       ├──────────────────────────────┤
       │  local variables             │  <- R5+0, R5-1, ...   (R5 指向局部变量底部)
低地址 └──────────────────────────────┘     <- R6 指向栈顶（向低地址增长）
```

> 说明：上表偏移取自课程讲义与 `translate.asm` / MT1 复习课的指令序列（`STR R0,R5,#3 ; store -1 in return value location`、`LDR R5,R5,#1 ; restore caller's frame pointer`）。个别资料把 linkage 三格的编号顺序写得不同，遇到冲突时以讲义中的**指令**为准（因为只有"返回值紧贴参数下方"这一顺序，调用者才能在不知道被调用者局部变量个数的情况下，用一条 `ADD R6,R6,#N` 同时弹出返回值与参数）。

【内存布局图解】（以 `tree_sum(0x8A00)` 在求左子树之和的瞬间为例）

```
R5+4  | root = 0x8A00       |  本帧参数（指向 50 号结点）
R5+3  | prev frame = 0x7Fxx |  调用者的 R5
R5+2  | return addr         |  调用者中 tree_sum 之后的那条指令地址
R5+1  | return value        |  尚未写入
R5+0  | total = 50          |  局部变量：root->value
R6 -> | 子调用 tree_sum(0x8B40) 的栈帧（更深，地址更低）
      └─────────────────────┘

       左子树返回后：total = 50 + 30；再调用 tree_sum(0x8C20) 求右子树
```

【与汇编的对应】 完整、逐句对照的 LC-3 翻译（结构偏移：`left = 0`、`value = 1`、`right = 2`）：

```
; ============================================================
; tree_sum (root)
; 参数: root 在 R5+4      返回: 和写入 R5+1 槽
; 局部: R5+0 = total
; ============================================================
TREE_SUM
        ADD   R6,R6,#-4         ; 申请 4 个槽: 3 个 linkage + 1 个局部变量
        STR   R5,R6,#1          ; 保存调用者的帧指针 (R5 = R6, 故写在 R5+1)
        ADD   R5,R6,#0          ; R5 = 局部变量底部
        STR   R7,R5,#2          ; *** 必须先存 R7: 后面的 JSR 会覆盖它 ***
        LDR   R0,R5,#4          ; R0 = root
        BRnp  TS_NONZERO        ; root != NULL ?
        AND   R0,R0,#0          ; 是: 返回 0
        STR   R0,R5,#1          ; 写入返回值槽 (R5+1)
        BRnzp TS_TEARDOWN
TS_NONZERO
        LDR   R1,R0,#1          ; R1 = root->value       (value 在偏移 1)
        STR   R1,R5,#0          ; total = root->value
; ---- total += tree_sum (root->left) ----
        LDR   R0,R0,#0          ; R0 = root->left        (left 在偏移 0)
        ADD   R6,R6,#-1
        STR   R0,R6,#0          ; 压入参数 root->left
        JSR   TREE_SUM          ; 调用 (R7 被覆盖, 但已在 R5+2 中有备份)
        LDR   R1,R6,#0          ; R1 = 返回值 (返回后位于栈顶)
        ADD   R6,R6,#2          ; 弹出返回值与参数
        LDR   R2,R5,#0          ; R2 = total
        ADD   R2,R2,R1
        STR   R2,R5,#0          ; total += 左子树之和
; ---- total += tree_sum (root->right) ----
        LDR   R0,R5,#4          ; 重新取回 root (参数槽一直有效)
        LDR   R0,R0,#2          ; R0 = root->right       (right 在偏移 2)
        ADD   R6,R6,#-1
        STR   R0,R6,#0
        JSR   TREE_SUM
        LDR   R1,R6,#0
        ADD   R6,R6,#2
        LDR   R2,R5,#0
        ADD   R2,R2,R1
        STR   R2,R5,#1          ; 返回值槽 = total
TS_TEARDOWN
        LDR   R7,R5,#2          ; 恢复返回地址 (两次 JSR 已经改过 R7)
        LDR   R5,R5,#1          ; 恢复调用者的帧指针
        ADD   R6,R6,#3          ; 弹掉局部变量与 linkage, 留下返回值在栈顶
        RET
```

对照要点：`p->value` → `LDR R1,R0,#1`；`p = p->left` → `LDR R0,R0,#0`；递归调用 → "压参数 + `JSR` + 读栈顶返回值 + 弹栈"四步；函数结尾统一走 `TS_TEARDOWN`，保证**多条 return 路径共享同一段收尾代码**。

常见错误与调试技巧

- **释放顺序错误**：写成 `free(n); free_tree(n->left);` 会读已释放内存（UB），通常表现为崩溃或"释放了不存在的指针"。 **调试**：`valgrind -q ./prog` 报 "Invalid read of size 8" 并把 `free_tree` 的行号指出来；正确顺序是左、右、自己。
- **BST 不变量被破坏**：只在直接孩子处比较大小（例如插入时写成与祖父比较），结果是 `search` 找不到明明插入过的值。 **调试**：写一个 `check_bst(root)`，用中序遍历检查输出严格递增；或 `gcc -fsanitize=address` 配合断言 `assert(prev < n->value)`。
- **忘记 `malloc` 失败检查**：`n = malloc(...); n->value = ...;` 在内存不足时对 `NULL` 解引用。 **调试**：`gdb -tui --args ./prog` 在 `SIGSEGV` 处 `bt` 看调用栈；养成 `if (NULL == n) { return NULL; }` 并与上层错误路径配合（参考 `read_packed_tree` 的层层回滚写法）。
- **递归没有基例或基例写错**：例如 `inorder` 忘了 `if (NULL == root) return;`，会立即解引用 `NULL` 崩溃；若基例写成 `return` 却漏了空指针检查，则表现为栈溢出。 **调试**：`gdb` 中 `bt 20` 查看重复帧判断递归是否收敛；`ulimit -s` 查看栈上限，用 `valgrind` 观察栈增长。
- **退化树导致栈溢出**：按有序数据插入 10 万个元素，`height = 99999`，递归遍历会耗尽 8 MB 栈。 **调试**：`gdb` 捕获 `SIGSEGV` 后 `bt` 显示成千上万个同名帧；改用迭代遍历（显式栈）或使用平衡树。
- **释放后继续用 `root`**：`free_tree(root); printf("%d\n", root->value);` 是 UB。 **调试**：`valgrind --leak-check=full ./prog` 同时报告泄漏与非法访问；释放后立即把指针置 `NULL` 并只通过一个包装函数访问。
- **忘记保存 R7 就递归（写 LC-3 时）**：第一次 `JSR` 后 `R7` 被覆盖，函数末尾 `RET` 跳回错误地址，程序"乱飞"。 **调试**：在 LC-3 模拟器中单步执行，观察 `RET` 前后 `PC` 是否落在调用点之后；规则是"**只要函数体内有 `JSR`，序言必须 `STR R7,R5,#2`，收尾必须 `LDR R7,R5,#2`**"。

关键要点

- 树的形状由指针字段表达，**地址毫无规律**；"层级"是逻辑关系，不是物理位置。
- BST 的核心是**整棵子树**意义上的有序不变量 `left < node < right`；中序遍历输出升序是它最直接的验证手段。
- 递归与树天生匹配：遍历、计数、求和、求高度、释放都是"递归左右 + 合并结果"的同一模板；**整树释放必须后序**（先孩子后自己）。
- 树高决定一切代价：查找 `O(h)`、递归栈 `O(h)`；有序插入会退化成链（实测 `n = 8` 时 `h = 7`），平衡或随机化才能保住 `O(log n)`。
- 在 LC-3 中，含递归的函数必须做**完整帧管理**：序言保存 `R5` 与 `R7` 并设置 `R5`，收尾用 `LDR R7,R5,#2`、`LDR R5,R5,#1`、`ADD R6,R6,#N` 三步恢复后 `RET`；跨调用需要保留的值一律放进栈帧局部变量。

思考题（带答案）

问题 1：下面这个 `size` 函数能正确统计结点数吗？如果树是退化的（只有左孩子），会发生什么？

```
static int32_t size(node_t* root)
{
    return 1 + size(root->left) + size(root->right);
}
```

答案：不能。它缺少基例 `if (NULL == root) { return 0; }`，一旦走到空子树就会解引用 `NULL` 而崩溃（在 LC-3 中则是 `LDR` 访问地址 0，读到的是系统区内容，行为不可预测）。补上基例后公式 `1 + size(left) + size(right)` 是正确的。退化树不会改变正确性，但会让递归深度等于 `n`，`n` 很大时导致栈溢出。

问题 2：给出一棵插入顺序，使 `50, 30, 70, 20, 40, 60, 80` 这七个值构成的 BST 高度达到 6，并说明为什么中序遍历仍然是升序。

答案：按升序插入 `20, 30, 40, 50, 60, 70, 80`（或降序）即可：每个新值都比上一个大，于是永远挂在右孩子位置，得到一条长为 7 的右链，高度 `= 6`（以边数计；本讲 `bst.c` 用 `1..8` 实测得到 `height = 7, nodes = 8`，规律一致）。中序仍然是升序，因为中序遍历只依赖"左子树 → 根 → 右子树"这一结构顺序，而链状树中每个结点的左子树都为空，访问顺序恰好就是插入时的升序。

问题 3：为什么下面这段 LC-3 序言是错的?

```
TREE_SUM
        ADD   R6,R6,#-4
        STR   R5,R6,#1
        ADD   R5,R6,#0
        ; 直接开始执行函数体，函数体中间有 JSR TREE_SUM
```

答案：缺少 `STR R7,R5,#2`。函数体会执行 `JSR TREE_SUM`，而 `JSR` 把返回地址写入 `R7`，覆盖了本函数进入时的返回地址；由于从未把它保存到栈帧，收尾处的 `LDR R7,R5,#2` 只会读到一个未初始化的槽，`RET` 于是跳到随机地址。规则是：**只要函数是"非叶"的（体内有任何 `JSR` / `JSRR`），序言就必须保存 `R7`，收尾必须恢复 `R7`**。

Lecture 19: 基本排序算法 (Basic Sorting Algorithms)

概述

本讲解决的核心问题是：给定一个数组与一个**序关系 (ordering)**，如何高效地把它重排成有序序列。 我们依次分析**插入排序 (insertion sort)**、**选择排序 (selection sort)**、**冒泡排序 (bubble sort)**、**归并排序 (merge sort)** 与**快速排序 (quicksort)** 的机制、代价与适应场景；随后说明任何基于比较的排序都不可能快过 `Ω(n log n)`，而整数键可用**计数排序/基数排序**突破这一下界。 最后引入**泛型排序 (generic sorting)**：`qsort(base, nmemb, size, compar)` 用**函数指针 (function pointer)** 把"比较规则"从算法中剥离出来，使同一份排序代码能处理 `int`、`double`、结构体与字符串——这正是 Lecture 12 回调思想最著名的应用。

核心概念与底层机制图解

- **排序问题的形式化与四个评价维度**：输入是数组与比较函数，输出是满足序关系的一个排列。
  - *直观解释*：像给一叠考卷按分数排队，规则可以自己定（升序、降序、先按年龄再按姓名）。
  - *底层机制图解*：评价排序算法看四件事：**时间**（最坏/平均/最好）、**空间**（是否原地 in-place）、**稳定性 (stability)**（相等元素是否保持原次序）、**适应性 (adaptivity)**（近乎有序时是否更快）。稳定排序是"多关键字排序"的前提（先按次关键字排，再按主关键字稳定排）。
  - *作用域与存储期*：所有排序都原地修改调用者数组；辅助数组（归并的 `aux`）由算法自己 `malloc`/`free`，属于 allocated storage duration。
- **插入排序 (insertion sort)**：把数组看成"左侧已排序 + 右侧待插入"，每次取出一个元素向左找到位置插入——像整理手中的扑克牌，每抓到一张新牌就插进已排好的牌中。
  - *底层机制图解*：第 `i` 轮把 `a[i]` 暂存为 `key`，从 `i-1` 向左扫描：只要 `a[j] > key` 就右移一格（`a[j+1] = a[j]`），遇到 `a[j] <= key` 即停，把 `key` 写入空位。**关键性质：内层循环会在第一个不大于 `key` 的元素处立即 `break`**。因此当数组已经有序时，每轮只做一次比较，总代价 `O(n)`；一般而言代价为 `O(n + 逆序对数量)`——这就是"插入排序在近乎有序的数据上极快"的根本原因。课程的 `isort.c` 正是这一算法对任意元素类型的泛型版本。
  - *作用域与存储期*：只用 `key` 一个 automatic 变量，空间 `O(1)`；把元素"右移"而不是交换，使**元素移动次数等于逆序对数量**，对"移动代价高的元素"更划算。
- **选择排序 (selection sort)**：第 `i` 轮在 `a[i..n-1]` 中选出最小值并与 `a[i]` 交换——像每次从剩下的考卷里挑出分数最低的那张放到队尾。
  - *底层机制图解*：比较次数恒为 `n(n-1)/2`（实测 `n = 7` 时为 `21`），与输入顺序**完全无关**；交换最多 `n-1` 次，是三种 $O(n^2)$ 算法中写内存最少的，但不是稳定排序。
  - *作用域与存储期*：空间 `O(1)`，只用一个 `min` 下标变量；因为交换次数少，在"写操作昂贵"的存储介质上有历史价值。
- **冒泡排序 (bubble sort) 与提前退出 (early exit)**：相邻两两比较，大的元素像气泡一样浮到末尾——像一排人比身高，相邻两人矮的往前站，一轮下来最高的被推到队尾。
  - *底层机制图解*：第 `i` 轮把第 `i` 大的元素放到 `a[n-1-i]`；用 `swapped` 记录本轮是否发生交换，整轮无交换即有序列退出。**加了提前退出后，有序输入只需 `n-1` 次比较**（实测 `cmp = 6`，与插入排序相同）；退化输入仍是 $O(n^2)$，且每次交换要 3 次赋值，实践中几乎总劣于插入排序。
  - *作用域与存储期*：空间 `O(1)`；`swapped` 是 automatic 变量，生命周期仅限本轮循环。
- **归并排序 (merge sort)**：把数组一分为二、各自递归排序、再把两个有序段**合并 (merge)**。
  - *直观解释*：两叠已经排好的牌，只要反复比较两叠的顶牌、取走较小的那张，就能合并成一叠有序牌。
  - *底层机制图解*：递推式 `T(n) = 2T(n/2) + O(n)`，展开得 `O(n log n)`（每层合并总代价 `O(n)`，共 `log n` 层）。**必须额外一块 `O(n)` 的辅助数组**（本讲 `sorts.c` 中的 `aux`），因为合并无法在原地安全完成；它是**稳定**的（相等元素优先取左段），且对链表特别友好（链表合并无需辅助数组）。比较次数几乎与输入无关（实测 `cmp = 14`）。
  - *作用域与存储期*：递归深度与栈开销均为 `O(log n)`；辅助数组由 `merge_sort` 分配、返回前 `free`。

- **快速排序 (quicksort)、Lomuto 与 Hoare 分区**：选一个**基准 (pivot)**，把数组分成"小于等于 pivot"与"大于 pivot"两段，再递归排序两段——像按身高把队伍分成矮的一队和高的一队，再各自排队。
  - *底层机制图解*：**Lomuto 分区**用 `a[hi]` 作 pivot，`i` 指向"小于等于 pivot 区域的末尾"：`j` 从 `lo` 扫到 `hi-1`，凡 `a[j] <= pivot` 就 `i++` 并与 `a[i]` 交换；最后把 pivot 换到 `i+1` 并返回该下标。**Hoare 分区**取中间元素作 pivot，`i`、`j` 两端相向而行，遇到"左边不小于 pivot、右边不大于 pivot"就交换，直到相遇并返回 `j`；递归区间是 `[lo, j]` 与 `[j+1, hi]`（**不是 `j-1`**，否则死循环）。Lomuto 代码简单；Hoare 交换更少（实测随机输入：Lomuto `cmp=11, swap=9`，Hoare `cmp=32, swap=5`），实践中更常用。
  - *作用域与存储期*：原地排序，空间只有递归栈 `O(log n)`（平均），最坏 `O(n)`。
- **最坏情况与基准选择 (worst case and pivot choice)**：当每次划分都极度不平衡时，快速排序退化为 `O(n²)`。
  - *直观解释*：如果每次挑中的"基准"都恰好是最大或最小的那个，队伍就永远只被分成"一个元素 + 其余全部"。
  - *底层机制图解*：对**已经有序**的输入使用"取末尾元素作 pivot"的 Lomuto，每轮刚好分出 `0` 个和 `n-1` 个元素，`T(n) = T(n-1) + O(n)` 给出 `O(n²)`。实测有序输入 `{1..7}`：Lomuto 需要 `cmp = 21 = n(n-1)/2`、`swap = 27`，而 Hoare（中间 pivot）`cmp = 26`、`swap = 0`；`n` 越大差距越是 `n²` 与 `n log n` 之别。三种常用对策：随机化 pivot、取"首/中/尾中位数"、小数组改用插入排序（cutoff）。
  - *作用域与存储期*：全部元素相等时 Hoare 仍把数组分成两半，避免 Lomuto 退化。
- **五种排序的对比 (comparison table)**：

算法	最好	平均	最坏	额外空间	稳定	适应性	`n=7` 实测比较/交换
插入排序 insertion	`O(n)`	`O(n²)`	`O(n²)`	`O(1)`	是	**强**（逆序对相关）	`15 / 17`
选择排序 selection	`O(n²)`	`O(n²)`	`O(n²)`	`O(1)`，写最少	否	无	`21 / 5`
冒泡排序 bubble	`O(n)`	`O(n²)`	`O(n²)`	`O(1)`	是	有（提前退出）	`20 / 11`
归并排序 merge	`O(n log n)`	`O(n log n)`	`O(n log n)`	`O(n)`	是	弱	`14 / 20`
快速排序 quicksort	`O(n log n)`	`O(n log n)`	`O(n²)`	`O(log n)` 栈	否	弱	`11 / 9`（Lomuto）

- **比较排序的下界 `Ω(n log n)`**：任何只通过"比较两个元素"获取信息的排序算法，最坏情况至少需要 `Ω(n log n)` 次比较。
  - *直观解释*：`n` 个元素共有 `n!` 种排列，算法必须能把它们全部区分开；每次比较只能得到"是/否"两种回答，因此至少需要 `log2(n!)` 个问题。

- *底层机制图解*：把算法执行过程画成一棵**决策树 (decision tree)**：内部结点是比较、叶子是输出排列。`n` 个元素有 `n!` 种可能排列，所以叶子数 ⩾ `n!`，高度 ⩾ `log2(n!)`。用 Stirling 近似 `log2(n!) ≈ n log2 n − 1.44n`，因此下界是 `Ω(n log n)`。具体数字：`n = 10^6` 时 `log2(n!) ≈ 1.9 × 10^7`，而冒泡排序的 `n(n−1)/2 ≈ 5 × 10^11` 次比较——相差四万倍。
- *作用域与存储期*：这条下界只约束**基于比较**的算法；不比较元素值（而是直接利用键的整数值）的算法不受它限制。

- **非比较排序 (non-comparison sorts)**：计数排序与基数排序利用"键是小范围整数"这一额外信息——把分数为 `k` 的考卷直接丢进第 `k` 号箱子，最后从 0 号箱子依次取走，不用互相比较。
  - *底层机制图解*：**计数排序 (counting sort)** 用大小为 `k`（键的取值范围）的计数数组，时间 `O(n + k)`、空间 `O(n + k)`，且稳定（关键是**从后往前**回写）；**基数排序 (radix sort)** 对 `d` 位数字做 `d` 轮稳定排序，时间 `O(d(n + k))`。代价是它们**只适用于整数键**，`k` 很大时空间爆炸。
  - *作用域与存储期*：计数/输出数组都是 allocated storage duration，函数返回前必须 `free`。
- **泛型排序与函数指针 (generic sorting and function pointers)**：把"如何比较"外置为一个回调函数，排序算法只依赖这个回调。
  - *直观解释*：像一台"通用分拣机"：机器只负责搬动与比较，至于"A 是否应该排在 B 前面"由你插入的规则卡片决定。
  - *底层机制图解*：`void qsort(void* base, size_t nmemb, size_t size, int (*compar)(const void*, const void*));`。`base` 是首地址、`nmemb` 是元素个数、`size` 是每元素的**字节数**，三者合起来让 `qsort` 能做通用指针算术：第 `i` 个元素地址是 `(char*)base + i * size`（必须先转 `char*`，因为 `void*` 不能做算术）。`compar` 必须返回**三路结果**（负/零/正）。这就是 Lecture 12 的回调：`qsort` 里写着 `(*compar)(p1, p2)`，具体调用谁要到运行时才知道（汇编层是一次 `JSRR`）。
  - *作用域与存储期*：比较函数通常是文件作用域的 `static` 函数，具有 static storage duration，其**地址**（函数指针值）在整个程序运行期间不变。

代码示例与底层机制分析

示例 1：五种排序算法在同一输入上的比较与交换次数

代码 (C) — `/tmp/ece220_l19/sorts.c`（节选算法本体；完整文件含计数与打印，已编译运行）：

```
static void insertion_sort(int32_t a[], int32_t n)
{
    int32_t i, j, key;
    for (i = 1; n > i; i++) {
        key = a[i];
        for (j = i - 1; 0 <= j; j--) {
            cmp_count++;
            if (a[j] <= key) {
                break;                  /* 已有序时立刻退出，这就是适应性 */
            }
            a[j + 1] = a[j];            /* 右移一格，而不是交换 */
            swap_count++;
        }
        a[j + 1] = key;
        swap_count++;
    }
}

static void selection_sort(int32_t a[], int32_t n)
{
    int32_t i, j, min;
    for (i = 0; n - 1 > i; i++) {
        min = i;
        for (j = i + 1; n > j; j++) {
            cmp_count++;
            if (a[j] < a[min]) {
                min = j;
            }
        }
        if (min != i) {
            swap(&a[i], &a[min]);
        }
    }
}

static void bubble_sort(int32_t a[], int32_t n)
{
    int32_t i, j, swapped;
    for (i = 0; n - 1 > i; i++) {
        swapped = 0;
        for (j = 0; n - 1 - i > j; j++) {
            cmp_count++;
            if (a[j] > a[j + 1]) {
                swap(&a[j], &a[j + 1]);
                swapped = 1;
            }
        }
        if (!swapped) {
            break;                      /* 提前退出：本轮无交换 => 已有序 */
        }
    }
}
```

```
static void merge(int32_t a[], int32_t lo, int32_t mid, int32_t hi, int32_t aux[])
{
    int32_t i = lo, j = mid + 1, k;
    for (k = lo; hi >= k; k++) {
        aux[k] = a[k];                  /* 先把整段拷进辅助数组 */
    }
    for (k = lo; hi >= k; k++) {
        if (i > mid) {
            a[k] = aux[j++];            /* 左半用尽 */
        } else if (j > hi) {
            a[k] = aux[i++];            /* 右半用尽 */
        } else {
            cmp_count++;
            if (aux[j] < aux[i]) {      /* 严格小于才取右边 => 稳定 */
                a[k] = aux[j++];
            } else {
                a[k] = aux[i++];
            }
        }
        swap_count++;
    }
}

static int32_t partition_lomuto(int32_t a[], int32_t lo, int32_t hi)
{
    int32_t pivot = a[hi], i = lo - 1, j;
    for (j = lo; hi > j; j++) {
        cmp_count++;
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);            /* pivot 归位 */
    return i + 1;
}

static int32_t partition_hoare(int32_t a[], int32_t lo, int32_t hi)
{
    int32_t pivot = a[lo + (hi - lo) / 2], i = lo - 1, j = hi + 1;
    for (;;) {
        do {
            i++;
            cmp_count++;
        } while (a[i] < pivot);
        do {
            j--;
            cmp_count++;
        } while (a[j] > pivot);
        if (i >= j) {
            return j;                   /* 返回 j，递归 [lo,j] 与 [j+1,hi] */
        }
        swap(&a[i], &a[j]);
```

```
    }
}
```

验证到的真实输出（输入 `{38, 27, 43, 3, 9, 82, 10}`，`gcc -g -std=c99 -Wall -Werror sorts.c -o sorts && ./sorts`）：

```
insertion sort            :   3   9  10  27  38  43  82   cmp=15 swap=17
selection sort            :   3   9  10  27  38  43  82   cmp=21 swap= 5
bubble sort               :   3   9  10  27  38  43  82   cmp=20 swap=11
merge sort                :   3   9  10  27  38  43  82   cmp=14 swap=20
quicksort (Lomuto)        :   3   9  10  27  38  43  82   cmp=11 swap= 9
quicksort (Hoare)         :   3   9  10  27  38  43  82   cmp=32 swap= 5

sorted input {3,9,10,27,38,43,82}:
  insertion               :   3   9  10  27  38  43  82   cmp= 6 swap= 6
  bubble                  :   3   9  10  27  38  43  82   cmp= 6 swap= 0
  selection               :   3   9  10  27  38  43  82   cmp=21 swap= 0
  quicksort Lomuto        :   3   9  10  27  38  43  82   cmp=21 swap=27

nearly sorted {3,9,10,27,38,43,82} with 82 moved to front:
  insertion               :   3   9  10  27  38  43  82   cmp=11 swap=12
  bubble                  :   3   9  10  27  38  43  82   cmp=11 swap= 6
  selection               :   3   9  10  27  38  43  82   cmp=21 swap= 6

worst case for quicksort, sorted input, n = 7:
  Lomuto, last pivot      :   1   2   3   4   5   6   7   cmp=21 swap=27
  Hoare, middle pivot     :   1   2   3   4   5   6   7   cmp=26 swap= 0
```

【代码做什么？】 1. 六种实现各自对同一乱序数组排序，输出全部为 `3 9 10 27 38 43 82`；右侧是比较次数与交换/移动次数。 2. 随机输入下：插入 `15/17`，选择 `21/5`（比较恒为 `n(n-1)/2`），冒泡 `20/11`，归并 `14/20`，Lomuto 快排 `11/9`（比较最少），Hoare 快排 `32/5`（交换最少）。 3. 有序输入下：插入排序只需 `6` 次比较（每轮一次 `break`），冒泡排序也只需 `6` 次（第一轮无交换即退出），而选择排序仍然是 `21` ——**它完全不具备适应性**。 4. 近乎有序输入（`82` 移到最前，只有 6 个逆序对）下插入排序 `cmp = 11`，少于随机输入的 `15`；有序输入下 Lomuto 快排的比较次数涨到 `21 = n(n-1)/2`，正是最坏情况的开端，而 Hoare 版（中间 pivot）`swap = 0`，没有退化成"1 和 n-1"的灾难。

【底层机制透视】 插入排序的内层循环 `break` 让它具有**输入敏感性**：运行时间是 `O(n + I)`，`I` 为逆序对个数。选择排序的内层循环没有 `break`，无论输入如何都跑满 `n(n-1)/2` 次，因此它是"**输入无关**"的——这在需要可预测延迟的实时系统里反而是优点。 归并排序的 `swap_count` 高（`20`）是因为**每一次写回都算一次移动**，它的比较与移动次数都是 `O(n log n)` 量级且常数稳定，这也是它在"外部排序"与"链表排序"中不可替代的原因。 `partition_hoare` 的 `do { i++; cmp_count++; } while (a[i] < pivot);` 依赖"pivot 一定在区间内"这一事实，因此**不会越界**；而 `partition_lomuto` 依赖 `a[hi]` 本身是 pivot，所以 `j` 只扫到 `hi-1`。 两个 partition 都通过 `swap` 修改变量：由于 `int32_t` 大小固定，这里直接用了一个 `int32_t` 临时变量；泛型版本则必须用 `memcpy`（见示例 4）。

【内存布局图解】（插入排序第 `i` 轮，`i = 4`、`key = a[4] = 9`）

```
   a[0] a[1] a[2] a[3] a[4] a[5] a[6]      i = 4, key = a[4] = 9
  +----+----+----+----+----+----+----+
  |  3 | 27 | 38 | 43 |  9 | 82 | 10 |   排序前：已排序区 a[0..3]，key 暂存于变量
  |  3 | 27 |  9 | 38 | 43 | 82 | 10 |   ① 43、38、27 依次右移（9 比它们都小）
  |  3 |  9 | 27 | 38 | 43 | 82 | 10 |   ② a[0]=3 <= 9，break；key 写入空位
  +----+----+----+----+----+----+----+
     ^ 已排序区扩展为 a[0..4]；本轮移动 I_i 个元素，总计 I = 逆序对数量
```

【与汇编的对应】（`a` 基址由 `LEA` 取得，元素偏移按字计算）

```
; ---- 插入排序内层：while (j >= 0 && a[j] > key) { a[j+1] = a[j]; j--; } ----
; R1 = key, R2 = &a[j]（进入循环前由 ADD R2,R3,R2 算好，循环内自减）, R3 = 数组基址, R0 = 临时量
; 说明：临时量只能取自 R0~R3，而 R0 要留给"比较/搬运"用，所以下标 j 不再单独占
;       一个寄存器（那就得动用 R4，违反约定）；循环用指针 R2 自减，边界判断改成
;       "指针是否已越过数组基址"，与 C 里的 j >= 0 完全等价
INS_IN  NOT   R0,R3
        ADD   R0,R0,#1
        ADD   R0,R2,R0        ; R0 = &a[j] - 数组基址 = j
        BRn   INS_PLACE       ; j < 0，停
        LDR   R0,R2,#0        ; R0 = a[j]
        NOT   R0,R0
        ADD   R0,R0,#1
        ADD   R0,R0,R1        ; R0 = key - a[j]
        BRzp  INS_PLACE       ; key >= a[j] ? 停（适应性：有序时立即退出）
        LDR   R0,R2,#0
        STR   R0,R2,#1        ; a[j+1] = a[j]
        ADD   R2,R2,#-1       ; &a[j]--（等价于 j--）
        BRnzp INS_IN
INS_PLACE
        ADD   R2,R2,#1
        STR   R1,R2,#0        ; a[j+1] = key
```

示例 2：Lomuto 分区的逐步过程

代码 (C) — `/tmp/ece220_l19/partition_trace.c`（核心函数，带 `verbose` 开关打印每一轮）：

```
static int32_t partition_lomuto(int32_t a[], int32_t lo, int32_t hi, int32_t verbose)
{
    int32_t pivot = a[hi];
    int32_t i = lo - 1;
    int32_t j;

    for (j = lo; hi > j; j++) {
        comparisons++;
        if (a[j] <= pivot) {
            i++;
            if (i != j) {
                swap(&a[i], &a[j]);
            }
        }
    }
    swap(&a[i + 1], &a[hi]);
    return i + 1;
}
```

验证到的真实输出（输入 `{7, 2, 1, 6, 8, 5, 3, 4}`）：

```
Lomuto trace:
  partition(lo=0, hi=7), pivot = a[7] = 4
    j=0: a[0]= 7 >   4, keep      ->  7  2  1  6  8  5  3  4
    j=1: a[1]= 2 <=  4, swap i=0  ->  2  7  1  6  8  5  3  4
    j=2: a[2]= 1 <=  4, swap i=1  ->  2  1  7  6  8  5  3  4
    j=3: a[3]= 6 >   4, keep      ->  2  1  7  6  8  5  3  4
    j=4: a[4]= 8 >   4, keep      ->  2  1  7  6  8  5  3  4
    j=5: a[5]= 5 >   4, keep      ->  2  1  7  6  8  5  3  4
    j=6: a[6]= 3 <=  4, swap i=2  ->  2  1  3  6  8  5  7  4
    place pivot at index 3
  after partition:  2  1  3  4  8  5  7  6   (pivot 4 fixed at index 3)
  partition(lo=0, hi=2), pivot = 3:  ->  2  1  3 | 4  8  5  7  6   (pivot 3 fixed at index 2
  partition(lo=0, hi=1), pivot = 1:  ->  1  2  3 | 4  8  5  7  6   (pivot 1 fixed at index 0
  partition(lo=4, hi=7), pivot = 6:  ->  1  2  3  4  5  6  7  8   (pivot 6 fixed at index 5)
  partition(lo=6, hi=7), pivot = 8:  ->  1  2  3  4  5  6  7  8   (pivot 8 fixed at index 7)
sorted               :  1  2  3  4  5  6  7  8
comparisons = 14, swaps = 9
```

【代码做什么？】 1. 第一次分区取 `a[7] = 4` 作 pivot，`i` 从 `-1` 开始；`j` 从 0 扫到 6，凡 `a[j] <= 4` 就把 `i` 前移并把该元素换到"小值区"末尾。 2. 扫描结束时数组为 `2 1 3 6 8 5 7 4`、`i = 2`；把 pivot 与 `a[i+1]` 交换得 `2 1 3 4 8 5 7 6`，**pivot 4 从此固定在下标 3**。 3. 递归处理左段 `[0,2]`（pivot 3）与右段 `[4,7]`（pivot 6），各自重复上述过程；每完成一次分区至少有一个元素归位，最终得到 `1 2 3 4 5 6 7 8`，全程 `comparisons = 14, swaps = 9`（关闭打印后重跑计数完全相同）。

【底层机制透视】 `i` 的语义是"小于等于 pivot 区域的最后一个下标"，循环不变式为 `a[lo..i] <= pivot`、`a[i+1..j-1] > pivot`、`a[j..hi-1]` 未检查、`a[hi] == pivot`——理解它就理解了 Lomuto 为什么正确。 分区是**原地**的，只用三个自动变量；每次分区结束时 pivot 落在最终位置，因此快速排序不需要"合并"步骤，这是它与归并排序最根本的结构差异（`quicksort` 先分后不管，`mergesort` 先递归后合

并)。从 trace 看每层比较次数为 7、2、1、3、1，总计 14 次，远小于 `n(n-1)/2 = 28`——因为选到的 pivot 接近中位数。

【内存布局图解】(第一次分区结束时)

```
下标:  0    1    2    3    4    5    6    7
     +----+----+----+----+----+----+----+----+
     |  2 |  1 |  3 |  4 |  8 |  5 |  7 |  6 |
     +----+----+----+----+----+----+----+----+
       <-- <= pivot -->  ^   <-- 未处理, > pivot -->
                         pivot 4 已归位 (下标 3, 永不再移动)

调用树:  [0..7] p=4 -> 分割点 3
          /                      \
    [0..2] p=3 -> 分割点 2    [4..7] p=6 -> 分割点 5
     /      \                  /      \
  [0..1]  空区间            空区间     [6..7] p=8 -> 分割点 7
   p=1 -> 分割点 0
```

【与汇编的对应】(Lomuto 内层循环；`a` 基址在 R3，`hi` 在 R5+5)

```
; ---- for (j = lo; j < hi; j++) if (a[j] <= pivot) { i++; swap(a[i], a[j]); } ----
; R1 = pivot, R2 = j, R3 = 数组基址, R0 = 临时量
; i 是本帧的局部变量, 按帧约定放在 R5+0: 通用寄存器只有 R0~R3 可当临时量,
; 而 pivot、j、数组基址与每次比较要用的临时量已经占满这四个, i 只能进帧
LOM_LOOP
        LDR   R0,R5,#5        ; R0 = hi (hi 是本帧参数)
        NOT   R0,R0
        ADD   R0,R0,#1
        ADD   R0,R0,R2        ; R0 = j - hi
        BRzp  LOM_END         ; j >= hi, 结束本轮分区
        ADD   R0,R3,R2
        LDR   R0,R0,#0        ; R0 = a[j]
        NOT   R0,R0
        ADD   R0,R0,#1
        ADD   R0,R0,R1        ; R0 = pivot - a[j]
        BRn   LOM_NEXT        ; a[j] > pivot, 跳过
        LDR   R0,R5,#0        ; R0 = i (i 在帧内, R5+0)
        ADD   R0,R0,#1        ; i++
        STR   R0,R5,#0        ; 写回 i; 随后用三次 LDR/STR 交换 a[i] 与 a[j] (i 从 R5+0 读回)
LOM_NEXT
        ADD   R2,R2,#1        ; j++
        BRnzp LOM_LOOP
LOM_END
        ; 最后 swap(a[i+1], a[hi]) 让 pivot 归位, 并返回 i+1 作为分割点
```

示例 3：`qsort` 与三路比较函数

代码 (C) — `/tmp/ece220_l19/qsort_demo.c` (节选)：

```
typedef struct player_t {
    char name[16];
    int32_t age;
    int32_t games;
} player_t;

/* 三路比较：负数表示 a < b，0 表示等价，正数表示 a > b */
static int cmp_int(const void* p1, const void* p2)
{
    const int32_t* a = p1;
    const int32_t* b = p2;

    if (*a < *b) {
        return -1;
    }
    if (*a > *b) {
        return 1;
    }
    return 0;
}

static int cmp_age(const void* p1, const void* p2)
{
    const player_t* a = p1;
    const player_t* b = p2;

    if (a->age < b->age) {
        return -1;
    }
    if (a->age > b->age) {
        return 1;
    }
    return 0;
}

static int cmp_age_then_name(const void* p1, const void* p2)   /* 多关键字 */
{
    const player_t* a = p1;
    const player_t* b = p2;
    int by_age = cmp_age(p1, p2);

    if (0 != by_age) {
        return by_age;
    }
    return strcmp(a->name, b->name);
}

/* 降序只需把两个操作数对调：cmp_games_desc 与 cmp_age 结构相同，
   只是把 a->age/b->age 换成 b->games/a->games */

/* qsort 传给比较函数的是"指向数组元素的指针"，元素本身是 char*，所以要再解引用一次 */
static int cmp_cstring(const void* p1, const void* p2)
{
```

```
    const char* const* s1 = p1;
    const char* const* s2 = p2;

    return strcmp(*s1, *s2);
}
```

验证到的真实输出：

```
integers before: 42 -7 42 0 13 -7 100 5
integers after : -7 -7 0 5 13 42 42 100

strings before: pear Apple fig apple banana Fig
strings after : Apple Fig apple banana fig pear

sorted by age:            Ed(19) Bo(25) Al(25) Di(25) Cy(31)   <- 25 岁三人次序未定义

sorted by age, then by name:
   Ed       age=19 games=55
   Al       age=25 games=12
   Bo       age=25 games=40
   Di       age=25 games=90
   Cy       age=31 games= 7

sorted by name:           Al Bo Cy Di Ed（按姓名字符串次序）

sorted by games, descending:
   Di       age=25 games=90
   Ed       age=19 games=55
   Bo       age=25 games=40
   Al       age=25 games=12
   Cy       age=31 games= 7
```

【代码做什么？】 1. `qsort(nums, 8, sizeof(nums[0]), cmp_int)` 把含重复值的整数数组排成 `-7 -7 0 5 13 42 42 100`，注意两个 `42` 的相对次序未定义（`qsort` **不保证稳定**）。 2. 字符串数组用 `cmp_cstring` 排成 `Apple Fig apple banana fig pear`——这是**ASCII 次序**（大写字母 `A`–`Z` 是 65–90，小写是 97–122），因此 `Apple` 与 `Fig` 排在小写单词之前，与"字典序（忽略大小写）"不同。 3. 同一组 `player_t` 数据用四个不同的比较函数各排一次：按年龄、按年龄再按姓名、按姓名、按场数降序；**排序代码一次都没改，改的只是比较函数**。"按年龄"与"按年龄再按姓名"的 25 岁组次序不同（`Bo, Al, Di` vs `Al, Bo, Di`），这是多关键字比较起作用的证据。

【底层机制透视】 `qsort` 只认识 `void*` 与字节大小，因此它内部的元素地址计算必然是 `(char*)base + i * size`；把 `void*` 转成 `char*` 是必须的，因为 `void*` 上的指针算术在 C 标准中没有定义。`int32_t` 的 `size = 4`、`player_t` 的 `size = 24`（16 字节 name + 4 age + 4 games），`qsort` 用同一段 `memcpy` 搬运逻辑处理二者。 比较函数的参数是**指向元素的指针**，不是元素本身。对 `int32_t nums[]`，元素是 `int32_t`，所以参数是 `int32_t*`；对 `char* words[]`，元素是 `char*`，所以参数是 `char**`——这就是 `cmp_cstring` 必须写两层解引用 `strcmp(*s1, *s2)` 的原因，也是学生最常犯的错误。 比较函数**必须**给出三路结果：写成 `return *a - *b;` 会在接近 `INT32_MIN/MAX` 时**有符号整数溢出**（UB），正确写法是显式 `<`、`>` 分支；返回值的大小无关紧要，只有符号有意义。在机器层，`qsort` 里的 `(*compar)(p1, p2)` 是

一次间接调用（LC-3 的 `JSRR`），使同一段排序代码获得"多态"能力——与 Lecture 12 的 I/O 通道函数指针表、示例 4 的 `is_smaller` 完全同源。

【内存布局图解】（`player_t team[5]` 与 `qsort` 眼中的它）

```
team（栈上 5 × 24 = 120 字节）
+----------------+----------------+----------------+
| name[0..15]    | age（4 字节）   | games（4 字节）|   每个元素 24 字节
+----------------+----------------+----------------+
^
第 0 个元素 = cmp_age 的 p1：qsort 传的是"指向这 24 字节的指针"，
函数内转成 player_t* 后按偏移访问 age；调用链 qsort -> (*compar) -> cmp_age
```

【与汇编的对应】（`(*compar)(p1, p2)` 的一次间接调用）

```
; ---- qsort 内部：p1 在 R5+4，p2 在 R5+5，compar 在 R5+6 ----
        LDR   R0,R5,#6        ; R0 = 函数指针（被调用函数的第一条指令地址）
        ADD   R6,R6,#-1
        LDR   R1,R5,#5
        STR   R1,R6,#0        ; 参数从右向左压栈：先 p2
        ADD   R6,R6,#-1
        LDR   R1,R5,#4
        STR   R1,R6,#0        ; 再 p1
        JSRR  R0              ; *** 间接调用：目标地址来自变量（多态）***
        LDR   R1,R6,#0        ; R1 = 比较结果
        ADD   R6,R6,#3        ; 弹出返回值与两个参数
        ADD   R1,R1,#0
        BRnz  Q_NO_SWAP       ; compar <= 0 已就绪，不交换
```

示例 4：课程的泛型插入排序 `isort.c`

代码 (C) — 课程公开源码 `Ccode/isort.c`（核心函数；为符合 ECE 220 编码规范，原文件的制表符缩进已规范化为 4 空格，代码逻辑逐字未改）：

```
static int32_t
isort (void* base, int32_t n_elts, size_t size,
       int32_t (*is_smaller) (void* t1, void* t2))
{
    char*   array = base;   /* array pointer (used for pointer arithmetic) */
    void*   current;        /* current element being placed into sorted subarray */
    int32_t sorted;         /* outer loop index; number of elements sorted */
    int32_t index;          /* inner loop index for placing current element */

    if (NULL == (current = malloc (size))) {
        return 0;
    }
    for (sorted = 2; n_elts >= sorted; sorted++) {
        memcpy (current, array + (sorted - 1) * size, size);
        for (index = sorted - 1; 0 < index; index--) {
            if ((*is_smaller) (current, array + (index - 1) * size)) {
                memcpy (array + index * size,
                        array + (index - 1) * size, size);
            } else {
                break;
            }
        }
        memcpy (array + index * size, current, size);
    }
    free (current);
    return 1;
}
```

验证到的真实输出（`gcc -g -std=c99 -Wall -Werror isort.c -o isort && ./isort`）：

```
integer sort -> 1: -10 22 30 50 73 99 104
double sort -> 1: -222.0000 -17.0000 3.1415 5.0000 33.0000 39.0000 60.0000 109.0000
string sort -> 1: ASCII Be Please alphabetical in instead. list not of order order. sort sur
```

【代码做什么？】 1. 启动时 `malloc(size)` 申请**恰好一个元素**大小的临时空间 `current`——这是泛型排序无法把元素放进 `int32_t` 变量时的通用解法；分配失败返回 0。 2. 外层 `for (sorted = 2; n_elts >= sorted; sorted++)` 把已排序区从长度 1 逐步扩展到 `n`。 3. 每轮用 `memcpy` 把 `array[(sorted-1)*size]` 拷进 `current`，然后内层从右向左比较：若 `current` 更小就把左边的元素整体右移 `size` 字节。 4. 一旦 `is_smaller` 返回假（`current` 不再更小）就 `break`，把 `current` 拷回空位；这正是插入排序的适应性来源。 5. `main` 用同一个 `isort` 分别排序 `int32_t`、`double`、`char*` 三种数组，打印结果与返回值 1（成功）。

【底层机制透视】 `char* array = base;` 是泛型指针算术的**关键字**：`void*` 不能做算术，转成 `char*` 后 `array + index * size` 就精确表示"第 index 个元素的起始字节地址"。若误写成 `int32_t*`，则 `+ index * size` 会再乘 4 倍，导致越界写。 元素搬运用 `memcpy(dest, src, size)` 而不是赋值：编译期不知道类型，只能按字节复制。`memcpy` 要求源与目标**不重叠**——本例右移时目标比源高恰好 `size`，属于相邻不重叠，安全；若整段搬移可能重叠则必须用 `memmove`。 `is_smaller` 的签名是 `int32_t (*)(void*,`

`void*)`，**非零表示第一个更小**——注意它与 `qsort` 的三路 `compar` 不同：`isort` 只需二路的"小于"判断（`return ((*int1) < (*int2));`）。字符串版本 `string_is_smaller` 的参数是 `char**`（`strcmp(*s1, *s2) < 0`），输出 `ASCII Be Please alphabetical ...` 是 `strcmp` 的 ASCII 次序（大写在前）而非字母表次序。

【内存布局图解】（`isort` 处理 `int32_t` 数组时的字节视图）

```
调用者数组 base（栈上）                          isort 的 current（堆上，size 字节）
+----+----+----+----+----+----+----+          +----+
| 38 | 27 | 43 |  3 |  9 | 82 | 10 |          | 27 |  <- 当前要插入的元素
+----+----+----+----+----+----+----+          +----+
 ^              ^                             ^
 base + 0*size  base + 2*size（char* 算术）     malloc(size) 返回的独立块
 memcpy(array + index*size, array + (index-1)*size, size)：每次搬动一个元素
```

【与汇编的对应】（元素地址计算与 `memcpy` 调用）

```
; ---- 泛型元素地址计算：array + index * size，必须按字节加 ----
; R1 = array（char* 基址）, R2 = index, R3 = size
        ADD   R0,R2,#0        ; R0 = index
        JSR   MULT            ; R0 = index * size（LC-3 无乘法指令，用子程序）
        ADD   R0,R1,R0        ; R0 = (char*)base + index * size
        ; 把 R0 作为参数压栈后调用 MEMCPY，即可复制任意 size 字节的元素
```

常见错误与调试技巧

- **比较函数参数类型写错**：把 `char*` 数组的比较函数写成 `const char*` 而非 `const char**`，导致 `strcmp` 把字符串内容当成指针使用，段错误。**调试**：`gcc -std=c99 -Wall -Werror` 会给出 incompatible pointer type 警告；GDB 中 `p *(char**)p1` 与 `p (char*)p1` 对比看哪个是合法地址。
- **比较函数返回差值导致溢出**：`return *a - *b;` 在 `int32_t` 极值附近溢出（UB），排序结果错乱。**调试**：`gcc -fsanitize=signed-integer-overflow -g`（或 `-ftrapv`）在溢出处中止；改用显式 `<` / `>` 分支返回 `-1/0/1`。
- **分区边界写错造成死循环 / 辅助数组未检查**：Hoare 分区若递归写成 `quick(a, lo, p - 1);`，当 `p == lo` 时左半区间不缩小而死循环；归并排序的 `aux = malloc(...)` 失败后直接使用则对 `NULL` 解引用。**调试**：前者用 `gdb` 的 `Ctrl-C` + `bt` 看递归深度爆炸，并打印 `lo/hi/p` 确认每次分割至少缩减一个元素（Hoare 必须递归 `[lo,p]` 与 `[p+1,hi]`）；后者用 `valgrind -q ./prog` 看 "Invalid write"。
- **`memcpy` 处理重叠区域 / 递归爆栈**：泛型排序中整段搬移若源与目标重叠必须改用 `memmove`；快速排序对有序输入的最坏递归深度为 `O(n)`，会耗尽 8 MB 栈。**调试**：前者用 `valgrind -q ./prog` 看 "overlapping" 或用 `x/8xb array` 逐字节核对；后者用 `ulimit -s` 查栈上限、`gdb` 捕获 `SIGSEGV` 后 `bt` 看是否成千上万个同名帧，再用随机/中位数 pivot 降低深度。
- **误以为 `qsort` 稳定**：想用"先按次关键字排、再按主关键字排"实现多关键字排序时，若 `qsort` 不稳定，结果是错的（本讲示例中两个 `42` 的次序确实未定义）。**调试**：改用稳定的归并排序，或用"比较函数里直接比较全部关键字"（如 `cmp_age_then_name`）——这是最实用的修正。

关键要点

- `O(n²)` 三种算法里，插入排序对近乎有序输入最好（代价 `O(n + 逆序对数)`），选择排序不具备适应性但写内存最少，冒泡排序加提前退出后最好情况可到 `O(n)`。
- 归并排序以 `O(n)` 辅助空间换取最坏情况 `O(n log n)` 与**稳定性**；快速排序原地、常数小、平均最快，但最坏 `O(n²)`，必须靠随机化或中位数 pivot 规避。比较排序的下界 `Ω(n log n)` 来自决策树叶子数 `n!`；计数/基数排序用"键可枚举"这一额外信息突破下界，但只适用于整数键。
- 泛型排序是把比较规则外置为**函数指针回调**：`qsort(base, nmemb, size, compar)` 用 `(char*)base + i * size` 做通用指针算术，比较函数必须返回三路结果；其参数是"指向元素的指针"（`int` 数组给 `int*`，`char*` 数组给 `char**`），这一层解引用是最常见的错误来源。

思考题（带答案）

问题 1：为什么插入排序对"近乎有序"的数组极快，而选择排序无论输入如何都同样慢？请用比较次数说明。

答案：插入排序第 `i` 轮把 `key` 与左侧元素比较，遇到第一个不大于它的元素就 `break`，因此总比较次数为 `n - 1 + I`（`I` 为逆序对个数）；近乎有序时 `I` 很小，总代价接近 `O(n)`（实测 7 个元素的有序输入只用 `6` 次比较）。选择排序第 `i` 轮必须扫描 `a[i+1..n-1]` 找最小值，没有提前退出的依据，比较次数恒为 `n(n-1)/2`（实测恒为 `21`）。

问题 2：下面的比较函数在什么输入下会出错？如何修正？

```
static int cmp(const void* p1, const void* p2)
{
    const int32_t* a = p1;
    const int32_t* b = p2;

    return *a - *b;
}
```

答案：当 `*a - *b` 超出 `int32_t` 范围时发生**有符号整数溢出**（UB）：例如 `*a = 2000000000`、`*b = -2000000000` 的差值为 `4 × 10^9`，超出 `INT32_MAX`，可能回绕成负数使排序颠倒。修正为显式三分支：`if (*a < *b) { return -1; } if (*a > *b) { return 1; } return 0;`。

Lecture 20: 面向对象编程概念及其在 C/C++ 中的实现 (Object-Oriented Concepts: Information Hiding and Encapsulation in C, and the Move to C++)

概述

本讲回答一个看似矛盾的问题：**面向对象 (object-oriented) 的四个核心思想——封装、信息隐藏、继承、多态——需要语言提供什么？** 答案是几乎不需要：它们是**设计思想**，在纯 C 里用「结构体 + 首参数为句柄的函

数 + 文件作用域 + 函数指针表」就能完整表达，而这正是你写过的 `mem220`、`stack`、`player` 模块在做的事。本讲先在 C 里把这些思想手工搭出来，再展示 C++ 编译器如何把同一套机制**自动化**：类与成员函数（隐含的 `this`）、访问控制、构造/析构与 RAII、`new` / `delete`、引用、运算符重载、虚函数与编译器生成的虚函数表 (vtable)。抓住「C++ 虚函数就是 C 的函数指针表，只是表由编译器代填」这一点，就把上一讲的函数指针与回调、ECE 120 的 LC-3 `JSRR` 间接跳转、以及后续的容器与迭代器缝成了同一条自底向上的线索。

核心概念与底层机制图解

- **封装 (Encapsulation)**：把**数据**与**作用于它的函数**捆绑为一个单元，只通过公开接口访问数据。*直观解释*：自动售货机——你只按面板按钮（接口），不拆开内部齿轮（数据）。*底层机制*：C 里的「捆绑」就是**一个结构体 + 一组首参数为该结构体指针的函数**；C++ 成员函数编译后的签名恰好就是这个形式。*作用域与存储期*：数据是 automatic（栈上 `struct`）或 dynamic（`malloc` / `new`），接口函数在 `.text`，存储期为整个程序。
- **信息隐藏 (Information Hiding)**：模块只暴露接口，隐藏「用什么数据结构实现」(David Parnas, 1972)。*直观解释*：餐厅菜单——你点「宫保鸡丁」，不必知道厨房用什么锅；厨房换设备（改数据结构）不影响菜单（接口）。*底层机制*：C 中**作用域就等于访问权限**——编译器看得见的名字你就能改；因此隐藏的唯一手段是**文件作用域**，把结构体定义、`static` 变量与函数全放进 `.c`，头文件只留**不完整类型 (incomplete type)** 与函数原型。*作用域与存储期*：对象状态在 `.c` 的 `static` 变量（static storage duration）或堆上（dynamic），外部只能透过句柄 (handle) 间接访问。
- **继承 (Inheritance)**：**数据继承**指子类型拥有父类型的全部字段，**函数继承**指能作用于父类型的函数也能作用于子类型；父类型称基类 (base class)，子类型称派生类 (derived class)。*直观解释*：「学生」也是一种「人」——人有的姓名、生日学生都有，针对「人」写的手续对「学生」同样管用。*底层机制*：ABI 层面的实现只有一句话——**把基类子对象作为派生类结构体的第一个成员**；C99 标准 6.7.2.1p13 规定指向结构体的指针经适当转换后指向其**初始成员**，故 `(base_t*)derived_ptr` 合法且**零指令开销**。*作用域与存储期*：父、子结构体在同一块内存连续存放，一个对象只有一次分配、一个存储期、一次释放。
- **多态 (Polymorphism)**：同一个调用表达式，按对象的**实际类型**在**运行期**选择不同实现。*直观解释*：同一句「开始工作」，对画家和司机说，执行的动作完全不同。*底层机制*：**两步间接**——对象里存 `vptr` 指向 vtable，vtable 是**每类一份**的函数指针表；`obj->vfun()` 编译成「取 vptr → 按固定偏移取函数地址 → `JSRR` /间接 `call`」，即讲义 slide 27–28 的「两次 load 加一次调用」。*作用域与存储期*：vtable 在只读段（`.rodata`），**每类一份**，存储期为整个程序；`vptr` **每对象一份**，随对象创建与销毁。
- **关键论断：四个概念是设计思想，不是语言特性。** C 没有 `class` / `private` / `virtual` 三个关键字，却能完整表达这四个思想——只是**由程序员手工保证**；C++ 把「手工保证」变成「编译器保证」：只在类定义里写一次类型层次与哪些函数要改写，编译器就替你排布数据、生成 vtable、填表。用讲义的话说，C++「产生的结构与函数通常与 C 中手工写出的一模一样」。
- **虚函数表指针 (vptr)**：含虚函数的对象里额外的一个指针字段，位于对象起始处（Itanium C++ ABI 约定），指向其**运行时类型**的 vtable。实测 `sizeof (Shape) = 40`、`sizeof (Circle) = 48`——这就是 `virtual` 的固定空间代价（C++ 的「pay only for what you use」：不用虚函数就不付）。vptr 在构造函数执行到**最内层派生类的函数体之前**被设为该层 vtable，析构时逐层回退，这就是「构造/析构期间不要调用虚函数」的原因。

图 1：基类子对象必须位于偏移 0（单继承的 ABI 实现）

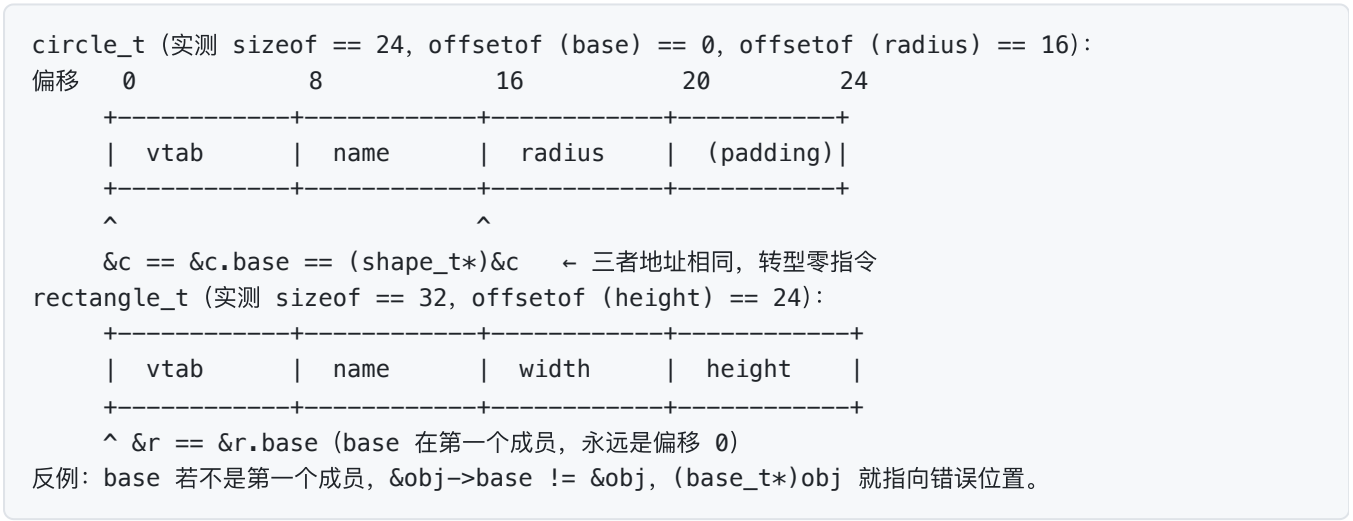
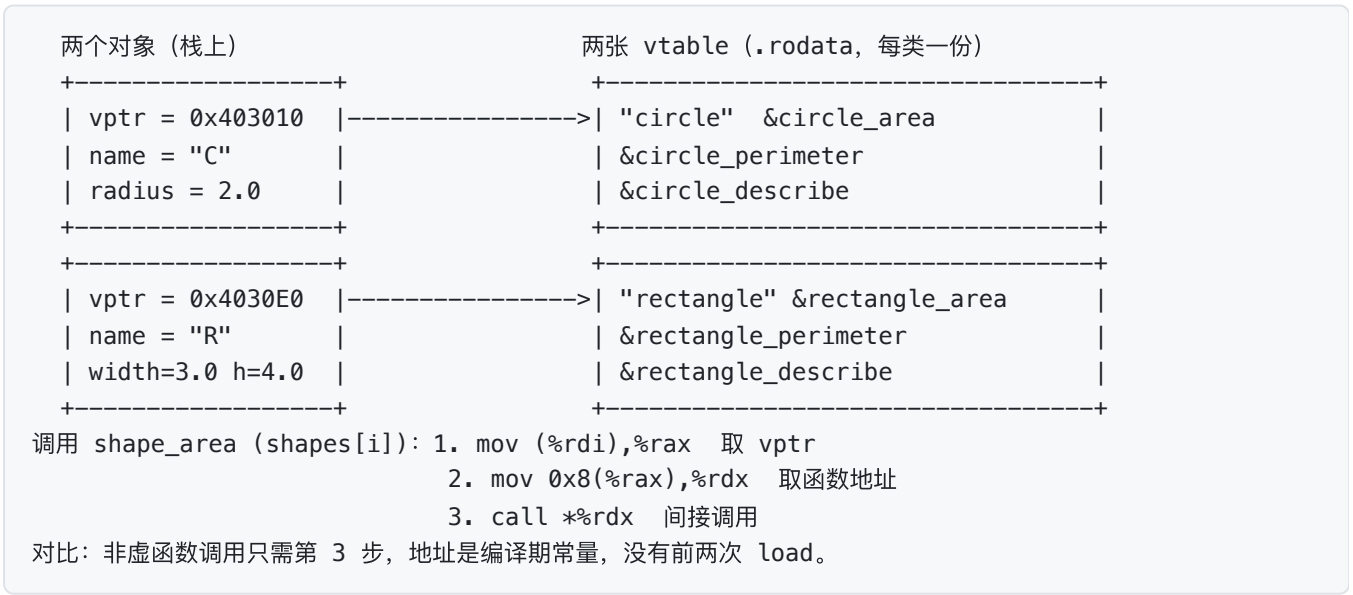


图 2：vtable 与动态分派（两次 load + 一次间接调用）



代码示例与底层机制分析

示例 1：C 中的信息隐藏 —— 真实 `mem220` 包与不透明句柄惯用法

代码 (C)

```
/* mem220.h —— 真实课程文件（节选）：头文件只暴露函数 */
void*   mem220_allocate   (size_t n_bytes);
int32_t mem220_reallocate (void** ptr_to_ptr, size_t n_bytes);
void    mem220_free       (void* ptr);
/* mem220.c —— 实现文件（节选）：头部结构体只在本文件可见 */
typedef struct mem_block_t mem_block_t;   /* 块头部，存在每块内存前端 */
struct mem_block_t { size_t size; mem_block_t* next; };
static uint8_t*     free_bytes;           /* 文件私有状态：static = 内部链接 */
static mem_block_t* mem_bin[MEM220_MAX_ALLOC_LOG+1];
void* mem220_allocate (size_t n_bytes)
{
    return (new_block + 1);               /* 查 bin 取块后，跳过头部返回 */
}
void mem220_free (void* ptr)
{
    mem_block_t* mem_block = ptr;
    int32_t      bin;
    if (NULL == ptr) { return; }
    bin = log2_ceil (mem_block[-1].size); /* ptr[-1] 就是头部！ */
    mem_block[-1].next = mem_bin[bin];
    mem_bin[bin] = &mem_block[-1];
}
/* 不透明句柄惯用法：把同一个模块写成「类」——只有不完整类型与原型 */
typedef struct memory_system memory_system_t;  /* 可声明指针，不能 sizeof */
memory_system_t* mem_system_create  (size_t capacity);       /* 构造函数 */
void             mem_system_destroy (memory_system_t* self); /* 析构函数 */
void* mem_system_allocate (memory_system_t* self, size_t n_bytes); /* 方法 */
/* mem_system.c —— 结构体定义与全部实现藏在这里 */
struct memory_system {          /* 只有本文件看得见这个定义 */
    size_t capacity;  size_t in_use;  size_t blocks_live;
    struct alloc_header_t* live_list;
};
void* mem_system_allocate (memory_system_t* self, size_t n_bytes)
{
    struct alloc_header_t* header;
    if (NULL == self || self->in_use + n_bytes + sizeof (*header) > self->capacity) {
        return NULL;                            /* 对象自己判定失败 */
    }
    if (NULL == (header = malloc (sizeof (*header) + n_bytes))) { return NULL; }
    header->next = self->live_list;  self->live_list = header;
    self->in_use += n_bytes;  self->blocks_live += 1;
    return (void*)(header + 1);                 /* 跳过头部 */
}
```

真实运行结果（`gcc -g -std=c99 -Wall -Werror`，两个包都实际编译运行过）：

```
# mem220.c + demo.c            # mem_system.c + handle_demo.c
MEM220_MAX_ALLOC = 1048576      greeting = "hello, ECE 220"   numbers = 0 1 4 9 16
all zero? yes                   in_use = 36   blocks_live = 2   blocks_freed = 0
data preserved? yes             free 两次后：in_use = 0, live = 0, freed = 2
allocate(0) = NULL              over-capacity request returns NULL
allocate(> MAX) = NULL
```

【代码做什么？】

1. `mem220_allocate(1000)` 把请求加上头部大小（64 位机实测 16 字节），用 `log2_ceil` 求 bin 号（10），优先从 bin 的空闲链表取块，返回 `new_block + 1` 即**跳过头部**的地址；`mem220_free` 用 `ptr[-1]` 找回头部并挂回 bin。
2. `mem_system_*` 版本改成让句柄 `self` 作首参数、用魔术数字自检，并统计 `in_use` / `blocks_live` ——实测 36/2，free 两次后归零；超容量请求返回 `NULL`，调用者绕不过这个检查。

【底层机制透视】

- **信息隐藏完全由文件作用域实现**：头文件没有 `struct mem_block_t` 定义，任何 `#include "mem220.h"` 的 `.c` 都**写不出** `buf[-1].size` ——不是被禁止，而是编译器不知道这些字段存在。此处 `static` 的含义是**内部链接**（名字不出本翻译单元），不是静态存储期。
- **`typedef struct memory_system memory_system_t;` 是句柄惯用法的核心**：这是**不完整类型**，`memory_system_t*` 完整（指针大小已知），`memory_system_t` 不完整（`sizeof` 编译失败）。调用者能传递、能保存句柄，却**永远无法**写 `ms->capacity`。
- **「句柄作第一个参数」就是「方法」**：`mem_system_allocate (ms, n)` 与 C++ 的 `ms->allocate (n)` 编译后**第一个整数参数寄存器**（x86-64 的 `%rdi`）里都是 `ms`，C++ 只是把它藏了起来。
- **负下标是合法 C**：`mem_block[-1]` 即 `*(mem_block - 1)`，指针算术以 `sizeof (mem_block_t)` 为单位，`p[-1]` 正好回退 16 字节落在头部起点——这就是 560 讲「用块上方头部保存管理信息」的实现。

【内存布局图解】

```
buf = mem220_allocate_and_zero (1000) 之后（实测 sizeof (mem_block_t) == 16）：
 +------------------+========================================+==========
 | mem_block_t 头部   |         调用者使用的 1000 字节           | 后续空闲区
 | size = 1024  next |                                        |
 +------------------+========================================+==========
 ^                   ^
 &block          buf = &block + 1   ← mem220_allocate 的返回值
句柄版本：栈上 ms ──► 堆上 struct memory_system {capacity, in_use, live_list}
                       live_list ──► [头部|数据] ──► [头部|数据]
调用者只能看到两端的数据；头部与 struct memory_system 全都不可见。
```

【与汇编的对应】

```
        LDR   R1, R4, #0          ; R1 = ms（句柄，作为第一个参数）
        LD    R0, SIZE_16         ; R0 = 16（第二个参数 n_bytes）
        JSR   mem_system_alloc    ; R7 ← 返回地址 = 下一条指令
        HALT                      ; 调用返回后停机（R0 = 新块地址；失败时为 0）
; ---- 下面是被调用者。真实程序里子程序通常放在别处，这里为便于并列阅读紧接其后；
;      关键是 JSR 之后必须还有一条调用者的指令，否则 R7 会指向子程序入口本身，
;      RET 就会跳回入口造成死循环。
mem_system_alloc                  ; 子程序入口：建立栈帧
        ADD   R6, R6, #-3         ; 3 个 linkage 字（旧帧指针 / 返回地址 / 返回值）
        STR   R5, R6, #0          ; 保存旧帧指针（设好 R5 后它就落在 R5+1）
        ADD   R5, R6, #-1         ; R5 = 本帧基址：R5+1 = 旧帧指针，R5+2 = 返回地址
        STR   R7, R5, #2          ; 保存返回地址（R5+2）
        LDR   R2, R1, #2          ; R2 = self->in_use（基址 + 常量偏移）
        ADD   R3, R2, R0          ; R3 = in_use + n
        LDR   R2, R1, #0          ; R2 = self->capacity（in_use 已用完）
                                  ; R4 是全局数据指针，不能当临时量，所以复用 R2
        NOT   R2, R2
        ADD   R2, R2, #1          ; R2 = -capacity
        ADD   R2, R2, R3          ; R2 = (in_use + n) - capacity
        BRp   alloc_fail          ; R2 > 0（超出容量）则失败返回 NULL
        ADD   R0, R1, #1          ; R0 = 新块地址（跳过头部）
        LDR   R7, R5, #2          ; 恢复返回地址（R5+2）
        LDR   R5, R5, #1          ; 恢复旧帧指针（R5+1）—— 与 L04/L08 的布局一致
        ADD   R6, R6, #3          ; 弹出 3 个 linkage 字，R6 回到进入时的位置
        RET                       ; 即 JMP R7
```

⚙️ **机器验证**：把这段清单配上一个模拟的 `memory_system` 结构（`capacity` 在偏移 0、`in_use` 在偏移 +2）后用 `ece220_data/lc3sim.py` 汇编并运行，四种情形都符合预期：

情形	结果
`capacity=100, in_use=3, n=16`	`R0 = MS+1`（返回块地址，成功）
`capacity=20, in_use=19, n=16`（19+16 > 20）	`R0 = 0`（走 `alloc_fail`）
`capacity=20, in_use=4, n=16`（4+16 = 20，**恰好装满**）	`R0 = MS+1`（成功）
`capacity=20, in_use=5, n=16`（5+16 = 21）	`R0 = 0`（失败）

四种情形下子程序都正常返回并 `HALT`，且 **`R5` 恢复成调用者进入时的帧指针、`R6` 回到调用者进入时的值** ——说明 prologue/epilogue 与 `R5+1` / `R5+2` 的约定严格配平。（具体执行了多少条指令取决于外面套的调用者代码，这里不把它当作结论；上面四行**结果**才是不随脚手架变化的。）边界行为也正确：**恰好装满不算超容量**，只有严格超出才失败。

顺带一提：这段清单在修正前还有一个更隐蔽的缺陷——`JSR` 的下一条指令就是子程序入口标签，于是 `R7` 指向入口本身，`RET` 会跳回去造成死循环；同时旧的"极简帧"把旧帧指针放在 `R5+0`，与全书的约定不符。两者现已一并修好。

示例 2：C 的 `shape`「类」、vtable 与跳转表（单继承与多态）

代码 (C)

```
/* shape.c —— 用 C 手工实现单继承 + 动态分派 */
typedef struct shape_vtab_t shape_vtab_t;   /* 每个类一份的「虚函数表」 */
struct shape_vtab_t {
    const char* class_name;
    double      (*area)      (const void* self);
    double      (*perimeter) (const void* self);
    void        (*describe)  (const void* self);
};
struct shape_t { const shape_vtab_t* vtab; const char* name; }; /* vptr 在头 */
struct circle_t    { shape_t base; double radius; };    /* 基类作首成员 */
struct rectangle_t { shape_t base; double width, height; };
static double circle_area (const void* self)
{
    const circle_t* c = self;
    return 3.14159265358979 * c->radius * c->radius;
}
static const shape_vtab_t CIRCLE_VTAB = {
    "circle", circle_area, circle_perimeter, circle_describe
};
static const shape_vtab_t RECTANGLE_VTAB = {
    "rectangle", rectangle_area, rectangle_perimeter, rectangle_describe
};
static double shape_area (const shape_t* self)  /* 「虚函数」分派器 */
{
    return (*self->vtab->area) (self);          /* 两次 load，然后间接调用 */
}
int main (void)
{
    circle_t c;  rectangle_t r;  shape_t* shapes[2];  size_t i;
    circle_init (&c, "C", 2.0);  rectangle_init (&r, "R", 3.0, 4.0);
    shapes[0] = &c.base;         shapes[1] = &r.base;  /* 向上转型：零指令 */
    for (i = 0; i < 2; i++) {
        printf ("%s: ", shapes[i]->name);
        shape_describe (shapes[i]);
        printf (" area = %.2f\n", shape_area (shapes[i]));
    }
    return 0;
}
/* 更朴素的形式：跳转表 —— 枚举值直接做数组下标 */
static op_func_t dispatch_table[NUM_OPS] = { op_add, op_mul, op_div, op_sub };
        op_t op = (op_t)row[2];  int32_t r = (*dispatch_table[op]) (row[0], row[1]);
```

真实运行结果与真实反汇编：

```
sizeof (shape_t) = 16, sizeof (circle_t) = 24
&c = 0x7ffe911c7230, &c.base = 0x7ffe911c7230, equal? yes
C: circle(radius=2.00) area = 12.57    R: rectangle(w=3.00,h=4.00) area = 12.00
sizeof (op_func_t) = 8 bytes; table base = 0x404040
add(12, 5) = 17    mul(12, 5) = 60    div(12, 0) = -1    sub(12, 5) = 7
0000000000401270 <shape_area>:          ← 真实 objdump 输出
  401280:       mov    (%rax),%rax           ; load 1: 取 vptr = self->vtab
  401283:       mov    0x8(%rax),%rdx        ; load 2: 取 vtab->area
  40128e:       callq  *%rdx                 ; 间接调用
```

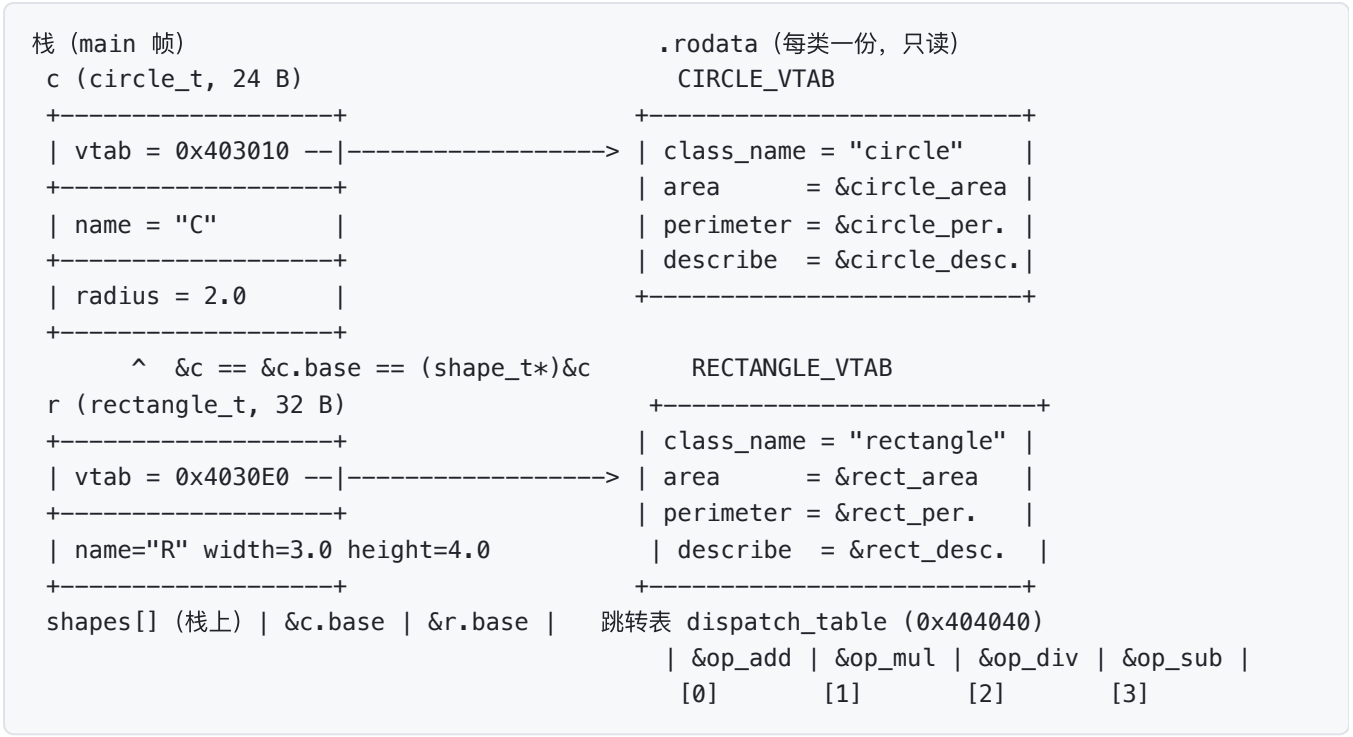
【代码做什么？】

1. `circle_init` / `rectangle_init` 把 `base.vtab` 指向**该类唯一的那张表**再填数据字段——这就是「构造函数」；`&c.base` 与 `&c` 实测**是同一个地址**，故 `shapes[0] = &c.base;` 不产生任何指令。
2. `shapes[i]->name` 按固定偏移 8 读取（**静态**绑定）；`shape_area (shapes[i])` 走 vtable：`i = 0` 调到 `circle_area`，`i = 1` 调到 `rectangle_area`。`dispatch_table` 是更朴素的版本：枚举顺序就是下标，实测函数指针都是 8 字节。

【底层机制透视】

- **反汇编完全印证讲义论断**：`shape_area` 里只有两条 `mov` 取地址，随后 `callq *%rdx`——正是 slide 27「两次内存读取」与 slide 28「Two loads followed by a call」。
- **vtable 每类一份，不是每对象一份**（讲义 slide 24–25）：若给每个对象都塞函数指针，1000 个 circle 就要 1000 份 `area` 指针；用 vtable 后只有 1 张表，对象只多一个 `vptr`。
- **向上转型安全的真正原因是 C99 6.7.2.1p13**（`base` 在偏移 0 使位模式相同）；**向下转型在 C 中不安全**，因为给定 `shape_t*` 无法知道后面是 `radius` 还是 `width` / `height`——讲义 slide 12–13 正是这个论点，解法是加 `type` 字段或用 vtab 里携带的类型信息。
- **跳转表与 `switch` 同源**：分支密集时编译器为 `switch` 生成的也是地址表，**跳转表是机器层面的通用机制，vtable 只是它的一个特例**。讲义 583 的 `isort (void* base, int32_t n_elts, size_t size, int32_t (*is_smaller)(void*, void*))` 是同一思想在泛型算法上的应用；`const void* self` 则是签名统一的折中（讲义 slide 20）。

【内存布局图解】



【与汇编的对应】

```
; (a) LC-3 实现 shape_area —— 讲义 slide 28 建议的练习
shape_area
        LDR   R1, R0, #0         ; R1 = self->vtab（vptr 在偏移 0）
        LDR   R2, R1, #1         ; R2 = vtab->area（表里第 1 个字）
        JSRR  R2                 ; 间接调用：多态的全部成本
        RET
; 对比：非虚函数调用，目标地址是编译期常量
        JSR   rectangle_area     ; 一条指令，无需任何 load
; (b) LC-3 跳转表 —— 与 dispatch_table 一一对应
        LEA   R2, DISPATCH       ; R2 = 跳转表基址
        LDR   R1, R0, #0         ; R1 = 操作码（作下标）
        ADD   R3, R1, R1         ; R3 = 2*op（每项 1 字，用 2 演示缩放）
        ADD   R3, R3, R2
        LDR   R3, R3, #0         ; R3 = DISPATCH[op]（函数入口地址）
        JSRR  R3                 ; 间接调用
DISPATCH  .FILL op_add           ; [0]
          .FILL op_mul           ; [1]
          .FILL op_div           ; [2]
```

示例 3：C++ 构造/析构与 RAII —— 真实的构造与析构顺序

代码 (C++)

```
// ctor_dtor.cpp
class Base {
public:
    Base ()  { std::cout << "  construct Base\n"; }
    virtual ~Base () { std::cout << "  destruct  Base\n"; }
private:
    Member m_;                        // 成员本身是对象：自动构造/析构
};
class Derived : public Base {
public:
    Derived () : Base (), first_("first"), second_("second")
    { std::cout << "  construct Derived body\n"; }
    ~Derived () override { std::cout << "  destruct  Derived body\n"; }
private:
    Tracer first_, second_;           // Tracer 的构造/析构都打印自己的名字
};
class Widget {
public:
    Widget () : n_(0), buf_(new char[8]) { }   // 数组元素用无参构造函数
    explicit Widget (std::int32_t n) : n_(n), buf_(new char[8]) { }
    ~Widget () { delete[] buf_; }    // 指针成员必须手工释放
private:
    std::int32_t n_;   char* buf_;
};
int main ()
{
    { Derived d; }                    // 离开作用域：析构自动发生
    Widget* w = new Widget (7);   delete w;
    Widget* arr = new Widget[3];  delete[] arr;
    return 0;
}
```

真实运行结果（`g++ -g -std=c++17 -Wall -Werror -o ctor_dtor ctor_dtor.cpp`）：

```
  construct Member    construct Base    construct first
  construct second    construct Derived body
--- Derived fully constructed ---
  destruct  Derived body    destruct  second    destruct  first
  destruct  Base            destruct  Member
--- array of 3 Widgets (no-argument ctor) ---
  Widget() is constructing element 0     (共三次，n_ 都是 0)
  ~Widget(0) releases its own buffer     (共三次，逆序)
```

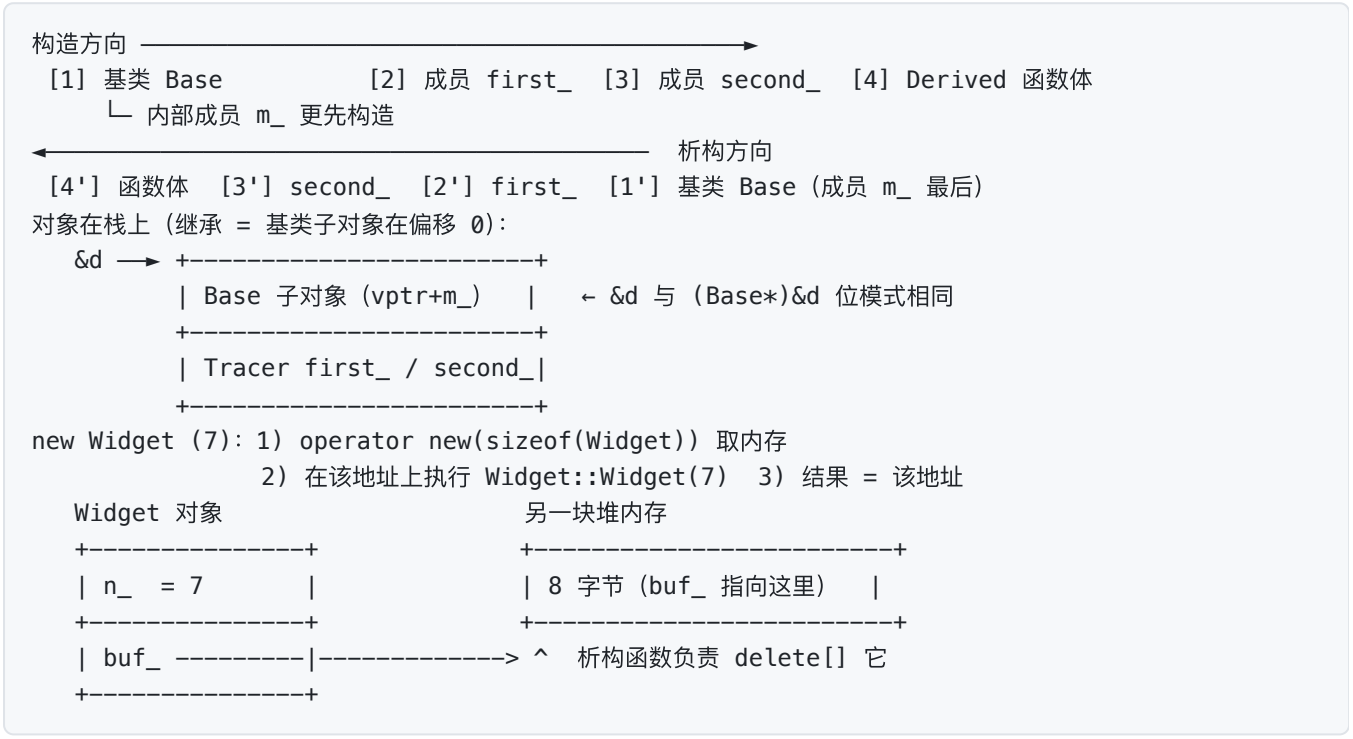
【代码做什么？】

1. 进入作用域构造 `Derived d`：输出证明构造顺序是 **基类 → 成员（按声明顺序）→ 自己的函数体**；离开作用域析构顺序**完全相反**：函数体 → 成员（逆序）→ 基类。
2. `new Widget (7)` 在堆上构造：构造函数在**分配之后**立即执行；`new Widget[3]` 对**每个元素**调用**无参构造函数**（实测三次 `Widget()`，字段 `n_` 均为 0），`delete[]` **逆序**析构三个元素。

【底层机制透视】

- **RAII（Resource Acquisition Is Initialization）**：资源在构造函数获取、在析构函数释放，于是「忘了释放」在 C++ 里**结构上不可能发生**——只要对象离开作用域，析构一定被调用；C 里必须在**每一条**返回路径上手工 `free`。
- **顺序规则来源**（讲义 593 slide 13）：初始化列表的执行顺序**不受书写顺序影响**，固定为「基类（按继承列表顺序）→ 成员（按声明顺序）」；析构顺序恰为构造的逆序，这是栈式内存管理的自然结果。初始化应写在初始化列表而不是函数体（slide 14），否则成员被默认构造一次后又赋值，属于无谓的工作。
- **析构函数不销毁「指针成员所指的对象」**：实测 `m_`、`first_`、`second_` 都自动析构，但 `buf_` 必须由 `~Widget` 里的 `delete[]` 显式释放。
- **析构函数不会在异常终止时运行**（slide 16）：`exit()`、段错误、除零崩溃都不运行析构函数；RAII 保护的是**正常控制流**。

【内存布局图解】



【与汇编的对应】

```
        LEA   R0, d                 ; R0 = 对象地址（相当于 this）
        JSR   Derived_ctor
Derived_ctor
        STR   R7, R5, #2            ; 保存返回地址
        ADD   R1, R0, #0            ; R1 = this（基类子对象在偏移 0）
        JSR   Base_ctor             ; 1) 先构造基类子对象
        LEA   R2, FIRST_STR
        ADD   R1, R0, #2            ; first_ 的偏移
        JSR   Tracer_ctor           ; 2) 再按声明顺序构造成员
        ADD   R1, R0, #4
        JSR   Tracer_ctor           ; second_
        RET                         ; 3) 最后执行构造函数体
```

示例 4：C++ `virtual`、隐藏的 `this` 指针与 vtable 探测

代码 (C++)

```
// this_ptr.cpp —— 成员函数有隐含的第一个参数
class Counter {
public:
    explicit Counter (std::int32_t start) : count_(start) {}
    void bump (std::int32_t by) { count_ += by; }  // 实为 this->count_ += by;
private:
    std::int32_t count_;
};
int main () { Counter c (10); c.bump (5); std::cout << c.value () << "\n"; }

// virtual.cpp —— 抽象基类 + 运行期多态
class Shape {
public:
    Shape (const std::string& name) : name_(name) {}
    virtual ~Shape () {}
    virtual double area () const = 0;                 // 纯虚：抽象基类
    virtual void   describe () const { std::cout << name_; }
    void tag () const { std::cout << "[shape tag]"; } // 非虚：静态绑定
private:
    std::string name_;
};
class Circle : public Shape {
public:
    Circle (double r) : Shape ("circle"), r_(r) {}
    double area () const override { return 3.14159265358979 * r_ * r_; }
    void   describe () const override { std::cout << "circle(r=" << r_ << ")"; }
private:
    double r_;
};
int main ()
{
    Circle c (2.0);  Rectangle r (3.0, 4.0);
    Shape* shapes[2] = { &c, &r };
    for (Shape* s : shapes) { s->describe (); s->area (); s->tag (); }
}
```

真实运行结果（含真实反汇编、真实 GDB 输出、真实符号表）：

```
000000000040123c <Counter::bump(int)>:      ← 真实 objdump 输出
  401240:       mov    %rdi,-0x8(%rbp)   ; 第 1 个整数参数寄存器 = this!
  401244:       mov    %esi,-0xc(%rbp)   ; 第 2 个 = by
  40124b:       mov    (%rax),%edx       ; edx = this->count_（偏移 0）
(gdb) break Counter::bump
(gdb) run
Breakpoint 1, Counter::bump (this=0x7fffffffb15c, by=5) at this_ptr.cpp:10
(gdb) info args
this = 0x7fffffffb15c
by = 5
sizeof (Shape) = 40   sizeof (Circle) = 48   sizeof (Rectangle) = 56
circle(r=2) area = 12.5664 [shape tag]   rectangle(w=3,h=4) area = 12 [shape tag]
address of b = 0x7ffe5ce32e70  b's vptr = 0x4020e8
address of d = 0x7ffe5ce32e60  d's vptr = 0x4020c0  different? yes  sizeof (Base) = 16
0000000000403160 V vtable for Shape          ← nm -C virtual
0000000000403130 V vtable for Circle
0000000000403100 V vtable for Rectangle
```

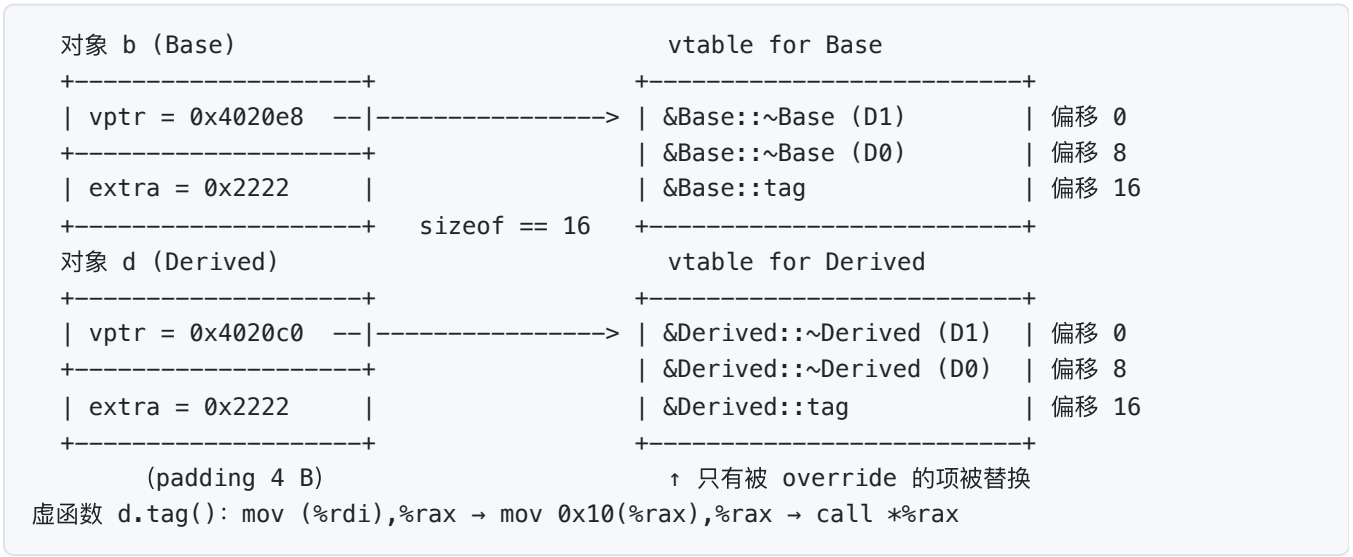
【代码做什么？】

1. GDB 显示成员函数参数列表里**第一个就是** `this=0x7fffffffb15c`——与 C 版本手写的 `self` 完全等价；反汇编确认 `%rdi`（x86-64 第一个整数参数寄存器）里放 `this`，`count_` 访问是 `(%rax)`，即偏移 0。
2. `vptr_probe` 读出两个**不同**的 vtable 地址；`nm -C` 显示 `vtable for Shape/Circle/Rectangle` 三个符号——**每类一份**，与示例 2 手写的 `CIRCLE_VTAB` / `RECTANGLE_VTAB` 一一对应。
3. 循环里 `s->describe()` 与 `s->area()` 动态分派（输出 `circle(...)` 与 `rectangle(...)`），而 `s->tag()` 永远输出 `[shape tag]`。

【底层机制透视】

- **`this` 是隐式的，不是没有**（讲义 591 slide 23）：类里的 `int32_t memFunc (char x, double* y);` 实际签名是 `int32_t memFunc (MyClass* this, char x, double* y);`，因此成员函数「遵守通常的调用约定」，不需要任何硬件支持。
- **`virtual` 必须写在基类里**（讲义 591 slide 37 的陷阱）：`virtual` **会**被派生类继承，但**不会反向传播**；只在派生类写 `virtual`，通过基类指针调用仍走基类版本。
- **两种绑定时机**：`describe` / `area` 是虚函数 → **运行期**按 vptr 决定；`tag` 不是 → **编译期**按指针静态类型决定，永远调 `Shape::tag`。含纯虚函数（`= 0`）的**抽象基类**不能实例化，只能作接口，强制每个派生类提供 `area`，把「忘了实现」变成编译错误。
- **`sizeof (Shape) = 40`** = `std::string` 成员 32 B + vptr 8 B；`Circle` 再加一个 `double` 与对齐共 48 B。空间代价是**每对象一个指针**，时间代价是**每次调用两次 load**——不用虚函数就不付这个代价。
- **容器与迭代器**（讲义 584）正是靠「基类指针 + 虚函数」实现的：容器代码只处理 `Animal*`，具体行为由运行时类型决定。讲义 584 的 `dl_execute_on_all (double_list_t* head, dl_execute_func_t func, void* arg)` 是同一思想的 C 版本——回调返回 `DL_CONTINUE` / `DL_REMOVE_AND_CONTINUE` / `DL_FREE_AND_CONTINUE` 等枚举值决定容器如何行动；`dl_first` 用 `head->next` 是否等于 `head` 判断空表。

【内存布局图解】



【与汇编的对应】

```
; LC-3 的虚函数调用：与示例 2 完全一样，只是表由编译器填
; R0 = 对象指针
        LDR   R1, R0, #0          ; R1 = *(R0+0) = vptr（vtab 在结构体最前面）
        LDR   R2, R1, #1          ; R2 = vtab[1] = 要调用的函数地址
        JSRR  R2                  ; 间接调用
        RET
; 对比：非虚成员函数，编译器直接生成 JSR，连 vptr 都不看
        JSR   Shape_tag           ; R0 = this 已经就位
```

示例 5：C++ 引用、运算符重载、模板 vs 虚函数

代码 (C++)

```
// refs.cpp —— 三种参数语义
void by_value     (std::int32_t  x) { x = 99; }   // 改副本，调用者看不到
void by_pointer   (std::int32_t* x) { *x = 99; }  // 调用点必须写 &
void by_reference (std::int32_t& x) { x = 99; }   // 静默的输出参数！
std::int32_t sum (const std::int32_t& a, const std::int32_t& b) { return a + b; }
std::int32_t& r = a;    // r 是 a 的别名

// overload.cpp —— 复数类与运算符重载
class Complex {
public:
    Complex (double re, double im) : re_(re), im_(im) {}
    Complex (double re) : re_(re), im_(0.0) {}     // 单参数构造 => 隐式转换
    Complex& operator+= (const Complex& rhs)
    { re_ += rhs.re_; im_ += rhs.im_; return *this; }
    friend Complex operator+ (const Complex& x, const Complex& y);
    friend Complex operator* (const Complex& x, const Complex& y);
private:
    double re_, im_;
};
Complex operator* (const Complex& x, const Complex& y)
{
    return Complex (x.re_ * y.re_ - x.im_ * y.im_,  // 返回整实例（栈上临时量）
                    x.re_ * y.im_ + x.im_ * y.re_);
}
// template_vs_virtual.cpp —— 两种多态
template <typename T> T twice (const T& x) { return x + x; }      // 编译期
class Animal { public: virtual std::string sound () const = 0; }; // 运行期
```

真实运行结果：

```
after by_value(a):     a = 1     p         = (1+2i)     twice(21)      = 42
after by_pointer(&a):  a = 99    p + q     = (4+1i)     twice(1.5)     = 3
after by_reference(a): a = 99    p * q     = (5+5i)     twice(string)  = abab
sum(a, b) = 101                  p*p + q*q = (5-2i)     Box<int>       = 42
a = 1234, r = 1234,              p * 2.0   = (2+4i)     Box<string>    = hello
&a == &r ? yes                   2.0 * p   = (2+4i)     animal says woof / meow
```

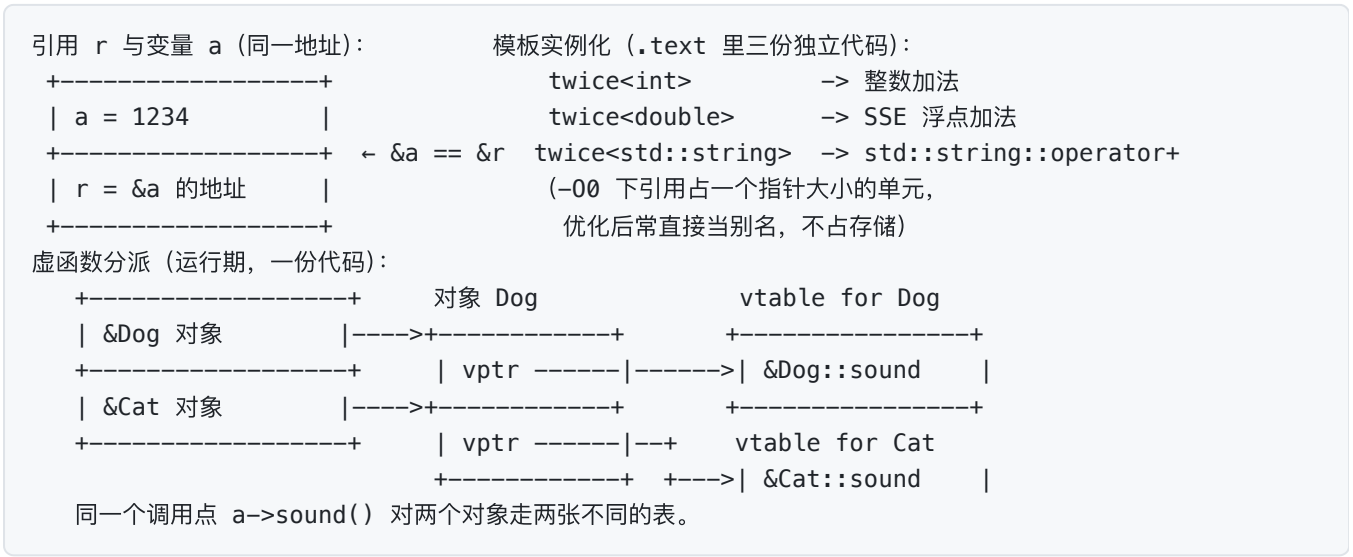
【代码做什么？】

1. `by_value(a)` 传副本，`a` 保持 1；`by_pointer(&a)` 与 `by_reference(a)` 都把 `a` 改成 99；`r = a` 后 `&a == &r` 为真——**引用就是别名**。
2. `p*p + q*q` 在 C 里要写 `complex_add (complex_multiply (P,P), complex_multiply (Q,Q))`，C++ 里直接写数学式（讲义 595 slide 6 的原始动机）。`2.0 * p` 与 `p * 2.0` 都工作：单参数构造函数 `Complex(double)` 提供从 `double` 的**隐式转换**，`friend` 函数让左右操作数对称。
3. `twice<int>` / `twice<double>` / `twice<std::string>` 是**三个独立函数实例**（编译期确定目标）；`Animal::sound()` 是同一调用点在运行期分派。

【底层机制透视】

- **引用在底层就是一个指针**（讲义 595 slide 16）：「引用被实现得与指针完全相同，但在语法上等价于它所指向的基类型。」实测 `&a == &r` 证实；区别只在引用**不能为 NULL**、**不能重新绑定**（单赋值，slide 17）。
- **引用是「容易被滥用」的特性**（slide 23–25）：把参数从值改成非常量引用，**调用点不会产生任何警告**，「某参数可能被改变」这一信息就消失了。讲义处方：**能用 `const` 引用就用 `const` 引用；需要修改的参数用指针**，这样调用点一定会出现 `&`。
- **运算符重载能毁掉可读性**（讲义 596 slide 3）：「C++ 允许极其细微的差别——请自担风险。」`operator+= (int)` 与 `operator+= (char)` 是两个不同重载，用起来和把两个变量命名成 `VaRiAbLe` 与 `vArIaBlE` 差不多。Lumetta 的建议很直接：**如果你不知道重载解析的答案，不要查，直接别用**——因为新函数可能「偷走」既有代码的调用（slide 6）。
- **`operator[]` 可能不再等价于 `*`**（596 slide 8）：C 里 `array[10]` 必然等价于 `*(array + 10)`；C++ 里前者调 `operator[]`，后者调 `operator+` 与 `operator*`，**两者可以定义得互不相容**。另外 `ALPHA b = a;` 用**拷贝构造**，`b = a;`（`b` 已存在）用 `operator=`，**重写其中一个不会重写另一个**，编译器**不会警告**（596 slide 10–11）。
- **返回实例会构造栈上临时量**（595 slide 12、28–32）：理论上 `complex b = a + a;` 会调用三次构造函数；实践中编译器用**具名返回值优化 (NRVO)**，由调用者在自己的栈帧里为 `b` 留空间、把指向它的指针作为隐含首参数传给 `operator+`，于是只构造一次。
- **模板 vs 虚函数**：模板是**编译期多态**（零运行时开销，但目标类型必须编译期已知，每类型生成一份代码）；虚函数是**运行期多态**（一次 vptr 间接加两次 load，但类型可以运行期才确定）。两者不可互相替代。

【内存布局图解】



【与汇编的对应】

```
; LC-3：引用参数与指针参数生成的代码完全相同
by_reference
        ; R0 = &x（引用参数传的就是地址）
        AND   R1, R1, #0
        ADD   R1, R1, #15
        ADD   R1, R1, #15
        ADD   R1, R1, #15
        ADD   R1, R1, #15
        ADD   R1, R1, #15
        ADD   R1, R1, #15        ; R1 = 90
        ADD   R1, R1, #9         ; R1 = 99
        STR   R1, R0, #0         ; *(&x) = 99，与 by_pointer 一模一样
        RET
by_value                         ; 只改自己栈帧里的副本，对外界无影响
        ADD   R0, R0, #1
        RET
```

示例 6：`new` / `delete`、数组与虚析构函数（Valgrind 实证）

代码 (C++) —— 第二段是明确标注「仅供演示、请勿模仿」的对照实验

```
// destructor_virtual.cpp —— 一个写错、一个写对
class BadBase {                      // 析构函数不是 virtual：错
public:
    BadBase () : buf_ (new char[1024]) { }
    ~BadBase () { delete[] buf_; }
private:
    char* buf_;
};
class BadDerived : public BadBase {
public:
    BadDerived () : extra_ (new char[1024]) { }
    ~BadDerived () { delete[] extra_; }
private:
    char* extra_;
};
class GoodBase {                     // 析构函数是 virtual：对
public:
    GoodBase () : buf_ (new char[1024]) { }
    virtual ~GoodBase () { delete[] buf_; }
private:
    char* buf_;
};
int main ()
{
    BadBase*  bad  = new BadDerived ();
    delete bad;    // 只运行 ~BadBase，extra_ 那 1024 字节泄漏
    GoodBase* good = new GoodDerived ();
    delete good;   // ~GoodDerived 再 ~GoodBase
    return 0;
}
```

```
// newdelete_mismatch.cpp —— 仅供演示，请勿模仿！两处未定义行为
    Tracked* a = new Tracked ();
    free (a);                  /* UB：析构函数根本不会运行 */
    Tracked* b = new Tracked[3];
    delete b;                  /* UB：错误释放器 + 只析构 1 个而非 3 个 */
```

真实运行结果与真实 Valgrind / 编译器输出：

```
  ~BadBase runs                          ~GoodDerived runs   ~GoodBase runs
==1221098== 1,024 bytes in 1 blocks are definitely lost in loss record 1 of 2
==1221098==    definitely lost: 1,024 bytes in 1 blocks
==1221098==    indirectly lost: 0 bytes in 0 blocks
==1221098==      possibly lost: 0 bytes in 0 blocks
==1221098==    still reachable: 4,096 bytes in 1 blocks
==1221098== ERROR SUMMARY: 1 errors from 1 contexts
# g++ -g -std=c++17 -Wall -Werror   真的把 new/delete 不匹配拦下来了！
newdelete_mismatch.cpp:30:10: error: 'void free(void*)' called on pointer returned
    from a mismatched allocation function [-Werror=mismatched-new-delete]
newdelete_mismatch.cpp:34:12: error: 'void operator delete(void*, std::size_t)' called
    on pointer returned from a mismatched allocation function
cc1plus: all warnings being treated as errors
# 去掉 -Werror 让它跑起来：析构次数不对，然后崩溃
  Tracked() acquired a 64-byte buffer       （共三次构造）
  ~Tracked() released its buffer           ← 只析构了 1 个，不是 3 个
munmap_chunk(): invalid pointer   Aborted (core dumped)
==1221491== Mismatched free() / delete / delete []
==1221491==    by 0x401220: main (newdelete_mismatch.cpp:30)
==1221491== Invalid free() / delete / delete[] / realloc()
==1221491==    by 0x401296: main (newdelete_mismatch.cpp:34)
==1221491==  Address 0x4da7d98 is 8 bytes inside a block of size 32 alloc'd
```

【代码做什么？】

1. `BadBase` 析构函数**不是**虚函数：`delete bad` 时编译器只看到静态类型 `BadBase*`，只调 `~BadBase`，`BadDerived` 的 `extra_` 那 1024 字节**永久泄漏**（Valgrind 实测「1,024 bytes definitely lost」）。
2. `GoodBase` 析构函数是 `virtual`：`delete good` 通过 vtable 找到 `~GoodDerived`，释放 `extra_` 后**自动接着**调用 `~GoodBase` 释放 `buf_`。
3. 对照实验里 `free(a)` 完全跳过析构函数；`delete b` 用在 `new Tracked[3]` 上时实测析构函数**只运行一次**，随后 `munmap_chunk(): invalid pointer` 崩溃。

【底层机制透视】

- **`malloc` 不调用构造函数，`free` 不调用析构函数**（讲义 594 slide 2）：构造 = 「分配内存」+「在内存上执行构造函数」两步，只有 `new` 做第二步，因此在 C++ 中**不要**用它们管理类实例。
- **`new[]` / `delete[]` 必须配对**（讲义 594 slide 6 的原话：「选错了就祝你好运找到那个 bug」）：实现通常在数组前面存一个**元素个数**供 `delete[]` 逐个析构，而 `delete` 不读这个计数，于是既用错释放器，也只析构第一个元素。

- **虚析构函数是「通过基类指针删除派生对象」的必要条件**（讲义 593 slide 17）：`ParentClass* p = new MyClass; delete p;` 若基类析构不是 virtual，**调用的就是错的那个**；加 `virtual` 的成本只是「对象多一个 vptr、销毁多一次间接调用」。
- **一个真实发现的更新**：讲义说编译器不会警告，但在本机 GCC 12.2.0 上 `-Wall -Werror` **确实**会报 `-Wmismatched-new-delete` 并拒绝编译。这是坚持使用 `-Wall -Werror` 的最有力理由之一——工具在进步，老经验需要被重新检验。
- **`new` 失败时抛异常**（594 slide 4）：默认会终止程序；要得到 `NULL` 需写 `new (std::nothrow) MyClass (...)`（含 `#include <new>`），此时构造函数不会被调用。**值初始化**（slide 8）：`new MyClass()`（带括号）先清零所有非实例字段再调构造函数；`new MyClass`（不带括号）不保证清零。

【内存布局图解】

```
delete bad;（析构函数不是 virtual）          delete good;（析构函数是 virtual）
  bad（静态类型 BadBase*）                    good（静态类型 GoodBase*）
  +------------------+                       +------------------+
  | 指向 BadDerived  |                       | 指向 GoodDerived |
  +------------------+                       +------------------+
  静态类型是 BadBase* ==>                     取 vptr -> 取析构函数地址 ->
  直接生成 call BadBase::~BadBase,            间接调用 -> ~GoodBase 自动接着跑
  不查 vtable ==> extra_ 泄漏 1024 字节        （实测输出 ~GoodDerived、~GoodBase）
new Tracked[3] 的实际内存（典型实现）：
 +-------------+------------+------------+------------+
 | 元素个数 = 3 | Tracked[0] | Tracked[1] | Tracked[2] |
 +-------------+------------+------------+------------+
 ^ delete[] 从这里读回 3，因此能析构 3 次
 ^ delete    不认识这个计数 -> 崩溃 / 只析构一次
```

【与汇编的对应】

```
; LC-3 中的「虚析构」：delete 必须通过对象的 vtab 找到真实析构函数
; R0 = 对象指针（静态类型是基类）
        LDR   R1, R0, #0          ; R1 = vptr
        LDR   R2, R1, #0          ; R2 = vtab[0] = 该对象的真实析构函数
        JSRR  R2                  ; 间接调用（先跑派生类析构函数体）
        RET                       ; 返回前自动 JSR 基类析构函数
; 若析构函数不是 virtual，编译器直接生成常量地址：
        JSR   Base_destructor     ; 永远只跑基类版本
```

【C 与 C++ 对照总结表】

关心的能力	纯 C 的做法	C++ 的做法	底层机制是否相同
数据与操作绑定	`struct` + 首参数为句柄的函数：`mem_system_allocate (ms, n)`	`class` + 成员函数：`ms->allocate (n)`	**相同**：`this` 就是那个首参数，都走 `%rdi`
隐藏实现	结构体定义放 `.c`，头文件只留不完整类型与原型的句柄惯用法	`private` / `protected` / `public` 访问说明符	**不同**：C 靠**文件作用域**（编译器看不见就无法访问），C++ 靠**编译器检查**，无硬件保护
多态分派	手写函数指针表（`shape_vtab_t`）+ 手动填表	`virtual` 函数 + 编译器生成的 vtable	**相同**：都是「两次 load + `JSRR` /间接 `call`」；vtable 每类一份、vptr 每对象一份
释放资源	到处手写 `free`，每条返回路径都要记得	析构函数在作用域结束时自动运行（RAII）	**不同**：C++ 的析构调用由编译器插入；而 `malloc` / `free` 对构造/析构一无所知
通用容器 / 泛型算法	`void*` + 元素大小 + 比较回调（讲义 583 `isort`、584 `dl_execute_on_all`）	`template <typename T>`	**不同**：`void*` 是**运行期**泛化（一份代码 + 指针间接），模板是**编译期**泛化（每类型一份，无间接）
借用别名而不复制	传 `T*`，调用点必须写 `&x`	传 `const T&`（只读）或 `T*`（可写）	**相同**：引用在底层就是一个指针；但不能为 NULL、不能重绑定

常见错误与调试技巧

- **基类子对象不在第一个成员**：`struct derived_t { int tag; base_t b; };`，现象是 `(base_t*)ptr` 之后访问的字段全是垃圾。**调试**：`printf ("offsetof = %zu\n", offsetof (derived_t, b));` 正常应为 0；或在 GDB 里对比 `p &d` 与 `p &d.b`。
- **函数指针类型写错**（漏 `const`、参数类型不符）：`-Wall` 报 incompatible pointer type。**调试**：不要用强转掩盖，用 `gcc -g -std=c99 -Wall -Werror` 让它拒绝编译；或在 GDB 里 `ptype CIRCLE_VTAB` 检查表的真实签名。
- **`virtual` 只写在派生类里**：`ParentClass* ptr = &m; ptr->aFunc();` 仍调用 `ParentClass::aFunc`。**调试**：在**基类**定义里给 `aFunc` 加 `virtual`；用 `nm -C yourprog | grep vtable` 确认真的生成了 vtable 符号。
- **`delete` 用在 `new[]` 得到的指针上**：析构次数不对，随后 `munmap_chunk(): invalid pointer` 崩溃。**调试**：`g++ -g -std=c++17 -Wall -Werror`（GCC 12 报 `-Wmismatched-new-delete`）；若已编译通过，用 `valgrind --leak-check=full ./prog` 找 `Mismatched free() / delete / delete []`。

- **基类析构函数不是 virtual 却通过基类指针删除**：派生类独有的资源静默泄漏。**调试**：`valgrind --leak-check=full --show-leak-kinds=all ./prog` 看 `definitely lost` 的大小与来源；只要层次里有资源成员，就给基类加 `virtual ~Base ()`。
- **在构造函数或析构函数里调用虚函数**：vptr 此时指向**正在构造/析构的那一层**，不会分派到派生类版本。**调试**：在 GDB 里 `break Derived::Derived`，用 `x/2gx this` 或 `print *this` 观察 vptr 的变化。
- **用 `printf` 当调试器而不是 GDB**：临时打印既污染代码又看不到栈帧。**调试**：改用 `gdb -tui --args ./prog arg1`；进 GDB 后 `b file.c:24`、`display i`、`p *ptr`、`bt`，观察完直接改源码重新编译，不必删除调试语句。

关键要点

- **面向对象的四个思想是设计思想，不是语言特性。** C 用「结构体 + 首参数为句柄的函数 + 文件作用域 + 函数指针表」就能完整表达封装、信息隐藏、继承与多态；C++ 的价值是让编译器**自动**完成这些手工劳动，并额外提供访问控制检查与自动清理。
- **单继承的 ABI 实现只有一句话：把基类子对象放在偏移 0。** 由此 `&derived == &derived.base`，向上转型零开销且合法（C99 6.7.2.1p13）；向下转型不安全，因为无法知道基类后面是什么。
- **C++ 的 `virtual` 与 C 的函数指针表是同一个机器机制。** 调用成本是「取 vptr、取函数地址、间接调用」——两次 load 加一次 `JSRR`/间接 `call`；vtable **每类一份**放在只读段，vptr **每对象一份**，代价换来运行期类型决定权。
- **析构函数是 C++ 相对 C 最真实的收益（RAII）。** 但要注意：`malloc`/`free` 不调用构造/析构、`new[]`/`delete[]` 必须配对、基类析构函数必须 `virtual`；异常终止（崩溃、`exit`）不会运行析构函数。
- **新特性都带新陷阱**：引用可能静默地变成输出参数（讲义 595「引用很容易被滥用」），运算符重载可以毁掉可读性（讲义 596「如果不知道答案，就别用」），拷贝构造与赋值不等价且编译器不警告。**收益来自克制地使用这些特性。**

思考题（带答案）

问题 1. 下面这段 C 代码为什么是**安全**的？如果把 `base` 移到 `derived_t` 的第二个成员，会发生什么？

```
typedef struct { const void* vtab; int id; } base_t;
typedef struct { base_t base; double radius; } derived_t;
base_t*   p = (base_t*)&d;   /* 安全吗？ */
```

答案. 安全。C99 6.7.2.1p13 规定：指向结构体对象的指针经适当转换后指向该结构体的**初始成员**。因为 `base` 是第一个成员，它与 `d` 地址相同，所以 `(base_t*)&d` 与 `&d.base` 位模式一致，**不产生任何指令**。若把 `base` 移到第二位，`&d.base` 就变成 `&d + sizeof(第一个字段)`（还可能加对齐填充），此时 `(base_t*)&d` 指向的位置**不是**一个 `base_t` 对象，属于未定义行为——`p->id` 会读到错误的内存。

问题 2. 下面 C++ 程序输出什么？为什么？

```
class Base {
public:
    Base () { who (); }
    virtual void who () const { std::cout << "Base::who\n"; }
    virtual ~Base () {}
};
class Derived : public Base {
public:
    Derived () { who (); }
    void who () const override { std::cout << "Derived::who\n"; }
};
int main () { Derived d; }
```

答案. 输出两行：先 `Base::who`，再 `Derived::who`。构造 `Derived d` 时**基类子对象先构造**，此期间对象 vptr 被设为该子对象的 vtable（`Base::who`），所以 `Base::Base()` 里的 `who()` 分派到 `Base::who`。等基类构造完成、进入 `Derived::Derived()` 的函数体**之前**，vptr 才被改写成 `Derived` 的 vtable，此时 `who()` 才分派到 `Derived::who`。这就是「构造函数期间不要调用虚函数」的原因：你以为会调用派生类版本，实际不会。

问题 3. 为什么 C++ 编译器生成的 vtable 不需要任何硬件支持？请从内存与指令两个层面说明。

答案. 因为 vtable 只是**普通的只读数据**，vptr 只是对象里的**普通指针字段**。实现多态只需要三条已有指令：一条 load（从对象取 vptr）、另一条 load（从 vtable 固定偏移取函数地址），再加一条间接跳转。LC-3 用 `LDR` + `LDR` + `JSRR` 就够（`JSRR` 本是为「子程序地址存在寄存器里」而设计，函数指针与回调一讲已经用过它）；x86-64 上对应 `mov (%rdi),%rax` + `mov 0x8(%rax),%rdx` + `call *%rdx`，本讲的真实反汇编输出已完全印证。所以多态是**编译器与 ABI 的约定**，不是 CPU 的特性。

Lecture 21: 调试工具与技术 (Testing, Debugging and Tooling: GDB, Valgrind and assert)

概述

本讲回答一个工程问题：**代码能编过、能跑，为什么还是错的？** 答案是错误分成若干**层次**，编译器只负责其中最浅的一层——语法与类型；链接错误、运行期崩溃、以及「能跑但答案错」的逻辑错误都必须靠**测试与调试**来抓。本讲先建立 Lumetta 的错误分类法 (error taxonomy)，再把工具链按「错误的可发现性」排列：编译标志（`-g -Wall -Werror -O0`）是第一道防线，GDB 用来观察运行中的程序状态（栈帧、变量、内存、vtable），Valgrind / UBSan 用来抓内存与未定义行为，`assert` 用来在开发期强制模块不变量 (invariant)，而**测试驱动 (test driver)** 与边界用例是唯一能覆盖逻辑错误的手段。这些技能直接对应教学目标第 8 条「能使用标准调试工具与技术测试和调试 C 程序」，也是 MP7 的全部内容。

核心概念与底层机制图解

- **错误分类法 (Error Taxonomy)**：按抽象层次自顶向下分四类（讲义 544 slide 6）。

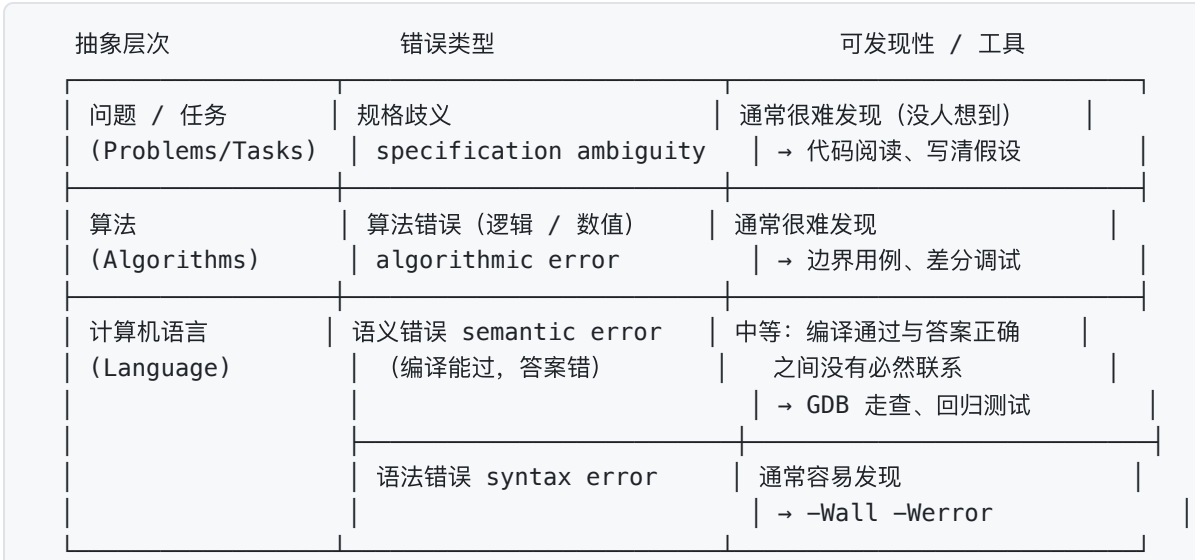
- *直观解释*：像装修——图纸错了很难发现（规范歧义），施工方案错了也难发现（算法错误），但把钉子钉歪了（语法错误）一眼就能看见。
- *底层机制图解*：层次越高越难发现，也越贵。**规格歧义 (specification ambiguity)** 是需求本身没写清楚（讲义 544 的例子：输入一开始就是 `-1` 时怎么办）；**算法错误 (algorithmic error)** 分逻辑与数值两类（讲义 544 的「loop swap sort」在 `8 3 4 7 9` 上成立，在 `12 4 1` 上失败；数值例子是不同机器浮点舍入方向不同导致 30% 误差）；**语义错误 (semantic error)** 是「实现写错了」，常是打字错误；**语法错误 (syntax error)** 是编译器能抓到的错误或警告。
- *作用域与存储期*：与内存无关——这类错误存在于**人类的设计与代码文本**中，不在运行时对象里。唯一的例外是运行期错误（UB、越界、泄漏），它们确实表现为内存状态被破坏。

- **编译期 vs 链接期 vs 运行期 vs 逻辑错误**：把上面的分类落到工具链上，就是本讲的操作骨架。
  - *直观解释*：编译期错误是「信写错了地址」，链接期错误是「信写好了但邮局找不到这个人」，运行期错误是「信寄到了但收信人当场晕倒」，逻辑错误是「信寄对了、人也没事，但内容是错的」。
  - *底层机制图解*：
    - **编译期**：`gcc` 的语义分析阶段报错，**必须**修好才能生成 `.o`。`-Wall -Werror` 把警告升级为错误。
    - **链接期**：`ld` 找不到符号（忘记实现、拼错函数名、忘记 `-l m`）。报错形如 `undefined reference to 'sqrt'`。
    - **运行期**：进程收到信号而终止——`SIGSEGV` (11) 段错误、`SIGFPE` (8) 整数除零、`SIGABRT` (6) 断言失败或 `abort`。退出码是 `128 + 信号号`，所以段错误在 shell 里看到 `139`，断言失败看到 `134`。
    - **逻辑错误**：进程正常退出（退出码 0）但输出错误。**这是唯一一类编译器完全帮不上忙的错误**，只能靠测试用例暴露。
  - *作用域与存储期*：编译期与链接期错误发生在**翻译单元**层面；运行期错误发生在**进程与内存**层面（栈、堆、只读段）；逻辑错误不属于任何存储期，它属于**输入与输出的关系**。
- **`-g` 与 `-O0`：让调试器看得见源码**。
  - *直观解释*：`-g` 是给可执行文件附上一张「机器码地址 ↔ 源文件行号 / 变量名」的对照表；`-O0` 是要求编译器**不要重排、不要合并、不要删除**代码，否则这张表就对不上现实。
  - *底层机制图解*：`-g` 把 DWARF 调试信息写进可执行文件的 `.debug_*` 段；GDB 用它把 `b file.c:24` 翻译成真实地址，把栈上的位模式解释成 `int32_t i`。`-O2` 会做寄存器分配、循环变换、常量传播，于是「第 24 行」可能对应另一段代码，变量可能被整个消除（GDB 显示 `<optimized out>`）。官方 GDB 快速参考页专门警告过这一点：某些情况下差异很大，甚至包括**行序重排**。
  - *作用域与存储期*：`-g` / `-O0` 只影响生成的可执行文件与调试信息，不改变程序的语义（形式上是如此）；调试完发布时通常切回 `-O2`。
- **GDB 的观察模型：栈帧 (stack frame) 与帧号 (frame number)**。
  - *直观解释*：backtrace 就是「我是怎么走到这一步的」——像翻看一叠便利贴，每张写着一个函数和它停在哪一行。

- *底层机制图解*：每次函数调用在栈上建立一帧。GDB 的 `bt` 输出里 `#0` 是**当前帧**（正在执行的函数），`#1` 是它的调用者，`#2` 再上一层。**关键约束**：`print` 只能看到当前帧可见的名字；在第 2 帧里打印第 1 帧的局部变量会失败。用 `frame N`（或 `f N`）切换上下文。讲义官方页面给出的示例输出：`#0 0x00001234 in main() at bar.c:12 #1 0x00004567 in foo() at bar.c:47 #2 0x00009876 in baz() at bar.c:56` 含义是 `main()` 在第 12 行调用了 `foo()`，`foo()` 在第 56 行调用了 `baz()`。
- *作用域与存储期*：帧的生命周期就是函数调用的生命周期——**函数返回后，它的局部变量存储期结束**，GDB 再打印就是垃圾值。官方页面明确提醒：「如果执行已经结束，打印变量值将不起作用。」

- **`.gdbinit` 与 TUI**：把重复劳动脚本化。
  - *直观解释*：`.gdbinit` 是 GDB 的「开机启动脚本」——每次打开 GDB 自动帮你设好断点、参数并运行。
  - *底层机制图解*：官方页面给出两步机制：在**家目录**的 `.gdbinit` 里写 `add-auto-load-safe-path <你的工作目录>`；在**工作目录**另建一个 `.gdbinit`，内容如 `set print pretty # 让结构体输出更易读 b baz # 在函数 baz 处断点 set args 4 25 # 为程序设置参数 run # 运行` 然后直接 `gdb ./foo` 即可。**若工作目录下没有 `.gdbinit`，GDB 就当作没有 gdbinit 文件**。TUI 模式用 `gdbtui ./bar` 或 `gdb ./bar --tui` 启动，用 `layout next` 在源码/汇编/寄存器窗口间切换，用 `C-x o` 切换焦点窗口，回车重复上一条命令。
  - *作用域与存储期*：家目录的 `.gdbinit` 是**全局**配置（自动加载安全路径）；工作目录的 `.gdbinit` 是**项目级**配置。本讲实测：未授权时工作目录的 `.gdbinit` 会被**静默忽略**，必须靠 `add-auto-load-safe-path` 或 `set auto-load safe-path /` 授权。

图 1：错误的层次结构与各层可用的工具（对应讲义 544 slide 6）



本讲工具与它们覆盖的层次：

```
-Wall -Werror ..... 语法/语义（编译期，最浅一层）
assert ............ 语义（模块不变量）
GDB ............... 语义 + 运行期（栈帧、变量、内存、vtable）
Valgrind / UBSan .. 运行期（越界、泄漏、未初始化、UB）
测试驱动 + 边界用例 ..... 语义 + 算法（唯一能抓逻辑错误的手段）
```

退出码速查（128 + 信号号）：

```
SIGSEGV(11) 段错误 → 139      SIGABRT(6) 断言/abort → 134
SIGFPE(8)  整数除零 → 136     正常退出 → 0（但答案可能是错的！）
```

代码示例与底层机制分析

示例 1：一个「能跑但答案错」的程序与一次完整的 GDB 会话

代码 (C)

```
/* buggy_stats.c —— 故意写错的程序（仅供演示） */
#define MAX_N 8
/* 返回数组前 n 个元素之和 */
static int32_t sum_first (const int32_t* arr, int32_t n)
{
    int32_t total = 0;
    int32_t i;
    for (i = 0; i <= n; i++) {   /* BUG：应为 i < n */
        total += arr[i];
    }
    return total;
}
static int32_t max_first (const int32_t* arr, int32_t n)
{
    int32_t best = arr[0];
    int32_t i;
    for (i = 1; i < n; i++) {
        if (arr[i] > best) { best = arr[i]; }
    }
    return best;
}
int main (void)
{
    int32_t data[MAX_N] = { 4, 8, 15, 16, 23, 42, 7, 3 };
    int32_t n = 4;
    printf ("sum_first = %d (expected 43)\n", sum_first (data, n));
    printf ("max_first = %d (expected 16)\n", max_first (data, n));
    return 0;
}
```

编译与运行（`gcc -g -std=c99 -Wall -Werror -o buggy_stats buggy_stats.c`）——**零警告、零错误、退出码 0**：

```
n         = 4
sum_first = 66 (expected 4+8+15+16 = 43)
max_first = 16 (expected 16)
```

下面是**实际执行**的 GDB 会话（`gdb -q ./buggy_stats`，命令与输出均为真实记录，仅删去部分重复行）：

```
(gdb) break sum_first
Breakpoint 1 at 0x401131: file buggy_stats.c, line 18.
(gdb) run
Breakpoint 1, sum_first (arr=0x7fffffffb130, n=4) at buggy_stats.c:18
(gdb) next
21          for (i = 0; i <= n; i++) { /* BUG: should be i < n */
(gdb) next
22              total += arr[i];
(gdb) display i
(gdb) display total
1: i = 0    2: total = 0
(gdb) next                                                          ← 此后每按一次 next 都打印 i 与 to
21          for (i = 0; i <= n; i++) { ... }     1: i = 0    2: total = 4
22              total += arr[i];                 1: i = 1    2: total = 4
21          for (i = 0; i <= n; i++) { ... }     1: i = 1    2: total = 12
22              total += arr[i];                 1: i = 2    2: total = 12
21          for (i = 0; i <= n; i++) { ... }     1: i = 2    2: total = 27
22              total += arr[i];                 1: i = 3    2: total = 27
21          for (i = 0; i <= n; i++) { ... }     1: i = 3    2: total = 43
22              total += arr[i];                 1: i = 4    2: total = 43   ← 此刻 i == n!
21          for (i = 0; i <= n; i++) { ... }     1: i = 4    2: total = 66   ← 又加了一次
(gdb) print i
$1 = 4
(gdb) print total
$2 = 66
(gdb) print arr[3]
$3 = 16
(gdb) print arr[4]
$4 = 23          ← 越界读到了第 5 个元素!
(gdb) x/8dw arr
0x7fffffffb130:        4       8       15      16        0x7fffffffb140:      23      42
(gdb) where
#0  sum_first (arr=0x7fffffffb130, n=4) at buggy_stats.c:21
#1  0x0000000000401239 in main () at buggy_stats.c:47
(gdb) info locals
total = 66
i = 4
```

追加的向量/指针观察命令（同一次会话中实测）：

```
(gdb) x/4dw arr
0x7fffffffb140:        4       8       15      16
(gdb) x/8xb arr
0x7fffffffb140:        0x04    0x00    0x00    0x00    0x08    0x00    0x00    0x00    ← 小端序
(gdb) info frame
Stack level 0, frame at 0x7fffffffb140:
 rip = 0x401131 in sum_first (buggy_stats.c:18); saved rip = 0x401239
 called by frame at 0x7fffffffb180
 Arglist at 0x7fffffffb130, args: arr=0x7fffffffb140, n=4
 Locals at 0x7fffffffb130, Previous frame's sp is 0x7fffffffb140
 Saved registers:
  rbp at 0x7fffffffb130, rip at 0x7fffffffb138
```

修复：把 `i <= n` 改成 `i < n`，重新编译运行得 `sum_first = 43 (expected 4+8+15+16 = 43)`。

【代码做什么？】

1. `sum_first` 用 `i <= n` 循环，因此当 `n = 4` 时循环执行 **5** 次（`i = 0,1,2,3,4`），多读了一个元素。
2. GDB 的 `display` 让每次暂停都自动打印 `i` 与 `total`：可见 `i` 从 3 跳到 4 时 `total` 从 43 变成 66——**43 + arr[4] = 43 + 23 = 66**，正好解释了输出。
3. `print arr[4]` 与 `x/8dw arr` 证实第 5 个元素是 23；因为数组本身有 8 个元素，这次越界读**没有崩溃**，只是悄悄算错。
4. `where`（即 `bt`）显示调用链只有两帧：`main` 在第 47 行调用 `sum_first`；`info locals` 显示当前帧只有 `total` 与 `i` 两个局部变量。

【底层机制透视】

- **为什么没有崩溃?** `arr` 指向 `main` 栈帧里的 `data[0..7]`。越界访问 `arr[4]` 仍然落在同一个数组对象内，只是**语义上越界**（函数承诺只看前 `n` 个），硬件与编译器都无法发现。若数组只有 4 个元素，越界就会读到 `n` 或返回地址等相邻数据，即使不崩溃也会得到随机答案。
- **`display` 与 `print` 的区别**：`print` 只求值一次，`display` 注册一个**每次暂停都自动重新求值**的表达式，并给它编号（实测 `1: i = 0`、`2: total = 0`）。这是观察循环变量最有效的手段。
- **`next` vs `step` vs `until` vs `finish`**：`next` 执行一整行（**不进入**被调函数），`step` 会**跳进**该行调用的函数，`until` 一直运行到**当前行之后的下一个源码行**（常用于快速走完剩余循环），`finish` 运行到**当前函数返回**并打印返回值。用错会让你在无关代码里迷路。
- **`x` 命令的格式**：`x/NFU addr` 中 N 是重复次数、F 是格式（`x` 十六进制、`d` 十进制、`b` 字节、`w` 4 字节字）、U 是单位大小。`x/8dw arr` 就是「从 `arr` 开始打印 8 个十进制的 4 字节字」；`x/8xb arr` 打印同样 8 个字节的十六进制，实测输出 `0x04 0x00 0x00 0x00 …` 直观展示了 x86-64 的**小端序 (little endian)**。
- **`-O0` 为什么重要**：本讲实测把同一程序用 `-O2` 编译后，`break sum_first` 落到了**第 21 行**而不是第 18 行，声明语句的行号在调试信息里已经不存在了。这就是官方 GDB 页面提醒「line 24 in qux.c 在编译后的程序里可能不匹配」的实证。

【内存布局图解】

```
main 的栈帧（高地址在上）                          低地址 → 高地址
 +--------------------------------+  ← 0x7fffffffb180 (caller frame)
 | ... main 的局部变量 ...         |
 | int32_t data[8]                |
 |  +----+----+----+----+----+----+----+----+
 |  |  4 |  8 | 15 | 16 | 23 | 42 |  7 |  3 |
 |  +----+----+----+----+----+----+----+----+
 |    ^                        ^
 |    | arr 指向这里            | arr[4]：函数本不该读，却读到了 23
 |    +------------------------+
 |  int32_t n = 4                 |
 +--------------------------------+  ← 0x7fffffffb140 (sum_first 帧底)
 | sum_first 的栈帧                |
 |  total = 0 → 66                |   ← info locals 显示的两个变量
 |  i     = 0 → 4 (i == n 时仍进入循环体)
 |  saved rbp / saved rip         |   ← info frame: rbp at 0x7fffffffb130
 +--------------------------------+       rip at 0x7fffffffb138
      低地址

GDB 视角：
  print arr[4]  → 23       数组内的越界读（不崩溃，静默算错）
  x/8dw arr     → 4 8 15 16 / 23 42 7 3     一次列出全部 8 个元素
  x/8xb arr     → 04 00 00 00 08 00 00 00   小端序：低位字节在前
```

【与汇编的对应】

```
; LC-3 中「i <= n 的循环」用 BRp/BRnz 实现——注意错误条件如何变成 BRz
; 正确版本：i < n，当 i >= n 时退出
loop_test
        NOT   R2, R1             ; R2 = ~n
        ADD   R2, R2, #1         ; R2 = -n
        ADD   R3, R0, R2         ; R3 = i - n
        BRzp  loop_done          ; i - n >= 0 则退出（即 i >= n）
        ; --- 循环体 ---
        LDR   R3, R5, #0         ; R3 = arr[i]（R3 在本次迭代里已用完）
        ADD   R2, R2, R3         ; total += arr[i]
        ADD   R0, R0, #1         ; i++
        BRNZP loop_test
loop_done
; 出错的版本把 BRzp 换成 BRp（仅当 i > n 才退出）：
;   BRp  loop_done              ; BUG：i == n 时会多执行一次循环体
; 这正是 C 里 "i <= n" 与 "i < n" 的差别，在汇编里只差一条指令的极性。
```

示例 2：把「跑一次对了」变成「每次都验证」——测试驱动

代码 (C)

```c
/* tests_stats.c —— 修复后的函数 + 打印 PASS/FAIL 的测试驱动 */
static int32_t sum_first (const int32_t* arr, int32_t n)
{
    int32_t total = 0;
    int32_t i;
    if (NULL == arr || 0 >= n) { return 0; }   /* 明确处理退化输入 */
    for (i = 0; i < n; i++) {                  /* FIXED: 原为 i <= n */
        total += arr[i];
    }
    return total;
}
static int tests_run    = 0;
static int tests_failed = 0;
static void check_i32 (const char* name, int32_t got, int32_t want)
{
    tests_run++;
    if (got == want) {
        printf ("PASS  %-34s got %d\n", name, got);
    } else {
        tests_failed++;
        printf ("FAIL  %-34s got %d, want %d\n", name, got, want);
    }
}
int main (void)
{
    int32_t none[1]  = { 0 };
    int32_t one[1]   = { 42 };
    int32_t many[8]  = { 4, 8, 15, 16, 23, 42, 7, 3 };
    int32_t equal[4] = { 5, 5, 5, 5 };
    int32_t neg[3]   = { -1, -2, -3 };
    check_i32 ("empty (n = 0)",      sum_first (none, 0), 0);   /* 边界 */
    check_i32 ("single element",     sum_first (one, 1), 42);   /* 边界 */
    check_i32 ("NULL pointer",       sum_first (NULL, 0), 0);   /* 边界 */
    check_i32 ("first four",         sum_first (many, 4), 43);  /* 普通情形 */
    check_i32 ("whole array",        sum_first (many, 8), 118); /* 暴露过 bug 的用例 */
    check_i32 ("all equal",          sum_first (equal, 4), 20);
    check_i32 ("all negative",       sum_first (neg, 3), -6);
    printf ("\n%d tests run, %d failed\n", tests_run, tests_failed);
    return (0 == tests_failed ? 0 : 1);   /* 失败则返回非零，供脚本判断 */
}
```

真实运行结果（`gcc -g -std=c99 -Wall -Werror -o tests_stats tests_stats.c && ./tests_stats`）：

```
PASS  empty (n = 0)                    got 0
PASS  single element                   got 42
PASS  NULL pointer                     got 0
PASS  first four                       got 43
PASS  whole array                      got 118
PASS  all equal                        got 20
PASS  all negative                     got -6
8 tests run, 0 failed          (退出码 0)
```

【代码做什么？】

1. 修复后 `sum_first` **显式处理退化输入**（`NULL` 或 `n <= 0` 返回 0），而不是依赖调用者保证前提。
2. `check_i32` 是极简测试框架：统计总数、打印 `PASS/FAIL`、失败时打印期望值与实际值。
3. 八个用例覆盖了边界（`n = 0`、`n = 1`、`NULL`）、普通情形（前 4 个）、**曾经暴露 bug 的用例**（整数组）、以及「全相等」「全负数」这类容易暴露比较符号错误的输入。
4. 全部通过时返回 0，任何一个失败返回非零——这让测试驱动可以被 Makefile 或 CI 脚本直接调用。

【底层机制透视】

- **「全数组」用例是回归测试 (regression test)。** 讲义 544 slide 30 的原则：**每次有人发现 bug，就加一个能暴露它的测试**，并在提交前跑通所有测试。这正是防止 bug 复发的最经济手段。
- **为什么必须测边界？** 讲义 544 slide 27 明确要求：让循环执行 **0 次或 1 次**、检查循环条件的**相等情形**、检查条件分支的**两个方向**、并思考可能的**溢出**。我们的 off-by-one bug 恰好只在 `i == n` 这个相等情形上体现——只测 `n = 1` 也可能因为巧合而通过。
- **全代码覆盖 (full code coverage) 只是起点**（讲义 545 slide 39、49）：必须让每条语句至少执行一次，但这还不够。讲义给出的反例是：`"0 0 0"` 这个测试暴露的 bug 出现在**已经被另一个测试覆盖过的语句**上，所以好的测试要同时考虑**代码的目的与结构**；方法上要把**白盒 (clear/white-box) 测试**（依据代码写测试）与**依据规格写测试**结合起来——只用白盒会漏掉「开发者根本没写进去的功能」，只用测试驱动开发则可能出现「为通过测试而写的假实现」（讲义 545 slide 5–8）。
- **「它跑对过一次」不是证据。** Brooks 的经验法则是 1/3 计划设计、1/6 写程序、**1/2 测试**（讲义 545 slide 2）。编译通过只说明语法对了，与答案正确**没有必然联系**。
- **测试要在写函数之前就开始设计。** 讲义 545 slide 10：「设计代码时就让它容易被测试」；先写一些测试会**迫使**你写出更可测的代码（例如把「计算」与「输入输出」分开，让 `sum_first` 只依赖参数而不是全局状态）。

【内存布局图解】

测试驱动的栈布局（每个 check_i32 调用建立一个新帧）：

```
   main 的栈帧
   +-----------------------------+  0x7ffd…b130
   | none[1]  = { 0 }            |     边界：n = 0 时不许读任何元素
   | one[1]   = { 42 }           |     边界：n = 1
   | many[8]  = { 4,8,15,16,23,42,7,3 }
   | equal[4] = { 5,5,5,5 }      |     全相等：暴露 < 与 <= 的混淆
   | neg[3]   = { -1,-2,-3 }     |     负数：暴露无符号/有符号错误
   | tests_run = 8, tests_failed = 0
   +-----------------------------+
   check_i32 的栈帧（每次调用独立）
   +-----------------------------+
   | name = "first four"         |     ← 指向 .rodata 里的字符串字面量
   | got = 43, want = 43         |
   | 返回地址 → main             |
   +-----------------------------+
   sum_first 的栈帧
   +-----------------------------+
   | arr, n（参数）               |
   | total = 43, i = 4           |     ← i == n 时循环条件为假，正确退出
   +-----------------------------+
```

每个用例都是一次「设置状态 → 调用 → 比较 → 记录」，全部在栈上完成，
没有全局可变状态污染测试之间的独立性。

【与汇编的对应】

```
; 测试驱动在 LC-3 上：把每个用例的期望值与实际返回值比较
        LEA   R0, many           ; R0 = 数组地址
        ADD   R1, R1, #4         ; R1 = n = 4
        JSR   sum_first
        ; 返回后 R0 = 实际值
        LD    R2, WANT_43        ; R2 = 期望值 43
        NOT   R2, R2
        ADD   R2, R2, #1         ; R2 = -43
        ADD   R2, R0, R2         ; R2 = 实际 - 期望
        BRz   case_pass
        LEA   R0, FAIL_STR
        PUTS                     ; TRAP x22：打印 FAIL
        ADD   R3, R3, #1         ; tests_failed++
        BRNZP next_case
case_pass
        LEA   R0, PASS_STR
        PUTS
next_case
; 关键点：比较用「减去期望值看是否为零」，与 C 里 got == want 完全对应。
```

示例 3：Valgrind —— 抓越界写、未初始化读与内存泄漏

代码 (C) —— 明确标注「仅供演示、请勿模仿」

```
/* leaky.c —— 三处缺陷，仅供演示 */
typedef struct { char* name; int id; } record_t;
static char* read_string (const char* src)
{
    char*  buf = malloc (8);       /* BUG 2 起点：8 字节装不下 15 个字符 */
    size_t i;
    for (i = 0; i <= strlen (src); i++) {
        buf[i] = src[i];           /* BUG 2：越界写（含结尾的 '\0'） */
    }
    return buf;
}
int main (void)
{
    record_t* records[2];
    int*      uninit;
    for (i = 0; i < 2; i++) {
        records[i] = malloc (sizeof (record_t));   /* BUG 1：从不 free */
        records[i]->id = i;  records[i]->name = NULL;
    }
    text = read_string ("hello, ECE 220");   /* BUG 2 */
    printf ("text = %s\n", text);
    free (text);
    uninit = malloc (sizeof (int));          /* BUG 3：未初始化就读 */
    if (10 > *uninit) { printf ("branch taken\n"); }
    free (uninit);
    return 0;
}
```

真实 Valgrind 输出（`gcc -g -std=c99 -Wall -Werror -o leaky leaky.c`，然后 `valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./leaky`，节选；`==NNNNNN==` 里的进程号每次运行都不同，其余内容逐字来自本机实测）：

```
==1215777== Invalid write of size 1
==1215777==    at 0x4011A3: read_string (leaky.c:24)
==1215777==    by 0x40121E: main (leaky.c:44)
==1215777==  Address 0x4a6f0e8 is 0 bytes after a block of size 8 alloc'd
==1215777==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==1215777==    by 0x40117B: read_string (leaky.c:20)
==1215777==
==1215777== Invalid read of size 1
==1215777==    at 0x484A604: strlen (vg_replace_strmem.c:495)
==1215777==    by 0x48E7207: __vfprintf_internal (in /usr/lib64/libc.so.6)
==1215777==  Address 0x4a6f0e8 is 0 bytes after a block of size 8 alloc'd
==1215777==
==1215777== Conditional jump or move depends on uninitialised value(s)
==1215777==    at 0x40125C: main (leaky.c:50)
==1215777==  Uninitialised value was created by a heap allocation
==1215777==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==1215777==    by 0x40124E: main (leaky.c:49)
==1215777==
==1215777== HEAP SUMMARY:
==1215777==     in use at exit: 4,128 bytes in 3 blocks
==1215777==   total heap usage: 5 allocs, 2 frees, 4,140 bytes allocated
==1215777==
==1215777== 32 bytes in 2 blocks are definitely lost in loss record 1 of 2
==1215777==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==1215777==    by 0x4011DC: main (leaky.c:38)
==1215777==
==1215777== 4,096 bytes in 1 blocks are still reachable in loss record 2 of 2
==1215777==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==1215777==    by 0x48EE693: _IO_file_doallocate (in /usr/lib64/libc.so.6)
==1215777==
==1215777== LEAK SUMMARY:
==1215777==    definitely lost: 32 bytes in 2 blocks
==1215777==    indirectly lost: 0 bytes in 0 blocks
==1215777==      possibly lost: 0 bytes in 0 blocks
==1215777==    still reachable: 4,096 bytes in 1 blocks
==1215777==         suppressed: 0 bytes in 0 blocks
==1215777== ERROR SUMMARY: 22 errors from 6 contexts (suppressed: 0 from 0)

# 程序本身的输出（它「正常」运行完了，退出码仍是 0）
text = hello, ECE 220
branch taken
records[1]->id = 1
```

关于 AddressSanitizer：下面这段不是本机输出。 ASan 在本环境**无法启动**（原因见【底层机制透视】），因此本节**无法**给出 ASan 的实测报告。下面的代码块只是**描述 ASan 报告的标准形式**，用来与上面的 Valgrind 报告对照，**不是从这台机器粘贴的真实输出，请勿当作实测证据**：

```
===== 以下为 ASan 报告的标准形式（示意，非本机实测输出） =====
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000018
WRITE of size 1 at 0x602000000018 thread T0
    #0 0x4c3a2b in read_string /home/user/leaky.c:24
    #1 0x4c3c1e in main /home/user/leaky.c:44
0x602000000018 is located 0 bytes to the right of 8-byte region
    [0x602000000010,0x602000000018)
allocated by thread T0 here:
    #0 0x4be1a7 in malloc
    #1 0x4c3b7b in read_string /home/user/leaky.c:20
SUMMARY: AddressSanitizer: heap-buffer-overflow /home/user/leaky.c:24 in read_string
```

两者定位同一个 bug 的策略不同：**ASan 在第一次越界发生的瞬间**就终止程序（退出码非零），并在同一条报告里同时给出**越界点与分配点**；**Valgrind 让程序继续跑完**，把所有问题一次性列出来（本例 22 个错误来自 6 个上下文）。ASan 更快（插桩而非翻译机器码）但必须重新编译，Valgrind 更慢但不需要改编译命令——本环境只能用后者。

【代码做什么？】

1. **BUG 1（泄漏）**：两个 `record_t` 从来没有 `free`，Valgrind 报 `32 bytes in 2 blocks are definitely lost`，并**精确指出分配位置**是 `main (leaky.c:38)`。
2. **BUG 2（越界写）**：`malloc (8)` 只给 8 字节，却要写入 15 个字符加结尾 `'\0'`。Valgrind 报 `Invalid write of size 1`，地址说明是 `0 bytes after a block of size 8 alloc'd`——**越过块末尾的第一个字节**。
3. 越界写破坏的正是堆块的元数据/相邻数据，因此后面 `printf ("%s", text)` 时 `strlen` 又触发 `Invalid read of size 1`（同一地址，块尾之后）。
4. **BUG 3（未初始化读）**：`malloc` 返回的内存内容是**任意位**，`10 > *uninit` 依赖这些垃圾位。Valgrind 报 `Conditional jump or move depends on uninitialised value(s)`，`--track-origins=yes` 进一步指出「该未初始化值来自 `main (leaky.c:49)` 的堆分配」。

【底层机制透视】

- **四类泄漏的含义**：`definitely lost` 是**没有任何指针能到达**的块（真泄漏，本程序 32 字节）；`indirectly lost` 是「只被已丢失块指向」的块；`possibly lost` 是「指针指向块内部而非块首」（例如指针被移动过）；`still reachable` 是**程序结束时仍由全局/栈上指针可达**的块（本程序的 4,096 字节来自 `stdout` 缓冲区，属于正常情况，**不是 bug**）。读懂这四档的差别，才能不被 `still reachable` 干扰判断。
- **Valgrind 的工作方式**：它是**动态二进制翻译 (dynamic binary translation)** 的模拟器——不重编译你的程序，而是把机器码翻译成自己的中间表示，给每个字节附带「已初始化/未初始化」和「可访问/不可访问」的标记位（shadow memory），因此能发现**硬件不会报错**的越界与未初始化读。代价是程序运行慢 10–50 倍。
- **程序"正常退出"却有 22 个错误**：这台程序退出码是 0，输出也「看起来对」，但 `ERROR SUMMARY: 22 errors from 6 contexts` 说明堆已被破坏。这正是「能跑不代表对」的最好例证。`--show-leak-`

`kinds=all` 让 `still reachable` 也显示出来；`--track-origins=yes` 则把未初始化值的**来源**一并报告。

- **`-fsanitize=address,undefined` 是另一条路线**：它在**编译期**插入检查代码（而非翻译机器码），因此更快但必须重新编译。**本环境只能用其中的 UBSan**：UndefinedBehaviorSanitizer 正常工作（示例 4 的诊断是实测），而 **AddressSanitizer 无法启动**（原因见下条）。
- **本环境的工具可用性（实测，请按此选择工具）**：`valgrind-3.19.0` **可用**，本节的泄漏/越界/未初始化报告**全部是它的真实输出**；`gdb` 10.2 **可用**（示例 1 的会话是真实记录）；`gcc` / `g++` 12.2.0 可用。**`-fsanitize=address` 不可用**——任何用该标志编译出的程序在**启动阶段**就直接失败，实测报错为 `ERROR: AddressSanitizer failed to allocate 0xdfff0001000 (15392894357504) bytes at address 2008fff7000 (errno: 12)` 与 `ReserveShadowMemoryRange failed while trying to map 0xdfff0001000 bytes. Perhaps you're using ulimit -v`。原因是本容器虚拟内存上限 `ulimit -v` = **32000000 KB（约 32 GB）**，而 ASan 需要预留约 **15 TB** 的 shadow 地址空间；该上限在本会话中无法提高（`ulimit -v unlimited` 无效）。**因此本讲的泄漏、越界与未初始化演示一律以 Valgrind 为准**，上面那段 ASan 报告已明确标注为「标准形式示意，非本机输出」。
- **本机 Valgrind 的一处已知噪声**：Valgrind 3.19.0 与本机 glibc 在退出阶段的 `__libc_freeres` 不完全兼容，每次运行会多出一行 `Process terminating with default action of signal 5 (SIGTRAP)`。**实测它出现在 `HEAP SUMMARY` 之前**（本次运行中位于输出的第 56 行），而 `HEAP SUMMARY` / `LEAK SUMMARY` / `ERROR SUMMARY` 随后都正常输出，退出码仍为 0，因此**不影响报告的有效性**——不要因为它以为程序或报告出了问题。上面贴出的节选已把这行省略。

【内存布局图解】

```
Valgrind 眼中的堆（每个字节带 shadow 标记位）：

  malloc(8) 返回的块（leaky.c:20）
  +--------+--------+--------+--------+--------+--------+--------+--------+
  | 'h'    | 'e'    | 'l'    | 'l'    | 'o'    | ','    | ' '    | 'E'    |
  +--------+--------+--------+--------+--------+--------+--------+--------+
   ↑ buf                                          ← 合法区域到此结束
                                                   ↓ 越界写开始
                        +--------+--------+--------+--------+--------+------+
                        | 'C'    | 'E'    | ' '    | '2'    | '2'    | '0'  | …
                        +--------+--------+--------+--------+--------+------+
  Valgrind: "Invalid write of size 1
             Address 0x4a6f0e8 is 0 bytes after a block of size 8 alloc'd"

  两个 record_t（从未 free）：
  records[] 数组（栈）                堆
  +----------------+                +-------------------+
  | &records[0] --|--------------->| id = 0, name=NULL|  ← 32 字节 definitely lost
  +----------------+                +-------------------+
  | &records[1] --|--------------->| id = 1, name=NULL|
  +----------------+                +-------------------+
  程序结束时这两个指针随栈帧消失 → 堆块**不可达** → definitely lost

  未初始化读（leaky.c:49）：
  +--------+--------+--------+--------+
  | ??     | ??     | ??     | ??     |   malloc 返回，内容任意
  +--------+--------+--------+--------+
   ↑ uninit
   if (10 > *uninit)  → 分支取决于垃圾位 → Valgrind 报 uninitialised value
```

【与汇编的对应】

```
; LC-3 中「分配与释放必须配对」的检查靠人工，Valgrind 相当于自动化的检查器
        LD    R0, SIZE_8          ; 请求 8 字节
        JSR   malloc              ; R0 = 块地址
        ADD   R1, R0, #0          ; R1 = buf
        ; 写入 16 字节（越界！）
        LEA   R2, SRC             ; R2 = "hello, ECE 220"
        AND   R3, R3, #0          ; i = 0
copy_loop
        LDR   R0, R2, #0          ; R0 = src[i]（R0 的返回值已经用完，可当临时）
        STR   R0, R1, #0          ; buf[i] = src[i]   ← 第 9 次起越界
        ADD   R1, R1, #1
        ADD   R3, R3, #1
        ADD   R0, R3, #-16        ; R0 = i - 16
        BRn   copy_loop
        ; 释放：忘记这一步，模拟器就永远收不回这 8 个字节
        JSR   free                ; 若删掉这行，就是"泄漏"
; LC-3 的 lc3sim 不会告诉你越界；Valgrind 会——这就是工具的价值。
```

示例 4：SIGSEGV、SIGABRT 与 `assert` / `perror`

代码 (C)

```c
/* assert_perror.c —— assert()、NDEBUG、errno 与 perror() */
#include <assert.h>
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
/* INTERNAL INVARIANT：只在数组有效且非空时调用，用 assert 强制这个契约 */
static int32_t internal_max (const int32_t* arr, int32_t n)
{
    int32_t best, i;
    assert (NULL != arr);
    assert (0 < n);
    best = arr[0];
    for (i = 1; i < n; i++) {
        if (arr[i] > best) { best = arr[i]; }
    }
    return best;
}
/* USER INPUT / ENVIRONMENT FAILURE：不是断言材料，必须报告并保持控制 */
static int read_file_or_report (const char* path, char* buf, size_t cap)
{
    FILE* fp = fopen (path, "r");
    if (NULL == fp) {
        fprintf (stderr, "cannot open '%s': ", path);
        perror (NULL);                    /* 打印 strerror(errno) */
        return -1;
    }
    if (NULL == fgets (buf, (int)cap, fp)) { fclose (fp); return -1; }
    fclose (fp);
    return 0;
}
int main (int argc, char* argv[])
{
    int32_t data[4] = { 3, 9, 4, 7 };
    char    line[128];
    printf ("internal_max = %d\n", internal_max (data, 4));
#ifdef NDEBUG
    printf ("NDEBUG is defined  -> assert() is compiled out\n");
#else
    printf ("NDEBUG is not defined -> assert() is active\n");
#endif
    if (0 != read_file_or_report ("/nonexistent/file.txt", line, sizeof (line))) {
        printf ("  handled the error, still running (errno=%d, %s)\n",
                errno, strerror (errno));
    }
    if (2 == argc && 0 == strcmp (argv[1], "crash-me")) {
        printf ("now calling internal_max(data, 0) ...\n");
        fflush (stdout);
        printf ("%d\n", internal_max (data, 0));   /* 违反不变量 */
    }
    return 0;
}
```

真实运行结果（三种编译/运行方式）：

```
# gcc -g -std=c99 -Wall -Werror -o assert_on  assert_perror.c ;  ./assert_on
internal_max = 9
NDEBUG is not defined -> assert() is active
fopen failure demo:
  handled the error, still running (errno=2, No such file or directory)
 (stderr: cannot open '/nonexistent/file.txt': No such file or directory)

# ./assert_on crash-me              ← 断言被违反
now calling internal_max(data, 0) ...
assert_on: assert_perror.c:23: internal_max: Assertion `0 < n' failed.
Aborted (core dumped)              ← 退出码 134 = 128 + SIGABRT(6)

# gcc -g -std=c99 -Wall -Werror -DNDEBUG -o assert_off assert_perror.c
# ./assert_off crash-me
NDEBUG is defined  -> assert() is compiled out
now calling internal_max(data, 0) ...
3                                  ← 没有崩溃，但返回的是垃圾值
```

【代码做什么？】

1. `internal_max` 用两个 `assert` 声明并**强制执行**它的前提条件（非空、`n > 0`）——这是**模块内部不变量**。
2. `read_file_or_report` 处理的是**用户输入/环境失败**（文件不存在）：它打印诊断、返回 `-1`、程序继续运行。实测 `errno = 2`（`ENOENT`）、`strerror` 给出 `No such file or directory`。
3. 断言版本在 `n = 0` 时**立刻崩溃**并打印文件名、行号、函数名与失败的表达式。
4. 用 `-DNDEBUG` 编译后断言被**完全消除**，程序返回垃圾值 `3`（读到了 `arr[0]` 之外的旧数据）——这演示了「发布版把断言关掉」的风险。

【底层机制透视】

- **`assert` 是什么**：`<assert.h>` 里的宏。它在表达式为假时调用 `__assert_fail`，后者向 `stderr` 打印诊断并调用 `abort()`；`abort()` 发送 `SIGABRT`，所以退出码是 `128 + 6 = 134`。**它是开发期工具，不是错误处理机制。**
- **`NDEBUG` 的作用**：定义 `NDEBUG` 后 `assert(expr)` 被展开为 `((void)0)`——**完全不求值 `expr`**。因此绝不能在 `assert` 里写有副作用的表达式（`assert(fclose(f) == 0)` 在发布版里根本不会执行）。教材与讲义都强调这一点；Ariane 5 事故的直接原因之一就是「断言被关掉后，整数溢出失去了保护」（MP7 的背景材料）。
- **什么时候用断言，什么时候必须做真正的错误处理**：
  - **断言（内部不变量）**：函数的前提条件（指针非空、数组非空）、类不变量（`in_use <= capacity`）、`switch` 的 `default` 分支（「不可能到达」）、循环不变量。这些在**正确的程序里永远不会触发**，触发就意味着**代码有 bug**。
  - **真正的错误处理（外部世界）**：`malloc` 返回 `NULL`、文件打不开、用户输入非法、网络断开。这些**不是 bug**，程序必须给出诊断并优雅地失败。讲义 544 slide 9 的说法是：「在模块边界断言所有要求」

——即断言「调用者必须满足的前提」，而用返回值/错误码处理「环境的不确定性」。

- **`errno` 与 `perror`**：`errno` 是每个线程一份的全局错误码，**只在库函数报错时才被设置**，且可能被后续调用覆盖——所以要在失败后**立刻**读取。`perror(s)` 打印 `s: ` 加上 `strerror(errno)`；实测 `perror(NULL)` 只打印错误描述。**不要在 `errno` 上做 `if (errno != 0)` 之类的检查**——成功调用不保证把它清零。
- **崩溃也是一种信息**：`SIGSEGV`（退出码 139）通常来自空指针解引用或野指针；`SIGFPE`（136）来自整数除零；`SIGABRT`（134）来自断言或 `abort`。本讲用 UBSan 实测：`divide_by_zero (7, 0)` 在打印 `runtime error: division by zero` 之后立刻收到 `Floating point exception (core dumped)`、退出码 136；`null_deref (NULL)` 报 `runtime error: load of null pointer of type 'int32_t'` 后收到 `Segmentation fault`、退出码 139。

【内存布局图解】

```
断言失败时的现场（assert_on crash-me）：

  internal_max 的栈帧
  +-----------------------------+
  | arr = &data[0]              |     ← assert(NULL != arr) 通过
  | n   = 0                     |     ← assert(0 < n) 失败!
  | best = ??（未初始化就用了）   |
  +-----------------------------+
         ↓ assert 宏展开（概念上）
  if (!(0 < n)) { __assert_fail("0 < n", "assert_perror.c", 23, "internal_max"); }

  输出：assert_on: assert_perror.c:23: internal_max: Assertion `0 < n' failed.
        Aborted (core dumped)          ← abort() → SIGABRT → 退出码 134

用 -DNDEBUG 编译后的同一段代码（断言被完全删除）：

  internal_max 的栈帧
  +-----------------------------+
  | arr = &data[0]              |
  | n   = 0                     |
  | best = arr[0] = 3           |     ← 循环体一次也不执行
  +-----------------------------+
  返回 3（垃圾值），程序继续运行，输出 "3"，退出码 0

  结论：断言让你在开发期**立刻**发现 bug，而不是在客户那里得到错误答案。

errno 的读取时机（必须在失败后立刻读，否则可能被覆盖）：
  fopen(...) → 返回 NULL  → errno = 2 (ENOENT)  ← 此刻读取才有效
  perror(NULL) → "No such file or directory"
  之后任何成功的库调用都可能改写 errno，所以不能事后检查。
```

【与汇编的对应】

```
; LC-3 里的"断言"：显式检查不变量，违反就打印并停机
internal_max
        LDR   R1, R0, #0          ; R1 = n（参数）
        BRp   n_ok                ; n > 0 则继续
        LEA   R0, ASSERT_MSG      ; 打印 "Assertion `0 < n' failed."
        PUTS
        HALT                      ; 立即停机 —— 相当于 abort()
n_ok
        ; ... 正常计算 ...
; 对比：正式的错误处理走"返回错误码"的路径，调用者决定怎么报告
read_file_or_report
        LD    R1, ERRNO_ADDR      ; R1 = &errno
        ; fopen 失败时把错误码写进 errno，然后返回 -1
        AND   R0, R0, #0
        ADD   R0, R0, #-1         ; 返回 -1 表示失败，但程序继续运行
        RET
```

示例 5：编译标志是廉价的第一道防线（`-Wall` vs `-Wall -Werror`）

代码 (C) —— 故意写得有问题

```
/* warn_demo.c —— 演示 -Wall 与 -Werror 的差别 */
int32_t compare (int32_t n)
{
    uint32_t i;
    for (i = 0; i < n; i++) {   /* -Wsign-compare（注意：C 里这由 -Wextra 打开） */
        if (10 == i) { return (int32_t)i; }
    }
    return -1;
}
int main (void)
{
    int32_t unused = 42;              /* -Wunused-variable */
    int32_t x;                        /* 从未初始化 */
    printf ("compare(20) = %d\n", compare (20));
    printf ("x might be %d\n", x);    /* -Wuninitialized */
    return 0;
}
```

真实编译输出（三种命令对比，括号内是真实退出码）：

```
### gcc -g -std=c99 -Wall -o warn_demo warn_demo.c          (退出码 0，仅警告)
warn_demo.c: In function 'main':
warn_demo.c:23:13: warning: unused variable 'unused' [-Wunused-variable]
warn_demo.c:27:5: warning: 'x' is used uninitialized [-Wuninitialized]
warn_demo.c:24:13: note: 'x' was declared here

### gcc -g -std=c99 -Wall -Werror -o warn_demo warn_demo.c   (退出码 1，编译失败)
warn_demo.c:23:13: error: unused variable 'unused' [-Werror=unused-variable]
warn_demo.c:27:5: error: 'x' is used uninitialized [-Werror=uninitialized]
warn_demo.c:24:13: note: 'x' was declared here
cc1: all warnings being treated as errors

### gcc -g -std=c99 -Wall -Wextra -o warn_demo warn_demo.c   (退出码 0)
warn_demo.c:13:19: warning: comparison of integer expressions of different
    signedness: 'uint32_t' {aka 'unsigned int'} and 'int32_t' {aka 'int'} [-Wsign-compare]
   13 |     for (i = 0; i < n; i++) {
      |                   ^
（另有与上面相同的两条警告）
```

【代码做什么？】

1. `-Wall` 报出 `unused` 未使用与 `x` 未初始化两条警告，但**仍然生成可执行文件**（退出码 0）——粗心的人会忽略它们。
2. 加上 `-Werror` 后同样的两条变成 `error:`，`cc1: all warnings being treated as errors`，**编译失败、退出码 1**，无法继续。
3. `-Wall` 在 C 中**不包含** `-Wsign-compare`；切换到 `-Wextra` 才报出 `uint32_t` 与 `int32_t` 的有符号/无符号比较问题。

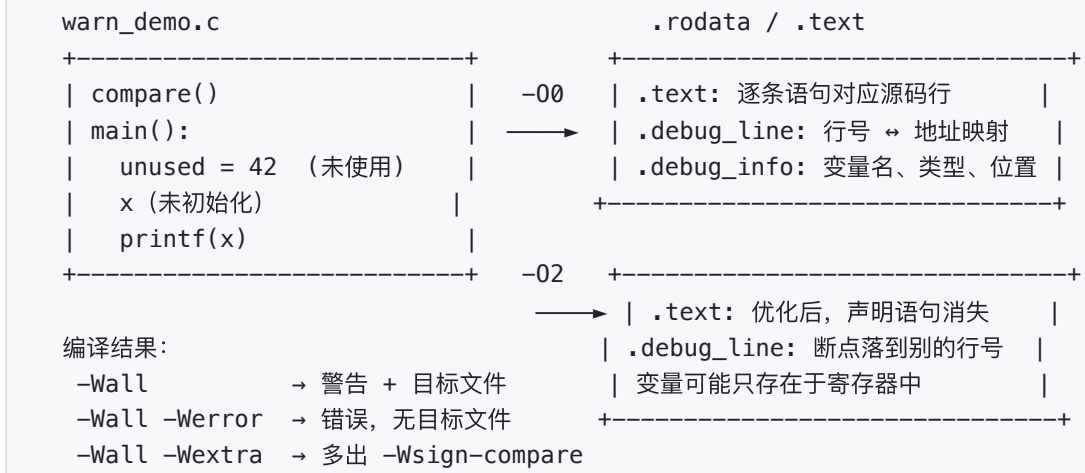
【底层机制透视】

- **课程要求的两条命令**（官方 C Coding Conventions 与 NOTES 规范）：不省略 `-std=c99 -Wall -Werror`：`gcc -g -std=c99 -Wall -Werror -o output_executable -l library1 source_file1.c ...` 典型库是 `c`（C 标准库）与 `m`（数学库）。
- **每个标志的作用**：
  - `-g`：生成调试信息（`.debug_*` 段），GDB 才能按源码行与变量名工作。
  - `-std=c99`：选定语言标准，避免编译器默认方言带来的意外（例如 `//` 注释、变长数组的行为）。
  - `-Wall`：打开一组常用警告。
  - `-Werror`：把所有警告升级为错误。这是**把「警告」变成「必须处理」**的唯一可靠手段。
  - `-O0`：禁止优化，保证「源码行号与变量」同调试器看到的一致。**注意 `-O0` 是「减 O 零」，与输出选项 `-o`（小写 o）完全不同**，官方 GDB 页面专门提醒过这一点。
  - `-O2`：开启优化。本讲实测把 `buggy_stats.c` 用 `-O2` 编译后，`break sum_first` 落在**第 21 行**而非第 18 行，声明语句的行号在调试信息里已经消失。变量还可能被优化掉，GDB 会显示 `<optimized out>`。

  - `-fsanitize=address,undefined`：插入运行期检查代码，抓越界、释放后使用、内存泄漏、整数溢出、除零等。代价是变慢。**注意：本环境只有 UBSan 部分可用，ASan 无法启动**（见示例 3 的实测报错），做内存检查请改用 Valgrind。
- **讲义 544 slide 31 的原话**：语法错误「容易避免也容易修复——打开所有警告（`-Wall`），修好所有警告与错误……但**不要靠猜**」。
- **讲义 544 slide 32 总结的技巧清单**（第 1 条就是本示例的延伸）：1) 代码阅读/结对编程；2) 避免做假设；3) 记录假设；4) 断言要求；5) 避免一件事有多种含义；6) 测边界用例；7) 在调试器里走遍所有路径；8) 用回归测试。

【内存布局图解】

同一份源码在三种编译方式下的产物对比：



官方要求的完整命令（ECE 220 Coding Conventions）：

```
gcc -g -std=c99 -Wall -Werror -o output_executable \
    -l library1 source_file1.c [source_file2.c] ...
```

调试时加 `-O0`；追求性能时换 `-O2`，但那时就不要指望逐行对照源码了。

【与汇编的对应】

```
; 「未初始化变量」在 LC-3 里的等价物：读一个从未写过的栈位置
        ; 正确做法：先初始化再使用
        AND   R1, R1, #0          ; R1 = 0（初始化）
        STR   R1, R5, #-1         ; 把 x 写进栈帧
        ; ...
        LDR   R1, R5, #-1         ; 之后读回来的才是确定值
; 错误做法：跳过初始化直接读，得到的是上一次函数调用留下的垃圾
        LDR   R1, R5, #-1         ; ← 无 BRzp 之类的检查能发现它
; 这正是 gcc 的 -Wuninitialized 与 Valgrind 能发现、而硬件发现不了的问题。
```

常见错误与调试技巧

- **在 `-O2` 下调试源码行号对不上**：断点落在错误的行，或变量显示 `<optimized out>`。**调试**：改回 `gcc -g -O0 -std=c99 -Wall -Werror` 重新编译；用 `info line file.c:24` 确认地址映射，用 `disassemble /m` 看机器码与源码的对照。
- **`print` 打印不了别的帧里的变量**：`No symbol "x" in current context`。**调试**：先 `bt` 看帧号，再 `frame 1`（或 `f 1`）切到目标帧；配合 `info locals`、`info args`、`info frame` 确认上下文。
- **程序崩溃后 `print` 无效**：官方页面提醒「如果执行已经结束，打印变量值将不起作用」。**调试**：在崩溃前设断点，或用 `run` 后立刻 `bt`；段错误刚发生时栈帧仍然完好，`bt` 与 `frame N` 仍然可用。
- **只靠 `printf` 调试**：临时打印污染代码，看不到栈帧，还常常改坏时序（尤其是在循环里）。**调试**：换成 `gdb -tui --args ./prog arg1`；用 `b file.c:24`、`display i`、`p *ptr`、`x/8xb arr`、`watch var`、`until`、`finish` 组合定位，不必删任何代码。
- **越界读写的表现时有时无**：今天不崩，明天在另一台机器上崩。原因见示例 1——越界常落在「同一数组或相邻栈数据」内，只改值不触发硬件异常。**调试**：首选 `valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog`（**本环境可用的内存检查工具**）；在其他机器上也可以 `gcc -fsanitize=address,undefined -g` 后运行（**本环境 ASan 因 `ulimit -v` 无法启动，故不要依赖它**）。
- **内存泄漏积累到程序被 OOM 杀掉**：`malloc` 成功但从不 `free`。**调试**：`valgrind --leak-check=full ./prog`，重点看 `definitely lost` 的大小与**分配位置**（Valgrind 直接给出文件名与行号）；`still reachable` 通常不用管。
- **`assert` 里写有副作用的表达式**：`assert(fclose(f) == 0);` 在 `-DNDEBUG` 的发布版里**根本不会执行**，于是文件不关闭。**调试**：`gcc -DNDEBUG` 编译一次并运行，确认行为不变；把副作用移出 `assert`。
- **把 `assert` 当成错误处理**：用户输入非法时 `assert` 会直接让程序崩溃。**调试**：区分「内部不变量」（用 `assert`）与「外部失败」（用返回值 + `errno` / `perror` 报告）；用 `errno` 时**必须在失败后立刻读取**，否则会被后续调用覆盖。

关键要点

- **编译器只抓最浅的一层错误。** 语法/类型错误由 `-Wall -Werror` 拦住；链接错误、运行期错误（越界、UB、泄漏）与逻辑错误必须靠工具与测试。**「编译通过」与「答案正确」之间没有必然联系**——本讲的 `buggy_stats.c` 零警告、退出码 0，却算错了答案。
- **`-g -std=c99 -Wall -Werror -O0` 是调试的标准起手式。** `-g` 提供行号/变量映射，`-O0` 保证映射真实；`-O2` 会让断点落错行、变量消失。发布时才切回 `-O2`。
- **GDB 的核心能力是「停在任意位置观察真实状态」。** 掌握 `break` / `run` / `next` / `step` / `until` / `finish` / `continue` 六种推进方式、`print` / `display` / `x` / `watch` 四种观察方式，以及 `bt` / `frame N` 的帧切换，就能覆盖绝大多数定位需求。`.gdbinit` 把常用断点与 `set args` 脚本化，TUI 提供源码/汇编/寄存器分窗视图。
- **内存问题的专用工具不可替代。** Valgrind 用动态二进制翻译抓越界、未初始化读与泄漏（并区分 `definitely/indirectly/possibly/still reachable`）；`-fsanitize=address,undefined` 用编译期插

桩抓得更快（但本环境因虚拟内存上限无法运行 ASan，UBSan 可用）。**绝不能靠「它这次没崩」来判断内存正确。**

- **测试纪律是唯一能覆盖逻辑错误的手段。** 用 `assert` 强制内部不变量，用返回值/`errno`/`perror` 处理外部失败；写一个打印 `PASS/FAIL` 的测试驱动，覆盖 `n = 0`、`n = 1`、`NULL`、全相等、全负数、最大值等边界，并为**每一个**发现过的 bug 补一个回归测试——`assert` 与测试的组合，才是「能跑」与「对」之间的桥。

思考题（带答案）

问题 1. 下面两个程序都被 `gcc -g -std=c99 -Wall -Werror` 成功编译并正常退出（退出码 0）。哪一个一定是错的？如何用一句话区分它们？

```
/* A */  int32_t sum_first (const int32_t* a, int32_t n)
         { int32_t t = 0; for (int32_t i = 0; i < n; i++) { t += a[i]; } return t; }
/* B 的循环条件 */  for (i = 0; i <= n; i++) { total += arr[i]; }
```

答案. B 一定是错的（越界读一个元素），A 是正确的。区分它们不能靠编译器——两者语法与类型都合法，警告级别也相同。区分手段是**测试边界用例**：让 `n = 0`（循环应执行 0 次）与检查循环条件的**相等情形**（`i == n` 时应退出）。这正是讲义 544 slide 27 的做法：测循环执行 0 次或 1 次、检查相等情形、检查两个分支方向、思考溢出。另一个手段是 `valgrind`：若数组恰好只有 `n` 个元素，B 会报 `Invalid read of size 4`。

问题 2. 为什么在 `gcc -O2` 下调试会失败，而在 `-O0` 下正常？请从「调试信息的含义」与「优化的作用」两方面回答。

答案. `-g` 生成的 DWARF 调试信息本质上是一张**映射表**：机器码地址 ↔ 源文件行号、以及「某个变量此刻存放在哪个寄存器或哪个栈偏移」。这张表只有在编译器**忠实保留**源码结构时才成立。`-O2` 会重排语句、把多个变量合并到同一寄存器、删除死代码、展开循环，于是：变量可能**根本不存在于任何存储位置**（GDB 显示 `<optimized out>`）；某段机器码可能对应多行源码或对应不到任何一行；断点按行号设置后会落在**另一段代码**上。本讲实测：同一程序用 `-O2` 编译后 `break sum_first` 落在第 21 行而不是第 18 行，声明语句在调试信息里已经不存在了。`-O0` 明确禁止这些变换，因此「源码行」与「机器指令」一一对应，调试信息才可信。

问题 3. 下面这段代码为什么是危险的？在什么情况下它会「看起来工作正常」？

```
char* buf = malloc (8);
strcpy (buf, "hello, ECE 220");
printf ("%s\n", buf);
```

答案. 危险之处：`malloc (8)` 只给了 8 字节（含结尾 `'\0'` 只够 7 个字符），而 `"hello, ECE 220"` 需要 **15** 字节（14 个字符 + `'\0'`）。`strcpy` 会写 15 字节，**越界 7 字节**，破坏堆块元数据或相邻对象——这是**未定义行为**，不是「可能出错」。它「看起来正常」的情形：`malloc` 通常从较大的 arena 里切块，越界写入的 7 字节往往落在**同一 arena 的填充区或尚未使用的空间**里，于是 `printf` 还能读回正确的字符串、程序还能正常退出（本讲实测的 `leaky.c` 就是这样：输出完全正确，Valgrind 却报了 22 个错误）。但一旦这些字节属于下一个堆块的头部或另一个对象，程序就会在**很久之后**的某次 `malloc`/`free` 上崩溃，表现为

`munmap_chunk(): invalid pointer` 之类与原始 bug 毫无关系的错误。正确写法是 `malloc (strlen(src) + 1)` 或直接用 `strdup`；定位手段是 `valgrind --leak-check=full --track-origins=yes ./prog`（本环境可用的工具），或换到 ASan 能运行的机器上用 `gcc -fsanitize=address -g`。

附录：C 语言与系统编程核心概念速查表

> 本速查表按类别汇总全课程的关键概念、代码模板、内存图与调试技巧，供复习与考试时快速检索。 所有代码模板均基于 ECE 220 的官方编码规范（4 空格缩进、始终使用花括号、`typedef struct` 风格）。

目录

1. 编译与工具链
2. 数据类型与内存表示
3. 运算符与优先级
4. 指针速查
5. 数组与指针算术速查
6. 字符串速查
7. 作用域与存储期
8. 函数与调用约定速查
9. 运行时栈与栈帧
10. 结构体、typedef 与信息隐藏
11. 动态内存管理
12. 动态内存分配器内部机制
13. 数据结构速查
14. 递归速查
15. 排序算法速查
16. 文件 I/O 速查
17. 格式化输入输出速查
18. LC-3 汇编速查
19. C 到 LC-3 翻译模板
20. 调试速查：GDB
21. 调试速查：Valgrind 与 Sanitizer

22. 错误分类与排查流程图

23. 常见内存错误图鉴

24. C 编程准则清单

---

1. 编译与工具链

官方编译命令

```
# ECE 220 标准编译命令（不要省略任何警告选项）
gcc -g -std=c99 -Wall -Werror -o output_executable \
    -l library1 [-l library2] ... source_file1.c [source_file2.c] ...
```

选项	作用	为什么重要
`-g`	生成调试符号	GDB 能看到变量名和源码行号；没有它 `gdb` 只能看汇编
`-std=c99`	使用 C99 标准	课程要求的语言子集；保证 `//` 注释、块内声明、`stdint.h` 可用
`-Wall`	打开所有常见警告	未初始化变量、隐式声明、可疑的 `=` 等
`-Werror`	把警告变成错误	强迫你面对每一个警告，而不是忽略它们
`-o out`	指定输出文件名	注意与 `-O0`（优化级别 0）不同，别写混
`-O0`	关闭优化	调试时使用；源码行与执行流一一对应
`-O2`	开启优化	变量可能被优化进寄存器或被删除，GDB 行为会"古怪"
`-lm`	链接数学库	用了 `sqrtf` 等函数时必须加，且**必须放在源文件之后**

调试与净化的编译变体

```
# 调试版：关闭优化，带符号
gcc -g -O0 -std=c99 -Wall -Werror -o prog prog.c

# 净化版：AddressSanitizer + UndefinedBehaviorSanitizer
gcc -g -O0 -std=c99 -Wall -fsanitize=address,undefined -o prog prog.c

# 泄漏检测版（Valgrind 不需要特殊编译，但需要 -g）
gcc -g -O0 -std=c99 -Wall -o prog prog.c
valgrind --leak-check=full --show-leak-kinds=all ./prog
```

多文件编译

```
gcc -g -std=c99 -Wall -Werror -o prog main.c stack.c util.c -lm
```

头文件用 `#include "my.h"`（双引号，先搜当前目录），系统头文件用 `#include <stdio.h>`（尖括号）。

---

2. 数据类型与内存表示

基本类型表（64 位机器，即 EWS 实验环境）

类型	大小	编码	最小值	最大值
`bool`	1 字节（实际只需 1 bit，编译器补齐）	0/1	`false`	`true`
`char`	8 bit	有符号整数	−128	127
`unsigned char`	8 bit	无符号整数	0	255
`short`	16 bit	有符号整数	−32 768	32 767
`unsigned short`	16 bit	无符号整数	0	65 535
`int`	32 bit	有符号整数	$\approx -2.1\times10^{9}$	$\approx 2.1\times10^{9}$
`unsigned int`	32 bit	无符号整数	0	$\approx 4.3\times10^{9}$
`long`	64 bit	有符号整数	$\approx -9.2\times10^{18}$	$\approx 9.2\times10^{18}$
`unsigned long`	64 bit	无符号整数	0	$\approx 1.8\times10^{19}$
`float`	32 bit	IEEE 754 单精度	-3.4×10^{38}	3.4×10^{38}（7 位十进制有效数字）
`double`	64 bit	IEEE 754 双精度	-1.8×10^{308}	1.8×10^{308}（16 位十进制有效数字）

> ⚠️ `int` 和 `long` 的大小**依赖平台**。课程参考明确指出：这些尺寸假设 64 位处理器。 在 32 位或 16 位平台（如 LC-3）上，`int` 可能更小。

派生类型表

类型	运算符	示例	大小	说明
指向 Z 的指针	`*`	`int *x;`	64 bit (8 字节)	一个存放 Z 类型对象地址的对象

类型	运算符	示例	大小	说明
n 个 Z 的数组	`[]`	`int x[4];`	n × sizeof(Z)	连续存放的 n 个 Z 类型对象
结构体	`struct`	`struct { int x, y; } v;`	成员大小之和（可能有填充）	顺序存放的成员对象集合

⚠️ **数组参数例外规则**（官方参考原文）：

```
void foo(int x[4]);   /* 编译器立即转换成： */
void foo(int *x);
```

> 数组作为函数参数时**立即退化为指针**，因此函数内部 `sizeof(x)` 得到的是**指针大小 8**， 而不是数组大小。这就是必须额外传递长度的根本原因。

字面量与后缀

后缀	示例	结果类型
`ul`	`42ul`	`unsigned long`
`u`	`42u`	`unsigned int`
`l`	`42l`	`long`
（无）	`42`	`int`
`f`	`42.0f`	`float`
`.`	`42.0`	`double`

名字	示例	描述	结果类型
字符字面量	`'c'`	某字符的 ASCII 值	`int`
字符串字面量	`"hello"`	NUL 结尾的 ASCII 数组	`char *`

字符串字面量的特殊性：它**隐式地在静态存储区分配并初始化**一个字符数组， 而指针本身分配在临时空间。唯一例外是用作 `char` 数组初始化器时：

```
char s[6] = "hello";   /* 等价于： */
char s[6] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

转义序列

序列	结果
`\0`	NUL（字符串结束符）
`\n`	换行
`\t`	制表符
`\\`	反斜杠
`\"`	双引号
`\'`	单引号
`\r`	回车

精确宽度类型（推荐在 ECE 220 中使用）

```
#include <stdint.h>
int8_t   /*  8 位有符号 */   uint8_t  /*  8 位无符号 */
int16_t  /* 16 位有符号 */   uint16_t /* 16 位无符号 */
int32_t  /* 32 位有符号 */   uint32_t /* 32 位无符号 */
int64_t  /* 64 位有符号 */   uint64_t /* 64 位无符号 */
intptr_t /* 能装下一个指针的整数 */
```

> **为什么 `intptr_t` 存在？** 64 位地址空间出现后，`int`（32 位）**装不下指针**了。`sbrk(intptr_t increment)` 的参数就是为此设计的。

---

3. 运算符与优先级

按优先级从高到低（常用部分）

优先级	运算符	结合性	说明
1	`()` `[]` `->` `.`	左→右	函数调用、下标、成员访问
2	`!` `~` `++` `--` `+`(一元) `-`(一元) `*`(解引用) `&`(取地址) `sizeof` `(type)`	右→左	一元运算符
3	`*` `/` `%`	左→右	乘除取模

优先级	运算符	结合性	说明
4	`+` `-`	左→右	加减（含指针算术）
5	`<<` `>>`	左→右	移位
6	`<` `<=` `>` `>=`	左→右	关系
7	`==` `!=`	左→右	相等
8	`&`	左→右	按位与
9	`^`	左→右	按位异或
10	`|`	左→右	按位或
11	`&&`	左→右	逻辑与（**短路**）
12	`||`	左→右	逻辑或（**短路**）
13	`?:`	右→左	条件运算符
14	`=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`	右→左	赋值
15	`,`	左→右	逗号

三个最容易记错的点

```c
/* ① 解引用与乘法用同一个符号，靠上下文区分 */
int *A, *B;
int c = (*A) * (*B);      /* 正确、可读 */
int c = *A**B;            /* 能编译，但没人读得懂 —— 别这么写 */

/* ② 后缀 ++ 与 * 的优先级 */
int arr[3] = {1, 2, 3};
int *p = arr;
int x = *p++;             /* 等价于 *(p++)：先取 *p=1，再 p 后移 → x=1，p 指向 arr[1] */
int y = (*p)++;           /* 取 arr[1] 的值 2，再让 arr[1] 变成 3 → y=2，arr[1]=3 */

/* ③ 赋值运算符优先级极低 */
if (a = b) { }            /* 编译通过（除非 -Werror 拦下）：这是赋值，不是比较！ */
if (a == b) { }           /* 正确写法 */
if (3 == a) { }           /* "Yoda 写法"：写错成 3 = a 会直接编译报错 */
```

位运算模板

```c
x & (1 << n)          /* 测试第 n 位是否为 1 */
x |= (1 << n)         /* 置第 n 位为 1 */
x &= ~(1 << n)        /* 清第 n 位为 0 */
x ^= (1 << n)         /* 翻转第 n 位 */
(x >> n) & 1          /* 取出第 n 位 */
x & 0xFFFF            /* 取低 16 位 */
x & (x - 1)           /* 清除最低位的 1（用于判断 2 的幂） */
(x & (x - 1)) == 0    /* x 是 2 的幂（x != 0） */
```

---

4. 指针速查

核心规则

```c
int   v = 42;
int  *p = &v;      /* p 的类型是 int*，值是 v 的地址 */
int **q = &p;      /* q 的类型是 int**，值是 p 的地址 */

*p        /* 解引用：得到 v 的值 42 */
&v        /* 取地址：得到 v 的地址（类型 int*） */
**q       /* 等价于 *p，即 v 的值 */
q         /* p 的地址 */
```

指针类型的读法：从右往左读。

```c
int  * p;      /* p 是一个指针，指向 int */
int ** p;      /* p 是一个指针，指向（指向 int 的指针） */
char * p;      /* p 是一个指针，指向 char */
```

声明陷阱（课程重点强调）

```
int *A, B;      /* A 是 int*，但 B 是 int ！！ */
int *A, *B;     /* 两个都是 int* —— 想声明多个指针必须每个都写 * */
```

指针的三条基本事实

1. **指针就是一个内存地址**——一个地址需要多少位取决于内存的可寻址性（64 位机器上是 8 字节）。
2. **编译器知道指针的类型**，因此能正确解读该地址上的比特（这是类型存在的核心意义）。
3. **声明一个指针只为指针本身分配空间**，不为它指向的东西分配空间。 如果你需要一个 `int` 供指针指向，必须另行声明。

```
int *p;         /* 只为 p 分配 8 字节，p 未初始化（野指针！） */
int  v;         /* 现在有了一个 int 供 p 指向 */
p = &v;         /* 让 p 指向它 */
*p = 42;        /* 安全地写入 */
```

指针常量与 NULL

```
int *p = NULL;          /* 空指针：明确表示"不指向任何东西" */
if (p != NULL) { ... } /* 解引用前必须检查 */
if (p) { ... }          /* 等价写法：NULL 在布尔上下文中为假 */
```

> `NULL` 通常定义为 `((void *)0)`。**永远不要解引用 NULL**—— 在 Linux 上会产生段错误（segmentation fault），因为虚拟地址 0 不映射到任何物理页。

用指针让函数修改调用者的变量（swap 模板）

```
void swap(int32_t *a, int32_t *b)
{
    int32_t tmp = *a;
    *a = *b;
    *b = tmp;
}

/* 调用：swap(&x, &y); —— 必须传地址，因为 C 是传值调用 */
```

指针的指针：让函数修改调用者的指针

```
/* 模板：分配成功时把新指针写回调用者 */
int32_t alloc_buffer(uint8_t **out, size_t n)
{
    uint8_t *p = malloc(n);
    if (p == NULL) {
        return -1;          /* 失败：不修改 *out，调用者的指针保持原样 */
    }
    *out = p;               /* 成功：写回 */
    return 0;
}

/* 调用 */
uint8_t *buf = NULL;
if (alloc_buffer(&buf, 1024) != 0) {
    fprintf(stderr, "allocation failed\n");
}
```

`realloc` 的正确写法（避免失败时泄漏）

```
/* ❌ 错误：realloc 失败返回 NULL，原指针被覆盖，内存泄漏 */
p = realloc(p, new_size);

/* ✅ 正确：用临时指针接住返回值 */
void *tmp = realloc(p, new_size);
if (tmp == NULL) {
    /* 原内存 p 仍然有效，可以继续使用或在这里 free */
    free(p);
    return -1;
}
p = tmp;
```

---

5. 数组与指针算术速查

数组的本质

```
int region[20];    /* 编译器分配 20 个连续 int，名为 region[0] … region[19] */
```

- `region` 这个表达式的类型是 `int *`，值是 `region[0]` 的地址。
- `region + N` 指向 `region[N]`，这叫**指针算术 (pointer arithmetic)**。
- `region[N]` 与 `*(region + N)` **完全等价**——方括号只是"加法 + 解引用"的简写。

指针算术的步长

```
int arr[5];
int *p = arr;

p + 1     /* 地址增加 sizeof(int) = 4 字节，不是 1 字节 */
p + 5     /* 增加 5 × 4 = 20 字节 */

/* 一般规则：ptr + n 的新地址 = (char *)ptr + n * sizeof(*ptr) */
```

> 课程幻灯片原话："*The amount added is the number of addresses required for 5 ints.*" 如果 `region` 是 `0x12345000`，那么 `region + 5` **不是** `0x12345005`， 而是 `0x12345014`（假设 `sizeof(int) == 4`）。

三种等价的遍历写法

```
int arr[5] = {10, 20, 30, 40, 50};

/* 写法 1：下标 */
for (int i = 0; i < 5; i++) { printf("%d ", arr[i]); }

/* 写法 2：指针 + 偏移 */
for (int i = 0; i < 5; i++) { printf("%d ", *(arr + i)); }

/* 写法 3：移动指针（推荐，最贴近机器） */
for (int *p = arr; p < arr + 5; p++) { printf("%d ", *p); }
```

> `arr + 5` 是 **one-past-the-end** 指针（尾后指针）。C 标准允许**计算**它并用于比较， 但**不允许解引用**它。

`sizeof` 陷阱

```
void f(int arr[10])
{
    /* ⚠️ 参数已退化为 int*，这里的 sizeof 是指针大小！ */
    size_t n = sizeof(arr);        /* 8，不是 40 */
    size_t m = sizeof(arr) / sizeof(arr[0]);  /* 8/4 = 2，完全错误！ */
}

int main(void)
{
    int arr[10];
    size_t n = sizeof(arr) / sizeof(arr[0]);  /* 40/4 = 10 ✅ 只在定义所在的函数里有效 */
    f(arr);    /* 必须另行传长度：改为 f(arr, 10) */
    return 0;
}
```

修复模板：永远显式传递长度。

```
void f(int32_t *arr, size_t n)
{
    for (size_t i = 0; i < n; i++) { /* ... */ }
}
```

数组名不是指针变量

	数组名 `arr`	指针变量 `p`
有自己的存储空间吗?	**没有**（它只是首元素地址的别名）	**有**（8 字节）
`sizeof`	整个数组的大小 (n × sizeof(T))	指针大小 (8)
可以 `arr = ...` 赋值吗?	**不可以**（不是左值）	可以
`&arr` 的类型	`int (*)[n]`（指向数组的指针）	`int **`
`&arr + 1` 的步长	n × sizeof(T)	8

二维数组的行主序 (row-major) 布局

```
int m[3][4];    /* 3 行 4 列，共 12 个 int，连续存放 */

/* 元素 m[i][j] 的地址公式： */
/*   base + (i * 4 + j) * sizeof(int)     ← 注意 4 是列数 */
```

```
内存布局（每格 4 字节）：
偏移:    0    4    8   12   16   20   24   28   32   36   40   44
      +----+----+----+----+----+----+----+----+----+----+----+----+
      |[0][0]|[0][1]|[0][2]|[0][3]|[1][0]|[1][1]|[1][2]|[1][3]|[2][0]|...
      +----+----+----+----+----+----+----+----+----+----+----+----+
        └──── 第 0 行 ────┘└──── 第 1 行 ────┘└──── 第 2 行 ────┘
```

三种"多维"声明的区别（高频考点）

```
int  m1[3][4];      /* 真正的二维数组：12 个 int 连续存放，共 48 字节 */
int *m2[3];         /* 3 个指针的数组，共 24 字节，每行可指向不同长度的内存（锯齿表） */
int (*m3)[4];       /* 一个指针，指向"含 4 个 int 的数组"，步长 16 字节 */
```

```
m1:  [ 12 个 int 连续 ]          m2:  [ p0 | p1 | p2 ]        m3:  p ──►[ int[4] ]
     m1[i] 退化为 int*                    ↓    ↓    ↓                 (只有一行)
     sizeof(m1)   = 48              [4 ints][4 ints][6 ints]      sizeof(m3) = 8
     sizeof(m1[0])= 16              sizeof(m2)   = 24             m3 + 1 步长 16
```

读法技巧：从变量名出发，先看右边，再看左边。

```
int  *m2[3];    /* m2 先与 [3] 结合 → m2 是数组；元素类型是 int* */
int (*m3)[4];   /* 括号强制 m3 先与 * 结合 → m3 是指针；指向 int[4] */
int *f(void);   /* f 先与 () 结合 → f 是函数；返回 int* */
int (*f)(void); /* 括号强制 f 先与 * 结合 → f 是指针；指向"返回 int 的函数" */
```

---

6. 字符串速查

C 字符串 = NUL 结尾的 char 数组

```
char s1[] = "hello";        /* 数组：6 字节（含 '\0'），可修改 */
char *s2  = "hello";        /* 指针：只占 8 字节，指向静态存储区的只读字面量 */
```

```
char s1[] = "hello";    ← 在栈上分配 6 字节
栈:  +-----+-----+-----+-----+-----+-----+
     | 'h' | 'e' | 'l' | 'l' | 'o' | \0  |
     +-----+-----+-----+-----+-----+-----+
     s1 是数组名，sizeof(s1) == 6

char *s2 = "hello";     ← 在栈上只分配 8 字节，指向全局数据段
栈:  +------------------+
     | s2 = 0x402008    |  (8 字节)
     +------------------+
              |
              v
全局数据段(只读): +-----+-----+-----+-----+-----+-----+
                  | 'h' | 'e' | 'l' | 'l' | 'o' | \0  |
                  +-----+-----+-----+-----+-----+-----+
```

> ⚠️ `s2[0] = 'H';` 是**未定义行为**（修改字符串字面量）。`s1[0] = 'H';` 是合法的。

标准字符串函数（`#include <string.h>`）

函数	原型	说明	陷阱
`strlen`	`size_t strlen(const char *s)`	返回**不含 NUL** 的长度	O(n)；循环里反复调用会变 $O(n^2)$
`strcpy`	`char *strcpy(char *dst, const char *src)`	复制含 NUL	**不检查目标缓冲区大小** → 溢出
`strncpy`	`char *strncpy(char *dst, const char *src, size_t n)`	最多复制 n 字节	n 用完时**可能不写 NUL**！

函数	原型	说明	陷阱
`strcat`	`char *strcat(char *dst, const char *src)`	追加	同样不检查大小
`strncat`	`char *strncat(char *dst, const char *src, size_t n)`	最多追加 n 字节	会自动补 NUL
`strcmp`	`int strcmp(const char *a, const char *b)`	比较，**返回 0 表示相等**	别写成 `if (strcmp(a,b))` 表示相等
`strncmp`	`int strncmp(const char *a, const char *b, size_t n)`	比较前 n 字节	
`strchr`	`char *strchr(const char *s, int c)`	查找字符，返回指针或 NULL	
`strstr`	`char *strstr(const char *h, const char *n)`	查找子串	
`strtok`	`char *strtok(char *s, const char *delim)`	切分（**会修改原串**）	非可重入；首参固定时返回 NULL 表示结束
`strtol`	`long strtol(const char *s, char **end, int base)`	字符串转整数	必须检查 `end` 与 `errno`

安全模板

```
/* 模板：安全复制（自己检查长度） */
int32_t safe_copy(char *dst, size_t dst_size, const char *src)
{
    size_t need = strlen(src) + 1;    /* +1 给 NUL */
    if (need > dst_size) {
        return -1;                    /* 目标太小，拒绝复制 */
    }
    memcpy(dst, src, need);
    return 0;
}
```

```
/* 模板：用 snprintf 做有界格式化（最省心的安全拼接） */
char buf[64];
int n = snprintf(buf, sizeof(buf), "value=%d, name=%s", v, name);
if (n < 0 || (size_t)n >= sizeof(buf)) {
    /* 被截断了 */
}
```

`strcmp` 的正确用法

```
if (strcmp(a, b) == 0) { /* 相等 */ }
if (strcmp(a, b) <  0) { /* a 在字典序上小于 b */ }
if (strcmp(a, b) != 0) { /* 不等 */ }

/* ❌ 常见错误：这样比较的是两个指针的地址，永远不为 0（字面量可能相同，但不可依赖） */
if (a == b) { }
```

---

7. 作用域与存储期

作用域（Scope）—— 名字在代码的哪里可见

种类	声明位置	可见范围
文件作用域	在所有函数之外，加 `static`	本文件内（推荐，避免全局污染）
全局作用域	在所有函数之外，不加 `static`	整个程序（**应尽量避免**）
函数作用域	函数体内	该函数内
块作用域	`{ }` 之内	该块内

```
static int32_t file_counter = 0;   /* 文件作用域：其他文件看不见 */

void f(void)
{
    int32_t local = 0;             /* 函数作用域 */
    {
        int32_t inner = 5;         /* 块作用域：出了这对花括号就消失 */
    }
    /* inner 在这里不存在 */
}
```

> **为什么避免全局变量？** 课程幻灯片给出的理由很实际： 想象一个 100 万行、20 个程序员的项目——你无法保证名字不冲突， 而且**你还得考虑链接进来的库代码**。收益极小，痛苦极大。 用 `static` 把作用域限制在文件内。

存储期（Storage Duration）—— 变量的寿命

存储期	何时分配/释放	存放在哪	典型例子
automatic（自动）	进入块时分配，离开块时释放	**栈**	普通局部变量
static（静态）	程序启动时分配，程序结束时释放	**全局数据段**	全局变量、`static` 局部变量、字符串字面量

存储期	何时分配/释放	存放在哪	典型例子
dynamic（动态）	显式 `malloc` 时分配，显式 `free` 时释放	**堆**	`malloc` 返回的内存

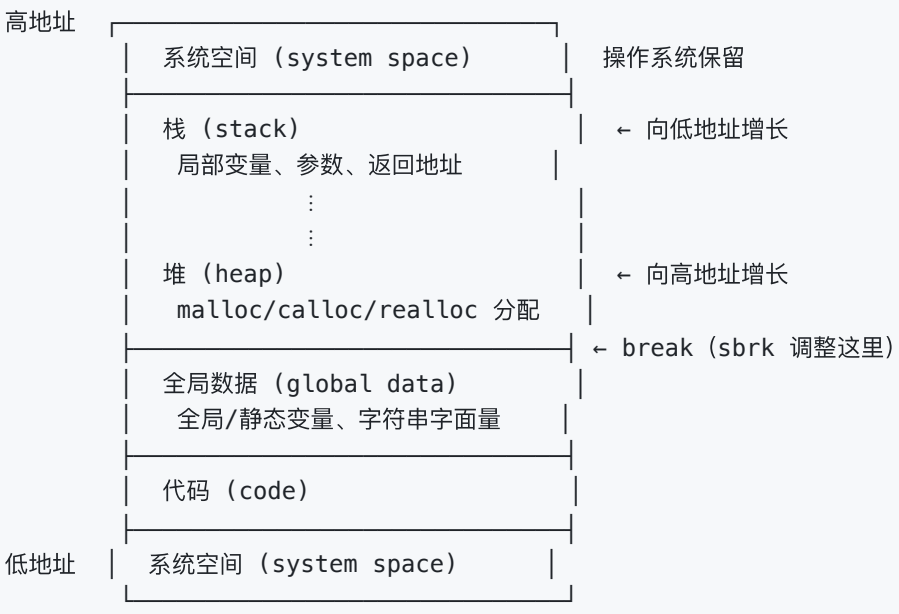
```
#include <stdio.h>

int32_t g = 1;                  /* static storage duration, 整个程序生命周期 */

void counter(void)
{
    static int32_t n = 0;       /* static storage duration, 但作用域只在本函数内 */
    n++;
    printf("called %d times\n", n);    /* 每次调用都递增，不会重置 */
}

void f(void)
{
    int32_t local = 5;          /* automatic: 每次进入 f 都重新分配 */
}
```

内存映射（Memory Map）



记忆要点：栈和堆**相向增长**，中间的空隙是它们共享的可用空间。 栈溢出（递归太深）和堆耗尽（忘记 `free`）从两端逼近同一个空隙。

---

8. 函数与调用约定速查

声明的三种形态

```
/* 1. 函数原型（声明）：告诉编译器返回类型和参数类型，不含函数体 */
int32_t max_of(int32_t a, int32_t b);

/* 2. 函数定义：包含函数体 */
int32_t max_of(int32_t a, int32_t b)
{
    return (a > b) ? a : b;
}

/* 3. 无参数的函数：用 (void) 而不是 () */
int32_t get_value(void);   /* ✅ 明确表示不接受参数 */
int32_t get_value();       /* ⚠️ 在 C 中表示"参数未指定"，不是"无参数" */
```

参数传递：一切皆传值

```
void try_to_change(int32_t x)   { x = 99; }        /* 改的是副本，调用者不变 */
void really_change(int32_t *px) { *px = 99; }      /* 改的是调用者的变量 */

int main(void)
{
    int32_t v = 1;
    try_to_change(v);        /* v 仍然是 1 */
    really_change(&v);       /* v 变成 99 */
    return 0;
}
```

底层原因：C 的参数通过栈传递（LC-3 上还有 R0–R3），传的是**实参的副本**。 要修改调用者的变量，你必须把**它的地址**（一个副本，但副本的内容是地址）传进去。

数组参数

```
/* 下面四种写法在编译器眼里完全一样，全都退化为 int32_t* */
void f(int32_t arr[10]);
void f(int32_t arr[]);
void f(int32_t *arr);
void f(int32_t *arr, size_t n);   /* ✅ 推荐：显式带长度 */
```

函数指针（高频考点）

```
/* 声明一个函数指针 */
int32_t (*cmp)(const void *, const void *);

/* 读法：从 cmp 出发 → 先看 (*cmp) 说明是指针 → 再看 (…) 说明指向函数
        → 返回 int32_t */

/* 赋值：函数名本身就是函数地址 */
cmp = my_compare;       /* ✅ 等价于 cmp = &my_compare; */

/* 调用 */
int32_t r = cmp(&a, &b);        /* ✅ 等价于 (*cmp)(&a, &b); */

/* 常见混淆 */
int32_t  *f(int32_t);           /* f 是函数，返回 int32_t* */
int32_t (*f)(int32_t);          /* f 是指针，指向返回 int32_t 的函数 */
int32_t (*tbl[4])(int32_t);     /* tbl 是数组，每个元素是函数指针（跳转表） */
```

回调与跳转表模板

```
/* 跳转表：用一个整数索引直接分派到不同函数 */
typedef int32_t (*op_fn)(int32_t, int32_t);

static int32_t op_add(int32_t a, int32_t b) { return a + b; }
static int32_t op_sub(int32_t a, int32_t b) { return a - b; }
static int32_t op_mul(int32_t a, int32_t b) { return a * b; }

static op_fn ops[] = { op_add, op_sub, op_mul };

int32_t dispatch(size_t which, int32_t a, int32_t b)
{
    if (which >= sizeof(ops) / sizeof(ops[0])) {
        return 0;
    }
    return ops[which](a, b);      /* 通过函数指针调用 */
}
```

> **与 LC-3 的联系**：这正是 LC-3 汇编里"跳转表"（`JSRR` 配合一个存地址的数组）的 C 版本。 两者都是"用数据（地址）取代一长串条件分支"。

---

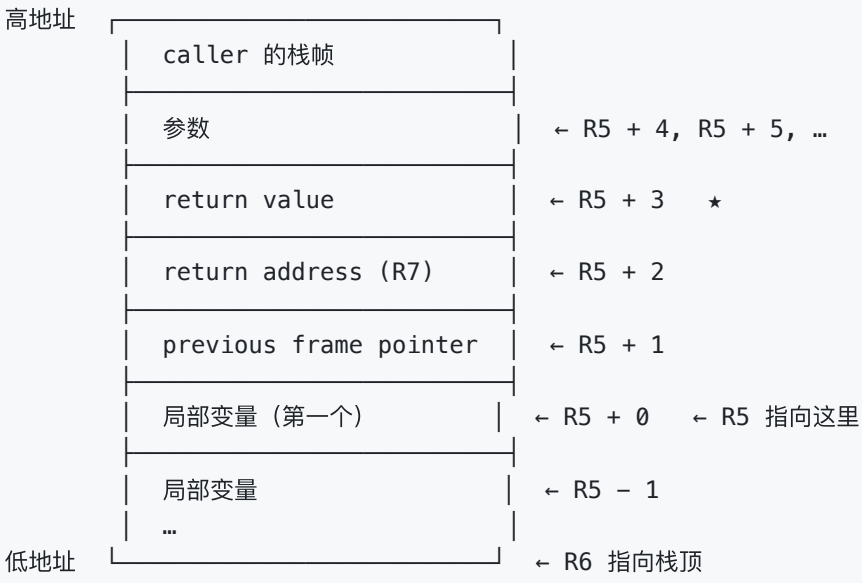
9. 运行时栈与栈帧

LC-3 调用约定寄存器

寄存器	用途	保存责任
`R0` — `R3`	传递参数、返回返回值	**caller-saved**（调用者负责保存）
`R4`	全局数据指针 (global data pointer)	全局约定
`R5`	帧指针 (frame pointer)	被调用者保存并恢复
`R6`	栈指针 (stack pointer)	被调用者保存并恢复
`R7`	返回地址 (return address)	被调用者保存并恢复

LC-3 栈帧布局

> **本节偏移已对照课程真实汇编核实**（`translate.asm` 的 `FIND_ABS`、`538-mt1-review` 讲义）。



记忆口诀：从 `R5` 往上数——`+1` 旧帧指针、`+2` 返回地址、`+3` **返回值**、`+4` 起参数； `R5+0` 往下是局部变量。

> **为什么返回值必须在 `R5+3`？** 因为调用者返回后要用**一条** `ADD R6, R6, #(nparams + 1)` 同时弹出参数**和**返回值。只有返回值紧贴在第一个参数（`R5+4`）下面，这一条指令才成立。

调用序列（caller 侧）

```
1. 把参数压栈（从右往左）          ADD R6, R6, #-1 ; STR R0, R6, #0
2. JSR 到子程序                      JSR SUB
3. 从 R0 取返回值                    ; 返回值写在 R0
4. 清理参数占用的栈空间             ADD R6, R6, #n
```

被调用序列（callee 侧）

课程真实代码（`translate.asm` 中的 `FIND_ABS`，`R5 = R6 + 0`，含 1 个局部变量）：

```
FIND_ABS
        ; ---- 建立栈帧 ----
        ADD     R6,R6,#-4       ; 4 个位置 = 3 个 linkage + 1 个局部变量
        STR     R5,R6,#1        ; 保存旧帧指针          → R5+1
        ADD     R5,R6,#0        ; 设置帧指针（R5 = R6）
        STR     R7,R5,#2        ; 保存返回地址          → R5+2
        ; ---- 函数体 ----
        LDR     R0,R5,#4        ; 读第一个参数 num      → R5+4
        STR     R0,R5,#0        ; 写局部变量            → R5+0
        STR     R0,R5,#3        ; 写返回值              → R5+3
        ; ---- 拆除栈帧 ----
        LDR     R7,R5,#2        ; 恢复返回地址
        LDR     R5,R5,#1        ; 恢复旧帧指针
        ADD     R6,R6,#3        ; 弹出局部变量与 linkage（返回值除外）
        RET                     ; = JMP R7
```

当函数**没有局部变量**时（`translate.asm` 中的 `MAIN`），帧指针设为 `R5 = R6 - 1`：

```
        ADD     R6,R6,#-3       ; 3 个位置，全部是 linkage
        STR     R5,R6,#0        ; → 对应 R5+1
        ADD     R5,R6,#-1       ; R5 = R6 - 1
        STR     R7,R5,#2        ; → R5+2
```

两种写法的**偏移量完全一致**：`R5+1`、`R5+2`、`R5+3`、`R5+4` 的含义不变。

注意：`ADD R6,R6,#3` 之后 `R6` 正好指向**返回值**所在位置（`R5+3`），所以调用者返回后可以直接读 `M[R6]` 取返回值，再用一条 `ADD R6, R6, #(nparams+1)` 弹干净。

为什么参数从右往左压栈?

因为 C 允许可变参数函数（如 `printf`）。 被调用者需要能够**先找到第一个参数**（即格式字符串），才能知道后面还有几个参数、各是什么类型。 把第一个参数放在**固定偏移**处（栈顶附近），就能保证无论传了多少个参数， 它总能被找到。如果从左往右压，第一个参数的位置会随参数个数变化，就找不到了。

```
printf("%d %s %f\n", i, s, d);
/*      ↑ 第一个参数必须在固定偏移，才能解析出后面三个 */
```

为什么同时需要 R5 和 R6?

`R6`（栈指针）在函数执行过程中会**不断移动**（每次压栈/弹栈都变）。 `R5`（帧指针）在函数建立栈帧后**保持不变**，因此可以用固定的偏移 （局部变量 `R5+0`、`R5-1`...，参数 `R5+4`、`R5+5`...）稳定地访问局部变量和参数。 这就是"用 R5 当基准，用 R6 当游标"。

编译器优化：栈帧可能不存在

课程幻灯片明确指出：**函数内部的栈帧使用不是接口问题，所以编译器可以自由优化。** 编译器可能：

- 把变量放进**寄存器**而不是栈
- **不保存 R7**（如果该函数不调用任何子程序）
- **完全不建立栈帧**

这就是为什么用 GDB 调试优化过的代码（`-O2`）时，变量会显示 `<optimized out>`。 **调试时务必用 `-O0`。**

返回局部变量指针 = 悬垂指针

```
/* ❌ 未定义行为：返回指向已销毁栈帧的指针 */
int32_t *bad(void)
{
    int32_t local = 42;
    return &local;      /* local 随栈帧销毁而失效 */
}

/* ✅ 正确：用堆分配（调用者负责 free） */
int32_t *good(void)
{
    int32_t *p = malloc(sizeof(int32_t));
    if (p != NULL) {
        *p = 42;
    }
    return p;           /* 调用者必须 free */
}
```

---

10. 结构体、typedef 与信息隐藏

定义与访问

```
typedef struct {
    int32_t x;
    int32_t y;
} point_t;

point_t p   = { 3, 4 };
point_t *pp = &p;

p.x         /* 通过结构体变量访问成员 */
pp->x       /* 通过指针访问成员：-> 就是 (*pp).x 的简写 */
(*pp).x     /* 与上一行完全等价 */
```

内存布局与填充（Padding）

```
typedef struct {
    char a;      /* 偏移 0，1 字节 */
    /* 3 字节填充 */
    int  b;      /* 偏移 4，4 字节 */
    char c;      /* 偏移 8，1 字节 */
    /* 3 字节尾部填充（为了让数组中的下一个元素对齐） */
} padded_t;

/* sizeof(padded_t) == 12，而不是 1+4+1 = 6 */
```

```
偏移:   0    1    2    3    4    5    6    7    8    9   10   11
      +----+----+----+----+----+----+----+----+----+----+----+----+
      | a  | 填充    |填充| b                   | c  |  填充       |
      +----+----+----+----+----+----+----+----+----+----+----+----+
```

调整成员顺序可以减小 sizeof：

```
typedef struct { char a; char c; int b; } packed_t;   /* sizeof == 8 */
```

> 这个技巧也是"为什么编译期能算出成员偏移"的关键：**偏移在编译期就是常量**，所以 `p->b` 会编译成一条 `LDR R1, R0, #offset` 形式的指令，**运行时没有任何查找开销**。

`typedef` 与 `enum`

```
/* 课程编码规范要求：定义 struct 和 enum 时总是使用 typedef */
typedef struct {
    float origin_x;
    float origin_y;
    float radius;
} circle;

typedef enum {
    APPLE,
    ORANGE,
    BANANA,
} fruit;
```

信息隐藏：不透明类型（Opaque Type / Handle Idiom）

头文件（接口，`mem220.h`）—— 只有声明，没有定义：

```
#if !defined(_MEM220_H)
#define _MEM220_H

#include <stdint.h>
#include <stddef.h>

/* 只前向声明，使用者看到的是一个不完整的类型 */
typedef struct memory_system memory_system_t;

memory_system_t *mem220_create(void);
void             mem220_destroy(memory_system_t *ms);
void            *mem220_allocate(memory_system_t *ms, size_t n_bytes);
void             mem220_free(memory_system_t *ms, void *ptr);

#endif /* !defined(_MEM220_H) */
```

实现文件（`mem220.c`）—— 只有这里有真正的定义：

```
struct memory_system {
    /* 私有表示：使用者完全不知道这些字段的存在 */
    uint8_t *heap_start;
    size_t   total_size;
    /* ... 空闲链表等内部状态 ... */
};
```

好处：

1. **封装**：使用者无法依赖内部表示，实现可以自由改变而不破坏调用者。
2. **不变量**：所有修改都必须经过函数，函数可以维护数据结构不变量。
3. **可测试性**：有清晰的接口边界，便于单元测试。
4. **这就是"类"的雏形**：`memory_system_t *` 相当于 `this` 指针， 每个函数第一个参数相当于一个"方法"。

头文件模板（含 include guard）

```
#if !defined(_MYMODULE_H)
#define _MYMODULE_H

#include <stdint.h>
#include <stddef.h>

typedef struct {
    int32_t value;
    char    name[32];
} record_t;

/* 函数声明，每行加注释说明契约 */
int32_t record_init(record_t *rec, const char *name, int32_t value);
void    record_print(const record_t *rec);

#endif /* !defined(_MYMODULE_H) */
```

---

11. 动态内存管理

四个函数的契约

函数	原型	成功	失败	注意
`malloc`	`void *malloc(size_t n)`	返回未初始化的 n 字节	返回 `NULL`	`malloc(0)` 返回值由实现定义，可能为 NULL
`calloc`	`void *calloc(size_t n, size_t sz)`	返回**清零**的 n×sz 字节	返回 `NULL`	会检查 n×sz 是否溢出
`realloc`	`void *realloc(void *p, size_t n)`	返回调整后的块	返回 `NULL`，**原块不变**	必须用临时指针接返回值
`free`	`void free(void *p)`	释放	—	`free(NULL)` 是合法的空操作

标准使用模板

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(void)
{
    size_t n = 10;

    /* 1. 分配 */
    int32_t *arr = malloc(n * sizeof(int32_t));
    if (arr == NULL) {                          /* 2. 必须检查 */
        fprintf(stderr, "Out of memory\n");
        return 1;
    }

    /* 3. 使用 */
    for (size_t i = 0; i < n; i++) {
        arr[i] = (int32_t)(i * i);
    }

    /* 4. 释放 */
    free(arr);
    arr = NULL;                                 /* 5. 置 NULL，防止悬垂指针 */

    return 0;
}
```

`sizeof` 的正确写法

```c
int32_t *a = malloc(10 * sizeof(int32_t));      /* ✅ 推荐：显式类型 */
int32_t *b = malloc(10 * sizeof(*b));           /* ✅ 也推荐：改类型时自动跟着变 */
int32_t *c = malloc(10 * sizeof(int));          /* ⚠️ 在本平台上恰好对，但不通用 */
int32_t *d = malloc(10 * 4);                    /* ❌ 魔法数字，禁止 */
```

动态数组增长（倍增策略）

```c
typedef struct {
    int32_t *data;
    size_t   count;
    size_t   capacity;
} vec_t;

int32_t vec_push(vec_t *v, int32_t value)
{
    if (v->count == v->capacity) {
        size_t new_cap = (v->capacity == 0) ? 4 : v->capacity * 2;
        int32_t *tmp = realloc(v->data, new_cap * sizeof(int32_t));
        if (tmp == NULL) {
            return -1;                  /* 原数据完好，调用者可继续使用 */
        }
        v->data     = tmp;
        v->capacity = new_cap;
    }
    v->data[v->count++] = value;
    return 0;
}
```

为什么倍增是 O(1) 摊还？ n 次 push 的总复制成本为 `1 + 2 + 4 + … + n/2 < n`，即 O(n)，摊到每次 push 就是 O(1)。 如果每次只加 1，总成本是 `1+2+…+n = O(n²)`，单次就是 O(n)。

倍增策略的代价（课程幻灯片量化）

以 2 倍增长为例：

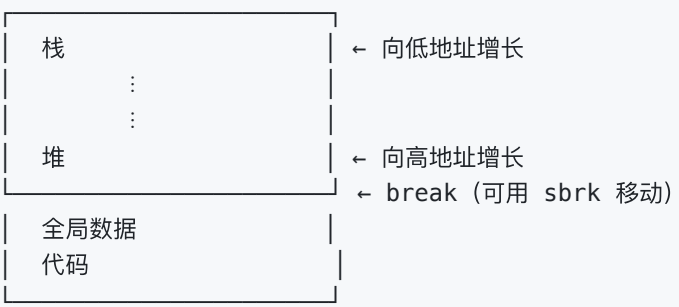
- **复制成本**：对 N 个元素，累计复制次数 ⩽ 2N
- **空间浪费**：约 **38%**（因为平均只用了一半容量）

删除：如果不要求顺序，把最后一个元素复制到被删位置，然后 `count--`，是 O(1)。 如果要求保序，删除是 O(n)。

---

12. 动态内存分配器内部机制

堆与 break



`sbrk` 系统调用

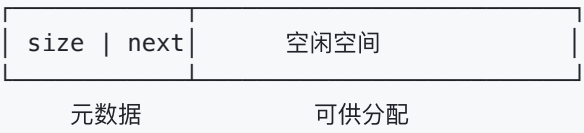
```
#include <unistd.h>
void *sbrk(intptr_t increment);
```

- 请求把 break 移动 `increment` 字节。
- **返回移动前**的 break 地址（即新分配区域的起始地址）。
- 失败时返回 `(void *)-1`。
- `increment` 可以是负数，用于**收缩**堆。

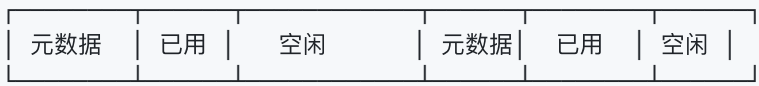
> 为什么参数是 `intptr_t` 而不是 `int`？因为 64 位地址空间下 `int`（32 位）装不下指针。

分配器的核心数据结构

空闲链表（free list）—— 每个空闲块头部藏有元数据：



分配后的堆布局：



为什么 `free(ptr)` 知道要释放多大？ 因为 `malloc` 在返回给用户的指针**前面** 存了块的元数据（大小、状态）。`free` 往后退几个字节就能读到它。 这也解释了为什么 **`free` 只能释放 `malloc` 返回的原始指针**——传一个偏移过的指针（如 `free(p + 1)`）会读到错误的元数据并破坏堆。

分配策略

策略	做法	优点	缺点
first-fit	用第一个足够大的空闲块	快	低地址处产生大量小碎片
best-fit	用最小的足够大的空闲块	空间利用率好	需要搜索整个空闲表；产生难以利用的微小碎片
worst-fit	用最大的空闲块	留下的剩余块较大	大块很快被消耗

课程讲授的是 **best-fit logarithmic allocator**（对数最佳适配分配器）， 它在 20 世纪被广泛使用了几十年。其思想是：把空闲块按**大小的对数**分桶， 这样寻找"最小的足够大"的块时，只需检查比请求大小略大的桶，而不是扫描整个链表。

分裂与合并

- **分裂 (splitting)**：一个空闲块比请求大得多时，切成"分配出去的部分 + 剩余空闲块"。 若剩余部分太小（小于最小块大小），干脆整块给出去。
- **合并 (coalescing)**：`free` 时，若相邻块也是空闲的，把它们合并成一个大块， 否则会产生**外部碎片**（总空闲够，但没有一个连续块够大）。

两类碎片

	定义	成因	缓解
内部碎片	分配出去的块**内部**未被使用的字节	对齐、最小块大小、分配粒度	减小粒度
外部碎片	空闲空间**总量够**但没有一个足够大的连续块	分配/释放的时序	合并相邻空闲块

---

13. 数据结构速查

动态数组 vs 链表

维度	动态数组（`vec_t`）	链表（`node_t *`）
随机访问 `[i]`	**O(1)**	O(n)
头部插入	O(n)	**O(1)**
尾部插入	O(1) 摊还	O(1)（带尾指针）
中间插入/删除	O(n)	O(1)（已有前驱指针时）
按值查找	O(n)	O(n)
每元素额外内存	无	**8 字节**（next 指针）
缓存局部性	**好**（连续）	差（分散在堆上）
增长能力	受连续空间限制	不受限制（只要有零散空间）
适用场景	频繁随机访问、元素数可预估	频繁中间插删、元素数未知且大

链表节点定义与核心操作

```
typedef struct node {
    int32_t      value;
    struct node *next;      /* 自引用：必须是指针，不能是 struct node */
} node_t;
```

```
head
  |
  v
+------+------+    +------+------+    +------+------+
|  10  |  ---|--->|  20  |  ---|--->|  30  | NULL |
+------+------+    +------+------+    +------+------+
 0x1000            0x2040            0x3080
  （节点在堆上分散，靠指针串起来，因此不能做指针算术）
```

遍历模板：

```
for (node_t *p = head; p != NULL; p = p->next) {
    /* 处理 p->value */
}
```

头插法（O(1)）：

```
node_t *push_front(node_t *head, int32_t value)
{
    node_t *n = malloc(sizeof(node_t));
    if (n == NULL) {
        return head;            /* 失败：返回原链表 */
    }
    n->value = value;
    n->next  = head;
    return n;                   /* 新头 */
}
```

删除（指针的指针写法，消除"删除头结点"的特例）：

```
int32_t list_remove(node_t **head, int32_t value)
{
    node_t **link = head;               /* link 指向"指向当前节点的指针" */
    while (*link != NULL) {
        if ((*link)->value == value) {
            node_t *victim = *link;
            *link = victim->next;       /* 让前驱（或 head）跳过它 */
            free(victim);
            return 0;
        }
        link = &(*link)->next;
    }
    return -1;                          /* 未找到 */
}
```

销毁整个链表（先存 next 再 free）：

```
void list_destroy(node_t *head)
{
    while (head != NULL) {
        node_t *next = head->next;      /* ⚠️ 必须先保存！ */
        free(head);
        head = next;
    }
}
```

> **经典 bug**：先 `free(head)` 再读 `head->next` 就是 use-after-free。 顺序必须是"存 next → free 当前 → 前进"。

二叉树

```
typedef struct tree_node {
    int32_t          value;
    struct tree_node *left;
    struct tree_node *right;
} tree_node_t;
```

BST 不变式：对每个节点，**左子树所有值 < 节点的值 < 右子树所有值**。

```
              50
            /    \
          30      70
         /  \    /  \
       20   40  60   80

中序遍历 (inorder, 左-根-右): 20 30 40 50 60 70 80   ← 升序!
前序遍历 (preorder, 根-左-右): 50 30 20 40 70 60 80
后序遍历 (postorder, 左-右-根): 20 40 30 60 80 70 50
```

遍历	顺序	典型用途
前序	根 → 左 → 右	复制树、序列化、打印目录结构
中序	左 → 根 → 右	**BST 升序输出**
后序	左 → 右 → 根	**释放整棵树**、计算子树聚合值

递归模板：

```
void inorder(const tree_node_t *root)
{
    if (root == NULL) {          /* 基本情况：空树 */
        return;
    }
    inorder(root->left);
    printf("%d ", root->value);
    inorder(root->right);
}

void tree_destroy(tree_node_t *root)
{
    if (root == NULL) {
        return;
    }
    tree_destroy(root->left);    /* 必须先释放子树 */
    tree_destroy(root->right);
    free(root);                  /* 最后释放自己（后序） */
}
```

BST 查找：

```
const tree_node_t *bst_find(const tree_node_t *root, int32_t key)
{
    while (root != NULL) {
        if (key == root->value) { return root; }
        root = (key < root->value) ? root->left : root->right;
    }
    return NULL;
}
```

> **复杂度**：O(树高)。平衡树高 ≈ $\log_2 n$；但**退化成链表的树高 = n**， 此时查找退化为 O(n)。这就是为什么需要平衡树。

---

14. 递归速查

递归的两个必要部分

1. **基本情况 (base case)**：不再递归、直接返回的情形。**没有它 = 无限递归 = 栈溢出。**
2. **递归情况 (recursive case)**：把问题化归为**更小的同类问题**。

标准模板

```
int32_t factorial(int32_t n)
{
    if (n <= 1) {              /* 基本情况 */
        return 1;
    }
    return n * factorial(n - 1);   /* 递归情况 */
}
```

递归的代价：栈帧链

```
factorial(4) 调用过程（栈向低地址增长）：

调用 factorial(4)   →  调用 factorial(3)  →  调用 factorial(2)  →  调用 factorial(1)
                                                                        ↓ 返回 1
                                                                   ↓ 返回 2*1=2
                                                              ↓ 返回 3*2=6
                                                         ↓ 返回 4*6=24

栈帧布局（每一层都有自己独立的一份 n）：
  高地址 ┌──────────────────┐
         │ factorial(4)     │  n=4，等着 3 的结果
         ├──────────────────┤
         │ factorial(3)     │  n=3，等着 2 的结果
         ├──────────────────┤
         │ factorial(2)     │  n=2，等着 1 的结果
         ├──────────────────┤
         │ factorial(1)     │  n=1，直接返回 1  ← 栈顶（R6 指向这里）
  低地址 └──────────────────┘
```

关键洞察：递归的"记忆"就是**栈**。每层调用有自己的局部变量和参数副本， 所以递归天然支持"回到上一层继续算"。这也是为什么递归深度受**栈大小**限制 （默认约 8 MB，即约 10 万层简单调用）。

递归 vs 迭代

维度	递归	迭代
代码可读性	树、回溯、分治类问题**明显更好**	线性扫描类问题更好
空间开销	O(深度) 栈帧	O(1)
速度	较慢（函数调用开销）	较快
栈溢出风险	**有**（深度受栈大小限制）	无
适用	自相似结构（树、分形、回溯）	简单累积、线性遍历

尾递归 (Tail Recursion)

递归调用是函数的**最后一个动作**（返回值直接就是递归调用的返回值）时， 编译器**可能**优化成循环，消除栈帧增长。但 **C 标准不保证**这种优化， 所以不要把"不会栈溢出"当作前提。

```
/* 尾递归形式：把累积量作为参数传下去 */
int32_t fact_tail(int32_t n, int32_t acc)
{
    if (n <= 1) {
        return acc;
    }
    return fact_tail(n - 1, n * acc);   /* 递归调用是最后一步 */
}
```

回溯 (Backtracking) 模板

```
/*
 * 通用回溯模板：
 *   1. 判断是否已经找到解 / 走不通 → 返回
 *   2. 标记当前位置（做选择）
 *   3. 递归探索所有邻居
 *   4. 撤销标记（回溯）
 */
int32_t solve(grid_t *g, int32_t r, int32_t c)
{
    /* 越界或已访问或不可通行 */
    if (r < 0 || r >= g->rows || c < 0 || c >= g->cols) { return 0; }
    if (g->cell[r][c] != OPEN)                          { return 0; }

    /* 到达目标 */
    if (r == g->goal_r && c == g->goal_c)               { return 1; }

    g->cell[r][c] = VISITED;        /* 做选择：标记 */

    /* 探索四个方向 */
    if (solve(g, r + 1, c) || solve(g, r - 1, c) ||
        solve(g, r, c + 1) || solve(g, r, c - 1)) {
        return 1;                   /* 找到解，逐层返回 */
    }

    g->cell[r][c] = OPEN;           /* 撤销选择：回溯 */
    return 0;
}
```

> **回溯的要点**：第 4 步的"撤销"必须与第 2 步的"标记"严格配对。 忘了撤销，算法就只能探索一条路径；撤销错了，会重复访问导致死循环。

递归树与复杂度：朴素 Fibonacci

```
int32_t fib(int32_t n)
{
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}
```

```
                  fib(5)
                 /      \
             fib(4)        fib(3)
            /     \       /     \
        fib(3)  fib(2)  fib(2)  fib(1)
        /    \   /   \   /   \
    fib(2) fib(1) ...  ...  ...
     /  \
  fib(1) fib(0)

节点总数 ≈ 2^n  →  时间复杂度 O(2^n)（实际约 O(1.618^n)）
注意 fib(3) 被计算了 2 次，fib(2) 被计算了 3 次 —— 大量重复子问题！
```

修复：动态规划/记忆化（memoization），把已算过的结果存起来，降到 O(n)。

Towers of Hanoi

```
n 个盘子需要 2^n - 1 次移动。
移动 n 个盘子的方法：
   1. 把上面 n-1 个盘子从 A 移到 B（借助 C）
   2. 把最大的盘子从 A 移到 C
   3. 把 n-1 个盘子从 B 移到 C（借助 A）
```

---

15. 排序算法速查

总览对比表

算法	最好	平均	最坏	额外空间	稳定	特点
插入排序	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✅	近乎有序时极快；小数组最快
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	❌	交换次数最少（恰好 n−1 次）
冒泡排序	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✅	加提前退出标志后近乎有序时 O(n)
归并排序	O(n log n)	O(n log n)	O(n log n)	**O(n)**	✅	稳定；可外排序；需要额外数组
快速排序	O(n log n)	O(n log n)	**$O(n^2)$**	O(log n)	❌	实际最快；最坏出现在已排序+糟糕选主元
堆排序	O(n log n)	O(n log n)	O(n log n)	O(1)	❌	原地且最坏有保证

算法	最好	平均	最坏	额外空间	稳定	特点
计数排序	O(n+k)	O(n+k)	O(n+k)	O(k)	✅	非比较；仅适合小范围整数

> **比较排序的下界**：任何只通过"比较两个元素"来决定顺序的算法， 最坏情况下至少要 Ω(n log n) 次比较（决策树有 n! 个叶子，高度 ⩾ $\log_2(n!) \approx n \log n$）。 这就是为什么计数排序必须"不比较"才能打破这个界。

插入排序模板

```
void insertion_sort(int32_t *a, size_t n)
{
    for (size_t i = 1; i < n; i++) {
        int32_t key = a[i];        /* 取出待插入的元素 */
        size_t  j   = i;
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];       /* 更大的元素整体右移一格 */
            j--;
        }
        a[j] = key;                /* 插到正确位置 */
    }
}
```

归并排序模板

```
static void merge(int32_t *a, int32_t *tmp, size_t lo, size_t mid, size_t hi)
{
    size_t i = lo, j = mid, k = lo;

    while (i < mid && j < hi) {
        /* 用 <= 保证稳定性：相等时优先取左边的 */
        tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    }
    while (i < mid) { tmp[k++] = a[i++]; }
    while (j < hi)  { tmp[k++] = a[j++]; }

    for (size_t m = lo; m < hi; m++) {
        a[m] = tmp[m];
    }
}

void merge_sort_rec(int32_t *a, int32_t *tmp, size_t lo, size_t hi)
{
    if (hi - lo < 2) {             /* 0 或 1 个元素：已有序 */
        return;
    }
    size_t mid = lo + (hi - lo) / 2;
    merge_sort_rec(a, tmp, lo, mid);
    merge_sort_rec(a, tmp, mid, hi);
    merge(a, tmp, lo, mid, hi);
}

void merge_sort(int32_t *a, size_t n)
{
    int32_t *tmp = malloc(n * sizeof(int32_t));
    if (tmp == NULL) {
        return;                    /* 分配失败：放弃（或回退到插入排序） */
    }
    merge_sort_rec(a, tmp, 0, n);
    free(tmp);
}
```

快速排序模板（Lomuto 分区）

```
static size_t partition(int32_t *a, size_t lo, size_t hi)
{
    /* hi 位置作为主元 */
    int32_t pivot = a[hi];
    size_t  i     = lo;            /* a[lo..i) 都是 <= pivot */

    for (size_t j = lo; j < hi; j++) {
        if (a[j] <= pivot) {
            int32_t t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
        }
    }
    int32_t t = a[i]; a[i] = a[hi]; a[hi] = t;   /* 主元归位 */
    return i;                      /* 主元的最终下标 */
}

void quick_sort_rec(int32_t *a, size_t lo, size_t hi)
{
    if (hi <= lo || hi - lo < 2) {
        return;
    }
    size_t p = partition(a, lo, hi - 1);
    if (p > lo)     { quick_sort_rec(a, lo, p); }
    if (p + 1 < hi) { quick_sort_rec(a, p + 1, hi); }
}
```

> **主元选择很重要**：总是取最后一个元素作主元，在**已排序**输入上会退化为 $O(n^2)$，且递归深度 $O(n)$。 实践中用"三者取中"（median-of-three）或随机化来避免。

`qsort` 与比较函数

```
#include <stdlib.h>

/* 比较函数的契约：返回 <0 表示 a 应排在 b 前；0 表示等价；>0 表示 a 应排在 b 后 */
int cmp_int_asc(const void *pa, const void *pb)
{
    int32_t a = *(const int32_t *)pa;
    int32_t b = *(const int32_t *)pb;
    if (a < b) { return -1; }
    if (a > b) { return  1; }
    return 0;
    /* ⚠️ 不要写 return a - b; —— 对 int32_t 可能溢出！ */
}

/* 调用 */
int32_t arr[100];
qsort(arr, 100, sizeof(arr[0]), cmp_int_asc);
```

按结构体不同字段排序：

```
typedef struct { char name[32]; int32_t score; } student_t;

static int cmp_by_score(const void *pa, const void *pb)
{
    const student_t *a = pa;
    const student_t *b = pb;
    if (a->score < b->score) { return -1; }
    if (a->score > b->score) { return  1; }
    return strcmp(a->name, b->name);       /* 分数相同时按名字，保证确定性 */
}

qsort(students, n, sizeof(student_t), cmp_by_score);
```

---

16. 文件 I/O 速查

标准流

流	含义	缓冲
`stdin`	标准输入（默认键盘 / 管道 / 重定向）	行缓冲或全缓冲
`stdout`	标准输出（默认终端 / 管道 / 重定向）	行缓冲（终端）/ 全缓冲（重定向）
`stderr`	标准错误（**不缓冲**）	无缓冲 —— 所以崩溃前的错误信息总能显示

`fopen` 模式

模式	含义	文件不存在时	文件存在时
`"r"`	只读	返回 `NULL`	从开头读
`"w"`	只写	创建	**截断为 0 字节**
`"a"`	追加写	创建	从末尾写
`"r+"`	读写	返回 `NULL`	从开头读写（不截断）
`"w+"`	读写	创建	截断
`"a+"`	读 + 追加	创建	读从开头，写从末尾
加 `b`	二进制模式（`"rb"`，`"wb"` 等）		Unix 上等同于文本模式

常用函数

函数	用途	返回值
`fopen(path, mode)`	打开	`FILE *` 或 `NULL`

函数	用途	返回值
`fclose(fp)`	关闭（**必须**）	0 成功，`EOF` 失败
`fprintf(fp, fmt, ...)`	格式化写	写入的字符数，负值表示错误
`fscanf(fp, fmt, ...)`	格式化读	**成功赋值的项数**（不是字符数！）
`fgetc(fp)` / `getc(fp)`	读一个字符	`int`（**必须是 int 才能装下 EOF**）
`fputc(c, fp)`	写一个字符	写的字符，或 `EOF`
`fgets(buf, n, fp)`	读一行（最多 n−1 字符）	`buf` 或 `NULL`（EOF/错误）
`fputs(s, fp)`	写字符串（不加换行）	非负值或 `EOF`
`fread(p, sz, n, fp)`	二进制读	**实际读到的元素个数**
`fwrite(p, sz, n, fp)`	二进制写	实际写入的元素个数
`feof(fp)`	是否已到文件尾	非零表示已到尾
`ferror(fp)`	是否出错	非零表示出错
`fflush(fp)`	强制把缓冲区写出	0 成功
`rewind(fp)` / `fseek`	移动文件位置	—
`remove(path)`	删除文件	0 成功
`perror(s)`	打印 `s` + `errno` 对应的错误信息	—

读取整个文件的模板

```c
#include <stdio.h>
#include <stdlib.h>

int32_t read_whole_file(const char *path, char **out, size_t *out_len)
{
    FILE *fp = fopen(path, "r");
    if (fp == NULL) {
        perror(path);              /* 打印 "path: No such file or directory" */
        return -1;
    }

    size_t cap  = 4096;
    size_t len  = 0;
    char  *buf  = malloc(cap);
    if (buf == NULL) {
        fclose(fp);
        return -1;
    }

    size_t got;
    while ((got = fread(buf + len, 1, cap - len, fp)) > 0) {
        len += got;
        if (len == cap) {                    /* 缓冲区满了，扩容 */
            char *tmp = realloc(buf, cap * 2);
            if (tmp == NULL) {
                free(buf);
                fclose(fp);
                return -1;
            }
            buf = tmp;
            cap *= 2;
        }
    }

    if (ferror(fp)) {                        /* 区分 EOF 与真错误 */
        free(buf);
        fclose(fp);
        return -1;
    }

    fclose(fp);
    *out     = buf;
    *out_len = len;
    return 0;
}
```

逐行读取的模板

```
char line[256];
while (fgets(line, sizeof(line), fp) != NULL) {
    /* 去掉行尾换行 */
    size_t n = strlen(line);
    if (n > 0 && line[n - 1] == '\n') {
        line[n - 1] = '\0';
    } else if (n == sizeof(line) - 1) {
        /* ⚠️ 行太长，缓冲区被填满且没有读到换行 —— 剩余部分还在文件里！
           必须把这一行剩下的字符读掉，否则会被当成下一行。 */
        int c;
        while ((c = fgetc(fp)) != '\n' && c != EOF) { }
    }
    /* 处理 line */
}
```

`while (!feof(f))` 为什么是错的

```
/* ❌ 错误写法 */
while (!feof(fp)) {
    fgets(line, sizeof(line), fp);
    printf("%s", line);          /* 最后一次循环会重复打印上一行！ */
}

/* ✅ 正确写法：用读取函数的返回值判断 */
while (fgets(line, sizeof(line), fp) != NULL) {
    printf("%s", line);
}
```

原因：`feof()` 只有在**尝试读取并撞到文件尾之后**才变成真。 它是"**已经**到过 EOF"的标志，不是"**将要**到 EOF"的预测。 所以 `while (!feof(f))` 会在最后一次多转一圈，此时 `fgets` 什么都没读到、 缓冲区仍是上次的内容，于是重复输出。

命令行重定向与管道

```
./prog < input.txt            # stdin 来自文件
./prog > output.txt           # stdout 写入文件（截断）
./prog >> output.txt          # stdout 追加
./prog 2> errors.txt          # stderr 重定向
./prog > out.txt 2>&1         # stdout 和 stderr 都到同一个文件
./prog < in.txt | sort | uniq -c    # 管道：程序的 stdout 接下一个程序的 stdin
```

> **程序设计原则**：让程序默认从 `stdin` 读、向 `stdout` 写、错误写到 `stderr`。 这样它就能自然地参与 Unix 管道，无需知道文件名。

解析文本的模板

```c
/* 用 strtok 切分（注意：会破坏原字符串，把分隔符改成 '\0'） */
char *save = NULL;
for (char *tok = strtok_r(line, " \t\n", &save);
     tok != NULL;
     tok = strtok_r(NULL, " \t\n", &save)) {
    /* 处理 tok */
}

/* 用 strtol 安全地把字符串转成整数 */
char *end;
errno = 0;
long v = strtol(tok, &end, 10);
if (end == tok || *end != '\0' || errno == ERANGE) {
    /* 转换失败或溢出 */
}

/* 用 sscanf 从字符串里提取格式化字段（比 strtok 更适合固定格式） */
int y, m, d;
if (sscanf(line, "%d-%d-%d", &y, &m, &d) == 3) {
    /* 成功解析 3 个字段 */
}
```

---

17. 格式化输入输出速查

`printf` / `scanf` 格式说明符（官方速查表）

说明符	格式描述	`printf` 类型	`scanf` 类型
`%c`	ASCII 字符	`char`	`char *`
`%d`	有符号十进制整数	`int`	`int *`
`%f`	带小数部分的十进制	`float`	`float *`
`%hd`	有符号十进制整数	`short`	`short *`
`%hhd`	有符号十进制整数	`char`	`char *`
`%hhu`	无符号十进制整数	`unsigned char`	`unsigned char *`
`%hhx`	十六进制	`unsigned char`	`unsigned char *`
`%hu`	无符号十进制整数	`unsigned short`	`unsigned short *`
`%hx`	十六进制	`unsigned short`	`unsigned short *`
`%ld`	有符号十进制整数	`long`	`long *`
`%lf`	带小数部分的十进制	`double`	`double *`

说明符	格式描述	`printf` 类型	`scanf` 类型
`%lu`	无符号十进制整数	`unsigned long`	`unsigned long *`
`%lx`	十六进制	`unsigned long`	`unsigned long *`
`%p`	十六进制地址	`T *`（任意指针）	`T **`
`%s`	ASCII 字符串	`char *`	`char *`
`%u`	无符号十进制整数	`unsigned int`	`unsigned int *`
`%x`	十六进制	`unsigned int`	`unsigned int *`

关键差异：`printf` 传值，`scanf` 传地址

```
int32_t v;
scanf("%d", &v);          /* ✅ scanf 需要地址 */
printf("%d\n", v);        /* ✅ printf 需要值 */
scanf("%d", v);           /* ❌ 漏了 & —— 编译警告，运行时段错误 */
```

唯一的例外：`%s` 在 `scanf` 中传的**已经是数组名**（即地址），所以不加 `&`：

```
char name[64];
scanf("%s", name);        /* ✅ name 本身就是地址 */
scanf("%s", &name);       /* 类型不同（char(*)[64]）但地址值相同，能跑，不推荐 */
```

三个经典陷阱

```
/* ① scanf 的 %c 会匹配空格和换行 —— 这通常不是你要的 */
scanf("%c", &ch);         /* ❌ 可能读到上一行残留的 '\n' */
scanf(" %c", &ch);        /* ✅ 格式串前面加一个空格：跳过所有前导空白 */

/* ② scanf 的 %s 不检查缓冲区大小 —— 缓冲区溢出风险 */
char buf[10];
scanf("%s", buf);                 /* ❌ 输入 "verylongstring" 就溢出了 */
scanf("%9s", buf);                /* ✅ 限制宽度为 sizeof(buf)-1 */

/* ③ scanf 的 %f 对应 float*，但 double 必须用 %lf */
float  f;  scanf("%f",  &f);     /* ✅ */
double d;  scanf("%f",  &d);     /* ❌ 类型不匹配，未定义行为 */
double d;  scanf("%lf", &d);     /* ✅ */
```

检查返回值（必须做）

```c
/* printf 返回写入的字符数，负值表示出错 */
int n = printf("value = %d\n", v);
if (n < 0) {
    perror("printf");
}

/* scanf 返回成功赋值的项数，不是字符数！ */
int count = scanf("%d %d", &a, &b);
if (count != 2) {
    /* 可能是输入格式不对，或者提前遇到 EOF */
    fprintf(stderr, "Expected two integers, got %d\n", count);
}
```

常用格式控制

```c
printf("%5d\n",   42);       /* 宽度 5，右对齐："   42" */
printf("%-5d|\n", 42);       /* 左对齐："42   |" */
printf("%05d\n",  42);       /* 补零："00042" */
printf("%.3f\n",  3.14159);  /* 3 位小数："3.142" */
printf("%8.2f\n", 3.14159);  /* 宽度 8，2 位小数："    3.14" */
printf("%x\n",    255);      /* 十六进制："ff" */
printf("%#x\n",   255);      /* 带前缀："0xff" */
printf("%X\n",    255);      /* 大写："FF" */
printf("%e\n",    1234.5);   /* 科学计数法 */
printf("%%\n",    0);        /* 输出一个百分号 */
printf("%zu\n",   sizeof(int)); /* size_t 必须用 %zu */
```

---

18. LC-3 汇编速查

寄存器

寄存器	名称与用途
`R0` — `R3`	通用；**caller-saved**；用于传参数和返回值
`R4`	全局数据指针 (global data pointer)
`R5`	帧指针 (frame pointer)
`R6`	栈指针 (stack pointer)
`R7`	返回地址 (return address)

内存映射 I/O 寄存器

地址	名称	作用
`xFE00`	KBSR (Keyboard Status Register)	**bit 15** = 1 表示有新按键可读
`xFE02`	KBDR (Keyboard Data Register)	**bit 7:0** = 按键的 ASCII 码
`xFE04`	DSR (Display Status Register)	**bit 15** = 1 表示可以写下一个字符
`xFE06`	DDR (Display Data Register)	**bit 7:0** = 要显示的字符

轮询 I/O 模板

```
; ---- 从键盘读一个字符（busy-wait）----
POLL_IN
        LDI     R1, KBSR        ; R1 = 键盘状态寄存器的内容
        BRz     POLL_IN         ; 若 bit15 == 0（无按键），继续轮询
        LDI     R0, KBDR        ; 读入字符到 R0
        RET

KBSR    .FILL   xFE00
KBDR    .FILL   xFE02
```

```
; ---- 向显示器写一个字符（busy-wait）----
POLL_OUT
        ST      R0, SAVE_R0     ; 保存要输出的字符（LDI 会破坏 R1，但 R0 需保留）
        LDI     R1, DSR
        BRz     POLL_OUT        ; 若显示器忙，继续等待
        LD      R0, SAVE_R0
        STI     R0, DDR         ; 写字符到显示数据寄存器
        RET

DSR     .FILL   xFE04
DDR     .FILL   xFE06
SAVE_R0 .BLKW   1
```

常用指令

> 下表是**指令速查**，不是一段可汇编的程序：其中的 `LABEL`、`value`、`n`、`"text"` 都是**占位符**， 用的时候要换成真实的标签名、立即数或字符串。

```
; ---- 算术与逻辑 ----
ADD     R0, R1, R2              ; R0 = R1 + R2（寄存器相加）
ADD     R0, R1, #5              ; R0 = R1 + 5（立即数，5 位补码，范围 -16..15）
AND     R0, R1, R2              ; R0 = R1 & R2
NOT     R0, R1                  ; R0 = ~R1
; 注意：没有 SUB、MUL、DIV —— 用 ADD 配合取负实现

; ---- 数据搬移 ----
LD      R0, LABEL               ; R0 = M[PC + offset]（PC 相对寻址，直接加载）
LDI     R0, LABEL               ; R0 = M[M[PC + offset]]（间接加载，两次访存）
LDR     R0, R1, #4              ; R0 = M[R1 + 4]（基址 + 偏移）
LEA     R0, LABEL               ; R0 = PC + offset（加载地址本身，不访存）
ST      R0, LABEL               ; M[PC + offset] = R0
STI     R0, LABEL               ; M[M[PC + offset]] = R0（间接存储）
STR     R0, R1, #4              ; M[R1 + 4] = R0

; ---- 控制转移 ----
BRnzp   LABEL                   ; 无条件跳转
BRz     LABEL                   ; 零则跳转（Z=1，即上一次运算结果为 0）
BRnp    LABEL                   ; 非零则跳转
BRn     LABEL                   ; 负数则跳转
BRp     LABEL                   ; 正数则跳转
JMP     R1                      ; 跳转到 R1 中的地址
JSR     LABEL                   ; 子程序调用；R7 = 返回地址（PC 相对，11 位偏移）
JSRR    R1                      ; 子程序调用；R7 = 返回地址，跳转到 R1（5 位寄存器）
RET                             ; = JMP R7，从子程序返回

; ---- TRAP ----
TRAP    x20                     ; GETC：读一个字符到 R0
TRAP    x21                     ; OUT：输出 R0 中的字符
TRAP    x22                     ; PUTS：输出以 R0 为首地址的字符串
TRAP    x23                     ; IN：提示并读入一个字符
TRAP    x24                     ; PUTSP：输出打包字符串（两字符一字节对）
TRAP    x25                     ; HALT：停止机器

; ---- 伪指令 ----
.FILL   value                   ; 在当前位置放一个字
.BLKW   n                       ; 保留 n 个字（Block Word），初值为 0
.STRINGZ "text"                 ; 存放 NUL 结尾的字符串
.ORIG   x3000                   ; 程序起始地址
.END                            ; 源文件结束
```

压栈与弹栈

```
; ---- PUSH R0 ----
        ADD     R6, R6, #-1     ; 栈向低地址增长：先移动指针
        STR     R0, R6, #0      ; 再存数据

; ---- POP R0 ----
        LDR     R0, R6, #0      ; 先取数据
        ADD     R6, R6, #1      ; 再移动指针
```

完整的子程序框架

```
; ========================================================
; 子程序：ADD3 —— 返回三个参数之和
; 输入：R0, R1, R2
; 输出：R0
; 帧布局：R5+0 = 无局部变量；R5+1 旧帧指针；R5+2 返回地址；R5+3 返回值
; ========================================================
ADD3
        ; ---- 建立栈帧 ----
        ADD     R6,R6,#-3       ; 3 个位置：3 个 linkage（本函数无局部变量）
        STR     R5,R6,#0        ; 保存旧帧指针（对应 R5+1）
        ADD     R5,R6,#-1       ; 设置帧指针：R5 = R6 - 1
        STR     R7,R5,#2        ; 保存返回地址（对应 R5+2）

        ; ---- 函数体 ----
        ADD     R0, R0, R1      ; R0 = R0 + R1
        ADD     R0, R0, R2      ; R0 = R0 + R2
        STR     R0,R5,#3        ; 写返回值（对应 R5+3）

        ; ---- 拆除栈帧 ----
        LDR     R7,R5,#2        ; 恢复返回地址
        LDR     R5,R5,#1        ; 恢复旧帧指针
        ADD     R6,R6,#3        ; 弹出 linkage（返回值留在栈上给调用者读）
        RET                     ; 返回：JMP R7
```

若函数**有 N 个局部变量**，通用形式如下（`N` 必须是编译期常量）：

> 下面是**示意模板**：`N` 是元变量，`#-(3+N)`、`#N`、`#(N-1)` 这种写法 **汇编器并不接受**—— 你必须先代入具体数字（例如 N=1 就写成 `#-4`、`#1`、`#0`）。上面两张代入验证表就是这么来的。

```
; 设 N = 局部变量个数
        ADD     R6,R6,#-(3+N)   ; 一次分配：3 个 linkage + N 个局部变量
        STR     R5,R6,#N        ; 把【旧】帧指针存到 R6+N（即未来的 R5+1）
        ADD     R5,R6,#(N-1)    ; 设置帧指针：R5 = R6 + (N-1)
        STR     R7,R5,#2        ; 保存返回地址 → R5+2
        ; 此时 R5+0, R5-1, …, R5-(N-1) 就是 N 个局部变量
```

代入验证（与课程真实代码逐字吻合）：

N	`ADD R6,R6,#-(3+N)`	`STR R5,R6,#N`	`ADD R5,R6,#(N-1)`	对应真实代码
1	`ADD R6,R6,#-4`	`STR R5,R6,#1`	`ADD R5,R6,#0`	`translate.asm` 的 `FIND_ABS` ✅
0	`ADD R6,R6,#-3`	`STR R5,R6,#0`	`ADD R5,R6,#-1`	`translate.asm` 的 `MAIN` ✅

> **注意保存顺序**：必须**先用 `R6` 作基址保存旧 `R5`**，然后才 `ADD R5,...` 设置新帧指针。 反过来写会把新 `R5` 当成旧值存起来， `RET` 时跳回一个错误的位置。
>
> **两条不变的规则**：① `R5+1` / `R5+2` / `R5+3` 永远是旧帧指针/返回地址/返回值， 与局部变量个数无关；② `R5+4` 起永远是第一个参数。

---

19. C 到 LC-3 翻译模板

> **这是 ECE 220 考试的核心能力**（教学目标第 1 条明确要求 "Be able to perform such a transformation manually"）。下面给出最常用的翻译模式。

变量声明的翻译

```
int32_t a;          /* 局部变量 */
int32_t b = 5;
```

```
; 假设 R5 指向局部变量底部
; 局部变量占 R5+0、R5-1、R5-2 …（向下增长）；R5+1 起是 linkage，不可用作局部变量！
; a 在 R5+0，b 在 R5-1
        AND     R0, R0, #0      ; R0 = 0（清空寄存器）
        STR     R0, R5, #0      ; a = 0（未初始化的局部变量内容不确定，这里示意）
        ADD     R0, R0, #5      ; R0 = 5
        STR     R0, R5, #-1     ; b = 5
```

赋值的翻译

```
a = b + c;
```

```
        LDR     R0, R5, #0      ; R0 = a
        LDR     R1, R5, #-1     ; R1 = b
        LDR     R2, R5, #-2     ; R2 = c
        ADD     R0, R0, R1      ; R0 = a + b
        ADD     R0, R0, R2      ; R0 = (a + b) + c
        STR     R0, R5, #-3     ; 结果存入临时/目标
```

`if` 语句的翻译

```
if (a < b) {
    x = 1;
} else {
    x = 2;
}
```

```
; 局部变量向下分配：a 在 R5+0，b 在 R5-1，x 在 R5-2
        LDR     R0, R5, #0      ; a
        LDR     R1, R5, #-1     ; b
        NOT     R1, R1
        ADD     R1, R1, #1      ; R1 = -b
        ADD     R0, R0, R1      ; R0 = a - b
        BRn     THEN            ; 若 a < b（结果为负）跳到 THEN
        ; ---- else 分支 ----
        AND     R0, R0, #0
        ADD     R0, R0, #2      ; R0 = 2
        STR     R0, R5, #-2     ; x = 2
        BRnzp   ENDIF
THEN
        AND     R0, R0, #0
        ADD     R0, R0, #1      ; R0 = 1
        STR     R0, R5, #-2     ; x = 1
ENDIF
```

> **关键技巧**：LC-3 没有 `SUB` 指令，所以 `a - b` 要写成 `a + (-b)`， 而取负是 `NOT` 加 `ADD #1`（补码取负）。

`while` 循环的翻译

```
while (i < n) {
    sum += i;
    i++;
}
```

```
; 局部变量：i 在 R5+0，n 在 R5-1，sum 在 R5-2
WHILE
        LDR     R0, R5, #0      ; i
        LDR     R1, R5, #-1     ; n
        NOT     R1, R1
        ADD     R1, R1, #1      ; -n
        ADD     R0, R0, R1      ; i - n
        BRzp    DONE            ; 若 i >= n，退出循环
        ; ---- 循环体 ----
        LDR     R0, R5, #0      ; i
        LDR     R1, R5, #-2     ; sum
        ADD     R1, R1, R0      ; sum + i
        STR     R1, R5, #-2     ; sum = sum + i
        LDR     R0, R5, #0
        ADD     R0, R0, #1      ; i + 1
        STR     R0, R5, #0      ; i = i + 1
        BRnzp   WHILE
DONE
```

`for` 循环的翻译

`for (init; cond; update) body;` 等价于 `init; while (cond) { body; update; }`。

数组访问的翻译

```
int32_t arr[5];
int32_t x = arr[2];
arr[3] = 7;
```

```
; 假设 arr 的基址在 R4（全局）或某个寄存器中
; arr[2] → LDR R0, R_base, #2    （字偏移，因为是字寻址）
        LDR     R0, R4, #2      ; R0 = arr[2]
        STR     R0, R5, #0      ; x = arr[2]

        ADD     R1, R1, #7      ; R1 = 7
        STR     R1, R4, #3      ; arr[3] = 7
```

```
/* 用指针遍历数组（等价形式）*/
for (int32_t *p = arr; p < arr + 5; p++) { sum += *p; }
```

```
; p 在 R2 中，sum 在 R3 中，arr 末地址在 R4 中
LOOP
        NOT     R0, R4
        ADD     R0, R0, #1      ; -end
        ADD     R0, R2, R0      ; p - end
        BRzp    LOOP_END        ; p >= end 时退出
        LDR     R0, R2, #0      ; *p
        ADD     R3, R3, R0      ; sum += *p
        ADD     R2, R2, #1      ; p++（字地址 +1）
        BRnzp   LOOP
LOOP_END
```

> **重要区别**：LC-3 是**字寻址**（每个地址一个 16 位字），所以 `p++` 就是地址加 1； 而在 64 位机器上按字节寻址，`int*` 的 `p++` 是地址加 4。**指针算术的步长永远等于 所指向类型的大小除以可寻址单位**。

函数调用的翻译

```
int32_t r = add3(1, 2, 3);
```

```
; ---- 调用者：把参数放入 R0-R3 ----
        AND     R0, R0, #0
        ADD     R0, R0, #1      ; R0 = 1（第一个参数）
        AND     R1, R1, #0
        ADD     R1, R1, #2      ; R1 = 2
        AND     R2, R2, #0
        ADD     R2, R2, #3      ; R2 = 3
        JSR     ADD3            ; 调用；R7 = 返回地址
        ; R0 中是返回值
        STR     R0, R5, #0      ; r = 返回值
```

> 对**参数多于 4 个**的函数，第 5 个及以后的参数必须**压栈**传递（从右往左压）。

结构体成员访问的翻译

```
typedef struct { int32_t x; int32_t y; } point_t;
point_t p;
p.y = 7;
```

```
; p 是 2 个字的局部变量，占 R5+0（p.x）与 R5-1（p.y）
; 注意：不能用 R5+1 —— 那是保存旧帧指针的位置！
        AND     R0, R0, #0
        ADD     R0, R0, #7
        STR     R0, R5, #-1     ; p.y = 7（基址 + 编译期常量偏移 1）
```

```
point_t *pp = &p;
int32_t v = pp->y;
```

```
; pp 在 R2 中
        LDR     R0, R2, #1      ; R0 = pp->y（基址 = pp 的值，偏移 = 1）
```

> **这就是结构体的全部秘密**：成员访问编译成"**基址 + 编译期常量偏移**"的一次访存。 偏移在编译期就确定了，运行时没有任何查表或计算开销。 `p.y` 和 `pp->y` 的唯一区别是基址来自 R5 偏移还是来自一个指针寄存器。

---

20. 调试速查：GDB

启动

```
gcc -g -O0 -std=c99 -Wall -Werror -o prog prog.c    # 必须先带 -g 编译

gdb ./prog                          # 无参数
gdb --args ./prog arg1 arg2         # 带命令行参数
gdb -tui ./prog                     # 打开 TUI 界面（显示源码）
```

命令表（官方速查表命令）

类别	命令	简写	作用
运行	`run`	`r`	启动程序（到第一个断点或结束）
	`run arg1 arg2`		带参数运行
	`set args 4 25`		设置运行参数
	`continue`	`c`	继续到下一个断点
	`quit`	`q`	退出 GDB
断点	`break baz`	`b baz`	在函数 `baz` 入口设断点
	`break qux.c:24`	`b qux.c:24`	在第 24 行设断点
	`clear qux.c:24`		删除该行断点
	`info breakpoints`		列出所有断点
	`delete N`	`d N`	删除 3 号断点
单步	`next`	`n`	执行下一行，**不进入**函数调用

类别	命令	简写	作用
	`step`	`s`	执行下一行，**进入**函数调用
	`until`	`u`	运行到当前循环的下一行（跳出循环）
	`finish`	`fin`	运行到当前函数返回
	`nexti` / `stepi`	`ni` / `si`	单步**一条机器指令**（无源码时用）
查看	`print corge`	`p corge`	打印变量的值
	`print/x var`		以十六进制打印
	`display corge`		每次暂停时自动打印该变量
	`undisplay N`		取消自动显示
	`list`	`l`	显示当前行附近的源码
	`layout src`		打开源码窗口
	`backtrace`	`bt`	显示调用栈（函数调用链）
	`frame 1`	`f 1`	切换到 1 号栈帧的上下文
	`info locals`		显示当前帧的所有局部变量
	`info args`		显示当前帧的参数
	`info frame`		显示当前栈帧的详细信息
	`ptype var`		显示变量的类型定义
内存	`x/8xb ptr`		从 `ptr` 起检查 8 个**字节**，十六进制
	`x/4dw arr`		检查 4 个**字**（4 字节），十进制
	`x/s str`		以字符串形式检查
	`p *ptr`		解引用打印
	`p arr[3]`		打印数组元素
	`p &var`		打印变量地址
修改	`set var x = 5`		运行时改变量值
	`watch var`		变量一变就暂停
	`call f(1, 2)`		手动调用函数

读懂 `backtrace`

```
(gdb) bt
#0  0x0000000000401198 in baz () at bar.c:56
#1  0x00000000004011c4 in foo () at bar.c:47
#2  0x00000000004011e9 in main () at bar.c:12
```

- **`#0` 是最内层**（程序现在停在这里），编号越大越靠外层。
- 读法：`main` 在第 12 行调用了 `foo`，`foo` 在第 47 行调用了 `baz`，`baz` 停在第 56 行。
- 用 `frame 1` 切到 `foo` 的上下文，才能 `print` `foo` 的局部变量。
- **在某个栈帧里只能访问该帧的变量**——`baz` 里 `print` 不到 `main` 的局部变量。

检查内存的 `x` 命令格式

```
x/[数量][格式][单位] 地址

格式：x=十六进制  d=十进制  u=无符号  o=八进制  t=二进制  c=字符  s=字符串  i=指令
单位：b=字节(1)  h=半字(2)  w=字(4)  g=巨字(8)

例：
x/8xb  ptr      → 8 个字节，十六进制
x/4dw  arr      → 4 个字（4 字节），十进制
x/2gx  &dbl     → 2 个 8 字节，十六进制
x/s    str      → 字符串
x/10i  main     → main 的前 10 条指令（反汇编）
```

`.gdbinit` 配置（自动加载断点）

在家目录创建 `~/.gdbinit`，内容：

```
add-auto-load-safe-path /home/user/ece220/mp08
```

在当前工作目录创建 `.gdbinit`，内容示例：

```
set print pretty            # 让结构体打印更易读
b main                      # 在 main 设断点
b list_insert               # 在 list_insert 设断点
set args input.txt          # 设置程序参数
run                         # 启动
```

之后直接 `gdb ./prog` 就会自动应用这些设置，无需每次重打。**这在 MP8 之后会非常省时间。**

示例调试会话（真实的 off-by-one bug）

```c
/* buggy.c */
#include <stdio.h>
int main(void)
{
    int arr[5] = { 10, 20, 30, 40, 50 };
    int sum = 0;
    for (int i = 0; i <= 5; i++) {      /* ⚠️ 应为 i < 5 */
        sum += arr[i];
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

```
$ gcc -g -O0 -std=c99 -Wall -o buggy buggy.c
$ gdb ./buggy
(gdb) b main
Breakpoint 1 at 0x1149: file buggy.c, line 4.
(gdb) r
Breakpoint 1, main () at buggy.c:4
4           int arr[5] = { 10, 20, 30, 40, 50 };
(gdb) n
5           int sum = 0;
(gdb) n
6           for (int i = 0; i <= 5; i++) {
(gdb) p arr
$1 = {10, 20, 30, 40, 50}
(gdb) p sizeof(arr)/sizeof(arr[0])
$2 = 5
(gdb) watch i
Hardware watchpoint 2: i
(gdb) c
...
(gdb) p i
$3 = 5
(gdb) p arr[5]              ← 越界访问!
$4 = 0
```

诊断：`arr` 只有 5 个元素（下标 0..4），但循环条件 `i <= 5` 让 `i` 取到 5， 访问了 `arr[5]` ——**越界**。**修复**：把 `<=` 改成 `<`。

常用查看技巧

```
# 查看某个指针指向的内容和它的地址
(gdb) p p
$1 = (int *) 0x7fffffffe4a0
(gdb) p *p
$2 = 42
(gdb) p &p
$3 = (int **) 0x7fffffffe498        # p 自己也有地址

# 查看数组的原始字节
(gdb) x/20xb arr
0x7fffffffe4a0: 0x0a  0x00  0x00  0x00  0x14  0x00  0x00  0x00
0x7fffffffe4a8: 0x1e  0x00  0x00  0x00  0x28  0x00  0x00  0x00
0x7fffffffe4b0: 0x32  0x00  0x00  0x00
# 注意：小端序！10 = 0x0a 存成 0a 00 00 00

# 查看结构体
(gdb) p *node
$4 = {value = 42, next = 0x602030}
(gdb) p node->next->value
$5 = 7
```

> **小端序观察**：`10` 的十六进制是 `0x0000000a`，但内存里是 `0a 00 00 00` —— 低位字节存在低地址。这是 x86-64 的小端序 (little-endian)，与 LC-3 一致。

---

21. 调试速查：Valgrind 与 Sanitizer

Valgrind Memcheck

```
gcc -g -O0 -std=c99 -Wall -o prog prog.c
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog
```

选项	作用
`--leak-check=full`	详细报告每一处泄漏的完整调用栈
`--show-leak-kinds=all`	显示全部四种泄漏类别
`--track-origins=yes`	追踪未初始化值的**来源**（较慢但极有用）
`--error-exitcode=1`	有错误时返回码为 1（便于脚本化测试）
`--gen-suppressions=all`	生成抑制文件（用于忽略库的内部问题）

读懂泄漏分类

类别	含义	严重性
definitely lost	没有任何指针指向这块内存	**真正的泄漏，必须修**
indirectly lost	指向它的指针本身也丢了（通常伴随 definitely lost）	**必须修**
possibly lost	只有指向块内部的指针（如 `p+1`），没有指向起始处	通常需要检查
still reachable	程序结束时仍有指针指向它，但没 `free`	常常可接受（如全局缓存）

实测输出（本机 `valgrind-3.19.0`，对一个 `malloc(10 * sizeof(int32_t))` 后未 `free` 的程序）：

```
==1221543==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==1221543==
==1221543== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1221543==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==1221543==    by 0x401137: main (leak.c:5)
==1221543==
==1221543== LEAK SUMMARY:
==1221543==    definitely lost: 40 bytes in 1 blocks
==1221543==    indirectly lost: 0 bytes in 0 blocks
==1221543==      possibly lost: 0 bytes in 0 blocks
==1221543==    still reachable: 0 bytes in 0 blocks
```

> 关键信息是 `by 0x401137: main (leak.c:5)`——它**直接指出泄漏发生在 `leak.c` 第 5 行**，即那次 `malloc`。

Valgrind 能抓到的错误类型

报告	含义
`Invalid read of size 4`	读取了越界或已释放的内存
`Invalid write of size 1`	写入了越界或已释放的内存
`Conditional jump depends on uninitialised value`	用未初始化的变量做判断
`Invalid free() / delete / delete[]`	`free` 了非 `malloc` 指针，或重复 `free`
`Mismatched free() / delete / delete[]`	C++ 中 `new` 配了 `free`
`Source and destination overlap in memcpy`	`memcpy` 区域重叠（应用 `memmove`）

AddressSanitizer（更快，编译期插桩）

```
gcc -g -O0 -std=c99 -Wall -fsanitize=address,undefined -o prog prog.c
./prog
```

优势：比 Valgrind 快得多（约 2 倍减速 vs 20 倍），且能找到**栈上的**越界访问 （Valgrind 主要覆盖堆）。

> ⚠️ **本笔记编写环境的实测说明**：ASan 需要预留约 15 TB 的**虚拟**地址空间作为影子内存 （shadow memory），而本机的 `ulimit -v` 为 32 GB，因此 ASan 无法启动，报错为：
>
> `> ==ERROR: AddressSanitizer failed to allocate 0xdfff0001000 ... > ReserveShadowMemoryRange failed while trying to map 0xdfff0001000 bytes. Perhaps you're using ulimit -v >`
>
> 若你在 EWS 实验室机器或自己的 Linux 上运行（无此虚拟内存限制），ASan 可以正常工作。 **本笔记中的实测输出均来自 Valgrind**（本机 `valgrind-3.19.0` 可用）。 下面展示的是 ASan 报告的**标准形式**，供你识别其输出结构，而非本机粘贴的输出。

```
==================================================================
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000018
WRITE of size 4 at 0x602000000018 thread T0
    #0 0x400b4d in main /tmp/overflow.c:7
    #1 0x7f... in __libc_start_main

0x602000000018 is located 0 bytes to the right of 16-byte region
[0x602000000008,0x602000000018)
allocated by thread T0 here:
    #0 0x7f... in malloc
    #1 0x400a9e in main /tmp/overflow.c:5
```

> **读法**：`heap-buffer-overflow` 说明越界；`0 bytes to the right of 16-byte region` 说明你正好写在 16 字节区域的右边界外——典型的 `i <= n` 写成 `i < n` 的错误。 两处调用栈分别告诉你**出错位置**和**分配位置**。

其他 Sanitizer

```
-fsanitize=undefined        # 未定义行为（有符号溢出、空指针解引用、越界移位、错位对齐）
-fsanitize=leak             # 仅泄漏检测（比 ASan 轻量）
-fsanitize=thread           # 数据竞争（多线程）
-fno-omit-frame-pointer     # 与 -fsanitize=address 配合，让栈更完整
```

三种工具的选择

工具	速度	覆盖范围	何时用
AddressSanitizer	快（~2×）	堆 + 栈 + 全局变量的越界、UAF、泄漏	日常开发首选
Valgrind	慢（~20×）	堆为主；能追踪未初始化值的来源	需要精确的未初始化值溯源时
手工 `assert` + 打印	最快	只覆盖你想到的东西	快速定位逻辑错误

---

22. 错误分类与排查流程图

四类错误（课程的错误分类学）

类别	何时暴露	典型表现	谁来抓
编译期错误	编译时	语法错误、类型错误、未声明标识符	编译器（`-Wall -Werror`）
链接期错误	链接时	`undefined reference to 'foo'`	链接器
运行期错误	运行时	段错误、`abort`、错误结果、随机行为	GDB、Valgrind、Sanitizer
逻辑错误	运行时	能跑完，结果不对	**只有测试能抓**

> **核心认识**：编译器只能抓第一类。第二类靠链接器。**第三、四类只能靠测试和调试工具。** 这就是为什么"它编译通过了"完全不能说明程序是对的。

排查流程

```
程序有问题
    │
    ├─ 编译不过？ ──────────────────────────► 读第一条错误信息（不是最后一条！）
    │                                          加 -Wall -Werror，逐个修
    │
    ├─ 链接不过？ ──────────────────────────► undefined reference:
    │                                          忘了加 .c 文件 / 忘了 -l库 / 拼错函数名
    │
    ├─ 崩溃（段错误）？ ────────────────────► 用 ASan 重编译运行
    │   │                                      gcc -g -fsanitize=address
    │   │                                      或 gdb → run → bt → 看 #0 帧
    │   │
    │   └─ 常见原因：野指针、解引用 NULL、数组越界、栈溢出（递归太深）
    │
    ├─ 结果不对？ ──────────────────────────► 先用 printf 缩小范围
    │   │                                      二分法：在中间打一个 printf，判断错误在前半还是后
    │   │                                      GDB: b 函数名 → n 单步 → p 变量 → watch 关键变
    │   │
    │   └─ 边界条件检查：空输入 / 单元素 / 全相同 / 最大值 / 负数
    │
    └─ 行为随机、时对时错？ ────────────────► 未初始化变量 或 越界写坏了别人的数据
                                               用 Valgrind --track-origins=yes
                                               或 ASan + MSan
```

段错误的五个常见原因

```
Segmentation fault（段错误）—— 访问了不属于你的虚拟地址
    │
    ├─ 1. 解引用未初始化的指针        int *p; *p = 5;
    ├─ 2. 解引用 NULL                   int *p = NULL; *p = 5;
    ├─ 3. 数组越界写                    arr[100] = 5;    （arr 只有 10 个元素）
    ├─ 4. 使用已释放的内存              free(p); *p = 5;
    ├─ 5. 递归太深导致栈溢出            void f(void){ f(); }
    └─ 6. scanf 忘了 &                  scanf("%d", v);    （v 被当作地址）
```

---

23. 常见内存错误图鉴

① 内存泄漏 (Memory Leak)

```c
/* ❌ 每次循环泄漏一块内存 */
for (int i = 0; i < 1000; i++) {
    int32_t *p = malloc(sizeof(int32_t));
    *p = i;
    /* 忘了 free(p)，而且 p 出了作用域就丢了 —— 这块内存再也找不回来了 */
}

/* ✅ 修复 */
for (int i = 0; i < 1000; i++) {
    int32_t *p = malloc(sizeof(int32_t));
    if (p == NULL) { return -1; }
    *p = i;
    /* ... 使用 p ... */
    free(p);
    p = NULL;
}
```

```
堆状态随时间变化：
开始:  [ 空闲 ................ ]
第 1 次: [已用][ 空闲 .......... ]
第 2 次: [已用][已用][ 空闲 .... ]
...
第 n 次: [已用][已用][已用]...[已用]  ← 堆耗尽，malloc 返回 NULL
                                    ↑ 累积的"已用"块全部无法访问，永不回收
```

检测：`valgrind --leak-check=full ./prog` → 看 "definitely lost"。

② 悬垂指针 / 释放后使用 (Dangling Pointer / Use-After-Free)

```
/* ❌ 释放后继续使用 */
int32_t *p = malloc(sizeof(int32_t));
*p = 42;
free(p);

printf("%d\n", *p);       /* ❌ use-after-free：内存可能已被重新分配或归还给 OS */
*p = 7;                   /* ❌ 更糟：写坏别人的数据 */

/* ✅ 修复：释放后立刻置 NULL */
free(p);
p = NULL;
/* 之后任何对 p 的解引用都会立即段错误（好过默默破坏数据） */
```

```
时间线：
  p ──► [块 A：42]
  free(p) 后：
  p ──► ???（指针值没变，但块 A 已归还给分配器）
        ↓
  p 仍指向原地址，但该内存可能已被分配给别的变量
        ↓
  解引用 p 会读到垃圾 / 写坏别人的数据 ── 行为不确定
```

检测：ASan 会在 UAF 时立即报 `heap-use-after-free` 并给出**释放点**和**使用点**两个栈。

③ 重复释放 (Double Free)

```
/* ❌ */
int32_t *p = malloc(sizeof(int32_t));
free(p);
free(p);                  /* ❌ 破坏分配器的空闲链表 */

/* ✅ */
free(p);
p = NULL;
free(p);                  /* ✅ free(NULL) 是合法的空操作 */
```

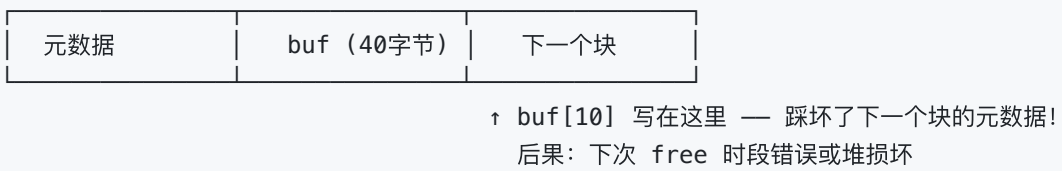
④ 越界访问 (Out-of-Bounds)

```
int32_t arr[10];

/* ❌ 循环写多了 */
for (int i = 0; i <= 10; i++) { arr[i] = 0; }   /* arr[10] 越界 */

/* ❌ 差一错误 */
int32_t *buf = malloc(10 * sizeof(int32_t));
buf[10] = 1;                                    /* 正好写在块尾之后 */
```

堆布局（越界写的破坏）：



检测：ASan（立即报错）或 Valgrind（`Invalid write of size 4`）。

⑤ 返回局部变量的地址

```
/* ❌ 返回指向已销毁栈帧的指针 */
int32_t *bad(void)
{
    int32_t local = 42;
    return &local;          /* local 所在栈帧已失效 */
}

/* ✅ 方案 A：调用者提供缓冲区 */
void good_a(int32_t *out) { *out = 42; }

/* ✅ 方案 B：堆分配，调用者负责 free */
int32_t *good_b(void) { int32_t *p = malloc(sizeof *p); if (p) { *p = 42; } return p; }

/* ✅ 方案 C：static 变量（注意：不可重入！） */
int32_t *good_c(void) { static int32_t v; v = 42; return &v; }
```

⑥ 未初始化的内存

```
int32_t *p = malloc(10 * sizeof(int32_t));
/* malloc 不保证清零！内容可能是任意值 */
printf("%d\n", p[0]);       /* ❌ 读到垃圾值 */

/* ✅ 用 calloc 清零 */
int32_t *q = calloc(10, sizeof(int32_t));
/* q[0..9] 全是 0 */
```

⑦ 结构体填充未初始化

```
typedef struct { char a; int b; char c; } s_t;
s_t s;
s.a = 'x'; s.c = 'y';       /* ❌ 忘了 s.b */
fwrite(&s, sizeof(s), 1, fp);   /* 把 b 的垃圾值和填充字节一起写进文件 */
```

⑧ `free` 非堆指针

```
int32_t arr[10];
free(arr);                  /* ❌ arr 在栈上，不是 malloc 来的 */
free(&arr[2]);              /* ❌ 偏移过的指针，元数据位置错误 */
```

内存错误速查表

错误	症状	检测工具
内存泄漏	长时间运行内存持续增长	Valgrind `--leak-check=full`、ASan
悬垂指针 / UAF	随机崩溃或错误结果	ASan、Valgrind
重复释放	`free(): double free detected`	ASan、Valgrind
越界读写	段错误或"幽灵"数据损坏	ASan、Valgrind
未初始化读取	结果不确定、时对时错	Valgrind `--track-origins=yes`、MSan
返回局部变量地址	函数返回后数据变成垃圾	编译器 `-Wreturn-local-addr`（在 `-Wall` 里）
`realloc` 失败覆盖指针	内存泄漏	代码审查 + Valgrind
结构体填充未清零	二进制输出不确定	`memset(&s, 0, sizeof(s))`

---

24. C 编程准则清单

内存与指针

1. **每个 `malloc` 都要检查 `NULL`。**
2. **每个 `malloc` 都要有对应的 `free`。** 画一张"所有权图"：谁分配、谁释放。
3. **`free` 之后立刻把指针置 `NULL`。**
4. **`realloc` 必须用临时指针接返回值。**
5. **不要返回指向局部变量的指针。**
6. **数组下标前先想清楚边界**：有效范围是 `0 .. n-1`，不是 `0 .. n`。
7. **`sizeof(arr)` 在函数参数上是指针大小**——必须显式传长度。

字符串

1. **永远不用 `gets`。** 用 `fgets`。
2. **`strcpy` / `strcat` 不检查长度**——用 `snprintf` 或自己检查。
3. **`strcmp` 返回 0 表示相等。**

4. **字符串字面量是只读的**，不要写 `char *s = "abc"; s[0] = 'x';`。

作用域与类型

1. **避免全局变量**，用 `static` 限制在文件内。
2. **用 `int32_t` 等定宽类型**而不是裸 `int`，特别是在需要确定宽度时。
3. **`int *A, B;` 里 `B` 不是指针**——每个指针都写 `*`。
4. **`typedef struct { ... } name_t;`** 遵循课程编码规范。

函数与接口

1. **每个函数只做一件事**，名字说明它做什么。
2. **函数超过一屏就考虑拆分。**
3. **头文件只暴露接口**，把表示藏在 `.c` 里（信息隐藏）。
4. **头文件必须加 include guard。**
5. **检查所有可能失败的函数的返回值**（`malloc`、`fopen`、`scanf`、`fclose`）。

流程与风格

1. **始终使用花括号**，即使只有一条语句。
2. **用 `=` 还是 `==` 想清楚**；把常量写在左边（`if (5 == x)`）可以借助编译器抓错。
3. **不要用 Tab**，用 4 个空格；行宽 ≤ 120 字符。
4. **`switch` 每个 `case` 都要有 `break`**（除非你**故意**要 fall-through 并写了注释）。

测试与调试

1. **编译时永远带 `-g -std=c99 -Wall -Werror`。**
2. **调试时永远用 `-O0`**，`-O2` 会让 GDB 显示的变量和源码对不上。
3. **测试的边界**：空、单元素、全相同、最大值、最小值、NULL。
4. **"它跑通了一次"不是证据。** 写能重复运行的测试。
5. **遇到崩溃先上 ASan**，遇到泄漏上 Valgrind，遇到逻辑错误用 GDB 单步加 `watch`。
6. **`assert` 表达"这里必须为真"的不变量**，不要用它处理用户输入错误。

附：LC-3 与 x86-64 概念对照表

概念	LC-3	x86-64（EWS Linux）
字长	16 位	64 位
寻址单位	字（16 位）	字节（8 位）

概念	LC-3	x86-64（EWS Linux）
通用寄存器数	8 个（R0—R7）	16 个（RAX, RBX, …, R15）
栈指针	R6	RSP
帧指针	R5	RBP
返回地址	R7	栈上（`call` 自动压栈）
参数传递	R0—R3，多余的压栈	RDI, RSI, RDX, RCX, R8, R9（System V），多余的压栈
返回值	R0	RAX
调用指令	`JSR` / `JSRR`	`call`
返回指令	`RET`（= `JMP R7`）	`ret`
减法	无（用 `NOT` + `ADD #1` 取负）	`sub`
乘法/除法	无	`imul` / `idiv`
指针大小	16 位（1 个字）	64 位（8 字节）
加载	`LD` / `LDI` / `LDR` / `LEA`	`mov` / `lea`（`lea` 用于算地址）
存储	`ST` / `STI` / `STR`	`mov`
条件跳转	`BRnzp`（基于 NZP 三个标志）	`jmp` / `je` / `jne` / `jl` / `jg`（基于 flags 寄存器）
按类型缩放指针算术	编译器自动（步长 = sizeof(T) / 2 字）	编译器自动（步长 = sizeof(T) 字节）
调用约定	课程定义的 LC-3 CIS	System V AMD64 ABI
系统调用	`TRAP` 向量（x20—x25）	`syscall` 指令 + 系统调用号
动态内存扩展	无（LC-3 无堆）	`sbrk` / `mmap`

> **最该记住的一条**：LC-3 的字寻址与 x86-64 的字节寻址**不影响指针算术的语义**—— 在两种机器上，`p + 1`（`p` 是 `int*`）都前进"一个 `int` 的距离"。 差别只在于这个距离在 LC-3 上是 1 个地址单位（1 个 16 位字）， 在 x86-64 上是 4 个地址单位（4 个字节）。 **指针算术的步长永远等于"所指向类型的大小"**，这是跨平台的不变真理。